
System RPL 编程

Programming in System RPL

第二版中文译稿

正文第 1-426 页

简要目录

第 1 章引言	13
第一部分 HP49 对象	16
第 2 章二进制整数 (BINT)	17
第 3 章实数	30
第 4 章复数	37
第 5 章整数 (ZINT)	39
第 6 章字符和字符串	40
第 7 章十六进制字符串	48
第 8 章标识符	50
第 9 章标记对象	51
第 10 章数组	52
第 11 章复合对象	54
第 12 章元对象	58
第 13 章单位对象	60
第 14 章符号对象	63
第 15 章图形对象 (Grobs)	65
第 16 章库对象和备份对象	69
第二部分通用系统 RPL 词条	70
第 17 章堆栈操作	71
第 18 章临时环境	75
第 19 章运行流控制	80
第 20 章条件结构	86
第 21 章循环	93
第 22 章错误处理	96
第 23 章虚拟堆栈	99
第 24 章内存操作	101
第 25 章时间与闹钟	104
第 26 章系统功能	106
第 27 章串行通信	108
第 28 章 HP49 文件管理器	109
第三部分输入与输出	113
第 29 章参数检查	114
第 30 章键盘控制	120
第 31 章使用 InputLine	123
第 32 章参数化外层循环	125
第 33 章使用 HP49 浏览器	132
第 34 章使用 HP48 浏览器	137
第 35 章创建窗口	143
第 36 章显示屏	153
第 37 章菜单	161
第 38 章 HP49 编辑器编程	166
第 39 章绘图	174
第四部分 HP49 计算机代数系统	176

第 40 章 HP49 CAS 简介	177
第 41 章 类型检查与转换	179
第 42 章 整数	180
第 43 章 矩阵	184
第 44 章 表达式操作	189
第 45 章 符号元对象处理	195
第 46 章 多项式	200
第 47 章 求根	204
第 48 章 微积分运算	206
第 49 章 求和	209
第 50 章 模运算	210
第 51 章 符号表	212
第 52 章 错误	213
第 53 章 CAS 配置	214
第 54 章 CAS 菜单	216
第 55 章 User RPL 命令的内部版本	217
第 56 章 杂项	220
第 VI 部分 索引	225
附录 F 按名称排序的词条	226
附录 G 按地址排序的词条	254

目录

第 1 章引言	13
1.1 您的第一个系统 RPL 程序	14
1.2 关于词条列表	14
第一部分 HP49 对象	16
第 2 章二进制整数 (BINT)	17
2.1 参考	17
2.1.1 内置二进制整数	17
2.1.2 压入多个 BINT	25
2.1.3 转换	26
2.1.4 算术函数	26
2.1.5 测试	28
第 3 章实数	30
3.1 参考	30
3.1.1 内置实数	30
3.1.2 内置扩展实数	32
3.1.3 与实数组组合的堆栈操作	32
3.1.4 转换	32
3.1.5 实数函数	33
3.1.6 扩展实数函数	34
3.1.7 测试	35
第 4 章复数	37
4.1 参考	37
4.1.1 内置复数	37
4.1.2 转换	37
4.1.3 函数	37
4.1.4 测试	38
第 5 章整数 (ZINT)	39
第 6 章字符和字符串	40
6.1 参考	40
6.1.1 内置字符	40
6.1.2 内置字符串	42
6.1.3 带堆栈操作的内置字符串	44
6.1.4 转换	44
6.1.5 管理	44
6.1.6 解析字符串	46
6.1.7 反编译	46
6.1.8 字符串测试	47
第 7 章十六进制字符串	48
7.1 参考	48
7.1.1 转换	48
7.1.2 通用函数	48
7.1.3 测试	49
第 8 章标识符	50

第 9 章标记对象	51
9.1 参考	51
第 10 章数组	52
10.1 参考	52
10.1.1 常规函数	52
10.1.2 转换	53
10.1.3 统计	53
第 11 章复合对象	54
11.1 参考	54
11.1.1 常规操作	54
11.1.2 构造	55
11.1.3 展开	56
11.1.4 列表	56
11.1.5 二级程序	57
第 12 章元对象	58
12.1 参考	58
12.1.1 堆栈函数	58
12.1.2 组合函数	58
12.1.3 元对象与对象操作	58
12.1.4 其他操作	59
第 13 章单位对象	60
13.1 参考	60
13.1.1 创建单位	60
13.1.2 通用函数	61
13.1.3 算术函数	61
13.1.4 测试	62
第 14 章符号对象	63
14.1 参考	63
14.1.1 通用操作	63
14.1.2 其他函数	64
14.1.3 符号元对象函数	64
第 15 章图形对象 (Grobs)	65
15.1 参考	65
15.1.1 内置 grob	65
15.1.2 尺寸	65
15.1.3 grob 操作	65
15.1.4 灰度图形	66
15.1.5 创建菜单标签 grob	67
15.1.6 将字符串转换为 grob	68
15.1.7 从其他对象创建 grob	68
第 16 章库对象和备份对象	69
16.1 参考	69
16.1.1 端口操作	69
16.1.2 ROM 指针	69
16.1.3 库	69
16.1.4 备份对象	69

第二部分通用系统 RPL 词条	70
第 17 章堆栈操作	71
17.1 参考	71
第 18 章临时环境	75
18.1 命名局部变量	75
18.2 无名局部变量	75
18.3 嵌套临时环境	76
18.4 其他绑定方式	76
18.5 参考	77
18.5.1 内置 ID 和 LAM	77
18.5.2 转换	77
18.5.3 临时环境命令字	77
第 19 章运行流控制	80
19.1 一些概念	80
19.2 运行流命令	80
19.3 一些示例	81
19.4 参考	83
19.4.1 引用对象	84
19.4.2 跳过对象	85
第 20 章条件结构	86
20.1 测试	86
20.2 If...Then...Else	86
20.3 Case	86
20.4 参考	87
20.4.1 布尔标志	87
20.4.2 一般测试	88
20.4.3 TRUE/FALSE 测试	88
20.4.4 二进制整数测试	90
20.4.5 实数与复数测试	91
20.4.6 元对象测试	91
20.4.7 一般对象测试	91
20.4.8 杂项	92
第 21 章循环	93
21.1 不定次数循环	93
21.1.1 不定次数循环的工作原理	93
21.2 定次数循环	94
21.2.1 DO 循环的工作原理	94
21.3 参考	94
21.3.1 不定次数循环	94
21.3.2 定次数循环	95
第 22 章错误处理	96
22.1 捕获错误	96
22.1.1 保护字	96
22.2 生成错误	97
22.3 参考	97
22.3.1 通用命令字	97

22.3.2 错误生成命令字	98
第 23 章虚拟堆栈	99
23.1 参考	99
第 24 章内存操作	101
24.1 参考	101
24.1.1 提取、存储和删除	101
24.1.2 目录	102
24.1.3 隐藏目录	102
24.1.4 临时内存	103
第 25 章时间与闹钟	104
25.1 参考	104
25.1.1 闹钟	104
第 26 章系统功能	106
26.1 参考	106
26.1.1 用户标志和系统标志	106
26.1.2 通用功能	107
第 27 章串行通信	108
27.1 参考	108
第 28 章 HP49 文件管理器	109
28.1 使用 Filer	109
28.1.1 Filer_Type 参数	109
28.1.2 Filer_Path 参数	109
28.1.3 Filer_List 参数	109
28.1.3.1 Name_Item	109
28.1.3.2 Location_Item	109
28.1.3.3 Action_Item	110
28.1.3.4 ExtraProgram_Item	111
28.1.3.5 Key_Shortcut	111
28.2 参考	112
第三部分输入与输出	113
第 29 章参数检查	114
29.1 参数数量	114
29.2 参数类型	114
29.2.1 示例	115
29.3 参考	116
29.3.1 类型检查	117
第 30 章键盘控制	120
30.1 按键位置	120
30.2 等待按键	121
30.3 参考	121
30.3.1 转换键码	121
30.3.2 等待按键	121
30.3.3 ATTN 标志	122
30.3.4 无效按键	122
30.3.5 用户键	122
第 31 章使用 InputLine	123

31.1 菜单键赋值	123
31.2 示例	124
31.3 参考	124
第 32 章参数化外层循环	125
32.1 参数化外层循环词条	125
32.2 屏幕显示	126
32.3 错误处理	126
32.4 实体键赋值	126
32.5 菜单键赋值	127
32.6 防止挂起环境	127
32.7 退出条件	127
32.8 示例	127
32.9 参考	130
第 33 章使用 HP49 浏览器	132
33.1 选择项元对象	132
33.2 标题字符串	132
33.3 初始所选项目	132
33.4 消息处理程序	132
33.5 浏览器与 lam	133
33.6 访问所选项目	133
33.7 保存和恢复屏幕	134
33.8 示例	134
33.9 参考	136
第 34 章使用 HP48 浏览器	137
34.1 ::Appl 参数	137
34.2 \$Title 参数	138
34.3 ::Converter 参数	138
34.4 {}Items 参数	138
34.5 Init 参数	138
34.6 浏览器的典型用法	138
34.7 示例	139
34.8 参考	141
34.8.1 浏览器使用的 NULLAM	142
第 35 章创建窗口	143
35.1 标签定义	143
35.2 字段定义	143
35.3 标签数与字段数	144
35.4 消息处理程序	144
35.5 标题	145
35.6 输入窗口的结果	145
35.7 示例	145
35.8 参考	147
35.8.1 输入窗口	147
35.8.2 输入窗口消息	149
35.8.2.1 IfMsgKeyPress —0	149
35.8.2.2 IfMsgLooseFocus —1	149

35.8.2.3 IfMsgNewField —2	149
35.8.2.4 IfMsgGetFocus —3	149
35.8.2.5 IfMsgGetFieldValue —4	149
35.8.2.6 IfMsgSetFieldValue —5	150
35.8.2.7 IfMsgGetFieldGrob —6	150
35.8.2.8 IfMsgSetFirstField —7	150
35.8.2.9 IfMsgFieldReset —10	150
35.8.2.10 IfMsgGetMenu —11	150
35.8.2.11 IfMsgGet3KeysMenu —12	151
35.8.2.12 IfMsgCancel —13	151
35.8.2.13 IfMsgCancelKey —14	151
35.8.2.14 IfMsgOK —15	151
35.8.2.15 IfMsgKeyOK —16	151
35.8.2.16 IfMsgChoose —17	151
35.8.2.17 IfMsgType —18	151
35.8.2.18 IfMsgCalc —19	152
35.8.2.19 IfMsgNewCommandLine —20	152
35.8.2.20 IfMsgOldCommandLine —21	152
35.8.2.21 IfMsgCommandLineValid —22	152
35.8.2.22 IfMsgDecompEdit —23	152
35.8.2.23 IfMsgNextChoose —24	152
35.8.2.24 IfMsgEdit —25	152
第 36 章显示屏	153
36.1 显示结构	153
36.2 准备显示屏	153
36.3 控制显示刷新	154
36.4 清空显示屏	154
36.5 显示文本	154
36.5.1 系统字体	154
36.5.2 微型字体	154
36.5.3 显示警告	155
36.6 参考	155
36.6.1 显示结构	155
36.6.2 准备显示屏	155
36.6.3 立即刷新	155
36.6.4 控制显示刷新	156
36.6.5 清空显示屏	157
36.6.6 指示符与模式控制	158
36.6.7 窗口坐标	158
36.6.8 滚动显示屏	158
36.6.9 显示对象	159
36.6.10 显示文本	159
36.6.11 字体	160
第 37 章菜单	161
37.1 菜单格式	161
37.2 菜单属性	162

37.3 参考	163
37.3.1 菜单属性	163
37.3.2 构建菜单	164
37.3.3 菜单显示	164
37.3.4 显示菜单标签	164
37.3.5 通用入口	165
第 38 章 HP49 编辑器编程	166
38.1 术语	166
38.2 示例	166
38.3 在编辑器中执行外部命令	168
38.4 参考	168
38.4.1 状态	168
38.4.2 插入文本	169
38.4.3 删除文本	169
38.4.4 移动光标	169
38.4.5 选择、剪切与粘贴以及剪贴板	170
38.4.6 搜索与替换	171
38.4.7 求值	171
38.4.8 启动编辑器	172
38.4.9 其他	172
第 39 章绘图	174
39.1 参考	174
第四部分 HP49 计算机代数系统	176
第 40 章 HP49 CAS 简介	177
40.1 本部分各章存在的问题	177
40.2 符号对象	177
40.3 几个示例	178
第 41 章类型检查与转换	179
41.1 参考	179
第 42 章整数	180
42.1 参考	180
42.1.1 内置整数	180
42.1.2 转换函数	180
42.1.3 一般整数运算	180
42.1.4 整数因式分解与素数	181
42.1.5 高斯整数	182
42.1.6 整数测试	182
第 43 章矩阵	184
43.1 参考	184
43.1.1 创建矩阵和重新确定矩阵维数	184
43.1.2 转换	184
43.1.3 测试	185
43.1.4 矩阵计算	185
43.1.5 线性代数和高斯约简	186
43.1.6 线性方程组求解器	186
43.1.7 其他矩阵操作	187

43.1.8 特征值、特征函数和约简	188
第 44 章 表达式操作	189
44.1 参考	189
44.1.1 基本运算和函数应用	189
44.1.2 三角函数和指数运算符	191
44.1.3 简化、求值和替换	191
44.1.4 合并与展开	192
44.1.5 三角变换	193
44.1.6 除法、最大公因数与最小公倍数	193
第 45 章 符号元对象处理	195
45.1 参考	195
45.1.1 基本表达式操作	195
45.1.2 基本运算与函数应用	195
45.1.3 三角函数和指数运算符	197
45.1.4 无穷与未定义值	198
45.1.5 展开与化简	199
45.1.6 测试	199
第 46 章 多项式	200
46.1 参考	200
46.1.1 多项式计算	200
46.1.2 因式分解	201
46.1.3 一般多项式运算	202
46.1.4 测试	203
第 47 章 求根	204
47.1 参考	204
47.1.1 求根与数值求解器	204
第 48 章 微积分运算	206
48.1 参考	206
48.1.1 极限与级数展开	206
48.1.2 导数	206
48.1.3 积分	208
48.1.4 部分分式	208
48.1.5 微分方程	208
48.1.6 拉普拉斯变换	208
第 49 章 求和	209
49.1 参考	209
第 50 章 模运算	210
50.1 参考	210
50.1.1 模运算	210
第 51 章 符号表	212
51.1 参考	212
第 52 章 错误	213
52.1 参考	213
第 53 章 CAS 配置	214
53.1 参考	214
第 54 章 CAS 菜单	216

54.1 参考	216
第 55 章 User RPL 命令的内部版本	217
55.1 参考	217
第 56 章杂项	220
56.1 参考	220
56.1.1 详细模式显示例程	220
56.1.2 求值	220
56.1.3 转换	220
56.1.4 Qpi	221
56.1.5 无穷	221
56.1.6 内置常量	221
56.1.7 列表应用	222
56.1.8 Irrquads	222
56.1.9 杂项	222
第 VI 部分索引	225
附录 F 按名称排序的词条	226
附录 G 按地址排序的词条	254

第 1 章 引言

如果您已经会用用户 RPL 编写程序（如果还不会，应先学会再继续阅读本书），那么您所了解的只是 HP49G 计算器能力的一部分。系统 RPL 编程语言使您能够完成许多难以想象的工作。例如，在系统 RPL 中可以处理所有可用的对象类型，而用户 RPL 只能访问其中一部分。您还可以进行 15 位精度的数学运算、使用含非数值元素的数组，等等。系统 RPL 也能完成用户 RPL 程序能够完成的工作，而且速度快得多。

不过，在开始介绍系统 RPL 之前，先回到用户 RPL，说明它实际上是如何工作的。我们知道您希望立刻进入正题，但以下信息对于正确理解系统 RPL 十分重要。

HP49 程序（包括用户 RPL 和系统 RPL 程序）在内部并不以命令名称存储，只存储对象的地址。每个地址只占 2.5 字节；若地址是 ROM 指针或 Flash 指针，则占用更多字节。程序运行时，实际执行的只是对该地址进行一种类似“GOSUB”的调用。这种存储方式有两个目的。2.5 字节短于大多数命令名称，因此程序占用的内存更少；而且执行程序时无须再根据名称查找地址，因此速度也更快。

多数情况下，一个地址会指向另一个程序，后者又继续跳转到其他程序，如此反复。计算器会记录应当返回的地址，因此您可以按需要进行任意多次跳转，无须操心返回位置。被调用的程序结束后，执行就回到原先的位置。当然，跳转最终必须落到某处：要么是机器语言程序，要么是只把自身压入堆栈的对象（数字、字符串等）。这与高级语言中调用函数或子程序的概念很相似。

但是，如果程序只是一串地址，又该如何编辑呢？

答案是，计算器中有一张用户命令名称与对应地址的表。因此，当您把用户 RPL 程序放入堆栈时，HP 会在表中查找内存所存地址对应的命令名称，然后以可读形式显示程序。您可以对其进行编辑；编辑完成后，系统再次查表，取得所输入命令的地址，并且只把这些地址存入内存。这就是编辑较长的用户 RPL 程序需要较长时间、而运行却很快的原因。

只要所有命令都有名称，这套机制就能正常工作。但实际情况是，有四千多个命令没有名称。这正是用户 RPL 与系统 RPL 的区别之一。手册所介绍的用户 RPL（即 « » 语言）只能访问有名称的命令。（实际上，只要知道命令地址，也可以通过 SYSEVAL、LIBEVAL 和 FLASHEVAL 访问无名命令，但这种做法效率不高，偶尔使用除外。）系统 RPL 则可以访问所有命令。

因此，系统 RPL 程序不能直接编辑，必须借助专用工具。附录 A 介绍了可用于编写系统 RPL 程序的工具。所需的一切功能要么已内置于计算器，要么包含在可下载到计算器的程序库中。

系统 RPL 编程功能更强、速度更快，因为它不进行错误检查。在系统 RPL 中，程序员必须确保不会发生错误，否则计算器可能崩溃。例如，若某条命令要求堆栈中有两个参数，而参数不存在或类型不符合函数要求，就可能导致热启动，甚至丢失内存数据。当然，系统提供了用于检查参数数量、参数类型和其他可能错误条件的命令。区别在于，通常只需在程序开始时检查一次所有参数是否齐全，后续无须反复检查。用户 RPL 的每条命令都会执行错误检查，因此会重复进行不必要的测试，拖慢程序运行。

此时您可能会问：“既然命令没有名称，如何用系统 RPL 编程？”如前所述，所有命令都有地址，因此可以用 PTR <address> 这样的结构直接调用地址，并执行该地址处的内容。不过，还有一种更简单的方法。

这些命令其实有名称，只是这些名称并不像用户命令名称那样存储在 HP49 中。

HP 设计团队为它们指定了名称，并把这些名称保存在创建系统 RPL 程序的工具中。您用这些名称编写程序，随后系统 RPL 编译器在表中查找名称，并将其转换为地址。这个过程称为编译或汇编。有些工具也能执行相反的操作，即把地址转换为命令名称；这个过程称为反编译或反汇编。

有些命令被归类为“受支持”：可以保证它们在计算器的所有 ROM 版本中都位于同一内存位置，也就是说地址不会改变，程序员可以放心使用。（但这并不表示它们在 HP48 和 HP49 等不同型号的计算器中也具有相同地址。）另一些命令则被归类为“不受支持”，无法保证它们在不同 ROM 版本中始终位于同一地址。

编译器的表中不列出不受支持的命令，因此不能只输入其名称，就期望生成的程序中出现对应地址。您必须直接按地址调用，或者自行给命令命名。在词条列表中，不受支持的词条名称用一对圆括号括起，例如 (CURSOR@@)。

不过，本书所列的所有不受支持词条都是稳定的。HP 设计团队指出，HP49G 中 025ECh-0B3C7h 和 25565h-40000h 两个地址范围很可能不会改变，其中的不受支持命令也包括在内。

实际上，词条共有三类。前面的说明主要针对普通的 2.5 字节地址，它们直接指向某个 ROM 地址；大多数词条都属于这一类。此外还有 ROM 指针词条和 Flash 指针词条。ROM 指针指向程序库内部的命令，其名称以 ~ 开头。Flash 指针只存在于 HP49 中，指向 Flash 存储器内的子程序，其名称以 ^ 开头。附录 A 将说明使用 HP49 编译器处理各类词条所需的操作。

1.1 您的第一个系统 RPL 程序

下面创建一个非常简单的系统 RPL 程序，并对其作详细说明。给定堆栈中的半径，该程序会计算圆的面积。编译方法见附录 A。如果您下载了示例文件，可以在其中找到名为 first 的程序。

```
1 ::  
CK1NOLASTWD (检查是否有参数)  
CK&DISPATCH1 (检查它是否为实数)  
BINT1 :: (如果是)  
5 %2 %^ (对半径求平方)  
%PI (将 PI 压入堆栈)  
%* (相乘)  
;  
;
```

在开始分析程序之前，必须注意系统 RPL 区分大小写，因此 pi、PI 和 pI 各不相同，输入时务必小心。此外，您可能已经猜到，() 之间的所有内容都视为注释。第一列为 * 的行也属于注释。

第一行含有二级程序（即程序）的起始标记::，称为 DOCOL。结束标记是;，称为 SEMI。

接下来是命令 CK1NOLASTWD。它检查堆栈中是否有一个参数；如果没有，则生成“Too Few Arguments”错误。下一条命令 CK&DISPATCH1 检查参数类型，并允许程序员针对不同的参数类型执行不同操作。本程序只支持一种参数类型：实数（这里用 BINT1 表示，即作为系统二进制整数的一；见第 2 章）。如果输入其他参数类型，则会生成“Bad Argument Type”错误。第 29 章将详细介绍参数检查。

随后是参数为实数时要执行的代码。请注意，这段代码位于:: 与; 之间，因为参数类型之后只应有一个对象。这里的这个对象是二级程序（子程序），属于一种复合对象：它本身只是一个对象，但内部还包含其他对象。因此，如果要对多个对象求值，就必须把它们包含在一个二级程序中。这类似于在 C 语言中用花括号括起多条语句，或在 Pascal 中把多条语句放在 begin 与 end 之间。

程序的其余部分非常简单。先把数字二压入堆栈，再将用户输入的半径提升到该次幂。

最后，把π压入堆栈，并与半径的平方相乘。此时，堆栈中就是圆的面积。

该程序长 25 字节，而对应的用户 RPL 程序«SQ π * →NUM»只需 20 字节。不过，在半径为 1 时，用户 RPL 版本用时 0.0156 秒，系统 RPL 版本只用 0.0019 秒。请注意，虽然这个系统 RPL 程序比等效的用户 RPL 程序更长，但一般并非如此。

1.2 关于词条列表

后续各章的堆栈图使用缩写表示各种对象类型。下表列出这些缩写：

缩写	含义
ob	任意对象
1...n	n 个对象
#	二进制整数 (BINT)
HXS	十六进制字符串 (用户级二进制整数)
CHR	字符
\$	字符串
T	TRUE
F	FALSE
flag	TRUE 或 FALSE
%	实数
%%	扩展实数
%C	复数
%%C	扩展复数
z, Z, ZINT	无限精度整数
N	正的无限精度整数
s, symb	符号对象
u, unit	单位对象
{}	列表
A, []	数组
V, []	向量
M, [[]]	矩阵
P	多项式，即 Q 的列表
Q	ZINT 或 P
meta, obl..obn #n	元对象
grob	图形对象
menu	菜单：程序或列表
sign	符号表

用户 RPL 的堆栈图还使用以下缩写：

缩写	含义
x, y	实数、列表或一般用户 RPL 对象
c, (,)	复数
#	十六进制字符串（用户级二进制整数）
缩写	含义
θ	角（实数）
m, n	整数（ZINT 或实数）
date	DD.MMYYYY 或 MM.DDYYYY 格式的日期
name	全局名称
prog, prg	程序
f, func	函数
F	f 的原函数

第一部分 HP49对象

第 2 章

二进制整数（BINT）

二进制整数是最常用的对象。它们并不是用户级二进制整数（即以 `#` 开头输入的数字）；后者实际上是十六进制字符串，将在第 7 章介绍。这里所说的是系统级二进制整数，简称 **bint**，用户不太容易直接访问。如果堆栈中出现一个 **bint**，它会显示为 `0 10h`。若您使用 HP49，可以尝试在堆栈中输入以下数字（务必反复核对是否正确）：`#3316Bh`。然后输入 `SYSEVAL` 并按 `ENTER`。根据当前使用的数值进制，堆栈中应出现 `0 10h`，也可能是 `0 16d`，甚至其他形式。**bint** 在内部始终采用十六进制。HP49 安装并挂接程序库 256 后（见附录 A），可分别使用 `RSB` 和 `SBB`，在实数、用户级二进制数与 **bint** 之间进行双向转换。

bint 之所以最常用，是因为大多数需要数值参数的命令都要求使用二进制整数，而用户函数通常要求实数。因此，创建 **bint** 应当很方便，事实也确实如此：只需在程序中以十进制形式直接输入，就能把一个 **bint** 压入堆栈。但并非总建议这样做，因为同样的直接输入方式也可以把实数压入堆栈；后文将说明如何区分二者。因此，最好使用结构 `# <hex>`。这样既可以确保得到二进制数，也能让代码更清晰。遗憾的是（也许反而是好事），这里必须使用十六进制表示。

HP49G ROM 中内置了许多二进制数。只需调用地址，就能把其中一个压入堆栈。由于它们几乎全都受支持，若要把 `#6h` 压入堆栈，只需使用命令字 `BINT6`。其主要优点是：输入 `# 6` 需要 5 字节，而 `BINT6` 与其他普通命令一样只占 2.5 字节（ROM 指针和 Flash 指针命令除外）。有些命令字能一次把两个甚至三个 **bint** 压入堆栈，节省的空间更多。下文列出内置 **bint**。

bint 的四种基本运算是 `#+`、`#-`、`#*` 和 `#/`，此外还有

下面列出的许多其他运算。

下面的示例分别用三种方法把三个 **bint** 压入堆栈：

```
1 ::  
13 (13d 或 Dh)  
# D (同一数值，使用推荐方法)  
BINT13 (在本例中，这种方法更短)  
5 ;
```

2.1 参考

2.1.1 内置二进制整数

地址	名称	说明
33107	BINT0	0d 0h 又名：ZERO, any
33111	BINT1	1d 1h 又名：ONE, real, MEMERR
3311B	BINT2	2d 2h 又名：TWO, cmp
33125	BINT3	3d 3h 又名：THREE, str
3312F	BINT4	4d 4h 又名：FOUR, arry
33139	BINT5	5d 5h 又名：FIVE, list
33143	BINT6	6d 6h 又名：SIX, id, idnt
3314D	BINT7	7d 7h 又名：SEVEN, lam
33157	BINT8	8d 8h 又名：EIGHT, seco
33161	BINT9	9d 9h 又名：NINE, symb
3316B	BINT10	10d Ah 又名：TEN, sym
地址	名称	说明
33175	BINT11	11d Bh 又名：ELEVEN, hxs
3317F	BINT12	12d Ch 又名：TWELVE, grob
33189	BINT13	13d Dh 又名：TAGGED, THIRTEEN
33193	BINT14	14d Eh 又名：EXT, FOURTEEN, unitob

地址	名称	说明
3319D	BINT15	15d Fh 又名: FIFTEEN, rompointer
331A7	BINT16	16d 10h 又名: REALOB, SIXTEEN
331B1	BINT17	17d 11h 又名: SEVENTEEN, 2REAL, REALREAL
331BB	BINT18	18d 12h 又名: EIGHTEEN
331C5	BINT19	19d 13h 又名: NINETEEN
331CF	BINT20	20d 14h 又名: TWENTY
331D9	BINT21	21d 15h 又名: TWENTYONE
331E3	BINT22	22d 16h 又名: TWENTYTWO
331ED	BINT23	23d 17h 又名: TWENTYTHREE
331F7	BINT24	24d 18h 又名: TWENTYFOUR
33201	BINT25	25d 19h 又名: TWENTYFIVE
3320B	BINT26	26d 1Ah 又名: REALSYM, TWENTYSIX
33215	BINT27	27d 1Bh 又名: TWENTYSEVEN
3321F	BINT28	28d 1Ch 又名: TWENTYEIGHT
33229	BINT29	29d 1Dh 又名: TWENTYNINE
地址	名称	说明
33233	BINT30	30d 1Eh 又名: REALEXT, THIRTY
3323D	BINT31	31d 1Fh 又名: THIRTYONE
33247	BINT32	32d 20h 又名: THIRTYTWO
33251	BINT33	33d 21h 又名: THIRTYTHREE
3325B	BINT34	34d 22h 又名: THIRTYFOUR
33265	BINT35	35d 23h 又名: THIRTYFIVE
3326F	BINT36	36d 24h 又名: TTHIRTYSIX
33279	BINT37	37d 25h 又名: THIRTYSEVEN
33283	BINT38	38d 26h 又名: THIRTYEIGHT
3328D	BINT39	39d 27h 又名: THIRTYNINE
33297	BINT40	40d 28h 又名: FORTY, FOURTY
332A1	BINT41	41d 29h 又名: FORTYONE
332AB	BINT42	42d 2Ah 又名: FORTYTWO
332B5	BINT43	43d 2Bh 又名: FORTYTHREE
332BF	BINT44	44d 2Ch 又名: FORTYFOUR
332C9	BINT45	45d 2Dh 又名: FORTYFIVE
332D3	BINT46	46d 2Eh 又名: FORTYSIX
332DD	BINT47	47d 2Fh 又名: FORTYSEVEN
332E7	BINT48	48d 30h 又名: FORTYEIGHT
地址	名称	说明
332F1	BINT49	49d 31h 又名: FORTYNINE
332FB	BINT50	50d 32h 又名: FIFTY

地址	名称	说明
	BINT51	51d 33h
33305		又名: FIFTYONE
	BINT52	52d 34h
3330F		又名: FIFTYTWO
	BINT53	53d 35h
33319		又名: FIFTYTHREE, STRLIST, THREEFIVE
	BINT54	54d 36h
33323		又名: FIFTYFOUR
	BINT55	55d 37h
3332D		又名: FIFTYFIVE
	BINT56	56d 38h
33337		又名: FIFTYSIX
	BINT57	57d 39h
33341		又名: FIFTYSEVEN
	BINT58	58d 3Ah
3334B		又名: FIFTYEIGHT
	BINT59	59d 3Bh
33355		又名: FIFTYNINE
	BINT60	60d 3Ch
3335F		又名: SIXTY
	BINT61	61d 3Dh
33369		又名: SIXTYONE
	BINT62	62d 3Eh
33373		又名: SIXTYTWO
	BINT63	63d 3Fh
3337D		又名: SIXTYTHREE
	BINT64	64d 40h
33387		又名: BINT40h, SIXTYFOUR, YHI
	BINT65	65d 41h
33391		又名: ARRYREAL
	BINT66	66d 42h
3339B		又名: FORTWO
	BINT67	67d 43h
333A5		又名: FOURTHREE
地址	名称	说明
	BINT68	68d 44h
333AF		又名: SIXTYEIGHT
	BINT69	69d 45h
333B9		又名: FOURFIVE
	BINT70	70d 46h
333C3		又名: SEVENTY
	BINT71	71d 47h
333CD		
	BINT72	72d 48h
333D7		
	BINT73	73d 49h
333E1		
	BINT74	74d 4Ah
333EB		又名: SEVENTYFOUR
	BINT75	75d 4Bh
333F5		
	BINT76	76d 4Ch
333FF		
	BINT77	77d 4Dh
33409		
	BINT78	78d 4Eh
33413		
	BINT79	79d 4Fh
3341D		又名: SEVENTYNINE
	BINT80	80d 50h
33427		又名: EIGHTY
	BINT81	81d 51h
33431		又名: EIGHTYONE, LISTREAL
	BINT82	82d 52h
3343B		又名: LISTCMP
	BINT83	83d 53h
33445		又名: FIVETHREE
	BINT84	84d 54h
3344F		又名: FIVEFOUR
	BINT85	85d 55h
33459		又名: 2LIST
	BINT86	86d 56h
33463		又名: FIVESIX
	BINT87	87d 57h
3346D		又名: LISTLAM
	BINT88	88d 58h
33477		

地址	名称	说明
33481	BINT89	89d 59h
3348B	BINT90	90d 5Ah
33495	BINT91	91d 5Bh
3349F	BINT92	又名: BINT_91d 92d 5Ch
地址	名称	说明
334A9	BINT93	93d 5Dh
334B3	BINT94	94d 5Eh
334BD	BINT95	95d 5Fh
334C7	BINT96	96d 60h
334D1	BINT97	又名: BINT_96d 97d 61h
334DB	BINT98	又名: IDREAL 98d 62h
334E5	BINT99	99d 63h
334EF	BINT100	100d 64h
334F9	BINT101	又名: ONEHUNDRED 101d 65h
33503	BINT102	102d 66h
3350D	BINT103	103d 67h
33517	BINT104	104d 68h
33521	BINT105	105d 69h
3352B	BINT106	106d 6Ah
33535	BINT107	107d 6Bh
3353F	BINT108	108d 6Ch
33549	BINT109	109d 6Dh
33553	BINT110	110d 6Eh
3355D	BINT111	111d 6Fh
33567	BINT112	又名: char 112d 70h
33571	BINT113	113d 71h
3357B	BINT114	114d 72h
33585	BINT115	115d 73h
3358F	BINT116	又名: BINT_115d 116d 74h
33599	BINT117	又名: BINT_116d 117d 75h
335A3	BINT118	118d 76h
335AD	BINT119	119d 77h
335B7	BINT120	120d 78h
335C1	BINT121	121d 79h
335CB	BINT122	122d 7Ah
335D5	BINT123	又名: BINT_122d 123d 7Bh
335DF	BINT124	124d 7Ch
地址	名称	说明
335E9	BINT125	125d 7Dh
335F3	BINT126	126d 7Eh
335FD	BINT127	127d 7Fh
33607	BINT128	128d 80h
		又名: BINT80h

地址	名称	说明
33611	BINT129	129d 81h
3361B	BINT130	130d 82h
33625	BINT131	又名: BINT130d, BINT_130d, XHI-1 131d 83h
3362F	(#8F)	又名: BINT_131d, BINT131d, XHI 143d 8Fh
33639	SYMBREAL	145d 91h
33643	(#92)	146d 92h
3364D	(#9A)	154d 9Ah
33657	SYMBUNIT	158d 9Eh
3EAFB	(#9F)	159d 9Fh
3366B	SYMOB	160d A0h
33675	SYMREAL	161d A1h
3367F	(#A2)	162d A2h
39E6B	(#A4)	164d A4h
33689	(#A5)	165d A5h
33693	SYMID	166d A6h
3369D	SYMLAM	167d A7h
336A7	(#A9)	169d A9h
336B1	SYMSYM	170d AAh
336BB	SYMEXT	174d AEh
3BD4C	(#AF)	175d AFh
336C5	(HXSREAL)	177d B1h
38275	(#BB)	187d BBh
336CF	(2HXS)	187d BBh
336D9	BINTC0h	192d C0h
3E7DA	(#C8)	200d C8h
336E3	2GROB	204d CCh
3BD65	(#CF)	207d CFh
336ED	TAGGEDANY	208d D0h
336F7	EXTREAL	225d E1h
33701	EXTSYM	234d EAh
3370B	2EXT	238d EEh
33715	ROMPANY	240d F0h
地址	名称	说明
3371F	BINT253	253d FDh
33729	BINT255d	255d FFh
33733	REALOBOB	256d 100h
3373D	#_102	258d 102h
33747	#SyntaxErr	262d 106h
33751	(BINT_263d)	263d 107h
3375B	(#110)	272d 110h
33765	3REAL	273d 111h
3E17B	(#111)	273d 111h
3376F	(Err#Kill)	291d 123h
33779	(Err#NoLstStk)	292d 124h

地址	名称	说明
2777E	(#12F)	303d 12Fh
33783	(#NoRoomForSt)	305d 131h
3378D	(#132)	306d 132h
33797	(REALSTRSTR)	307d 133h
337A1	(#134)	308d 134h
337AB	(#135)	309d 135h
337B5	(#136)	310d 136h
337BF	(#137)	311d 137h
337C9	(#138)	312d 138h
337D3	(#139)	313d 139h
337DD	(#13A)	314d 13Ah
337E7	(#13B)	315d 13Bh
337F1	(#13D)	317d 13Dh
337FB	(#13E)	318d 13Eh
33805	INTEGER337	337d 151h
3380F	(#200)	512d 200h
33819	(Err#NoLstArg)	517d 205h
3A1C2	(#304)	772d 304h
33823	STRREALREAL	785d 311h
3B9FA	(#313)	787d 313h
3C11E	(#410)	1040d 410h
3B928	(#411)	1041d 411h
3382D	(ARRYREALREAL)	1041d 411h
33837	(#412)	1042d 412h
3BA2D	(#414)	1044d 414h
3B93D	(#415)	1045d 415h
33841	(#444)	1092d 444h
3C10F	(#450)	1104d 450h
地址	名称	说明
3B952	(#451)	1105d 451h
3384B	(ARRYLISTREAL)	1105d 451h
33855	(#452)	1106d 452h
3BA18	(#454)	1108d 454h
3B913	(#455)	1109d 455h
3A12D	(#4FF)	1279d 4FFh
3385F	(#510)	1296d 510h
33869	(#511)	1297d 511h
3BA09	(#515)	1301d 515h
33873	(#550)	1360d 550h
277F6	(#605)	1541d 605h
27800	(#606)	1542d 606h
2780A	(#607)	1543d 607h
27814	(#608)	1544d 608h
2781E	(#609)	1545d 609h

地址	名称	说明
27828	(#60A)	1546d 60Ah
27832	(#60B)	1547d 60Bh
2783C	(#60C)	1548d 60Ch
27846	(#60D)	1549d 60Dh
2768E	(#60E)	1550d 60Eh
27698	(#60F)	1551d 60Fh
3387D	(IDREALOB)	1552d 610h
276AC	(#611)	1553d 611h
276B6	(#612)	1554d 612h
276C0	(#613)	1555d 613h
276CA	(#614)	1556d 614h
276D4	(#615)	1557d 615h
276DE	(#616)	1558d 616h
276E8	(#617)	1559d 617h
27792	(#618)	1560d 618h
2779C	(#619)	1561d 619h
277A6	(#61A)	1562d 61Ah
277B0	(#61B)	1563d 61Bh
277BA	(#61C)	1564d 61Ch
277C4	(#61D)	1565d 61Dh
277CE	(#61E)	1566d 61Eh
277D8	(#61F)	1567d 61Fh
277E2	(#620)	1568d 620h
277EC	(#621)	1569d 621h
地址	名称	说明
276F2	(#622)	1570d 622h
276FC	(#623)	1571d 623h
27706	(#624)	1572d 624h
27710	(#628)	1576d 628h
2771A	(#629)	1577d 629h
27724	(#62A)	1578d 62Ah
2772E	(#62B)	1579d 62Bh
27738	(#62C)	1580d 62Ch
27742	(#62D)	1581d 62Dh
27788	(#62E)	1582d 62Eh
33887	(IDLISTOB)	1616d 650h
33891	(#700)	1792d 700h
3C17A	(#710)	1808d 710h
3C16B	(#750)	1872d 750h
08DF7	(#7FF)	2047d 7FFh
27878	(#800)	2048d 800h
3B976	(#822)	2082d 822h
3C83C	(#82C)	2092d 82Ch
3B967	(#855)	2133d 855h

地址	名称	说明
3C81E	(#85C)	2140d 85Ch
3389B	(#861)	2145d 861h
338A5	(#862)	2146d 862h
338AF	(#865)	2149d 865h
338B9	(#86E)	2158d 86Eh
3E7FF	(#8F1)	2289d 8F1h
3E759	(#8FD)	2301d 8FDh
3E7E9	(#9F1)	2545d 9F1h
3E743	(#9FD)	2557d 9FDh
2774C	(#A01)	2561d A01h
27756	(#A02)	2562d A02h
27882	Attn#	2563d A03h
338C3	ATTNERR	2563d A03h
27760	(#A04)	2564d A04h
2776A	(#A05)	2565d A05h
27774	(#A06)	2566d A06h
338CD	(#A11)	2577d A11h
338D7	(#A12)	2578d A12h
338E1	(#A1A)	2586d A1Ah
338EB	(#A21)	2593d A21h
地址	名称	说明
338F5	(#A22)	2594d A22h
338FF	(#A2A)	2602d A2Ah
33909	(#A61)	2657d A61h
33913	(#A62)	2658d A62h
3391D	(#A65)	2661d A65h
33927	(#A6E)	2670d A6Eh
33931	(#AA1)	2721d AA1h
3393B	(#AA2)	2722d AA2h
33945	(#AAA)	2730d AAAh
3394F	(#C06)	3078d C06h
33959	(#C07)	3079d C07h
33963	(#C08)	3080d C08h
3396D	Connecting	3082d C0Ah
33977	(#C0B)	3083d C0Bh
3C800	(#C2C)	3116d C2Ch
3C7E2	(#C5C)	3164d C5Ch
3B904	(#C22)	3106d C22h
3B8F5	(#C55)	3157d C55h
33981	#CArmErr	3583d DFFh
3398B	EXTOBOB	3584d E00h
3C8D0	(#2111)	8465d 2111h
03FEF	(#2614)	9748d 2614h
03FF9	(#2686)	9862d 2686h

地址	名称	说明
03F8B	TYPEREAL	10547d 2933h
03FDB	(TYPEEREL)	10581d 2955h
03FA9	TYPEIDNT	10568d 2948h
03F95	(TYPECMP)	10615d 2977h
03F9F	(TYPELIST)	10868d 2A74h
20D6F	(TYPERRP)	10902d 2A96h
03FBD	(TYPESYMB)	10936d 2AB8h
03FE5	(TYPEEXT)	10970d 2ADAh
03FA9	(#2E48)	11848d 2E48h
03FD1	(TYPELAM)	11885d 2E6Dh
3C8DF	(#5B11)	23313d 5B11h
3D50D	(#A110)	41232d A110h
3D52B	(#A1A0)	41376 A1A0h
3D51C	(#AA10)	43536d AA10h
2C4D2	(#AAA0)	43680d AAA0h
3B7AD	(#BBBB)	48059d BBBBh
地址	名称	说明
08F1F	(#D6A8)	54952d D6A8h
38266	(#FFFF)	65535d FFFFh
03880	(#102A8)	66216d 102A8h
091B4	(#2D541)	185665d 2D541h
350F5	(#37258)	225880d 37258h
0803F	(#414C1)	267457d 414C1h
08ECE	(#536A8)	341672d 536A8h
0657E	(#61441)	398401d 61441h
33995	#EXITERR	458752d 70000h
03826	(#A8241)	688705d A8241h
39277	(#B437D)	738173d B437Dh
038DC	(#E13A8)	922536d E13A8h
3399F	MINUSONE	1048575d FFFFh

2.1.2 压入多个 BINT

地址	名称	说明
37287	ZEROZERO	(→ #0 #0)
37294	#ZERO#ONE	(→ #0 #1)
37305	#ZERO#SEVEN	(→ #0 #7)
36B12	ONEONE	(→ #1 #1) 又名: ONEDUP
37315	#ONE#27	(→ #1 #27d)
37328	#TWO#ONE	(→ #2 #1)
3733A	#TWO#TWO	(→ #2 #2)
3734A	#TWO#FOUR	(→ #2 #4)
3735C	#THREE#FOUR	(→ #3 #4)
3736E	#FIVE#FOUR	(→ #5 #4)
37380	ZEROZEROZERO	(→ #0 #0 #0)

地址	名称	说明
	ZEROZEROONE	(→ #0 #0 #1)
37394	ZEROZEROTWO	(→ #0 #0 #2)
373A8	DROPZERO	(ob → #0)
3558C	2DROP00	(ob ob → #0 #0)
355A5	DROPONE	(ob → #1)
3596D	DUPZERO	(ob → ob ob #0)
36AD6	DUPONE	(ob → ob ob #1)
36AEA	DUPTWO	(ob → ob ob #2)
36B26	SWAPONE	(ob ob' → ob' ob #1)
36AFE		
地址	名称	说明
	ZEROSWAP	(ob → #0 ob)
35E75	ZEROOVER	(ob → ob #0 ob)
360BB	ZEROFALSE	(→ #0 F)
36568	ONESWAP	(ob → #1 ob)
35EA2	ONEFALSE	(→ #1 F)
3657C		

2.1.3 转换

地址	名称	说明
	COERCE	(% → #)
262F1	COERCEDUP	(% → # #)
35D08	COERCESWAP	(ob % → # ob)
35EB6	COERCE2	(% %' → # #')
3F481	%ABSCOERCE	(% → #)
262EC	(COERCE&CKSGN)	(% → # flag) 实数大于 0 时返回 TRUE，否则返回 FALSE。
2F244	C%>#	(C% → # #')
2F31F	HXS>#	(hxs → #)
05A03	2HXSLIST?	({ hxs hxs' } → # #') 把由两个 hxs 组成的列表转换为两个 bint。输入无效时生成 “Bad Argument Value” 错误。
2F17E	CHR>#	(chr → #)
05A51	^Z2BIN	(Z → #) 把 Z 转换为 bint。溢出时返回 FFFFF；负数则返回 0。
0EF006	^Z>#	(z → #) 把 Z 强制转换为 #。Z<0 或 Z>9999 时生成溢出错误。使用 10000 可确保 #*6 能在 5 个半字节的字段中以 BCD 表示。
19D006	^COERCE2Z	(z2 z1 → #2 #1) 把两个 ZINT 转换为 bint。
0F0006		

2.1.4 算术函数

地址	名称	说明
	#+	(# #' → #+ #')
03DBC	#1+	(# → #+1)
03DEF	#2+	(# → #+2)
03E2D	#3+	(# → #+3)
355FD	#4+	(# → #+4)
35602	#5+	(# → #+5)
35607	#6+	(# → #+6)
3560C	#7+	(# → #+7)
35611	#8+	(# → #+8)
35616		

地址	名称	说明
3561B	#9+	(# → #+9)
35620	#10+	(# → #+10)
3562A	#12+	(# → #+12)
03DE0	#-	(##' → #-#')
2F13D	(CK#-)	(##' → #') 如果 #' 大于 #, 则返回 #0; 否则返回 #-#'。
03E0E	#1-	(# → #-1)
03E4E	#2-	(# → #-2)
355DF	#3-	(# → #-3)
355DA	#4-	(# → #-4)
355D5	#5-	(# → #-5)
355D0	#6-	(# → #-6)
03EC2	#*	(##' → #*#')
2632D	#*OVF	(##' → #*#') 0 ≤ 结果 ≤ FFFFF
03E6F	#2*	(# → #*2)
356B8	#6*	(# → #*6)
3569B	#8*	(# → #*8)
35675	#10*	(# → #*10)
03EF7	#/	(##' → #r #q)
03E8E	#2/	(# → #/2) 向下取整。
36815	#1-	(##' → #-#'+1) 又名: #-+1
36851	#1-+	(##' → #+#+-1) \$1-+ 是 EXTABLE 中的笔误。 又名: #+-1, \$1-+
地址	名称	说明
35552	#-#2/	(##' → (-#-#')/2)
357FC	#+DUP	(##' → #+#+#+#')
35E39	#+SWAP	(ob # #' → #+##' ob)
36093	#+OVER	(ob # #' → ob #+##' ob)
3581F	#-DUP	(##' → #-#'-#-#')
35E4D	#-SWAP	(ob # #' → #-#-' ob)
360A7	#-OVER	(ob # #' → ob #-#-' ob)
35830	#1+DUP	(# → #+1 #+1)
35E61	#1+SWAP	(ob # → #+1 ob)
2F222	#1+ROT	(ob ob' # → ob' #+1 ob)
35841	#1-DUP	(# → #-1 #-1)
28071	#1-SWAP	(ob # → #-1 ob) 又名: pull
3601B	#1-ROT	(ob ob' # → ob' #-1 ob)
281D5	#1-UNROT	(ob ob' # → #-1 ob ob')
35E89	#1-1SWAP	(# → 1 #-1) 返回 bint ONE 和运算结果。
35912	DUP#1+	(# → # #+1)
3571E	DUP#2+	(# → # #+2)
35956	DUP#1-	(# → # #-1)
3674D	2DUP#+	(##' → ##' #+##') 又名: DUP3PICK#+
3683D	DROP#1-	(# ob → #-1)
357BB	SWAP#-	(##' → #'-#)

地址	名称	说明
3592B	SWAP#1+	(# ob → ob #+1) 又名: SWP1+
29786	('RSWAP#1+)	(# → nob #+1) nob 是运行流中的下一个对象。
28099	SWAP#1+SWAP	(# ob → #+1 ob)
36829	SWAP#1-	(# ob → ob #-1)
280AD	SWAP#1-SWAP	(# ob → #-1 ob)
28989	(SWAPDROP#1-)	(ob # → #-1)
367ED	SWAPOVER#-	(# #' → #' #-#)
36775	OVER#+	(# #' → #' #'+#)
367C5	OVER#-	(# #' → #' #-#)
36761	ROT#+	(# ob #' → ob #' +#)
367B1	ROT#-	(# ob #' → ob #' -#)
36801	ROT#1+	(# ob ob' → ob ob' #+1)
28001	ROT#1+UNROT	(# ob ob' → #+1 ob ob')
地址	名称	说明
35E07	ROT#+SWAP	(# ob #' → #' +# ob) 又名: ROT+SWAP
36789	3PICK#+	(# ob #' → # ob #' +#)
3679D	4PICK#+	(# ob1 ob2 #' → # ob1 ob2 #' +#)
35E20	4PICK#+SWAP	(# ob1 ob2 #' → # ob1 #' +# ob2) 又名: 4PICK+SWAP
35511	#MIN	(# #' → #')
3551D	#MAX	(# #' → #')
03EB1	#AND	(# #' → #') 按位 AND。

2.1.5 测试

地址	名称	说明
03D19	#=	(# → flag)
03D4E	#<>	(# → flag)
03CE4	#<	(# → flag)
03D83	#>	(# → flag)
03CC7	#0<>	(# → flag)
03CA6	#0=	(# → flag)
3530D	#1<>	(# → flag)
352FE	#1=	(# → flag)
36711	#2<>	(# → flag)
352F1	#2=	(# → flag)
352E0	#3=	(# → flag)
366FD	#5=	(# → flag)
366BC	#<3	(# → flag)
36739	#>1	(# → flag) 又名: ONE#>
358C2	2DUP#<	(# #' → # #' flag)
358F8	2DUP#>	(# #' → # #' flag)
363CE	ONE_EQ	(# → flag) 使用 EQ 测试。
35268	OVER#=#	(# #' → # flag)
358DC	2DUP#=#	(# #' → # #' flag)
36694	OVER#0=	(# #' → # #' flag)

地址	名称	说明
352BD	DUP#0=	(# → # flag)
366A8	OVER#<	(# #' → # flag)
地址	名称	说明
3531C	DUP#1=	(# → # flag)
36725	OVER#>	(# #' → # flag)
3532B	DUP#0<>	(# → # flag)
366D0	DUP#<7	(# → # flag) 如果参数小于 #7，则返回 TRUE。
36676	2#0=OR	(# # → flag) 如果任一参数为零，则返回 TRUE。

第 3 章 实数

创建实数有两种方法。第一种是不加任何前缀，直接输入数字。但这种方法也可用来创建 **bint**。那么，编译器如何判断您要的是实数还是 **bint**？如果数字含有小数点和 / 或指数，它就是实数；否则就是 **bint**。

由于两者容易混淆，建议使用结构 `% <dec>`。这样可以确保得到实数，代码也更易读。

与 **bint** 一样，系统也内置了许多实数，具体见下文列表。

实数的基本运算是 `%+`、`%-`、`%*`、`%/` 和 `%^`。除此之外还有许多运算，也列在下文。

还有一种用户和用户 **RPL** 程序都不能直接访问的实数，即扩展实数（也称长实数）。它们的工作方式与普通实数相同，但有两点区别：精度为 15 位，而普通实数为 12 位；指数范围为 -50000 到 50000。

扩展实数使用 `%% <dec>` 创建。如果堆栈中出现扩展实数，其显示方式与普通实数相同，但始终采用科学记数法。基本运算也相同，只是命令前缀由 `%` 改为 `%%`。这里需要明确一点：在用户 **RPL** 中，`+` 可以对各种对象执行加法，例如实数、用户级二进制整数（即后文将介绍的十六进制字符串），还可以向列表添加元素等。在系统 **RPL** 中，命令字 `%+` 只适用于两个实数。两个二进制整数相加必须使用 `#+`，扩展实数相加则使用 `%%+`。如果用错误类型的参数调用函数，系统可能崩溃。

用命令 `%>%%` 可把实数转换为扩展实数，反向转换使用 `%%>%`。用 **UNCOERCE** 可把 **bint** 转换为普通实数，反向转换函数是

COERCE。下文列出更多转换函数及其他与实数相关的函数。

3.1 参考

3.1.1 内置实数

地址	名称	说明
2FB0A	<code>%-MAXREAL</code>	-9.99E499
2FAB1	<code>%-9</code>	-9
2FA9C	<code>%-8</code>	-8
2FA87	<code>%-7</code>	-7
2FA72	<code>%-6</code>	-6
2FA5D	<code>%-5</code>	-5
2FA48	<code>%-4</code>	-4
2FA33	<code>%-3</code>	-3
2FA1E	<code>%-2</code>	-2
2FA09	<code>%-1</code>	-1
2FB34	<code>%-MINREAL</code>	-1E-499
2F937	<code>%0</code>	0
2FB1F	<code>%MINREAL</code>	1E-499
27118	<code>%.1</code>	.1
339BE	<code>%.5</code>	.5
339D3	<code>(%-.5)</code>	-.5
2F94C	<code>%1</code>	1
270EE	<code>(%1.8)</code>	1.8
2F961	<code>%2</code>	2
339A9	<code>%e</code>	e
2F976	<code>%3</code>	3
2FAC6	<code>%PI</code>	π
2F98B	<code>%4</code>	4
2F9A0	<code>%5</code>	5
2F9B5	<code>%6</code>	6

地址	名称	说明
2F9CA	%7	7
2F9DF	%8	8
2F9F4	%9	9
339E8	%10	10
2FCE6	%11	11
地址	名称	说明
2FCFB	%12	12
2FD10	%13	13
2FD25	%14	14
2FD3A	%15	15
2FD4F	%16	16
2FD64	%17	17
2FD79	%18	18
2FD8E	%19	19
2FDA3	%20	20
2FDB8	%21	21
2FDCD	%22	22
2FDE2	%23	23
2FDF7	%24	24
2FE0C	%25	25
2FE21	%26	26
2FE36	%27	27
2FE4B	(%28)	28
2FE60	(%29)	29
2FE75	(%30)	30
2FE8A	(%31)	31
2FE9F	(%32)	32
2FEB4	(%33)	33
2FEC9	(%34)	34
2FEDE	(%35)	35
27103	%80	80
27E5D	%100	100
339FD	%180	180
33A12	(%200)	200
33A3C	(%400)	400
33A27	%360	360
2FC7D	(%1200)	1200
2FC92	(%2400)	2400
2FCA7	(%4800)	4800
4EA22	(%TICKSsec)	8192
2FCBC	(%9600)	9600
2FCD1	(%15360)	15360
4EA37	(%TICKSmin)	491520
4EA4C	(%TICKShour)	29491200
4EA61	(%TICKSday)	707788800

地址	名称	说明
4EA76	(%TICKSweek)	4954521600
2FAF5	%MAXREAL	9.99E499
2F180	1REV	(→ 6.28318530718) (→ 360.) (→ 400.) 根据当前角度模式，返回一周所对应的角度。

3.1.2 内置扩展实数

地址	名称	说明
2FB49	%%0	0
2FBE5	%%.1	0.1
30DC8	%%.4	0.4
2FBFF	%%.5	0.5
2DA11	cF	0.555... 用于 C↔F 转换的%5/9。
2FB63	%1	1
2DA2B	cC	1 用于 C↔K 转换。
2FB7D	%2	2
2FB97	%3	3
2FADB	%PI	π
30017	PI/180	$\pi/180$
2FBB1	%4	4
2FBCB	%5	5
27A89	%2PI	2π
30BEA	%7	7
2FC19	%10	10
30CC7	%12	12
30CEB	%60	60

3.1.3 与实数组合的堆栈操作

地址	名称	说明
282CC	(DROP%0)	(ob → %0)

3.1.4 转换

地址	名称	说明
2FFAC	%>%	(% → %)
35ECA	%>%SWAP	(ob % → % ob)
2FF9B	%>%	(%% → %)
30E47	2%>%	(% % → % % %)
30E5B	2%>%	(% % %' → % %')
262F6	UNCOERCE	(# → %)
3F495	UNCOERCE2	(# # → % %)
36BFA	UNCOERCE%	(# → % %)
2EFCA	HXS>%	(hxs → %)
05D2C	C%>%	(C% → %re %im)
2B3FD	%IP>#	(% → #IP(ABS(%))) 同时执行 ABS。
0F6006	^Z>R	(Z → %) 把 ZINT 转换为实数。
18A006	^Z2%	(Z → % %) 把整数转换为长实数。

地址	名称	说明
197006	$\wedge$ OBJ2REAL	(z/% $\rightarrow$ %) 把 ob 转换为实数。

3.1.5 实数函数

地址	名称	说明
3035F	%+	(% %' $\rightarrow$ %+')
25E69	%+SWAP	(ob % %' $\rightarrow$ %+%' ob)
26F36	%1+	(% $\rightarrow$ %1)
3036C	%-	(% %' $\rightarrow$ %-')
26F4A	%1-	(% $\rightarrow$ %1)
30346	%>%%-	(% %' $\rightarrow$ %%-%')
303A7	%*	(% %' $\rightarrow$ %*%')
地址	名称	说明
35C18	%10*	(% $\rightarrow$ %*10)
303E9	%/	(% %' $\rightarrow$ %/')
3045B	%^	(% %' $\rightarrow$ %^%')
302EB	%ABS	(% $\rightarrow$ %')
3030B	%CHS	(% $\rightarrow$ -%)
302C2	%SGN	(% $\rightarrow$ -1/0/1)
3049A	%1/	(% $\rightarrow$ 1/%)
30489	%>%%1/	(% $\rightarrow$ 1/%%)
304F4	%SQRT	(% $\rightarrow$ $\sqrt{\%}$)
304E1	%>%%SQRT	(% $\rightarrow$ $\sqrt{\%}\%$)
3051A	%EXP	(% $\rightarrow$ e^%)
3052D	%EXPM1	(% $\rightarrow$ e^%-1)
30559	%LN	(% $\rightarrow$ LN%)
30592	%LNP1	(% $\rightarrow$ LN(%+1))
3056C	%LOG	(% $\rightarrow$ LOG%)
305A5	%ALOG	(% $\rightarrow$ 10^%)
305DA	%SIN	(% $\rightarrow$ SIN%)
3062B	%COS	(% $\rightarrow$ COS%)
3067C	%TAN	(% $\rightarrow$ TAN%)
306AC	%ASIN	(% $\rightarrow$ ASIN%)
306DC	%ACOS	(% $\rightarrow$ ACOS%)
3070C	%ATAN	(% $\rightarrow$ ATAN%)
30799	%SINH	(% $\rightarrow$ SINH%)
307C5	%COSH	(% $\rightarrow$ COSH%)
307D8	%TANH	(% $\rightarrow$ TANH%)
307EB	%ASINH	(% $\rightarrow$ ASINH%)
307FE	%ACOSH	(% $\rightarrow$ ACOSH%)
30811	%ATANH	(% $\rightarrow$ ATANH%)
3031B	%MANTISSA	(% $\rightarrow$ %mant)
30824	%EXPONENT	(% $\rightarrow$ %expn)
30938	%FP	(% $\rightarrow$ %frac)
3094B	%IP	(% $\rightarrow$ %int)
30971	%FLOOR	(% $\rightarrow$ %maxint <=%)

地址	名称	说明
3095E	%CEIL	(% → %minint >=%)
305C7	%MOD	(% %' → %rem)
30723	%ANGLE	(%x %y → %ang)
30746	%>%ANGLE	(%x %y → %%ang)
30F14	RNDXY	(% %places → %')
地址	名称	说明
30F28	TRCXY	(% %places → %')
3084D	%COMB	(% %' → COMB(%, %'))
30860	%PERM	(% %' → PERM(%, %'))
30837	%NFACT	(% → %!) 计算数字的阶乘。
30AAF	%FACT	(% → gamma(%+1)) 计算 gamma(x+1)。
3046C	%NROOT	(% %n → %') 计算实数的第 %n 次根。等同于用户函数 XROOT。
300F9	%MIN	(% %' → %lesser)
300E0	%MAX	(% %' → %greater)
35DBC	%MAXorder	(% %' → %max %min)
309AD	%RAN	(→ %random) 返回下一个随机数。
30A2F	%RANDOMIZE	(%seed →) 系统级 RDZ: 为随机数生成器设置种子。
30A66	DORANDOMIZE	(% →) 把给定数字存为随机数种子。
303B4	%OF	(% %' → %'/% * 100)
303F6	%T	(% %' → %pcttotal)
3041B	%CH	(% %' → %pcchange)
3000D	%D>R	(%deg → %rad)
30040	%R>D	(%rad → %deg)
30E79	%REC>%POL	(%r %ang → %x %y)
30EA6	%POL>%REC	(%x %y → %r %ang)
30EDD	%SPH>%REC	(%r %ang %ph → %x %y %z)

3.1.6 扩展实数函数

地址	名称	说明
3032E	%%+	(%% %%' → %%+%%')
3033A	%%-	(%% %%' → %%-%')
30385	%%*	(%% %%' → %%*%')
3602F	%%*ROT	(ob ob' %% %%' → ob' %%+%%' ob)
35EDE	%%*SWAP	(ob %% %%' → %%+%%' ob)
36C7C	%%*UNROT	(ob ob' %% %%' → +%+%%' ob ob')
地址	名称	说明
303D3	%%/	(%% %%' → %%/%')
36C22	SWAP%%/	(%% %%' → %%")
36BE6	%%/>%	(%% %%' → %)
3044A	%%^	(%% %%' → %%^%')
51D006	^CK%%SQRT	(%% → %%/C%%)
30612	%%SINRAD	(%% → %%')
30767	%%ANGLERAD	(%% → %%')
302DB	%%ABS	(%% → %%abs)
306F3	%%ACOSRAD	(%% → %%rad)

地址	名称	说明
3073A	%%ANGLE	(%%x %%y → %%ang)
30757	%%ANGLEDEG	(%%x %%y → %%deg)
306C3	%%ASINRAD	(%% → %%rad)
302FB	%%CHS	(%% → -%%)
3047D	%%1/	(%% → 1/%%)
30642	%%COS	(%% → %%cos)
30653	%%COSDEG	(%%deg → %%cos)
307B2	%%COSH	(%% → %%cosh)
30663	%%COSRAD	(%%rad → %%cos)
30507	%%EXP	(%% → e^%%)
30546	%%LN	(%% → ln %%)
30984	%%FLOOR	(%% → %%maxint) 又名: %%INT
3057F	%%LNPI	(%% → %%ln(%%+1))
300C7	%%MAX	(%% %%' → %%max)
30E83	%%R>P	(%%x %%y → %%radius %%angle)
30EB0	%%P>R	(%%r %%ang → %%x %%y)
305F1	%%SIN	(%% → %%sin)
30602	%%SINDEG	(%%deg → %%sin)
30780	%%SINH	(%% → %%sinh)
304D5	%%SQRT	(%% → √%%)
30693	%%TANRAD	(%%rad → %%tan)

3.1.7 测试

地址	名称	说明
302AC	%=	(%%' → flag)
302B7	%<>	(%%' → flag)
3025C	%<	(%%' → flag)
302A1	%<=	(%%' → flag)
30275	%>	(%%' → flag)
3028B	%>=	(%%' → flag)
30156	%0=	(% → flag)
36C0E	DUP%0=	(% → flag)
301BA	%0<>	(% → flag) 可用于把用户标志转换为系统标志。
30123	%0<	(% → flag)
30184	%0>	(% → flag)
301E2	%0>=	(% → flag)
3020A	%%<	(%% %%' → flag)
30296	%%<=	(%% %%' → flag)
3026A	%%>	(%% %%' → flag)
30280	%%>=	(%% %%' → flag)
30145	%%0=	(%% → flag)
301A6	%%0<>	(%% → flag)
30112	%%0<	(%% → flag)
301F6	%%0<=	(%% → flag)
30173	%%0>	(%% → flag)

地址	名称	说明
301CE	%%0>=	(%% → flag)

第 4 章 复数

可用结构 `C% <real> <imag>` 在程序中插入复数。实部和虚部均为十进制形式的实数。如果堆栈中已有实部和虚部，命令 `%>C%` 会用它们创建复数。命令 `C%>%` 接受一个复数，并返回其实部和虚部。

系统还提供用户不能直接访问的扩展复数（也称长复数）。这类复数的实部和虚部都是扩展实数。可用 `C%% <real> <imag>` 将其插入程序，其中实部和虚部均为扩展实数。它们在堆栈中的显示方式与普通复数相同，但始终采用科学记数法。

下文列出所有与复数相关的命令，包括数学运算。

4.1 参考

4.1.1 内置复数

地址	名称	说明
	<code>C%0</code>	$(0,0)$
27DE4	<code>C%1</code>	$(1,0)$
27E09	<code>C%-1</code>	$(-1,0)$
27DBF	<code>C%%1</code>	$(\%1,\%0)$
27E2E		

4.1.2 转换

地址	名称	说明
	<code>C%%>C%</code>	$(C\% \rightarrow C\%)$
261D9	<code>%>C%</code>	$(\%re\ \%im \rightarrow C\%)$
05C27	<code>SWAP%>C%</code>	$(\%im\ \%re \rightarrow C\%)$
362F2	<code>Re>C%</code>	$(\%re \rightarrow C\%)$
261FC	<code>C>Re%</code>	$(C\% \rightarrow \%re)$
25E9C	<code>C>Im%</code>	$(C\% \rightarrow \%im)$
25E9B	<code>^E%>C%%</code>	$(\%re\ \%im \rightarrow C\%) $ 把长实数转换为长复数。
18C006	<code>%>C%</code>	$(\%re\ \%im \rightarrow C\%)$
261CF	<code>C%>%</code>	$(C\% \rightarrow \%re\ \%im)$
25E82	<code>C%>%SWAP</code>	$(C\% \rightarrow \%im\ \%re)$
25E83	<code>C%%>%</code>	$(C\% \rightarrow \%re\ \%im)$
05DBC	<code>^C2C%%</code>	$(C \rightarrow C\%) $ 把高斯整数转换为长复数。
188006	<code>^ZZ2C%%ext</code>	$(Zre\ Zim \rightarrow C\%) $ 把高斯整数转换为长复数。
189006	<code>^C%>C%%</code>	$(C\% \rightarrow C\%) $ 把复数转换为长复数。
18B006	<code>^RIXCext</code>	$(Zre\ Zim \rightarrow C) $ 把整数转换为复数。
15E006	<code>^IRXCext</code>	$(Zim\ Zre \rightarrow C) $ 把整数转换为复数。
15F006		

4.1.3 函数

地址	名称	说明
	<code>C%C^C</code>	$(C\% C\%' \rightarrow C\%'')$
25E8F	<code>C%C^R</code>	$(C\% \% \rightarrow C\%')$
25E90	<code>C%R^C</code>	$(\% C\% \rightarrow C\%')$
25E94	<code>C%ABS</code>	$(C\% \rightarrow \%)$
25E84	<code>^CZABS</code>	$(complex \rightarrow real) $ 绝对值。
50C006	<code>C%CHS</code>	$(C\% \rightarrow -C\%)$
261ED	<code>C%1/</code>	$(C\% \rightarrow 1/C\%)$
25E81	<code>C%SQRT</code>	$(C\% \rightarrow \sqrt{C\%})$
25E98		

地址	名称	说明
25E95	C%SGN	(C% → C%/C%ABS)
地址	名称	说明
261F2	C%CONJ	(C% → C%')
25E88	C%ARG	(C% → %)
25E91	C%EXP	(C% → e^C%)
25E92	C%LN	(C% → ln C%)
25E93	C%LOG	(C% → log C%)
25E87	C%ALOG	(C% → 10^C%)
25E96	C%SIN	(C% → sin C%)
25E8D	C%COS	(C% → cos C%)
25E99	C%TAN	(C% → tan C%)
25E89	C%ASIN	(C% → asin C%)
25E85	C%ACOS	(C% → acos C%)
25E8B	C%ATAN	(C% → atan C%)
25E97	C%SINH	(C% → sinh C%)
25E8E	C%COSH	(C% → cosh C%)
25E9A	C%TANH	(C% → tanh C%)
25E8A	C%ASINH	(C% → asinh C%)
25E86	C%ACOSH	(C% → acosh C%)
25E8C	C%ATANH	(C% → atanh C%)
261DE	C%%CHS	(C%% → -C%%)
261E3	C%%CONJ	(C%% → C%%')
515006	^ARG2	(im re → arg(ob)) ARG。
517006	^QUADRANT	(re im ?re>0 ?im>0 → newre newim %) 返回 Z0、Z1、Z-2 或 Z-1，使相应复数的辐角为 Z * π/2 + theta，其中θ 位于区间 [0,π/2]。
51E006	^C%%SQRT	(C%% → C%%')

4.1.4 测试

地址	名称	说明
261E8	C%0=	(C% → flag)
261D4	C%%0=	(C%% → flag)

第 5 章 整数 (ZINT)

这是 HP49 新增的一种对象。整数（简称 ZINT）是一种可以表示任意大整数的数值类型。

多数情况下，您不必专门处理用户作为程序参数输入的整数。类型检查机制（见[第 29.2 节](#)）通常会自动把 ZINT 转换为实数。

不过，如果您希望直接处理整数，系统也提供了若干针对 ZINT 的函数。由于这种对象类型实际上属于 HP49 计算机代数系统 (CAS)，这里不再说明这些函数；有关 ZINT 的文档，请参见[第 42 章](#)。

第 6 章

字符和字符串

字符和字符串是两种用于保存文本的数据类型。

用户不能直接使用字符对象。一个字符对象只能保存一个字符。您可以用 `CHR <char>` 创建字符，也可以使用下文列出的众多内置字符。用 `CHR>#` 可将字符转换为 **bint**；返回的 **bint** 是该字符的 **ASCII** 码。反向转换使用 `#>CHR`。在程序中可用 `$ "<string>"` 插入字符串，也可以直接写成 `"<string>"`。下文列出了一些内置字符串。命令 `CHR>$` 可将字符转换为字符串。

`LEN$` 和 `&$` 是两个简单实用的字符串函数。`LEN$` 以 **bint** 返回字符串的长度（字符数），`&$` 则连接两个字符串。若要取得子字符串，即字符串的一部分，请使用 `SUB$`。它接受三个参数：原字符串、起始位置（**bint**）和结束位置（也是 **bint**）。位置从 1 开始计数，函数会返回起始字符到结束字符之间的全部内容（含两端）。另一个函数 `POS$` 用于搜索：待搜索字符串放在堆栈第 3 层，要查找的字符或字符串放在堆栈第 2 层，指定起始位置的 **bint** 放在堆栈第 1 层。函数会把查找内容第一次出现的位置以 **bint** 返回到堆栈第 1 层；若未找到，则返回 **#0**。其他相关函数见下文列表。

6.1 参考

6.1.1 内置字符

地址	名称	说明
33D2B	CHR_00	'\00'（字符 0d、00h） 空字符。
33F77	CHR_Newline	'\0a'（字符 10d、0Ah） 换行字符。
33D32	CHR_...	'...'（字符 31d、1Fh）
33F93	CHR_Space	' '（字符 32d、20h） 空格字符。
33D39	CHR_DblQuote	'"'（字符 34d、22h）
33D40	CHR_#	'#'（字符 35d、23h）
33F70	CHR_LeftPar	'('（字符 40d、28h）
33F85	CHR_RightPar	')'（字符 41d、29h）
33D47	CHR_*	'*'（字符 42d、2Ah）
33D4E	CHR_+	'+'（字符 43d、2Bh）
33D55	CHR_,	','（字符 44d、2Ch）
33D5C	CHR_-	'-'（字符 45d、2Dh）
33D63	CHR_.	'.'（字符 46d、2Eh）
33D6A	CHR_/	'/'（字符 47d、2Fh）
33D71	CHR_0	'0'（字符 48d、30h）
33D78	CHR_1	'1'（字符 49d、31h）
33D7F	CHR_2	'2'（字符 50d、32h）
33D86	CHR_3	'3'（字符 51d、33h）
33D8D	CHR_4	'4'（字符 52d、34h）
33D94	CHR_5	'5'（字符 53d、35h）
33D9B	CHR_6	'6'（字符 54d、36h）
33DA2	CHR_7	'7'（字符 55d、37h）
33DA9	CHR_8	'8'（字符 56d、38h）
33DB0	CHR_9	'9'（字符 57d、39h）
33DB7	CHR_:	':'（字符 58d、3Ah）
33DBE	CHR_;	';'（字符 59d、3Bh）
33DC5	CHR_<	'<'（字符 60d、3Ch）
33DCC	CHR_=	'='（字符 61d、3Dh）

地址	名称	说明
33DD3	CHR_>	'>' (字符 62d、3Eh)
33DDA	CHR_A	'A' (字符 65d、41h)
33DE1	CHR_B	'B' (字符 66d、42h)
地址	名称	说明
33DE8	CHR_C	'C' (字符 67d、43h)
33DEF	CHR_D	'D' (字符 68d、44h)
33DF6	CHR_E	'E' (字符 69d、45h)
33DFD	CHR_F	'F' (字符 70d、46h)
33E04	CHR_G	'G' (字符 71d、47h)
33E0B	CHR_H	'H' (字符 72d、48h)
33E12	CHR_I	'I' (字符 73d、49h)
33E19	CHR_J	'J' (字符 74d、4Ah)
33E20	CHR_K	'K' (字符 75d、4Bh)
33E27	CHR_L	'L' (字符 76d、4Ch)
33E2E	CHR_M	'M' (字符 77d、4Dh)
33E35	CHR_N	'N' (字符 78d、4Eh)
33E3C	CHR_O	'O' (字符 79d、4Fh)
33E43	CHR_P	'P' (字符 80d、50h)
33E4A	CHR_Q	'Q' (字符 81d、51h)
33E51	CHR_R	'R' (字符 82d、52h)
33E58	CHR_S	'S' (字符 83d、53h)
33E5F	CHR_T	'T' (字符 84d、54h)
33E66	CHR_U	'U' (字符 85d、55h)
33E6D	CHR_V	'V' (字符 86d、56h)
33E74	CHR_W	'W' (字符 87d、57h)
33E7B	CHR_X	'X' (字符 88d、58h)
33E82	CHR_Y	'Y' (字符 89d、59h)
33E89	CHR_Z	'Z' (字符 90d、5Ah)
33FA1	CHR_['	'[' (字符 91d、5Bh)
33FA8	CHR_]'	']' (字符 93d、5Dh)
33F9A	CHR_UndScore	'_' (字符 95d、5Fh)
33E90	CHR_a	'a' (字符 97d、61h)
33E97	CHR_b	'b' (字符 98d、62h)
33E9E	CHR_c	'c' (字符 99d、63h)
33EA5	CHR_d	'd' (字符 100d、64h)
33EAC	CHR_e	'e' (字符 101d、65h)
33EB3	CHR_f	'f' (字符 102d、66h)
33EBA	CHR_g	'g' (字符 103d、67h)
33EC1	CHR_h	'h' (字符 104d、68h)
33EC8	CHR_i	'i' (字符 105d、69h)
33ECF	CHR_j	'j' (字符 106d、6Ah)
33ED6	CHR_k	'k' (字符 107d、6Bh)
33EDD	CHR_l	'l' (字符 108d、6Ch)

地址	名称	说明
33EE4	CHR_m	'm' (字符 109d、5Dh)
33EEB	CHR_n	'n' (字符 110d、6Eh)
33EF2	CHR_o	'o' (字符 111d、6Fh)
33EF9	CHR_p	'p' (字符 112d、70h)
33F00	CHR_q	'q' (字符 113d、71h)
33F07	CHR_r	'r' (字符 114d、72h)
33F0E	CHR_s	's' (字符 115d、73h)
33F15	CHR_t	't' (字符 116d、74h)
33F1C	CHR_u	'u' (字符 117d、75h)
33F23	CHR_v	'v' (字符 118d、76h)
33F2A	CHR_w	'w' (字符 119d、77h)
33F31	CHR_x	'x' (字符 120d、78h)
33F38	CHR_y	'y' (字符 121d、79h)
33F3F	CHR_z	'z' (字符 122d、7Ah)
33FAF	CHR_{	'{' (字符 123d、7Bh)
33FB6	CHR_}	'}' (字符 125d、7Dh)
33F5B	CHR_Angle	'∠' (字符 128d、80h)
33F69	CHR_Integral	'∫' (字符 132d、84h)
33F62	CHR_Deriv	'∂' (字符 136d、88h)
33F46	CHR_→	'→' (字符 141d、8Dh)
33F4D	CHR_<<	'«' (字符 171d、ABh)
33F54	CHR_>>	'»' (字符 187d、BBh)
33F7E	CHR_Pi	'π' (字符 135d、87h)
33F8C	CHR_Sigma	'Σ' (字符 133d、85h)
33FBD	CHR_<=	'≤' (字符 137d、89h)
33FC4	CHR_>=	'≥' (字符 138d、8Ah)
33FCB	CHR_<>	'≠' (字符 139d、8Bh)

6.1.2 内置字符串

地址	名称	说明
055DF	NULL\$	""
33B55	SPACE\$	空字符串。 " "
33B39	NEWLINE\$	又名: tok_ "\0a" 换行符。
地址	名称	说明
27195	CRLF\$	"\0d\0a" 回车符和换行符。
340A4	\$_RAD	"RAD"
340B4	\$_GRAD	"GRAD"
33FF2	\$_XYZ	"XYZ"
33FE2	\$_R<Z	"R∠Z" "R<angle>Z"
33FD2	\$_R<<	"R∠∠" "R<angle><angle>"
34076	\$_EXIT	"EXIT"
34064	\$_ECHO	"ECHO"
34088	\$_Undefined	"Undefined"

地址	名称	说明
	\$_<>>	"«»"
34002	\$_{}	"{}"
34010	\$_[]	"[]"
3401E	\$_''	""
3402C		两个单引号。
	\$_::	"::"
3403A	\$_LRParens	"()"
34048	\$_2DQ	"\""
34056		两个双引号。
	tok,	","
33B91		""
	tok'	一个单引号。
33B85		"_"
	tok-	
33BFD		"."
	tok.	
33B9D		"g"
2D848	tok_g	
	tok_m	"m"
2D86D		"s"
	tok_s	
2D8AD		"{"
	tok{	
33A77		"«"
	tok<<	
33AD7		"="
	tok=	
33C09		"→"
	tok->	
272D9		"0"
	tok0	
33C4D		"1"
	tok1	
33C59		"8"
	tok8	
33CAD		"9"
	tok9	
33CB9		"<ESC>"
	tokESC	转义字符。
33ABF		
地址	名称	说明
	tokexponent	"E"
33AE3		"\""
	tokquote	一个双引号。
33B79		"#"
	toksharp	
33A8F		"\$"
	(tok\$)	
33AA7		"&"
	(tok&)	
33AB3		"*"
	(tok*)	
33BD9		"+"
	(tok+)	
33BF1		"/"
	(tok/)	
33BE5		"2"
	(tok2)	
33C65		"3"
	(tok3)	
33C71		"4"
	(tok4)	
33C7D		"5"
	(tok5)	
33C89		"6"
	(tok6)	
33C95		"7"
	(tok7)	
33CA1		"8"
	(tok8;)	
33BA9		"»"
	(tok>>)	
33ACB		"∠"
	(tokanglesign)	
33AEF		"GROB"
	(tokCTGROB)	
33C2D		"C\$"
	(tokCTSTR)	
33C3F		"∂"
	(tokDER)	
33C21		

地址	名称	说明
	(\$DER)	"der"
33B45	(toklparen)	"{"
33BB5	(tokrparen)	"}"
33BC1	(tokSIGMA)	"Σ"
33AFB	(tokSQRT)	"√"
33C15	(tokUNKNOWN)	"UNKNOWN"
33B61	(tokuscore)	"_"
33A9B	(tokWHERE)	" "
33B07	(tok[)	"["
33A6B	(tok])	"]"
33A51	(tok^)	"^"
33BCD	(tok{)}	"}"
33A83	(14SPACES\$)	" "
33B13		由 14 个空格组成的字符串。

6.1.3 带堆栈操作的内置字符串

地址	名称	说明
35D94	NULL\$SWAP	(ob → \$ ob) 先执行 NULL\$, 再执行 SWAP。
04D3E	DROPNULL\$	(ob → NULL\$) 先执行 DROP, 再执行 NULL\$。
25EEC	NULL\$TEMP	(→ \$) 在临时内存中创建空字符串 (先执行 NULL\$, 再执行 TOTEMPOB)。

6.1.4 转换

地址	名称	说明
25F77	#>\$	(# → \$) 由 bint 创建十进制字符串。
25F72	#:>\$	(# → "#:") 由 bint 创建字符串, 并在末尾添加冒号和空格。例如: "1: "
25F0F	a%>\$	(% → \$) 按当前显示模式把实数转换为字符串。 又名: a%>\$,
05BE9	ID>\$	(id/lam → \$) 把标识符转换为字符串。
25EB3	DOCHR	(% → \$) 创建由指定编号所对应字符组成的字符串。
0F1006	^Z>\$	(Z → \$) 把 Z 转换为十进制字符串。
2EFC1	hxs>\$	(hxs → \$) 使用当前显示模式和字长。
2EFC0	HXS>\$	(hxs → \$) 先执行 hxs>\$, 再添加进制字符。

6.1.5 管理

地址	名称	说明
05A75	#>CHR	(# → chr) 返回 ASCII 码为指定值的字符。
37AA5	CHR>\$	(chr → \$* Strings) 把字符转换为字符串。
05636	LEN\$	(\$ → #length) 返回以字节计的长度。
357E2	DUPLEN\$	(\$ → \$#) 先执行 DUP, 再执行 LEN\$。
05622	OVERLEN\$	(\$ ob → \$ ob #len) 先执行 OVER, 再执行 LEN\$。
361DA	NEWLINE\$&\$	(\$ → "\$\0a") 在字符串末尾添加换行符。 又名: NEWLINE&\$
2F31A	APNDCRLF	(\$ → \$') 在字符串末尾添加回车符和换行符。
050ED	CAR\$	(\$ → chr) 以字符串形式返回字符串的第一个字符; 若为空字符串, 则返回 NULL\$。
0516C	CDR\$	(\$ → \$') 返回去掉第一个字符后的字符串; 若为空字符串, 则返回 NULL\$。
378FA	POS\$	(\$search \$find #start → #pos) 从位置 #start 开始, 在 \$search 中查找 \$find。返回 \$find 的位置; 若未找到, 则返回 0。与 POSCHR 是同一词条。

地址	名称	说明
378FA	POCHR	(\$search chr #start → #pos) 与 POS\$ 是同一词条。
37906	POS\$REV	(\$search \$find #limit → #pos) 从 #limit 向后搜索至 #1。与 POSCHRREV 是同一词条。
37906	POSCHRREV	(\$search chr #start → #pos) 与 POS\$REV 是同一词条。
地址	名称	说明
25EA0	COERCE\$22	(\$ → \$') 如果字符串超过 22 个字符，则将其截断为 21 个字符，并在末尾添加“...”。
2F16D	Blank\$	(#len → \$) 创建由指定数量空格组成的字符串。
2EEF0	PromptIdUtil	(id ob → \$) 创建格式为“id: ob”的字符串。
25EF8	SEP\$NL	(\$ → \$' \$') 在第一个换行符处拆分字符串。\$' 是第一个换行符之前的子字符串，\$' 是其后的子字符串。
09A003	(`WRAP\$)	(\$ #width → \$') 用 NEWLINE 替换 SPACE 字符，使文本适合给定的 #width。供 ViewStrObject 使用。速度很快 (bang 型)。
05733	SUB\$	(\$ #start #end → \$') 返回指定位置之间的子字符串。
3628E	#1-SUB\$	(\$ #start #end+#1 → \$') 先执行 #1-，再执行 SUB\$。
362A2	1_#1-SUB\$	(\$ #end → \$') 返回从第一个字符到指定位置前一个字符的子字符串。 又名：1_#1-SUB
362B6	LAST\$	(\$ #start → \$') 返回从指定起始位置到字符串末尾的子字符串（包含起始位置）。
362CA	#1+LAST\$	(\$ #start-#1 → \$') 返回从指定起始位置到字符串末尾的子字符串（不包含起始位置）。
35DA8	SUB\$SWAP	(ob \$ # #' → \$' ob) 先执行 SUB\$，再执行 SWAP。
2A5CA	SUB\$1#	(\$ # → #') 返回指定位置字符的 ASCII 码所对应的 bint。
地址	名称	说明
34C82	EXPAND	(hxs #nibs → hxs') 在字符串末尾添加空字符。由于 #nibs 表示半字节数，因此应使用所需空字符数两倍的数值。
05193	&\$	(\$ \$' → \$+\$') 连接两个字符串。
36FF6	&\$SWAP	(ob \$ \$' → \$+\$' ob) 先执行 &\$，再执行 SWAP。
353CD	!append\$	(\$ \$' → \$+\$') 先尝试 &\$；若内存不足，则执行!!append\$?。
3533C	!insert\$	(\$ \$' → \$'+\$) 先执行 SWAP，再执行!append\$。
35F6A	!append\$SWAP	(ob \$ \$' → \$+\$' ob) 先执行!append\$，再执行 SWAP。
35369	!!append\$?	(\$ \$' → \$+\$') 如果目标位于临时对象内存中，则尝试“原地”追加。
353F7	!!append\$	(\$ \$' → \$+\$') 尝试“原地”追加。
353EB	!!insert\$	(\$ \$' → \$'+\$) 尝试“原地”插入。
0525B	>H\$	(\$ chr → \$') 在字符串开头添加字符。
052EE	>T\$	(\$ chr → \$') 在字符串末尾添加字符。
35BD7	APPEND_SPACE	(\$ → \$') 在字符串末尾添加空格。
35346	SWAP&\$	(\$ \$' → \$'+\$) 连接两个字符串。
2EED3	TIMESTR	(%dt %tm → "dy dt tm") 按当前格式返回时间的字符串表示。例如： "WED 06/24/98 10:00:45A"
25E7C	AND\$	(\$1 \$2 → \$') 逻辑 AND。若两个字符串长度不同，则生成错误。
地址	名称	说明
25EF0	OR\$	(\$ \$' → \$') 逻辑 OR。若两个字符串长度不同，则生成错误。
25F0D	XOR\$	(\$ \$' → \$') 逻辑 XOR。若两个字符串长度不同，则生成错误。
2F1A7	CHARSEEDIT	(→) HP49 字符浏览器。这是一个交互式应用，可将其中的字符回显到命令行。

6.1.6 解析字符串

地址	名称	说明
25EB7	DOSTR>	(\$ → ?) STR→ 的内部版本。
2EF62	palparse	(\$ → ob T) (\$ → \$ #pos \$' F) 尝试把字符串解析为对象。成功时返回对象和 TRUE；否则返回错误位置、字符串中有问题的部分 \$' 和 FALSE。如果字符串含有多个参数，所得对象将是包含这些对象的二级程序。
25E68	!*trior	(F → <SKIP>) (T T → <COLA>)
25E67	!*triand	(T T →) (F T → F T <SEMI>)
26206	tok8cktrior	(\$1 \$1 → :: \$1 <Ob1> ;) (\$1 \$2 → :: \$1 <Ob2> <Rest> ;)
261BB	tok8trior	(GNT data \$1 \$1 → :: GNT data GetNextToken ;) (GNT data \$1 \$2 → :: \$1 <Ob1> <Rest> ;)
29E67	nultrior	(NULL\$ → :: ;) (\$ → :: \$ <Ob1> <Rest> ;)
25EDB	GetNextToken	(hxs-mask \$ #start → hxs-mask \$ #next \$token)
地址	名称	说明
2F33C	getmatchtok	(hxs-mask \$ #loc \$_tok → hxs-mask \$ #next \$match)
2EF6E	ParseFail	(ob \$parsed #pos \$' →) 调用 DispBadToken 重新编辑已解析的字符串，并显示 “Syntax Error”。
2EF6F	DispBadToken	(ob \$parsed #pos \$' →) 重新编辑已解析的字符串，并将光标定位到错误位置。供 ParseFail 使用。

6.1.7 反编译

地址	名称	说明
2F191	!DcompWidth	(# →) 设置反编译所得字符串的宽度（字符数）。该宽度用于截断生成的字符串（供堆栈显示）或将其拆分为多行（主要供编辑）。请注意，大多数反编译入口都会把此值重置为堆栈宽度或编辑器宽度。若要确保采用当前宽度且不被改写，请使用 stkdecomp\$w 和 editdecomp\$w。
2F190	DcompWidth@	(→ #) 提取反编译所得字符串的宽度（字符数）。
26459	setStdWid	(→) 根据系统标志 72（堆栈小字体），将 DcompWidth 设置为堆栈显示的标准值 19 或 30 个字符。
2645E	setStdEditWid	(→) 根据系统标志 73（编辑小字体），将 DcompWidth 设置为编辑宽度 21 或 32 个字符。
25F13	stkdecomp\$w	(ob → \$) 使用当前 DcompWidth 反编译对象以供堆栈显示；若字符串过长，则将其截断。
地址	名称	说明
25E6D	1stkdecomp\$w	(ob → \$) 调用 setStdWid，并反编译对象以供堆栈显示；必要时截断字符串。
2A842	Decomp1Line	(ob → \$) 与 1stkdecomp\$w 相同。
2A904	RPNDecomp1Line	(ob → \$) 与 Decomp1Line 相同，但执行期间强制采用 RPN 模式（清除系统标志 95）。
25E6F	>Review\$	(id → \$) 根据变量名及其内容生成字符串；内容用 Decomp1Line 反编译，供查看键显示。
2A8E4	DecompStd1Line32	(ob → \$) 将 DcompWidth 设为 32，并使用 stkdecomp\$w 反编译。
2A9C4	RPNDecompStd1Line32	(ob → \$) 与 DecompStd1Line32 相同，但执行期间强制采用 RPN 模式（清除系统标志 95）。
2A8C9	DecompStd1Line	(ob → \$) 调用 setStdWid 并反编译；若所得字符串过长，则将其截断。
2A9A4	RPNDecompStd1Line	(ob → \$) 与 DecompStd1Line 相同，但执行期间强制采用 RPN 模式（清除系统标志 95）。
2A893	Decomp#Disp	(ob # → \$) 调用 setStdWid，反编译 ob（仅处理用户 RPL 组成部分），将字符串拆分为多行，并返回前 #+1 行。用于在堆栈第 1 层进行多行显示。
2A964	RPNDecomp#Disp	(ob # → \$) 与 Decomp#Disp 相同，但执行期间强制采用 RPN 模式（清除系统标志 95）。
地址	名称	说明
2A878	Decomp#Line	(ob # → \$) 与 Decomp#Disp 类似，但返回的字符串是待显示各行的内部表示。用于在堆栈第 1 层进行多行显示。
2A944	RPNDecomp#Line	(ob # → \$) 与 Decomp#Line 相同，但执行期间强制采用 RPN 模式（清除系统标志 95）。

地址	名称	说明
25F11	editdecomp\$w	(ob → \$) 反编译整个对象以供编辑，但只反编译其中的用户 RPL 组成部分。一些系统 RPL 入口（如 TakeOver）会被直接跳过，另一些则写成“External”。使用当前 DcompWidth 将所得字符串拆分为多行。
25ECE	EDITDECOMP\$	(ob → \$) 调用 setStdEditWid，然后像 editdecomp\$w 一样反编译以供编辑。
2A85D	DecompEdit	(ob → \$) 与 EDITDECOMP\$ 相同。
2A924	RPNDecompEdit	(ob → \$) 与 DecompEdit 相同，但执行期间强制采用 RPN 模式（清除系统标志 95）。
2AA43	AlgDecomp	(ob → \$) 调用 DecompEdit，并在其前后执行若干检查。
25EAA	DECOMP\$	(ob → \$) 调用 setStdwid，并反编译整个对象（仅处理用户 RPL 组成部分）。以 DcompWidth 为宽度将字符串拆分为多行。
39CB3	(ob&\$)	(ob \$ → "ob\$") 对 ob 应用 DECOMP\$，并将结果与字符串连接。
39C9F	(\$&ob)	(\$ ob → "\$ob") 对 ob 应用 DECOMP\$，并将结果与字符串连接。
地址	名称	说明
25EB1	DO>STR	(\$ → \$) (ob → \$) →STR 的内部版本。
1A7006	^DO>STRID	(id/ob → \$) 与 DO>STR 类似，但对 id 不加引号。
2A8AE	DecompEcho	(ob → \$) 调用 setStdEditWid，并将整个对象（仅用户 RPL）反编译为单行。
2A984	RPNDecompEcho	(ob → \$) 与 DecompEcho 相同，但执行期间强制采用 RPN 模式（清除系统标志 95）。
2F1BF	Decomp%Short	(% #width → \$) 将实数反编译为宽度为 #width 的字符串。它会按需舍去较低有效位或补零；必要时也会超过 #width。例如，“-1.e-33”至少需要 7 个字符，因此即使 #width 更小，也会使用 7 个字符。%0 始终反编译为“0”。
001004	^FSTR1	(ob → \$) stkdecomp\$w、1stkdecomp\$w、Decomp1Line 和 DecompStd1Line32 使用的反编译器。调用前必须先设置 DcompWidth。
003004	^FSTR3	(ob # → \$) Decomp#Line 使用的反编译器。调用前必须先设置 DcompWidth。
004004	^FSTR4	(ob → \$) editdecomp\$w、DecompEdit 和 EDITDECOMP\$ 使用的反编译器。调用前必须先设置 DcompWidth。
005004	^FSTR5	(ob → \$) DecompEcho 使用的反编译器。调用前必须先设置 DcompWidth。
地址	名称	说明
006004	^FSTR6	(ob # → \$) Decomp#Line 使用的反编译器。调用前必须先设置 DcompWidth。
007004	^FSTR7	(ob → \$) DO>STR 使用的反编译器。调用前必须先设置 DcompWidth。
009004	^FSTR9	(ob → \$) DecompStd1Line 使用的反编译器。调用前必须先设置 DcompWidth。
00D004	^FSTR13	(ob → \$) DECOMP\$ 使用的反编译器。调用前必须先设置 DcompWidth。
35B82	palrompdcmp	(romptr → \$ T) 为用户 RPL 堆栈反编译 ROM 指针。若该 ROM 指针有名称，则返回其名称；否则返回“XLIB n m”。

6.1.8 字符串测试

地址	名称	说明
0556F	NULL\$?	(ob → flag)
36252	DUPNULL\$?	(ob → ob flag)
2F321	CkChr00	(\$ → \$ flag) 如果字符串含有任何空字符，则返回 FALSE。

第 7 章

十六进制字符串

十六进制字符串就是本手册中所谓的二进制整数（Binary Integers，简称 **bint**）；它们可以按多种进制表示。在系统 RPL 中，这类对象称为十六进制字符串。它们使用结构 HXS <len> <hexbody> 创建。len 是字符串长度（半字节数或十六进制位数），并以十六进制形式表示；hexbody 则是字符串的实际内容。由于 HP 的内部体系结构，必须按逆序输入内容，这是较容易出错之处。例如，要得到十六进制字符串 #12AD7h，必须输入 HXS 5 7DA21；要得到 #12345678h，则输入 HXS 8 87654321。在系统 RPL 中，十六进制字符串可以具有任意长度；而在用户 RPL 中，其长度限制为 16 个半字节，即 64 位。

使用命令 HXS># 和 #>HXS，可在十六进制字符串与 bint 之间相互转换。使用 #>%（或 HXS>%）和 %>#，可在 HXS 与实数之间相互转换。

下面列出更多与十六进制字符串有关的命令。

7.1 参考

7.1.1 转换

地址	名称	说明
059CC	#>HXS	(# → hxs) 长度固定为 5。
2EFCB	%>#	(% → #) 将实数转换为 hxs。此命令本应命名为 %>HXS。

7.1.2 通用函数

地址	名称	说明
2EFBE	WORDSIZE	(→ #) 以 bint 返回当前字长。
2EFAA	dostws	(# →) 设置当前字长。
055D5	NULLHXS	HXS 0 将空 hxs 压入堆栈。
0518A	&HXS	(hxs hxs' → hxs'') 将 hxs' 追加到 hxs，返回 hxs''。
34C82	EXPAND	(hxs #nibs → hxs') 向 hxs 末尾追加 #nibs 个零半字节。
05616	LENHXS	(hxs → #nibs) 以半字节为单位返回长度。
05815	SUBHXS	(hxs #m #n → hxs') 返回 hxs 的子字符串。
2EFB9	bit+	(hxs hxs' → hxs'') 将两个 hxs 相加。
2EFC8	bit%#+	(% hxs → hxs') 将实数与 hxs 相加，返回 hxs。
2EFC9	bit%#+	(hxs % → hxs') 将 hxs 与实数相加，返回 hxs。
2EFBA	bit-	(hxs hxs' → hxs'') 从第一个 hxs 中减去第二个 hxs。
2EFC6	bit%#-	(% hxs → hxs') 从实数中减去 hxs，返回 hxs。
2EFC7	bit%#-	(hxs % → hxs') 从 hxs 中减去实数，返回 hxs。
2EFBC	bit*	(hxs hxs' → hxs'') 将两个 hxs 相乘。
2EFC4	bit%#*	(% hxs → hxs') 将实数与 hxs 相乘，返回 hxs。
2EFC5	bit%#*	(hxs % → hxs') 将 hxs 与实数相乘，返回 hxs。
2EFBD	bit/	(hxs hxs' → hxs'') 用第二个 hxs 除第一个 hxs。
2EFC2	bit%#/	(% hxs → hxs') 用 hxs 除实数，返回 hxs。
2EFC3	bit%#/	(hxs % → hxs') 用实数除 hxs，返回 hxs。
2EFAC	bitAND	(hxs hxs' → hxs'') 按位 AND。
2EFAD	bitOR	(hxs hxs' → hxs'') 按位 OR。
2EFAE	bitXOR	(hxs hxs' → hxs'') 按位 XOR。
2EFAF	bitNOT	(hxs → hxs') 按位 NOT。
2EFB8	bitASR	(hxs → hxs') 算术右移一位。最高有效位（符号位）保持不变。

地址	名称	说明
2EFB6	bitRL	(hxs → hxs') 循环左移一位。
2EFB7	bitRLB	(hxs → hxs') 循环左移一个字节。
2EFB4	bitRR	(hxs → hxs') 循环右移一位。
2EFB5	bitRRB	(hxs → hxs') 循环右移一个字节。
2EFB0	bitSL	(hxs → hxs') 左移一位。
2EFB1	bitSLB	(hxs → hxs') 左移一个字节。
2EFB2	bitSR	(hxs → hxs') 右移一位。
2EFB3	bitSRB	(hxs → hxs') 右移一个字节。

7.1.3 测试

地址	名称	说明
2EFCC	HXS==HXS	(hxs hxs' → %flag) == 测试。
2F0EE	HXS#HXS	(hxs hxs' → %flag) ≠ 测试。
地址	名称	说明
2EFCE	HXS<HXS	(hxs hxs' → %flag) < 测试。
2EFCD	HXS>HXS	(hxs hxs' → %flag) > 测试。
2EFCE	HXS>=HXS	(hxs hxs' → %flag) ≥ 测试。
2F0EF	HXS<=HXS	(hxs hxs' → %flag) ≤ 测试。

第 8 章 标识符

标识符用于表示存储在内存中的对象名称（即变量）。对用户而言，标识符在堆栈中显示在一对单引号之间，即"。在系统 RPL 中，标识符用结构 ID <name> 创建。使用该结构时，堆栈中不一定会得到标识符，因为它总会被求值。例如，若变量 anumber 中存有 123.45，而程序中的某处写有 ID anumber，该标识符会被求值并提取变量内容，因此堆栈中将得到 123.45。若要把 id 本身放入堆栈，请使用 ' ID <name>。如第 19 章所述，命令' 会把其后的对象放入堆栈，这称为引用。不过，如果不存在名为 <name> 的变量，ID <name>（不带'）也会把该 id 放入堆栈。这与您在 HP49 命令行中不加引号输入变量名时的行为类似。

您可以用 \$>ID 将字符串转换为 id；反向转换由 ID>\$ 完成。

此外还有另一类标识符：临时标识符，简称 lam。它们用于创建局部变量，第 18 章将进一步介绍。临时标识符用 LAM <name> 创建，其工作方式与普通 id 基本相同。

由于 id 与内存访问密切相关，处理 id 的函数列在第 24 章。

第 9 章 标记对象

若要在程序中插入**标记对象**，请使用结构 TAG <tag> <object>。**tag** 是不带引号的字符串，**object** 可以是任意对象。例如，要创建 0: 1790，可使用 TAG 0 % 1790。一个对象可以带有多个标记，但这种做法通常没有太大用处。

命令字 >TAG 根据堆栈第 2 层中的对象和堆栈第 1 层中表示标记的字符串创建**标记对象**。%>TAG 的工作方式相同，但用实数为对象添加标记。ID>TAG 则用标识符为对象添加标记。若要移除对象的全部标记，请调用 STRIPTAGS。下表还列出了若干与**标记对象**有关的命令。

请注意，程序员通常无须操心**标记对象**，因为类型分派机制（见第 29.2 节）可以自动去除传给程序的参数上的标记。

9.1 参考

地址	名称	说明
05E81	>TAG	(ob \$tag → tagged) 为对象添加标记。
2F266	USER\$>TAG	(ob \$tag → tagged) 字符串最多可含 255 个字符。
2F223	%>TAG	(ob % → tagged) 按当前显示模式把实数转换为字符串，并用该字符串为对象添加标记。
05F2E	ID>TAG	(ob id/lam → tagged) 用标识符或 lam 为对象添加标记。
37B04	TAGOBS	(ob \$tag → tagged) (ob.. { \$.. } → tagged...) 为一个或多个对象添加标记。
地址	名称	说明
37ABE	STRIPTAGS	(tagged → ob) 去除对象的全部标记。
37AEB	STRIPTAGS12	(tagged ob' → ob ob') 去除堆栈第 2 层中对象的全部标记。

第 10 章

数组

HP49G中实际上有两组用于表示数组的对象。第一组自 HP48 起就已存在，本章将介绍这一组：普通数组（对用户而言只能包含实数或复数）以及用户无法访问的链接数组。HP49 又引入了一种表示数组的新对象，即符号矩阵。由于符号矩阵实际上属于 HP49 CAS，因此将在第 43 章介绍。

在用户 RPL 中，数组只能由实数或复数组成。在系统 RPL 中，数组可以包含任意对象，甚至可以包含数组。但请注意，即使数组看起来像复合对象，它也不是复合对象（见第 11 章）。此外，一个数组只能包含一种对象类型。在 MASD 中，数组按如下方式输入：

```
1 ARRAY [[ % 1. % 2. %3. ]
          [ % 4. % 5. %6. ]]
```

这与在普通 HP49 命令行中输入数组没有太大区别。

您也可以创建由普通（非扩展）实数或复数组成的数组：先按顺序把这些数放入堆栈，再在堆栈第 1 层放入一个表示数组维数的列表；该列表中的维数必须是实数，而不是 bint。然后执行 ^XEQ>ARRAY。该函数会进行错误检查，确保参数数量足够且类型受支持。

函数 ^ARSize 返回数组的元素数。^DIMLIMITS 可取得数组维数，它返回一个由表示各维大小的 bint 组成的列表。若要取得数组中的一个元素，请把元素编号放在堆栈第 2 层，把数组放在堆栈第 1 层，然后执行 GETATELN。若找到该元素，将返回元素和 TRUE；若元素不存在，则只返回 FALSE。下文还列出了更多数组函数。

此外还有链接数组。链接数组与普通数组相似，不同之处在于它带有一张指针表，表中保存着指向数组各对象的指针。

这种结构可以加快数组元素的访问速度：访问链接数组中的对象时，只需读取表中指向该对象的指针并直接跳转到相应位置；而普通数组则需要顺序搜索。

本节列出的入口都针对普通数组（尽管其中一些也适用于 CAS 的符号矩阵）。符号矩阵专用入口见第 43 章。

10.1 参考

10.1.1 常规函数

地址	名称	说明
0371D	GETATELN	(# [] → ob T) (# [] → F) 从数组中取得一个元素。
16D006	^MDIMS	([[]] → #rows #cols T) ([] → #elem F) 返回数组大小。等同于 HP48 命令 MDIMS。
35FD8	MDIMSDROP	([2D] → #m #n) 先执行 MDIMS，再执行 DROP。
16E006	^DIMLIMITS	([] → { # }) ([[]] → { # # }) 与用户 RPL 命令 SIZE 类似，返回数组大小，但各维长度为 bint 而不是实数。等同于 HP48 命令 DIMLIMITS。
35E006	^ARSize	([] → #) 返回数组中的最大 # 值（即元素数）。
36183	OVERARSize	([] ob → [] ob #elts) 先执行 OVER，再执行 ARSize。
260F8	PULLREALEL	([%] # → [%] %) 取得实数元素。
260F3	PULLCMPEL	([C%] # → [C%] C%) 取得复数元素。
地址	名称	说明
26102	PUTEL	([%] % # → [%]') ([C%] C% # → [C%]') 将元素写入指定位置。写入前转换为“short”格式。警告：事先未复制到 tempob。
26107	PUTREALEL	([%] % # → [%]') 将实数元素写入指定位置。警告：事先未复制到 tempob。
260FD	PUTCMPEL	([C%] C% # → [C%]') 将复数元素写入指定位置。警告：事先未复制到 tempob。
33B006	^MATTRAN	(M → M') 矩阵转置。
331006	^Yext	(v2 v1 → ob) 计算符号向量的标量积，不作检查。

10.1.2 转换

地址	名称	说明
169006	<code>^BESTMATRIXTYPE</code>	<code>(ob → ob) </code> 把元素为实数或复数的符号矩阵转换为数值数组。
172006	<code>^CKNUMARRY</code>	<code>(ob → ob) </code> 检验 <code>ob</code> 是否为数值数组，并尝试把符号数组转换为数值数组。
178006	<code>^MATRIX2ARRAY</code>	<code>([] → [])</code>
001007	<code>^ListToArray</code>	<code>([[]] → [[]]) </code> 尝试把符号矩阵转换为数值数组。 <code>({} / {} → [] / [[]] TRUE)</code> <code>({} / {} → FALSE) </code> 若可行，把列表或嵌套列表转换为普通数组并返回 <code>TRUE</code> ；否则返回 <code>FALSE</code> 。
17F006	<code>^XEQ>ARRAY</code>	<code>(obl...obn { %n } → [])</code> <code>(obl1...obmn { %m %n } → [[mxn]]) </code> 仿照 <code>→ARRAY</code> 构造矩阵。
17C006	<code>^XEQARRAY></code>	<code>([] → obl...obn meta-array) </code> 仿照 <code>→ARRAY</code> 展开矩阵。
地址	名称	说明
002007	<code>^ArryToMatrix</code>	<code>([] → M) </code> 把数组转换为符号数组。

10.1.3 统计

地址	名称	说明
2EEDA	<code>STATCLST</code>	<code>(→) </code> 清空 <code>ΣDAT</code> 。
2EEDB	<code>STATSADD%</code>	<code>(% →) </code> 内部 <code>Σ+</code> 。
2EEDC	<code>STATN</code>	<code>(→ N) </code> 内部 <code>NΣ</code> 。
2EEDF	<code>STATSMIN</code>	<code>(→ %) </code> 内部 <code>MINΣ</code> 。
2EEDD	<code>STATSMAX</code>	<code>(→ %) </code> 内部 <code>MAXΣ</code> 。
2EED E	<code>STATMEAN</code>	<code>(→ %)</code> <code>(→ []) </code> 内部 <code>MEAN</code> 。
2EEEE0	<code>STATSTDEV</code>	<code>(→ %)</code> <code>(→ []) </code> 内部 <code>SDEV</code> 。
2EEEE1	<code>STATTOT</code>	<code>(→ %)</code> <code>(→ []) </code> 内部 <code>TOT</code> 。
2EEEE2	<code>STATVAR</code>	<code>(→ %)</code> <code>(→ []) </code> 内部 <code>VAR</code> 。

第 11 章 复合对象

复合对象内部包含其他对象。与数组不同，同一个复合对象可以包含不同类型的对象。在引言中，我们已经接触过复合对象，当时使用二级程序把若干命令组合成一个对象。

所有复合对象的结构都相似：开头是一个随对象种类而异的命令字，结尾是命令字 SEMI。

除二级程序外，复合对象还包括列表、符号对象（见第 14 章）和单位对象（见第 13 章）。

创建列表时，以 { 开始、以 } 结束，并可在其中放入任意数量、任意类型的对象。二级程序以 :: 和 ; 作为定界符。

若要连接两个复合对象，请把它们放入堆栈并执行 &COMP。若只想向复合对象的头部（开头）或尾部（末尾）添加一个对象，请先把复合对象放入堆栈，再放入该对象，然后分别执行 >HCOMP 或 >TCOMP。若要取得复合对象的长度（即对象数，以 bint 表示），只需把复合对象放在堆栈第 1 层并执行 LENCOMP。若要把复合对象展开为它的全部对象和一个计数（类似用户 RPL 命令 OBJ→），请使用 INNERCOMP；唯一的区别是，对象数以 bint 返回。若要取得复合对象中的一个对象，请把复合对象放在堆栈第 2 层，把对象位置（自然也是 bint）放在堆栈第 1 层，然后执行 NTHELCOMP。位置超出范围时只会得到 FALSE；否则会得到该对象和 TRUE。NTHCOMPDROP 等于上述入口后接 DROP。若要取得复合对象的一部分，请使用 SUBCOMP：堆栈第 3 层放复合对象，第 2 层放起始位置，第 1 层放结束位置；除非另有说明，从此处起所有数值参数均为 bint。返回结果是一个同类型复合对象，包含从起始位置到结束位置的所有元素（含两端）。该函数会检查索引是否越界；越界时返回空复合对象。如果结束位置小于起始位置，也会返回空复合对象。

其他命令列在下面的参考一节中。

11.1 参考

11.1.1 常规操作

地址	名称	说明
0521F	&COMP	(comp comp' → comp') 连接两个复合对象。
052FA	>TCOMP	(comp ob → comp+ob) 将 ob 添加到复合对象的尾部（末尾）。
052C6	>HCOMP	(comp ob → ob+comp) 将 ob 添加到复合对象的头部（开头）。
39C8B	(SWAP>HCOMP)	(ob comp → ob+comp) 先执行 SWAP，再执行 >HCOMP。
05089	CARCOMP	(comp → ob_head) (comp_null → comp_null) 返回复合对象的第一个对象；若参数为空复合对象，则返回空复合对象。
361C6	?CARCOMP	(comp T → ob) (comp F → comp) 若标志为 TRUE，则执行 CARCOMP。
05153	CDRCOMP	(comp → comp-ob_head) (comp_null → comp_null) 返回去掉第一个对象后的复合对象；若参数为空复合对象，则返回空复合对象。
2825E	(2NELCOMPDROP)	(comp → ob2) 取得复合对象的第二个元素。
2BC006	^LASTCOMP	(comp → ob) 取得复合对象的最后一个元素。先执行 DUPLCOMP，再执行 NTHCOMPDROP。
0567B	LENCOMP	(comp → #n) 返回复合对象的长度（对象数）。
3627A	DUPLCOMP	(comp → comp #n) 先执行 DUP，再执行 LENCOMP。
055B7	NULLCOMP?	(comp → flag) 若复合对象为空，则返回 TRUE。
36266	DUPNULLCOMP?	(comp → comp flag) 先执行 DUP，再执行 NULLCOMP?。
地址	名称	说明
056B6	NTHELCOMP	(comp #i → ob T) (comp #i → F) 返回复合对象中的指定元素和 TRUE；若找不到，则只返回 FALSE。
35BC3	NTHCOMPDROP	(comp #i → ob) 先执行 NTHELCOMP，再执行 DROP。
35D58	NTHCOMDDUP	(comp #i → ob ob) 先执行 NTHCOMPDROP，再执行 DUP。

地址	名称	说明
376EE	POSCOMP	(comp ob pred → #i) (comp ob pred → #0) (例如: pred = ' %<) 对复合对象的每个元素与 ob 执行 pred, 并返回第一个使 pred 返回 TRUE 的对象索引。若没有元素使其返回 TRUE, 则返回 #0。以下程序返回 #4: :: { %1 %2 %3 %4 %5 %6 %7 } %0 ' %< POSCOMP ;
3776B	EQUALPOSCOMP	(comp ob → #pos) (comp ob → #0) 以 EQUAL 为测试的 POSCOMP。
37784	NTHOF	(ob comp → #i) (ob comp → #0) 先执行 SWAP, 再执行 EQUALPOSCOMP。
0FD006	^ListPos	(ob { } → #i / #0) 等同于 NTHOF, 但速度更快; 不过它只适用于列表。
37752	#=POSCOMP	(comp # → #i) (comp # → #0) 以 #= 为测试的 POSCOMP。
05821	SUBCOMP	(comp #m #n → comp') 返回子复合对象, 并事先完成全部索引检查。
376B7	matchob?	(ob comp → T) (ob comp → ob F) 若 ob 与复合对象中的任一元素 EQUAL, 则返回 TRUE。
地址	名称	说明
371B3	Embedded?	(ob1 ob2 → flag) 若 ob2 嵌入在 ob1 中, 或 ob2 与 ob1 是同一对象, 则返回 TRUE; 否则返回 FALSE。
37798	Find1stTrue	(comp test → ob T) (comp test → F) 用 test 检验每个元素。第一个使 test 返回 TRUE 的元素会与 TRUE 一同压入堆栈。若没有任何对象使 test 返回 TRUE, 则把 FALSE 压入堆栈。例如, 以下程序返回 %4 和 TRUE: :: { %1 %2 %2 %4 %5 %6 } ' %0< Find1stTrue ;
377C5	Lookup	(ob test comp → nextob T) (ob test comp → ob F) 用 test 检验复合对象中每个奇数位置的元素 (第 1、第 3、……个)。若某次 test 返回 TRUE, 则返回被检验元素之后的对象以及 TRUE。若没有对象使 test 返回 TRUE, 则返回 FALSE。例如, 以下程序返回 %6 和 TRUE: :: %0' %< { %1 %2 %3 %4 %5 %6 } Lookup ;
377DE	Lookup.1	(ob test → nextob T) (ob test → ob F) 返回堆栈: (comp →) 复合对象已通过 >R 预先压入运行流时使用的 Lookup。由 Lookup 调用。
37829	EQLookup	(ob comp → nextob T) (ob comp → ob F) 以 EQ 为测试的 Lookup。
地址	名称	说明
37B54	NEXTCOMPOB	(comp #ofs → comp #ofs' ob T) (comp #ofs → comp F) 返回从起点起指定半字节偏移处的对象。若该对象是 SEMI (即已到达复合对象末尾), 则返回 FALSE。要取得第一个元素, 请用 FIVE 作为偏移值, 以跳过前导码; ZERO 也可使用。

11.1.2 构造

此外还有一些快捷命令字, 可用指定数量的对象构造列表和二级程序, 见以下各节。

地址	名称	说明
05459	{ }N	(obn..obl #n → { obn..obl })
05445	::N	(obl..obn #n → :: obl..obn ;)
0546D	SYMBN	(obl..obn #n → symb) 构造符号对象。
05481	EXTN	(obl..obn #n → u) 构造单位对象。
293F8	P{ }N	(obl..obn #n → { }) 构造列表; 期间可能发生垃圾回收。

11.1.3 展开

地址	名称	说明
054AF	INNERCOMP	(comp → obn..ob1 #n)
3622A	DUPINCOMP	(comp → comp obn..ob1 #n)
3623E	SWAPINCOMP	(comp obj → obj obn..ob1 #n)
35BAF	INCOMPDROP	(comp → obn..ob1)
35C68	INNERDUP	(comp → obn..ob1 #n #n)
2F0EC	ICMPDRPRTRDP	(comp → obn...ob4 ob2 ob1) 先执行 INCOMPDROP, 再执行 ROTDROP。
3BADA	(INNERCOMP>%)	(comp → obn..ob1 %n)
366E9	INNER#1=	(comp → obn..ob1 flag)
地址	名称	说明
157006	^SYMBINCOMP	(symb → ob1 .. obN #n) (ob → ob #1) ({} → {} #1) 将符号对象展开为元对象。其他对象则通过把 #1 压入堆栈转换为单对象元对象。
12A006	^2SYMBINCOMP	(ob1 ob2 → meta1 meta2) 对两个对象分别执行 ^SYMBINCOMP。
158006	^CKINNERCOMP	({} → ob1 .. obN #n) (ob → ob #1) 将列表展开为元对象。其他对象则通过把 #1 压入堆栈转换为单对象元对象。

11.1.4 列表

地址	名称	说明
055E9	NULL{ }	(→ {}) 将空列表压入堆栈。
36ABD	DUPNULL{ }?	({} → { } flag)
159006	^DUPCKLEN{ }	({} → { } #n) (ob → ob #1) 返回列表长度; 若参数不是列表, 则返回 1。
29D18	ONE{ }N	(ob → { ob })
36202	TWO{ }N	(ob1 ob2 → { ob1 ob2 })
36216	THREE{ }N	(ob1 ob2 ob3 → { ob1 ob2 ob3 })
361EE	#1-{ }N	(ob1..obn #n+1 → {})
2B42A	PUTLIST	(ob #i { } → {}') 替换指定位置的对象。假定 #i 有效。
2FC006	^INSERT{ }N	({ } ob # → {}') 在给定位置向列表插入对象。该位置必须小于列表长度; 若位置为零, 则使用 >TCOMP。
2FB006	^NEXTPext	(list → list1 list2) 把 list 中第一个对象的所有出现项提取到 list2, 其余对象保存在 list1 中。list1 = list-list2。
地址	名称	说明
2FD006	^COMPRIMext	({} → {}') 删除列表中的重复项。
15A006	^CKCARCOMP	({} → ob1) (ob → ob) 若参数为列表, 则返回其第一个元素; 否则返回对象本身。
2EF5A	apndvarlst	({ } ob → {}') 若 ob 尚不在列表中, 则将其追加到列表。
0FE006	^AppendList	({ } ob → {}') 等同于 apndvarlst, 但速度更快。
4EB006	^prepvarlist	({ } ob → {}') 若 ob 不在列表中, 则将其添加到列表开头; 若 ob 已在列表中, 则将其移到列表开头。
100006	^SortList	(L pred → L') 按给定谓词对列表排序。pred 是检验两个元素的程序; 如果第一个元素应排在第二个元素之前, 它返回 FALSE。例如, 按数值顺序排序时, 可用 > 测试作为谓词。
28A006	^PIext	({} → ob) 返回列表全部元素的乘积。
25ED3	EqList?	(ob →) ob 是否为方程列表? 若 ob 是至少包含两个元素的列表, 且第二个元素本身不是列表, 则返回 T。

11.1.5 二级程序

地址	名称	说明
055FD	NULL::	(→ :: ;) 返回空二级程序。
37073	Ob>Seco	(ob → :: ob ;) 先执行 ONE，再执行::N。
3705A	?Ob>Seco	(ob → :: ob ;) 若对象不是二级程序，则执行 Ob>Seco。
37087	2Ob>Seco	(ob1 ob2 → :: ob1 ob2 ;) 先执行 TWO，再执行::N。
地址	名称	说明
3631A	::NEVAL	(ob1..obn #n → ?) 先执行::N，再执行 EVAL。

第 12 章 元对象

元对象（简称 **meta**）由 **n** 个对象及其计数（以 **bint** 表示）组成。元对象可视为复合对象的另一种表示。命令字 **INNERCOMP** 会把任意复合对象展开为元对象；反向转换则根据所需复合对象的种类，由若干不同命令字完成。可用命令字列在[第 11.1.2 节](#)中。

请注意，单独一个零就是一个（空）元对象，即空元对象。

有若干堆栈操作可以把元对象当作单个对象处理。一般而言，这些堆栈操作的名称使用小写。不过，有些命令字的名称完全不能反映此用途，因为它们并非总用于元对象；命名时考虑的是它们的另一种用途。

此外还有用户元对象。它与元对象类似，但计数以实数而不是 **bint** 表示，不过并不常见。

12.1 参考

12.1.1 堆栈函数

地址	名称	说明
0326E	NDROP	(meta →) 本应命名为 drop。
37032	DROPNDROP	(meta ob →) 本应命名为 DROPdrop。
35FB0	#1+NDROP	(ob meta →) 本应命名为 dropDROP。 又名：N+1DROP
地址	名称	说明
28211	NDROPFALSE	(meta → F) 本应命名为 dropFALSE。
391006	^NDROPZERO	(obn..obl #n → #0) 将元对象替换为空元对象。 本应命名为 dropZERO。
29A5D	psh	(meta1 meta2 → meta2 meta1) 本应命名为 swap。
29A8F	roll2ND	(meta1 meta2 meta3 → meta2 meta3 meta1) 本应命名为 rot。
29B12	unroll2ND	(meta1 meta2 meta3 → meta3 meta1 meta2) 本应命名为 unrot。
3695A	SWAPUnNDROP	(meta1 meta2 → meta2) 本应命名为 swapdrop。
36FA6	metaROTDUP	(meta1 meta2 meta3 → meta2 meta3 meta1 meta1) 本应命名为 rotdup。

12.1.2 组合函数

地址	名称	说明
296A7	top&	(meta1 meta2 → meta1&meta2)
2973B	pshtop&	(meta1 meta2 → meta2&meta1)
36FBA	ROTUntop&	(meta1 meta2 meta3 → meta2 meta3&meta1)
36FCE	roll2top&	(meta1 meta2 meta3 → meta3 meta1&meta2) 又名：rolltwotop&
2963E	psh&	(meta1 meta2 meta3 → meta1&meta3 meta2)

12.1.3 元对象与对象操作

地址	名称	说明
3592B	SWAP#1+	(# ob → ob #+1) 又名：SWP1+
34431	DUP#1+PICK	(n..1 #n → n..1 #n n)
34504	get1	(ob meta → meta ob)
36147	OVER#2+UNROL	(meta ob → ob meta)
29693	psh1top&	(meta ob → ob&meta)
28071	pull	(meta&ob → meta ob) 又名：#1-SWAP
28085	pullrev	(ob&meta → meta ob)
29821	psh1&	(meta1 meta2 ob → ob&meta1 meta2)

地址	名称	说明
298C0	psh1&rev	(meta1 meta2 ob → ob&meta1 meta2)
2F193	UobROT	(ob meta1 meta2 → meta1 meta2 ob)
29754	pullpsh1&	(meta1 meta2&ob → ob&meta1 meta2)
406006	^addt0meta	(meta1&ob meta2 → meta1 meta2) 删除 meta1 的最后一个对象。
29972	pshzer	(meta → #0 meta)
36946	SWAPUnDROP	(ob meta → meta)
2F38E	xnsngeneral	(meta → LAM3&meta&LAM1) 使用 LAM1 和 LAM3 的内容。
2F38F	xnsngeneral	(meta → meta&LAM3&LAM1) 使用 LAM1 和 LAM3 的内容。

12.1.4 其他操作

地址	名称	说明
3760D	SubMetaOb	(meta #start #end → meta') 取得一个子元对象，并执行范围检查。
地址	名称	说明
37685	SubMetaOb1	(ob1..obi..obn #n #i #n #i → ob1..obi #n #i) 若随后执行 SWAPDROP，可用此函数取出元对象的前 i 个对象。示例： :: %1 %2 %3 %4 %5 BINT5 BINT3 BINT5 BINT3 SubMetaOb1 ; 结果为： %1 %2 %3 #5 #3
33F006	^submeta	(meta #begin #end → meta') 从元对象中提取一个子元对象。
2F356	metatail	(ob1..obn-i..obn #i #n+1 → ob1..ob..obn-i #n-i obn-i+1..obn #i) #n 是元对象中的对象数。该函数取出元对象末尾的 #i 个元素，并用它们创建一个新元对象。示例： :: %1 %2 %3 %4 %5 BINT2 BINT6 metatail ; 结果为： %1 %2 %3 #3 %4 %5 #2
385006	^metasplit	(meta #i → meta1 meta2) 在位置 i 将一个元对象拆分为两个元对象。meta1 含 #i 个元素，meta2 含 #n-i 个元素。
39F006	^metaEQUAL?	(meta2 meta1 → meta2 meta1 flag) 测试两个元对象是否相等。
3BF006	^EQUALPOSMETA	(Meta ob → Meta ob #pos) 返回 ob 在 Meta 中最后一次出现的位置。如果元对象的某个分量是列表或符号对象，还会搜索 ob 是否嵌套在该分量中。
3C0006	^EQUALPOS2META	(Meta2 Meta1 ob → Meta2 Meta1 ob #pos) 返回 ob 在 Meta1 或 Meta2 中最后一次出现的位置。如果位于 meta2，则 #pos >0；如果位于 meta1，则 #pos <0 (#pos=MINUSONE-#)。
198006	^METAINT?	(Meta → Meta flag) 测试 Meta 是否为整数。
地址	名称	说明
199006	^METAPOSINT?	(Meta → Meta flag) 测试 Meta 是否为小于 Zsmall 的正整数。

第 13 章 单位对象

单位是另一类复合对象。在程序中加入单位对象其实并不困难，只是比较繁琐。
单位对象以 UNIT 开始，以;结束。其内部使用若干命令来定义单位。理解单位表示方式的最佳方法，是对其进行反汇编。单位对象 9.8_m/s^2 可由以下代码创建：

```
1 ::
  UNIT
    % 9.8
    "m"
5  "s"
    %2
    um^
    um/
    umEND
10 ;
;
```

可以看到，单位是用逆波兰方式创建的，所用字包括 um^、um*、um/ 和 umP。前三者的含义很容易判断。最后一个用于创建单位前缀运算符，例如 kilo、mega、milli 等。先将前缀作为字符或字符串输入，再输入单位名称；所有这些操作都以字符或字符串形式接收单位名称。执行 umP 后即可创建带前缀的单位，随后按需调用其他函数。单位定义以 umEND 结束。它把最先输入的数值部分与单位部分连接起来。如果使用第 6 章列出的内置字符和字符串，上述代码还可以缩短。
由于单位属于复合对象，可以使用 INNERCOMP 等命令将单位拆解为元对象。也可以使用 EXTN 从元对象创建单位，参见第 12 章。下一页的程序会把单位 m/s 加到堆栈中的数值上：

```
1 ::
  CK1NOLASTWD
  CKREAL
  "m"
5  "s"
    um/
    umEND
    BINT5 EXTN
;
```

请注意，执行 um 系列字时，它们只会把自身压入堆栈。
可以对单位执行多种操作，完整列表如下。最重要的是含义直观的 UM+、UM-、UM*、UM/ 和 UFACT；工作方式与用户命令 CONVERT 相同的 UMCONV；等效于 UBASE 的 UMSI；以及返回单位数值部分的 U>nbr。

13.1 参考

13.1.1 创建单位

地址	名称	说明
	um*	* 标记符
2D74F	um/	/ 标记符
2D759	um^	^ 标记符
2D763	umP	字符前缀运算符
2D76D	umEND	单位结束运算符
2D777	EXTN	(ob1..obn #n → u) 创建一个单位对象。
05481		

13.1.2 通用函数

地址	名称	说明
2F099	U>NCQ	(u → n % cf % qhxs) 返回数值、换算到基本单位的换算因子，以及如下形式的向量： [kg m A s K cd mol r sr ?] 其中每个元素表示对应单位的指数。例如，1_N U>NCQ 将返回： %%1 %%1 [1 1 0 -2 0 0 0 0 0] 因为它等价于 1_kg*m/s^2。
2F07A	UM>U	(% u → u') 替换单位对象的数值部分。
2F08C	UMCONV	(u1 u2 → u1') 把 u1 的单位换算为 u2 的单位。
2F090	UMSI	(u → u') 等效于用户命令 UBASE。
2F095	UMU>	(u → % u') 返回单位的数值部分和规范化单位部分。
2F019	UNIT>\$	(u → \$) 把单位转换为字符串。
2F07B	U>nbr	(u → %) 返回单位的数值部分。
2F098	Unbr>U	(u % → u') 替换单位的数值部分。
2F09A	TempConv	??? 由 UMCONV 用于温度单位换算。
25EE4	KeepUnit	(% ob ob' → % ob) (% ob u → u' ob) 如果堆栈第 1 层的对象是单位对象，则用堆栈第 3 层的数值替换其数值部分；否则只执行 DROP。

13.1.3 算术函数

地址	名称	说明
2F081	UM+	(u u' → u'')
2F082	UM-	(u u' → u'')
2F080	UM*	(u u' → u'')
2F083	UM/	(u u' → u'')
2F07D	UM%	(u %percent → u')
2F07E	UM%CH	(u u' → %)
2F07F	UM%T	(u u' → %)
2F08F	UMMIN	(u u' → u?)
2F08E	UMMAX	(u u' → u?)
2F096	UMXROOT	(u u' → u'')
2F08A	UMABS	(u → u')
2F08B	UMCHS	(u → u')
2F092	UMSQ	(u → u')
2F093	UMSQRT	(u → u')
2D949	UMSIGN	(u → u')
2D95D	UMIP	(u → u')
2D971	UMFP	(u → u')
2D985	UMFLOOR	(u → u')
2D999	UMCEIL	(u → u')
2D9CB	UMRND	(u → u')
2D9EE	UMTRC	(u → u')
2F08D	UMCOS	(u → u')
2F091	UMSIN	(u → u')
2F094	UMTAN	(u → u')

13.1.4 测试

地址	名称	说明
2F087	UM=?	(u u' → %flag)
2F07C	UM#?	(u u' → %flag)
2F086	UM<?	(u u' → %flag)
2F089	UM>?	(u u' → %flag)
2F085	UM<=?	(u u' → %flag)
2F088	UM>=?	(u u' → %flag)
地址	名称	说明
2F076	puretemp?	([] []' → [] []' flag) 检查两个数组是否都表示 纯温度单位，即两个数组是否都等于： [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]

第 14 章 符号对象

符号对象，也称代数表达式，是另一类复合对象。其结构与单位对象非常相似。符号对象以 SYMBOL 开始，以;结束；其内部采用逆波兰方式构造表达式。

如果需要在程序中加入符号对象，对下列方程进行反汇编即可看出其写法：

$$R = \frac{V}{I}$$

```
1 ::  
  SYMBOL  
    ID R  
    ID V  
5   ID I  
    x/  
    x=  
;  
;
```

可以看到，变量由标识符表示，而函数则使用用户可访问的函数；在系统 RPL 中，这些函数名以小写 x 为前缀。要从元对象创建符号对象，请使用 SYMBN。加入函数时必须将其引用，即在命令前放置命令'，使该命令被压入堆栈而不是立即执行。第 19.2 节将更详细地说明对象引用。

在 HP49 上，新 CAS 包含了大多数与符号对象有关的入口。这些入口在本书的 CAS 部分中说明，主要见第 44 章和第 45 章。不过，为保持兼容，HP48 上原有的一些入口仍被保留，列于下文。

14.1 参考

14.1.1 通用操作

地址	名称	说明
0546D	SYMBN	(ob1..obn #n → sym)
2BD8C	(SYMBN:)	ob1..obn #n → symb 依次执行 'R、SWAP#1+ 和 SYMBN。它使用堆栈中的元对象和运行流中的下一个对象创建符号对象，并把该对象添加到符号对象末尾。
286E7	symcomp	(ob → ob') 如果 ob 已是符号对象，则不执行任何操作；否则执行 ONE SYMBN。
2F073	SWAPcompSWAP	(ob ob' → ob'' ob') 执行 SWAP symcomp SWAP。
28ACE	(DROP?symcomp)	(%/C%/Z/id/lam ob' → %/C%/Z/id/lam) (ob ob' → symb) 先丢弃 ob'。如果堆栈中的对象是实数、复数、zint、标识符或 lam，则不再执行其他操作；对于其他对象，调用 symcomp 创建只含一个对象的符号对象。
293A3	(?symcomp)	(%/C%/Z/id/lam #1 → %/C%/Z/id/lam) (ob #1 → symb) (ob # → symb) 如果 # 为 BINT1，则调用 DROP? symcomp；如果是其他数值，则调用 SYMBN。
25EA2	CRUNCH	(ob → %) 用户命令 → NUM 的内部版本。
2F110	(FINDVARS)	(sym → { }) 返回方程中的变量列表，并递归搜索方程所含的程序和函数。
462006	^EQUATION?	(ob → ob flag) 如果 ob 是以 x= 结束的符号对象，则返回 TRUE。
地址	名称	说明
463006	^USERFCN?	(ob → ob flag) 如果 ob 是以 xFCNAPPLY 结束的符号对象，则返回 TRUE。
29CB9	uncrunch	(→) 只为下一条命令清除数值结果标志（系统标志 3）。示例： SYMCOLCT = :: uncrunch colct ;

地址	名称	说明
2BCA2	cknumdsptchl	(sym → symf) 供单参数函数使用，根据数值结果标志选择符号例程或数值例程。用法： :: ccknumdsptchl <sym> <num> ; 在数值模式下，对堆栈第 1 层对象执行 CRUNCH，并对 <num> 执行 COLA。在符号模式下，调用 ckseval1:。 示例： :: ccknumdsptchl MetaRE xRE ; (sym sym → symf) 供双参数函数使用，根据当前数值模式对函数求值。用法：:: sscknum2 <sym> <num> ; 在数值模式下，对两个参数执行 CRUNCH，并对 <num> 执行 COLA；否则调用 cksseval2:。示例： SYM+ = :: sncknum2 Meta+ x+ ; (sym % → symf) 用法：:: sncknum2 <sym> <num> ; 在符号模式下使用 cksneval2:。示例： SYM+O = :: sncknum2 Meta+Con x+ ; (% sym → symf) 用法：:: nscknum2 <sym> <num> ; 在符号模式下使用 ckseval2:。示例： O+SYM = :: nscknum2 Con+Meta x+ ;
2BB21	sscknum2	
2BB3A	sncknum2	
2BB53	nscknum2	

14.1.2 其他函数

地址	名称	说明
2EF26	SYMSHOW	(sym id/lam → symf)
2F2A9	XEQSHOWLS	(sym {} → symf)

14.1.3 符号元对象函数

地址	名称	说明
29986	pshzerpsharg	(meta → M_last M_rest) 压入元对象中的最后一个子表达式。如果该元对象是有效表达式，M_rest 将为空。
3701E	pZpargSWAPUn	(meta → M_rest M_last) 先执行 pshzerpsharg，再执行 psh。
36FE2	plDRPpZparg	(meta&ob → M_last M_rest) 丢弃 ob，再调用 pshzerpsharg。
3F1006	^DIVMETAOBJ	(o1...on #n ob → {o1/ob...on/ob}) 将元对象的所有元素除以 ob，并测试 o=1。

第 15 章

图形对象 (Grobs)

图形对象简称 **grob**，用于表示图像、绘图等。如果要编写在屏幕上绘图的程序，就必须了解 **grob**，因为屏幕内容本身就是一个 **grob**；程序需要在该 **grob** 上绘制，或向其中插入另一个 **grob**。

下面的参考部分列出了创建、操作和显示图形对象所用的字。

处理图形时，请牢记以下两点：

1. 一些 **grob** 操作不会创建副本，而是直接修改原 **grob**。因此，堆栈中所有指向该对象的指针都会反映修改。可使用 **CKREF** 确保对象是唯一的。临时内存和引用计数详见第 24.1.4 节。此类操作称为“bang 型”，命令名通常带有感叹号，例如 **GROB!** 或 **GROB!ZERO**。这类操作也不执行错误检查，因此参数不正确或超出范围时可能破坏内存。
2. 将 **grob** 放入显示 **grob** 的最佳命令是 **XYGROBDISP**。该字会检查待放入 **HARDBUFF** 的 **grob** 是否超出边界；必要时，它会扩大 **HARDBUFF**，使 **grob** 能够完整容纳。

15.1 参考

15.1.1 内置 grob

地址	名称	说明
27AA3	(NULLGROB)	(→ grob) 0x0 空 grob。
27D3F	CROSSGROB	(→ grob) 5x5 十字光标 (“+”)。
27D5D	MARKGROB	(→ grob) 5x5 标记符号 (“x”)。
27D7B	(StdLabelGrob)	21x8 普通菜单键
2E25C	(InvLabelGrob)	21x8 反显菜单键
0860B0	~grobAlertIcon	9x9 警告 grob
0870B0	~grobCheckKey	21x8 勾选键菜单 grob。 由一个勾号和菜单 grob 中的“CHK”构成。

15.1.2 尺寸

地址	名称	说明
26085	GROBDIM	(grob → #height #width)
25EBB	DUPGROBDIM	(grob → grob #height #width)
36C68	GROBDIMw	(grob → #width)
2F324	CKGROBFITS	(g1 g2 #n #m → g1 g2' #n #m) 如果 g2 无法放入 g1，则缩小 g2。
2F320	CHECKHEIGHT	(grob #height →) 强制 grob (ABUFF/GBUFF) 至少具有 64 行。

15.1.3 grob 操作

地址	名称	说明
2607B	GROB!	(grob1 grob2 #x #y →) 把 grob1 存入 grob2。 bang 型。
2EFDB	(GROB+)	(grob1 grob2 → grob) 用逐位 OR 合并两个 grob；两个 grob 尺寸不同时报错。
地址	名称	说明
2F342	GROB+#	(flag grob1 grob2 #x #y → grob') 把 grob2 插入 grob1 的指定位置。flag 为 TRUE 时使用 OR，为 FALSE 时使用 XOR。执行前会完成所有必要检查。
26080	GROB!ZERO	(grob #x1 #y1 #x2 #y2 → grob') 清空 grob 的一个矩形区域。bang 型。
368E7	GROB!ZERODRP	(grob #x1 #y1 #x2 #y2 →) 清空 grob 的一个矩形区域。可能只适合用于文本 grob 或图形 grob，参见显示组织部分。bang 型。
2612F	SUBGROB	(grob #x1 #y1 #x2 #y2 → grob') 返回 grob 的指定部分。
25F0E	XYGROBDISP	(#x #y grob →) 以 (#x,#y) 为左上角，把 grob 存入 HARDBUFF；必要时扩大 HARDBUFF。
25ED8	GROB>GDISP	(grob →) 存储新的图形 grob。

地址	名称	说明
260B2	MAKEGROB	(#height #width → grob) 创建空白 grob。
2F0DB	MAKEPICT#	(#w #h →) 创建空白图形 grob。最小尺寸为 131x64；尺寸更小时会自动调整。
2609E	INVGROB	(grob → grob') 反转 grob 的数据位。bang 型。
260E4	PIXON	(#x #y →) 在文本 grob 中置位像素。
260DF	PIXOFF	(#x #y →) 在文本 grob 中清除像素。
260EE	PIXON?	(#x #y → flag) 文本 grob 中的该像素是否已置位？
260DA	PIXON3	(#x #y →) 在图形 grob 中置位像素。
260D5	PIXOFF3	(#x #y →) 在图形 grob 中清除像素。
地址	名称	说明
260E9	PIXON?3	(#x #y → flag) 图形 grob 中的该像素是否已置位？
280C1	ORDERXY#	(#x1 #y1 #x2 #y2 → #x1' #y1' #x2' #y2') 对二进制整数排序，使其适合定义 grob 中的矩形。如果 #x2<#x1，则交换 #x1 和 #x2；如果 #y2<#y1，则交换 #y1 和 #y2。
280F8	ORDERXY%	(%x1 %y1 %x2 %y2 → %x1' %y1' %x2' %y2') 实数版本的 ORDERXY#。
2EF9F	LINEON	(#x1 #y1 #x2 #y2 →) 在文本 grob 中绘制直线。
2EFA0	LINEOFF	(#x1 #y1 #x2 #y2 →) 清除文本 grob 中的一条直线。
2EFA1	TOGLINE	(#x1 #y1 #x2 #y2 →) 切换文本 grob 中的一条直线。
2EFA2	LINEON3	(#x1 #y1 #x2 #y2 →) 在图形 grob 中绘制直线。
2F13F	DRAWLINE#3	(#x1 #y1 #x2 #y2 →) 在图形 grob 中绘制直线，不要求 x1<x2。
2EFA3	LINEOFF3	(#x1 #y1 #x2 #y2 →) 清除图形 grob 中的一条直线。
2EFA4	TOGLINE3	(#x1 #y1 #x2 #y2 →) 切换图形 grob 中的一条直线。
2F382	TOGGLELINE#3	(#x1 #y1 #x2 #y2 →) 切换图形 grob 中的一条直线，不要求 x1<x2。
2F32C	DRAWBOX#	(#x1 #y1 #x2 #y2 →) 在图形 grob 中绘制矩形。
2EF03	DOLCD>	(→ grob) 返回当前显示内容。
2EF04	DO>LCD	(grob →) 把 grob 送往显示屏。
0BF007	^GROBADDext	(grob2 grob1 → grob) 纵向拼接 grob；grob2 位于 grob1 上方。

15.1.4 灰度图形

地址	名称	说明
25592	SubRepl	(grb1 grb2 #x1 #y1 #x2 #y2 #W #H → grb1') 以 REPLACE 模式，用 grb2 的一部分替换 grb1 的一部分。
25597	SubGor	(grb1 grb2 #x1 #y1 #x2 #y2 #W #H → grb1') 以 OR 模式，用 grb2 的一部分替换 grb1 的一部分。
2559C	SubGxor	(grb1 grb2 #x1 #y1 #x2 #y2 #W #H → grb1') 以 XOR 模式，用 grb2 的一部分替换 grb1 的一部分。
25565	LineW	(grb #x1 #y1 #x2 #y2 → grb') 绘制白线。
2556F	LineG1	(grb #x1 #y1 #x2 #y2 → grb') 绘制浅灰色线。
25574	LineG2	(grb #x1 #y1 #x2 #y2 → grb') 绘制深灰色线。
2556A	LineB	(grb #x1 #y1 #x2 #y2 → grb') 绘制黑线。
25579	LineXor	(grb #x1 #y1 #x2 #y2 → grb') 对一条线执行 XOR。
2F218	CircleW	(grb #Cx #Cy #r → grb') 绘制白色圆。
2F216	CircleG1	(grb #Cx #Cy #r → grb') 绘制浅灰色圆。
2F217	CircleG2	(grb #Cx #Cy #r → grb') 绘制深灰色圆。
2F215	CircleB	(grb #Cx #Cy #r → grb') 绘制黑色圆。

地址	名称	说明
2F219	CircleXor	(grb #Cx #Cy #r → grb') 对一个圆执行 XOR。
2557E	Sub	(grb #x1 #y1 #x2 #y2 → grb' flag) 取得 grob 的一部分。
25583	Repl	(grb1 grb2 #x #y → grb1') 以 REPLACE 模式把 grb2 复制到 grb1 中。
地址	名称	说明
25588	Gor	(grb1 grb2 #x #y → grb1') 以 OR 模式把 grb2 复制到 grb1 中。
2558D	Gxor	(grb1 grb2 #x #y → grb1') 以 XOR 模式把 grb2 复制到 grb1 中。
255A1	Grey?	(grob → flag) 该 grob 是否为灰度 grob?
255B0	ScrollVGrob	(grb #W #X #Yd #Ys #h → grb') 上下滚动图形对象的一部分。
255BA	PixonW	(grb #x #y → grb') 把一个像素设为白色。
255C4	PixonG1	(grb #x #y → grb') 把一个像素设为浅灰色。
255C9	PixonG2	(grb #x #y → grb') 把一个像素设为深灰色。
255BF	PixonB	(grb #x #y → grb') 把一个像素设为黑色。
255CE	PixonXor	(grb #x #y → grb') 对一个像素执行 XOR。
255D3	FBoxW	(grb #x1 #y1 #x2 #y2 → grb') 创建白色实心矩形。
255D3	FBoxG1	(grb #x1 #y1 #x2 #y2 → grb') 创建浅灰色实心矩形。
255D8	FBoxG2	(grb #x1 #y1 #x2 #y2 → grb') 创建深灰色实心矩形。
255DD	FBoxB	(grb #x1 #y1 #x2 #y2 → grb') 创建黑色实心矩形。
255E2	FBoxXor	(grb #x1 #y1 #x2 #y2 → grb') 对实心矩形执行 XOR。
255E7	LBoxW	(grb #x1 #y1 #x2 #y2 → grb') 绘制白色矩形。
255EC	LBoxG1	(grb #x1 #y1 #x2 #y2 → grb') 绘制浅灰色矩形。
255F1	LBoxG2	(grb #x1 #y1 #x2 #y2 → grb') 绘制深灰色矩形。
255F6	LBoxB	(grb #x1 #y1 #x2 #y2 → grb') 绘制黑色矩形。
255FB	LBoxXor	(grb #x1 #y1 #x2 #y2 → grb') 对矩形执行 XOR。
地址	名称	说明
2F21B	ToGray	(grb → grb'/grb) 把黑白 grob 转换为灰度 grob。
2F21A	Dither	(grb → grb'/grb) 把灰度 grob 转换为黑白 grob。
255B5	Distance	(#Δx #Δy → #SQRT(Δx^2+Δy^2)) 计算两点之间的距离。

15.1.5 创建菜单标签 grob

地址	名称	说明
2E166	MakeStdLabel	(\$ → grob) 创建标准菜单标签。
2E189	MakeBoxLabel	(\$ → grob) 创建带边框的标签。
2E1EB	MakeDirLabel	(\$ → grob) 创建目录标签。
2E24D	MakeInvLabel	(\$ → grob) 创建反显标签。
25E7F	Box/StdLabel	(\$ flag → grob) flag 为 TRUE 时创建带边框标签，否则创建标准标签。
25F01	Std/BoxLabel	(\$ flag → grob) flag 为 TRUE 时创建标准标签，否则创建带边框标签。
25E80	Box/StdLbl:	(→ grob) 使用运行流中的下两个对象执行 Box/StdLabel。 用法: :: Box/StdLbl: \$ <test>;
2E0D5	Grob>Menu	(#col grob →) 把 grob 显示为菜单标签。
2E0F3	Str>Menu	(#col \$ →) 把字符串显示为菜单标签。
2E11B	Id>Menu	(#col id →) 把标识符显示为菜单标签。
2E107	Seco>Menu	(#col :: →) 执行 EVAL，再执行 DoLabel。

地址	名称	说明
25886	DoLabel	(#col ob →) 如果 ob 属于支持的类型，则显示菜单标签；否则产生“Bad Argument Type”错误。

15.1.6 将字符串转换为 grob

地址	名称	说明
25F7C	\$>GROB	(\$ → grob) 使用系统字体把字符串转换为 grob。换行符不会另起一行。
25F86	\$>GROBCR	(\$ → grob) 使用系统字体把字符串转换为 grob。换行符会另起一行。
25F81	\$>grob	(\$ → grob) 使用迷你字体把字符串转换为 grob。换行符不会另起一行。
25F8B	\$>grobCR	(\$ → grob) 使用迷你字体把字符串转换为 grob。换行符会另起一行。
05F0B3	(~\$>grobOrGROB)	(\$ → grob) 根据系统标志 90，使用当前字体或迷你字体把字符串转换为 grob。
25F24	RIGHT\$3x6	(\$ #n → flag grob) 先用迷你字体把字符串转换为 grob，再取得从第 #n 列之后开始的所有字符。如果 #n 大于 grob 的宽度，则 flag 为 FALSE，并返回整个 grob。
25FEF	CENTER\$3x5	(grob #x #y \$ #w → grob') 使用迷你字体从字符串创建 grob，并把它嵌入指定位置 (#x,#y)。该 grob 以 #x 为中心，顶部置于 #y。#w 是所创建 grob 的最大宽度；文字过宽时会被截断。bang 型。
地址	名称	说明
2E2AA	MakeLabel	(\$ #w #x grob → grob') 使用 CENTER\$3x5 且令 y=5，把 \$ 插入 grob。
25FF9	LEFT\$3x5	(grob #x #y \$ #w → grob') 与 CENTER\$3x5 类似，但文字左上角定位在 #x。
26071	ERASE&LEFT\$3x5	(grob #x #y \$ #w → grob') 与 LEFT\$3x5 类似，但先清除背景。
26008	LEFT\$3x5Arrow	(grob #x #y \$ #w → grob') 与 LEFT\$3x5 类似；如果文字无法容纳，则把最后一个字符替换为字符 31（点状符号），表示文字已被截断。
2601C	LEFT\$3x5CR	(grob #x #y \$ #w #h → grob') 与 LEFT\$3x5 类似，但会解释字符串中的换行符并另起新行。附加参数 #h 表示文字 grob 的最大高度。
26012	LEFT\$3x5CRArrow	(grob #x #y \$ #w #h → grob') 与 LEFT\$3x5CR 类似，但用箭头表示截断。
25FF4	CENTER\$5x7	(grob #x #y \$ #w → grob') 与 CENTER\$3x5 相同，但使用系统字体。
25FFE	LEFT\$5x7	(grob #x #y \$ #w → grob') 与 CENTER\$5x7 类似，但文字左上角定位在 #x。
2606C	ERASE&LEFT\$5x7	(grob #x #y \$ #w → grob') 与 LEFT\$5x7 类似，但先清除背景。
26003	LEFT\$5x7Arrow	(grob #x #y \$ #w → grob') 与 LEFT\$5x7 类似；如果必须截断文字，则把最后一个字符替换为字符 31（箭头）。
26017	LEFT\$5x7CR	(grob #x #y \$ #w → grob') 与 LEFT\$5x7 类似，但会解释换行符。
2600D	LEFT\$5x7CRArrow	(grob #x #y \$ #w → grob') 与 LEFT\$5x7CR 类似，但用箭头表示截断。

15.1.7 从其他对象创建 grob

地址	名称	说明
019004	^EQW3GROB	(ob → ext grob #0) (ob → #2)
01A004	^EQW3GROBStk	(ob → ext grob #0) (ob → #2)
01F004	^EQW3GROBmini	(ob → ext grob #0) (ob → #2)
01E004	^EQW3GROBsys	(ob → ext grob #0) (ob → #2)
0BE007	^XGROBext	(ob → grob) 把对象转换为 grob。
0C0007	^DISPLAYext	(grob ob → grob') 先把 ob 转换为 grob，再将其加入原 grob。

第 16 章

库对象和备份对象

库是非常复杂的对象，用于容纳一组命令。其中一些命令有名称，可由用户访问；另一些没有名称，因此是“隐藏”的。HP49使用备份对象保存整个 HOME 目录的内容，以便以后恢复。两类对象都附带 CRC 码，因此可以验证其完整性。

ROM 指针（有时称为 XLIB 名称）是指向库中某条命令的指针。访问库中无名命令的唯一方法是使用 ROM 指针。ROM 指针保存库编号（通常称为 id）和命令编号。

在程序中插入 ROM 指针时，使用以下结构：ROMPTR <lib> <cmd>，其中 <lib> 是库编号，<cmd> 是命令编号；两个编号均以十六进制形式指定。ROM 指针与标识符一样总会自动执行，因此若要把 ROM 指针本身放入堆栈，必须将其引用，参见第 19.2 节。

16.1 参考

16.1.1 端口操作

地址	名称	说明
25EEB	NEXTLIBBAK	(#addr → backup/library #nextaddr) 取得下一个库对象或备份对象。

16.1.2 ROM 指针

地址	名称	说明
07E50	#>ROMPTR	(#lib #cmd → ROMPTR) 创建 ROM 指针。
08CCC	ROMPTR>#	(ROMPTR → #lib #cmd) 拆分 ROM 指针。
07E99	ROMPTR@	(ROMPTR → ob T) (ROMPTR → F) 提取 ROM 指针所指内容。
35C40	DUPROMPTR@	(ROMPTR → ROMPTR ob T) (ROMPTR → ROMPTR F) 先执行 DUP，再执行 ROMPTR@。
35A88	?>ROMPTR	(ob → ob') 如果 ROM-WORD? 和 TYPECOL? 的结果为真，则执行 RPL@。
35AAB	?ROMPTR>	(ob → ob') 如果 TYPEROMP? 为真，并且所指内容通过 INHARDROM? 检查，则返回其内容。
35BFF	RESOROMP	(→ ob) 提取运行流中的下一个对象；该对象必须是 ROM 指针。
34FCD	ROM-WORD?	(ob → flag)
34FC0	DUPROM-WORD?	(ob → ob flag)

16.1.3 库

地址	名称	说明
07709	TOSRRP	(# →) 把库挂接到 HOME 目录。
076AE	OFFSRRP	(# →) 从 HOME 目录解除挂接库。
2F2A7	XEQSETLIB	(% →) ATTACH 的内部版本。
07638	SETHASH	(hxs #libnum →) 可能有缺陷？

16.1.4 备份对象

地址	名称	说明
081D9	BAKNAME	(bak → id T) 返回备份对象的名称。
0905F	BAK>OB	(bak → ob) 获取备份对象。

第二部分 通用系统 RPL 词条

第 17 章 堆栈操作

在系统 RPL 中，堆栈的用法几乎与用户 RPL 中相同。基本操作相同，只有名称略有变化：DUP、2DUP（相当于用户 RPL 的 DUP2）、NDUP（DUPN）、DROP、2DROP（DROP2）、NDROP（DROPN）、OVER、PICK、SWAP、ROLL、UNROLL（ROLLD）、ROT、UNROT 和 DEPTH。

除非另有说明，凡需要或返回数值参数的命令都使用二进制整数（bint），而不是实数。

有许多命令会依次完成两个甚至三个操作。参考一节列出了这些命令。下表以便于查阅的形式列出了一些实用组合：

	DUP	DROP	SWAP	OVER	ROT	UNROT	
DUP	DUPDUP	DROPDUP	SWAPDUP	OVERDUP	ROTDUP	UNROTDUP	
DROP	-	2DROP	SWAPDROP	-	ROTDROP	UNROTDROP	
SWAP	-	DROPSWAP	-	OVERSWAP	ROTSWAP	UNROTSWAP	
OVER	DUPDUP	DROPOVER	SWAPOVER	2DUP	ROTOVER	UNROTOVER	
ROT	DUPROT	DROPROT	SWAPROT	-	UNROT	-	
UNROT	DUPUNROT	-	ROTSWAP	OVERUNROT	-	ROT	
SWAPDROP	-	DROPSWAPDROP	-	DROPDUP	DROPSWAP	UNROTSWAPDRO	
DROPDUP	-	-	SWAPDROPDUP	-	-	-	
DROPSWAP	-	-	SWAPDROPSWAP	-	ROTDROPSWAP	SWAPDROP	
2DROP	-	3DROP	-	-	ROT2DROP	UNROT2DROP	
2DUP	-	-	SWAP2DUP	-	ROT2DUP	-	
3PICK	DUP3PICK	-	SWAP3PICK	OVERDUP	-	-	
4PICK	-	-	SWAP4PICK	-	-	-	
5PICK	-	-	OVER5PICK	-	-	-	
4ROLL	-	-	SWAP4ROLL	-	-	-	
4UNROLL	DUP4UNROLL	-	-	-	-	-	
ROT2DROP	-	-	ROTRROT2DROP	SWAPDROP	ROTRROT2DROP	-	

17.1 参考

本节用数字 1、2……n 表示不同的对象；这些对象不一定是任何类型的数。

地址	名称	说明
03188	DUP	(ob → ob ob)
35CE0	DUPDUP	(ob → ob ob ob)
2D5006	^3DUP	(3 2 1 → 3 2 1 3 2 1)
28143	NDUPN	(ob #n → ob..ob #n) (ob #0 → #0)
35FF3	DUPROT	(1 2 → 2 2 1)
3457F	DUPUNROT	(1 2 → 2 1 2) 又名：SWAPOVER
36133	DUPROLL	(1..n #n → 1 3..n #n 2)
3432C	DUP4UNROLL	(1 2 3 → 3 1 2 3)
3611F	DUPPICK	(n..1 #n → n..1 #n n-1)
35D30	DUP3PICK	(1 2 → 1 2 2 1) 又名：2DUPSWAP
34431	DUP#1+PICK	(n..1 #n → n..1 #n n)
031AC	2DUP	(1 2 → 1 2 1 2)
35D30	2DUPSWAP	(1 2 → 1 2 2 1) 又名：DUP3PICK
36CA4	2DUP5ROLL	(1 2 3 → 2 3 2 3 1)
031D9	NDUP	(1..n #n → 1..n 1..n)
03244	DROP	(1 →)
357CE	DROPDUP	(1 2 → 1 1)
37032	DROPNDROP	(1..n #n ob →)
35733	DROPSWAP	(1 2 3 → 2 1)
3574D	DROPSWAPDROP	(1 2 3 → 2) 又名：ROT2DROP, XYZ>Y
36007	DROPROT	(1 2 3 4 → 2 3 1)
3606B	DROPOVER	(1 2 3 → 1 2 1)

地址	名称	说明
03258	2DROP	(1 2 →)
341D2	3DROP	(1 2 3 →) 又名: XYZ>
341D7	4DROP	(1..4 →) 又名: XYZW>
341DC	5DROP	(1..5 →)
341E8	6DROP	(1..6 →)
341F4	7DROP	(1..7 →)
0326E	NDROP	(1..n #n →)
35FB0	#1+NDROP	(ob 1..n #n →) 又名: N+1DROP
地址	名称	说明
2F0A1	RESETDEPTH	(obl..obn obn+1..obx #n → obl..obn) 保留 #n 个堆栈层, 并丢弃其余所有堆栈层。
0314C	DEPTH	(1..n → 1..n #n)
28187	reversym	(1..n #n → n..1 #n)
03223	SWAP	(1 2 → 2 1)
3576E	SWAPDUP	(1 2 → 2 1 1)
368B5	SWAP2DUP	(1 2 → 2 1 2 1)
3421A	SWAPDROP	(1 2 → 2) 又名: XY>Y
35857	SWAPDROPDUP	(1 2 → 2 2)
35872	SWAPDROPSWAP	(1 2 3 → 3 1) 又名: UNROTDROP, XYZ>ZX
341BA	SWAPROT	(1 2 3 → 3 2 1) 又名: UNROTSWAP, XYZ>ZYX
36C90	SWAP4ROLL	(1 2 3 4 → 2 4 3 1) 又名: XYZW>YWZX
3457F	SWAPOVER	(1 2 → 2 1 2) 又名: DUPUNROT
36CB8	SWAP3PICK	(1 2 3 → 1 3 2 1)
35018	2SWAP	(1 2 3 4 → 3 4 1 2)
03295	ROT	(1 2 3 → 2 3 1)
3579C	ROTDUP	(1 2 3 → 2 3 1 1)
35CA4	ROT2DUP	(1 2 3 → 2 3 1 3 1)
341A8	ROTDROP	(1 2 3 → 2 3) 又名: XYZ>YZ
3574D	ROT2DROP	(1 2 3 → 2) 又名: DROPSWAPDROP, XYZ>Y
34195	ROTDROPSWAP	(1 2 3 → 3 2) 又名: XYZ>ZY
3416E	ROTSWAP	(1 2 3 → 2 1 3) 又名: XYZ>YXZ
343BD	ROTROT2DROP	(1 2 3 → 3) 又名: UNROT2DROP, XYZ>Z
35CCC	ROTOVER	(1 2 3 → 2 3 1 3)
3423A	4ROLL	(1 2 3 4 → 2 3 4 1) 又名: FOURROLL, XYZW>YZWX
3588B	4ROLLDROP	(1 2 3 4 → 2 3 4)
35F06	4ROLLSWAP	(1 2 3 4 → 2 3 1 4)
地址	名称	说明
36043	4ROLLROT	(1 2 3 4 → 2 4 1 3) 又名: FOURROLLROT
360E3	4ROLLOVER	(1 2 3 4 → 2 3 4 1 4)
34257	5ROLL	(1 2 3 4 5 → 2 3 4 5 1) 又名: FIVEROLL
358A7	5ROLLDROP	(1 2 3 4 5 → 2 3 4 5)
34281	6ROLL	(1..6 → 2..6 1) 又名: SIXROLL

地址	名称	说明
342EA	7ROLL	(1..7 → 2.7 1) 又名: SEVENROLL
342BB	8ROLL	(1..8 → 2.8 1) 又名: EIGHTROLL
03325	ROLL	(1..n #n → 2..n 1)
35FC4	ROLLDROP	(1..n #n → 2..n)
35D80	ROLLSWAP	(1..n #n → 2..n-1 1 n)
344F2	#1+ROLL	(ob 1..n #n → 1..n ob)
34517	#2+ROLL	(a b 1..n #n → b 1..n a)
2D6006	^#3+ROLL	(obn+3...obn...ob1 #n → obn+2...ob1 obn+3)
344DD	#+ROLL	(1..n+m #n #m → 2..n+m 1)
344CB	#-ROLL	(1..n-m #n #m → 2..n-m 1)
3422B	UNROT	(1 2 3 → 3 1 2) 又名: 3UNROLL, XYZ>ZXY
35D1C	UNROTDUP	(1 2 3 → 3 1 2 1)
35872	UNROTDROP	(1 2 3 → 3 1) 又名: SWAPDROPSWAP, XYZ>ZX
343BD	UNROT2DROP	(1 2 3 → 3) 又名: ROTROT2DROP, XYZ>Z
341BA	UNROTSWAP	(1 2 3 → 3 2 1) 又名: SWAPROT, XYZ>ZYX
360CF	UNROTOVER	(1 2 3 → 3 1 2 1)
3422B	3UNROLL	(1 2 3 → 3 1 2) 又名: UNROT, XYZ>ZXY
34331	4UNROLL	(1 2 3 4 → 4 1 2 3) 又名: FOURUNROLL, XYZW>WXYZ
35D44	4UNROLLDUP	(1 2 3 4 → 4 1 2 3 3)
343CF	4UNROLL3DROP	(1 2 3 4 → 4) 又名: XYZW>W
36057	4UNROLLROT	(1 2 3 4 → 4 3 2 1)
地址	名称	说明
34357	5UNROLL	(1 2 3 4 5 → 5 1 2 3 4) 又名: FIVEUNROLL
3438D	6UNROLL	(1..6 → 6 1..5) 又名: SIXUNROLL
35BEB	7UNROLL	(1..7 → 7 1..6)
3615B	8UNROLL	(1..8 → 8 1..7)
28225	(9UNROLL)	(1..9 → 9 1..8)
3616F	10UNROLL	(1..10 → 10 1..9)
0339E	UNROLL	(1..n #n → n 1..n-1)
34552	#1+UNROLL	(ob 1..n #n → n ob 1..n-1)
34564	#2+UNROLL	(a b 1..n #n → n a b 1..n-1)
3453D	#+UNROLL	(1..n+m #n #m → n+m 1..n+m-1)
3452B	#-UNROLL	(1..n-m #n #m → n-m 1..n+m-1)
032C2	OVER	(1 2 → 1 2 1)
35CF4	OVERDUP	(1 2 → 1 2 1 1)
35D6C	OVERSWAP	(1 2 → 1 1 2) 又名: OVERUNROT
35D6C	OVERUNROT	(1 2 → 1 1 2) 又名: OVERSWAP
36CF4	OVER5PICK	(1 2 3 4 → 1 2 3 4 3 1)
37046	2OVER	(1 2 3 4 → 1 2 3 4 1 2)
34485	3PICK	(1 2 3 → 1 2 3 1)
35F1A	3PICKSWAP	(1 2 3 → 1 2 1 3)
360F7	3PICKOVER	(1 2 3 → 1 2 3 1 3)

地址	名称	说明
36CCC	3PICK3PICK	(1 2 3 → 1 2 3 1 2)
2F1C6	DROP3PICK	(1 2 3 4 → 1 2 3 1)
3448A	4PICK	(1 2 3 4 → 1 2 3 4 1)
35F2E	4PICKSWAP	(1 2 3 4 → 1 2 3 1 4)
36CE0	SWAP4PICK	(1 2 3 4 → 1 2 4 3 1)
3610B	4PICKOVER	(1 2 3 4 → 1 2 3 4 1 4)
3448F	5PICK	(1 2 3 4 5 → 1 2 3 4 5 1)
34494	6PICK	(1..6 → 1..6 1)
34499	7PICK	(1..7 → 1..7 1)
3449E	8PICK	(1..8 → 1..8 1)
344A3	(9PICK)	(1..9 → 1..9 1)
344A8	(10PICK)	(1..10 → 1..10 1)
032E2	PICK	(1..n #n → 1..n 1)
34436	#1+PICK	(1..n #n-1 → 1..n 1)
34451	#2+PICK	(1..n #n-2 → 1..n 1)
地址	名称	说明
34465	#3+PICK	(1..n #n-3 → 1..n 1)
34474	#4+PICK	(1..n #n-4 → 1..n 1)
34417	#+PICK	(1..n+m #n #m → 1..n+m 1)
34405	#-PICK	(1..n-m #n #m → 1..n-m 1)

第 18 章 临时环境

系统 RPL 的局部变量（也称临时变量或 **lambda** 变量）与用户 RPL 中的局部变量具有相同的工作方式和用途。您可以为它们赋值，并在变量存在期间提取或更改这些值。存储的值通过局部标识符引用（也称 **lambda** 标识符，简称 **lam**）。这些标识符与引用内存中所存变量的全局标识符非常相似（参见第 8 章），但这些变量只会临时存在。

不过，两者有一个区别：在系统 RPL 中，可以为局部变量指定 **null**（即空）名称，从而实际上使其成为无名变量。这样既节省内存，速度也快得多。在学习如何创建和使用无名局部变量之前，先来了解如何使用普通的命名局部变量。

18.1 命名局部变量

创建命名局部变量的方法与在用户 RPL 中创建临时变量非常相似。您必须创建一个由局部标识符（简称 **lam**）组成的列表，并执行 **BIND** 命令。要提取其中某个变量的内容，只需输入相应的局部标识符。要存储新值，请将该值和 **lam** 放入堆栈，然后执行 **STO**。要从内存中移除局部变量，请使用 **ABND**（“**abandon**”的缩写）。系统不会检查代码中的 **BIND** 与 **ABND** 是否成对，因此可以根据需要将它们放在不同的程序中。但这也意味着，您必须确保每个 **BIND** 都有一个对应的 **ABND**。

下面这个小程序创建两个局部变量，提取它们的内容并为其赋予新值（程序名为 **LAM1**）：

```
1 ::
%2 %3
{
  LAM first

  5 LAM sec
}
BIND (first 中存放 2, sec 中存放 3)
LAM first (从 first 提取内容 - 2)
LAM sec (从 sec 提取内容 - 3)
10 DUP
' LAM first
STO (将新内容存入 first)
%+ (结果为 5)
' LAM sec
15 STO (将求和结果存入 sec)
ABND (从内存中删除变量)
;
```

18.2 无名局部变量

如上所述，可以使用无名局部变量。从技术上说，它们有名称，即空（**null**）名称；但所有“无名”变量的名称都相同。由于无法按名称标识它们，因此必须使用位置语法。上面的程序可按以下方式改用空名称临时变量重写（现在命名为 **LAM2**）：

```
1 ::
%2 %3
{ NULLLAM NULLLAM }
BIND
5 2GETLAM (提取 2)
1GETLAM (提取 3)
DUP
2PUTLAM
%+
10 1PUTLAM
ABND
;
```

编号顺序与堆栈层的顺序相同：1GETLAM 对应原先位于堆栈第 1 层的内容，2GETLAM 对应原先位于堆栈第 2 层的内容，依此类推。系统提供了可直接提取和存储前 22 个变量的词条（1GETLAM 到 22GETLAM，以及与它们对应的 PUTLAM 词条）。若要访问编号大于 23 的变量（这种情况可能不常发生），请使用 GETLAM；它接受一个表示变量编号的 **bint**，并返回该变量的

内容；PUTLAM 接受一个对象和变量编号，并将该对象存入指定变量。

18.3 嵌套临时环境

完全可以同时使用两个或更多临时环境。创建时无需任何特殊处理：只需在放弃前一个环境之前再次使用 DOBIND 或 BIND。遇到 ABND 时，它总是对应最近的 BIND。

如果只使用命名 lam，则无需任何特殊处理。名称不会发生混淆，除非您重新定义已有变量（但这样只会使程序变得非常混乱）。不过，只要至少一个临时环境包含无名 lam，就必须注意编号。

请注意，GETLAM 命令字不一定指向无名局部变量：1GETLAM 提取最近绑定的变量，2GETLAM 提取它前一个绑定的变量，依此类推。（绑定 lam 时，会从编号最大的堆栈层开始，逐层向编号最小的堆栈层进行，因此最后绑定的变量，其内容原先位于堆栈第 1 层。）也可以使用 GETLAM 命令字访问命名 lam。

由于临时环境的性质，在所有其他变量之前似乎还存在一个额外的局部变量，用于内部管理。要访问前一个环境中的无名 lam，必须在第二次绑定之前所用的编号上，加上当前环境中绑定的变量数，再加 1。

以下程序（名为 LAM3）将帮助说明上述内容：

```
1 ::
%2
%1
{
5 LAM n2
LAM n1
}
BIND
1GETLAM (返回 1)
10 2GETLAM (返回 2)

%4
%3
{
15 NULLLAM
NULLLAM
}
BIND
1GETLAM (返回 3)
20 2GETLAM (返回 4)
4GETLAM (返回 1)
5GETLAM (返回 2)
ABND
ABND
25 ;
```

首先，此程序将 2 绑定到 n2，将 1 绑定到 n1，但从未使用这些名称。程序改用 1GETLAM 访问最近绑定的值，即 1；也可以通过 LAM n1 访问该值。随后，2GETLAM 返回倒数第二个绑定值，即 2。

再绑定一个环境时，情况会变得更复杂；这一次绑定的是无名 lam。现在，1GETLAM 返回 3，它属于新环境，并且是最后绑定的变量。同样，2GETLAM 返回的值也来自第二批绑定的变量。

如果要访问先前编号为 1 的变量，就必须在原编号上加上新环境中绑定的变量数（即 2），再加 1（内部管理用的伪变量）。因此，要获得第二次绑定之前 1GETLAM 会返回的值，就将 3 加到 1 上，得到 4GETLAM。它会如预期返回 1。同样，5GETLAM 返回 2，也就是第二次绑定之前 2GETLAM 返回的值。

自然，在执行第一个 ABND（对应值 4 和 3 的绑定）之后，1GETLAM 和 2GETLAM 将再次分别返回 1 和 2。

如果您已理解上述内容，那么在需要时嵌套临时环境就不会有问题。

18.4 其他绑定方式

首先，可以不使用 lam 列表，而是始终将每个 lam 放入堆栈，随后放入要绑定的变量数，并执行 DOBIND 命令而不是 BIND。实际上，BIND 就是 :: INNERCOMP DOBIND ；。

绑定大量局部变量时，不必输入以下代码（占用 67.5 字节）：

```
1 ...
{ NULLLAM NULLLAM NULLLAM NULLLAM NULLLAM NULLLAM
NULLLAM NULLLAM NULLLAM NULLLAM NULLLAM NULLLAM
NULLLAM NULLLAM NULLLAM NULLLAM NULLLAM NULLLAM
5 NULLLAM NULLLAM NULLLAM NULLLAM NULLLAM NULLLAM }
BIND
...
```

而可使用以下代码：它只占用 12.5 字节，可节省 55 字节：

```
... NULLLAM TWENTYFOUR NDUPN {}N BIND ...
```

不过，如果随后还要展开复合程序，为什么要先创建它？将 {}N BIND 替换为 DOBIND，还可再节省 2.5 字节。也可以使用 TWENTYFOUR ' NULLLAM CACHE。不过，这样会额外创建一个变量来保存计数值，因此必须将前面示例中的变量位置编号加 1。

反编译代码时，有时会看到如下内容：

```
... ZEROZEROZERO BINT3 DOBIND ...
```

这是绑定 3 个空名称变量的另一种方法。之所以可行，是因为可以使用任何固定地址 ROM 对象来代替 NULLLAM，本例中使用的就是 ZERO。

以下结构是为 N 个空名称变量创建临时环境的最紧凑方式。

```
N  创建 N 个空名称变量的命令
1  1LAMBIND
2  ZEROZEROTWO DOBIND
2  FPTR2 ^2LAMBIND
3  FPTR2 ^3LAMBIND
4  4NULLLAM{} BIND
N  NULLLAM #N NDUPN DOBIND
```

18.5 参考

18.5.1 内置 ID 和 LAM

地址	名称	说明
272FE	NULLID	(→ id) 空（即内容为空的）标识符
2B3AB	NULLLAM	(→ lam) 将 NULLLAM 压入堆栈。
27155	'IDX	(→ id) 将未经求值的 ID X 压入堆栈。
272F3	(ID_EQ)	ID EQ
27937	(ID_SIGMADAT)	ID ΣDAT

18.5.2 转换

地址	名称	说明
05B15	\$>ID	(\$ → ID)
362DE	DUP\$>ID	(\$ → \$ ID)

18.5.3 临时环境命令字

地址	名称	说明
074D0	BIND	(obn..obl {lamn..lam1} →) 将 n 个对象绑定到 n 个名称各异的 lam。
074E4	DOBIND	(obn..obl lamn..lam1 #n →) 将 n 个对象绑定到 n 个名称各异的 lam。
36518	1LAMBIND	(ob →) 将一个对象绑定到空名称 lam。
36513	DUP1LAMBIND	(ob → ob) 先执行 DUP，再执行 1LAMBIND。
155006	^2LAMBIND	(ob1 ob2 →) 将两个对象绑定到空名称 lam。
156006	^3LAMBIND	(ob1 ob2 ob3 →) 将三个对象绑定到空名称 lam。
地址	名称	说明
0DE0B0	~nNullBind	(obn..obl #n →) 将 #n 个对象绑定到空名称 lam。1LAM 中为计数，2LAM 中为第一个对象。反编译为： :: ' NULLLAM CACHE ;
36A77	dvarlsBIND	(ob →) 将 ob 绑定到 LAM 'dvar。
07497	ABND	(→) 放弃最顶层临时环境。
34D00	CACHE	(obn..obl #n lam →) 将所有对象绑定到同一名称。1LAM 中为计数。
34EBE	DUMP	(NULLLAM → ob1..obn #n) CACHE 的逆操作。始终进行垃圾回收。

地址	名称	说明
34D58	SAVESTACK	(→) 将堆栈缓存到 SAVELAM。
34FA6	undo	(→) 转储 SAVELAM。
07943	@LAM	(lam → ob T) (lam → F) 尝试从 lam 提取对象。成功时返回对象和 TRUE，否则只返回 FALSE。
07D1B	STOLAM	(ob lam →) 尝试将对象存储到 lam。如果找不到 lam，则生成“Undefined Local Name”错误。
075A5	GETLAM	(#n → ob) 获取从顶部起第 n 个局部名称的内容。
34616	1GETLAM	(→ ob)
34620	2GETLAM	(→ ob)
3462A	3GETLAM	(→ ob)
34634	4GETLAM	(→ ob)
3463E	5GETLAM	(→ ob)
34648	6GETLAM	(→ ob)
34652	7GETLAM	(→ ob)
3465C	8GETLAM	(→ ob)
34666	9GETLAM	(→ ob)
34670	10GETLAM	(→ ob)
地址	名称	说明
3467A	11GETLAM	(→ ob)
34684	12GETLAM	(→ ob)
3468E	13GETLAM	(→ ob)
34698	14GETLAM	(→ ob)
346A2	15GETLAM	(→ ob)
346AC	16GETLAM	(→ ob)
346B6	17GETLAM	(→ ob)
346C0	18GETLAM	(→ ob)
346CA	19GETLAM	(→ ob)
346D4	20GETLAM	(→ ob)
346DE	21GETLAM	(→ ob)
346E8	22GETLAM	(→ ob)
346F2	(23GETLAM)	(→ ob)
346FC	(24GETLAM)	(→ ob)
34706	(25GETLAM)	(→ ob)
34710	(26GETLAM)	(→ ob)
3471A	(27GETLAM)	(→ ob)
075E9	PUTLAM	(ob #n →) 将新内容存入从顶部起第 n 个局部名称。
34611	1PUTLAM	(ob →)
3461B	2PUTLAM	(ob →)
34625	3PUTLAM	(ob →)
3462F	4PUTLAM	(ob →)
34639	5PUTLAM	(ob →)
34643	6PUTLAM	(ob →)
3464D	7PUTLAM	(ob →)
34657	8PUTLAM	(ob →)
34661	9PUTLAM	(ob →)
3466B	10PUTLAM	(ob →)

地址	名称	说明
34675	11PUTLAM	(ob →)
3467F	12PUTLAM	(ob →)
34689	13PUTLAM	(ob →)
34693	14PUTLAM	(ob →)
3469D	15PUTLAM	(ob →)
346A7	16PUTLAM	(ob →)
346B1	17PUTLAM	(ob →)
346BB	18PUTLAM	(ob →)
346C5	19PUTLAM	(ob →)
346CF	20PUTLAM	(ob →)
地址	名称	说明
346D9	21PUTLAM	(ob →)
346E3	22PUTLAM	(ob →)
346ED	(23PUTLAM)	(ob →)
346F7	(24PUTLAM)	(ob →)
34701	(25PUTLAM)	(ob →)
3470B	(26PUTLAM)	(ob →)
34715	(27PUTLAM)	(ob →)
34797	DUP4PUTLAM	(ob → ob) 先执行 DUP，再执行 4PUTLAM。
364FF	1GETABND	(→ 1lamob) 先执行 1GETLAM，再执行 ABND。
35DEE	1ABNDSWAP	(ob → 1lamob ob) 先执行 1GETABND，再执行 SWAP。
35F42	1GETSWAP	(ob → 1lamob ob) 先执行 1GETLAM，再执行 SWAP。
2F318	1GETLAMSWP1+	(# → 1lamob #+1) 先执行 1GETLAM，再执行 SWAP#1+。
3632E	2GETEVAL	(→ ?) 先执行 2GETLAM，再执行 EVAL。
3483E	GETLAMPAIR	(#n → #n ob lam F) (#n → #n T) 获取局部名称的内容和名称 (10 = 1lam, 20 = 2lam, 依此类推)。
347AB	DUPTEMPENV	(→) 复制最顶层临时环境 (清除保护命令字)。
2B3A6	1NULLLAM{ }	(→ { }) 将含一个 NULLLAM 的列表压入堆栈。
271F4	(2NULLLAM{ })	(→ { }) 将含两个 NULLLAM 的列表压入堆栈。
27208	(3NULLLAM{ })	(→ { }) 将含三个 NULLLAM 的列表压入堆栈。
2B3B7	4NULLLAM{ }	(→ { }) 将含四个 NULLLAM 的列表压入堆栈。

第 19 章 运行流控制

到目前为止，您所见的命令都不会影响正常的程序流程。前面给出的所有程序都按顺序工作，从第一条命令执行到最后一条命令，期间不会改变这一顺序。然而，除最简单的程序外，通常都需要打破默认的执行顺序。有时需要将程序的某一部分重复执行若干次，或者只在特定条件下执行某些操作。

本章介绍一些会影响正常执行顺序的低级词条。上述情形可以用循环（见第 21 章）和条件结构（见第 20 章）等高级结构实现，而且您使用这些结构的频率很可能会高于使用下面的大多数词条。不过，本章还会介绍一些概念，帮助您理解系统 RPL 程序的工作方式，以及如何改变正常的程序流程。

19.1 一些概念

正如您从“引言”中所知，编译后的系统 RPL 程序由一系列指向内存地址的指针组成。程序的基本执行方式是：跳转到指针所指的地址，执行该处的内容，返回程序，再跳转到下一个地址，如此继续。

实际情况要复杂一些，因为程序中还会嵌入实数、字符串，甚至其他程序（二级程序）等对象。要正确处理这些对象，需要一些“魔法”（实际上只是精心编写的代码），但说明其处理方式超出了本文档的范围。只需假定发现对象时就会“执行”它。对于实数或字符串等大多数对象，这意味着将对象自身压入堆栈；对于二级程序，这意味着执行其内容；对于标识符等其他对象，这意味着尝试提取并执行其内容，或者仅将标识符自身压入堆栈。

由于对象按顺序执行，因此需要某个变量始终指向下一个待执行的对象。这个变量称为解释器指针，存储在 CPU 寄存器中。每执行完一个对象，该指针就前移并指向下一个对象。

程序中遇到 DUP 时，实际发生的过程如下：程序中真正存储的只有地址 #03188h。系统跳转到该地址；该地址处有一段机器语言代码。执行这段代码后，解释器指针前移，系统再跳转到下一个对象，无论该对象是什么。

例如，要执行 INCOMPDROP，情况会稍微复杂一些。在该命令的地址处有一个二级程序对象，其内容恰好是:: INNERCOMP DROP ；。问题在于，必须切换到这个（子）程序，执行其全部内容，然后返回调用 INCOMPDROP 的程序。由于子程序完全可能包含更多子程序，因此需要某种堆栈。执行二级程序（或任何其他复合程序）时，调用程序中位于该复合程序之后的对象地址会被压入这个堆栈。随后，通过让解释器指针依次指向复合程序中的各个对象来执行它。执行结束时，从返回堆栈弹出一个地址，并从该地址继续执行。这个地址就是前一个程序中下一个对象的地址，因此执行可以正确恢复。这个堆栈称为返回堆栈。

以上说明并不完整，但应当足以让您大致了解其工作原理。系统 RPL 程序还有许多细节，若逐一详述会使说明过于冗长复杂，因此本书不再给出这种详细说明。

另一个重要概念是运行流。运行流是当前正在执行的对象之后的一系列对象。例如，在以下程序中执行' 命令期间：

```
:: ' DUP :: EVAL ; % 1. ;
```

运行流包含三个对象。第一个是命令 DUP。第二个是内部包含 EVAL 命令的二级程序（不是 EVAL 命令本身，也不只是::），第三个是实数一。有若干命令字（包括下文将介绍的'）从运行流中的下一个对象取得参数，而不像大多数命令那样从数据堆栈取得参数。因此，'的“参数”就是命令 DUP。

现在您应当已经明白本章为何称为“运行流控制”：这里的命令会影响运行流，也就是影响组成程序的各个对象的执行顺序。

19.2 运行流命令

这里介绍的命令是可用的基本操作。在下文参考部分中，您会看到多条将这些命令与其他命令组合起来的命令。

命令	堆栈与说明
'	(→ ob) 这很容易理解：它将紧随其后的对象（即运行流中的第一个对象）压入堆栈。被压入的对象不会执行；执行从该对象之后的对象恢复。例如： :: %1 %2 ' %+ EVAL ; 等价于： :: %1 %2 %+ ; 这种不对下一个对象求值、而是将其压入堆栈的操作，称为引用下一个对象。

命令	堆栈与说明
'R	(→ ob) 将返回堆栈最顶层指针所指向的对象压入数据堆栈，并跳过这个被压入的对象。换言之，包含当前正在执行的复合程序的那个复合程序中的第一个对象，会被压入数据堆栈并被跳过。不过，如果本应压入的对象是 SEMI，则改为压入一个空复合程序。例如，RESOROMP 命令与 ROMPTR@ 类似，但其参数位于运行流中紧随其后的位置（见第 16 章）。RESOROMP 的定义如下： :: 'R ROMPTR@ DROP ; 它只是将 RESOROMP 后面的对象压入堆栈，并调用 ROMPTR@。
ticR	(→ ob TRUE) (→ FALSE) 该命令与 'R 类似，但如果没有可压入的对象，它不会压入空复合程序，而是返回 FALSE。如果有对象可以压入，则将该对象与 TRUE 一同压入。
命令	堆栈与说明
>R	(comp →) 将指向参数所给复合程序主体的指针压入返回堆栈。这意味着当前二级程序结束时，执行不会直接返回调用当前复合程序的二级程序。在此之前，会先执行作为参数给出的复合程序；只有该复合程序结束后，执行才会在调用当前复合程序的二级程序中恢复。例如，下面的代码会按 3、2、1 的顺序将这些实数压入堆栈： :: ' :: % 1 ; >R % 3 % 2 ;
R>	(→ ::) 将返回堆栈中的第一个指针弹出，并把该指针所指内容构成一个二级程序压入数据堆栈。换言之，它把调用当前二级程序的二级程序中剩余的命令作为一个二级程序压入堆栈。当前二级程序结束后，这些命令将不再自动执行。例如，下面的代码会按 3、2、1 的顺序将这些实数压入堆栈： :: :: R> EVAL % 1 ; % 3 % 2 ;
R@	(→ ::) 与 R> 相同，但不会弹出返回堆栈。若在上一个命令的同一示例中将 R> 改为 R@，则会返回 3、2、1、3 和 2。
IDUP	(→) 将解释器指针压入返回堆栈。这意味着当前二级程序结束后，系统会跳转到 IDUP 后面的对象，从而再次执行当前二级程序的剩余部分。
RDROP	(→) 弹出返回堆栈。也就是说，调用当前二级程序的二级程序中剩余的对象将不会执行。
RDUP	(→) 复制返回堆栈顶层的地址。
RSWAP	(→) 交换返回堆栈顶端的两个地址。
?SEMI	(flag →) 如果标志为 TRUE，则跳过当前二级程序的剩余部分。
命令	堆栈与说明
COLA	(→) 只执行运行流中的下一个对象，并跳过当前二级程序的剩余部分。下面的程序只会将 1 压入堆栈： :: COLA % 1 % 2 % 3 ; 下文将介绍 COLA 的一些实用用法。
SKIP	(→) 跳过运行流中的下一个对象。若将上面程序中的 COLA 替换为 SKIP，则会将 2 和 3 压入堆栈。
?SKIP	(flag →) 如果标志为 TRUE，则执行 SKIP。

19.3 一些示例

第一个示例说明 COLA 在递归中的一种实用用法。假设有下面两个程序用于计算一个数的阶乘：
fact:

```
1 ::  
  CKREAL  
  { LAM x } BIND  
  %1 (阶乘计算的初始值)  
5 factiter  
  ABND  
;
```

factiter:

```
1 ::  
  LAM x %0= ?SEMI (若 x=0，则退出)
```

```

    LAM x %* (乘以当前值)
    LAM x %1- ' LAM x STO
5 COLA factiter
;

```

请注意，在递归调用 `factiter` 之前有命令字 `COLA`。如果没有它，程序将需要许多返回堆栈层，而且每一层都指向 `SEMI`。使用 `COLA` 后，不会向返回堆栈压入任何内容。`factiter` 只是被直接调用，并不存储解释器应当跳回位置的地址。因此程序始终只使用固定数量的返回堆栈层。

不过，`COLA` 并非只用于这种情形；在其他情况下它也非常有用。假设在您的项目中，经常需要执行一种 **case**（见第 20.3 节），判断某个实数是否等于 3。与其每次都重复写 “`%3 % = case`”，不如编写一个程序完成这项操作（类似内置命令字 `%1=case`）。

第一次尝试可能会写成下面这个程序：

```

:: %3 % = case ;

```

然而，这样并不能工作。原因在于，`case` 从运行流，也就是当前执行的程序中取得参数，而不是从调用它的复合程序中取得参数。这意味着 `case` 的参数是 `;`，并非我们想要的对象。但有一个解决办法：在 `case` 前使用 `COLA`。`COLA` 会在执行其后的命令后丢弃运行流的剩余部分，从某种意义上把当前命令与调用它的复合程序合并起来。因此，如果在 `case` 前添加 `COLA`，并将这个新子程序嵌入另一个程序，如下所示：

```

:: ... :: %3 % = COLA case ; <act_T> <act_F> ...

```

其效果就像下面的代码：

```

:: ... %3 % = case <act_T> <act_F> ...

```

这正是我们所需要的。因此，定义这个子程序的正确方式是在 `case` 前使用 `COLA`。这是一个常用组合，所以有一条快捷命令 `COLAcase`，等价于依次执行 `COLA` 和 `case`。还有其他类似的命令字，参见下文的参考部分。

下一个示例使用一种将在第 22 章介绍的错误捕获结构，它是 Wolfgang Rutenberg 编写的 OT49 库（见第 A.3.1 节）中的命令 **D**。该命令的用法如下：

```

« D ... »

```

也就是说，它通常用作程序中的第一条命令（当然，该程序也可以是系统 **RPL** 程序，而不只是用户 **RPL** 程序）。它使程序在关闭屏幕显示的状态下执行，因此速度会略快一些。从 **D** 之后直到二级程序末尾的所有内容都会“盲目”执行。运行 **D** 时，它会关闭屏幕显示；二级程序执行结束时，屏幕显示会重新开启。但如果程序结束后无须专门调用任何内容，怎样才能做到这一点呢？答案很简单：操纵返回堆栈。下面是该命令的反汇编结果：

```

1 ::
  Code
  R>

  ERRSET
5 COMPEVAL
  ERRTRAP
  ::
    'REVAL
  ERRJMP
10 ;
  Code
;

```

第一个代码对象关闭屏幕显示。它是一小段简单的机器语言代码，但说明它超出了本书的范围。随后，`R>` 把调用 **D** 的复合程序余部送入数据堆栈，由 `COMPEVAL` 求值。该程序唯一的难点是，即使程序发生错误，也必须重新开启屏幕显示。如果发生错误，就会执行 `ERRTRAP` 后面的对象。首先，`'REVAL` 将当前复合程序之后的第一个对象送入数据堆栈并执行它；这个对象恰好是第二个代码对象，用于开启屏幕显示。随后用 `ERRJMP` 再次触发错误。如果没有错误，则执行会直接进入第二个代码对象，并结束程序。

作为处理返回堆栈的另一个示例，我们来研究同样来自 OT49 库的列表处理程序 `DoL`。该命令要求堆栈第 2 层中有一个列表，堆栈第 1 层中有任意对象（通常是命令或函数）。它会针对列表中的每个元素依次对该对象求值，并把结果收集到另一个列表中，然后返回这个新列表。该程序使用了一些尚未学过的内容，例如循环和虚拟堆栈。您可以暂时跳过此示例，稍后再回来阅读。下面给出省略参数检查部分后的代码：

```
1 ::
  OVER
  >R (将列表元素压入返回堆栈)
  ticR (尝试取得第一个元素)
5 NOTcaseDROP (若列表为空, 则丢弃该对象)
  PushVStack&Clear (保存当前堆栈)
  BINT0
  GetElemBotVStack (取得列表的第一个元素)
  BEGIN
10 BINT1
  GetElemBotVStack
  xEVAL (取得对象并求值)
  RSWAP

  ticR (取得列表中的下一个元素)
15 WHILE (仍有元素时重复)
  RSWAP
  REPEAT
  DEPTH
  {}N (收集结果)
20 PopVStackAbove (取得已保存的堆栈)
  4UNROLL3DROP (丢弃参数和第一个对象)
;
```

这个程序乍看可能有些难以理解, 但它对返回堆栈的操纵非常巧妙。

程序首先使用 >R, 把一个指向列表内容的指针插入返回堆栈。如果后来不移除这些对象, 那么程序结束后, 列表中的每个对象都会被求值。

然后使用 ticR 取得列表中的第一个对象。这也会推进返回堆栈中的指针, 使其指向第二个元素。如果列表为空, ticR 就返回 FALSE。此时会丢弃待求值的对象, 并将空列表保留为程序的结果。

至少有一个元素时, 真正有趣的部分就开始了。整个堆栈被保存为一个虚拟堆栈层, 而此前由 ticR 取得的列表第一个元素则被提取到“新”堆栈中。

随后开始一个循环。这里使用的循环与用户 RPL 的 WHILE...REPEAT...END 循环非常相似, 详见第 21 章。程序提取待求值的对象并对其求值, 然后再用 ticR 取得列表中的下一个元素。不过, 由于命令字 BEGIN 向返回堆栈压入了某些内容 (说明见第 21.1.1 节), 因此必须使用 RSWAP, 把指向列表元素的指针重新放到返回堆栈第 1 层, 使 ticR 能够取得其中一个元素。如果确实取得了元素, 就再次执行 RSWAP, 将返回堆栈恢复到原来的状态, 然后循环重新开始, 继续对对象求值, 如此反复。没有更多元素时, 控制会转到命令字 REPEAT 之后。所有结果被收集到一个列表中, 然后在结果列表上方恢复先前保存的堆栈。最后, 程序只需丢弃原始列表、待求值的对象以及列表的第一个对象; 将堆栈压入虚拟堆栈时, 这三个对象都在堆栈中。

如您所见, 该程序把返回堆栈用作存储位置; 压入其中的复合程序从未执行, 因为其中的元素被逐个移除, 直到没有任何内容可供执行。

19.4 参考

地址	名称	说明
06E8E	NOP	(→) 什么也不做。
06EEB	'R	(→ ob) 将返回堆栈中的下一个对象 (即位于当前复合程序上一层的复合程序中的第一个对象) 压入堆栈并跳过它。如果返回堆栈顶层为空 (含 SEMI), 则压入一个空二级程序, 且不推进指针。
06F66	'REVAL	(→ ?) 先执行 'R, 再执行 EVAL。
36A27	'R'R	(→ ob1 ob2) 执行两次 'R。
34BEF	ticR	(→ ob T) (→ F) 将返回堆栈中的下一个对象与 TRUE 压入堆栈; 如果返回堆栈顶层的主体为空, 则只压入 FALSE。在后一种情况下, 该层会被丢弃。
36A4A	'RRDROP	(→ ob) 先执行 'R, 再执行 RDROP。
06F9F	>R	(:: →) 将:: 压到返回堆栈顶层 (跳过前导码, 即该复合程序会自动执行)。
0701F	R>	(→ ::) 根据返回堆栈顶层所指流的主体创建一个二级程序, 将该层弹出, 并把二级程序压入堆栈。
07012	R@	(→ ::) 与 R> 类似, 但不会弹出返回堆栈。
0716B	IDUP	(→) 将顶层主体压入返回堆栈。
06F8E	EVAL	(ob → ?) 对对象求值。

地址	名称	说明
262FB	COMPEVAL	(comp → ?) EVAL 只会把列表原样压回堆栈，而此命令会执行该列表。
34BAB	2@REVAL	(→ ?) 对上两层流中的第一个对象求值。
地址	名称	说明
34BBB	3@REVAL	(→ ?) 对上三层流中的第一个对象求值。
34A31	GOTO	(→) 跳转到当前流中的下一个地址。该地址是一个五半字节地址，而不是系统二进制整数。它只能用于跳转到程序中部，不能跳转到程序前导码。
34A46	?GOTO	(flag →) 如果为 TRUE，则跳转；否则跳过五个半字节。
34A59	NOT?GOTO	(flag →) 如果为 FALSE，则跳转；否则跳过五个半字节。
26111	RDUP	(→) 复制返回堆栈顶层。
06FB7	RDROP	(→) 弹出返回堆栈。
343E1	2RDROP	(→) 弹出两个返回堆栈层。
343F3	3RDROP	(→) 弹出三个返回堆栈层。
36342	DROPRDROP	(ob →) 先执行 DROP，再执行 RDROP。
3597F	RDROPCOLA	(→) 先执行 RDROP，再执行 COLA。
34144	RSWAP	(→) 在返回堆栈中执行交换。
368C9	RSKIP	(→) 跳过返回堆栈中的第一个对象（即位于当前复合程序上一层的复合程序中的第一个对象）。
2B8BE	(OBJ>R)	(ob →) 将一个对象压入返回堆栈，例如用于临时存储。如果 ob 是列表，则将整个列表放到流中，而不是分别放入其中的各个元素。
2B8E6	(R>OBJ)	(→ ob) 从返回堆栈取得一个对象。
0312B	SEMI	(→) 对当前流的剩余部分执行 DROP。

19.4.1 引用对象

地址	名称	说明
06E97	'	(→ nob (nextob)) 将当前流中的下一个对象压入堆栈并跳过它。
3696E	DUP'	(ob → ob nob) 先执行 DUP，再执行'。
36996	DROP'	(ob → nob) 先执行 DROP，再执行'。
36982	SWAP'	(ob1 ob2 → ob2 ob1 nob) 先执行 SWAP，再执行'。
369AA	OVER'	(ob1 ob2 → ob1 ob2 ob1 nob) 先执行 OVER，再执行'。
369BE	STO'	(ob id/lam → nob) 先执行 STO，再执行'。
369D2	TRUE'	(→ T nob) 将 TRUE 和下一个对象压入堆栈。
369FF	FALSE'	(→ F nob) 将 FALSE 和下一个对象压入堆栈。
369E6	ONEFALSE'	(→ #1 F nob) 将 ONE、FALSE 和下一个对象压入堆栈。
36A13	#1+'	(# → #+1 nob) 先执行 #1+，再执行'。
36306	'NOP	(→ NOP) 将 NOP 压入堆栈。
3619E	'ERRJMP	(→ ERRJMP) 将 ERRJMP 压入堆栈。
2B90B	'DROPFALSE	(→ DROPFALSE) 将 DROPFALSE 压入堆栈。
25E6A	'DoBadKey	(→ DoBadKey) 将 DoBadKey 压入堆栈。
25E6B	'DoBadKeyT	(→ DoBadKey T) 将 DoBadKey 和 TRUE 压入堆栈。
2F32E	DROPDEADTRUE	(ob → DoBadKey T) 让用户倒地毙命，然后压入 TRUE。
36BBE	('x*)	(→ x*) 将 x*（用户 RPL 命令字 *）压入堆栈。
地址	名称	说明
36BD2	'xDER	(→ xDER) 将 xDER（用户 RPL 命令字 0）压入堆栈。
27B43	'IDFUNCTION	(→ xFUNCTION) 将 xFUNCTION（用户 RPL 命令字 FUNCTION）压入堆栈。

地址	名称	说明
27B6B	'IDPOLAR	($\rightarrow$ xPOLAR) 将 xPOLAR (用户 RPL 命令字 POLAR) 压入堆栈。
27B7F	'IDPARAMETER	($\rightarrow$ xPARAMETRIC) 将 xPARAMETRIC (用户 RPL 命令字 PARAMETRIC) 压入堆栈。
29ED0	'Rapndit	(meta obl...ob4 $\rightarrow$ meta&ob obl...ob4) 从运行流取得 ob, 并将其追加到从堆栈第 5 层开始的 meta。
36AA4	'xDEREQ	(ob $\rightarrow$ flag) ob 是否等于用户 RPL 命令字 xDER?

19.4.2 跳过对象

地址	名称	说明
06FD1	COLA	执行下一个 obj, 并丢弃当前流的剩余部分。
36A63	ONECOLA	先执行 ONE, 再执行 COLA。
3635B	SWAPCOLA	先执行 SWAP, 再执行 COLA。
3636F	XYZ>ZCOLA	先执行 UNROT2DROP, 再执行 COLA。
34AD3	COLA_EVAL	返回到前一个流, 并对其中的第一个 obj 求值。
35994	COLACOLA	丢弃当前流的剩余部分, 并在上一层流中执行 COLA。
0714D	SKIP	跳过运行流中的 1 个 obj。
35715	skipcola	先执行 SKIP, 再执行 COLA。
3570C	2skipcola	先执行 2SKIP, 再执行 COLA。
35703	3skipcola	先执行 3SKIP, 再执行 COLA。
356D5	5skipcola	跳过 5 个对象, 再执行 COLA。
363FB	COLASKIP	丢弃当前流的剩余部分, 并跳过上一层流中的一个 obj。

第 20 章 条件结构

系统 RPL 中的条件结构与用户 RPL 略有不同。第一个区别是，在用户 RPL 中，“假”由实数零表示，任何其他值都表示“真”。在系统 RPL 中，“假”由命令字 FALSE 表示，而“真”由命令字 TRUE 表示（令人惊讶！）。执行这些命令字时，它们只会将自身压入堆栈。所有进行测试的命令都会返回其中之一。IT 或 case 等命令字则以其中之一作为参数。

必要时，可以用 COERCEFLAG 将 TRUE 或 FALSE 转换为（实数）0 或 1。没有像 UNCOERCEFLAG 这样专门执行反向转换的函数，但%0<> 完全可以完成这项工作。

有许多命令会将 TRUE、FALSE 或它们的某种组合压入堆栈，参见下表。

系统 RPL 也提供布尔运算符 NOT、AND、OR 和 XOR，并提供了若干组合形式，列表见下文。

20.1 测试

测试命令字接受一个或多个参数，对参数进行某种比较后返回 TRUE 或 FALSE。针对每种对象类型的测试列在相应对象类型章节的参考部分中。对象类型测试见第 29 章。其他类型的测试列在下文的参考部分中。

这些测试中最重要的是 EQ 和 EQUAL。两者都接受两个对象并返回一个标志。EQ 检查对象是否相同，也就是是否占用相同的内存地址；EQUAL 则检查对象的内容是否相等。区别在于，:: BINT2 # 2 EQUAL ; 返回 TRUE；但若把 EQUAL 替换为 EQ，该程序就返回 FALSE，因为一个对象是位于地址 #3311B 的内置二进制整数（bint）2，另一个则是地址不可预测的 bint，但可以肯定它不在 ROM 中。

再举一例：如果您把一个字符串放入堆栈第 1 层并按 ENTER，EQ

和 EQUAL 都会返回 TRUE。但是，如果您输入一个字符串，随后再次输入完全相同的字符串，则只有 EQUAL 会返回 TRUE。这是因为两个字符串的内容相同，但它们是内存中的不同对象，各自占据不同的内存地址；它们只是恰好具有相同内容。在可行的情况下，应在程序中使用 EQ，因为它比 EQUAL 更快。

20.2 If...Then...Else

多数情况下，您会使用 IT 和 ITE 命令创建这类条件结构：

命令字	堆栈与操作
IT	(flag →) 如果标志为 TRUE，则执行下一个对象；否则跳过该对象。
ITE	(flag →) 如果标志为 TRUE，则执行下一个对象并跳过第二个对象；如果为 FALSE，则跳过一个对象并执行第二个对象。

以下代码片段将零变为一，但不对其他数进行任何操作：

```
... DUP %0= IT %1+ ...
```

如果两个对象相等，以下代码将输出"Equal"；否则输出"Not equal"：

```
... EQUAL $ "Equal" $ "Not equal" ...
```

当然，需要执行多个命令时，必须将它们包含在一个二级程序中。

20.3 Case

CASE 命令字是 IT、SKIP 和 COLA 的组合（见第 19 章）。基本命令字是 case，此外还有将其与测试和其他命令组合起来的各种命令字。

case 从堆栈第 1 层取得一个标志。如果该标志为 TRUE，则执行下一个对象

但只执行这一个对象：运行流的其余部分会被丢弃。因此，TRUE case 等同于 COLA。如果标志为 FALSE，则跳过下一个对象，并从该对象之后继续执行。因此，FALSE case 等同于 SKIP。

以下示例说明如何构造一种常见的 case 结构，类似于其他语言（甚至用户 RPL）中的结构。它会输出一个表示堆栈第 1 层中 bint 的字符串。

```
1 ::  
  DUP #0=      case $ "Zero"  
  DUP BINT1 #= case $ "One"  
  DUP BINT2 #= case $ "Two"  
5  ...  
  ;
```

有许多命令字会将 `case` 与其他命令字组合在一起，`OVER#=case` 就是其中之一。它的作用不难推断：先执行 `OVER`，然后由 `#=` 比较两个 `bint`，最后 `case` 如前所述执行。使用这个命令字，上述代码可改写为：

```
1 ::
    BINT0 OVER#=case $ "Zero"
    BINT1 OVER#=case $ "One"
    BINT2 OVER#=case $ "Two"
5  ...
    ;
```

在下面的参考部分中，您将看到一份命令字列表，这些命令字除执行 `case` 外，还执行其他操作。这些命令字由前置部分、`case` 本身和后置部分组成。有些除 `case` 外只有前置部分或后置部分，有些则两者皆有。前置部分表示在 `case` 之前执行的命令，其作用通常很容易理解；例如，`NOTcase` 等同于先执行 `NOT`，再执行 `case`。后置部分则较为复杂，因为有两类后置部分。第一类后置部分以大写字母书写。当测试为 `TRUE` 时，将执行这类后置部分中的命令。您只需提供 `FALSE` 情况下的操作。例如，以下片段

```
... caseDROP <FalseAction> ...
```

等同于

```
... case DROP <FalseAction> ...
```

第二类由后置部分以小写字母书写的命令字组成。在这种情况下，当测试为 `TRUE` 时，后置部分中的命令会与 `case` 后面的对象一起执行。换言之，以下片段

```
... casedrop <TrueAction> <FalseAction> ...
```

等同于

```
... case :: DROP <TrueAction> ; <FalseAction> ...
```

遗憾的是，某些词条的命名有误，没有遵循此约定。这些词条已在下方说明中明确标出。

此外，下面大多数命令字的“堆栈图”并不是真正的堆栈图。箭头左侧照例表示调用该词条之前的堆栈内容。`ob1` 和 `ob2` 表示不同的对象。`f1` 和 `f2` 表示不同的标志；`T` 表示 `TRUE`，`F` 表示 `FALSE`。`#m` 和 `#n` 是满足 `#m` 小于 `#n` 的两个二进制整数。`#set` 表示某个标志的编号，且该标志已设置；`#clr` 表示某个标志的编号，且该标志已清除。箭头右侧表示当堆栈与箭头左侧匹配时将执行的对象。初始运行流的形式为：

```
:: <test_word> <ob1> ... <obn> ;
```

在这些图中，`<rest>` 表示位于 `<rest>` 前一个对象之后的所有对象。箭头右侧还会出现不带尖括号的对象。这些是该命令字运行后留在数据堆栈中的对象，而不是运行流中的对象。

20.4 参考

20.4.1 布尔标志

地址	名称	说明
2602B	COERCEFLAG	(T → %1) (F → %0) 将系统标志转换为用户标志，并丢弃当前运行流。
301BA	%0<>	(% → flag) 可用于将用户标志转换为系统标志。
03A81	TRUE	(→ T)
27E87	TrueTrue	(→ T T)
地址	名称	说明
36540	TrueFalse	(→ T F) 又名：TRUEFALSE
03AC0	FALSE	(→ F)
36554	FalseTrue	(→ F T) 又名：FALSETRUE
283E8	FalseFalse	(→ F F)

地址	名称	说明
27E9B	failed	(→ F T)
35280	DROPTTRUE	(ob → T)
2D7006	^2DROPTTRUE	(ob ob' → T)
35289	DROPFALSE	(ob → F)
35B32	2DROPFALSE	(ob1 ob2 → F)
28211	NDROPFALSE	(ob1..obn #n → F)
2812F	SWAPTRUE	(ob1 ob2 → ob2 ob1 T)
374BE	SWAPDROPTTRUE	(ob1 ob2 → ob2 T)
35EF2	XYZ>ZTRUE	(ob1 ob2 ob3 → ob3 T)
2962A	RDROPFALSE	(→ F) 将 FALSE 放入堆栈，并丢弃当前运行流的其余部分。
03AF2	NOT	(flag → flag') 若输入为 TRUE，则返回 FALSE；反之亦然。
03B46	AND	(flag1 flag2 → flag) 仅当两个标志均为 TRUE 时返回 TRUE。
03B75	OR	(flag1 flag2 → flag) 任一标志为 TRUE 时返回 TRUE。
03ADA	XOR	(flag1 flag2 → flag) 两个标志不同时返回 TRUE。
365F9	ORNOT	(flag1 flag2 → flag) 任一标志为 TRUE 时返回 FALSE。
35C7C	NOTAND	(flag1 flag2 → flag) 当 flag1 为 TRUE 且 flag2 为 FALSE 时返回 TRUE。
35CB8	ROTAND	(flag1 ob flag2 → ob flag) 任一标志为 TRUE 时返回 TRUE。

20.4.2 一般测试

地址	名称	说明
03B2E	EQ	(ob1 ob2 → flag) 如果两个对象是同一对象，即在内存中占据相同的物理空间，则返回 TRUE。只比较对象的地址。
36621	2DUPEQ	(ob1 ob2 → ob1 ob2 flag) 先执行 2DUP，再执行 EQ。
3664E	EQOR	(flag ob1 ob2 → flag') 先执行 EQ，再执行 OR。
3607F	EQOVER	(ob3 ob1 ob2 → ob3 flag ob3) 先执行 EQ，再执行 OVER。
3663A	EQ:	(ob → flag) 对当前运行流中的下一个对象执行 EQ。
36635	DUPEQ:	(ob → ob flag) 先执行 DUP，再执行 EQ:。
03B97	EQUAL	(ob1 ob2 → flag) 如果两个对象相等（但不一定是同一对象），即它们的前导码和内容均相同，则返回 TRUE。
3660D	EQUALNOT	(ob1 ob2 → flag) 如果两个对象不同，则返回 TRUE。
36662	EQUALOR	(flag ob1 ob2 → flag') 先执行 EQUAL，再执行 OR。
0FF006	^Contains?	(ob1 ob2 → ob1 ob2 flag) 测试 ob1 是否包含 ob2。如果 ob1 是符号对象，则在 ob1 中搜索嵌入的 ob2；如果 ob1 是列表，则遍历 ob1 以查找直接匹配项；否则，测试 ob1 与 ob2 是否相等。

20.4.3 TRUE/FALSE 测试

地址	名称	说明
34AA1	?SEMI	(T → :: ;) (F → :: <ob1> <rest> ;)
34A92	NOT?SEMI	(T → :: <ob1> <rest> ;) (F → :: ;)
3692D	?SEMIDROP	(ob T → :: ob ;) (ob F → :: <ob1> <rest> ;)
34BD8	NOT?DROP	(ob T → :: ob <ob1> <rest> ;) (ob F → :: <ob1> <rest> ;)
35F56	?SWAP	(ob1 ob2 T → :: ob2 ob1 <ob1> <rest> ;) (ob1 ob2 F → :: ob1 ob2 <ob1> <rest> ;)

地址	名称	说明
35DDA	?SKIPSWAP	(ob1 ob2 T → :: ob1 ob2 <ob1> <rest>;) (ob1 ob2 F → :: ob2 ob1 <ob1> <rest>;)
35F97	?SWAPDROP	(ob1 ob2 T → :: ob1 <ob1> <rest>;) (ob1 ob2 F → :: ob2 <ob1> <rest>;)
35F7E	NOT?SWAPDROP	(ob1 ob2 T → :: ob2 <ob1> <rest>;) (ob1 ob2 F → :: ob1 <ob1> <rest>;)
070FD	RPIT	(T ob → :: ob <ob1> <rest>;) (F ob → :: <ob1> <rest>;) ob 实际上会被执行，而不是被压入堆栈。
070C3	RPITE	(T ob1 ob2 → :: ob1 <ob1> <rest>;) (F ob1 ob2 → ob2 <ob1> <rest>;) ob1 或 ob2 实际上会被执行，而不是被压入堆栈。
34AF4	COLARPITE	(T ob1 ob2 → :: ob1 ;) (F ob1 ob2 → :: ob2 ;) ob1 或 ob2 实际上会被执行，而不是被压入堆栈。
34B4F	2'RCOLARPITE	返回复合程序，并在其中执行 ITE。
34A22	IT	(T → :: <ob1> <rest>;) (F → :: <ob2> <rest>;)
0712A	?SKIP	(T → :: <ob2> <rest>;) (F → :: <ob1> <rest>;) 又名: NOT_IT
地址	名称	说明
34B3E	ITE	(T → :: <ob1> <ob3> <rest>;) (F → :: <ob2> <rest>;)
36865	COLAITE	(T → :: <ob1>;) (F → :: <ob2>;)
34ABE	ITE_DROP	(ob T → :: <ob2> <rest>;) (ob F → :: ob <ob1> <rest>;)
36EED	ANDITE	(f1 f2 → :: <ob1> <ob3> <rest>;) (f1 f2 → :: <ob2> <rest>;)
349F9	case	(T → :: <ob1>;) (F → :: <ob2> <rest>;)
34A13	NOTcase	(T → :: <ob2> <rest>;) (F → :: <ob1>;)
36D4E	ANDcase	(f1 f2 → :: <ob1>;) (f1 f2 → :: <ob2> <rest>;)
36E6B	ANDNOTcase	(f1 f2 → :: <ob1>;) (f1 f2 → :: <ob2> <rest>;)
359E3	ORcase	(f1 f2 → :: <ob1>;) (f1 f2 → :: <ob2> <rest>;)
3495D	casedrop	(ob T → :: <ob1>;) (ob F → :: ob <ob2> <rest>;)
3494E	NOTcasedrop	(ob T → :: ob <ob2> <rest>;) (ob F → :: <ob1>;)
34985	case2drop	(ob1 ob2 T → :: <ob1>;) (ob1 ob2 F → :: ob1 ob2 <ob2> <rest>;)
34976	NOTcase2drop	(ob1 ob2 T → :: ob1 ob2 <ob2> <rest>;) (ob1 ob2 F → :: <ob1>;)
349B1	caseDROP	(ob T → :: ;) (ob F → :: ob <ob1> <rest>;)
349C6	NOTcaseDROP	(ob T → :: ob <ob1> <rest>;) (ob F → :: ;)
368FB	casedrptru	(ob T → T) (ob F → :: ob <ob1> <rest>;) 注: 应命名为 caseDRPTRU。
365B3	casedrpfls	(ob T → F) (ob F → :: ob <ob1> <rest>;) 注: 应命名为 caseDRPFLLS。
地址	名称	说明
36B3A	NOTcsdrpfls	(ob T → :: ob <ob1> <rest>;) (ob F → F) 注: 应命名为 NOTcaseDRPFLLS。
349D6	case2DROP	(ob1 ob2 T → :: ;) (ob1 ob2 F → :: ob1 ob2 <ob1> <rest>;)
349EA	NOTcase2DROP	(ob1 ob2 T → :: ob1 ob2 <ob1> <rest>;) (ob1 ob2 F → :: ;)
365CC	case2drpfls	(ob1 ob2 T → F) (ob1 ob2 F → :: ob1 ob2 <ob1> <rest>;) 注: 应命名为 case2DRPFLLS。
3652C	caseTRUE	(T → T) (F → :: <ob1> <rest>;)
36914	NOTcaseTRUE	(T → :: <ob1> <rest>;) (F → T)
365E5	caseFALSE	(T → F) (F → :: <ob1> <rest>;)

地址	名称	说明
2B2C5	NOTcaseFALSE	(T → :: <obl> <rest>;) (F → F)
359AD	COLAcase	(T → :: <obl>;) (F → :: <ob2> <rest>;) 丢弃当前运行流的其余部分，并在上一层运行流中执行 case。
359C8	COLANOTcase	(T → :: <ob2> <rest>;) (F → :: <obl>;) 丢弃当前运行流的其余部分，并在上一层运行流中执行 NOTcase。

20.4.4 二进制整数测试

地址	名称	说明
363B5	#=?SKIP	(#m #n → :: <ob2> <rest>;) (#m #n → :: <obl> <rest>;)
363E2	#>?SKIP	(#m #n → :: <obl> <rest>;) (#m #n → :: <ob2> <rest>;)
地址	名称	说明
35C54	#=ITE	(#m #n → :: <obl> <ob3> <rest>;) (#m #n → :: <ob2> <rest>;)
36F29	#<ITE	(#m #n → :: <obl> <ob3> <rest>;) (#m #n → :: <ob2> <rest>;)
36F3D	#>ITE	(#m #n → :: <ob2> <rest>;) (#m #n → :: <obl> <ob3> <rest>;)
348D2	#=case	(#m #n → :: <obl>;) (#m #n → :: <ob2> <rest>;)
348E2	OVER#=case	(#m #n → :: #m <obl>;) (#m #n → :: #m <ob2> <rest>;)
34939	#=casedrop	(#m #n → :: <obl>;) (#m #n → :: #m <ob2> <rest>;) 注：应命名为 OVER#=casedrop。
36590	#=casedrpfls	(#m #n → F) (#m #n → :: #m <obl> <rest>;) 注：应命名为 OVER#=caseDRPFLS。
36D9E	#<>case	(#m #n → :: <ob2> <rest>;) (#m #n → :: <obl>;)
36D76	#<case	(#m #n → :: <obl>;) (#m #n → :: <ob2> <rest>;)
36DCB	#>case	(#m #n → :: <ob2> <rest>;) (#m #n → :: <obl>;)
34A7E	#0=?SEMI	(#0 → :: ;) (# → :: <obl> <rest>;)
36383	#0=?SKIP	(#0 → :: <ob2> <rest>;) (# → :: <obl> <rest>;)
36F15	#0=ITE	(#0 → :: <obl> <ob3> <rest>;) (# → :: <ob2> <rest>)
36ED4	DUP#0=IT	(#0 → :: #0 <obl> <rest>;) (# → :: # <ob2> <rest>;)
36F51	DUP#0=ITE	(#0 → :: #0 <obl> <ob3> <rest>;) (# → :: # <ob2> <rest>;)
348FC	#0=case	(#0 → :: <obl>;) (# → :: <ob2> <rest>;)
348F7	DUP#0=case	(#0 → :: #0 <obl>;) (# → :: # <ob2> <rest>;)
3490E	DUP#0=csedrp	(#0 → :: <obl>;) (# → :: # <ob2> <rest>;)
地址	名称	说明
36D21	DUP#0=csDROP	(#0 → :: ;) (# → :: # <obl> <rest>;)
36D8A	#1=case	(#1 → :: <obl>;) (# → :: <ob2> <rest>;)
3639C	#1=?SKIP	(#1 → :: <ob2> <rest>;) (# → :: <obl> <rest>;)
36DB2	#>2case	(#0/#1/#2 → :: <ob2> <rest>;) (# → :: <obl>;)
25E72	?CaseKeyDef	(# #' → :: ' obl T ;) (# #' → :: <ob2> <rest>;) 比较两个 bint。若相等，则引用运行流中的下一个对象，并将该对象与 TRUE 一同返回。
25E73	?CaseRomptr@	(# #' → ob T) (# #' → F) (# #' → :: <ob2> <rest>;) 比较两个 bint。若相等，则尝试解析运行流中的下一个对象；该对象必须是 ROM 指针。ROMPTR@ 成功时会压入 TRUE，因此该词条可直接用于按键处理程序。

20.4.5 实数与复数测试

地址	名称	说明
2B149	%0=case	(%0 → :: %0 <ob1>;) (ob → :: ob <ob2> <rest>;)
36DDF	j%0=case	(%0 → :: <ob1>;) (ob → :: <ob2> <rest>;)
2B15D	C%0=case	(C%0 → :: C%0 <ob1>;) (ob → :: ob <ob2> <rest>;)
2B11C	num0=case	(0 → :: 0 <ob1>;) (ob → :: ob <ob2> <rest>;) 对于此测试, 实数 零和复数零都会构成 TRUE 条件。
2B1A3	%1=case	(%1 → :: %1 <ob1>;) (ob → :: ob <ob2> <rest>;)
地址	名称	说明
2B1C1	C%1=case	(C%1 → :: C%1 <ob1>;) (ob → :: ob <ob2> <rest>;)
2B176	num1=case	(1 → :: 1 <ob1>;) (ob → :: ob <ob2> <rest>;) 对于此测试, 实数 一和复数一都会构成 TRUE 条件。
2B20C	%2=case	(%2 → :: %2 <ob1>;) (ob → :: ob <ob2> <rest>;)
2B22A	C%2=case	(C%2 → :: C%2 <ob1>;) (ob → :: ob <ob2> <rest>;)
2B1DF	num2=case	(2 → :: 2 <ob1>;) (ob → :: ob <ob2> <rest>;) 对于此测试, 实数 二和复数二都会构成 TRUE 条件。
2B289	%-1=case	(%-1 → :: %-1 <ob1>;) (ob → :: ob <ob2> <rest>;)
2B2A7	C%-1=case	(C%-1 → :: C%-1 <ob1>;) (ob → ob <ob2> <rest>;)
2B25C	num-1=case	(-1 → :: -1 <ob1>;) (ob → :: ob <ob2> <rest>;) 对于此测试, 实数 -1 和复数 -1 都会构成 TRUE 条件。

20.4.6 元对象测试

地址	名称	说明
2AFFB	MEQ1stcase	(meta&ob1 ob2 → ob1=ob2 ? case) Meta&ob1 ob2 ob1=ob2 ? case
2AF37	AEQ1stcase	(meta&ob → ob=nob ? case) Meta&ob ob=nob ? case
2B01B	MEQopscase	(meta1&ob1 meta2&ob2 ob3 →) Meta1&ob1 Meta2&ob2 ob3
2B06A	AEQopscase	meta1&ob1 meta2&ob2 Meta1&ob1 Meta2&ob2
2B083	Mid1stcase	(meta&ob → ob is id) lam ? case Meta&ob ob is id or lam ? case
地址	名称	说明
2AE32	M-1stcasechs	(Meta&NEG → Meta COLA) (Meta → Meta SKIP) (Meta&(%<0) → Meta&ABS(%) COLA) Meta&NEG Meta COLA ; Meta Meta SKIP Meta&(%<0) Meta&ABS(%) COLA

20.4.7 一般对象测试

地址	名称	说明
36EBB	EQIT	(ob1 ob1 → :: <ob1> <rest>;) (ob1 ob2 → :: <ob2> <rest>;)
36F01	EQITE	(ob1 ob1 → :: <ob1> <ob3> <rest>;) (ob1 ob2 → :: <ob2> <rest>;)
36D3A	jEQcase	(ob1 ob1 → :: <ob1>;) (ob1 ob2 → :: <ob2> <rest>;)
34999	EQcase	(ob1 ob1 → :: ob1 <ob1>;) (ob1 ob2 → :: ob1 <ob2> <rest>;) 注: 应命名 为 OVEREQcase。
359F7	REQcase	(ob → :: ob <ob2>;) (ob → :: ob <ob3> <rest>;) 使用运行流中的下 一个对象执行 EQcase。
34920	EQcasedrop	(ob1 ob1 → :: <ob1>;) (ob1 ob2 → :: ob1 <ob2> <rest>;) 注: 应命名 为 OVEREQcasedrop。

地址	名称	说明
35A10	REQcasedrop	(ob → <ob2>;) (ob → <ob3> <rest>;) 使用运行流中的下一个对象执行 EQcasedrop。
36D62	EQUALcase	(ob1 ob1 → :: <ob1>;) (ob1 ob2 → :: <ob2> <rest>;)
36E7F	EQUALNOTcase	(ob1 ob1 → :: <ob2> <rest>;) (ob1 ob2 → :: <ob1>;)
36D08	EQUALcasedrp	(ob ob1 ob2 → :: <ob1>;) (ob ob1 ob2 → :: ob <ob2> <rest>;)
地址	名称	说明
2AD81	EQUALcasedrop	(ob1 ob2 → :: <ob1>;) (ob1 ob2 → :: ob1 <ob2> <rest>;)
29E99	tok=casedrop	(\$ \$' → :: <ob1>;) (\$ \$' → :: \$ <ob2> <rest>;) 注：应命名为 OVERTok=casedrop。
2ADBD	nonopcase	(seco → :: seco <ob2> <rest>;) (ob → :: ob <ob1>;)
2B0CC	idntcase	(id → :: id <ob1>;) (ob → :: ob <ob2> <rest>;)
36E93	dIDNTNcase	(id → :: id <ob2> <rest>;) (ob → :: ob <ob1>;)
2B0EF	idntlamcase	(id/lam → :: id <ob1>;) (ob → :: ob <ob2> <rest>;)
36DF3	REALcase	(% → :: <ob1>;) (ob → :: <ob2> <rest>;)
36EA7	dREALNcase	(% → :: % <ob2> <rest>;) (ob → :: ob <ob1>;)
36E07	dARRAYcase	([] → :: [] <ob1>;) (ob → :: ob <ob2> <rest>;)
36E43	dLISTcase	({} → :: {} ob1 ;) (ob → :: ob <ob2> <rest>;)
260C6	NOTLISTcase	({} → :: {} <ob2> <rest>;) (ob → :: ob <ob1>;)
260D0	NOTSECOcase	(seco → :: seco <ob2> <rest>;) (ob → :: ob <ob1>;)
260CB	NOTROMPcase	(romp → :: romp <ob2> <rest>;) (ob → :: ob <ob1>;)
2ADE0	numblstcase	(%/C%/[]/[L] → :: <ob1>;) (ob → :: ob2 <rest>;) 如果为%、C%、[]或[L]，则执行 COLA；否则执行 SKIP。

20.4.8 杂项

地址	名称	说明
36F65	UserITE	(#set → :: <ob1> <ob3> <rest>;) (#clr → :: <ob2> <rest>;)
36F79	SysITE	(#set → :: <ob1> <ob3> <rest>;) (#clr → :: <ob2> <rest>;)
36C4F	caseDoBadKey	(T → :: DoBadKey ;) (F → :: <ob1> <rest>;) 又名：caseDEADKEY
36C36	caseDrpBadKy	(ob T → :: DoBadKey ;) (ob F → :: ob <ob1> <rest>;)
361B2	caseERRJMP	(T → :: ERRJMP ;) (F → :: <ob> <rest>;)
36B53	caseSIZEERR	(T → :: SIZEERR ;) (F → :: <ob> <rest>;)
36B67	NcaseSIZEERR	(T → :: <ob> <rest>;) (F → :: SIZEERR ;)
36BAA	NcaseTYPEERR	(T → :: <ob1> <rest>;) (F → :: TYPEERR ;)
25EEE	NoEdit?case	(→ :: <ob1> <rest>;) (→ :: <rest>;) 测试当前是否没有活动的编辑行。
36E57	EditExstCase	(→ :: <ob1> <rest>;) (→ :: <rest>;) 测试当前是否有活动的编辑行。
2BE36	(ALGcase)	(→ :: <ob1>;) (→ :: <ob2> <rest>;) 测试是否处于代数模式，并执行 case。

第 21 章

循环

与用户 RPL 一样，系统 RPL 中有两类循环：不定次数循环和定次数循环。不定次数循环是指事先不知道会执行多少次的循环；它会不断重复，直到满足特定条件。其创建方式与用户 RPL 的不定次数循环非常相似。另一方面，定次数循环会执行在开始前指定的次数。它的创建方式与用户 RPL 并不完全相同，但使用简单，而且功能更强。例如，您可以在循环运行期间改变其运行次数。

在以下说明中，除非另有注明，<> 之间的元素可以由多个对象组成。

21.1 不定次数循环

在系统 RPL 中，可以用三种方式创建不定次数循环。第一种是 WHILE 循环，其结构如下：

```
1 BEGIN
  <test clause>
  WHILE
    <loop object>
5 REPEAT
```

这种循环先执行测试子句 (<test clause>)；如果测试结果为 TRUE，就执行循环对象 (<loop object>)，然后重新开始循环。如果测试返回 FALSE，则从 REPEAT 之后继续执行。如果第一次测试就返回 FALSE，该循环一次也不会执行。

这种循环要求 <loop object> 是单个对象。多数情况下，该对象会是复合程序。

第二种不定次数循环是 UNTIL 循环，其结构如下：

```
1 BEGIN
  <loop clause>
  UNTIL
```

这种循环始终至少执行一次。命令字 UNTIL 需要一个标志。如果标志为 FALSE，就再次执行循环子句 (<loop clause>)；如果为 TRUE，则从 UNTIL 之后继续执行。

还有第三种不定次数循环：

```
1 BEGIN
  <loop object>
  AGAIN
```

这种循环不进行测试。若要退出该循环，必须触发错误条件，或直接操纵返回堆栈。如果循环代码中有多个不同位置需要决定是重复循环还是退出循环，这种形式会很有用。

21.1.1 不定次数循环的工作原理

不定次数循环由命令字 BEGIN、WHILE、REPEAT、UNTIL 和 AGAIN 组合而成。这些命令字并无特殊之处，与其他命令一样，只是组合后可以构成循环。它们通过操纵运行流和返回堆栈工作，因此请确保您理解这些概念；如有疑问，请参见第 19.1 节。

命令字	堆栈与操作
BEGIN	(→) 将解释器指针复制到返回堆栈。
UNTIL	(flag →) 如果标志为 TRUE，则弹出返回堆栈；否则将解释器指针设为返回堆栈最顶层的地址，但不弹出该层。
WHILE	(flag →) 如果标志为 TRUE，则什么也不做；否则弹出返回堆栈，并跳过运行流中的下两个对象。
REPEAT	(→) 将解释器指针设为返回堆栈最顶层的指针，但不弹出该层。
命令字	堆栈与操作
AGAIN	(→) 将解释器指针设为返回堆栈最顶层的地址，但不弹出该层。

根据以上说明，您应当已经理解这些循环的工作方式，也应明白 BEGIN...WHILE...REPEAT 循环为什么要求 WHILE 和 REPEAT 之间只有一个对象。

21.2 定次数循环

定次数循环由 DO 和 LOOP（或其他等价命令字）创建。DO 从堆栈取得两个 **bint**，分别表示终止值和起始值。起始值存储为当前索引，可以用 INDEX@ 提取；终止值可以用 ISTOP@ 提取。您可以分别用 INDEXSTO 和 ISTOPSTO 存入新的当前索引值和终止值。

与 DO 配套的命令是 LOOP 和 +LOOP。前者将索引值加一，并检查新值是否大于或等于终止值；若是，则退出循环，否则再次执行循环。+LOOP 的工作方式类似，但它将索引增加堆栈第 1 层中的 **bint**。

DO 循环的标准形式为：

```
stop start DO <loop clause> LOOP
```

它会针对从 start 到 stop-1 的每个索引值执行循环子句（<loop clause>）。请注意，终止值比在用户 RPL 中使用的值大，因此务必留意。另外，“stop”值位于“start”值之前。

系统提供了若干与 DO 循环配合使用的命令字，例如 ONE_DO。它们列在下文中。

下面是一个简单循环的示例，它会把 **bint** #1h、#2h、#3h 和 #4h 输出到堆栈：

```
1 ::
  BINT5 BINT1
  DO
    INDEX@
  5 LOOP
;
```

它可以改写为：

```
1 ::
  BINT5 ONE_DO
  INDEX@
  LOOP
  5 ;
```

21.2.1 DO 循环的工作原理

如果您对返回堆栈和运行流等概念已有一定了解（见第 19.1 节），本节将说明 DO 循环的工作原理。

执行命令字 DO 时，它会把解释器指针压入返回堆栈；此时该指针指向 DO 后的第一个对象。DO 还会创建一个 DoLoop 环境，用于存储初始值和终止值。

随后正常继续执行，运行 DO 与 LOOP 之间的所有命令。

执行 LOOP 时，它会递增最近创建的 DoLoop 环境中的当前值。如果该值大于或等于该环境的终止值，就销毁该环境，并从返回堆栈弹出一层。这样会移除指向 DO 后第一个对象的指针，执行则从 LOOP 之后正常继续。如果该值较小，就把解释器指针设为返回堆栈顶层的值，使执行从 DO 后的第一个对象重新开始。

21.3 参考

21.3.1 不定次数循环

地址	名称	说明
0716B	IDUP	(→) 将解释器指针压入返回堆栈。
071A2	BEGIN	(→) 将解释器指针压入返回堆栈。
地址	名称	说明
071AB	AGAIN	(→) 将解释器指针设为返回堆栈最顶层的值，但不弹出该层。
071E5	REPEAT	(→) 将解释器指针设为返回堆栈最顶层的值，但不弹出该层。
071C8	UNTIL	(flag →) 若为 FALSE，则执行 AGAIN；否则执行 RDROP。
3640F	NOT_UNTIL	(flag →) 先执行 NOT，再执行 UNTIL。
35B96	#0=UNTIL	(# → #) 实际上应命名为 DUP#0=UNTIL。
071EE	WHILE	(flag →) 若为 TRUE，则什么也不做；否则先执行 RDROP，再执行 2SKIP。
36428	NOT_WHILE	(flag →) 先执行 NOT，再执行 WHILE。

地址	名称	说明
36441	DUP#0<>WHILE	(# →) 请尝试推断它的作用。

21.3.2 定次数循环

地址	名称	说明
073F7	DO	(#stop #start →)
073C3	ZERO_DO	(#stop →)
364C8	DUP#0_DO	(#stop → #stop)
073CE	ONE_DO	(#stop →)
073DB	#1+_ONE_DO	(#stop →)
364E1	toLEN_DO	({} → {}) 从 ONE 循环到 #elements。
07334	LOOP	(→)
073A5	+LOOP	(# →) 按指定数值递增索引。
364AF	DROPLoop	(ob →)
36496	SWAPLoop	(ob1 ob2 → ob2 ob1)
34AAD	SEMILOOP	(→)
07221	INDEX@	(→ #) 提取最顶层循环计数器的值。
地址	名称	说明
3645A	DUPINDEX@	(ob → ob #)
3646E	SWAPINDEX@	(ob1 ob2 → ob2 ob1 #)
36482	OVERINDEX@	(ob1 ob2 → ob1 ob2 ob1 #)
367D9	INDEX@#-	(# → #')
07270	INDEXSTO	(# →) 存储新的最顶层循环计数器值。
07249	ISTOP@	(→ #) 提取最顶层循环终止值。
07295	ISTOPSTO	(# →) 存储新的最顶层循环终止值。
283FC	ISTOP-INDEX	(→ #)
07258	JINDEX@	(→ #) 提取次顶层循环计数器值。
072AD	JINDEXSTO	(# →) 存储新的次顶层循环计数器值。
07264	JSTOP@	(→ #) 提取次顶层循环终止值。
072C2	JSTOPSTO	(# →) 存储新的次顶层循环终止值。
3709B	ExitAtLoop	(→) 不会立即退出循环，而是把终止值存为零，因此在遇到下一个 LOOP 之前，所有对象仍会被求值。 又名：ZEROISTOPSTO

第 22 章 错误处理

系统 RPL 程序发生错误时，程序通常会中止，并弹出显示错误消息的消息框。不过，有时需要让程序捕获错误，并在可能的情况下继续执行，或者以不同方式表明发生了错误。

另一些时候，程序需要主动生成错误。例如，如果用户向程序提供了无效输入，程序应以 “Invalid Argument Type” 错误中止，而不应冒着使计算器崩溃的风险继续运行。

22.1 捕获错误

您可以使用以下结构截获错误处理子系统的执行，也就是捕获程序生成的错误：

```
1 ::
...
ERRSET
:: <suspect objects> ;
5 ERRTRAP
:: <if-error objects> ;
...
;
```

当然，如果 <suspect objects> 和 / 或 <if-error objects> 只有一个对象，就不必把它们放在二级程序中。

其工作方式如下：如果 <suspect objects> 生成错误，执行便从 <if-error objects> 继续；否则将跳过它，从后面的内容继续执行。

<if-error objects> 的处理方式完全由您决定。通常，它会处理错误，然后继续执行或退出程序。可用 ERROR@ 提取当前错误号，使程序能够针对不同类型的错误采取不同操作。错误消息及其编号见附录 E。

22.1.1 保护字

每个临时环境（见第 18 章）、DO/LOOP 环境（见第 21 章）和虚拟堆栈层（见第 23 章）都有一个保护字。它用于让错误处理子系统区分哪些环境是在设置错误捕获之前创建的，哪些是在之后创建的。这样，一旦发生错误，设置错误捕获之后创建的所有环境都会被删除。例如，请看以下代码：

```
1 ::
...
1LAMBIND
...
5 TEN ZERO_DO
ERRSET ::
...
1LAMBIND
...
10 FIVE ONE_DO
<suspect object is here>
LOOP
ABND
;
15 ERRTRAP ::
<error handling>
;
LOOP
...
20 ABND
;
```

如果生成错误，该错误就会被捕获。借助保护字，内层 DO/LOOP 环境和临时环境都会被删除。

创建这些环境时，其保护字被设为零。ERRSET 会递增三类环境中各自最新环境的保护字。这样，这些环境的保护字就变为非零值。（创建环境时，保护字已初始化为零。）

ERRTRAP 和 ERRJMP 会按从新到旧的顺序删除这些环境，直到在每一类环境中找到一个保护字非零的环境。这样的环境在设置错误捕获之前就已经存在，

因为 ERRSET 已经递增了它们的保护字。这样，设置捕获之后创建的所有环境都会被删除，因为它们的保护字仍为零。ERRTRAP 和 ERRJMP 还会递减首次找到的保护字非零环境的保护字，从而使存在多层嵌套时这一过程也能正确工作。

22.2 生成错误

错误处理子系统由命令字 `ERRJMP` 调用。如果已经设置错误捕获，就执行相应的错误处理程序；如果没有设置，则执行默认错误处理程序。

多数情况下，生成错误后会默认错误处理程序处理。该默认处理程序会发出蜂鸣声（如果已启用此功能），并在消息框中显示错误说明。

显示的消息取决于两项内容：一是错误号，它决定错误消息，例如“`Bad Argument Type`”或“`Too Few Arguments`”；二是最后存储的命令名。

第 29 章介绍的 `CK<n>` 命令字会自动存储最后一个命令名。如该章所述，如果您编写的程序不属于库，就不应存储命令名，否则会显示一个难看的名称。

要定义错误号，请使用命令字 `ERRORSTO`。它需要一个 `bint` 参数，即错误编号。错误列表见附录 E。

有些命令字会自动完成这一过程并生成常见错误，例如 `SETTYPEERROR`。下文参考部分列出了这些命令字。此外，还有一些生成 `CAS` 错误消息的快捷命令字，详见第 52 章。

不过，有时需要生成内置错误列表中没有的错误消息。为此，首先使用命令 `EXITMSGSTO` 存储所需消息，然后将 `#70000` 存为错误号。请注意，系统内置了名为 `#EXITERR` 的 `bint`，其中保存的就是该数值。最后调用 `ERRJMP` 即可。

上述过程可用 `DO#EXIT` 和 `DO$EXIT` 简化。前者取得一个 `bint` 参数，存储该数值并调用 `ERRJMP`。后者用于字符串：它取得一个字符串参数并执行上文所述操作。不过，这两个词条还会调用

`AtUserStack`，通知错误处理系统不要删除堆栈中的任何对象。因此，如果堆栈中有应当删除的对象（由您的程序压入），请勿使用这些命令字。发生错误时自动删除堆栈中非用户对象的机制将在第 29.1 节详细说明。

22.3 参考

22.3.1 通用命令字

地址	名称	说明
26067	ERRBEEP	(→) 发出蜂鸣声。
04CE6	ERROR@	(→#) 返回当前错误号。
04D0E	ERRORSTO	(#→) 存储新的错误号。
36883	ERROROUT	(#→) 存储新的错误号并调用 <code>ERRJMP</code> 。
04D33	ERRORCLR	(→) 将零存为新的错误号。
04ED1	ERRJMP	(→) 调用错误处理子系统。
04E07	GETEXITMSG	(→\$) 获取 <code>EXITMSG</code> （用户定义的错误消息）。
04E37	EXITMSGSTO	(\$→) 将 \$ 存为 <code>EXITMSG</code> 。
25EAE	DO#EXIT	(#→) 存储新的错误号，执行 <code>AtUserStack</code> ，然后执行 <code>ERRJMP</code> 。
25EB0	DO%EXIT	(%→) 与上项相同，但以实数作为参数。
25EAF	DO\$EXIT	(\$→) 将字符串存为 <code>EXITMSG</code> ，将 <code>#70000</code> 存为错误号，执行 <code>AtUserStack</code> ，然后执行 <code>ERRJMP</code> 。
04EA4	ABORT	(→) 执行 <code>ERRORCLR</code> 和 <code>ERRJMP</code> 。
地址	名称	说明
04E5E	ERRSET	(→) 设置新的错误捕获点。
04EB8	ERRTRAP	(→) 错误捕获标记。即使没有发生错误，也会删除 <code>ERRSET</code> 之后创建的所有临时环境。
04D87	JstGetTHEMSG	(#→\$) 从消息表中取得消息。要从库中取得消息，请使用公式： <code>libnum*#100+msgnum</code> 。 又名： <code>JstGETTHEMSG</code>
04D64	GETTHEMSG	(#→\$) 若为 <code>#70000</code> ，则执行 <code>GETEXITMSG</code> ；否则执行 <code>JstGetTHEMSG</code> 。
39332	(?GETMSG)	(#→\$msg) 如果参数是 <code>bint</code> ，则执行 <code>JstGETTHEMSG</code> 取得消息；其他参数原样返回。

22.3.2 错误生成命令字

地址	名称	说明
04FB6	SETMEMERR	Error 001h 生成 “Insufficient Memory” 错误。
05016	SETROMPERR	Error 004h 生成 “Undefined XLIB Name” 错误。
04FF2	SETPORTNOTAV	Error 00Ah 生成 “Port Not Available” 错误。
26134	SYNTAXERR	Error 106h 生成 “Invalid Syntax” 错误。
260C1	NOHALTERR	Error 126h 生成 “HALT Not Allowed” 错误。
26116	SETCIRCERR	Error 129h 生成 “Circular Reference” 错误。
262E2	SETSTACKERR	Error 201h 生成 “Too Few Arguments” 错误。
地址	名称	说明
262DD	SETTYPEERR	Error 202h 生成 “Bad Argument Type” 错误。
262D8	SETSIZEERR	Error 203h 生成 “Bad Argument Value” 错误。
262E7	SETNONEXTERR	Error 204h 生成 “Undefined Name” 错误。
2F458	SETIVLERR	Error 304h 生成 “Undefined Result” 错误。
2F37B	SetIOPARErr	Error C12h 生成 “Invalid IOPAR” 错误。
3721C	Sig?ErrJump	(# →) 如果错误号是 {13E 123 DFF} 中的任意一个，则调用 ERRJMP。
25F10	ederr	(→) 供使用 savefmt1 保存当前显示格式的应用程序使用的错误处理程序。它先调用 rstfmt1，然后输出错误。

第 23 章 虚拟堆栈

HP49提供“虚拟堆栈”功能。它是一组可操作 RPN 堆栈的命令，基本上可以保存堆栈，然后再将其恢复。

实际上，系统中存在一个由多个堆栈构成的堆栈（也许可以称为“元堆栈”）。最顶层的堆栈在正常情况下也是唯一的堆栈，它就是普通 RPN 堆栈：用户在其中输入对象，命令从中取得参数并将结果返回其中。下文称其为 RPN 堆栈。由这些堆栈组成的集合，更准确地说是堆栈，则称为“虚拟堆栈”。

您可以把整个 RPN 堆栈或其中一部分压入虚拟堆栈，使这些压入的对象成为虚拟堆栈中的一层。下文把虚拟堆栈中的一个层称为“虚拟堆栈层”。压入 RPN 堆栈后，您可以任意操作当前 RPN 堆栈，并可随时恢复先前压入的内容。您也可以再压入一个堆栈，这样除了可独立使用的“新”RPN 堆栈之外，还会存有两个虚拟堆栈层。

每个已压入的虚拟堆栈层都保存若干对象及其对象数。对象数在压入虚拟堆栈层时确定，之后无法再加入更多对象。以元对象形式返回虚拟堆栈层的命令字会同时返回该对象数，其他命令字则不会。压入时，以元对象形式压入堆栈的命令字允许只压入堆栈的一部分，其他命令字则会压入 RPN 堆栈中的全部内容。不过，您可以把一个并非以元对象形式压入的堆栈层按元对象形式弹出，也可以把堆栈按元对象形式压入而不按元对象形式弹出。两者唯一的区别是是否返回元素个数。

几乎所有 HP49 应用程序都会使用虚拟堆栈。当您需要立即保存完整堆栈而又不想占用太多内存时，它非常有用，而且速度很快。

例如，正是虚拟堆栈使您能够在窗口中输入完整命令行，并在字段中取得该命令行的结果。假设您在窗口中输入 DROP，即使堆栈原本不为空，也会得到“Too Few Arguments”错误。HP49 在运行该命令之前，会先把堆栈保存到虚拟堆栈中，然后运行命令。命令运行完毕后，它会在新堆栈之上恢复先前压入的虚拟堆栈层。

虚拟堆栈位于 TEMPOB 的第一个对象所包含的字符串中。其结构与局部变量堆栈相似，由若干块组成，并且保护机制与局部变量完全相同。如果捕获错误，在 ERRSET 与 ERRTRAP 之间创建的虚拟堆栈层会自动删除，就像局部变量块一样。详细信息见第 22.1.1 节。

有关虚拟堆栈的应用示例，请参见第 19.3 节的 DoL 列表处理程序和第 34.7 节的 HP48 Browser 示例。下面列出处理虚拟堆栈的命令。

23.1 参考

地址	名称	说明
25F1E	PushVStack	(obn..obl → obn..obl) 虚拟堆栈: (→ [obn..obl]) 将 RPN 堆栈压入虚拟堆栈; RPN 堆栈保持不变。
25F1F	PushVStack&Clear	(obn..obl →) 虚拟堆栈: (→ [obn..obl]) 执行 PushVStack, 然后清空 RPN 堆栈。
25F1A	PopMetaVStackDROP	(→ obn..obl) 虚拟堆栈: ([obn..obl] →) 将最顶层虚拟堆栈层弹入 RPN 堆栈。RPN 堆栈原有内容会保留。(名称中的 Meta 表示会返回对象数, 而 DROP 会在之后将其删除。)
地址	名称	说明
25F1B	PopVStack	(obm..obl → obn'..obl') 虚拟堆栈: ([obn'..obl'] →) 将最顶层虚拟堆栈层弹入 RPN 堆栈。RPN 堆栈原有内容会丢失。
25F17	GetMetaVStackDROP	(→ obn..obl) 虚拟堆栈: ([obn..obl] → [obn..obl]) 将最顶层虚拟堆栈层中的对象插入 RPN 堆栈, 虚拟堆栈保持不变。(名称中的 Meta 表示会返回对象数, 但该对象数会由 DROP 删除。)
25F18	GetVStack	(obm..obl → obn'..obl') 虚拟堆栈: ([obn'..obl'] → [obn'..obl']) 将最顶层虚拟堆栈层复制到 RPN 堆栈。虚拟堆栈不变, 但当前 RPN 堆栈会丢失。
26265	PushMetaVStack	(obn..obl #n → obn..obl #n) 虚拟堆栈: (→ [obn..obl]) 将 #n 个对象压入一个新的虚拟堆栈层。RPN 堆栈中的其他对象不会被压入, RPN 堆栈保持不变。
25F1D	PushMetaVStack&Drop	(obn..obl #n →) 虚拟堆栈: (→ [obn..obl]) 先执行 PushMetaVStack, 再删除已压入的对象。RPN 堆栈中的其他对象既不会被压入, 也不会被删除。

地址	名称	说明
25F19	PopMetaVStack	(→ obn..obl #n) 虚拟堆栈: ([obn..obl] →) 将最新虚拟堆栈层的内容插入 RPN 堆栈, 随后压入对象数。RPN 堆栈原有内容不会丢失。
2624C	GetMetaVStack	(→ obn..obl #n) 虚拟堆栈: ([obn..obl] → [obn..obl]) 将最顶层虚拟堆栈层中的对象及其对象数插入 RPN 堆栈。虚拟堆栈保持不变。
25F20	PushVStack&Keep	(obn..obl obm'..obl' #m → obm'..obl' #m) 虚拟堆栈: (→ [obn..obl]) 把 RPN 堆栈中不属于元对象的内容, 也就是位于它“上方”的内容, 压入新的虚拟堆栈层并从 RPN 堆栈中删除, 同时保留元对象。
25F21	PushVStack&KeepDROP	(obn..obl obm'..obl' #m → obm'..obl') 虚拟堆栈: (→ [obn..obl]) 执行 PushVStack&Keep, 然后执行 DROP。
25F1C	PopVStackAbove	(obm'..obl' → obn..obl obm'..obl') 虚拟堆栈: ([obn..obl] →) 将最顶层虚拟堆栈层的内容弹入 RPN 堆栈 (如同 PopMetaVStackDROP), 但放在 RPN 堆栈当前内容的上方。此操作可撤销 PushVStack&Keep 或 PushVStack&KeepDROP。
地址	名称	说明
26215	DropVStack	(→) 虚拟堆栈: ([obn..obl] →) 从虚拟堆栈中删除最顶层虚拟堆栈层。
26229	GetElemTopVStack	(#i → obi) 虚拟堆栈: ([obn..obl] → [obn..obl]) 从顶部开始计数, 返回最顶层虚拟堆栈层中的第 i 个对象。“从顶部开始计数”是指对象 #0 位于编号最高的层 n, 对象 #1 位于 n-1 层, 依此类推。注意: 不检查 #i 是否有效。
2626F	PutElemTopVStack	(new_ob #i →) 虚拟堆栈: ([obn..ob(n-i)..obl] → [obn..new_ob..obl]) 从顶部开始计数, 用 new_ob 替换最顶层虚拟堆栈层中的第 i 个对象。注意: 不检查 #i 是否有效。
26224	GetElemBotVStack	(#i → obi) 虚拟堆栈: ([obn..obl] → [obn..obl]) 从底部开始计数, 返回最顶层虚拟堆栈层中的第 i 个对象。“从底部开始计数”是指 #0 为编号最低层中的对象 (通常视为第 1 层), #1 位于第 2 层, 依此类推。注意: 不检查 #i 是否有效。
2626A	PutElemBotVStack	(new_ob #i →) 虚拟堆栈: ([obn..obi..obl] → [obn..new_ob..obl]) 从底部开始计数, 用 new_ob 替换最顶层虚拟堆栈层中的第 i 个对象。注意: 不检查 #i 是否有效。
地址	名称	说明
26233	GetVStackProtectWord	(→ #) 底层操作: 获取最后一个 VStack 层的保护字。
2622E	SetVStackProtectWord	(# →) 底层操作: 设置最后一个 VStack 层的保护字。

第 24 章 内存操作

与用户命令 STO 和 RCL 基本等价的命令字是 CREATE、STO 和 @:

命令字	堆栈与操作
CREATE	(ob id →) 创建名称为 id、内容为 ob 的变量。如果 ob 是当前目录或包含当前目录，就会发生错误（“Directory Recursion”）。此命令字不会检查是否已经存在名为 id 的变量；即使存在，也会再创建一个变量。
STO	(ob id →) (ob lam →) 对于 lam，会把临时标识符重新绑定到 ob；如果 lam 未绑定，则发生错误。对于 id，STO 会尝试用 ob 替换名为 id 的变量内容。如果找不到该名称的变量，就创建一个新变量。
@	(id → ob TRUE) (id → FALSE) (lam → ob TRUE) (lam → FALSE) 尝试返回变量或临时标识符中存储的内容。成功时返回所存对象和 TRUE；如果找不到该名称的变量或 lam，则只返回 FALSE。对于变量，搜索从当前目录开始，并在必要时逐级向上搜索父目录。

STO 和 @ 有一个问题：例如，如果把 SIN 作为参数，函数的整个主体都会存入变量。因此，最好使用 SAFESTO 和 SAFE@。它们的工作方式分别类似于 STO 和 @，但会自动把 ROM 主体转换为 XLIB 名称（SAFESTO），并再转换回来（SAFE@）。

请注意，这些词条及其他词条名称中的 `SAFE` 仅表示会执行上述转换，

并不表示它们在其他方面也有安全保障。

还有许多与内存有关的命令字，详见下表。

24.1 参考

24.1.1 提取、存储和删除

地址	名称	说明
0797B	@	(id/lam $\rightarrow$ ob T) (id/lam $\rightarrow$ F) 基本提取函数。
35C2C	DUP@	(id/lam $\rightarrow$ id/lam ob T) (id/lam $\rightarrow$ id/lam F) 先执行 DUP, 再执行 @。
35A5B	SAFE@	(id/lam $\rightarrow$ ob T) (id/lam $\rightarrow$ F) 对于 lam, 执行 @; 对于 id, 对找到的 ob 执行 ?ROMPTR>。
35A56	DUPSAFE@	(id/lam $\rightarrow$ id/lam ob T) (id/lam $\rightarrow$ id/lam F) 先执行 DUP, 再执行 SAFE@。
25EF7	SAFE@_HERE	(id $\rightarrow$ ob F) (id $\rightarrow$ T) 与 SAFE@ 相同, 但只在当前目录中工作。
2F064	Sys@	(ID $\rightarrow$ ob T) (ID $\rightarrow$ F) 临时切换到 HOME 目录并在其中执行 @。
2F2A3	XEQRCL	(id $\rightarrow$ ob) 与 SAFE@ 相同, 但找不到变量时会报错。也可用于 lam, 但得到的错误不正确。
2F24E	LISTRCL	({path id} $\rightarrow$ ob) 从指定路径提取对象。
地址	名称	说明
07D27	STO	(ob id/lam $\rightarrow$) 对于 id, 此词条假定 ob 不是 pco。替换对象时, 会把该对象复制到 TEMPOB 并更新指针。对于 lam, 如果 lam 未绑定则报错。
35A29	SAFESTO	(ob id/lam $\rightarrow$) 对于 id, 存储之前对对象执行 ?>ROMPTR。
2F380	SysSTO	(ob ID $\rightarrow$) 临时切换到 HOME 目录并在其中执行 STO。
25E79	XEQSTOID	(ob id/lam $\rightarrow$) 与 SAFESTO 相同, 但只会存储到当前目录, 且不会覆盖目录。 又名: ?STO HERE
25F0C	XEQStoKey	(ob ID $\rightarrow$)
3E823	xSTO>	(ob id $\rightarrow$) (ob symb $\rightarrow$) 与 xSTO 类似, 但如果堆栈第 1 层参数为符号对象, 则使用其中第一个元素作为目标变量。

地址	名称	说明
0BD007	^PROMPTSTO1	(id/lam →) 输入变量值并将其存储。
085D3	REPLACE	(newob oldob → newob) 用 newob 替换内存中的 oldob。
08C27	PURGE	(id →) 删除变量；事先不进行类型检查。
25E78	?PURGE_HERE	(id →) 与 PURGE 类似，但只在当前目录中工作。
1D3006	^SAFEPURGE	(idnt/lam →) 如果 idnt/lam 存在，则将其删除。
08696	CREATE	(ob id →) 在当前目录中创建变量。如果 id 是当前目录或包含当前目录，则报错。假定 id 不是 pco。
地址	名称	说明
25EC4	DoHere:	(→) 仅以当前目录为作用域，对运行流中的下一个对象求值。
36A8B	'LAMLNAMESTO	(ob →) 将 ob 存入 LAM LAMLNAME。

24.1.2 目录

地址	名称	说明
25EA1	CREATEDIR	(id →) 创建空目录。首先调用?PURGE_HERE 删除原对象。
08326	LASTRAM-WORD	(rrp → ob T) (rrp → F) 提取目录中的第一个对象。
25EE7	LastNonNull	(rrp → ob T) (rrp → F) 提取目录中名称非空的第一个对象。
08376	PREVRAM-WORD	(ob → ob' T) (ob → F) 提取目录中的下一个对象。
25EF2	PrevNonNull	(ob → ob' T) (ob → F) 提取目录中名称非空的下一个对象。
082E3	RAM-WORDNAME	(ob → id) 提取当前目录中对象的名称。
25F14	XEQPGDIR	(id →) 删除目录；事先检查引用等。
2F296	XEQORDER	({id1 id2..} →) 通过把给定变量移到目录开头，对目录中的变量排序。
25EB9	DOVARS	(→ {id1 id2..}) 返回当前目录中的变量列表。
地址	名称	说明
25EB8	DOTVARS%	(% → { }) 返回当前目录中用户类型由该数字指定的变量列表。给出单个数字时，这是内部 TVARS。
0BD002	^DOTVARS{ }	({# #' ...} → { }) 返回当前目录中用户类型由列表中任意数字指定的变量列表。这是 TVARS 程序的核心。
25EF1	PATHDIR	(→ {HOME dir1 dir2..}) 返回当前路径。
2F265	UPDIR	(→) 转到父目录。
08D5A	CONTEXT@	(→ rrp) 提取当前目录。
08D08	CONTEXT!	(rrp →) 设置新的当前目录。
08DD4	SYSRRP?	(rrp → flag) 判断 rrp 是否为 HOME。
08D92	HOMEDIR	(→) 将 HOME 设为当前目录。 又名：SYSCONTEXT
3712C	SaveVarRes	(→) 将当前目录和上一个目录绑定到两个名称为空的 lam。
37186	RestVarRes	(→) 首先把 HOME 同时设为当前目录和上一个目录（以防发生错误），然后从 1LAM 和 2LAM 恢复当前目录和上一个目录。

24.1.3 隐藏目录

地址	名称	说明
3714A	SetHiddenRes	(→) 将隐藏目录设为当前目录和上一个目录。
370C3	WithHidden	(→ ?) 在隐藏目录中执行下一个命令。
370AF	RclHiddenVar	(id → ob T) (id → F) 从隐藏目录提取变量。等同于:: WithHidden @ ;
37104	StoHiddenVar	(ob id →) 将变量存入隐藏目录。等同于:: WithHidden STO ;
37118	PuHiddenVar	(id →) 从隐藏目录删除变量。等同于:: WithHidden PURGE ;

24.1.4 临时内存

堆栈中的对象位于一个称为“临时内存”的区域。顾名思义，它用于临时存储。

在堆栈中复制对象时，并不会真正创建对象副本：堆栈只包含指向对象的指针，复制的也只是这个指针。

修改对象时，多数命令会自动为相应对象创建新副本，然后修改该副本。换言之，如果您在堆栈中输入一个字符串，按 **ENTER**，然后编辑该字符串，现在就会得到两个不同的字符串。这只是因为编辑前创建了字符串副本。如果没有创建副本，两个字符串都会被修改，因为它们实际上是同一个对象。

有少数命令在编辑对象之前不会创建副本。这意味着该对象的所有副本，无论位于堆栈还是存储在内存中，都会同时被修改。有时这是所需行为，有时则不是。这些命令有时称为“bang 型”命令。本书出现这类命令时，会在其说明中注明。使用这些命令时，必须小心，不要同时修改过多

个对象。您可以使用 TOTEMPOB 或 CKREF 为对象再创建一个副本；这样便可安全使用这些“bang 型”命令。

地址	名称	说明
06657	TOTEMPOB	(ob → ob') 将对象复制到 TEMPOB，并返回指向新副本的指针。
35C90	TOTEMPSWAP	(ob1 ob2 → ob2' ob1) 先执行 TOTEMPOB，再执行 SWAP。
25E9F	CKREF	(ob → ob') 如果对象位于 TEMPOB 中、未嵌入复合程序且未被引用，则什么也不做；否则将其复制到 TEMPOB 并返回副本。
3700A	SWAPCKREF	(ob1 ob2 → ob2 ob1') 先执行 SWAP，再执行 CKREF。
06B4E	INTEMNOTREF?	(ob → ob flag) 如果对象位于 TEMPOB 区域、未嵌入复合程序且未被引用，则返回该对象和 TRUE；否则返回该对象和 FALSE。
01E0E8	~INTEMPOB?	(ob → ob flag)

第 25 章 时间与闹钟

本章列出与时间、日期及内部闹钟列表有关的词条。

25.1 参考

地址	名称	说明
26120	SLOW	(→) 延迟 15 毫秒。
26125	VERYSLOW	(→) 延迟 300 毫秒。
2F37E	SORTASLOW	(→) 延迟 1.2 秒 (4 × VERYSLOW)。
2612A	VERYVERYSLOW	(→) 延迟 3 秒。
2F2D4	dowait	(%secs →) 等待指定的秒数。
3005E	%>HMS	(% → %hms) 从十进制转换为 H.MMSS 格式。
30912	%H>HMS	(%% → %%hms) 与 %>HMS 相同，但用于长实数。
30077	%HMS>	(%hms → %) 从 H.MMSS 格式转换为十进制。
3008B	%HMS+	(%hms1 %hms2 → %hms) 将两个 hms 格式的时间相加。
300B3	%HMS-	(%hms1 %hms2 → %hms) 将两个 hms 格式的时间相减。
2EECF	TOD	(→ %time) 返回当前时间。
地址	名称	说明
2F388	VerifyTOD	(%time → %time) 检查时间是否有效；无效时报错。
2EED0	DATE	(→ %date) 返回当前日期。
2EED2	DATE+DAYS	(%date %days → %date') 给日期加上指定天数。
2EED1	DDAYS	(%date1 %date2 → %days) 返回两个日期之间的天数。
2EED7	CLKTICKS	(→ hxs) 返回计时滴答数。 又名： SysTime
2EED3	TIMESTR	(%dt %tm → "dy dt tm") 使用当前格式返回时间的字符串表示。例如： "WED 06/24/98 10:00:45A"
2F329	Date>d\$	(%date → \$) 使用当前格式返回日期的字符串表示。
2F381	TOD>t\$	(%time → \$) 使用当前格式返回时间的字符串表示。
2F1AB	Date>hxs13	(%date → hxs) 将日期转换为计时滴答数。
2F003	(Ticks>Date)	(hxs → %date) 从内部闹钟列表格式的 hxs 返回日期。
2F002	(Ticks>TOD)	(hxs → %time) 从内部闹钟列表格式的 hxs 返回时间。
2F004	(Ticks>Rpt)	(hxs → %rpt) 将内部闹钟列表格式的 hxs 转换为重复间隔。

25.1.1 闹钟

内部闹钟列表采用以下格式：

```
{ { hxs action } { ... } ... }
```

每个 hxs 的长度为 24 个半字节。最低有效的 13 个半字节表示日期和时间的计时滴答值；接下来的 10 个半字节表示重复间隔（如果有）；最高有效的半字节表示闹钟状态，例如待处理、已确认等。

地址	名称	说明
2F178	ALARMS@	(→ { }) 返回内部闹钟列表。
2F37F	STOALM	(%date %time acti %rep → %) 存储闹钟。 %repeat 是两次重复之间的计时滴答数。每秒有 8192 个滴答，每分钟 60 秒，每小时 60 分钟，因此要创建每小时重复一次的闹钟，%repetition 应为 8192*60*60 = 29491200。返回表示该闹钟在列表中位置的实数。
2F0AC	PURGALARM%	(% →) 内部 DELALARM。
2F314	RCLALARM%	(%n → { }) 提取第 n 个闹钟。列表采用 STOALARMLS 的格式。

地址	名称	说明
25FA9	ALARM?	($\rightarrow$ flag) 如果闹钟到期，则返回 TRUE。

第 26 章 系统功能

下面列出与系统有关的功能，例如配置计算器的某些方面以及关闭计算器。这里还说明处理用户标志和系统标志的功能。

26.1 参考

26.1.1 用户标志和系统标志

地址	名称	说明
2614D	SetSysFlag	(# →) 设置编号为 # 的系统标志。
26044	ClrSysFlag	(# →) 清除编号为 # 的系统标志。
26170	TestSysFlag	(# → flag) 如果系统标志已设置，则返回 TRUE。
26152	SetUserFlag	(# →) 设置编号为 # 的用户标志。
26049	ClrUserFlag	(# →) 清除编号为 # 的用户标志。
26175	TestUserFlag	(# → flag) 如果用户标志已设置，则返回 TRUE。
2F259	RCLSYSF	(→ hxs) 提取第 1 至 64 号系统标志。
2F25F	(STOSYSF)	(hxs →) 存储第 1 至 64 号系统标志。
2F23E	DOSTOSYSF	(hxs →) 存储第 1 至 64 号系统标志，并检查 LASTARG 标志是否改变。
地址	名称	说明
2F25A	(RCLSYSF2)	(→ hxs) 提取第 65 至 128 号系统标志。
2F260	(STOSYSF2)	(hxs →) 存储第 65 至 128 号系统标志。
2F25B	RCLUSERF	(→ hxs) 提取第 1 至 64 号用户标志。
2F261	(STOUSERF)	(hxs →) 存储第 1 至 64 号用户标志。
2F25C	(RCLUSERF2)	(→ hxs) 提取第 65 至 128 号用户标志。
2F262	(STOUSERF2)	(hxs →) 存储第 65 至 128 号用户标志。
2F3A9	(STOALLF)	(hxs usr hxs sys →) 存储第 1 至 64 号用户标志和系统标志。第一个参数是用户标志，第二个参数是系统标志。
2F3AA	(STOALLF2)	(hxs sys1 hxs usr1 hxs sys2 hxs usr2 →) 需要 4 个 hxs，并将它们存为用户标志和系统标志。
3B76C	(DOSTOALLF2)	({ } →) 存储系统标志和用户标志。需要包含两个或四个 hxs 的列表。前两个依次是第 1 至 64 号系统标志和用户标志；如果还有后两个，则依次是第 65 至 128 号系统标志和用户标志。
25F23	SaveSysFlags	(→) 把系统标志保存到一个虚拟堆栈层。
25F22	RestoreSysFlags	(→) 从虚拟堆栈恢复系统标志，并弹出该层。
2ABF0	RunSafeFlags	运行流: (ob →) 对运行流中的下一个对象求值，并在求值前后保存和恢复系统标志。使用 DoRunSafe。此命令非常有用。
地址	名称	说明
2AB69	RunInApprox	运行流: (ob →) 对运行流中的下一个对象求值；清除系统标志 20、21，并设置系统标志 22、105、102、120。
2AC0E	DoRunSafe	(ob → hxs1 hxs2) 对 ob 求值，并把求值之前的系统标志压入堆栈。由 RunSafeFlags 和 RunSafeFlagsNoError 使用。
2ABD7	RunSafeFlagsNoError	运行流: (ob →) :: 'R DoRunSafe 2DROP ;
2EFA5	DOHEX	(→) 将 HEX 字符串的堆栈显示格式切换为十六进制。
2EFA8	DODEC	(→) 将 HEX 字符串的堆栈显示格式切换为十进制。
2EFA6	DOBIN	(→) 将 HEX 字符串的堆栈显示格式切换为二进制。
2EFA7	DOOCT	(→) 将 HEX 字符串的堆栈显示格式切换为八进制。
2EFBF	BASE	(→ #) 返回 #10h、#10d、#10b 或 #10o。按十进制值分别为：十六进制基数 16、十进制基数 10、八进制基数 8、二进制基数 2。

地址	名称	说明
2605D	DOSTD	(→) 用户命令字 STD 的内部版本。
26053	DOFIX	(# →) 用户命令字 FIX 的内部版本。
26058	DOSCI	(# →) 用户命令字 SCI 的内部版本。
2604E	DOENG	(# →) 用户命令字 ENG 的内部版本。
261A7	savefmt1	(→) 保存当前数值格式，并切换到 STD 模式。
地址	名称	说明
261A2	rstfmt1	(→) 恢复由 savefmt1 保存的数值格式。只能保存一组标志；这些词条不可嵌套。
2FFDB	SETRAD	(→) 将角度模式设为 RAD。
25EF3	RAD?	(→ flag) 判断角度模式是否为 RAD。
2FFBD	SETDEG	(→) 将角度模式设为 DEG。
2FFEF	SETGRAD	(→) 将角度模式设为 GRAD。
25EBA	DPRADIX?	(→ flag) 如果当前小数分隔符为 “.”，则返回 TRUE。

26.1.2 通用功能

地址	名称	说明
25EB2	DOBEEP	(%freq %dur →) 发出蜂鸣声，类似于用户函数 BEEP。
261AC	setbeep	(#ms #Hz →) 同样会发出蜂鸣声。
041A7	TurnOff	(→) 内部 OFF。
041ED	DEEPSLEEP	(→ flag) 使 HP 进入深度睡眠模式。如果出现 “Invalid Card Data” 消息，则返回 TRUE。
01118	LowBat?	(→ flag) 如果电池电量低，则返回 TRUE。
0426A	ShowInvRomp	(→) 闪现 “Invalid Card Data” 消息。
2EE5D	?FlashAlert	(→) 显示系统警告。
05F42	GARBAGE	(→) 强制执行垃圾回收。
05F61	MEM	(→ #) 返回可用内存量，以半字节为单位。不执行垃圾回收。（用户命令会执行。）
地址	名称	说明
05902	OSIZE	(ob → #) 返回对象大小，以半字节为单位。强制执行垃圾回收。
05944	OCRC	(ob → #nib hxs) 以半字节为单位返回大小，并以 hxs 返回校验和。
2F257	OCRC%	(ob → hxs %bytes) 返回校验和及字节数。
2F267	VARSIZE	(id → hxs %bytes) 返回指定变量的校验和及字节数。
394C8	INHARDROM?	(ob → ob flag) 判断对象地址是否小于 #80000h。
05AB3	CHANGETYPE	(ob #prolog → ob') 更改对象的前导码，并执行 TOTEMPOB。
25F90	>LANGUAGE	(# →) 设置消息的当前语言。x→LANGUAGE 的内部版本。
25F95	LANGUAGE>	(→ #) 返回消息的当前语言。xLANGUAGE→ 命令的内部版本。
256BE	NOBLINK	(→) 清除 BLINKFLAG，即 SysNib5。
25E71	?BlinkCursor	(→) 如果处于 App 模式或编辑行中，使光标闪烁。

第 27 章 串行通信

下面列出的词条可让程序员编写通过 HP49G 串行接口与其他设备通信的程序。

27.1 参考

地址	名称	说明
2EEBB	SENDLIST	({} →) 内部 SEND。
2EEBC	GETNAME	(\$/id/lam →) 内部 KGET。
2EEBD	DOFINISH	(→) 内部 FINISH。
2EEBE	DOPKT	(\$ \$' →) 内部 PKT。
2EEC1	DOBAUD	(% →) 内部 BAUD。
2EEC2	DOPARITY	(% →) 内部 PARITY。
2EEC3	DOTRANSIO	(% →) 内部 TRANSIO。
2EEC4	DOKERRM	(→ \$) 内部 KERRM。
2EEC5	DOBUFLEN	(→ % 0/1) 内部 BUFLen。
2EEC6	DOSBRK	(→) 内部 SBRK。
2EEC7	DOSRECV	(% →) 内部 SRECV。
地址	名称	说明
2EEC9	CLOSEUART	(→) 内部 CLOSEIO。
2EECB	DOCR	(→) 内部 CR。
2EECD	DODELAY	(% →) 内部 DELAY。
2F31A	APNDCRLF	(\$ → \$') 在字符串末尾附加回车符和换行符。
2716D	StdIOPAR	(→ {}) 默认 IOPAR: { 9600 0 0 0 3 1 }。
2EEBF	GetIOPAR	(→ %baud % % % % %) 提取 IOPAR, 并将其各元素展开到堆栈中。
2F062	StoIOPAR	({} →) 把 I/O 参数列表存储在 HOME 目录的 IOPAR 变量中。
2F37B	SetIOPARErr	(→) 抛出 IOPAR 错误: “Invalid IOPAR”。
2F34F	KVISLF	(\$ → \$') 与 KVIS 类似, 但会为 PC 在每个换行符前插入 <cr>。
2F34E	KVIS	(\$ → \$') 将特殊字符转换为二合字符, 以便以 ASCII 传输到 PC。
2F34D	KINVISLF	(\$ → \$') 将字符串中的二合字符转换回字符, 并删除行末的 <cr>。
2F389	VERSTRING	(→ \$) 返回版本字符串。

第 28 章

HP49 文件管理器

HP49 File Manager（简称 **Filer**）允许程序员编写各种处理文件的应用程序。
使用 **Filer** 的两个内置应用程序是 **File Manager** 和 **Font Browser**。

28.1 使用 Filer

调用 **Filer** 的通用词条是 `^FILER_MANAGERTYPE`。它取得三个参数：`Filer_Type`、`Filer_Path` 和 `Filer_List`。

28.1.1 Filer_Type 参数

此参数用于选择应显示的对象类型。它是一个列表，其中包含允许显示的对象前导码地址。
例如，**HP49 Font Browser** 只显示字体、目录和备份对象，因此 `Filer_Type` 指定为 `{ DOFONT DORRP DOBAK }`。
如果要浏览所有类型的对象，此参数只需为 `{ ZERO }`。这种情况很常见，因此系统提供了一个词条，可把该列表作为参数传给 `^FILER_MANAGERTYPE`。它名为 `^FILER_MANAGER`。使用该词条时，只需指定另外两个参数。

28.1.2 Filer_Path 参数

此参数指定初始路径，可以采用以下形式：

值	含义
<code>{ }</code>	从 HOME 开始。
<code>{ FOO.DIR }</code>	从 HOME/FOO.DIR 开始。
值	含义
<code>:n:{ }</code>	从端口 <code>n</code> 开始。
<code>:n:{ FOO }</code>	从端口 <code>n</code> 中的备份对象 <code>FOO</code> 开始。

28.1.3 Filer_List 参数

此参数指定菜单键和实体键分配。它是一个列表，每个菜单键对应一个元素。每个菜单键由一个包含三至五个参数的列表表示。
通用结构如下：

```
1 {
  {
    (Item 1)
    Name_Item
    5 Location_Item
    Action_Item
    [ ExtraProgram_Item (if 16 <= Action_Item <= 23) ]
    [ Key_Shortcut ]
  }
  10 {
    (Item 2)
    ...
  }
  ..
  15 }
```

下面逐一说明各子列表中的元素。

28.1.3.1 Name_Item

此元素指定菜单中显示的内容。它可以是字符串、`grob`，或求值后返回字符串或 `grob` 的程序。

28.1.3.2 Location_Item

此元素用于控制绑定到该菜单的操作何时可以运行。它可以是 `bint`，也可以是求值后返回 `bint` 的程序。
共有五个可用值，见下表。“常量”列给出编程时应使用的常量名称；这些常量定义在示例 `include` 目录中的 `filer.h` 文件内。

值	常量	含义
0	<code>fEverywhere</code>	操作可在任何位置运行。

值	常量	含义
1	fVar	只有当用户正在浏览 HOME 目录或其子目录时，操作才能运行。
2	fNoLib	用户正在浏览库时，操作不能运行。
3	fNoBackup	用户正在浏览端口中的备份对象时，操作不能运行。
4	fHomePort	操作只能在端口根目录中运行。

28.1.3.3 Action_Item

此元素定义用户按下相应菜单软键或分配的实体键时会发生什么（见第 28.1.3.5 节）。它可以是 **bint**，也可以是求值后返回 **bint** 的程序。

您可以调用 **Filer** 的内置功能，也可以定义自己的功能。下表列出可用的内置操作。“常量”仍表示 **filer.h** 中定义的常量名称。

值	常量	操作
0	cBip	发出蜂鸣声。
1	cInfo	HP49G 中尚未实现。
2	cHexa	HP49G 中尚未实现。
3	cView	查看对象。
4	cArbo	显示目录树。
5	cUp	向上移动亮选。
6	cMaxUp	将亮选移到第一个项目。
7	cDown	向下移动亮选。
8	cMaxDown	将亮选移到最后一个项目。
9	cSelect	标记所选变量。
10	cUpDir	转到父目录。
11	cDownDir	进入亮选的目录。
12	cPreviousMenu	显示上一菜单页。
13	cNextMenu	显示下一菜单页。
14	cEVAL	对亮选的变量求值。
15	cSwapHeader	在两个可用页眉行之间切换。
24	cDetails	在显示变量信息与仅显示变量名称之间切换。
25	cEDIT	编辑所选变量。
26	cCOPY	复制所选变量。
值	常量	操作
27	cMOVE	移动所选变量。
28	cRCL	提取所选变量的内容。
29	cPURGE	删除所选变量。
30	cRENAME	重命名所选变量。
31	cCRDIR	创建目录。
32	cORDER	重新排列当前目录中的变量。
33	cSEND	使用 Kermit 发送所选变量。
34	cHALT	暂时挂起 Filer 。
35	cEDITB	在最合适的编辑器中编辑所选变量。
36	cRECV	使用 Kermit 接收变量。
37	cQUIT	退出 Filer 。
38	cPageUp	将 Filer 内容向上滚动一屏。
39	cPageDown	将 Filer 内容向下滚动一屏。
40	cNewObject	创建新变量。
41	cSort	打开含有多个变量排序选项的对话框。

要运行自定义程序，**Action_Item** 参数应在 16-23 范围内。这八个值分别指定堆栈中的内容以及程序的调用方式。

下表列出调用方式：

值	说明
16	只提取当前路径。 1: 路径
17	提取当前所选对象的名称和内容。 3: 路径 2: 对象 1: 名称
18	与上项等效，但处理多个所选对象。 2n + 2: 路径 . . . 5: 对象 2 4: 名称 2 3: 对象 1 2: 名称 1 1: 对象数 (bint)

值	说明
19	对每个对象调用一次程序。对每个对象压入的内容与调用方式 #17 相同。
20	只提取当前对象的名称。 2: 路径 1: 名称
21	提取所有所选对象的名称。 n + 2: 路径 . . . 3: 名称 2 2: 名称 1 1: 名称数 (bint)
22	仅通过地址字符串提取当前对象。 2: 路径 1: 字符串
23	通过地址字符串提取所选对象。 2: 路径 1: 字符串

这里不再说明自定义调用 22 和 23，因为它们不太实用，而且用法稍复杂。

在端口根目录下对库调用程序时，名称适用以下特殊规则：

- 对于调用 17 和 18，名称为库标题，例如 “Emacs 1.09 CD&Pivo”。
- 对于调用 19，名称为字母 “L” 加库编号，例如 “L1790”。
- 对于调用 20 和 21，名称为以实数表示的库编号。

28.1.3.4 ExtraProgram_Item

使用自定义程序时，此元素保存要调用的程序。

还有一些可能有用的附加功能：

- 如果从 VAR 启动程序，程序会在当前浏览的目录中启动。
- 除调用 16 可以在空目录中运行外，程序只能针对所选对象调用。
- 默认情况下，程序运行后会刷新屏幕、重新解析工作目录，并丢失当前选择。可通过在堆栈中留下 FALSE 来阻止这些操作。例如：

```
1 { "INFO"
fEverywhere
BINT20
:: SWAPDROP (Remove the path)
5 "Name selected is:" DISPROW1
ID>$ DISPROW2
FALSE
;
}
```

- 如果希望程序运行后强制 Filer 退出，请在堆栈中留下 “TakeOver”。例如：

```
{ "QUIT" fEverywhere BINT16 :: DROP ' TakeOver ; }
```

这等效于以下使用内置调用的代码：

```
{ "QUIT" fEverywhere cQUIT }
```

28.1.3.5 Key_Shortcut

使用此参数可以把程序分配给按键。该参数是形如 # axx 的 bint，其中 a 为 0 或 1，分别表示不带 ALPHA 和带 ALPHA；xx 为按键代码，还可选择加 #40 表示左移位键，或加 #C0 表示右移位键。

如果要把程序分配给左移位键 + TOOL，数值应为 #049：#09 表示 TOOL 键，#40 表示左移位键。

注：此参数必须是列表中的第五个元素。因此，如果使用内置调用，就必须把词条定义成类似下面这样：

```
{ "TITLE" fEverywhere cQUIT TakeOver # 12F }
```

这会把程序分配给 ALPHA-ON。

28.2 参考

地址	名称	说明
067004	^Filer	(→) 调用标准 Filer 。
06D004	^FILER_MANAGER	({path} {args} →) 自定义 Filer ，可浏览所有对象类型。
06E004	^FILER_MANAGERTYPE	({types} {path} {args} →) {args} = { item1 item2 ... } item = {name loc action [prog] [key]} ... } 仅为所选类型创建自定义 Filer 。

第三部分 输入与输出

第 29 章 参数检查

当系统 RPL 程序可由用户直接访问时，检查程序所需的所有参数是否都在堆栈中以及类型是否有效非常重要。在用户 RPL 中无需操心这些问题，因为系统会自动检查。系统 RPL 中只有极少数命令会执行这种检查，因此这项任务留给程序员完成。乍看之下这似乎是缺点，实际上却是优点：您只需在程序开始时检查一次参数。这样生成的代码速度很快，不像用户 RPL 那样每个命令都会检查参数。

29.1 参数数量

要检查特定数量的参数，请使用下列命令之一。它们会检查堆栈中的参数是否足够；如果不足，则生成“Too Few Arguments”错误。

命令	适用情况
CK0, CK0NOLASTWD	不需要参数。
CK1, CK1NOLASTWD	需要一个参数。
CK2, CK2NOLASTWD	需要两个参数。
CK3, CK3NOLASTWD	需要三个参数。
CK4, CK4NOLASTWD	需要四个参数。
CK5, CK5NOLASTWD	需要五个参数。

CK<n> 命令会保存其所在命令的名称；如果发生错误，就显示该名称。详细信息见第 22 章。这意味着它们只能用于库中，因为如果程序不属于库而发生错误，错误消息会显示为类似“XLIB 1364 36 Error:”的内容。不属于库的程序应使用 CK<n>NOLASTWD，它不会保存命令名。

除了检查指定数量的参数外，这些命令字还会“标记”堆栈，以便在发生错误时删除程序压入堆栈的对象，不在堆栈中留下无用内容。其原理是在第 n 层上方“标记”堆栈，其中 n 是所需参数的数量。例如，如果程序使用 CK2 或 CK2NOLASTWD，而堆栈中有三个参数，可以把堆栈想象成这样：

```
3:      10.  
2:      3.  
1:      5.5
```

该标记并不固定在这一层；随着元素压入或弹出，它也会移动。程序压入 bint #1h 后，堆栈如下：

```
4:      10.  
3:      3.  
2:      5.5  
1:      #1h
```

此时，如果程序中发生错误，标记“下方”的所有对象都会被删除。这会删除程序压入的所有对象；如果程序参数仍在堆栈中，也会一并删除。这是 HP49G 的标准行为。

这些命令字除了检查参数数量并提供错误恢复机制外，还会把参数保存为上次参数；如果启用了此功能，就可用用户命令 LASTARG 取回。如果发生错误且该功能已启用，参数会自动恢复。

对于不取得参数但可由用户访问的程序，仍应使用 CK0；如果程序不属于库，则使用 CK0NOLASTWD。这样可把堆栈中的全部对象标记为归用户所有，并为错误恢复标记堆栈。

如果程序使用由堆栈指定的参数数量（例如 DROPN），请使用 CKN 或 CKNNOLASTWD。这些命令字先检查堆栈第 1 层中的实数，再检查堆栈中是否有指定数量的对象。堆栈在第 2 层标记，但只有该实数保存到 LASTARG。最后把实数转换为 bint。

29.2 参数类型

命令字 CK&DISPATCH1 和 CK&DISPATCH0 可让程序根据所给参数的类型执行不同操作。其用法如下：

```
1 ...  
CK&DISPATCH1  
#type1 action1  
#type2 action2  
5 #type3 action3  
...  
#type_n action_n  
;
```

类型 / 操作对由 SEMI (;) 终止。如果分派之后还要对所有参数类型执行一些操作，就需要把整个 CK&DISPATCH1 块包含在另一个二级程序中。

CK&DISPATCH0 的工作方式如下：它检查堆栈是否与 #type1 中的定义匹配。如果匹配，就执行 action1，然后从 SEMI 之后继续执行程序。（每个操作必须是单个对象；如果要执行多个操作，就必须把它们全部放在一个二级程序中，也就是放在 :: 与 ; 之间。）如果类型定义与堆栈不匹配，就检查 #type2，依此类推。如果没有找到匹配项，则生成 “Bad Argument Type” 错误。

即使程序只接受一种参数组合，此命令对于检查参数是否属于给定类型仍然很有用。

CK&DISPATCH0 与 CK&DISPATCH1 的区别在于，后者第一次遍历未找到匹配项后，会移除参数的所有标记，把 ZINT 转换为实数，然后进行第二次遍历。只有第二次遍历仍未找到匹配项时，才生成 “Bad Argument Type” 错误。每个类型定义都是形如 #nnnnn 的 bint。每个 n 都是十六进制数字，按照下表表示堆栈相应位置中的对象。第一个 n 表示堆栈第 5 层的对象，第二个表示第 4 层，依此类推。因此，#00201 表示堆栈第 3 层为复数、第 2 层为任意对象、第 1 层为实数；#000A6 表示第 2 层为 hxs、第 1 层为 id。还有一些以 F 结尾的两位对象类型编号。每使用一个这样的编号，可检查的参数数量就会减少。例如，#13F4F 表示第 3 层为实数，

第 2 层为扩展实数，第 1 层为扩展复数。

分派代码	用户类型	对象类型
0	n/a	任意对象
1	0	实数
2	1	复数
3	2	字符串
4	3、4 或 29	数组或矩阵
5	5	列表
6	6	标识符（全局）
7	7	LAM（临时标识符）
8	8、18 或 19	二级程序
9	9	符号对象
A	n/a	符号类
B	10	十六进制字符串
C	11	图形对象（GROB）
D	12	标记对象
E	13	单位对象
0F	14	ROM 指针（XLIB 名称）
1F	20	Bint
2F	15	目录
3F	21	扩展实数
4F	22	扩展复数
5F	23	链接数组
6F	24	字符
7F	25	代码对象
8F	16	库
9F	17	备份对象
AF	26	库数据
BF	27	访问指针
CF	30	字体对象
DF	27	迷你字体对象
EF	27	外部对象 4（未使用）
FF	28	ZINT

此外还有 CK<n>&Dispatch 命令字，其中 <n> 是 1 至 5 的数字。这些命令字把 CK<n> 与 CK&DISPATCH1 组合在一起。因为它们使用 CK<n>，也就会保存最后一个命令名，所以只能用于库命令。

29.2.1 示例

通过反汇编并研究内置命令字，您可以学到很多内容，不仅限于参数检查，还包括许多其他方面。

TYPE 命令展示了分派的示例。下面是它的反汇编结果：

```
1 ::
CK1
::
CK&DISPATCH0
5 real %0
cmp %1
str %2
array XEQTYPEARRY
list %5
10 id %6
lam %7
seco TYPESEC (returns 8, 18 or 19)
symb %9
hxs %10
15 grob %11
TAGGED %12
unitob %13
```

```
rompointer %14
BINT31 %20 (#)
20 BINT47 %15
# 3F %21 (%%)
# 4F %22 (C%%)
# 5F %23 (LNKARRAY)
# 6F %24 (CHR)
25 # 7F %25 (CODE)
# 8F %16
# 9F %17
# AF %26 (Library Data)
# CF % 30 (Font)
30 # FF % 28 (ZINT)
any %27 (External)
;
SWAPDROP
;
```

在此示例中，可以使用 CK&DISPATCH1 代替 CK&DISPATCH0，因为表中已经明确列出标记对象。

由于列表最后一项是 **any**，其他未列出的任何对象都会返回类型 **27**。

TYPE 属于库，因此原本可以使用命令 CK1&Dispatch。之所以没有使用，是为了节省 ROM 空间。内层复合程序实际上就是系统 RPL 命令 XEQTYPE 的主体。这样，系统 RPL 程序员就可以调用函数返回对象类型，不必承担检查对象是否存在的额外开销，也无需复制分派机制。

请注意这些对象名称。它们是内置 bint 的别名。内置 bint 列表见第 2 章。

29.3 参考

地址	名称	说明
262B0	CK0	(→) 将当前命令保存到 LASTCKCMD。把堆栈第 1 层以下的位置标记到 STACKMARK。
262B5	CK1	(ob → ob) 将当前命令保存到 LASTCKCMD。确认堆栈中至少有一个对象；否则生成“Too Few Arguments”错误。把堆栈标记保存到 STACKMARK。如果启用了 Last Arg，则保存该参数。
262BA	CK2	(ob1 ob2 → ob1 ob2) 与 CK1 类似，但检查是否至少有两个参数。
262BF	CK3	(ob1...ob3 → ob1...ob3) 与 CK1 类似，但检查是否至少有三个参数。
262C4	CK4	(ob1...ob5 → ob1...ob5) 与 CK1 类似，但检查是否至少有四个参数。
262C9	CK5	(ob1...ob5 → ob1...ob5) 与 CK1 类似，但检查是否至少有五个参数。
262CE	CKN	(ob1...obn %n → ob1...obn #n) 检查堆栈第 1 层是否为实数，然后检查是否有该数量的参数，最后把实数转换为 bint。
26292	CK0NOLASTWD	(→) 与 CK0 类似，但不保存当前命令。
地址	名称	说明
26297	CK1NOLASTWD	(ob → ob) 与 CK1 类似，但不保存当前命令。
2629C	CK2NOLASTWD	(ob1 ob2 → ob1 ob2) 与 CK2 类似，但不保存当前命令。
262A1	CK3NOLASTWD	(ob1...ob3 → ob1...ob3) 与 CK3 类似，但不保存当前命令。
262A6	CK4NOLASTWD	(ob1...ob4 → ob1...ob4) 与 CK4 类似，但不保存当前命令。
262AB	CK5NOLASTWD	(ob1...ob5 → ob1...ob5) 与 CK5 类似，但不保存当前命令。
25F25	CKNNOLASTWD	(ob1...obn %n → ob1...obn #n) 与 CKN 类似，但不保存当前命令。
2631E	CK&DISPATCH0	(→) 根据堆栈参数进行分派。
26328	CK&DISPATCH1	(→) 根据堆栈参数进行分派；必要时移除标记并把实数转换为 ZINT。
26323	CK&DISPATCH2	(→) 等同于 CK&DISPATCH1。
26300	CK1&Dispatch	(→) 将 CK1 与 CK&DISPATCH1 组合。
26305	CK2&Dispatch	(→) 将 CK2 与 CK&DISPATCH1 组合。
2630A	CK3&Dispatch	(→) 将 CK3 与 CK&DISPATCH1 组合。
2630F	CK4&Dispatch	(→) 将 CK4 与 CK&DISPATCH1 组合。

地址	名称	说明
26314	CK5&Dispatch	(→) 将 CK5 与 CK&DISPATCH1 组合。
25F9A	0LASTOWDOB!	(→) 清除上一个 CK<n> 命令保存的命令名。 又名: 0LASTOWDOB!、0LastRomWrd!
2EF6C	AtUserStack	(→)
25E9E	CK1NoBlame	:: CK0NOLASTTWD 0LASTOWDOB! ; (→) :: 0LASTOWDOB! CK1NOLASTTWD ;
地址	名称	说明
354CB	'RSAVEWORD	(→) 将实际复合程序上方的复合程序中的第一个对象存入 LASTCKCMD。 又名: 'RSaveRomWrd
26319	EvalNoCK	(comp → ?) 对复合程序求值, 但不将其保存为当前命令。如果第一个命令是 CK<n>&Dispatch, 则用 CK&DISPATCH1 替换; 如果第一个命令是 CK<n>, 则跳过它; 其他任何第一个命令也会被跳过。
25F29	(EvalNoCK:)	运行流: (ob →) 以运行流中的下一个对象作为参数执行 EvalNoCK。
2A9E9	RunRPN:	运行流: (ob →) 在 RPN 模式开启时对运行流中的下一个对象求值, 也就是清除系统标志 95。求值后, 系统标志恢复为原值。

29.3.1 类型检查

地址	名称	说明
36B7B	CKREAL	(% → %) (z → %) 检查是否为实数。如果是 ZINT, 则转换为实数; 否则执行 SETTYPEERR。
184006	^CK1Z	(\$/#/hxs → z) 检查是否为整数。把字符串、bint 或 hxs 转换为 ZINT; 其他对象类型会报错。
185006	^CK2Z	(ob ob' → z z') 与 ^CK1Z 类似, 但用于两个对象。
186006	^CK3Z	(ob ob' ob'' → z z' z'') 与 ^CK1Z 类似, 但用于三个对象。
地址	名称	说明
3D2B4	CKSYMBTYPE	(→) 检查引用名称, 即作为符号对象的名称。
2EF07	nmetasyms	(meta → meta) 检查元对象是否包含%、C%、unit、id、lam 或 symb。
03C64	TYPE	(ob → #prolog) 返回对象前导码的地址。
3BC43	XEQTYPE	(ob → ob %type) 用户命令字 TYPE 的系统版本, 但会保留对象。
3511D	TYPEREAL?	(ob → flag)
35118	DUPTYPEREAL?	(ob → ob flag) 又名: DTYPEREAL?
3512C	TYPECMP?	(ob → flag)
35127	DUPTYPECMP?	(ob → ob flag)
3510E	TYPECSTR?	(ob → flag)
35109	DUPTYPECSTR?	(ob → ob flag) 又名: DTYPECSTR?
35136	DUPTYPEARRY?	(ob → ob flag) 又名: DTYPEARRY?
3513B	TYPEARRY?	(ob → flag ???)
35292	TYPERARRY?	(ob → flag)
352AD	TYPECARRY?	(ob → flag)
35195	TYPELIST?	(ob → flag)
35190	DUPTYPELIST?	(ob → ob flag) 又名: DTYPELIST?
3504B	TYPEIDNT?	(ob → flag)
35046	DUPTYPEIDNT?	(ob → ob flag)
350E1	TYPELAM?	(ob → flag)
350DC	DUPTYPELAM?	(ob → ob flag)
194006	^TYPEIDNTLAM?	(ob → flag) 测试 ob 是否为 ID 或 lam。
2F0D4	(ILnot?)	(ob → ob flag) 测试 ob 是否既不是 ID 也不是 LAM。

地址	名称	说明
35168	TYPESYMB?	(ob → flag)
35163	DUPTYPESYMB?	(ob → ob flag)
350FF	TYPEHSTR?	(ob → flag)
350FA	DUPTYPEHSTR?	(ob → ob flag)
35186	TYPEGROB?	(ob → flag)
地址	名称	说明
35181	DUPTYPEGROB?	(ob → ob flag)
351A4	TYPETAGGED?	(ob → flag)
3519F	DUPTYPETAG?	(ob → ob flag)
351B3	TYPEEXT?	(ob → flag) ob 是否为单位对象?
351AE	DUPTYPEEXT?	(ob → ob flag) ob 是否为单位对象?
3514A	TYPEROMP?	(ob → flag)
35145	DUPTYPEROMP?	(ob → ob flag)
350F0	TYPEBINT?	(ob → flag)
350EB	DUPTYPEBINT?	(ob → ob flag)
35159	TYPERRP?	(ob → flag)
35154	DUPTYPERRP?	(ob → ob flag)
3503C	TYPECHAR?	(ob → flag)
35037	DUPTYPECHAR?	(ob → ob flag)
35177	TYPECOL?	(ob → flag) ob 是否为二级程序?
35172	DUPTYPECOL?	(ob → ob flag) ob 是否为二级程序? 又名: DTYPECOL?
350D2	TYPEAPLET?	(ob → flag)
350CD	DUPTYPEAPLET?	(ob → ob flag)
35087	TYPEFLASHPTR?	(ob → flag)
35082	DUPTYPEFLASHPTR?	(ob → ob flag)
350C3	TYPEFONT?	(ob → flag)
350BE	DUPTYPEFONT?	(ob → ob flag)
350B4	TYPELNGCMP?	(ob → flag)
350AF	DUPTYPELNGCMP?	(ob → ob flag)
350A5	TYPELNGREAL?	(ob → flag)
350A0	DUPTYPELNGREAL?	(ob → ob flag)
35096	TYPEZINT?	(ob → flag)
35091	DUPTYPEZINT?	(ob → ob flag)
182006	^TYPEZ?	(ob → flag)
183006	^DUPTYPEZ?	(ob → ob flag)
114007	^TYPEGAUSSINT?	(ob → flag) 检查 ob 是否为高斯整数。
115007	^DTYPEGAUSSINT?	(ob → ob flag) 检查 ob 是否为高斯整数。
地址	名称	说明
116007	^DUPTYPEGAUSSINT?	(ob → ob flag) 检查 ob 是否为高斯整数。
187006	^CK1Cext	(ob → flag) 检查对象是否为整数或高斯整数。
181006	^CKALG	(ob → ob) 检查对象是否为 real/cmplx/unit 或 idnt/lam/symbolic 类型。
25E77	?OKINALG	(ob → ob flag) 对象是否可用于代数对象中?
171006	^DTYPEFMAT?	(ob → ob flag) 测试对象是否为符号矩阵。
191006	^IDNTLAM?	(ob → ob flag) 测试 ob 是否为 idnt 或 lam。

地址	名称	说明
192006	<code>^FLOAT?</code>	<code>(ob → ob flag) </code> 测试 <code>ob</code> 是否为实数或复数。
195006	<code>^REAL?</code>	<code>(ob → ob flag) </code> 测试 <code>ob</code> 是否为 <code>real</code> 、 <code>zint</code> 或 <code>hxs</code> 。
196006	<code>^TYPEREALZINT?</code>	<code>(ob → flag) </code> 测试 <code>ob</code> 是否为 <code>real</code> 、 <code>zint</code> 或 <code>hxs</code> 。
193006	<code>^CKSYMREALCMP</code>	<code>(ob → ob) </code> 如果 <code>ob</code> 不是实数、复数或符号对象，则引发“Bad Argument Type”错误。

第 30 章 键盘控制

系统 RPL 程序可通过多种方式获取用户输入：

- 从堆栈获取；
- 等待键盘击键；
- 使用内部 INPUT；
- 使用内部 INFORM；
- 建立参数化外层循环；
- 以及其他方法。

您已经了解了如何直接从堆栈获取输入。后续各章将介绍 InputLine、ParOuterLoop 和输入窗口的用法。本章将介绍如何读取键盘击键。

30.1 按键位置

在用户 RPL 中，按键表示形式为%rc.p。在系统 RPL 中，按键由两个二进制整数表示。第一个数通常称为 #KeyCode，范围从 1（F1 键）到 51（ENTER 键），按从左到右、从上到下的顺序表示每个按键。上箭头的代码为 10，被视为第二行的第四个按键。左箭头、下箭头和右箭头的代码分别为 14、15 和 16，被视为第三行的一部分。

第二个数 #Plane 按下表表示修饰键状态：

#Plane	修饰键	#Plane	修饰键
1	无	4	ALPHA
2	左移位	5	ALPHA、左移位
3	右移位	6	ALPHA、右移位

可使用以下词条在两种表示形式之间转换：

```
Ck&DecKeyLoc ( %rc.p → #KeyCode #Plane )
CodePl>%rc.p ( #KeyCode #Plane → %rc.p )
```

有时，移位键不作为其他按键的修饰键处理，而是被当作独立按键。此时它们的键码为 40h（左移位键）、C0h（右移位键）和 80h（ALPHA 键）。

HP48 只有上面列出的六个按键平面。HP49 又引入了五个平面，即移位保持键。这类击键是在按下其他按键时保持按住移位键。在用户 RPL 中，这些按键通过在%rc.p 表示值上加 0.01 来表示。例如，键码 11.21 表示按住左移位键的同时按下 F1 键。

在系统 RPL 中，移位保持键有两种编码方式。第一种方式（下称编码 A）保持键码 #kc 不变，并使用新的平面 #8...#C。第二种方式（编码 B）使用 #1...#6 范围内的平面，并将移位键的键码加到键码 #kc 上。

下表列出了在 HP49G 上按 F1 键的所有方式及其不同编码。

平面	移位键	用户 RPL %rc.pl	A: #kc	A: #pl	B: #kc	B: #pl
1	未移位	11.1	1h	1h	1h	1h
2	左移位	11.2	1h	2h	1h	2h
3	右移位	11.3	1h	3h	1h	3h
4	ALPHA	11.4	1h	4h	1h	4h
5	ALPHA、左移位	11.5	1h	5h	1h	5h
6	ALPHA、右移位	11.6	1h	6h	1h	6h
7	未使用					
8	左移位保持	11.21	1h	8h	41h	2h
9	右移位保持	11.31	1h	9h	C1h	3h
10	ALPHA 保持	11.41	1h	Ah	81h	4h
11	ALPHA、左移位保持	11.51	1h	Bh	41h	5h
12	ALPHA、右移位保持	11.61	1h	Ch	C1h	6h

大多数但并非全部与按键有关的系统 RPL 词条都能处理移位保持击键。参考部分为每个相关词条说明了这一点。以 #kc 和 #pl 为参数的系统 RPL 词条（如 CodePl>%rc 或 Key>StdKeyOb）接受两种编码形式（A 和 B）。返回 #kc 或返回 #kc 和 #pl 的词条（如 Ck&DecKeyLoc 和 GETTOUCH）全都使用编码 B。编码 A 似乎更便于分派。若要把编码 B 转换为编码 A，可使用：

```
:: SWAP 64 #/ ROTSWAP #0=?SKIP #6+ ;
```

30.2 等待按键

WaitForKey 是一个便于等待按键的词条。该命令使 HP49 进入低功耗状态，并等待按键被按下。随后，它把键码返回到堆栈第 2 层，把平面返回到堆栈第 1 层。下面还列出了用于其他情况的词条。

遗憾的是，WaitForKey 不处理移位保持键。因此，下面给出一个行为与 WaitForKey 相同但会返回扩展键码（编码 B）的短程序。该程序包含一个访问按键缓冲区的代码对象，这里不作解释。您可以在示例目录的 keyboard 目录中找到此程序。

```
1 ::
WaitForKey (普通 WaitForKey)
CODE (获取扩展键码)
GOSBVL =SAVPTR
5 D0= 047DF
A=DAT0 A
D0=A
A=0 A
A=DAT0 1
10 A=A-1 P
A=A+A A
CD0EX
C=C+A A
CD0EX
15 D0=D0+ 2
A=DAT0 B
GOVLNG =PUSH#ALOOP

ENDCODE
ROTDROPSWAP (用扩展值替换键码)
20 ;
```

如果希望程序返回编码 A 而不是编码 B，只需将 ROTDROPSWAP 替换为：

```
BINT63 #>case #6+
```

30.3 参考

30.3.1 转换键码

地址	名称	说明
25EA7	Ck&DecKeyLoc	(%rc.p → #kc #p) 将用户按键表示格式转换为系统格式。能处理移位保持键。
25EA9	CodePl>%rc.p	(#kc #p → %rc.p) 将系统按键表示格式转换为用户格式。能处理移位保持键。
25EDC	H/W>KeyCode	(# → #') 将移位键的键码偏移量转换为移位键的键码，即 80h->32d、40h->37d、C0h->42d。
25EEA	ModifierKey?	(#kc #pl → flag) 判断按键是否为右移位键、左移位键或 ALPHA 键这三个修饰键之一。

30.3.2 等待按键

地址	名称	说明
261CA	FLUSHKEYS	(→) 清空按键缓冲区。 又名：FLUSH
地址	名称	说明
04708	CHECKKEY	(→ #kc T) (→ F) 返回按键缓冲区中的下一个按键（如果有），但不将其弹出。能处理移位保持键。
04714	GETTOUCH	(→ #kc T) (→ F) 从按键缓冲区弹出下一个按键（如果有）。能处理移位保持键。
25ED6	GETKEY	(→ #kc flag) 从按键缓冲区取得一次击键，必要时等待。返回按键和 TRUE。如果发生异常，则返回 FALSE；不处理该异常。能处理移位保持键。
25ED7	GETKEY*	(→ #kc T) (→ F F) (→ {Alrmlist} T F) 从按键缓冲区取得一次击键，必要时等待。返回按键和 TRUE。如果发生异常（错误或闹钟），则处理该异常并返回 FALSE。能处理移位保持键。

地址	名称	说明
25ED9	GetKeyOb	(→ ob) 等待一次击键，并返回与该按键关联的对象。能处理移位保持键。
25EC5	DoKeyOb	(ob →) 如同 ob 已赋给某个按键且该键已被按下一样执行 ob。
047C7	REPKEY?	(#kc → flag) 如果该按键正被按住，则返回 TRUE。
25EE3	KEYINBUFFER?	(→ flag) 如果按键缓冲区中至少有一个按键，则返回 TRUE。
25F0B	WaitForKey	(→ #kc #flag) 返回下一次完整击键。不处理移位保持键。
地址	名称	说明
2F268	Wait/GetKey	(% → ?) 内部 WAIT 命令。不处理移位保持键。

30.3.3 ATTN 标志

地址	名称	说明
25FAE	ATTN?	(→ flag) 如果已按下 CANCEL，则返回 TRUE。
25E70	?ATTNQUIT	(→) 如果已按下 CANCEL，则对程序执行 ABORT。又名：?ATTN_QUIT
25E9D	CK0ATTNABORT	(→) 由用户 RPL 程序定界符 x« 和 x» 以及 xUNTIL 执行。主要只是执行?ATTNQUIT。
25EED	NoAttn?Semi	(→) 如果尚未按下 CANCEL，则丢弃运行流的其余部分。
05040	ATTNFLG@	(→ #) 提取 CANCEL 键计数器。
05068	ATTNFLGCLR	(→) 清除 CANCEL 键计数器。不影响按键缓冲区。

30.3.4 无效按键

地址	名称	说明
25EBF	DoBadKey	(→) 发出蜂鸣声。
25ECD	DropBadKey	(ob →) 发出蜂鸣声。
25E6E	2DropBadKey	(ob ob' →) 发出蜂鸣声。

30.3.5 用户键

如果没有任何按键赋值，内部按键赋值列表就是空列表。如果存在一个或多个赋值，该列表包含 51 个子列表，每个子列表代表一个按键。每个子列表要么为空，表示该按键没有赋值；要么包含 12 个元素，每个元素代表一个平面的赋值。各平面见第 30.1 节中的表。没有赋值的平面必须用空列表表示。第七个列表始终为空。

地址	名称	说明
25F09	UserKeys?	(→ flag) 执行 BINT62 TestSysFlag。
25967	GetUserKeys	(→ {}) 返回用户键列表（内部格式）。
2F3B3	(AsnKey)	(ob #kc #p →) 将对象赋给以系统格式指定的按键。
25621	(NonUsrKeyOK?)	(→ flag) 如果未定义的按键会执行其正常操作，则返回 TRUE。
25617	(SetNUsrKeyOK)	(→) 使未定义的按键执行其正常操作。
2561C	(ClrNUsrKeyOK)	(→) 使未定义的按键在按下时只发出蜂鸣声。
25EE5	Key>StdKeyOb	(#kc #pl → ob) 提取按键的标准赋值。这是在 USER 模式关闭时生效的赋值。
25EE6	Key>U/SKeyOb	(#kc #pl → ob) 如果 USER 模式开启，则提取赋给按键的用户对象；如果 USER 模式关闭，则改为提取标准赋值。
255006	^KEYEVAL	(% → ?) 对击键求值。如果%为负数，则始终对标准按键赋值求值。

第 31 章

使用 InputLine

InputLine 命令是用户命令 INPUT 在系统中的等效词条。两者用法相似，作用也相近：

- 在屏幕顶部显示提示；
- 启动键盘输入模式；
- 初始化编辑行；
- 接受输入，直到按下 ENTER；
- 解析输入、对输入求值，或直接返回用户输入；
- 如果通过 ENTER 退出环境，则返回 TRUE；如果通过 ON/CANCEL 中止，则返回 FALSE。

堆栈中必须包含以下参数：

名称	说明
\$Prompt	输入期间要显示的提示。
\$EditLine	初始编辑行。
CursorPos	初始光标位置。可以按绝对字符位置指定，也可以用包含行号和列号的双元素列表指定。在两种情况下，#0 都表示编辑行、行或列的末尾。所有数字自然都应指定为 bint。
#Ins/Rep	光标的初始插入 / 替换模式： • #0 当前模式 • #1 插入模式 • #2 替换模式
名称	说明
#Entry	初始输入模式： • #0 当前输入模式加程序输入模式 • #1 仅程序输入模式 • #2 程序输入模式和代数输入模式
#AlphaLock	初始 ALPHA 模式： • #0 当前模式 • #1 启用 ALPHA • #2 禁用 ALPHA
ILMenu	初始菜单，采用下文指定的格式。通常指定为 FALSE，表示不更改菜单。
#ILMenu	初始菜单行号（通常为 BINT1，用于显示第一页）。
AttnAbort?	标志： • TRUE：CANCEL 中止输入 • FALSE：CANCEL 只清空编辑行
#Parse	编辑行的处理方式： • #0 将编辑行作为字符串返回（不求值） • #1 将编辑行作为字符串返回，并返回解析后的对象 • #2 解析编辑行并求值

如果 AttnAbort? 为 TRUE，用户在 InputLine 活动期间按下 CANCEL 时，编辑操作会中止。如果它为 FALSE，CANCEL 只会清空编辑行。如果编辑行原本已经为空，则会中止 InputLine 并返回 FALSE。

根据 #Parse 的值，将按照下表返回不同的值：

#Parse	堆栈	说明
#0	\$Editline TRUE	仅编辑行（不求值）
#1	\$Editline obs TRUE	编辑行和解析后的对象
#2	obl ... obn TRUE	求值所得对象
	FALSE	按下 CANCEL 中止

31.1 菜单键赋值

任何应用程序都可以通过 ILMenu 参数指定初始菜单。InputLine 启动时会显示此菜单。所有菜单键都可以在未移位、左移位和右移位平面上设置赋值。循环退出时，会完整恢复原来的菜单。

ILMenu 参数是一个列表；在少数情况下，也可以是一个返回列表的程序。其格式见第 37.1 节。如果不想更改当前菜单，也可以直接把此参数指定为 FALSE。

请注意，各操作必须以词条 TakeOver 开头，用于标记这些操作应在命令行处于活动状态时运行。

31.2 示例

下面是一个 InputLine 示例。它会提示您输入姓名；如果编辑操作未被中止，就显示该姓名。

```
1 ::
$ "Your name:" (提示)
NULL$ (初始编辑行)
#ZERO#ONE (光标位于末尾，插入模式)
5 ONEONE (程序模式，启用 ALPHA)
NULL{} (无菜单)
ONE (菜单行)
FALSE (CANCEL 清空编辑行)
ZERO (返回字符串)
10 InputLine
NOT?SEMI (若为 FALSE 则退出)
$ "Your name is "
SWAP&$ (拼接字符串与姓名)
CLEARLCD (清空屏幕)
15 DISPROW1 (在第 1 行显示字符串)
SetDAsTemp (冻结屏幕)
;
```

31.3 参考

地址	名称	说明
2EF5F	InputLine	(args → \$ T) (args → \$ obl..obn T) (args → obl..obn T) (args → F) args = \$pr \$line #pos #l/R #l/A #alph menu #row atn #parse
2F154	(input\$)	(\$1 \$2 → \$3) 当堆栈第 1 层为字符串时，用户命令 INPUT 执行的就是这个词条。
2F155	(input{})	(\$1 {} → \$3) 当堆栈第 1 层为列表时，用户命令 INPUT 执行的就是这个词条。

第 32 章 参数化外层循环

参数化外层循环是一种系统 RPL 结构，可用来创建完整的应用程序：它接收击键，并根据按下的按键执行不同操作。这一过程会按需要重复，直到出现退出条件。大多数情况下，会设置一个用于停止循环的按键，例如 CANCEL 或 DROP。通常，它用于处理屏幕显示的程序。POL 的复杂用法包括输入窗口（第 35 章）和浏览器（第 33 章、第 34 章）。请注意，POL 是一种非常通用的结构，因此需要较为复杂的参数。简单应用程序有时可以用围绕 WaitForKey（第 30.2 节）的循环配合直接处理屏幕显示来实现，这种方式更简单、也更紧凑。

建立参数化外层循环需要九个参数：

参数名称	说明
AppDisplay	每次对按键求值前都会对该对象求值。它应负责处理按键本身未完成的屏幕更新，并对错误作特殊处理。
AppKeys	实体键分配，格式见下文。
NonAppKeyOK?	标志：如果为 TRUE，则未由应用程序分配的实体键会执行其正常操作；否则只发出蜂鸣声。
DoStdKeys?	标志：如果为 TRUE，则对非应用程序按键使用标准按键定义，而不使用默认按键处理。
AppMenu	菜单规格，采用第 37.1 节所述格式；也可以指定 FALSE，以保持当前菜单不变。
#AppMenuPage	初始菜单页。通常为 BINT1，用于显示第一页。
参数名称	说明
SuspendOK?	标志：如果为 TRUE，任何会创建挂起环境并重新启动系统外层循环的用户命令都会改为引发错误。
ExitCond	每次更新屏幕以及对按键求值前，都会对该对象求值。如果结果为 TRUE，则退出循环。
AppError	按键求值期间发生错误时，要对其求值的错误处理对象。

参数设置完毕后，调用 ParOuterLoop。该词条本身不生成任何结果，但任何按键赋值都可以按需要把结果返回到堆栈，或以其他方式返回结果。

32.1 参数化外层循环词条

参数化外层循环通过调用以下词条构成，并配有适当的错误处理。这些词条都不返回任何内容；只有 POLSetUI 取得参数，即 ParOuterLoop 所需的同一组九个参数。

词条	操作
POLSaveUI	将当前用户界面保存在临时环境中。
POLSetUI	根据所给参数设置当前用户界面。
POLKeyUI	显示、读取按键并对按键求值；处理错误，并根据 POLSetUI 所指定的用户界面退出。
POLRestoreUI	恢复 POLSaveUI 保存的用户界面，并放弃临时环境。
POLResUI&Err	恢复用户界面并重新引发错误。用于参数化外层循环内未处理的错误。

词条 ParOuterLoop 反编译为：

```
1 ::
POLSaveUI (保存当前用户界面)
ERRSET :: (开始错误陷阱)
POLSetUI (设置新用户界面)

5 POLKeyUI (处理击键)
;
ERRTRAP
POLResUI&Err (若发生错误，则恢复)
(保存的用户界面与错误)
10 POLRestoreUI (恢复保存的用户界面)
;
```

如果不使用 ParOuterLoop 而直接使用上述词条，必须提供与上面代码同等级别的错误保护。

需要注意：参数化外层循环在保存当前用户界面时会创建临时环境，在恢复保存的用户界面时会放弃该环境。这意味着，不能在循环内使用 1GETLAM 等对最顶层临时环境进行操作的词条，除非变量是在调用 POLSaveUI 后创建，并在调用 POLRestoreUI 前放弃。对于调用 POLSaveUI 前创建的临时环境，应使用具名临时变量。

32.2 屏幕显示

在参数化外层循环中，用户负责设置和更新屏幕显示；系统不提供默认显示。

屏幕可通过两种方式更新：使用参数 “AppDisplay”，或使用按键赋值。例如，当用户按下按键移动光标时，按键赋值可以向 “AppDisplay” 传递信息（往往是隐式传递），由它负责更新屏幕；也可以由按键赋值对象自行处理显示。哪种方法效率更高取决于具体情况。在下面的示例中，AppKeys 只在 lam 中设置 grob 的位置，AppDisplay 则绘制该 grob。

32.3 错误处理

如果按键处理期间发生错误，就会执行 AppError。该对象负责处理参数化外层循环运行期间生成的任何错误。AppError 应确定具体错误并采取相应操作。也可以直接把 ERRJMP 指定为 AppError，这表示应用程序不处理任何错误。

32.4 实体键赋值

在参数化外层循环中，六个基本平面（见第 30.1 节）上的任何按键都可以被赋予新功能。参数 AppKeys 指定要分配的按键及其操作。

如果某按键未由应用程序分配，且参数 NonAppKeyOK? 为 TRUE，那么当参数 DoStdKeys? 为 TRUE 时会执行标准按键定义；当该参数为 FALSE 时，则执行 USER 键赋值（如果存在）。如果 NonAppKeyOK? 为 FALSE，则发出警告蜂鸣声，除此之外不执行任何操作。

大多数情况下，NonAppKeysOK? 应设为 FALSE。

AppKeys 参数是一个二级程序，必须以键码和平面为参数，并返回所需的按键定义和 TRUE；如果应用程序不处理该按键，则返回 FALSE。具体堆栈图如下：

```
( #KeyCode #Plane → KeyDef TRUE )
( #KeyCode #Plane → FALSE )
```

建议按以下形式设置按键赋值：

```
1 BINT1 #=casedrop :: (处理未移位平面) ;
BINT2 #=casedrop :: (处理左移位平面) ;
...
2DROPFALSE
```

每个平面的处理程序通常采用以下形式：

```
1 BINT7 ?CaseKeyDef :: TakeOver <process APPS key> ;
BINT9 ?CaseKeyDef :: TakeOver <process TOOL key> ;
...
DROPFALSE
```

?CaseKeyDef 在这种情况下非常实用，因为它等效于 #=casedrop :: ' <keydef> TRUE ；。使用此词条可以缩短代码，并提高定义的可读性。?CaseKeyDef 的用法如下：

```
... #KeyCode #TestKeyCode ?CaseKeyDef <keydef> ...
```

如果 #TestKeyCode 等于 #KeyCode，?CaseKeyDef 会丢弃这两个值，将 <KeyDef> 和 TRUE 压入堆栈，然后退出二级程序。否则，它只丢弃 #TestKeyCode，跳过 <KeyDef> 并继续执行。

如果要处理按住移位键的组合键，也可以做到。扩展键码（编码 B，见第 30.1 节）会传给 AppKeys 程序的堆栈第 5 层和第 6 层。只需让 AppKeys 以下列代码片段开头：

```
4DROP 2DUP 2DUP
```

然后照常进行分派即可。

32.5 菜单键赋值

您可以指定参数化外层循环启动时显示的菜单。**AppMenu** 参数的格式基本上与 **InputLine** 的 **ILMenu** 参数相同，后者见第 37.1 节。

区别在于，此处不需要 **TakeOver**，因为输入行未处于活动状态。

另外，由于实体键分配的优先级高于菜单键分配，因此应在 **AppKeys** 参数的每个按键平面定义中加入以下代码：

```
DUP#<7 casedrpfls
```

按下键码小于 7 的按键（即某个软键）时，这段代码会压入 **FALSE**。**FALSE** 将强制执行标准分配，而该分配会执行 **AppMenu** 参数定义的操作。

为使其生效，**NonAppKeysOK?** 参数必须为 **TRUE**，这样菜单键才能按正常方式工作，即执行 **AppMenu** 参数指定的操作。

32.6 防止挂起环境

您的应用程序可能需要对任意命令和用户参数求值，但不希望当前环境被 **HALT** 或 **PROMPT** 命令挂起。**SuspendOK?** 参数为 **FALSE** 时，会取消这些命令以及其他任何会挂起环境的命令，并生成“**HALT Not Allowed**”错误，**AppError** 可处理该错误。如果参数为 **TRUE**，应用程序必须做好处理其后果的准备。正如 **RPLMAN.DOC** 所写：“这里的危险很多，而且很严重。”

几乎所有应用程序都应把 **SuspendOK?** 参数设为 **FALSE**。

32.7 退出条件

ExitCond 参数是一个在每次按键求值之前求值的对象。如果求值结果为 **TRUE**，循环就会退出；否则继续运行。例如，可以把 **ExitCond** 定义为 **' LAM exit**。按下“**quit**”键时，只需执行 **TRUE ' LAM exit STO**，循环就会退出。自然，必须事先创建该 **lam**，并用 **FALSE** 对其初始化。

32.8 示例

下面的程序是一个使用参数化外层循环的应用程序示例。它创建了一个环境，用户可以在屏幕上移动一个小图形。既可使用方向键移动，也可使用菜单键移动。两种情况下，如果先按左移位键，图形都会一次移动 10 步，而不是 1 步。代码还会确保图形不超出屏幕边界。

下面的图 32.1 显示了该程序运行时的画面。



图 32.1：POL 示例

```
1 ::
* 为所用按键定义名称，使代码更简洁，
* 也更易读
DEFINE kpNoShift BINT1
5 DEFINE kpLeftShift BINT2
DEFINE kcUpArrow BINT10
DEFINE kcLeftArrow BINT14
DEFINE kcDownArrow BINT15
```

```

DEFINE kcRightArrow BINT16
10 DEFINE kcLeftShift BINT37
DEFINE kcOn BINT47
* 无需参数
CKONOLASTWD
15

* 准备屏幕显示
RECLAIMDISP (清空屏幕并调整显示区大小)
ClrDA1IsStat (暂时禁用时钟)
20 * 笑脸 grob。下一行必须只占一行。
GROB 7C 310003100008F000060300810C004000104000102000202401201000401000401000401000401104402104202J
↵ 6032048F010400010810C0006030008F000
FIFTYSIX (框的初始 x 坐标)
25 EIGHTEEN (框的初始 y 坐标)
FALSE (初始退出条件)
{
  LAM MrSmile
  LAM x
  30 LAM y
  LAM exit?
} BIND (绑定局部变量)
* 以下复合程序是显示更新对象。
35 * 它会清空屏幕并绘制笑脸 grob。
,
::
CLEARVDISP (清空屏幕)
LAM MrSmile (提取笑脸 grob)
HARDBUFF (提取当前显示缓冲区)
40 LAM x LAM y (笑脸坐标)
GROB! (REPL)
DispMenu.1 (显示菜单)
;
45 * 以下复合程序是按键操作处理程序。
,
::
kpNoShift #=casedrop ::
DUP#<7 casedrpfls (启用软键)
kcUpArrow ?CaseKeyDef
50 ::
LAM y DUP
BINT1 #<ITE
:: DROP ERBEEP ;

:: #1- ' LAM y STO ;
55 ;
kcDownArrow ?CaseKeyDef
::
LAM y DUP
BINT36 #>ITE
60 :: DROP ERBEEP ;
:: #1+ ' LAM y STO ;
;
kcLeftArrow ?CaseKeyDef
::
65 LAM x DUP
BINT1 #<ITE
:: DROP ERBEEP ;
:: #1- ' LAM x STO ;
;
70 kcRightArrow ?CaseKeyDef
::
LAM x DUP
BINT111 #>ITE
:: DROP ERBEEP ;
75 :: #1+ ' LAM x STO ;
;
kcOn ?CaseKeyDef
:: TRUE ' LAM exit? STO ;
kcLeftShift #=casedrpfls
80 DROP ' DoBadKeyT
;
kpLeftShift #=casedrop ::
DUP#<7 casedrpfls (启用软键)
kcUpArrow ?CaseKeyDef
85 ::
LAM y DUP
BINT10 #<ITE
:: DROPZERO ERBEEP ;
:: BINT10 #- ;

```

```

90 ' LAM y STO
;
kcDownArrow ?CaseKeyDef
::
LAM y DUP
95 BINT27 #>ITE
:: DROP BINT27 ERBEEP ;
#10+
' LAM y STO

;
100 kcLeftArrow ?CaseKeyDef
::
LAM x DUP
BINT10 #<ITE
:: DROPZERO ERBEEP ;
105 :: BINT10 #- ;
' LAM x STO
;
kcRightArrow ?CaseKeyDef
::
110 LAM x DUP
BINT102 #>ITE
:: DROP BINT112 ERBEEP ;
#10+
' LAM x STO
115 ;
kcLeftShift #=caserpfls
DROP ' DoBadKeyT
;
2DROP ' DoBadKeyT
120 ;
* 按键定义
TrueTrue
125 * 菜单定义
{ { "Up" {
::
LAM y DUP
BINT1 #<ITE
130 :: DROP ERBEEP ;
:: #1- ' LAM y STO ;
;
::
LAM y DUP
135 BINT10 #<ITE
:: DROPZERO ERBEEP ;
:: BINT10 #- ;
' LAM y STO
;
140 }
}
{ "Down" { ::
LAM y DUP

BINT36 #>ITE
145 :: DROP ERBEEP ;
:: #1+ ' LAM y STO ;
;
::
LAM y DUP
150 BINT37 #>ITE
:: DROP BINT37 ERBEEP ;
#10+
' LAM y STO
;
155 }
}
{ "Left" { ::
LAM x DUP
BINT1 #<ITE
160 :: DROP ERBEEP ;
:: #1- ' LAM x STO ;
;
::
LAM x DUP
165 BINT10 #<ITE

```

```
:: DROPZERO ERBBEEP ;
:: BINT10 #- ;
' LAM x STO
;
170 }
}
{ "Right" { ::
LAM x DUP
BINT111 #>ITE
175 :: DROP ERBBEEP ;
:: #1+ ' LAM x STO ;
;
::
LAM x DUP
180 BINT102 #>ITE
:: DROP BINT112 ERBBEEP ;
#10+
' LAM x STO
;
185 }
}
NullMenuKey
{ "Quit" :: TRUE ' LAM exit? STO ; }

}
190 ONEFALSE (第一菜单行, 不允许挂起环境)
' LAM exit? (退出条件)
' ERRJMP (错误处理程序)
ParOuterLoop (运行参数化外层循环)
RECLAIMDISP (调整显示区大小并清空屏幕)
195 ClrDAsOK (重绘屏幕)
;
```

32.9 参考

地址	名称	说明
2B475	ParOuterLoop	(Disp Keys NonAppKeys? DoStdKeys? menu #row suspendOK? ExitCond AppErr →)
2B4AC	POLSaveUI	(Disp Keys NonAppKeys? DoStdKeys? menu #row suspendOK? ExitCond AppErr →) 将当前 UI 保存到 LAMSavedUI。
2B542	POLSetUI	<see>ParOuterLoop 设置新 UI, 参数与 ParOuterLoop 相同。
2B628	POLKeyUI	(→) 根据已设置的 UI 显示、读取按键并对其求值。
2B6CD	POLRestoreUI	(→) 从 LAMSavedUI 恢复保存的 UI。
2B6B4	POLResUI&Err	(→) 恢复保存的 UI 并执行 ERRJMP。
29F25	AppDisplay!	(ob →)
29F35	AppDisplay@	(→)
29F55	AppKeys!	(ob →)
29F75	AppKeys0	???
2A055	AppExitCond!	(ob →)
2A065	AppExitCond@	(→ ob)
2A145	AppError!	(ob →)
2A158	AppError@	(→ ob)
地址	名称	说明
25690	AppMode?	(→ flag) 当前是否有 POL 处于活动状态?
25695	SetAppMode	(→)
2569A	ClrAppMode	(→)
2564D	SetNAppKeyOK	(→)
2565A	DoStdKeys?	(→ flag)
2565F	SetDoStdKeys	(→)
25F04	SuspendOK?	(→ flag) 当前用户界面是否允许挂起?

地址	名称	说明
27E72	nohalt	(→ ob) :: LAM 'nohalt ;
25671	SetAppSuspOK	(→)
25676	ClrAppSuspOK	(→)

第 33 章

使用 HP49 浏览器

浏览器是用户 RPL 命令 CHOOSE 所创建选择框的底层引擎。不过，它的能力远不止该命令所提供的功能。HP49G 计算器中有两个浏览器引擎：旧引擎自 HP48G 系列起就已存在，新引擎则仅见于 HP49G 型号。本章将介绍更易于使用的新引擎。新引擎具有旧引擎没有的一些功能，但旧引擎也具有新引擎没有的一些重要功能，例如全屏模式 and 多项目选择。下一章将介绍旧引擎。

有多个闪存指针可用于访问浏览器引擎。这些闪存指针不受官方支持，但很可能保持稳定。

它与用户 RPL 的 CHOOSE 命令之间的主要区别在于，您可以指定消息处理程序，用它提供自定义菜单、处理击键以及执行其他一些操作。

主入口为 FPTR 2 72 (^Choose3)。其堆栈效果如下：

```
( meta $title #initial ::message → ob TRUE )  
或  
( meta $title #initial ::message → FALSE )
```

返回哪一种结果，取决于用户是选择了项目还是取消了操作。

也可以用 FPTR 2 74 代替 FPTR 2 72。区别在于，该入口不会在虚拟堆栈中保存原始 meta 的副本，并且返回的是索引而不是所选对象。索引从 0 开始，而不是从 1 开始。

33.1 选择项元对象

meta 是一个元对象（见第 12 章），其中包含应在选择框中显示的项目。所有对象类型均可使用，显示时会对其进行反编译。

33.2 标题字符串

\$title 是标题。它会显示在选择框顶部的小框中。如果它是空字符串，则不显示标题。当选择框的内容不需要进一步说明时，这一点很有用。省略标题可以为额外一行项目腾出空间。

33.3 初始所选项目

选择引擎启动时，已经有一个项目处于亮选状态。通常是第一个项目，但也可以用 #initial 参数选择其他项目。编号从 0 开始，而不是从 1 开始。

33.4 消息处理程序

::message 是一个程序，即消息处理程序。消息处理程序是一种向应用程序传递数量可变的可选参数的通用方法。应用程序会把不同的“消息”（通常是 bint）放在堆栈第 1 层，并可能把附加参数放在其他堆栈层，然后调用消息处理程序。处理程序可以决定是否处理该消息。若要处理，就应完成相应工作并返回 TRUE；若要忽略，就应丢弃该 bint 并返回 FALSE。因此，空消息处理程序就是命令 DROPFALSE（可用'DROPFALSE 很方便地把它压入堆栈）。使用用户命令 xCHOOSE 时，它只会提供 DROPFALSE，并把处理转交给更通用的引擎。

消息处理程序可以处理以下消息：

消息	消息名称及含义
BINT1	MsgDispBox 此消息与选择框的显示有关，目前还没有得到充分理解。该消息的消息处理程序堆栈效果似乎为： (#1 → ::prog TRUE) (#1 → FALSE)
BINT2	MsgDispTitle 此消息应显示标题。如果不处理，则使用给定的 \$title 参数绘制标题。 (#2 → TRUE) (#2 → FALSE)
BINT3	MsgEndInit 此消息在选择框初始化之后、控制权交给 POL 之前执行。 (#3 → TRUE) (#3 → FALSE)

消息	消息名称及含义
BINT4	MsgKeyPress 这是一个按键处理程序，与 POL 使用的处理程序类似。用户按键时，会以键码、按键平面（见第 30.1 节）和堆栈中的消息 BINT4 调用消息处理程序。它应返回按键定义（地址或二级程序）、TRUE，并再返回一个 TRUE。 如果不处理该按键，则必须返回 FALSE。该消息的消息处理程序堆栈效果如下： (#kc #pl #4 → KeyDef TRUE TRUE) 确实，TRUE 要返回两次！ (#kc #pl #4 → FALSE)
BINT5	MsgMenu 此消息必须返回选择过程中向用户显示的菜单。系统会对该消息的返回值求值，以取得菜单。移动亮选项目时，菜单不会自动更新，但可以用消息 #6 强制更新。如果菜单有多页，就必须在按键处理程序中处理 NXT 和 PREV 键，因为默认情况下不会处理它们。 (#5 → { menu list } TRUE) (#5 → ::prog_returning_list TRUE) (#5 → FALSE)
消息	消息名称及含义
BINT6	MsgEndEndDisp 选择框重新显示完成后（因为您更改了所选项目），会调用此消息。可将 24LAM 设为 FALSE，强制更新菜单显示。请参见下面的示例。 (#6 → TRUE) (#6 → FALSE)

33.5 浏览器与 lam

浏览器 POL 使用 24 个匿名局部变量，因此您最好不要依赖自己的匿名局部变量，而应使用命名局部变量。浏览器引擎使用的一些重要匿名 LAM 如下：

LAM	内容
1LAM	Quit。若要让 POL 退出，请将其设为 TRUE。
2LAM	DispOffset。所选项目相对于 DispTop 的索引。
3LAM	DispTop。当前屏幕上可见的第一个选择项目的索引。
17LAM	消息处理程序。
14LAM	重新显示程序。
24LAM	DisplayMenu。将其设为 FALSE 可强制重新显示菜单。

33.6 访问所选项目

要让浏览器执行选择项目以外的工作，就必须编写可通过按键处理程序或菜单调用的程序。这些程序最重要的任务之一，就是确定当前项目是什么。选择框引擎会在虚拟堆栈中保留选择列表的两个副本，您可以用它们取得当前项目。虚拟堆栈第 1 层中的列表顺序已经反转，选择框中已经显示过的项目会转换为字符串（受标志 -85 影响）。虚拟堆栈第 3 层中保存着原始列表的副本。可用以下代码片段取得当前项目的索引：

```
:: 2GETLAM 3GETLAM #+ ;
```

索引从 0 开始。

可使用以下方法之一访问当前项目：

- 1. 获取反编译后的字符串。这种方法速度很快，而且只需 8 字节。

```
:: 2GETLAM 3GETLAM #+ GetElemBotVStack ;
```

- 2. 获取原始项目。没有受支持的方法可直接访问虚拟堆栈第 3 层，因此必须把它挖出来，并在随后恢复堆栈。下面是一种实现方式（35 字节）：

```
1 ::  
GetVStackProtectWord PopMetaVStack  
GetVStackProtectWord PopMetaVStack  
2GETLAM 3GETLAM #+  
5 GetElemTopVStack  
1LAMBIND  
PushMetaVStack&Drop SetVStackProtectWord  
PushMetaVStack&Drop SetVStackProtectWord  
1GETABND  
10 ;
```

这段代码看起来很复杂，但速度也相当快，而且 ROM 中目录的 Help 键实际使用的就是它。

3. 如果觉得方法 2 太长，可以保留原始列表的副本，例如保存在名为“mylist”的 LAM 中。如果在调用浏览器闪存指针之前这样做，就可用以下代码取得当前项目：

```
:: LAM mylist 2GETLAM 3GETLAM #+ #1+ NTHCOMPDROP ;
```

33.7 保存和恢复屏幕

如果要通过菜单键或其他按键暂时离开选择框，执行某个会占用屏幕的操作，就必须在该操作前后保存并恢复当前屏幕。这是因为浏览器 POL 只对屏幕进行尽可能少的更新；从临时操作返回时，屏幕会显示异常且无法自行恢复。有两个简单的闪存指针可用于保存和恢复屏幕：

FPTR 2 88 保存当前屏幕。

FPTR 2 89 恢复保存的屏幕。

请注意，这些命令使用特定的存储位置，因此不能用于嵌套选择框，即一个选择框创建另一个选择框，而后者又需要为临时操作保存屏幕。在这种情况下，需要保存并恢复 HARDBUFF 和 HARDBUFF2 的副本。

33.8 示例

下面给出浏览器引擎的应用示例。该程序显示数字 1-100 供多选，并返回由所有所选值组成的列表。按平方根键会在消息框中显示当前数字的平方根。在菜单中，按 F1 会把当前所选数字的反编译形式（一个字符串！）加入返回列表。按 F2 会显示有关选择框的帮助文本。另有一个 F3 菜单键，它本身不执行任何操作，但会显示所选数字是偶数还是奇数。由于该显示会变化，需要使用消息 6 强制更新菜单。F5 和 F6 分别执行常规 CANCL 和 OK 操作。

图 33.1 显示了该程序运行时的画面。



图 33.1: 49 浏览器示例

```
1 ::
101 ONE_DO
INDEX@ UNCOERCE
LOOP
5 100 P{}N (创建包含 1-100 的列表)
DUP

NULL{} (用于收集结果的空列表)
{ LAM mylist LAM res } BIND (保存列表副本)
INNERCOMP (为 FPTR 2 72 展开)
10 "REALS" (标题)
0 (初始位置)
,

:: (按键处理程序)
4 OVER#=case
15 ::
DROP DUP#1= 3PICK 23 #=-
ANDcase (按下 SQRT 键)
::
2DROP (丢弃键码)
20 '
::
LAM mylist
2GETLAM 3GETLAM
#+ #1+
25 NTHCOMPDROP (获取当前值)
%SQRT DO>STR (计算平方根)
```

```

FlashWarning (显示)
;
TrueTrue (是, 处理此按键)
30 ;
FALSE (不处理其他按键)
;
5 OVER#=case (提供菜单)
::
35 DROP
' ::
NoExitAction (不保存为 LastMenu)
{ { "->{}" (" 添加到列表" 菜单键)
:: TakeOver
40 LAM res (获取当前列表)
2GETLAM (以字符串形式获取元素)
3GETLAM #+
GetElemBotVStack
>TCOMP (加入列表)
45 ' LAM res STO (存储当前列表)
; }
{ "?" (" 帮助" 菜单项)
:: TakeOver
FPTR 2 88 (保存当前屏幕)
50 DOCLLCD (清空屏幕)
ZEROZERO (接下来是帮助文本)

"->{} ADD
? HELP
SQRT DISP ROOT"
55 $>GROBCR
XYGROBDISP (显示帮助文本)
WaitForKey (等待任意按键)
2DROP
FPTR 2 89 (恢复屏幕)
60 ;
}
{ :: (要显示的按钮)
TakeOver ("even" 或 "odd")
LAM mylist (列表)
65 2GETLAM
3GETLAM #+ #1+ (获取当前元素)
NTHCOMPDROP
DUP (测试是否为偶数)
%2 %/
70 %FLOOR
%2 %* %= ITE
"even" "odd" (返回正确标签)
;
NOP (按下时不执行任何操作)
75 }
NullMenuKey (第 4 个菜单键为空)
{ "CANCL" (默认 CANCL 操作)
FPTR 2 77 }
{ "OK" (默认 OK 操作)
80 FPTR 2 76 }
}
;
TRUE (是, 提供菜单)
;
85 6 OVER#=case (强制更新菜单)
:: DROP FalseFalse
24 PUTLAM ;
DROFFALSE (其他消息)
; (不处理)
90 FPTR 2 72 (运行 CHOOSE 引擎)
ITE
::
DROP (丢弃当前值)
LAM res (返回列表)
95 TRUE (压入 TRUE)
;

```

```
FALSE (CANCL: 返回 FALSE)
ABND (释放局部变量)
;
```

33.9 参考

地址	名称	说明
072002	(^Choose3)	(meta \$title #pos ::handler → ob T) (meta \$title #pos ::handler → F) 主选择引擎。
074002	(^Choose3Index)	(meta \$title #pos ::handler → #idx T) (meta \$title #pos ::handler → F) 与 ^Choose3 相同，但返回所选项目的索引，而不是项目本身。#idx 从 0 开始。
070002	(^Choose2)	(meta \$title #pos → ob T) (meta \$title #pos → F) 使用空消息处理程序调用 Choose3Index。这实际就是： :: 'DROPFALSE FPTR2 ^Choose3Index ;
073002	(^Choose3Save)	(meta \$title #pos ::handler → ob T) (meta \$title #pos ::handler → F) 在调用 Choose3 前后保存并恢复 HARDBUFF/2。
005002	(^sysCHOOSE)	(\$title {} %sel → ob %1) (\$title {} %sel → %0) 等效于用户 RPL 命令 CHOOSE。
075002	(^ChooseDefHandler)	(→ ::handler) 将默认消息处理程序（CAT 键使用的处理程序）压入堆栈。
地址	名称	说明
088002	(^SaveHARDBUFF)	(→) 将 HARDBUFF 和 HARDBUFF2 保存到安全位置。
089002	(^RestoreHARDBUFF)	(→) 恢复用 SaveHARDBUFF 保存的 HARDBUFF 和 HARDBUFF2。
077002	(^Choose3OK)	(→) 按下 OK 或 ENTER 时，由 Choose3 执行的 OK 操作。
076002	(^Choose3CANCL)	(→) 按下 CANCL 或 ON 时，由 Choose3 执行的 CANCEL 操作。

第 34 章

使用 HP48 浏览器

HP48 浏览器（在 HP49 中仍然保留）可以完成许多操作。基本上，它显示一个项目列表，您可以从中选择一个或多个项目（新式 HP49 浏览器则只允许选择一个项目），并通过菜单键或实体键赋值对这些项目执行操作。这个“旧”引擎具有 HP49 浏览器所没有的一些功能，例如全屏模式。不过，它的用法也更复杂。与窗口引擎（见第 35 章）一样，它具有数千种功能，通常也有多种方法可以完成同一件事。调用浏览器的词条是 Choose。它要求堆栈中有五个参数。根据退出方式（后文详述），它随后会返回结果和 TRUE，或者仅返回 FALSE。堆栈图如下：

```
( ::Appl $Title ::Converter {}Items Init → result TRUE ) 或  
( ::Appl $Title ::Converter {}Items Init → FALSE )
```

result 可以是列表，也可以是单个对象，取决于是否启用了 对勾和多项选择。

34.1 ::Appl 参数

这是一个程序，可用于配置浏览器的多个方面。其工作方式与其他消息处理程序相同：调用时，堆栈中会有一个表示消息代码的 bint。如果程序处理该消息，就应返回该消息所需的全部数据和 TRUE；否则返回 FALSE。这意味着 DROPFALSE（可用命令'DROPFALSE 将其压入堆栈）可作为此参数的有效值；它表示不处理任何消息，并始终使用默认值。

下面说明其中一些消息：

代码（十进制）	说明及堆栈效果
57	浏览器将在屏幕上显示的行数。默认值取决于当前字体以及系统标志 90。 (→ #)
58	浏览器行的高度。一般不需要更改。 (→ #)
59	浏览器行的宽度。如果项目数可能大于页面容量，请为箭头显示留出空间。 (→ #)
60	浏览器采用全屏模式时应返回 TRUE，采用窗口模式时应返回 FALSE。默认采用窗口模式。 (→ flag)
61	若允许显示对勾，从而支持选择多个项目，则应返回 TRUE；否则返回 FALSE。默认不允许显示对勾。 (→ flag)
62	返回项目数。如果程序在执行期间更改项目数，就必须处理此消息。 (→ #)
63	返回浏览器选择框左上角的坐标。您可能不需要更改默认值。 (→ #x #y)
64	此消息应返回初始选中项与页面顶部之间的差值。务必确保该差值小于当前所选项的索引，并小于页面大小；否则计算器可能崩溃。 (→ #)
65	需要绘制背景时会调用此消息。其操作可用于在背景上绘制其他内容。 (→)
代码（十进制）	说明及堆栈效果
66	需要绘制标题时会调用此消息。其操作应在 HARDBUFF 中绘制标题。多数情况下不处理此消息，而使用 \$Title 参数绘制标题。 (→)
67	以 grob 形式返回标题。多数情况下不处理此消息，而使用 \$Title 参数绘制标题。 (→ grob)
68	如果未定义消息 67，则调用此消息以 grob 形式返回标题，但仅用于全屏模式。 (→ grob)
69	如果未定义消息 67，则调用此消息以 grob 形式返回标题，但仅用于窗口模式。 (→ grob)
70	如果 \$Title 参数不是空字符串，则调用此消息来返回标题字符串。它会覆盖 \$Title 参数。 (→ \$)
74	此消息应绘制浏览器的所有可见行。 (→)
79	此消息应显示浏览器的一行。如果这是选中行，就应以反显方式绘制该行，或用其他方式标明它是选中行。 (# →)

代码（十进制）	说明及堆栈效果
80	此消息是通过 {}Items 参数提供项目的另一种方式。调用时会给出项目编号，此消息应返回该项目。可以返回任意对象；随后会调用 ::Converter 将其转换为字符串。如果希望浏览器中的项目动态变化，可以使用此消息。不过，在这种情况下，消息 82 可能更合适。 (# → ob)
81	此消息将一个项目转换为 grob（覆盖 ::Converter 参数）。它应返回尺寸为 7NULLAMx8NULLAM 的 grob。如果启用了勾，项目被勾选时，必须在 grob 中包含勾。 (# → grob)
代码（十进制）	说明及堆栈效果
82	此消息与消息 80 类似，但返回的对象已经是字符串，之后不会调用 ::Converter。如果使用此消息，就不必编写 ::Converter。 (# → \$)
83	返回一个说明菜单的列表。该列表的格式与 InputLine 和窗口所用格式相同，见第 37.1 节。 (→ {})
85	浏览器启动并完成全部设置后，会调用此消息。 (→)
86	项目被勾选或取消勾选时会调用此消息。默认操作能很好地处理项目的勾选和取消勾选，因此通常不需要处理此消息。 (# →)
87	浏览器退出前会调用此消息。 (→)
91	按下 ON 键或 CANCL 菜单键后会调用此消息。如果返回 TRUE，浏览器就会退出；如果返回 FALSE，浏览器将继续运行。 (→ flag)
96	按下 ENTER 键或 OK 菜单键后会调用此消息。如果返回 TRUE，浏览器就会退出；如果返回 FALSE，浏览器将继续运行。 (→ flag)

34.2 \$Title 参数

此参数指定标题。消息 66、67、68、69 和 70 可以覆盖此参数。

34.3 ::Converter 参数

这是一个二级程序，用于将列表中使用的任意类型对象转换为供显示的字符串。该二级程序的堆栈图为：

(ob → \$)

如果处理了消息 81 或 82，就不必编写此转换程序。不过，浏览器允许用户先按 ALPHA，再按某个字母，以搜索名称以该字母开头的对象并跳转到该对象。即使提供了上述消息，这项功能仍然需要 ::Converter 参数。因此，应确保此参数能以某种方式返回字符串。DO>STR 词条在这里非常有用。

34.4 {}Items 参数

可以在这里指定一个对象列表，也可以指定空列表，并使用消息 80、81 或 82 提供项目。

34.5 Init 参数

此参数可以是二进制整数，也可以是列表。如果它是 bint 0，浏览器将作为查看器运行，不允许选择项目。如果是其他 bint，则表示初始所选项目。

如果启用了多项选择，也可以改为指定一个 bint 列表，表示初始勾选的项目。

34.6 浏览器的典型用法

阅读上述消息和参数说明后，您可能已经注意到，可以用多种方式提供构成可浏览列表的项目，也可能因此感到困惑。下面列出其中两种方式。

- 可以通过 {}Items 参数提供项目，并提供一个 ::Converter，将其中的某个项目转换为字符串。无需处理消息 80、81 或 82。如果程序运行期间项目列表不会变化，这种方法很合适。
- 可以让 {}Items 列表为空，并将项目列表存放在其他位置（最可能是在某个 lam 中）。然后使用消息 80、81 或 82 返回项目。如果使用消息 81 或 82，返回的项目已经是 grob 或字符串，::Converter 可以是空二级程序。也可以用消息 80 返回某个对象，再用 ::Converter 将其转换为字符串。如果程序运行期间项目会发生变化，这种方法很合适。使用这种技术时，还必须处理消息 62。

浏览器中的项目数发生变化时，执行以下代码，使浏览器适应该变化：

```
1 ::
ROMPTR 0B3 03E (重新读取元素数)
ROMPTR 0B3 026 (重新读取宽度)
18GETLAM (索引 #Index)
5 12GETLAM (元素数 #NumOfElements)
DUP#0=IT
DROPONE
#MIN (若 #NumOfElements)
(减少, 则减小 #index)
10 18PUTLAM
FALSE ROMPTR 0B3 019 (重新计算偏移量)
;
```

34.7 示例

此示例使用浏览器让用户输入方程列表（灵感来自 **Y=** 窗口，但有很大差别）。列表起初为空，随后用户向其中添加方程。还可以编辑或删除方程。

该程序处理消息 **62** 和 **82**：分别在请求时返回项目数，以及已经转换为字符串的方程。方程存储在命名 **LAM** 中。程序还会处理其他一些消息，以配置浏览器的其他方面。

图 34.1 显示了该程序运行时的画面。

```
1 ::
NULL{} (从空列表开始)
' LAM EQS
```

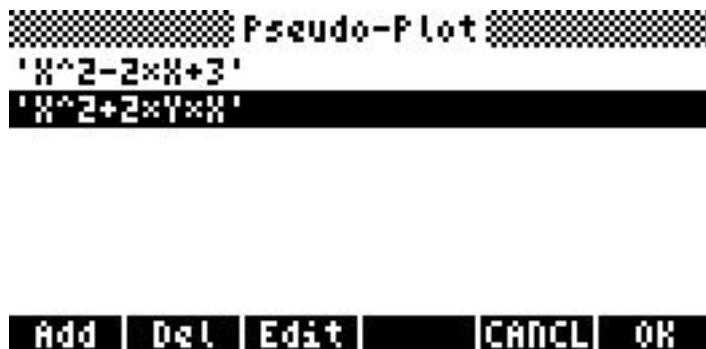


图 34.1: '48 浏览器示例

```
1 DOBIND
5
' :: (:::Appl 参数)
60 #=casedrop TrueTrue (使用全屏模式)
62 #=casedrop :: (元素数)
LAM EQS LENCOMP
10 DUP#0=IT
#1+
TRUE
;
82 #=casedrop :: (以字符串形式返回第 n 个元素)
15 LAM EQS SWAP
NTHELCOMP
ITE
:: (转换为字符串)
setStdWid
20 FPTR2 ^FSTR7
;
"No equations"
TRUE
;
25 83 #=casedrop :: (菜单)
{
{ "Add"
::
PushVStack&Clear (保存堆栈)
30 DoNewEqw
```

```

DEPTH
#0<> IT (添加方程)
::
LAM EQS SWAP
35 >TCOMP
' LAM EQS
STO
ROMPTR B3 3E (重新读取元素数)
;
40 PopMetaVStackDROP
;
}
{ "Del"
::
45 LAM EQS
INNERDUP
#0=case DROP (若为空则退出)
PushVStack&Keep (保存堆栈内容)
reversym
50 DROP
18GETLAM
ROLL
DROP
18GETLAM #1-
55 UNROLL
DEPTH
{}N
' LAM EQS STO
PopMetaVStackDROP (恢复堆栈)
60 ROMPTR B3 3E (重新读取元素数)
18GETLAM (更改所选项目)
12GETLAM (如有必要)
#MIN
18PUTLAM
65 FALSE ROMPTR B3 19
;
}
{ "Edit"
::
70 LAM EQS (获取项目)

```

```

18GETLAM
NTHELCOMP
NOT?SEMI (若为空则退出)
FPTR2 ^EQW3Edit (编辑)
75 NOT?SEMI (若未更改则退出)
18GETLAM
LAM EQS
PUTLIST (替换)
' LAM EQS STO
80 ;
}
NullMenuKey
{ "CANCL" FPTR2 ^DoCKeyCancel }
{ "OK" FPTR2 ^DoCKeyOK }
85 }
TRUE
;
DROPFALSE
;
90
"Pseudo-Plot" (标题)
' NULL:: (转换程序)
NULL{} (无项目, 改用消息提供)
BINT1 (初始选中的元素)
95
ROMPTR2 ~Choose (运行浏览器)
ABND
;

```

34.8 参考

地址	名称	说明
0000B3	Choose	(::Appl \$Title ::Convert {} offset → {} ' T) (::Appl \$Title ::Convert {} offset → ob T) (::Appl \$Title ::Convert {} offset → F) 如果启用了 对勾, 返回值为列表; 否则只返回所选对象。只有用户按下 CANCEL 时才返回 FALSE。
0050B3	ChooseMenu0	(→ {}) 带有“OK” 的菜单。
0060B3	ChooseMenu1	(→ {}) 带有“CANCL”、“OK” 的菜单。
0070B3	ChooseMenu2	(→ {}) 带有“CHK”、“CANCL”、“OK” 的菜单。
0630B3	ChooseSimple	(\$title {items} → ob T) (\$title {items} → F) HP48 选择引擎的简易接口。在 HP49G 上调用 ^RunChooseSimple。
004002	^RunChooseSimple	(\$title {items} → ob T) (\$title {items} → F) HP48 选择引擎的简易接口。
09F002	^DoCKeyCheck	(→) 切换当前项目的勾选状态。
0A0002	^DoCKeyChAll	(→) 勾选所有元素。
0B0002	^DoCKeyUnChAll	(→) 取消勾选所有项目。
09E002	^DoCKeyCancel	(→) 模拟 CANCEL。
09D002	^DoCKeyOK	(→) 模拟 OK。
地址	名称	说明
0B3002	^LEDispPrompt	(→) 重绘标题。
0B2002	^LEDispList	(→) 重绘浏览器各行。
0B1002	^LEDispItem	(# →) 重绘一行。
0150B3	(BBMoveTo)	(# →) 将选择移至指定行并更新屏幕显示。
0190B3	(BBRecalOff&Disp)	(flag →) 重新计算所选项目在当前页中的偏移量; 如果 flag 为 TRUE, 则重绘各行。
0220B3	(BBRunEntryProc)	(→) 向::Appl 发送消息 85, 从而运行用户定义的启动过程。
0230B3	(BBReReadPageSize)	(→) 重新读取页面大小 (消息 57)。
0240B3	(BBReReadHeight)	(→) 重新读取浏览器行高 (消息 58)。
0250B3	(BBReReadCoords)	(→) 重新读取浏览器选择框的坐标 (消息 63)。
0260B3	(BBReReadWidth)	(→) 重新读取浏览器行宽 (消息 59)。
0280B3	(BBRunENTERAction)	(→) 向::Appl 发送消息 96, 从而运行 OK 操作。它不检查返回值, 也不会退出浏览器。
0290B3	(BBRunCanclAction)	(→) 向::Appl 发送消息 91, 从而运行 CANCEL 操作。它不检查返回值, 也不会退出浏览器。
02F0B3	(BBReDrawBackgr)	(→) 重绘背景。
0370B3	(BBGetNGrob)	(#n → grob) 将第 n 个元素作为 grob 返回。
地址	名称	说明
0380B3	(BBGetNStr)	(#n → \$) 将第 n 个元素作为字符串返回。
03B0B3	(BBRereadChkEnbl)	(→) 重新读取是否启用对勾 (消息 61)。
03C0B3	(BBRereadFullScr)	(→) 重新读取是否使用全屏模式 (消息 60)。
03D0B3	(BReReadMenus)	(→) 重新读取菜单 (消息 83)。
03E0B3	(BBReReadNElems)	(→) 重新读取元素数 (消息 62)。
03F0B3	(BBGetN)	(#n → ob) 返回第 n 个元素。
04B0B3	(BBIIsChecked?)	(#n → flag) 返回给定元素是否已勾选。
0520B3	(BBUpArrow)	(→ grob) 将向上箭头作为 grob 返回。
0530B3	(BBDownArrow)	(→ grob) 将向下箭头作为 grob 返回。
0540B3	(BBSpace)	(→ grob) 将一个空格作为 grob 返回。
0590B3	(BBPgDown)	(→) 向下翻一页。

地址	名称	说明
05A0B3	(BBPgUp)	(→) 向上翻一页。
05B0B3	(BBEmpty?)	(→ flag) 如果浏览器没有元素，则返回 TRUE。
05C0B3	(BBGetDeflhtHeight)	(→ #) 根据将使用的字体返回行高。此值是浏览器的默认行高，等同于 FPTR 2 64。
0190E0	BRRclC1	(→) :: LAM 'BR5 ;

34.8.1 浏览器使用的 NULLLAM

浏览器使用大量匿名 LAM 来存储自身信息。说明如下：

LAM	说明	类型
1	由 CACHE 使用	n/a
2	POL 退出条件	flag
3	初始显示状态。该值是如下格式的列表： { DA1IsStatFlag DA2bEditFlag DA1BadFlag DA2aBadFlag DA2bBadFlag DA3BadFlag }	{ }
4	运行浏览器前的菜单	grob 131x8
5	运行浏览器前的屏幕	grob 131x56
6	页内偏移量	#
7	浏览器行高	#
9	浏览器左上角在 HARDB UFF 中的 x 坐标	#
10	浏览器左上角在 HARDB UFF 中的 y 坐标	#
11	页面大小	#
12	元素数	#
13	菜单	{ }
14	是否全屏?	flag
15	已勾选项目的索引列表	flag
16	是否启用对勾?	flag
17	为 TRUE 表示浏览器，为 FALSE 表示查看器	flag
18	当前所选索引	#
19	{ }Items	{ }
20	::Converter	::
21	\$Title	\$
22	::Appl	::

第 35 章 创建窗口

窗口为输入程序所需的数据提供图形界面。数据通过多个字段输入；这些字段就是供用户填入相应数据的“位置”。HP49的许多功能都会使用窗口。按下 **MODE** 键即可看到一个窗口。

在用户 RPL 中可以使用 **INFORM** 命令创建窗口，但这并非易事；在系统 RPL 中则更加困难。不过，这样做有若干优势：在用户 RPL 中只能使用文本字段，而在系统 RPL 中还可以使用复选框或选择字段。您还可以限制有效输入的范围，并让字段在执行期间显示或隐藏。最后，系统 RPL 的窗口运行速度也快得多。

窗口使用 **^IfMain** 命令创建；该命令是一个闪存指针，需要许多参数。这些参数分为三类：标签定义、字段定义和常规信息。每个标签定义和字段定义又由若干参数组成。

^IfMain 命令指向 HP49 中的新窗口引擎。旧的 HP48 引擎仍然存在，并经过了一些提速改进；旧的 **DoInputForm** 命令也仍然存在，基于该命令创建的窗口仍可工作。两个入口使用相同的参数，但消息处理方式（见下文第 35.4 节）已经改变。两个引擎之间存在少量不兼容之处。

下表给出了 **^IfMain** 命令的常规参数结构：

参数	说明
label_1	标签定义
...	
label_n	
field_1	字段定义
...	
field_n	
#labels	标签数
参数	说明
	字段数
#fields	
MessageHandler	见下文第 35.4 节
Title	显示在屏幕顶部的标题

35.1 标签定义

每个标签定义由三个参数组成：

参数	说明
label_text	要显示的文本
#x_offset	X 坐标
#y_offset	Y 坐标

label_text 是一个字符串，将使用微型字体把它转换为 **grob**。该文本会显示在指定坐标处。坐标由两个表示标签在屏幕上 **x**、**y** 位置的 **bint** 给出。左上角的坐标为 (0, 0)，坐标值向下和向右递增。

新窗口引擎还支持以 **grob** 作为参数；该 **grob** 会直接显示在指定坐标处。

35.2 字段定义

每个字段定义由十三个参数组成：

参数	说明
MessageHandler	见下文第 35.4 节
#x_offset	X 坐标
#y_offset	Y 坐标（通常为标签的 Y 坐标 - 1）
#Length	字段长度
#Height	字段高度（通常为 8）
#FieldType	字段类型，有效值见下文
#AllowedTypes	有效对象类型的列表
Decompile	见下文
"HelpString"	帮助字符串
ChooseData	见下文
ChooseDecompile	见下文
参数	说明
	重置值
ResetValue	
InitValue	初始值

消息处理程序将在下文说明。

x、**y** 位置指定字段出现的位置，其作用方式与标签定义中的 **x**、**y** 位置相似。长度和高度也是两个 **bint**，用来指定字段的大小。

字段类型是一个定义字段种类的 **bint**：

十进制值	字段类型
1	文本字段：用户可以输入任何内容。
23	扩展文本字段（仅限 DoInputForm 引擎）：用户可以输入任何内容，也可以使用 Filer 选择变量。
12	选择字段：用户必须从有效值列表中选择。
2	组合字段：用户可以从值列表中选择，也可以输入其他值。
32	复选框字段。

#AllowedTypes 参数用于文本字段、扩展文本字段和组合字段：它是一个 **bint** 列表，表示该字段允许输入的对象类型。对象类型见第 29.2 节的表格。必须使用“User Type”列中的值，并将其指定为 **bint**。其他字段应指定 MINUSONE。文本字段和组合字段也可以指定 MINUSONE，这表示接受所有种类的对象。在扩展文本字段中，此类型列表还用于限制 Filer 中显示的变量。

Decompile 是一个 **bint**，指定输入的对象应如何显示在屏幕上。其含义取决于置位的位。首先应以 BINT2 或 BINT4 为基础：前者表示使用当前模式反编译数字，后者表示使用 STD 模式。如果字段不保存数字，选择哪一个值并无太大差别。

指定对象的基本反编译方式后，还可以设置一些标志作进一步配置。如果希望显示字段值时使用微型字体，就给该值加 1。如果给该值加 8，则只显示字符串的第一个字符。如果字段保存的对象是列表，或者实际上是其他复合对象，则可以给该值加 16 或 32，以分别取得该复合对象的第一个或第二个对象，并显示

该对象；具体显示方式遵循其他值所定义的规则。此选项在使用选择字段时有时很有用，但不适用于普通文本字段。

注：DoInputForm 不支持使用微型字体反编译的字段。

也可以将 **Decompile** 参数指定为 BINT0。这样将不进行反编译：字段中只能使用字符串，并且字符串会以普通字体显示，不带引号。

下一个参数指定帮助字符串。当该字段获得焦点时，此字符串会显示在屏幕最后一行。您可以输入任意所需内容。

ChooseData 参数仅用于列表字段和组合字段。其他类型应将此参数设为 MINUSONE。此参数是供用户选择的值列表。当反编译值中包含数值 16 时，可以使用如下列表：

```
{ { "label1" <foo> } { "label2" <bar> } { ... } ... }
```

这样只会显示每个子列表中的第一个对象，但会返回整个子列表。（与用户命令 INFORM 的行为相同。）

使用 DoInputForm（但不是 ^IfMain）时，还可以在文本字段的 **ChooseData** 参数中指定字符串。这表示该文本字段允许用户浏览内存中存储的变量，并将某个变量的内容用作字段值。

显然，^IfMain 会忽略 **ChooseDecompile** 参数。只需将其指定为与 **Decompile** 参数相同的值。

重置值和初始值分别是窗口重置时和首次显示时字段中显示的内容。它们应当是该字段所允许类型的对象；对于列表字段，应当是 **ChooseData** 列表中的一个元素。对于复选字段，请使用 TRUE 或 FALSE。将其中一个或两个参数指定为 MINUSONE，即可使文本字段或组合字段留空。

35.3 标签数与字段数

这是两个 **bint**，分别表示已定义的标签数和字段数。请注意，由于它们是两个不同的值，因此可以定义仅向用户显示某种信息的标签，也可以定义没有任何标签的字段。

35.4 消息处理程序

与 HP49 的其他输入/输出应用程序一样，输入窗口使用消息处理程序，使程序员能够更细致地控制输入窗口。每个字段都有一个消息处理程序，输入窗口本身也有一个。每当字段或输入窗口发生某种“值得关注”的事件，以及输入窗口初始化期间，都会传递消息。

与其他消息处理程序一样，系统调用您提供的程序时，会在堆栈第 1 层放置一个消息编号（**bint**），有时还会提供其他参数。如果程序处理该消息，就应返回该消息所要求的结果（有时无需返回任何内容）。如果不处理该消息，则应丢弃消息编号，并将 FALSE 压入堆栈，同时保留其他所有参数。因此，完全不处理任何消息的消息处理程序就是 DROPFALSE；如您所知，可用'DROPFALSE 很方便地将其压入堆栈。

处理消息时，可以使用下文参考部分列出的词条，从输入窗口提取信息或修改输入窗口。第 35.8.2 节将说明 ^IfMain 可用的每一种消息。DoInputForm 使用的消息与之不同。

如果只处理一条消息，消息处理程序可以采用以下模板：

```
1 ' ::
  IfMsgGetFocus (或其他任意消息)
  #=case
```

```

::
5 * 此处为消息处理代码
TRUE (告知系统该消息已处理)
;
FALSE (表示其他消息未处理)
;

```

处理两条或更多消息时，可以采用以下模板：

```

1 , ::
IfMsgOK OVER#=case
::
* 代码。不要忘记返回 TRUE。
5 ;
IfMsgType OVER#=case

::
* 消息代码。
10 ;
* 还可以继续添加更多消息。
DROPPFALSE (其他消息未处理)
15 ;

```

35.5 标题

这是一个字符串，会以小字体显示在屏幕顶部。如果其长度超过 32 个字符（即屏幕宽度），字符串将被截断，并在末尾添加“...”。

使用 `^IfMain` 时，可以用自定义 `grob` 代替字符串进行显示。该 `grob` 的大小应为 131x7 像素。

35.6 输入窗口的结果

如果用户按 `ENTER` 退出输入窗口，堆栈输出为：

```

N+1: field_1
N: field_2
...
2: field_n
1: TRUE

```

如果使用 `CANCEL` 退出窗口，则只返回 `FALSE`。

每个字段的值取决于该字段允许的类型，以及列表字段可能值的指定方式。如果字段为空，则返回 `xNOVAL`。

35.7 示例

此示例模仿 `HP49` 的传输对话框，但远非完整实现。两者存在许多差异，而且此示例除了显示一个输入窗口之外，自然不具备任何实际功能。

代码定义了所有标签和字段；输入窗口带有一个简单的消息处理程序，用来处理两条消息：一条用于设置最初获得焦点的字段，另一条用于配置最后三个软键，使其外观与传输对话框中的软键相同。（不过，我们的这些按键在按下时只会发出蜂鸣声……）

运行此程序时的屏幕如下面的图 35.1 所示。



图 35.1：输入窗口示例

```

1 ::
* 标签定义
"Port:" 1 10
"Type:" 70 10
5 "Name:" 1 19
"Fmt:" 1 28
"Xlat:" 49 28
"Chk:" 104 28
"Baud:" 1 37
10 "Parity:" 49 37
"OvrW" 111 37
* 字段定义
' DROPFALSE (消息处理程序)
15 26 9 24 8 (位置和大小)
BINT12 (字段类型: 选择字段)
MINUSONE (类型, 此处不适用)
BINT0 (不反编译)
"Choose transfer port" (帮助文本)
20 { "Wire" } (可选项)
BINT0 (ChooseDecompile——忽略)
"Wire" DUP (初始值和重置值)

' DROPFALSE
25 92 9 36 8
BINT12
MINUSONE
BINT0
"Choose type of transfer"
30 { "Kermit" "XModem" }
BINT0
"Kermit" DUP
' DROPFALSE (消息处理程序)
35 25 18 103 8 (位置和大小)
BINT1 (字段类型: 文本字段)
{ BINT5 BINT6 } (允许标识符和列表)
BINT2 (以堆栈外观反编译)
"Enter names of vars to transfer" (帮助文本)
40 MINUSONE (ChooseDate——不适用)
MINUSONE (ChooseDecompile——忽略)
MINUSONE DUP (初始为空)
' DROPFALSE
45 20 27 18 8
BINT12
MINUSONE
BINT0
"Choose transfer format"
50 { "Bin" "ASC" }
BINT0
"Bin" DUP
' DROPFALSE
55 74 27 24 8
BINT12
MINUSONE
BINT0
"Choose character translations"
60 { "None" "Newl" "\8D159" "\8D255" }
BINT0
"\8D255" DUP
' DROPFALSE
65 122 27 7 8
BINT12
MINUSONE
BINT0

"Choose checksum type"
70 { "1" "2" "3" }
BINT0
"3" DUP
' DROPFALSE
75 20 36 24 8
BINT12
MINUSONE
BINT0
"Choose baud rate"
80 { "1200" "2400" "4800" "9600" "15300" }
BINT0

```

```
"9600" DUP
' DROPFALSE
85 74 36 24 8
BINT12
MINUSONE
BINT0
"Choose parity"
90 { "None" "Odd" "Even" "Mark" "Spc" }
BINT0
"None" DUP
' DROPFALSE
95 104 36 ZEROZERO
BINT32
MINUSONE
DUP
"Overwrite existing variables?"
100 MINUSONE
DUP
TrueTrue
9 9 (标签数和字段数)
105 ' :: (InputForm 消息处理程序)
BINT7 OVER#=case :: (设置最初获得焦点的字段)
DROP
TWO
TRUE
110 ;
BINT12 OVER#=case :: (配置菜单软键)
DROP
{ { "RCV" DoBadKey }

{ "KGET" DoBadKey }
115 { "SEND" DoBadKey }
}
TRUE
;
DROPFALSE
120 ;
"TRANSFER" (标题)
FPTR2 ^IfMain (运行)
;
```

35.8 参考

35.8.1 输入窗口

地址	名称	说明
020004	^IfMain	(11..ln fld..fm #n #m msg \$ → ob1..obn T) (11..ln fld..fm #n #m msg \$ → F) l = \$ #x #y f = msg #x #y #w #h #type legal dec \$hlp ChDat ChDec res init 使用新引擎启动输入窗口。
2C371	DoInputForm	(11..ln fld..fm #n #m msg \$ → ob1..obn T) (11..ln fld..fm #n #m msg \$ → F) l = \$ #x #y f = msg #x #y #w #h #type legal dec \$hlp ChDat ChDec res init 使用旧引擎启动输入窗口。
0050B0	~IFMenuRow1	(→ {}) 返回输入窗口第一行菜单所用的菜单。
地址	名称	说明
0060B0	~IFMenuRow2	(→ {}) 返回输入窗口第二行菜单所用的菜单。
021004	^IfSetFieldVisible	(# T/F(fld/lbl) T/F(val) →) (# T/F(fld/blb) #0 → T/F(val)) 切换字段或 标签的可见与不可见状态。第二个参数指定 # 表示字 段还是标签。第三个参数是要设置的值；第三个参数为 ZERO 时，表示提取当前设置。
022004	^IfSetSelected	(# T/F(fld/lbl) T/F(val) →) (# T/F(fld/blb) #0 → T/F(val)) 切换字段或 标签的选中与未选中状态；选中时，该对象会在屏幕上反 白显示。
023004	^IfSetGrob	(# T/F(fld/lbl) grb →) 设置字段或标签的 grob (会修改保存在数据字符串中的数据)。
024004	^IfSetFieldValue	(val # →) 设置字段值，并执行包括设置 GROB 在 内的完整处理。

地址	名称	说明
026004	^IfGetFieldValue	(# → val) 获取第 N 个字段的值。
027004	^IfGetCurrentField Value	(→) 获取当前字段的值。
025004	^IfSetCurrentValue Value	(val →) 设置当前字段的值。
028004	^IfGetFieldMessage Handler	(# → prg) 提取字段的消息处理程序。
029004	^IfGetFieldType	(# → #type) 提取字段类型。
02A004	^IfGetFieldObjects Type	(# → {}) 提取字段的对象类型列表。
02B004	^IfGetFieldDecompObject	(# → val) 提取字段的反编译值。

地址	名称	说明
02C004	^IfGetFieldChooseData	(# → {}) 提取字段的选择数据。
02D004	^IfGetFieldChooseDecomp	(# → val) 提取选择字段的反编译值。
02E004	^IfGetFieldResetValue	(# → val) 提取字段的重置值。
02F004	^IfSetFieldResetValue	(val # →) 更改字段的重置值。
030004	^IfGetFieldInternalValue	(# → val) 提取字段的内部值。
031004	^IfDisplayFromData	(→) 将数据字符串显示到屏幕上，并处理命令行的大小。
032004	^IfGetNbFields	(→ #n) 从数据字符串中提取字段数。
033004	^IfCheckSetValue	(# val →) 勾选或取消勾选复选字段。
034004	^IfCheckFieldtype	(ob → ob flag) 检查对象是否满足当前字段的类型要求。
04C004	^IfGetPrlgFromTypes	({} → {}) (#FFFF → #0) 生成字段所允许的前导码列表。
035004	^IfReset	(→) 重置所有字段，将各字段的重置值设为当前值。用于将数据列表展开到堆栈上，以便对其进行操作。
036004	^IfSetField	(# →) 将另一个字段设为“current”（当前）字段。
037004	^IfKeyChoose	(→ val) (→) 如果当前字段是选择字段，则显示可选项并让用户选择。仅当用户未按 CANCEL 时才返回值。

地址	名称	说明
038004	^IfKeyEdit	(→ (cmd line)) 如果可以，则编辑当前字段值。选择字段和标签选择字段无法编辑。
039004	^IfKeyTypes	(→ (cmd line)) (→) 显示一个选择框，其中列出此字段所有可能的类型。仅当用户以 OK 作答时才打开命令行。
03A004	^IfKeyCalc	(→ val) 将字段值压入堆栈并执行 HALT，以便用户计算新值。
03B004	^IfKeyInvertCheck	(→) 反转当前复选字段的值。
03C004	^IfONKeyPress	(→) ON 键处理程序。让用户有机会执行自己的程序，并询问 IF（输入窗口）是否可以退出。如果可以退出，则压入 FALSE（通过 ON 退出时视为取消），并将 LAM 'Quit 设为 TRUE。
03D004	^IfEnterKeyPress	(→) ENTER 键处理程序。让用户有机会执行自己的程序，并询问 IF（输入窗口）是否可以退出。如果可以，则将字段值压入堆栈，再压入 TRUE（如果已确认），并将 LAM 'Quit 设为 TRUE。
03F004	^IfSetHelpString	(\$dat #n \$/# → \$dat') 设置与字段关联的帮助字符串。此词条供自动 IF（输入窗口）生成程序使用，不应以其他方式使用。

地址	名称	说明
040004	^IfSetTitle	(\$dat grb/\$/# → \$dat') 修改 DataString 的标题部分。此词条供自动 IF（输入窗口）生成程序使用，不应以其他方式使用。
04A004	^IfInitDepth	(→) 初始化内部深度计数器。运行会修改堆栈的命令时必须使用此词条。
042004	^IfMain2	(\$dat handl {} → F) (\$dat handl {} → ob1...obn T) 内部 Inform Box 主程序。修改 DataString 的标题部分。此词条供自动输入窗口生成程序使用，不应以其他方式使用。
043004	^IfPutFieldsOnStack	(→ ob1...obn) 将每个字段的外部值压入堆栈。

地址	名称	说明
044004	<code>^IfSetFieldPos</code>	<code>(# T/F(fld/lbl) #x #y #w #h →) </code> 更改对象的大小和位置。注：不能更改标签或复选字段的大小或 X 位置。
045004	<code>^IfGetFieldPos</code>	<code>(# T/F(fld/lbl) → #x #y #w #h) </code> 获取对象的大小和位置。
047004	<code>^IfSetAllLabelsMessages</code>	<code>(\$dat bmsg #n → \$dat) </code> 设置一组标签的文本。
048004	<code>^IfSetAllHelpStrings</code>	<code>(\$dat bmsg #n → \$dat) </code> 设置所有字段的帮助字符串。
04D004	<code>^IsUncompressDataString</code>	<code>(\$dc → \$dat) </code> 解压缩经过压缩的数据字符串。

35.8.2 输入窗口消息

这些消息名称都是对应数值的 DEFINE。您可以在 include 子目录中的 inputform.h 文件里找到这些 DEFINE。

35.8.2.1 IfMsgKeyPress —0

每次击键后都会发送此消息，先发送给活动字段，再发送给输入窗口。如果字段处理了该消息，就不再执行输入窗口的正常按键处理。

输入：
2: #KeyPlane
1: #KeyCode
输出（若已处理）：
2: ::Key_Handler_Program
1: TRUE
输出（若未处理）：
3: #KeyPlane
2: #KeyCode
1: FALSE

35.8.2.2 IfMsgLooseFocus —1

当某字段即将失去焦点时，会向该字段发送此消息。您可以在这里执行任何操作，包括让该字段重新取得焦点。如果这样做，就不会向该字段发送 IfMsgGetFocus 消息。

输入：
1: #Field_That_Will_Get_Focus
输出：
2: #Field_That_Will_Get_Focus
1: TRUE 或 FALSE

35.8.2.3 IfMsgNewField —2

在新字段即将获得焦点之前，会向 IF（输入窗口）发送此消息。没有输入，输出可以是 TRUE 或 FALSE。

35.8.2.4 IfMsgGetFocus —3

当字段刚刚获得焦点时，会向该字段发送此消息。没有输入，输出可以是 TRUE 或 FALSE。

35.8.2.5 IfMsgGetFieldValue —4

此消息发送给当前字段。其输入是字段的内部数据，可用此消息返回外部值（即屏幕上显示的值）。例如，将此消息与 IfMsgSetFieldValue 消息配合使用，在具有一组固定值时，就可以只存储当前元素的偏移量，而不存储元素本身。

输入：
1: 内部值
输出（若已处理）：
2: 外部值
1: TRUE
输出（若未处理）：
2: 内部值
1: FALSE

35.8.2.6 IfMsgSetFieldValue —5

这是 `IfMsgGetFieldValue` 的对应消息：它以“外部”（或用户）值作为输入，并应返回内部值。如果希望在字段值每次更改后都调用一条消息，就应使用此消息。当然，也可以让作为输入提供的值保持不变。

```
输入：
1: 外部值
输出（若已处理）：
2: 内部值
1: TRUE
输出（若未处理）：
2: 外部值
1: FALSE
```

35.8.2.7 IfMsgGetFieldGrob —6

此消息发送给当前字段。如果决定处理它，就必须设置字段中显示的 `grob`（可以使用词条 `^IfSetGrob`）。完成此操作后，输入窗口中原本用于执行此操作的标准代码将不会被调用。

```
输入：
2: #Field
1: Value
输出：
2: #Field
1: TRUE 或 FALSE
```

下面是处理此消息的一个示例：

```
1 ::
OVER TRUE ROT SWAP (数值、数值、TRUE、值)
$>grob
FPTR2 ^IfSetGrob
5 ;
```

35.8.2.8 IfMsgSetFirstField —7

初始化期间，会向输入窗口处理程序发送此消息，以取得最先被选中的字段编号。返回 `TRUE` 还是 `FALSE` 没有区别；如有需要，只需更改该编号。

```
输入：
1: #Field
输出：
2: #Field
1: TRUE 或 FALSE
```

35.8.2.9 IfMsgFieldReset —10

当字段即将重置时，会向该字段发送此消息。如有需要，可以修改字段值。

```
输入：
1: Value
输出：
2: Value, 可能已修改
1: TRUE 或 FALSE
```

35.8.2.10 IfMsgGetMenu —11

初始化期间，会向输入窗口处理程序发送此消息；它可用于为输入窗口提供菜单。菜单采用第 37.1 节所述的格式。

```
输入：
1: Menu
输出（若已处理）：
3: 原菜单
2: 新菜单
1: TRUE
输出（若未处理）：
2: 原菜单
1: FALSE
```

35.8.2.11 IfMsgGet3KeysMenu —12

此消息可用于更改标准输入窗口菜单第一行的最后三个软键。如果处理此消息，则应返回一个包含三个子列表的列表，每个子列表都是一个按键定义。

```

输入：无
输出（若已处理）：
    2: List
    1: TRUE
输出（若未处理）：
    1: FALSE

```

35.8.2.12 IfMsgCancel —13

此消息允许用户替换默认的退出处理程序。按下 ON 键或 CANCEL 软键时会调用此消息。如果处理此消息，就不会运行任何标准代码。用户应更改 LAM 'Quit 的值，向 POL 指明应结束输入窗口。

```

输入：无
输出（若已处理）：
    1: TRUE
输出（若未处理）：
    1: FALSE

```

35.8.2.13 IfMsgCancelKey —14

当用户请求通过 CANCEL 键结束输入窗口时，会向输入窗口处理程序发送此消息。例如，如果输入无效，程序员可以阻止输入窗口结束。

```

输入：无
输出（若已处理）：
    2: TRUE 或 FALSE
    1: TRUE
输出（若未处理）：
    1: FALSE

```

处理此消息时，堆栈第 2 层中的 TRUE 表示应结束输入窗口，FALSE 表示应继续。

35.8.2.14 IfMsgOK —15

此消息与 IfMsgCancel 类似，但用于 OK 软键或 ENTER 键。

35.8.2.15 IfMsgKeyOK —16

当用户请求通过 OK 键结束输入窗口时，会向输入窗口处理程序发送此消息。例如，如果输入无效，程序员可以阻止输入窗口结束。

```

输入：无
输出（若已处理）：
    2: TRUE 或 FALSE
    1: TRUE
输出（若未处理）：
    1: FALSE

```

处理此消息时，堆栈第 2 层中的 TRUE 表示应结束输入窗口，FALSE 表示应继续。

35.8.2.16 IfMsgChoose —17

用户在选择字段中按下 CHOOS 软键时，会先向该字段发送此消息；如果字段未处理，再发送给输入窗口。如果其中任一方处理了此消息，就不会运行标准代码，您必须自行显示选择框。

此消息没有参数。显示选择框后，如果希望阻止标准代码执行，应返回 TRUE。

35.8.2.17 IfMsgType —18

按下 TYPES 软键时，会向输入窗口发送此消息。如果处理此消息，就不会执行标准代码。

此消息没有参数。显示选择框后，如果希望阻止标准代码执行，应返回 TRUE。

35.8.2.18 IfMsgCalc —19

按下 CALC 软键时，会向输入窗口发送此消息。如果处理此消息，就不会执行标准代码。

此消息没有参数。显示选择框后，如果希望阻止标准代码执行，应返回 TRUE。

35.8.2.19 IfMsgNewCommandLine —20

在输入窗口中新建命令行时，会发送此消息。系统不关心该消息是否得到处理；它只是让程序员有机会执行所需的任何操作。此消息没有输入，输出只是 TRUE 或 FALSE，两者没有任何区别。

35.8.2.20 IfMsgOldCommandLine —21

当输入窗口中的命令行被取消时，会发送此消息。更多详细信息请参见上面的 IfMsgNewCommandLine 消息。

35.8.2.21 IfMsgCommandLineValid —22

命令行得到确认时，会向字段发送此消息。

输入：无；存在一条命令行
输出（若已处理）：无命令行；堆栈中有元素及 TRUE
输出（若未处理）：
1: FALSE

35.8.2.22 IfMsgDecompEdit —23

当对象需要反编译以供编辑时，会向字段发送此消息。

输入：
1: Object
输出（若已处理）：
2: String
1: TRUE
输出（若未处理）：
1: FALSE

35.8.2.23 IfMsgNextChoose —24

在选择字段中按下 +/- 键时，会向该字段发送此消息。如果处理此消息，就不会执行默认操作。除了 TRUE/FALSE 之外，此消息没有输入和输出。

35.8.2.24 IfMsgEdit —25

在字段中按下 EDIT 软键时，会发送此消息。输入是字段的当前值；输出可以为空、经过修改的命令行、堆栈中的某些内容，或者经过修改的字段。如果处理此消息，就不会执行默认代码。

第 36 章 显示屏

在 **System RPL** 中编程时，程序员可使用两个屏幕：图形屏幕（例如可在 **Plot** 应用程序中看到，在用户 **RPL** 中称为 **PICT**）和文本屏幕（即标准堆栈环境中可见的图形）。只要可能，就应使用后者并保持图形屏幕不变，因为图形屏幕被视为用户资源，程序不应更改它。

36.1 显示结构

HP49 的系统 **RAM** 中有三个专用于显示的图形对象（下文称为 **grob**）。以下命令分别返回这些 **grob**：

命令	Grob
ABUFF	文本 grob（堆栈）
GBUFF	图形 grob（PICT）
HARDBUFF	文本 grob 或图形 grob，取决于当前哪一个处于活动状态。
HARDBUFF2	菜单标签

需要注意的是，上述词条只返回指向 **grob** 的指针，因此，只要修改该 **grob**，屏幕也会自动随之改变。多数情况下，这正是所需的行为；如果不希望如此，请在使用上述任一词条后调用 **TOTEMPOB**，以便在临时内存中创建一份独立副本。有关临时内存和对象引用的更多信息，请参阅第 24.1.4 节。

文本 **grob** 和图形 **grob** 均可放大，也可滚动。菜单标签 **grob** 的大小固定为 131x8 像素。

命令 **TOADISP** 使文本 **grob** 可见，命令 **TOGDISP** 使图形 **grob** 可见。

文本 **grob** 分为三个区域。显示区域编号为 1、2 和 3。许多词条名称中带有“**DA**”，其含义是“**Display Area**”（显示区域）。图 36.1 标出了各显示区域。



图 36.1：显示区域

显示区域 2 实际上又分为两个区域：2a 和 2b。通常只有区域 2a 可见，并占据整个 DA 2。

36.2 准备显示屏

两个词条用于取得文本显示的控制权：**RECLAIMDISP** 和 **ClrDA1IsStat**。前者执行以下操作：

- 确保当前显示的是文本屏幕；
- 清空文本屏幕；
- 如有必要，将文本屏幕的大小调整为默认的 131x56 像素。

此词条的工作方式与用户词条 **CLLCD** 非常相似，区别在于 **CLLCD** 从不调整文本屏幕的大小。

词条 **ClrDA1IsStat** 是可选的，但多数情况下都应使用。它会暂时停止跳动时钟的显示。大多数图形程序都不希望显示该时钟。

不需要菜单时，可使用词条 **TURNMENUOFF** 隐藏菜单，并将文本 **grob** 放大到 131x64 像素。使用 **TURNMENUON** 可重新开启菜单。有关菜单的更多信息，请参阅第 37 章。

使用文本屏幕的应用程序建议采用以下模板：

```
1  ::  
  ClrDA1IsStat      (暂停时钟)  
  RECLAIMDISP      (设置、清空文本屏幕并调整其大小)  
  TURNMENUOFF      (如果需要，关闭菜单)  
  
5  <application>  
  
  ClrDAsOK          (重绘 LCD)
```

原书第 271 页

```
-or-
10 SetDAsTemp      (冻结整个屏幕)
;
```

36.3 控制显示刷新

在某些程序中，应用程序结束后不希望重绘屏幕，而希望屏幕保持冻结，以使用户查看结果，就像用户 RPL 命令 FREEZE 那样。另一些情况下，则需要让显示恢复正常。系统 RPL 中有若干词条可实现这些目的。下面列出最常用的词条；完整列表见下文参考部分。

词条	作用
SetDA1Temp	冻结显示区域 1。
SetDA2OKTemp	冻结显示区域 2。
SetDA3Temp	冻结显示区域 3。
SetDA12Temp	冻结显示区域 1 和 2。
SetDAsTemp	冻结整个屏幕。
ClrDA1OK	重绘显示区域 1。
ClrDA2OK	重绘显示区域 2。
ClrDA3OK	重绘显示区域 3。
ClrDAsOK	重绘整个屏幕。

36.4 清空显示屏

以下词条可清空 HARDBUFF 的全部或部分内容。请记住，HARDBUFF 指当前显示的 grob，可能是文本显示 grob，也可能是图形显示 grob。除 BLANKIT 外，这些词条都不取得或返回参数。

词条	作用
CLEARVDISP	清空整个 HARDBUFF。
BlankDA1	清空显示区域 1。
BlankDA2	清空显示区域 2。
BlankDA12	清空显示区域 1 和 2。
Clr16	清空顶部 16 行。
Clr8	清空顶部 8 行。
Clr8-15	清空第 8 至 15 行（状态栏第 2 行）。
CLCD10	清空状态区和堆栈区。
CLEARLCD	清空整个屏幕。
BLANKIT	(#start_row #rows →) 从 HARDBUFF 中 清空 #rows 行。

36.5 显示文本

HP49 中有两种字体：“系统字体”和“微型字体”。用户可以更改这两种字体，但在任一时刻只能访问两种字体。系统字体（准确地说，是其字符）的高度可以变化，但字符宽度始终为 5 像素。微型字体的大小固定，每个字符均为 3x5 像素。

有些命令可以直接用系统字体显示文本，却没有直接用微型字体显示文本的命令。对于后一种情况，必须先把文本转换为 grob，再显示该 grob。下面只介绍最常用的命令；完整列表见下文参考部分。

36.5.1 系统字体

要用系统字体显示文本，请使用命令 DISPROW1、DISPROW2……DISPROW10。这些命令取得一个字符串作为参数，并将其显示在屏幕的指定行。请注意，根据所选系统字体的大小以及菜单是否显示，某些命令可能无法使用。即使采用最大的系统字体，也始终可以安全地在前七行显示文本。

36.5.2 微型字体

如上所述，使用微型字体显示文本较为复杂。首先，将一个字符串放入堆栈，然后运行命令 \$>grob。该命令会返回一个以微型字体表示此字符串的 grob。接下来，需要在屏幕上显示该 grob，可以使用 GROB! 或 XYGROBDISP。有关这些词条的更多信息以及 grob 的一般说明，请参阅第 15 章。在同一章中，还可找到一些用微型字体显示文本时可能更为方便的其他命令。

原书第 272 页

36.5.3 显示警告

词条 `FlashWarning` 用于显示警告消息。它先发出蜂鸣声，然后在消息框中显示给定字符串。用户必须按下 `OK` 才能继续。

也可以使用 `FlashMsg` 代替 `FlashWarning`。`FlashMsg` 在状态栏中显示文本，并且不会发出蜂鸣声。为了在状态区显示消息，它会使用词条 `DISPSTATUS2`；该词条取得一个包含换行符的字符串，并使用状态区的两行来显示该字符串。短暂停顿后，屏幕恢复到之前的状态，程序继续执行。

36.6 参考

36.6.1 显示结构

地址	名称	说明
26166	TOADISP	(→) 将文本屏幕设为活动屏幕。
2616B	TOGDISP	(→) 将图形屏幕设为活动屏幕。
25FA4	ABUFF	(→ textgrob) 将文本 grob 返回到堆栈。
26076	GBUFF	(→ graphgrob) 将图形 grob 返回到堆栈。HP49 中 <code>ExitAction!</code> 的 <code>extable</code> 地址与此相同，但这一定是个错误。
地址	名称	说明
2608F	HARDBUFF	(→ dispgrob) 将当前 grob 返回到堆栈。
26094	HARDBUFF2	(→ menugrob) 将菜单 grob 返回到堆栈。
25EDE	HARDHEIGHT	(→ #height) 返回 <code>HARDBUFF</code> 的高度。
25ED5	GBUFFGROBDIM	(→ #height #width) 返回图形 grob 的尺寸。

36.6.2 准备显示屏

地址	名称	说明
25EF4	RECLAIMDISP	(→) 激活文本 grob ，清空它并将其设为默认大小。
2EE7D	ClrDAIsStat	(→) 停止时钟显示。
2EEFD	MENUOFF?	(→ flag) 如果菜单 grob 已关闭，则返回 <code>TRUE</code> 。
2F034	TURNMENUOFF	(→) 关闭菜单显示，并放大 <code>ABUFF</code> 以填满屏幕。
2F031	TURNMENUON	(→) 开启菜单 grob 。
2EEFC	MENUOFF	(→)
26247	GetHeader	(→ #) 取得以行数表示的状态栏大小 (0-2)。
26283	SetHeader	(# →) 设置以行数表示的状态栏大小 (0-2)。
26099	HEIGHTENGROB	(grob #rows →) 增加图形 grob 或文本 grob 的高度。
260A3	KILLGDISP	(→) 通过将图形屏幕设为 <code>NULLGROB</code> 来清空它。参见 <code>DOERASE</code> 。
2EEF9	DOERASE	(→) 清除图形显示 grob ，但不改变其大小。

36.6.3 立即刷新

地址	名称	说明
2EF67	SysDisplay	(→) 立即重新显示所有需要刷新的区域，不等待当前命令执行完毕。
2F19F	?DispCommandLine	(→) 必要时立即重新显示命令行。
2F19E	DispCommandLine	(→) 立即重新显示命令行。
2EE5A	DispEditLine	(→) 仅调用 <code>DispCommandLine</code> 。
2DFCC	?DispMenu	(→) 如果缓冲区内没有等待处理的按键，则立即重新显示菜单。更好的写法是： :: DA3OK?NOTIT ?DispMenu ;
2DFF4	DispMenu.1	(→) 立即显示菜单。
2DFE0	DispMenu	(→) :: DispMenu.1 SetDAsValid;
2C341	?DispStack	(→) 必要时立即重新显示堆栈。

地址	名称	说明
2C311	?DispStatus	(→) 必要时立即重新显示状态区。
2C305	DispStatus	(→) 立即显示状态区。
2C2F9	DispStsBound	(→) 在 y=14 处显示一条水平线，通常用来分隔状态栏和堆栈。
2A7F7	DispTimeReq?	(→ flag) 是否需要显示时间？它会检查系统标志 40 以及其他某项。
2F300	DispILPrompt	(→) 重新显示 InputLine 提示，即刷新 InputLine 运行时命令行与状态栏之间的区域。要求 4LAM 中有一个字符串（提示文本）。
地址	名称	说明
26260	nDISPSTACK	(\$prompt #height #header flag flag →) 由 DispILPrompt 使用。

36.6.4 控制显示刷新

地址	名称	说明
2EE8D	ClrDA1OK	(→)
2EE8E	ClrDA2aOK	(→)
2EE8F	ClrDA2bOK	(→)
2EE90	ClrDA2OK	(→)
2EE6E	ClrDA3OK	(→)
2EE6D	ClrDAsOK	(→)
2EE62	DA1OK?	(→ flag)
2EE63	DA3OK?	(→ flag)
2EE66	DA2aLess1OK?	(→ flag)
2BF3A	DA1OK?NOTIT	(→) 依次执行 DA1OK?、NOT 和 IT。
2BF53	DA2aOK?NOTIT	(→) 依次执行 DA2aOK?、NOT 和 IT。
2BF6C	DA2bOK?NOTIT	(→) 依次执行 DA2bOK?、NOT 和 IT。
2BF85	DA3OK?NOTIT	(→) 依次执行 DA3OK?、NOT 和 IT。
2EE69	SetDA1Temp	(→)
2EE8A	SetDA2aTemp	(→)
2EE6A	SetDA2bTemp	(→)
2EEA7	ClrDA2bTemp	(→)
2F37A	SetDA2OKTemp	(→)
2EE6B	SetDA3Temp	(→)
2EE71	SetDA12Temp	(→)
2EE64	SetDAsTemp	(→)
2EEA5	SetDA2bTempF	(→)
2EE67	SetDA1Valid	(→)
2EF98	SetDA2aValid	(→)
2EE68	SetDA2bValid	(→)
2EE91	SetDA2Valid	(→)
2EF99	SetDA3Valid	(→)
地址	名称	说明
2EEA0	SetDA3ValidF	(→)
2EE78	SetDA1Bad	(→)
2EE74	ClrDA1Bad	(→)
2EEB0	DA1Bad?	(→ flag)
2EE79	SetDA2aBad	(→)
2EE75	ClrDA2aBad	(→)

地址	名称	说明
2EEB1	DA2aBad?	(→ flag)
2EE7A	SetDA2bBad	(→)
2EEB3	ClrDA2bBad	(→)
2EEB2	DA2bBad?	(→ flag)
2EE7B	SetDA3Bad	(→)
2EEB5	ClrDA3Bad	(→)
2EEB4	DA3Bad?	(→ flag)
2EE72	SetDA1NoCh	(→)
2EE73	SetDA2aNoCh	(→)
2EE76	SetDA2bNoCh	(→)
2EE81	ClrDA2bNoCh	(→)
2EEB7	DA2bNoCh?	(→ flag)
2EE93	SetDA2NoCh	(→)
2EE6F	SetDA12NoCh	(→)
2EE77	SetDA3NoCh	(→)
2EE70	SetDA13NoCh	(→)
2EE94	SetDA23NoCh	(→)
2EE65	SetDA12a3NCh	(→) 又名: SetDA12a3NoCh
2F379	SetDA123NoCh	(→)
2EE7C	SetDAsNoCh	(→)
2EE6C	SetDA2aEcho	(→)
2EEAC	SetDA1IsStat	(→)
2EEAE	SetNoRollDA2	(→)
2EEAF	ClrNoRollDA2	(→)
2EEAB	DA1IsStatus?	(→ flag)
2EE7F	SetDA2bIsEdL	(→)
2EE7E	DA2bIsEdL?	(→ flag)
2EE80	ClrDA2bIsEdL	(→)

36.6.5 清空显示屏

地址	名称	说明
25E7E	BLANKIT	(#startrow #rows →) 从 HARDBUFF 的 #startrow 开始清空 #rows 行。
26021	CLEARVDISP	(→) 清空 HARDBUFF。
2EED4	Clr8	(→) 清空顶部 8 行 (状态栏第 1 行)。
2EED5	Clr8-15	(→) 清空状态栏第 2 行。
2F15E	Clr16	(→) 清空顶部 16 行。
2EF5E	BlankDA1	(→) 从 HARDBUFF 中清空状态区。
2F31C	BlankDA2a	(→) 清空显示区域 DA2a。
2F31B	BlankDA2	(→) 清空显示区域 DA2a 和 DA2b。
2EE5C	BlankDA12	(→) 清空显示区域 DA1 和 DA2。
261C0	CLCD10	(→) 清空状态区和堆栈区。
261C5	CLEARLCD	(→) 清空整个屏幕。
2EF05	DOCLLCD	(→) 与用户词条 CLLCD 相同。

36.6.6 指示符与模式控制

地址	名称	说明
2613E	SetLeftAnn	(→) 设置左移位指示符。
2603A	ClrLeftAnn	(→) 清除左移位指示符。
26148	SetRightAnn	(→) 设置右移位指示符。
地址	名称	说明
2603F	ClrRightAnn	(→) 清除右移位指示符。
26139	SetAlphaAnn	(→) 设置 alpha 指示符。
26035	ClrAlphaAnn	(→) 清除 alpha 指示符。
25EE9	LockAlpha	(→) 设置 alpha 模式、指示符等。
25F08	UnLockAlpha	(→) 清除 alpha 模式、指示符等。
2649F	(ClrBusyAnn)	(→) 清除忙碌指示符。
26143	SetPrgmEntry	(→) 设置程序输入模式。
2610C	PrgmEntry?	(→ flag) 是否已设置程序输入模式?
25EBE	Do1st/2nd+:	(→ :: <ob1> ; (PRG mode)) (→ :: <ob2> <rest> ; (no PRG mode)) 如果处于程序模式, 则执行其后的下一个对象; 如果不处于程序模式, 则从其后的第二个对象开始执行运行流的其余部分。
25719	SetAlgEntry	(→) 设置代数输入模式。
2571E	ClrAlgEntry	(→) 清除代数输入模式。
256EA	AlgEntry?	(→ flag) 是否已设置代数输入模式?
25EDF	ImmedEntry?	(→ flag) 如果处于立即输入模式 (程序输入模式和代数输入模式均已清除), 则返回 TRUE。
25E74	?ClrAlg	(→) 如果已设置 AlgEntry 模式, 则将其清除。
25E75	?ClrAlgSetPr	(→) 如果已设置 AlgEntry 模式, 则将其清除, 并设置 ProgramEntry 模式。

36.6.7 窗口坐标

地址	名称	说明
2F384	TOP8	(→ HBgrob #x1 #y #x1+131 #y1+8) 返回状态栏第 1 行的坐标。
2F36C	Rows8-15	(→ HBgrob #x1 #y1+8 #x1+131 #y1+16) 返回状态栏第 2 行的坐标。
2F383	TOP16	(→ HBgrob #x1 #y1 #x1+131 #y1+16) 返回状态区的坐标。
2617F	WINDOWCORNER	(→ #x #y) 取得窗口角点的坐标。
2EED6	HBUFF_X_Y	(→ HBgrob #x #y) 返回当前 grob 和窗口坐标。
2F352	LEFTCOL	(→ #x) 取得最左列的 x 坐标。
2F36B	RIGHTCOL	(→ #x) 取得最右列的 x 坐标。
2F385	TOPROW	(→ #y) 取得最上行的 y 坐标。
2F31D	BOTROW	(→ #y) 取得最下行的 y 坐标。
26198	WINDOWXY	(#x #y →) 设置角点坐标。

36.6.8 滚动显示屏

地址	名称	说明
26193	WINDOWUP	(→) 将屏幕向上移动 1 像素。
26184	WINDOWDOWN	(→) 将屏幕向下移动 1 像素。
26189	WINDOWLEFT	(→) 将屏幕向左移动 1 像素。
2618E	WINDOWRIGHT	(→) 将屏幕向右移动 1 像素。
2F370	SCROLLUP	(→) 将屏幕向上移动 1 像素, 并检查是否按下了相应按键。

地址	名称	说明
2F36D	SCROLLEDOWN	(→) 将屏幕向下移动 1 像素，并检查是否按下了相应按键。
2F36E	SCROLLLEFT	(→) 将屏幕向左移动 1 像素，并检查是否按下了相应按键。
2F36F	SCROLLRIGHT	(→) 将屏幕向右移动 1 像素，并检查是否按下了相应按键。
2F34A	JUMPTOP	(→) 跳至屏幕顶部。
2F347	JUMPBOT	(→) 跳至屏幕底部。
2F348	JUMPLEFT	(→) 跳至屏幕左端。
2F349	JUMPRIGHT	(→) 跳至屏幕右端。
2F38D	WINDOWTOP?	(→ flag) 窗口是否位于顶部？
2F38A	WINDOWBOT?	(→ flag) 窗口是否位于底部？
2F38B	WINDOWLEFT?	(→ flag) 窗口是否位于左端？
2F38C	WINDOWRIGHT?	(→ flag) 窗口是否位于右端？

36.6.9 显示对象

地址	名称	说明
2F21D	ViewObject	(ob →)
2F21E	ViewStrObject	(flag \$ → F) flag 决定是否允许切换 TEXT/GRAPH。
2F21F	ViewGrobObject	(flag grob → F) flag 决定是否允许切换 TEXT/GRAPH。
地址	名称	说明
25F12	sstDISP	(ob →) 在状态栏中显示 ob，用于调试时的单步执行。
0C1007	^SCROLLExt	(grob →) 启动 PICT 环境。
2EF61	WINDOW#	(#x #y →) 内部 PVIEW，从给定坐标开始显示 PICT。

36.6.10 显示文本

地址	名称	说明
25EB4	DODISP	(ob %row →) 在指定行显示任意对象。
25FB8	DISPROW1	(\$ →) 又名：DISP@01,BIGDISPROW1
25EAB	DISPROW1*	(\$ →) 相对于窗口角点显示。
25FBD	DISPROW2	(\$ →) 又名：DISP@09,BIGDISPROW2
25EAC	DISPROW2*	(\$ →) 相对于窗口角点显示。
25FC2	DISPROW3	(\$ →) 又名：DISP@17,BIGDISPROW3
25FC7	DISPROW4	(\$ →) 又名：DISP@25,BIGDISPROW4
25FCC	DISPROW5	(\$ →)
261F7	DISPROW6	(\$ →)
25FD1	DISPROW7	(\$ →)
25FD6	DISPROW8	(\$ →) 是否可用取决于字体大小以及菜单是否开启。
25FDB	DISPROW9	(\$ →) 是否可用取决于字体大小以及菜单是否开启。
25FE0	DISPROW10	(\$ →) 是否可用取决于字体大小以及菜单是否开启。
地址	名称	说明
25FB3	DISPN	(\$ #row →) 又名：BIGDISPN
25EBC	Disp5x7	(\$ #start #max →) 从 #start 开始分多行显示字符串，所用行数不超过 #max。必须手动指定换行。超过 22 个字符的片段会被截断，并在末尾添加“...”。
25EAD	DISPSTATUS2	(\$ →) 使用两行在状态区显示消息。
38C00	(DISPST2&FREEZE)	(\$ →) 执行 DISPSTATUS2 并冻结状态区。
2EEFF	DispCoord1	(\$ →) 使用微型字体在菜单 grob 中显示 \$。

地址	名称	说明
2F32B	DISPCOORD2	(\$ →) 使用微型字体在菜单 grob 中显示 \$ 并等待按键，然后刷新菜单显示。
25FE5	DISPLASTROW	(\$ →) 在堆栈显示的最后一行，即菜单正上方显示 \$。
25FEA	DISPLASTROWBUT1	(\$ →) 在堆栈显示的最后一行显示 \$。如果菜单已开启，菜单可能会覆盖所显示的文本。
25ED4	FlashMsg	(\$ →) 在状态区显示消息，然后恢复正常显示。
2EE61	FlashWarning	(\$ →) 在消息框中显示消息并发出蜂鸣声，等待用户按下 OK。
2F1A5	AskQuestion	(\$ → flag) 使用该字符串在选择框中向用户提出“是/否”问题。
02E002	^DoAlert	(\$ →) 显示警告消息框。
2EE60	DoWarning	(\$ →) 显示消息、发出蜂鸣声并冻结状态区。
地址	名称	说明
007002	^Ck&DoMsgBox	(\$ #x #y grob menu → T) 显示一个消息框，其左上角带有 grob ，并使用指定的菜单。 #x 和 #y 的含义不明确。
0040B1	~MsgBoxMenu	(→ {}) 消息框菜单，其中只有 OK 键。

36.6.11 字体

地址	名称	说明
2621A	FONT>	(→ font) 提取系统字体。
2625B	MINIFONT>	(→ minifont) 提取当前微型字体。
25F15	>FONT	(font →) 设置系统字体。
2620B	>MINIFONT	(minifont →) 设置当前微型字体。
26288	StackLineHeight	(→ #) 返回文本 grob 的高度减去状态栏和菜单所占高度后的值。
26242	GetFontStkHeight	(→ #) 返回堆栈字体高度（用于显示堆栈行）。又名：StackFontHeight

第 37 章 菜单

菜单行分为六个部分，每个按键对应一部分；每部分高 8 像素、宽 21 像素。HARBDUFF2 中各菜单键标签的起始列如下：

十六进制	十进制	软键
0	0	第一个软键 (F1)
16	22	第二个软键 (F2)
2C	44	第三个软键 (F3)
42	66	第四个软键 (F4)
58	88	第五个软键 (F5)
6E	110	第六个软键 (F6)

命令 DispMenu.1 重新显示当前菜单；命令 DispMenu 重新显示当前菜单，然后调用 SetDA3Valid 冻结菜单显示区（显示区 3）。

下面这些词条把若干种对象转换为菜单标签，并在指定列显示它们：

词条	堆栈及操作
Str>Menu	(#col \$ →) 创建并显示标准菜单标签。
Id>Menu	(#col id →) 提取 id，并根据其内容显示标准标签或目录标签。
Grob>Menu	(#col grob →) 把 grob 显示为菜单标签。
Seco>Menu	(#col :: →) 对二级程序求值，并用结果创建和显示适当的菜单标签。

下面这些词条把字符串转换为四种可用的 grob。它们都以字符串为参数并返回 grob：

词条	操作
MakeStdLabel	创建黑色标签（标准标签）。
MakeBoxLabel	创建内部带框的标签。
MakeDirLabel	创建目录标签（上方带横条）。
MakeInvLabel	创建白色标签（与 Solver 中的标签类似）。

37.1 菜单格式

菜单可以是列表：

```
{ MenuKey1 MenuKey2 ... MenuKeyN }
```

也可以是程序：

```
:: <Settings> { MenuKey1 MenuKey2 ... MenuKeyN } ;
```

该程序返回这样的列表，还可以选择性地更改 InitMenu 所安装的默认菜单属性。

每个菜单键都可以采用以下任一种形式：

- NullMenuKey
- KeyObj
- { LabelObj KeyProcNS }
- { LabelObj { KeyProcNS KeyProcLS } }
- { LabelObj { KeyProcNS KeyProcLS KeyProcRS } }

LabelObj 是要显示为标签的对象。如果它是一个以 TakeOver 为首个命令的程序，系统会在堆栈中放入标签的 x 位置后对该程序求值；程序必须返回 LabelDef 程序所需的参数（通常是标签的 x 位置 and 要显示为标签的对象）。

如果不覆盖 LabelDef 命令（大多数情况下都不会覆盖），LabelObj 可以是任意对象，但通常是字符串或 21x8 grob。

KeyProc 是按下按键时执行的操作。它由一个特殊执行器执行；该执行器会根据对象类型采取适当操作。如果 KeyProc 是一个以 TakeOver 为首个命令的程序，它将覆盖通常的执行器。此处 NS 表示未按移位键时按下菜单键所执行的操作（可理解为 No-Shift）。同样，LS 和 RS 分别表示按左移位键或右移位键后按下该键时执行的操作。

37.2 菜单属性

HP49 的菜单系统极为灵活。除了通常的操作外，菜单还具有许多属性，用于定义标签的外观、按键时执行的具体操作、上下文变化或安装其他菜单时执行的操作等。

菜单包含以下属性：

属性	堆栈及操作
MenuDef	当前菜单。
MenuKeys	列表中的菜单键。
MenuRow	菜单页。
LabelDef	菜单的标签构建器。
MenuRowAct	菜单行变化或重新安装 LastMenu 时执行的操作。
ExitAction	菜单变化时执行的操作。通常，此操作会把当前菜单保存为 LastMenu 。
TrackAct	上下文（当前目录）变化时执行的操作。
ReviewKey	按下 REVIEW 键（右移位 DOWN ）时执行的操作。
MenuKeysNS	未按移位键时按下菜单键所执行的操作。
MenuKeysLS	按左移位键后按下菜单键所执行的操作。
MenuKeysRS	按右移位键后按下菜单键所执行的操作。
BadMenu?	是否必须重绘菜单？
Rebuild?	菜单行是否已经变化？
Track?	如果上下文已经变化，是否有程序要执行？

TrackAct 属性的示例如下：

- **SolverMenu** 以 **DoSolveMenu** 作为 **TrackAct**，因为可能有另一个 **EQ** 变量可供使用。
- **Custom** 菜单只会重新启动自身，因为 **CST** 变量的值可能已经变化。
(**CstTrack** = :: **NoExitAction** **MenuDef**@ **InitMenu** ;)

大多数菜单属性都可以通过受支持的入口修改。下面给出一个示例。以下程序设置经过修改的 **VAR** 菜单，使变量免受“左移位键加菜单键”操作的覆盖。

```
1 ::
MenuMaker
::
ROMPTR A9 2 (内置 VAR 菜单)
5 '
::
DUP
DUPTYPECSTR? NOT_IT DECOMP$ (创建字符串)
SWAP
10 ID prtct
ITE (选择标签类型)
MakeInvLabel
MakeStdLabel
Grob>Menu (显示标签)
15 ;
LabelDef!
'
::
ID prtct
20 NOTcase xSTO
DECOMP$ "\0A is protected" &$
FlashWarning
;
MenuKeyLS!
25 ;
DoMenuKey
;
```

该程序完成以下几项工作：

1. 取得通常的 **VAR** 菜单，以便将其传给 **DoMenuKey**。
2. 修改 **LabelDef** 属性，使受保护的变量在菜单中显示为反显标签。
3. 修改 **MenuKeyLS** 属性，使相关变量受保护时以错误消息退出。

ID prtct 中的程序是一个 ID 选择器，其堆栈效果为 (id → flag)，必须判断给定 ID 是否应受保护。下面给出几种可能的实现：

1. :: **DROPTTRUE** ; (所有变量均受保护。)
2. :: **DROPFALSE** ; (所有变量均不受保护。)
3. :: **ID>\$ CAR\$ CHR_ % EQUAL** ; (名称以“%”开头的变量受保护。)
4. 列表中的所有变量：

```

:: (列表中的所有变量)
ID PROTECTED (存储在该变量中)
OVER EQUALPOSCOMP (' PROTECTED' 中的变量受保护)
#0<>
;

```

请注意，如果变量 prtct 不存在，或不符合所需的堆栈效果，计算器可能会崩溃。您可能需要修改程序，以便更好地防范用户错误。还要注意，这只能防止通过左移位键加菜单键进行存储；自然，它不能防止使用 STO 命令或文件管理器进行存储。

37.3 参考

37.3.1 菜单属性

地址	名称	说明
04A41	GETDF	(#menukey → ob) 从 THOUGHTAB 取得菜单键的定义。#menukey = #1..#6。
04A0B	GETPROC	(#menukey → ob) 从 THOUGHTAB 取得菜单键的定义。#menukey = #1..#6；使用 #7 时取得执行器。
2580E	SetRebuild	(→) 设置表示菜单需要重建的标志。
260B7	MenuRow!	(#n →) 设置菜单行。#n 不是行号，而是该行中第一个菜单键的索引，即 1,7,13,...。
260BC	MenuRow@	(→ #n) 提取当前菜单页中第一个菜单键的索引。第一页返回 1，第二页返回 7，第三页返回 13，依此类推。
地址	名称	说明
260A8	LastMenuRow!	(#n →) 设置上一菜单的菜单行。#n 不是行号，而是该行中第一个菜单键的索引，即 1,7,13,...。
260AD	LastMenuRow@	(→ #n) 提取上一菜单当前行中第一个菜单键的索引。第一页返回 1，第二页返回 7，第三页返回 13，依此类推。
25845	MenuDef@	(→ menu) 提取当前菜单定义。menu 是 MenuList、程序或 Rompointer。
25908	LastMenuDef!	(menu →) 设置上一菜单的定义。menu 是 MenuList、程序或 Rompointer。
2590D	LastMenuDef@	(→ menu) 提取上一菜单的定义。menu 是 MenuList、程序或 Rompointer。
25EFB	SaveLastMenu	(→) 把当前菜单的行和定义存储为上一菜单。
25EDA	GetMenu%	(→ %)
25863	MenuRowAct!	(ob →) 把 ob 存储为菜单的 RowAct 属性。
25EE2	InitTrack:	(→) 如果目录发生变化，则执行运行流中的下一个程序。VAR 菜单用它在目录变化时设置第一个菜单行，CST 菜单则用它重建菜单。
地址	名称	说明
25877	LabelDef!	(ob →) 存储一个用于显示菜单标签的程序。该程序的堆栈效果为： (#col ob →) 例如，LIBS 命令使用以下程序使所有菜单标签看起来都像目录： :: DUPNULL\$? ITE MakeStdLabel MakeDirLabel Grob>Menu ; 执行期间，INDEX@ 中保存菜单键编号。
2589F	MenuKeyLS!	(ob → ob) 设置左移位菜单键的操作。该程序接收菜单项的操作部分作为参数，即 {ob-NS ob-LS ob-RS}。
258B3	MenuKeyRS!	(ob → ob) 设置右移位菜单键的操作。该程序接收菜单项的操作部分作为参数，即 {ob-NS ob-LS ob-RS}。
2588B	MenuKeyNS!	(og → ob) 设置未移位菜单键的操作。该程序接收菜单项的操作部分作为参数，即 ob-NS 或 {ob-NS ob-LS ob-RS}。
25890	MenuKeyNS@	(→ ob) 提取未移位菜单键的操作。
25EFC	SetKeysNS	(ob →) 把 ob 设置为 MenuKeysNS，并把 DoBadKey 设置为 LS 和 RS 操作。
25F02	StdMenuKeyLS	({ob-NS ob-LS ob-RS} → ?) 标准菜单的 MenuKeyLS 内容。
25F03	StdMenuKeyNS	(ob-NS → ?) ({ob-NS ob-LS ob-RS} → ?) 标准菜单的 MenuKeyNS 内容。

地址	名称	说明
27FED	NullMenuKey	(→) 定义菜单列表时，用作空菜单键的占位符。
258C7	ReviewKey!	(ob →) 存储一个在按下 REVIEW 键 (RS DOWN) 时调用的程序。该程序的堆栈效果为： (→)
258EF	(ExitAction!)	(ob →) 把 ob 存储为退出操作。
25EEF	NoExitAction	(→) 把 NOP 设置为 ExitAction。其主要用途是在安装新菜单时，避免把当前菜单保存为上一菜单。

37.3.2 构建菜单

地址	名称	说明
275C6	TakeOver	(→) 覆盖默认菜单键执行器。如果它是程序中的第一个词条，该程序就可以在编辑模式下使用。如果它是菜单键标签槽内程序的第一个词条，系统会对该程序求值，以取得标签对象（很可能是 grob）。
275EE	Modifier	(→)
27620	MenuMaker	:: TakeOver ; (→ ob) 引用运行流中的下一个对象，并且也提供 TakeOver。其反汇编为： :: TakeOver 'R ; 通常按以下方式使用： :: MenuMaker menu InitMenu ; (menu →) menu 为 {} 或:: settings {} ;。Settings 会覆盖 InitMenu 安装的默认设置。
25EE0	InitMenu	(menu →)
25EC6	DoMenuKey	:: SetDA12NoCh InitMenu ;

地址	名称	说明
25EE1	InitMenu%	(%mnu.pg →) (%0 →)
25F00	StartMenu	(menu #n →) #n 是页面上第一个菜单键的索引；第一页使用 1，第二页使用 7，依此类推。StartMenu 执行 ExitAction（上一菜单！），设置默认菜单属性和菜单页，然后对 menu 求值，把结果存入 MenuKeys，并执行 SetThisRow。
25EFE	SetThisRow	(→) 构建新的 TOUCHTAB，并执行 SetBadMenu。
25EE8	LoadTouchTbl	(MenuKey1 .. MenuKeyN #n →) 根据各菜单键构建新的 TOUCHTAB。

37.3.3 菜单显示

地址	名称	说明
2EF66	SysMenuCheck	(→) 检查菜单有效性。如果 DA3NoCh? 为真，则不执行任何操作；如果 Track? 为真，则执行?DoTrackAct@；如果 Rebuild? 为真，则执行 SetThisRow。
2DFCC	?DispMenu	(→) 如果缓冲区中没有等待处理的按键，则立即重新显示菜单。更好的写法是： :: DA3OK?NOTIT ?DispMenu ;
2DFF4	DispMenu.1	(→) 立即显示菜单。
2DFE0	DispMenu	(→) :: DispMenu.1 SetDAsValid ;

37.3.4 显示菜单标签

地址	名称	说明
2E0D5	Grob>Menu	(#col grob →) 把 grob 显示为菜单标签。
2E0F3	Str>Menu	(#col \$ →) 把字符串显示为菜单标签。
2E11B	Id>Menu	(#col id →) 把 id 显示为菜单标签。
2E107	Seco>Menu	(#col :: →) 执行 EVAL，再执行 DoLabel。
25886	DoLabel	(#col ob →) 如果 ob 属于支持的类型，则显示菜单标签；否则生成“Bad Argument Type”错误。
2E2AA	MakeLabel	(\$ #w #x grob → grob') 使用 CENTER\$3x5 并令 y=5，把 \$ 插入 grob。
08E007	^WRITEMENU	(\$6...\$1 →) 把六个字符串显示为菜单键。

37.3.5 通用入口

地址	名称	说明
25EA6	CheckMenuRow	(# → # #')
25EFD	SetSomeRow	(#n →) 其中 Mod(n,FFFFFFh)= 0。
2589A	DoMenuKeyNS	(#n →)
275FD	MenuKey	(→) 从运行流中取得 NOB。
2F15B	CLEARMENU	(→)
25F2B	CHECKMENU	(→)
3EA01	(CST)	(→ ob) 对 ID CST 求值。
2C2C0	nCustomMenu	(→) 安装 CST 菜单。
25EFF	SolvMenuInit	(→) 设置 Solver 菜单所需的 MenuKeyNS/LS/RS、ReviewKey 和 LabelDef 属性。
地址	名称	说明
25EC3	DoFirstRow	(→) 设置当前菜单的第一行。

第 38 章

HP49 编辑器编程

HP49G 内置的编辑器比 HP48 上的编辑器快得多，也更易用。不过，它是通用编辑器；对于特定应用，如果无需另写一套全新的编辑器就能添加一些功能，会非常有用。HP49G ROM 包含许多受支持的入口，可供程序操纵编辑器。这些入口可用于编写编辑器扩展。

38.1 术语

以下术语会在本章中频繁出现。

术语	别名或记号	含义
EditLine	Buffer; Command line; \$buf	当前正在编辑的字符串。堆栈效果中用 \$buf 表示。
Cursor position	#cpos	光标在 EditLine 中的位置，以 bint 表示。
Current line		编辑器中的当前行，即从光标之前的 NEWLINE 之后开始，直到下一个 NEWLINE 字符为止的子字符串。
Editor window		当正在编辑的文本过长和/或过宽时，HP49G 屏幕只显示其中一部分，即窗口。移动光标时，必须重新定位窗口，使新位置可见。
Selection	\$sel	当起始标记和结束标记处于活动状态时，可以选择缓冲区中的一个区域。堆栈效果中用 \$sel 表示所选子字符串。
术语	别名或记号	含义
Word-start	#ws	单词的起始位置，即字符串中前一个字符是 SPACE 或 NEWLINE、后一个字符是非空白字符的位置。下面若干命令会处理这种位置。
Invisible chars		HP49G 可以用不同的字体和样式显示文本。为切换字体和样式，文本中会插入表示字体或样式变化的特殊标记。这些 3 字符序列不可见，但会计入字符串长度和光标位置。一些 Editor 命令能够识别这些字符串，并进行复杂计算，以剪切和粘贴带属性的文本。这当然会使这些命令比原本可能达到的速度更慢。如果不使用字体和样式，就不必担心这些问题。

38.2 示例

有关以下示例所用具体入口的信息，请参阅后面的“参考”一节。

1. 选择当前行并将其复制到剪贴板。

```
1 ::
TakeOver
CMD_END_LINE (移到行尾)
RCL_CMD_POS (提取位置)
5 CMD_STO_FIN (存储为标记)
CMD_DEB_LINE (移到行首)
RCL_CMD_POS (提取位置)
CMD_STO_DEBUT (存储为标记)
CMD_COPY (复制到剪贴板)
10 ;
```

使用内置命令 SELECT.LINE 可以更简短地完成此操作。以下程序与上面的程序等效。

```
1 ::
TakeOver

SELECT.LINE
CMD_COPY
5 ;
```

2. 在单行中插入":;"模板，并把光标置于":;"与";"之间。

```
1 ::
TakeOver
":;"
CMD_PLUS
5 CMD_BAK
CMD_BAK
;
```

3. 插入多行`:: ;`模板，并把光标置于第二行，另加一层缩进。

```
1 ::
TakeOver
"::\0A\0A;"
CMD_PLUS
5 CMD_UP
SPACE$ (把缩进调整为额外 2 个空格)
CMD_PLUS
;
```

4. 转到下一个标签。标签是以`"**"`开头的行。

```
1 ::
TakeOver
"\0A*" (换行符后跟星号)
FindStrInCmd (查找该字符串)
5 IT
:: (如果成功)
DROP (丢弃 #end)
#1+ (修正位置以越过 NL)
STO_CURS_POS (设置新的光标位置)
10 ;
DROP (丢弃搜索字符串)
;
```

5. Emacs 库的 RPLCPL 命令（见第 A.6 节）用于在 Editor 中补全名称。它需要找到光标之前的单词片段。可按以下方式完成：

```
1 ::
RCL_CMD (提取 EditLine)
RCL_CMD_POS (当前位置)
DUP (GET.W<- 所需的参数)
5 GET.W<- (单词起始位置)
#1+SWAP (为 SUB$ 准备参数)
SUB$ (取得子字符串)
;
```

6. 把当前行的缩进改为 `#N` 个空格。`#N` 是预期位于堆栈第 1 层的 `bint`。该命令不改变空行和以`"**"`开头的行。

```
1 ::
Blank$ (创建缩进字符串)
CMD_DEB_LINE (移到行首)
RCL_CMD
5 RCL_CMD_POS
#1+ SUB$1 (查看该行第一个字符)
BINT42 ('*' 的 ASCII 码)
OVER#=case (该行是否以 '*' 开头?)
:: 2DROP (清理)
10 CMD_DOWN (转到下一行并退出)
;
BINT32 >#?SKIP (该行以非空白字符开头)
::
CMD_END_LINE (该行以空白字符开头)
15 RCL_CMD_POS (记住行尾位置)
CMD_DEB_LINE (返回行首)
DO>Skip (跳到下一个单词)
RCL_CMD_POS
#<ITE
20 DROPDROP (如果已经在下一行，则退出)
DoFarBS (删除第一个单词前的空白)
;
CMD_PLUS (插入空格)
CMD_DEB_LINE (返回行首)
25 CMD_DOWN (下一行)
;
```

38.3 在编辑器中执行外部命令

要在编辑器中使用新命令，必须把它们赋给某个按键，或放入菜单中。请注意，您编写的每个命令都必须在二级程序中以 TakeOver 作为第一个词条，否则该命令不能在编辑器中执行。

下面给出一个简单的 InputLine 环境示例。它定义一个初始菜单，其中有两个命令，分别用于选择当前行和清空 EditLine。有关 InputLine 的更多信息，请参阅第 31 章。

```
1 ::
"Edit this!" (提示)
"" (初始字符串)
zero (光标位置)
5 zerozerozero (模式)
{
{
"SLINE"
:: (选择行的程序)
10 TakeOver
SELECT.LINE
;
}
{
15 "CLEAR"
:: (清空 EditLine 的程序)
TakeOver
DEL_CMD
;
20 }
}
ONE (初始菜单行)
TRUE (中止标志)
ZERO (解析)
25 InputLine (然后开始!)
;
```

38.4 参考

38.4.1 状态

地址	名称	说明
257A2	EditLExists?	(→ flag) 是否存在 EditLine?
2EEED	NoEditLine?	(→ flag) 是否不存在 EditLine?
2F196	RCL_CMD	(→ \$) 把当前命令行的副本返回到堆栈。与 EDITLINE\$ 相同。
2EEEB	EDITLINE\$	(→ \$) 把当前命令行的副本返回到堆栈。与 RCL_CMD 相同。
2F197	RCL_CMD2	(→ \$) 与 RCL_CMD 类似，但如果没有足够的内存把 EditLine 复制到堆栈，它会把当前 EditLine 移入 TEMPOB。当然，这会删除当前 EditLine。
2EF87	RCL_CMD_POS	(→ #) 提取当前光标位置。
26585	CURSORE	(→ #) 提取当前光标位置。
26594	(CURSOR_PART)	(→ #) 提取当前光标所在的行。
2F158	(THISCHAR)	(→ chr) 返回光标下的字符。在文件末尾返回 CHR_00。
2EEEA	CURSOR_END?	(→ flag) 检查光标是否位于行尾或文件末尾。其实方式是把当前字符与换行符和 CHR_00 比较。
264CC	FIRSTC@	(→ #) 显示窗口左边缘所在的列。
26030	CURSOR_OFF	(→ #) 光标列相对于显示窗口左边缘的位置。
地址	名称	说明
2EF91	CAL_CURS_POS	(#1 #c → #) 根据行号和列号计算当前 EditLine 中的位置。结果可供 STO_CURS_POS 使用，将光标移至该位置。如果 #line 大于 EditLine 的行数，则计算最后一行中的位置。
2EF90	CAL_CURS_POS_VIS	(#1 #c → #) 与 CAL_CURS_POS 类似，但会忽略不可见字符。结果可供 STO_CURS_POS_VIS 使用，将光标移至该位置。

地址	名称	说明
2F199	RCL_CMD_MODE	(→ \$) 提取包含当前编辑器设置的字符串。使用 HALT 暂时离开编辑器，或者调用必须暂时更改设置的程序时，可以与 STO_CMD_MODE 配合使用，以保存和恢复 EditLine 的状态。
2F198	STO_CMD_MODE	(\$ →) 存储与 RCL_CMD_MODE 所取得字符串类似的模式字符串。

38.4.2 插入文本

地址	名称	说明
2EF74	CMD_PLUS	(\$ →) 在 EditLine 的当前光标位置插入字符串。
2F194	CMD_PLUS2	(\$ →) 用新字符串替换整个当前 EditLine。当内存不足以复制堆栈第 1 层中的字符串时，将该字符串移出 TEMPOB。您必须确保该字符串未以任何方式被引用。光标会移至新字符串的末尾。
地址	名称	说明
2F195	CMD_PLUS3	(\$ →) 与 CMD_PLUS2 相同，但不改变光标位置。用于在 HALT 之后恢复命令行上下文。
2EF97	InsertEcho	(\$ →) 在 EditLine 的当前光标位置插入字符串。
2EEE4	Echo\$Key	(\$ /chr →) 与 CMD_PLUS 相同。
2F11C	Echo\$NoChr00	(\$ →) 在 EditLine 的当前光标位置插入字符串。
25EC1	DoDelim	(→) 从运行流中取得一个字符或字符串并将其插入。
25EC2	DoDelims	(→) 从运行流中取得一个字符或字符串，将其插入并把光标向左移回一个字符。
25795	INSERT_MODE	(→) 开启插入模式。在插入模式下，新字符不会覆盖旧字符。
2577F	(TogInsert)	(→) 切换插入/覆盖标志。
25790	INSERT?	(→ flag) 如果插入模式处于活动状态，则返回 TRUE。

38.4.3 删除文本

地址	名称	说明
2EF82	CMD_DEL	(→) 删除编辑器中的下一个字符。与 LS+DEL 相同。如果执行此词条时按住 BS，HP49G 会认为您按下了该键并希望重复执行。
地址	名称	说明
2EF81	CMD_DROP	(→) 在编辑器中执行退格。删除光标前的字符。与 BS 键相同。如果执行此词条时按住 BS，HP49G 会认为您按下了该键并希望重复执行。
2EF95	DEL_CMD	(→) 清空整个 EditLine。
2EEE7	InitEdLine	(→) :: DEL_CMD ;
2F2F0	DO<Del	(→) 向左删除至单词开头。与编辑器 TOOL 菜单中的 ←DEL 按钮相同。
2F2F1	DO>Del	(→) 向右删除至下一个单词的开头。与编辑器 TOOL 菜单中的 DEL→ 按钮相同。
2F2F9	DODEL.L	(→) 删除当前行中的所有字符。如果该行已为空，则删除 NEWLINE。与编辑器 TOOL 菜单中的 DELL 按钮相同。
2F2DD	DoFarBS	(→) 删除至行首。与编辑器 TOOL 菜单中的 RS+←DEL 相同。
2F2DE	DoFarDel	(→) 删除至行尾。与编辑器 TOOL 菜单中的 RS+Del→ 相同。

38.4.4 移动光标

地址	名称	说明
2EF8B	STO_CURS_POS	(# →) 存储光标位置。把光标移至指定位置，并在必要时重新定位编辑器窗口，以确保光标位置可见。如果必须水平滚动窗口，此命令会把窗口左边缘设为光标所在列，并尽可能显示光标右侧的文本。但是，如果把窗口边缘移至第零列时光标仍然可见，则优先采用该位置。
2EF8C	STO_CURS_POS2	(# →) 与 STO_CURS_POS 相同，但把编辑器窗口的右边缘移至光标所在列。

地址	名称	说明
2EF8D	STO_CURS_POS3	(# →) 与 STO_CURS_POS 相同，但不检查样式/字体切换序列。因此，STO_CURS_POS 始终确保光标最终位于可见字符之前，而此命令允许把光标置于不可见转义序列内部。
2EF8E	STO_CURS_POS4	(# →) 在编辑器窗口定位方面与 STO_CURS_POS2 相同，在处理不可见字符方面与 STO_CURS_POS3 相同。
2EF8F	STO_CURS_POS_VIS	(# →) 与 STO_CURS_POS 类似，但忽略不可见字符。因此，如果您查看字符串并希望转到所看到的第 5 个字符，请使用此词条。
2F378	SetCursor	(# →) (# #) →) 把光标设到给定位置。使用列表参数时，两个数分别为行和列。
地址	名称	说明
2EF7C	CMD_NXT	(→) 把光标移至下一个字符，类似于右方向键。
2EF7B	CMD_BAK	(→) 把光标向左移动。与左方向键相同。
2EF80	CMD_DOWN	(→) 把光标移至下一行。与下方向键相同。
2EF7F	CMD_UP	(→) 把光标移至上一行，类似于上方向键。
2EF7D	CMD_DEB_LINE	(→) 把光标移至行首。与 RS+LEFT 相同。
2EF7E	CMD_END_LINE	(→) 把光标移至行尾。与 RS+RIGHT 相同。
2EF7A	CMD_PAGED	(→) 把光标向下移动一页，类似于 LS+DOWN。
2EF77	CMD_PAGEL	(→) 把光标向左移动一页，类似于 LS+LEFT。
2EF78	CMD_PAGER	(→) 把光标向右移动一页，类似于 LS+RIGHT。
2EF79	CMD_PAGEU	(→) 把光标向上移动一页，类似于 LS+UP。
2F2EE	DO<Skip	(→) 向左跳至单词开头。与编辑器 TOOL 菜单中的←SKIP 按钮相同。
2F2EF	DO>Skip	(→) 向右跳至下一个单词的开头。与编辑器 TOOL 菜单中的 SKIP→ 按钮相同。
2F2E4	DO>BEG	(→) 转到选择内容的开头（如果选择处于活动状态），否则转到 EditLine 的开头。与编辑器 TOOL 菜单中的→BEG 按钮相同。
地址	名称	说明
2F2E5	DO>END	(→) 转到选择内容的末尾。与编辑器 TOOL 菜单中的→END 按钮相同。没有选择内容时不移动。
2F2E6	GOTOLABEL	(→) 弹出包含 EditLine 中各标签的 CHOOSE 选择框。与编辑器 TOOL/GOTO 菜单中的 LABEL 按钮相同。

38.4.5 选择、剪切与粘贴以及剪贴板

地址	名称	说明
2EF83	CMD_STO_DEBUT	(# →) 设置开始标记，类似于 RS+BEGIN，但从堆栈取得位置。
2EF84	CMD_STO_FIN	(# →) 设置结束标记，类似于 RS+END，但从堆栈取得位置。
2EF85	RCL_CMD_DEB	(→ #) (→ #0) 提取 BEGIN 标记的位置。如果选择已被清除，则返回 ZERO。
2EF86	RCL_CMD_FIN	(→ #) (→ #0) 提取 END 标记的位置。如果选择已被清除，则返回 ZERO。
2F2DC	ClearSelection	(→) 取消选择已选文本，但不更改编辑器内容。把开始标记和结束标记都设为 ZERO。
2EF93	VERIF_SELECTION	(→ flag) 当 END 标记不为 ZERO 时返回 TRUE，表示选择处于活动状态。在对选择内容执行任何操作之前，请用此命令进行检查。
2EF8A	CMD_COPY	(→) 复制所选字符串，类似于 RS+COPY。
地址	名称	说明
2EF88	CMD_CUT	(→) 剪切字符串。实际上只是“删除”，并不复制到剪切缓冲区。因此，“正常”的 CUT 应为： :: CMD_COPY CMD_CUT ;
2F2FA	CMD_COPY.SBR	(→ \$) 把选择内容作为字符串放入堆栈。此命令可感知字体/样式。不建议使用它，因为如果不先把光标重新定位至选择内容的开头，它可能取得错误的文本样式。如果不使用字体，以下程序的作用类似： :: RCL_CMD RCL_CMD_DEB RCL_CMD_FIN SUB\$;

地址	名称	说明
2EF94	PASTE.EXT	(\$ →) 从堆栈粘贴，并处理字体和样式。在光标位置插入堆栈第 1 层中的字符串。它可以在粗体文本等内容的正中间插入普通文本。如果不使用样式或不同字体，CMD_PLUS 可能更快。
2F2E1	SELECT.LINE	(→) 选择当前行，并把光标置于行首。选择内容不包括行尾的 NEWLINE 字符。
2F2E2	SELECT.LINEEND	(→) 选择当前行，并把光标置于行尾。选择内容不包括行尾的 NEWLINE 字符。
2A085	(Clipboard!)	(\$ →) 把字符串存入剪贴板。
2A095	(Clipboard@)	(→ \$) 把剪贴板内容提取到堆栈。
2A0A5	(Clipboard0)	(→) 清空剪贴板。
2A0B5	(Clipboard?)	(→ flag) 剪贴板中是否有内容？

38.4.6 搜索与替换

地址	名称	说明
2F2F3	GET.W->	(→ #) 返回当前光标位置右侧下一个单词开头的位置。请注意此命令与 GET.W<- 的不对称性。
2F2F4	GET.W<-	(# → #') 从堆栈取得一个位置，并返回该位置左侧最近的单词开头的位置。请注意此命令与 GET.W-> 的不对称性。
2F2F2	FindStrInCmd	(\$find → \$find \$start \$end T) (\$find → \$find F) 从当前光标位置开始，在 EditLine 中查找字符串。搜索字符串保留在堆栈中，可能是为了进行重复搜索。返回匹配项的开始位置、结束位置和一个标志。此函数遵循内部区分大小写搜索标志的设置。
2F2E8	DOFIND	(→) 与编辑器 TOOL/SEARCH 菜单中的 FIND 按钮相同。弹出 FIND 窗口。
2F2EA	DONEXT	(→) 查找下一个匹配项。与编辑器 TOOL/SEARCH 菜单中的 NEXT 按钮相同。
2F2E9	DOREPL	(→) 与编辑器 TOOL/SEARCH 菜单中的 REP 按钮相同。弹出 REPLACE 窗口。
2F2EB	DOREPLACE	(→) 替换当前匹配项。与编辑器 TOOL/SEARCH 菜单中的 R 按钮相同。
地址	名称	说明
2F2EC	DOREPLACE/NEXT	(→) 替换当前匹配项并移至下一个匹配项。与编辑器 TOOL/SEARCH 菜单中的 R/N 按钮相同。
2F2ED	REPLACEALL	(→) 替换缓冲区中的所有匹配项。与编辑器 TOOL/SEARCH 菜单中的 ALL 按钮相同。
2F2FC	REPLACEALLNOSCREEN	(→) 与 REPLACEALL 类似，但不更新屏幕。这样速度快得多。

38.4.7 求值

地址	名称	说明
2F2DF	EditSelect	(→) 编辑当前选择内容。打开编辑器时只装入选择内容，随后即可编辑。按下 ENTER 后，编辑后的文本会重新插入先前的编辑环境。
2F2E3	EVAL.LINE	(→) 对当前行求值，并以求值结果替换该行。与 EVAL.SELECTION 类似，但不必先选择该行。
2F2FB	EVAL.SELECTION	(→) 对当前选择内容求值，并以求值结果替换选择内容。与编辑器 TOOL 菜单中的 EXEC 按钮相同。
地址	名称	说明
2F2F8	EXEC_CMD	(cmd algflag → obsel) 对 EditLine 中的选择内容运行命令。它取得两个参数：要运行的命令，以及一个指示应用该命令前应如何编译选择内容的标志。如果该标志为 TRUE 且 ALG 模式已开启，则使用 ALG 编译器，并从结果中移除 DOTAG :: xEVAL 序言。如果结果还要由另一个编辑器编辑，请使用此词条。选择内容作为对象留在堆栈第 1 层。
0B954	(RunInNewContext)	(ob →) 保存当前用户界面，对 ob 求值，然后恢复用户界面。可用于从一个应用程序内部运行另一个应用程序。

38.4.8 启动编辑器

```
::
"SOME STRING"
DUPLEN$ SWAP (get length)
InitEdLine (clear the editline)
EditString (string to editline)
STO_CURS_POS2 (cursor at end)
"Starting editor..."
FlashMsg (display before edit)
;
```

地址	名称	说明
2EEE9	EditString	(\$ →) 当前程序退出时开始编辑该字符串。如果程序退出时应激活编辑器，请使用此入口。在此入口前使用 InitEdLine 可清空 EditLine（如果需要）；否则，该字符串将插入当前 EditLine。此入口之后的所有代码都会先执行，然后才把控制权交给编辑器应用程序。例如：
2F19A	ViewLevel1	(ob → ob') 编辑堆栈第 1 层中的对象。
2F1AF	AlgObEdit	(ob → ob') 在代数模式下代替 ViewLevel1 使用。不执行 STARTED 和 EXITED。
2B2F2	(CallEditCmd:)	(ob → ob') 对运行流中的下一个对象求值，该对象通常是 ObEdit 之类的编辑命令。如果求值返回 FALSE，则把保存在临时变量中的原对象恢复到堆栈。如果求值返回 TRUE，则从堆栈中移除该 TRUE。
2EEE5	EditLevel1	(ob → ob')
2F1AE	ObEdit	(ob → ob' T) (ob → F) 编辑对象。用户取消时只返回 FALSE；否则返回修改后的对象和 TRUE。
011004	EQW3Edit	(symb → symb' T) (symb → F) 打开公式编辑器编辑表达式。以 ENTER 退出时返回新表达式和 TRUE；以 CANCEL 退出时只返回 FALSE。

38.4.9 其他

地址	名称	说明
25ED2	EditMenu	(→ {}) 返回编辑器菜单。
2EF73	?Space/Go>	(→) 插入一个 SPACE 字符，除非光标位置之前已经有一个空格。如果要确保随后回显的内容与其前面的单词至少相隔一个空格，请使用此词条。
地址	名称	说明
2EF76	AddLeadingSpace	(\$ → \$') 如果堆栈第 1 层中的字符串不是以空格开头，且编辑器中的光标位于非空白字符之后，则在字符串前添加一个空格。因此，下列程序会插入 DUP，并确保其两侧都有空格： :: "DUP" AddLeadingSpace AddTrailingSpace CMD_PLUS ;
2EF75	AddTrailingSpace	(\$ → \$') 如果堆栈第 1 层中的字符串尚未以空格结尾，则在字符串末尾添加一个空格。
2EF9A	CommandLineHeight	(→ #pix) 返回 EditLine 可见部分所占的像素行数。
2F2DB	DOTEXTINFO	(→) 显示 EditLine 信息屏幕。与编辑器 TOOL 菜单中的 INFO 按钮相同。
2F2F6	GET_CUR_FONT.EXT	(→ #) 返回光标下字符所用字体的 ID（以系统二进制整数表示）。
2EF96	NO_AFFCMD	(→) 通知下一次 CMD_PLUS 调用不要更新屏幕。如果在用户看到结果前还要插入更多内容，使用此词条可提高速度。
2F19E	DispCommandLine	(→) 重新显示命令行。
2F19F	?DispCommandLine	(→) 必要时重新显示命令行。
地址	名称	说明
2F2F7	PUT_STYLE	(# →) 更改当前点的样式。如果选择内容处于活动状态，则更改选择内容中文本的样式；否则更改随后输入文本的样式。它从堆栈取得一个 BINT，作为样式编号。编者认为，编辑器 TOOL/STYLE 菜单中的 ITALI 按钮可以用以下程序实现：

```
:: ERRSET PUT_STYLE
ERRTRAP ERRJMP ;
```

PUT_STYLE 不会 ABND 其临时环境，因此需要使用 ERRTRAP 结构来避开此错误。

地址	名称	说明
2F2F5	PUT_FONTE	(# →) 更改当前点的字体。其工作方式与 PUT_STYLE 命令类似。
2F2E7	SELECT.FONT	(→) 弹出 CHOOSE 选择框以选择字体。与编辑器 TOOL/STYLE 菜单中的 FONT 按钮相同。
2F2E0	ViewEditGrob	(→) 在光标附近查看当前正在 EditLine 中编辑的 grob。如果 EditLine 包含 GROB 10 10 FFFFFF..., 请把光标移至第一个"10"的"1"上。随后此入口点会显示该 grob。
地址	名称	说明
2EF92	XLINE_SIZE?	(ob → flag) 检查光标是否位于当前行之外。在 HP49G 编辑器中，可以把光标移至行尾右侧，而不实际延长该行。只有实际插入文本，或使用 CMD_DEL 把下一行接到该位置时，该行才会延长。如果光标不在换行符处或其左侧，此入口返回 TRUE。请注意，它会先从堆栈取得一个任意对象，因此调用前要在堆栈中放入某个对象。
27F47	<DelKey	(→ }) 返回←DEL 菜单键。
27F9A	>DelKey	(→ }) 返回 DEL→ 菜单键。
27EAF	<SkipKey	(→ }) 返回←SKIP 菜单键。
27EFB	>SkipKey	(→ }) 返回 SKIP→ 菜单键。
2EEE6	InitEd&Modes	(→) :: InitEdLine InitEdModes ;
2EEE7	InitEdLine	(→) :: DEL_CMD ;
2EEE8	InitEdModes	(→)
2F05E	SaveLastEdit	(\$ →) 如果历史记录已开启，则调用 CMD_STO。
2F326	CMDSTO	(\$ →) 把字符串添加到最近 4 条命令的列表中；可通过 CMD 键访问该列表。

第 39 章 绘图

本章中的命令涉及与绘图有关的各方面。此处的词条主要处理 PPAR 变量，其中包含绘图所用的参数。该变量是一个包含以下参数的列表：

```
{ (x min, y min) (x max, y max) indep res axes type depend}
```

各参数的含义如下:

参数	说明	默认值
(x_min, y_min)	表示查看范围左下角坐标的复数。	(-6.5,-3.1)
(x_max, y_max)	表示查看范围右上角坐标的复数。	(6.5,3.2)
indep	自变量。	X
res	分辨率。表示绘制点之间间隔的数值。	0
axes	表示坐标轴交点坐标的复数。它也可以是一个列表，用来表示该坐标及许多其他细节；本书不介绍这些细节。	(0,0)
type	指定绘图类型的（用户）命令名称。	FUNCTION
depend	因变量。	Y

39.1 参考

地址	名称	说明
2F162	CHECKPICT	($\rightarrow$) 检查 GBUFF 的大小。如果小于 131x64, 则把 GBUFF 恢复为默认大小 (131x64)。
2EF06	CKPICT	($\times$ PICT $\rightarrow$) 检查堆栈第 1 层是否为用户词 $\times$ PICT。如果是其他对象, 则产生错误 (SETTYPEERR)。
2F258	PICTRCL	($\times$ PICT $\rightarrow$ grob) 执行 CKPICT, 然后提取 GBUFF 并执行 TOTEMPOB。
2F355	MAKEPVARs	($\rightarrow$ { }) 在当前目录中创建默认 PPAR 变量并返回其值。
2F163	CHECKPVARs	($\rightarrow$ { }) 把当前路径中 PPAR 的内容提取到堆栈。如果 PPAR 不存在, 则在当前目录中创建它。如果现有 PPAR 无效, 则产生 "Invalid PPAR" 错误。
2F33D	GETPARAM	(# $\rightarrow$ ob) 从 PPAR 中提取第 # 个项目。不进行错误检查!
2F0FF	GETXMIN	($\rightarrow$ %) 如果 PPAR 列表中存在 XMIN, 则提取它; 否则在当前目录中创建默认 PPAR。
2F366	PUTXMIN	(% $\rightarrow$) 设置新的 XMIN 值。必要时创建 PPAR。
2F0FE	GETXMAX	($\rightarrow$ %) 如果 PPAR 列表中存在 XMAX, 则提取它; 否则在当前目录中创建默认 PPAR。
2F365	PUTXMAX	(% $\rightarrow$) 设置新的 XMAX 值。必要时创建 PPAR。
地址	名称	说明
2F100	GETYMIN	($\rightarrow$ %) 如果 PPAR 列表中存在 YMIN, 则提取它; 否则在当前目录中创建默认 PPAR。
2F368	PUTYMIN	(% $\rightarrow$) 为 YMIN 设置新值。必要时创建 PPAR。
2F10E	GETYMAX	($\rightarrow$ %) 如果 PPAR 列表中存在 YMAX, 则提取它; 否则在当前目录中创建默认 PPAR。
2F367	PUTYMAX	(% $\rightarrow$) 为 YMAX 设置新值。必要时创建 PPAR。
2F107	GETPMin&MAX	($\rightarrow$ C% C%) 返回 PMin 和 PMax。
2EEF2	PUTINDEP	(ID $\rightarrow$) 当参数为 ID 时使用的内部 $\times$ INDEP。
2EEF3	PUTINDEPLIST	({ $\rightarrow$) 当参数为列表时使用的内部 $\times$ INDEP。
2F0E8	INDEPVAR	($\rightarrow$ id) 提取自变量。如果它是列表, 则提取第一个元素。 :: GETINDEP DUPTYPELIST? ?CARCOMP ;
2F106	GETINDEP	($\rightarrow$ id) ($\rightarrow$ { }) 提取 PPAR 中的自变量字段。
2EEF5	GETPTYPE	($\rightarrow$ name) 使用 GETPARAM 提取绘图类型。
2EEF6	PUTPTYPE	(name $\rightarrow$) 设置新的绘图类型。必要时创建 PPAR。

地址	名称	说明
2F10D	GETRES	(→ %) 使用 GETPARAM 提取绘图分辨率。
2EEF4	PUTRES	(% →) 设置新的绘图分辨率。必要时创建 PPAR。
2F33E	GETSCALE	(→ % %') 提取绘图比例参数。
地址	名称	说明
2EEF1	PUTSCALE	(% %' →) 设置新的绘图比例。必要时创建 PPAR。
2EEEF	AUTOSCALE	(→) 内部 AUTO。
2EF60	DOGRAPHIC	(→) 设置 PICTURE 的滚动模式，实质上等同于 { }
25ECF	EQUATION	(→ ob) 提取存储在'EQ' 变量中的当前方程。
2F339	GetEqN	(#n → ob T) (#n → NULL\$ F) 如果 EQ 是方程列表，则获取第 #n 个方程。
25EB5	DORCLE	(→ ob) 提取 EQ 变量的内容；如果该变量不存在，则报错。
25EB6	DOSTOE	(ob →) 将 ob 存入变量 EQ。
2F297	XEQPURGEPICT	(xPICT →) 如果堆栈第 1 层的对象是 xPICT，则清除图形显示；否则报错。
2F105	GDISPCENTER	(→) 移至图形显示区的中心。
2EF01	DOPX>C	({ hxs hxs' } → C%) 将由两个十六进制字符串组成的列表转换为复数。用于绘图坐标。其逆运算是 DOC>PX。
2EF02	DOC>PX	(C% → { hxs hxs' }) 将复坐标点转换为由两个 HXS 数组成的列表。其逆运算是 DOPX>C。

第四部分

HP49计算机代数系统

第 40 章

HP49 CAS简介

HP49G 的主要创新之一，是内置了功能强大的计算机代数系统（CAS）。HP49G CAS 主要源自最初为 HP48 计算器编写的 ALG48 和 ERABLE 库。不过，在 HP49G 上，CAS 已完全集成到操作系统中，因此当参数需要 CAS 处理时，User RPL 运算符能够透明地访问 CAS。大量受支持的入口点可用于访问 CAS 的内部命令，使用户能够编写自己的程序和命令，以处理符号对象、矩阵和任意精度整数。

40.1 本部分各章存在的问题

本书 CAS 命令参考列表的初始版本源自 ALG48 和 ERABLE 的源文件¹。这种方法的问题在于：源代码中的各个条目并未完全按照功能排列。每个源文件处理 CAS 命令的某一领域，同时还在同一文件中包含许多实用程序例程。因此，例如元对象处理例程可能会出现在若干不同位置。不同文件中甚至还存在相似的此类例程（它们似乎执行相同的操作）。我们已投入大量精力按功能重新排列这些条目，但也认识到这项工作只取得了部分成功。要完成这项工作，还需要更深入地了解 CAS 及其内部机制。

完整的 CAS 文档还应包含大量有关 CAS 对象内部表示的资料，以及许多说明如何使用这些命令的示例。希望 Bernard Parisse 有一天能抽出时间，完整记录 HP49G CAS 的内部机制。目前，我们收录了他提供给我们的一份文档的略经编辑版本，其中介绍了 CAS 的一些重要方面。本部分的其余内容将仅列出入口点参考。

¹ 实际上，源自 Bernard Parisse 提供给我们的例程头的精简版本。

40.2 符号对象

CAS 对符号标量以及由这些对象组成的向量或矩阵进行运算。符号标量有 3 种表示形式，下面以表达式 ‘2X’ 为例说明。

用户表示	一个 SYMBOL 对象，即复合对象。示例表达式如下所示： SYMBOL Z2 ID X x* ;
元表示	将 SYMBOL 对象展开到堆栈中。对于 '2*X'，4 个展开对象 Z2、ID、X、x* 位于堆栈第 5 层至第 2 层，计数 #4 位于堆栈第 1 层。
列表表示	相对于变量列表（{ X }）的多项式系数（本例中为 { 2 0 }）。

从用户表示转换为元表示由 SYMBINCOMP 完成（它是 INNERCOMP 的泛化版本，可以处理整数等非符号对象）。

当有理规范形式无关紧要时，使用元表示来处理运算。它比符号表示更高效，因为不必展开和重建符号对象，所有操作都在堆栈上完成。第 12 章介绍了元对象的堆栈操作。一元和二元运算符的名称通常是在运算符名称前加上 addt（例如 addtSIN）。CASCOMPEVAL 是对元对象执行复杂操作的例程示例。它执行类似 COMPEVAL 的循环，但堆栈中使用的是元对象而不是符号对象。

当有理规范形式很重要时，使用列表表示。这适用于有理分式的积分、有理式简化、拉普拉斯变换、级数展开以及类似运算。转换的第一步，是找出使表达式为有理式所对应的变量列表。例如，

$$\frac{\sin(x) + y}{\cos(x) + y} \quad (40.1)$$

相对于 { sin(x) cos(x) y } 是有理式。给定一个符号对象或符号对象的列表/数组，用户级命令 LVAR 或 System RPL 命令 LVARext 会返回该变量列表。然后，按照该顺序，将表达式转换为关于此变量列表的两个多元多项式之商。

高斯整数表示为包含两个元素的次对象：

```
:: imaginary_part real_part ;
```

虚部和实部必须为整数。

平方根表示为 irrquad:

```
:: x« a b c x» ;
```

该对象表示 $a+b*\sqrt{c}$ 。

多项式定义为一个系数列表，其中的系数本身是多项式、常数（整数或高斯整数）或 irrquad。在这些多项式上构造的有理分式表示为 SYMBOL num deno x/；其中 num 和 deno 是互素多项式（在精确模式下）。

转换为列表表示的主要例程是 VXXLext。主要的反向转换例程是 R2SYM。有若干专用例程可用于转换符号对象的列表或元对象、将符号对象转换为元表示，或将列表表示转换为符号对象的元表示。这些专用例程效率更高，但使用难度也更大。

列表对象上的有理运算符已经实现（QAdd、QSub、QDiv、QMul、QNeg、RPext），同时还实现了欧几里得除法，并针对整数或高斯整数等情况提供了专用版本。

40.3 几个示例

在下列示例中，每行的注释表示执行当前命令后堆栈中的对象。

符号对象的有理式简化可以编码为

```
1 :: (symb)
FPTR2 ^LVARExt (symb lvar)
FPTR2 ^VXXLext (lvar n/d)
FPTR2 ^R2SYM (symb)
5 ;
```

“列表形式”的两个符号向量的标量积

```
1 :: (x y)
INNERCOMP #1+ROLL INNERCOMP
DUP#1= casedrop FPTR2 ^QMul
get1 (y1,...,yn-1,x1,...,xn,#n,yn)
5 ROTSWAP FPTR2 ^QMul (y1,...,yn-1,x1,...,xn-1,#n,xn*yn)
OVER ONE_DO
ROT 3PICK #2+PICK

FPTR2 ^QMul
FPTR2 ^QAdd
10 LOOP
(y1, ..., yn-1, #n, X.Y)
OVER #1+UNROLL #1- NDROP
;
```

第 41 章

类型检查与转换

本章中的词条用于检查第 40 章所述的特殊 CAS 对象，并在这些不同种类的对象之间进行转换。

41.1 参考

地址	名称	说明
157006	<code>^SYMBINCOMP</code>	<code>(symb → ob1 .. obN #n)</code> <code>(ob → ob #1)</code> <code>({} → {} #1)</code> 将符号对象展开为元对象。对于其他对象，则通过把 #1 压入堆栈，将其转换为仅含一个对象的元对象。
12A006	<code>^2SYMBINCOMP</code>	<code>(ob1 ob2 → meta1 meta2)</code> 对两个对象执行 <code>^SYMBINCOMP</code> 。
4D7006	<code>^VXXLext</code>	<code>(ob Lvar → Q)</code> 将对象转换为内部形式。该对象可以是符号对象、符号向量或符号矩阵。如果转换未成功，则清除 <code>vxxxflag</code> 。
400006	<code>^R2SYM</code>	<code>(lvar ob → ob)</code> 对标量对象进行反向转换。
4D8006	<code>^METALISTVXXL</code>	<code>(Meta → Meta)</code> 根据 LAM1 中的变量转换元对象的所有元素。
4D9006	<code>^VXXLFext</code>	<code>(n/d → z1/z2)</code> 转换不依赖任何变量的分数。
地址	名称	说明
4DA006	<code>^VXXL1ext</code>	<code>(n → z)</code> 转换不依赖任何变量的对象。
4DB006	<code>^VXXL0</code>	<code>(ob → Q)</code> 根据 LAM1 中的 <code>Lvar</code> 转换对象。
4DC006	<code>^VXXL2NR</code>	<code>(Meta → Q)</code> 将符号元对象转换为内部形式 (<code>LAM1=Lvar</code>)。设置 <code>nocareflag</code> 可避免平方根问题。
4DD006	<code>^VXXL2</code>	<code>(Meta → Q)</code> 将符号元对象转换为内部形式 (<code>LAM1=Lvar</code>)。
167006	<code>^TYPEIRRQ?</code>	<code>(ob → flag)</code> <code>ob</code> 是否为 <code>irrquad</code> ?
168006	<code>^DTYPEIRRQ?</code>	<code>(ob → ob flag)</code> 先执行 <code>DUP</code> ，再执行 <code>^TYPEIRRQ?</code> 。
177006	<code>^CKMATRIXELEM</code>	<code>(ob → ob)</code> 检查 <code>ob</code> 是否为有效的内部矩阵元素。有关用户矩阵元素，请参见 <code>CK[JNCK</code> 。
18F006	<code>^CKFPOLYext</code>	<code>(ob → ob)</code> 如果列表包含次对象或空列表，则产生错误。
190006	<code>^CK2FPOLY</code>	<code>(ob ob → ob ob)</code> 对两个对象执行 <code>CKFPOLYext</code> 。
19E006	<code>^CLEANIDLAM</code>	<code>(ob → ob)</code> 在不需要 <code>SYMB</code> 时将其消除。

第 42 章 整数

本章列出处理任意精度整数的函数。任意精度整数是 HP49 CAS 提供的一种新数值类型。有关该类型的说明，请参见第 5 章。

您会注意到，这里没有整数基本算术运算的词条。这是因为并不存在专用于整数的此类词条。请改用第 46 章所列的多项式词条，如 `^QAdd`、`^QMul` 等。

42.1 参考

42.1.1 内置整数

地址	名称	说明
	<code>^DROPZ0</code>	$(ob \rightarrow z0)$
2E0006		
	<code>^DROPZ1</code>	$(ob \rightarrow z1)$
2DF006		
	<code>^2DROPZ0</code>	$(21 \rightarrow z0)$
392006		
	<code>^NDROPZ0</code>	$(obn \dots ob1 \#n \rightarrow z0)$ 用 Z0 替换元对象。
3B3006		
	<code>^NDROPZ1</code>	$(obn \dots ob1 \#n \rightarrow z1)$ 用 Z1 替换元对象。
3B4006		

42.1.2 转换函数

地址	名称	说明
	<code>^#>Z</code>	$(\# \rightarrow Z)$ 将 bint 转换为 zint。
0EE006		
	<code>^R>Z</code>	$(\% \rightarrow z)$ 将实数转换为 zint。如果该数不是整数，请勿调用此词条。
0F5006		
地址	名称	说明
	<code>^R2Zext</code>	$(\% \rightarrow \% / Z)$ 将实数转换为 zint；如果该数不是整数，则转换为长实数并设置近似模式。
18D006		
	<code>^H>Z</code>	$(HXS \rightarrow Z / Error)$ 检查 HXS 是否为正确的 zint 数，并将其修整。
0ED006		
	<code>^S>Z</code>	$(\$ \rightarrow z)$ 将字符串中的十进制数转换为 zint。
0F2006		
	<code>^S>Z?</code>	$(\$ \rightarrow z T)$ $(\$ \rightarrow \$ F)$ 如果可以，则将字符串转换为 zint 并返回 TRUE；否则保留原字符串并返回 FALSE。
0F3006		
	<code>^CK1Z</code>	$(\$/\#/hxs \rightarrow Z)$ 检查整数。将字符串、bint 或 hxs 转换为 zint；其他对象类型会产生错误。
184006		
	<code>^CK2Z</code>	$(ob \ ob' \rightarrow Z \ Z')$ 与 <code>^CK1Z</code> 类似，但用于两个对象。
185006		
	<code>^CK3Z</code>	$(ob \ ob' \ ob'' \rightarrow Z \ Z' \ Z'')$ 与 <code>^CK1Z</code> 类似，但用于三个对象。
186006		
	<code>^CK&CONVINT</code>	$(symb \rightarrow zint)$ $(symb \rightarrow :: zint \ zint')$ 检查 sym 是否为 zint 或高斯整数，并进行转换。
202006		
	<code>^CK&CONV2INT</code>	$(symb \ symb' \rightarrow zint \ zint')$ $(symb \ symb' \rightarrow :: zint1 \ zint2 ; :: zint3 \ zint4 ;)$ 检查两个 sym 是否为 zint 或高斯整数，并进行转换。
203006		
	<code>^CONVBACKINT</code>	$(zint c \rightarrow symb)$
205006		
	<code>^CONVBACK2INT</code>	$(zint c \ zint c \rightarrow symb \ symb)$
204006		
	<code>^Z>ZH</code>	$(Z \rightarrow Z')$ 将十进制 Z 转换为十六进制 Z。
0F4006		
	<code>^Z2Sext</code>	$(Z \rightarrow '$Z')$ 将 Z 转换为数值字符串。该数被嵌入符号对象中，以便在代数式中使用。
18E006		

42.1.3 一般整数运算

地址	名称	说明
	<code>^ZTrim</code>	$(Z \rightarrow Z')$ 去除 Z 中不必要的前导半字节。计算表示该数所需的半字节数；如果它等于已用半字节数，则快速退出。否则分配新对象，复制有效尾数半字节并附加原符号。
101006		
	<code>^ZAbs</code>	$(Z \rightarrow Z)$ 取 Z 的绝对值。如果 Z 已为正，则不执行任何操作；否则复制对象并更改符号。
102006		
	<code>^ZABS</code>	$(Z \rightarrow Z')$ 绝对值。
50B006		

地址	名称	说明
0E0006	<code>^ZSQRT</code>	$(Z \rightarrow Z' \text{ flag})$ 计算平方根的整数部分。如果该数为完全平方数，则 flag 为 TRUE，表示返回结果是精确的。
3D0006	<code>^Mod</code>	$(Z \ Z_n \rightarrow Z')$ 计算 Z 模 N。
0DD006	<code>^ZMod</code>	$(Z_1 \ Z_2 \rightarrow Z')$
105006	<code>^ZNMax</code>	$(Z_1 \ Z_2 \rightarrow \text{NormMax}[Z_1, Z_2])$ 返回绝对值较大的整数。(如果 $ Z_1 \geq Z_2 $ ，则返回 Z1；如果 $ Z_1 < Z_2 $ ，则返回 Z2。)
106006	<code>^ZNMin</code>	$(Z_1 \ Z_2 \rightarrow \text{NormMin}[Z_1, Z_2])$ 返回绝对值较小的整数。(如果 $ Z_1 \leq Z_2 $ ，则返回 Z1；如果 $ Z_1 > Z_2 $ ，则返回 Z2。)
10D006	<code>^ZBits</code>	$(Z \rightarrow Z \ \#bits)$ 计算 Z 使用的位数。
10E006	<code>^ZBit?</code>	$(Z \ \#bit \rightarrow Z \ \text{flag})$ 测试 Z 中的某一位是否置位。与 ZBits 不同，计数从零开始。
2B7006	<code>^ZGCDext</code>	$(Z_2 \ Z_1 \rightarrow Z)$ 整数最大公约数。
2B8006	<code>^ZGcd</code>	$(Z_2 \ Z_1 \rightarrow Z)$ 此词条与 ZGCDext 相同。
地址	名称	说明
3D6006	<code>^IEGCDext</code>	$(a \ b \rightarrow d \ u \ v)$ 整数的贝祖等式。 $d = au + bv = \text{gcd}(a, b)$ 。
3D9006	<code>^INEGCD</code>	$(a \ b \rightarrow d \ u \ v)$
07C007	<code>^#FACT</code>	$(\# \rightarrow Z)$ 计算整数的阶乘。对 #0 - #FFFF 范围内的所有数均可正常工作，但数值达到某一点后会产生内存不足错误。
576006	<code>^factzint</code>	$(z \rightarrow z!)$ 长整数的阶乘。
215006	<code>^PA2B2</code>	$(z/\% \rightarrow a + bi)$ 内部 PA2B2。

42.1.4 整数因式分解与素数

地址	名称	说明
0C9006	<code>^ZFactor</code>	$(Z_s \rightarrow L_f)$ 分解带符号长整数。
0CA006	<code>^NFactor</code>	$(z \rightarrow \{\})$ 分解正长整数。
0CB006	<code>^NFactorSpc</code>	$(z \rightarrow \{\})$ 对正长整数进行半因式分解。它是在常规因式分解的基础上增加一次‘hopeless?’测试。
0CD006	<code>^SFactor</code>	$(S \rightarrow L_f)$ 分解短整数。使用 Pollard Rho 算法，并假定试除已经完成。因此，小于 4012009 的任何因子均已知为素数；对于更大的因子，在调用实际 Pollard Rho 算法之前先进行素性测试。Pollard Rho 不按大小顺序找出因子，因此完成全部因式分解后会对结果排序。
地址	名称	说明
0CE006	<code>^SPollard</code>	$(S \rightarrow S_1 \ S_2)$ 使用 Pollard Rho 算法将短整数分解为两部分。调用此子程序前应完成试除和素性测试，否则可能陷入无限循环。随机数生成器以用户级 RAND 命令为模型，但起始值不同。
0CF006	<code>^BFactor</code>	$(N \rightarrow L_f)$ 分解长整数。使用 Brent-Pollard 算法，并假定试除已经完成。找到小因子后，调用 SFactor 完成短整数的全部因式分解。由于因式分解可能耗时很久，因此使用执行时间测试来中止对极长整数的分解（每个合数的限制为 60s）。退出时对因子排序。
0D0006	<code>^BrentPow</code>	$(Z_a \ Z_1 \ Z_2 \ Z_n \ \#k \rightarrow Z)$ 用于 Brent-Pollard 因式分解的模 * + ^ 运算。输出为 $Z_1 * Z_2 + Z_a \bmod Z_n$ ，重复 k 次。注意，k=0 与 k=1 给出相同结果。此外，当 k≠0 时， $Z_1 \neq Z_2$ 没有意义。假定所有参数均为正数，并假定 $Z_a < 16$ 。在某些情况下 k 可能很大，因此使用 Montgomery 乘法或许更合适。
0D1006	<code>^ZPrime?</code>	$(Z \rightarrow \text{flag})$ 正整数的素性测试。据 Pinch 所述，商业软件包默认仅使用约 5-10 个底数，最多约 25 个。最新版本通常实现确定性测试。
0D2006	<code>^ZIsPrime?</code>	$(Z \rightarrow \text{flag})$ 正整数的概率素性测试。
地址	名称	说明
0D3006	<code>^SIsPrime?</code>	$(S \rightarrow \text{flag})$ 测试正短 Z 是否为素数。M-R 测试对整数 ≤ 3 失效，因此开始时单独测试这些数。为方便起见，也把 0 和 1 定义为素数。
0D4006	<code>^BIsPrime?</code>	$(S \rightarrow \text{flag})$ 测试正长 Z 是否为素数。

地址	名称	说明
0D5006	<code>^BRabin</code>	$(Z \rightarrow \#base \rightarrow Z \text{ flag})$ 对正长整数执行 Miller-Rabin 测试。如果 <code>base</code> 证明该数为合数，则返回 TRUE；否则返回 FALSE。
0D6006	<code>^ZTrialDiv2</code>	$(Z \rightarrow Z' \ \#n)$ 从整数中除去因子 2。 <code>#n</code> 是从该数中提取出的 2 的幂。符号也会得到正确处理，尽管 ALG48 中从不需要这样做（绝对 Z）。
0D7006	<code>^ZTrialPrime?</code>	$(Z \rightarrow \text{flag})$ 对正整数进行试除素性测试。适用于 $Z \geq 3$ ($Z=2$ 时返回 FALSE)。
0D8006	<code>^ZTrialDiv</code>	$(Z \rightarrow Mf \ Z')$ 对正整数进行试除。如果 Z' 为一，则已完成全部因式分解。长整数试除并不太慢，因为除以短整数很快。还会检查商，因此小于 2000^2 的最后一个因子也会自动检出。
0C7006	<code>^Prime+</code>	$(Z \rightarrow Z')$ 返回下一个素数 ($Z' > Z$)。
0C8006	<code>^Prime-</code>	$(Z \rightarrow Z')$ 返回上一个素数 ($Z' < Z$)。

42.1.5 高斯整数

地址	名称	说明
114007	<code>^TYPEGAUSSINT?</code>	$(ob \rightarrow \text{flag})$ 检查 <code>ob</code> 是否为高斯整数。
115007	<code>^DTYPEGAUSSINT?</code>	$(ob \rightarrow ob \ \text{flag})$ 检查 <code>ob</code> 是否为高斯整数。
116007	<code>^DUPTYPEGAUSSINT?</code>	$(ob \rightarrow ob \ \text{flag})$ 检查 <code>ob</code> 是否为高斯整数。
187006	<code>^CK1Cext</code>	$(ob \rightarrow \text{flag})$ 检查对象是否为整数或高斯整数。
15D006	<code>^CXRIext</code>	$(C \rightarrow Zre \ Zim)$ 返回高斯整数的实部和虚部。
2B5006	<code>^CGCDext</code>	$(C2 \ C1 \rightarrow C)$ 高斯整数的最大公约数。
4D5006	<code>^CSQFFext</code>	$(C \rightarrow \{ \text{factor1} \ \text{mult1} \ \dots \ \text{factn} \ \text{multn} \})$ 高斯整数的因式分解。这并不是 C 在高斯整数上的完全分解，因为 c 的实部与虚部的最大公约数只在 R 上分解。
4D4006	<code>^SECOSQFFext</code>	$(:: x \ll a \ b \ c \ x \gg \rightarrow \{ \text{fact1} \ \text{mult1} \ \dots \ \text{factn} \ \text{multn} \})$ 分解 <code>irrquad</code> 和高斯整数。
4D6006	<code>^SUMSQRext</code>	$(Z \rightarrow Z \ C)$ 返回高斯整数 C ，使 $ C ^2=Z$ 。 Z 必须为 2，或者满足 $Z \equiv 1 \pmod{4}$ 。如果 $Z \not\equiv 1 \pmod{4}$ ，则产生“ Z is not 1 mod 4”错误。 Z 应为素数，以确保解存在。
518006	<code>^CNORMext</code>	$(C \rightarrow C ^2)$ 高斯整数的模平方。

42.1.6 整数测试

地址	名称	说明
265C1	<code>Z=</code>	$(Z \ Z' \rightarrow \text{flag})$
265C6	<code>Z<></code>	$(Z \ Z' \rightarrow \text{flag})$
265BC	<code>Z<</code>	$(Z \ Z' \rightarrow \text{flag})$
265D0	<code>Z<=</code>	$(Z \ Z' \rightarrow \text{flag})$
265B7	<code>Z></code>	$(Z \ Z' \rightarrow \text{flag})$
265CB	<code>Z>=</code>	$(Z \ Z' \rightarrow \text{flag})$
0F8006	<code>^QIsZero?</code>	$(Q \rightarrow \text{flag})$ 测试 Q 是否为零。假定列表只包含列表或十六进制数！
0F7006	<code>^DupQIsZero?</code>	$(Q \rightarrow Q \ \text{flag})$ 复制 Q 并测试 Q 是否为零。假定列表只包含列表或十六进制数！
0FA006	<code>^ZIsOne?</code>	$(Z \rightarrow \text{flag})$ 测试 Z 是否为 1。
0F9006	<code>^DupZIsOne?</code>	$(Z \rightarrow Z \ \text{flag})$ 复制 Z ；如果 Z 为 1，则返回 TRUE。
109006	<code>^DupZIsTwo?</code>	$(Z \rightarrow Z \ \text{flag})$ 如果 Z 为 2，则返回 TRUE。
0FC006	<code>^ZIsNeg?</code>	$(Z \rightarrow \text{flag})$ 测试 Z 是否为负数。
0FB006	<code>^DupZIsNeg?</code>	$(Z \rightarrow Z \ \text{flag})$ 测试 Z 是否为负数。
10A006	<code>^DupZIsEven?</code>	$(Z \rightarrow Z \ \text{flag})$ 测试 Z 是否为偶数。
107006	<code>^ZNLT?</code>	$(Z1 \ Z2 \rightarrow \text{flag})$ 如果 $ Z1 < Z2 $ ，则为 TRUE。
19A006	<code>^OBJINT?</code>	$(z/\% \rightarrow z \ \text{flag})$ 测试 <code>Obj</code> 是否为整数。
19B006	<code>^OBJPOSINT?</code>	$(z/\% \rightarrow z \ \text{flag})$ 测试 <code>Obj</code> 是否为小于 Z_{small} 的正整数。

地址	名称	说明
19C006	^CKINT>0	(Obj $\rightarrow$ Obj flag) 测试 Obj 是否为严格正整数。
198006	^METAINT?	(Meta $\rightarrow$ Meta flag) 测试 Meta 是否为整数。
地址	名称	说明
199006	^METAPOSINT?	(Meta $\rightarrow$ Meta flag) 测试 Meta 是否为小于 Zsmall 的正整数。
0CC006	^DupTypeS?	(Z $\rightarrow$ Z flag) 测试 Z 是否为短整数 (≤ 64 位)。

第 43 章

矩阵

CAS的符号矩阵是 HP49 上用于表示矩阵的一种新对象。与 HP48 以来已有的旧式数组对象不同，这些矩阵内部可以包含符号表达式。数组内部也可以包含不同类型的对象。

这种矩阵实际上是复合对象，因此可以对其使用第 11 章中的函数。

下列矩阵 $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ 的反汇编应当能够说明如何使用 MASD 创建矩阵，以及矩阵为何实际上是复合对象：

```
1 MATRIX
  MATRIX
    ZINT 1
    ZINT 2
  5 ;
  MATRIX
    ZINT 3
    ZINT 4
  ;
10 ;
```

还应注意，下文所述用于处理符号矩阵的大多数函数（即使不是全部）也适用于由列表构成的列表。原因很明显：这些矩阵与列表的列表具有相同结构，只有 prolog 地址发生变化。

第 10.1 节中介绍的一些用于处理旧式 HP48 数组的词条也适用于符号矩阵。

43.1 参考

43.1.1 创建矩阵和重新确定矩阵维数

地址	名称	说明
371006	<code>^MATIDN</code>	<code>(M/z/% → M')</code> 创建单位矩阵。
372006	<code>^MATCON</code>	<code>(M ob → [ob])</code> 根据矩阵创建常量矩阵。
373006	<code>^MAKEARRY</code>	<code>({#el} symb → [])</code> <code>({#rows #cols} symb → [[]])</code> 根据 ob 类型创建常量矩阵/数组。
345006	<code>^DIMRANM</code>	<code>({} → M')</code> 根据维数创建符号随机矩阵。
344006	<code>^MATRANM</code>	<code>(M → M')</code> 将矩阵的所有元素改为随机生成的元素。
374006	<code>^OBJDIMS2MAT</code>	<code>(ob {} → M)</code> 根据维数和 ob 创建常量矩阵。
375006	<code>^LCPROG2M</code>	<code>(#n #m prg → M)</code> 使用程序填充指定大小的矩阵。prg 必须接受两个参数并返回一个参数。进入 MAKE2DMATRIX 时，以 Z 整数形式提供索引。
376006	<code>^MAKE2DMATRIX</code>	<code>(#n #m prg → M)</code> 根据大小和程序创建矩阵（带堆栈检查）。prg 必须接受 2 个参数并返回 1 个参数。进入 MAKE2DMATRIX 时，以 Z 整数形式提供索引。
377006	<code>^make2dmatrix</code>	<code>(#n #m prg → meta-M)</code> 根据大小和程序创建元矩阵（带堆栈检查）。prg 必须接受 2 个参数并返回 1 个参数。进入 make2dmatrix 时，以 Z 整数形式提供索引。
341006	<code>^MATREDIM</code>	<code>(M {} → M')</code> 更改矩阵大小，按需删除元素和/或添加零。
地址	名称	说明
342006	<code>^VRRDM</code>	<code>([]/[[]] {} → [])</code> 向量右侧重新定维：在右侧添加 0。
343006	<code>^VRRDMmeta</code>	<code>(meta #l → meta-#l)</code> 元对象右侧重新定维：在右侧添加 0。

43.1.2 转换

地址	名称	说明
16A006	<code>^{}TO[]</code>	<code>({} → [])</code> 将列表的列表表示转换为矩阵。不检查元素类型。
17A006	<code>^LIST2MATRIX</code>	<code>({} → [])</code> <code>({} → [[]])</code> <code>(ob → ob)</code> 将符号列表转换为矩阵。不检查矩阵是否有效；可使用 DTYPFMAT? 进行检查。
16B006	<code>^[]TO{ }</code>	<code>([] → {})</code> 将矩阵转换为列表的列表。

地址	名称	说明
179006	<code>^MATRIX2LIST</code>	$([] \rightarrow \{\})$ $([][] \rightarrow \{\{\}\})$ $(ob \rightarrow ob) $ 将符号矩阵转换为列表。
17E006	<code>^ARRAY2MATRIX</code>	$([] \rightarrow [])$ $([][] \rightarrow []) $ 必要时将数组转换为符号数组。
175006	<code>^SAMEMATRIX</code>	$(M1\ M2 \rightarrow M1\ M2\ flag) $ 如果其中一个对象是符号数组，则将两个数组都转换为符号形式。符号矩阵返回 TRUE，数值矩阵返回 FALSE。
176006	<code>^SAMEMATSCTYPE</code>	$(M\ ob \rightarrow M\ ob\ flag) $ 如果 M 是数值矩阵且 ob 不是浮点数，则将矩阵转换为符号形式。符号形式返回 TRUE，数值形式返回 FALSE。
003007	<code>^ArryToList</code>	$([]/[][] \rightarrow \{\}/\{\{\}\}) $ 将普通数组转换为列表的列表；遇到符号数组时出错。
地址	名称	说明
17D006	<code>^MATEXPLODE</code>	$([[ob1..obn]] \rightarrow ob1..obn\ [[ob1..obn]]) $ 展开符号矩阵的元素，并保留原矩阵。

43.1.3 测试

地址	名称	说明
16C006	<code>^DUPNULL[]?</code>	$(ob \rightarrow ob\ flag) $ 测试对象是否为空数组。
359006	<code>^NULLVECTOR?</code>	$(v \rightarrow flag) $ 如果向量为空，则返回 TRUE。
16F006	<code>^CKSAME SIZE</code>	$(array1\ array2 \rightarrow array1\ array2\ flag) $ 测试 array1 和 array2 的大小是否相同。
170006	<code>^DTYPENDO?</code>	$(ob \rightarrow ob\ flag) $ 测试对象是否为方形符号矩阵。将数值数组转换为符号矩阵。
173006	<code>^2DMATRIX?</code>	$(ob \rightarrow ob\ flag) $ 测试对象是否为二维矩阵。

43.1.4 矩阵计算

地址	名称	说明
320006	<code>^MAT+</code>	$(M2\ M1 \rightarrow M2+M1) $ 矩阵加法。
321006	<code>^MADD</code>	$(M2\ M1 \rightarrow M2+M1) $ 矩阵加法。
322006	<code>^MAT-</code>	$(M2\ M1 \rightarrow M2-M1) $ 矩阵减法。
323006	<code>^MSUB</code>	$(M2\ M1 \rightarrow M2-M1) $ 矩阵减法。
324006	<code>^VADD</code>	$(V2\ V1 \rightarrow V2+V1) $ 向量加法。
325006	<code>^V SUB</code>	$(V2\ V1 \rightarrow V2-V1) $ 向量减法。
326006	<code>^MAT*</code>	$(M2\ M1 \rightarrow M2*M1) $ 执行矩阵乘积，并检查大小和类型。
327006	<code>^MMMULT</code>	$(M2\ M1 \rightarrow M2*M1) $ 矩阵乘法。
328006	<code>^MVMULT</code>	$(M\ V \rightarrow V')$ 矩阵与向量的乘积。
329006	<code>^SCL*MAT</code>	$(ob\ M \rightarrow M*ob) $ 标量乘矩阵。
32A006	<code>^MAT*SCL</code>	$(M\ ob \rightarrow M*ob) $ 矩阵乘标量。
地址	名称	说明
32B006	<code>^VPMULT</code>	$(V\ ob \rightarrow V')$ 向量乘以标量。
335006	<code>^MATSQUARE</code>	$(M \rightarrow M*M) $ 计算矩阵的平方。
32C006	<code>^MAT^</code>	$(M\ z/\% \rightarrow M')$ 计算矩阵的整数次幂。
32D006	<code>^MATCROSS</code>	$([]\ []' \rightarrow []) $ 计算向量积。
32E006	<code>^MATDOT</code>	$(V2\ V1 \rightarrow ob) $ 经检查后计算标量积。
32F006	<code>^RNDARRY</code>	$(M\ \% \rightarrow M) $ 对数组进行四舍五入。
330006	<code>^TRCARRY</code>	$(M\ \% \rightarrow M) $ 截断数组。
332006	<code>^MAT/SCL</code>	$(M\ ob \rightarrow M/ob) $ 矩阵除以标量。
333006	<code>^MAT/</code>	$(V\ M \rightarrow M^{-1}*V) $ 用矩阵“除”向量。
334006	<code>^MATCHS</code>	$(M \rightarrow -M) $ 改变矩阵各元素的符号。
34E006	<code>^MATINV</code>	$(M \rightarrow M^{-1}) $ 计算逆矩阵。
336006	<code>^MATCONJ</code>	$(M \rightarrow M')$ 计算矩阵的共轭。

地址	名称	说明
337006	<code>^MATRE</code>	$(M \rightarrow \text{re}[M])$ 取矩阵的实部。
338006	<code>^MATIM</code>	$(M \rightarrow \text{im}[M])$ 取矩阵的虚部。
339006	<code>^MATTRACE</code>	$(M \rightarrow \text{trace})$ 计算矩阵的迹。
33A006	<code>^MATTRN</code>	$(M \rightarrow M')$ 对矩阵进行转置和共轭。
33C006	<code>^mattran</code>	$(M \rightarrow \text{Meta-}M')$ 转置矩阵，并返回元矩阵。
33D006	<code>^mattrn</code>	$(\text{Meta-}M \rightarrow \text{Meta-}M')$ 转置元矩阵。
346006	<code>^MATDET</code>	$(M \rightarrow \text{det})$ 通过完全展开（而非行约简）计算行列式。
347006	<code>^MATRDET</code>	$(M \rightarrow \text{det})$ 使用行约简计算行列式。
348006	<code>^MATFNORM</code>	$(M \rightarrow \text{ob})$ 计算 Frobenius 范数。
349006	<code>^MATRNORM</code>	$(M \rightarrow \text{ob})$ 计算行范数。
地址	名称	说明
34A006	<code>^MATCNORM</code>	$(M \rightarrow \text{ob})$ 计算列范数。
174006	<code>^MATRIXDIM</code>	$(\text{ob} \rightarrow \#)$ 返回对象的符号矩阵维数。

43.1.5 线性代数和高斯约简

地址	名称	说明
34C006	<code>^MATREF</code>	$(M \rightarrow M')$ 返回矩阵的行阶梯形。
34B006	<code>^MATRREF</code>	$(M \rightarrow M')$ 返回矩阵的简化行阶梯形。
34F006	<code>^MATREFRREF</code>	$(M \text{ \#full_ref} \rightarrow M \text{ list } M')$ 如果 #full_ref 为 1，则返回简化行阶梯形；否则仅返回行阶梯形。
367006	<code>^MATRIXRCI</code>	$(\text{ncol } i \text{ } M \text{ const} \rightarrow M')$ 将符号矩阵 M 的第 #i 行乘以常数。 ncol 不使用；之所以保留它，是因为从 laRCI 内部调用时的堆栈状态如此。
368006	<code>^MATRIXRCIJ</code>	$(\text{ncol } \#i \text{ } \#j \text{ } M \text{ const} \rightarrow M')$ 执行 $L_j \leftarrow c \cdot L_i + L_j$ 。 ncol 不使用；之所以保留它，是因为从 laRCI 内部调用时的堆栈状态如此。
350006	<code>^INXREDext</code>	$(\text{Lvar } \#full_ref \text{ } M \rightarrow \text{Lvar pivot } M)$ 对矩阵执行索引约简。
351006	<code>^METAMATRED</code>	$(\text{Meta-}M \text{ Lvar } \#full_red \rightarrow \text{meta-}M \text{ Lvar pivot})$ 对元矩阵执行约简。
352006	<code>^METAPIVOT</code>	$(\text{meta-}M \text{ \#l } \#c \rightarrow \text{meta-}M \text{ \#l } \#l' \text{ } \#c' \text{ flag})$ 从第 #l 行开始在第 #c 列中搜索主元。未找到主元时 flag 为 FALSE ；找到时， #l' 为所在行，且 #c 更新为 #c' 。
354006	<code>^PIVOTFLOAT</code>	$(\text{float} \rightarrow \text{float_modulus})$ 返回浮点数的模，用于主元选择。
34D006	<code>^MATRANK</code>	$(M \rightarrow Z/\%)$ 计算矩阵的秩。

43.1.6 线性方程组求解器

地址	名称	说明
080007	<code>^LINSOLV</code>	$(b \text{ } a \rightarrow y)$ 求解 $y'=ay+b$ 。
0F4007	<code>^SOLVEMETASYST</code>	$(\text{meta-}M \rightarrow d \text{ meta-sol } T)$ $(\text{meta-}M \rightarrow F)$ 以元表示形式求解线性方程组。 meta-sol 已约简为相同的分母 d 。
0F5007	<code>^REDUCEMETASYST</code>	$(\text{meta-}M \rightarrow \text{meta-}M')$ 约简以元表示形式给出的线性方程组。
0F6007	<code>^REDUCEMETAPSYST</code>	$(\text{meta-}M \rightarrow \text{meta-}M')$ 约简以元表示形式给出的线性方程组，但不约简元矩阵的最后一列。这适用于求解参数位于最后一列的线性方程组。
0F7007	<code>^SOLVECRAMER</code>	$(\text{meta-}M \rightarrow d \text{ meta-sol } T)$ $(\text{meta-}M \rightarrow F)$ 求解 Cramer 方程组。 meta-matr 必须完全约简。 meta-sol 被约简为相同的分母。当维数不匹配时， d flag 为 FALSE 。
355006	<code>^SYSText</code>	$(M \text{ linc} \rightarrow \text{linc linc' res cas_p})$ 求解矩阵形式的线性方程组。
356006	<code>^STOSYSText</code>	$(M2 \text{ } M1 \rightarrow M2 \text{ list})$ 存储线性方程组。
357006	<code>^MAKESYSText</code>	$(M \text{ eq } M_{\text{inc}} \rightarrow M_{\text{eq}} M_{\text{lidnt}} \text{ flag})$ 将线性方程转换为矩阵，并检查方程相对于 lidnt 是否为线性。

43.1.7 其他矩阵操作

地址	名称	说明
35A006	<code>^FINDELN</code>	<code>({ } A → # flag)</code> 返回数组中元素 <code>{ }</code> 的索引 <code>#</code> 。
地址	名称	说明
35B006	<code>^PULLEL[S]</code>	<code>(A # → A el)</code> 从数组中提取索引为 <code>#</code> 的元素。为提高数组处理速度，数组类型测试由汇编代码执行。
35C006	<code>^BANGARRY</code>	<code>(el # M → M')</code> 将 <code>el</code> 放入矩阵 <code>M</code> 中索引为 <code>#</code> 的位置。
35D006	<code>^PUT[]</code>	<code>(el #i V → V)</code> 用指定元素替换向量的第 <code>#i</code> 个分量。
17B006	<code>^LENMATRIX</code>	<code>([] → #el)</code> <code>([[]] → #row)</code> 返回数值矩阵的元素数或符号矩阵的行数。
33E006	<code>^MATSUB</code>	<code>(M rmin n rows cmin n cols { #m #n } → M')</code> 从矩阵中提取子矩阵。
340006	<code>^MATREPL</code>	<code>(M1 M2 → M2')</code> 用源矩阵 <code>M1</code> 替换目标矩阵 <code>M2</code> 的一部分。 <code>LAM1</code> 到 <code>9</code> 必须按 <code>llib/LMain.s</code> 中的方式绑定 (<code>9:r 8:c 7:dmatrix 6:f 5:md 4:nd 3:smatrix 2:ms 1:ns</code>)。从矩阵 <code>d</code> 的第 <code>r</code> 行、第 <code>c</code> 列开始复制。
35F006	<code>^MATRIX>DIAG</code>	<code>(A n cols+1 ndiags → V)</code> 提取对角项。之所以使用 <code>n cols+1</code> ，是因为 <code>MATRIX>DIAG</code> 在 <code>la>DIAG</code> 内部调用。
360006	<code>^MATRIXDIAG></code>	<code>(ncol+1 diagV dlen dims{ } → M)</code> 根据对角项向量构造矩阵。
361006	<code>^la+ELEMsym</code>	<code>(V ob %i → V')</code> 在符号向量的第 <code>%i</code> 行插入元素。
362006	<code>^INSERTROW[]</code>	<code>(V ob #i → V)</code> <code>(M V #i → M')</code> 在符号向量/矩阵的第 <code>#i</code> 行插入元素/向量。检查 <code>0 < #i < #n + 1</code> ，但不检查矩阵/向量的大小。
363006	<code>^insertrow[]</code>	<code>(ob #i meta → meta)</code> 在元对象的第 <code>#i</code> 个位置插入元素/向量。检查 <code>0 < #i < #n + 1</code> ，但不检查向量大小。
地址	名称	说明
364006	<code>^INSERTCOL[]</code>	<code>(M V #i → M')</code> 在符号矩阵的第 <code>#i</code> 列插入向量。检查 <code>0 < #i < #n + 1</code> ，但不检查矩阵/向量的大小。
365006	<code>^INSERT[]ROW[]</code>	<code>(M3 M2 #i → M)</code> 从第 <code>#i</code> 行开始，将矩阵 <code>M2</code> 插入矩阵 <code>M3</code> 。检查 <code>0 < #i < #n+1</code> ，但不检查矩阵大小。
366006	<code>^INSERT[]COL[]</code>	<code>(M3 M2 #i → M)</code> 从第 <code>#i</code> 列开始，将矩阵 <code>M2</code> 插入矩阵 <code>M3</code> 。检查 <code>0 < #i < #n + 1</code> ，但不检查矩阵大小。
369006	<code>^MATRIXCSWAP</code>	<code>(M #c #c' → M)</code> 交换符号矩阵的第 <code>c</code> 列和第 <code>c'</code> 列。
36A006	<code>^MATRIXRSWAP</code>	<code>(M #r #r' → M)</code> 交换符号矩阵的第 <code>r</code> 行和第 <code>r'</code> 行。
0AC003	<code>^SWAPROWS</code>	<code>(M % %' → M')</code> 交换矩阵中的两行。 <code>xRSWP</code> 的内部版本。
36B006	<code>^MATRIX-ROW</code>	<code>(M #r → M' lr)</code> 从 <code>M</code> 中提取第 <code>#r</code> 行。检查边界。
36C006	<code>^METAMAT-ROW</code>	<code>(meta-M #r → meta-M lr)</code> 从元矩阵中提取第 <code>#r</code> 行。检查边界。
36D006	<code>^MATRIX-COL</code>	<code>(M #c → M cc)</code> 从矩阵中提取第 <code>#c</code> 列。检查边界。
36E006	<code>^METAMATCSWAP</code>	<code>(meta-M #c #c' → meta-M)</code> 交换元矩阵的第 <code>c</code> 列和第 <code>c'</code> 列。
36F006	<code>^METAMATRSWAP</code>	<code>(meta-M #l #l' → meta-M)</code> 交换元矩阵（或向量）的第 <code>l</code> 行和第 <code>l'</code> 行。
370006	<code>^STOMAText</code>	<code>(M →)</code> 将矩阵存入当前目录中的 <code>'MATRIX'</code> 。
地址	名称	说明
378006	<code>^ADDMATOBJext</code>	<code>(array ob → array array)</code> <code>(ob array → array array)</code> 用于数值矩阵与符号对象的加法。
379006	<code>^VUNARYOP</code>	<code>(v op → V)</code> 应用一元运算 <code>op(v[i])</code> 得到 <code>V[i]</code> 。
37A006	<code>^VBINARYOP</code>	<code>(V2 V1 binop → V)</code> 即使 <code>V2</code> 与 <code>V1</code> 的维数不同也能工作。
37B006	<code>^PEVAL</code>	<code>(V r → P[r])</code> Horner 求值，其中 <code>V</code> 的元素表示多项式的系数。

43.1.8 特征值、特征函数和约简

地址	名称	说明
37C006	<code>^MATEGVL</code>	<code>(M → V) </code> 与 <code>EGVL</code> 类似，计算矩阵的特征值。
37F006	<code>^MATEGV</code>	<code>(M → V) </code> 与 <code>EGV</code> 类似，计算矩阵的特征值/特征向量。
37E006	<code>^MADJ</code>	<code>(M → M^-1 P[M] P[lambda]) </code> 计算逆矩阵、矩阵多项式和特征多项式。
380006	<code>^JORDAN</code>	<code>(M → pmin pcar {evect} {eval})</code> <code>(pmax pcar → pmin pcar {evect} {eval}) </code> 计算特征值/特征函数。
22D006	<code>^FLAGJORDAN</code>	<code>(M →) </code> <code>JORDAN</code> 的内部版本。
381006	<code>^QXA</code>	<code>(symb lidnt → M lidnt) </code> 将符号二次型转换为矩阵二次型。
224006	<code>^FLAGQXA</code>	<code>(symb lidnt → M lidnt) </code> <code>QXA</code> 的内部版本。
382006	<code>^AXQ</code>	<code>(M lidnt → symb lidnt) </code> 将矩阵二次型转换为符号二次型。
225006	<code>^FLAGAXQ</code>	<code>(M lidnt → symb lidnt) </code> <code>AXQ</code> 的内部版本。
383006	<code>^GAUSS</code>	<code>(symb → D P symb') </code> 对二次型执行高斯约简（符号形式）。
226006	<code>^FLAGGAUSS</code>	<code>(symb lidnt → symb') </code> <code>GAUSS</code> 的内部版本。
384006	<code>^SYLVESTER</code>	<code>(M → D P) </code> 对二次型执行高斯约简（矩阵形式）。
227006	<code>^FLAGSYLVESTER</code>	<code>(M → P D) </code> <code>SYLVESTER</code> 的内部版本。
228006	<code>^PCAR</code>	<code>([[]] → symb) </code> <code>PCAR</code> 的内部版本。

第 44 章 表达式操作

本章中的条目用于操作以符号对象形式表示的表达式。
(有关处理元对象形式符号表达式的条目, 请参见第 45 章。) 这些条目涉及表达式的合并与展开、三角和指数变换, 以及在表达式中代入值。

44.1 参考

44.1.1 基本运算和函数应用

地址	名称	说明
125006	<code>^x+ext</code>	(ob2 ob1 $\rightarrow$ ob2+ob1) 符号加法, 检查无穷大。
126006	<code>^x-ext</code>	(ob2 ob1 $\rightarrow$ ob2-ob1) 符号减法, 检查无穷大。
127006	<code>^x*ext</code>	(ob2 ob1 $\rightarrow$ ob2*ob1) 符号乘法, 检查无穷大。
129006	<code>^x/ext</code>	(ob2 ob1 $\rightarrow$ ob2/ob1) 符号除法, 检查无穷大。
12B006	<code>^x^ext</code>	(ob power $\rightarrow$ ob^power) 幂运算。
12C006	<code>^EXPAND^</code>	(x y $\rightarrow$ x^y=exp[y*ln[x]]) 带简化的幂运算。如果 y 是整数的分数, 则改用 XROOT^。
4FB006	<code>^QNeg</code>	(ob $\rightarrow$ -ob) 符号取负。
4FC006	<code>^RNEGext</code>	(ob $\rightarrow$ -ob) 符号取负。
地址	名称	说明
4FA006	<code>^SWAPRNEG</code>	(ob2 ob1 $\rightarrow$ ob1 -ob2) 先执行 SWAP, 再进行符号取负。
4FE006	<code>^RREext</code>	(ob $\rightarrow$ Re (ob)) 符号实部。
4FD006	<code>^SWAPRRE</code>	(ob2 ob1 $\rightarrow$ ob1 Re (ob2)) 先执行 SWAP, 再执行 RREext。
500006	<code>^RIMext</code>	(ob $\rightarrow$ Im (ob)) 符号虚部。
4FF006	<code>^SWAPRIM</code>	(ob1 ob2 $\rightarrow$ ob2 Im (ob1)) 先执行 SWAP, 再执行 RIMext。
501006	<code>^xREext</code>	(symb $\rightarrow$ symb') 复数的实部。仅展开 + - * / ^。
503006	<code>^xIMext</code>	(symb $\rightarrow$ symb') 复数的虚部。仅展开 + - * / ^。
505006	<code>^RCONJext</code>	(ob $\rightarrow$ Conj (ob)) 符号复共轭。
50D006	<code>^xABSext</code>	(ob $\rightarrow$ abs (ob)) 符号 ABS 函数。
50A006	<code>^RABSext</code>	(ob $\rightarrow$ abs (ob)) 内部 ABS。使用内部表示。
52A006	<code>^xINVext</code>	(ob $\rightarrow$ 1/ob) 符号求逆。
557006	<code>^xSYMINV</code>	(symb $\rightarrow$ 1/symb) 符号求逆。
553006	<code>^xSQext</code>	(symb $\rightarrow$ sq (symb)) 符号平方。
555006	<code>^xSYMSQ</code>	(symb $\rightarrow$ symb^2) 符号平方。
51B006	<code>^XSQRext</code>	(ob $\rightarrow$ sqrt (ob)) 不处理正负号。
51C006	<code>^XSQRext</code>	(ob $\rightarrow$ sqrt (ob)) 如果 nocareflag 已清除, 则尝试返回正平方根。
52B006	<code>^xvext</code>	(ob $\rightarrow$ sqrt (ob)) 符号平方根, 检查 0 和 1。
552006	<code>^xSYMSQRT</code>	(symb $\rightarrow$ sqrt (symb)) 符号平方根。
521006	<code>^CKLN</code>	(ob $\rightarrow$ ln (ob)) 对分数进行特殊处理的符号 LN。不使用内部表示。
地址	名称	说明
522006	<code>^xLNNext</code>	(ob $\rightarrow$ ln (ob)) 不处理分数的符号 LN。
525006	<code>^EXPANDLN</code>	(ob $\rightarrow$ ln (ob)) 使用内部表示的符号 LN。在切换到内部表示之前, 检查 ABS、0 和 1; 在实数模式下还检查 ob=exp(x)。
528006	<code>^REALLN</code>	(ob $\rightarrow$ ln (ob)) 实数参数的内部自然对数。
526006	<code>^CMPLXLN</code>	(ob $\rightarrow$ ln (ob)) 内部复数自然对数。
527006	<code>^LNATANext</code>	(ob $\rightarrow$ ln (ob)) 复数的内部自然对数。

地址	名称	说明
529006	<code>^xEXPext</code>	$(y \text{ d } n \rightarrow \exp(y*n/d*i*\pi))$ 符号 EXP, 检查 0、无穷大和 $i*k*\pi/12$, 其中 k 为整数。检查 $d=1,2,3,4,6$ 。
52C006	<code>^xCOSext</code>	$(ob \rightarrow \cos(ob))$ 符号 COS, 检查 0 和 $\pi/12$ 的倍数。还检查 $ob=\text{acos}(x)$ 或 $ob=\text{asin}(x)$ 。
536006	<code>^xSYMCOS</code>	$(ob \rightarrow \cos(ob))$ 符号余弦。
533006	<code>^xACOSext</code>	$(ob \rightarrow \text{acos}(ob))$ 符号 ACOS。检查 0、无穷大和表值。
53F006	<code>^xSYMACOS</code>	$(ob \rightarrow \text{acos}(ob))$ 符号反余弦。
52D006	<code>^xSINext</code>	$(ob \rightarrow \sin(ob))$ 符号 SIN, 检查 0 和 $\pi/12$ 的倍数。还检查 $ob=\text{acos}(x)$ 或 $ob=\text{asin}(x)$ 。
538006	<code>^xSYMSIN</code>	$(ob \rightarrow \sin(ob))$ 符号正弦。
532006	<code>^xASINext</code>	$(ob \rightarrow \text{asin}(ob))$ 符号 ASIN。检查 0、无穷大和表值。
53D006	<code>^xSYMASIN</code>	$(ob \rightarrow \text{asin}(ob))$ 符号反正弦。
52E006	<code>^xTANext</code>	$(ob \rightarrow \tan(ob))$ 符号 TAN。检查 0 和 $\pi/12$ 的倍数。还检查 $ob=\text{atan}(x)$ 。
53A006	<code>^xSYMTAN</code>	$(ob \rightarrow \tan(ob))$ 符号正切。
534006	<code>^xATANext</code>	$(ob \rightarrow \text{atan}(ob))$ 符号 ATAN。检查 0、无穷大和表值。
541006	<code>^xSYMATAN</code>	$(ob \rightarrow \text{atan}(ob))$ 符号反正切。
52F006	<code>^xCOSHext</code>	$(ob \rightarrow \cosh(ob))$ 符号 COSH。检查 0、无穷大和 $\text{acosh}(x)$ 。
545006	<code>^xSYMCOSH</code>	$(ob \rightarrow \cosh(ob))$ 符号双曲余弦。
地址	名称	说明
54E006	<code>^xACOSHext</code>	$(\text{symb} \rightarrow \text{acosh}(\text{symb}))$ 符号 ACOSH。
550006	<code>^xSYMACOSH</code>	$(\text{symb} \rightarrow \text{acosh}(\text{symb}))$ 符号反双曲余弦。
530006	<code>^xSINHext</code>	$(ob \rightarrow \sinh(ob))$ 符号 SINH。检查 0、无穷大和 $\text{asinh}(x)$ 。
543006	<code>^xSYMSINH</code>	$(ob \rightarrow \sinh(ob))$ 符号双曲正弦。
54B006	<code>^xASINHext</code>	$(\text{symb} \rightarrow \text{symb}')$ 符号 ASINH。
54D006	<code>^xSYMASINH</code>	$(\text{symb} \rightarrow \text{asinh}(\text{symb}))$ 符号反双曲正弦。
531006	<code>^xTANHext</code>	$(ob \rightarrow \tanh(ob))$ 符号 TANH。检查 0 和 $\text{atanh}(x)$ 。
547006	<code>^xSYMTANH</code>	$(ob \rightarrow \tanh(ob))$ 符号 TANH。
548006	<code>^xATANHext</code>	$(\text{symb} \rightarrow \text{symb}')$ 符号 ATANH。
54A006	<code>^xSYMATANH</code>	$(ob \rightarrow \text{atanh}(ob))$ 符号反双曲正切。
55F006	<code>^xSYMFLOOR</code>	$(\text{symb} \rightarrow \text{symb}')$ 符号下取整。
561006	<code>^xSYMCEIL</code>	$(\text{symb} \rightarrow \text{symb}')$ 符号上取整。
563006	<code>^xSYMIP</code>	$(\text{symb} \rightarrow \text{symb}')$ 取符号对象的整数部分。
565006	<code>^xSYMFP</code>	$(\text{symb} \rightarrow \text{symb}')$ 取符号对象的小数部分。
567006	<code>^xSYMXPON</code>	$(\text{symb} \rightarrow \text{symb}')$ 取符号对象的指数部分。
569006	<code>^xSYMMANT</code>	$(\text{symb} \rightarrow \text{symb}')$ 取符号对象的尾数部分。
56B006	<code>^xSYMLNP1</code>	$(\text{symb} \rightarrow \text{symb}')$ 对符号对象执行 LNPI。
56D006	<code>^xSYMLOG</code>	$(\text{symb} \rightarrow \text{symb}')$ 对符号对象执行 LOG。
56F006	<code>^xSYMALOG</code>	$(\text{symb} \rightarrow \text{symb}')$ 对符号对象执行 ALOG。
571006	<code>^xSYMEXPM1</code>	$(\text{symb} \rightarrow \text{symb}')$ 对符号对象执行 EXPM1。
572006	<code>^factorial</code>	$(\text{symb} \rightarrow \text{symb}!)$ 符号阶乘。
573006	<code>^facts</code>	$(\text{symb} \rightarrow \text{symb}!)$ 符号阶乘。
575006	<code>^xSYMFACT</code>	$(\text{symb} \rightarrow \text{symb}!)$ 符号阶乘。
578006	<code>^xSYMNOT</code>	$(\text{symb} \rightarrow \text{symb}')$ 对符号对象执行 NOT。
128006	<code>^x=ext</code>	$(ob2 \text{ ob1} \rightarrow ob2=ob1)$ 符号相等关系。

44.1.2 三角函数和指数运算符

地址	名称	说明
408006	<code>^COS2TAN/2</code>	(symb → symb') $x \rightarrow (1 - \{\tan(x/2)\}^2) / (1 + \{\tan(x/2)\}^2)$
40B006	<code>^SIN2TAN/2</code>	(symb → symb') $x \rightarrow 2 \tan(x/2) / (1 + \{\tan(x/2)\}^2)$
40E006	<code>^TAN2TAN/2</code>	(symb → symb') $x \rightarrow 2 \tan(x/2) / (1 - \{\tan(x/2)\}^2)$
412006	<code>^COS2TAN</code>	(symb → symb2) $x \rightarrow 1 / \sqrt{1 + \{\tan(x)\}^2}$
414006	<code>^SIN2TAN</code>	(symb → symb') $x \rightarrow \tan(x) / \sqrt{1 + \{\tan(x)\}^2}$
41A006	<code>^LNP12LN</code>	(symb → symb') $x \rightarrow \ln(x+1)$
41B006	<code>^LOG2LN</code>	(symb → symb') $x \rightarrow \log(x)$
41C006	<code>^ALOG2EXP</code>	(symb → symb') $x \rightarrow \text{alog}(x)$
41D006	<code>^EXPM2EXP</code>	(symb → symb') $x \rightarrow \exp(x) - 1$
41E006	<code>^SQRT2LNEXP</code>	(symb → symb') $x \rightarrow \exp(\ln(x)/2)$
41F006	<code>^sqrt2lnexp</code>	(meta → meta') $x \rightarrow \exp(\ln(x)/2)$
420006	<code>^TAN2EXP</code>	(symb → symb') $x \rightarrow (\exp(i2x) - 1) / (i * (\exp(i2x) + 1))$
422006	<code>^ASIN2LN</code>	(symb → symb') $x \rightarrow i * \ln(x + \sqrt{x^2 - 1}) + \pi / 2.$
424006	<code>^ACOS2LN</code>	(symb → symb') $x \rightarrow \ln(x + \sqrt{x^2 - 1}) / i$
427006	<code>^TAN2SC</code>	(symb → symb') $x \rightarrow \sin(x) / \cos(x)$
42A006	<code>^SIN2TC</code>	(symb → symb') $x \rightarrow \cos(x) * \tan(x)$
42C006	<code>^COS2ext</code>	(symb → symb') $x \rightarrow \sqrt{1 - \{\sin(x)\}^2}.$
42E006	<code>^SIN2ext</code>	(symb → symb') $x \rightarrow \sqrt{1 - \{\cos(x)\}^2}.$

地址	名称	说明
431006	<code>^ATAN2ASIN</code>	(symb → symb') $x \rightarrow \text{asin}(x / \sqrt{x^2 + 1})$
434006	<code>^ASIN2ATAN</code>	(symb → symb') $x \rightarrow \text{atan}(x / \sqrt{1 - x^2})$
437006	<code>^ASIN2ACOS</code>	(symb → symb') $x \rightarrow \pi / 2 - \text{acos}(x)$
43C006	<code>^ACOS2ASIN</code>	(symb → symb') $x \rightarrow \pi / 2 - \text{asin}(x)$
43D006	<code>^ATAN2LNext</code>	(symb → symb') $x \rightarrow i / 2 * \ln((i+x) / (i-x))$
440006	<code>^TAN2SC2</code>	(symb → symb') $x \rightarrow (1 - \cos(2x)) / \sin(2x)$
442006	<code>^TAN2CS2</code>	(symb → symb') $x \rightarrow \sin(2x) / (1 + \cos(2x))$
444006	<code>^SIN2EXPext</code>	(symb → symb') $x \rightarrow (e^{(i*x)} - 1 / e^{(i*x)}) / (2i)$
446006	<code>^COS2EXPext</code>	(symb → symb') $x \rightarrow (e^{(i*x)} + 1 / e^{(i*x)}) / 2$
448006	<code>^SINH2EXPext</code>	(symb → symb') $x \rightarrow (e^x - 1 / e^x) / 2$
44A006	<code>^COSH2EXPext</code>	(symb → symb') $x \rightarrow (e^x + 1 / e^x) / 2$
44C006	<code>^TANH2EXPext</code>	(symb → symb') $x \rightarrow (e^{2x} - 1) / (e^{2x} + 1)$
44E006	<code>^ASINH2LNext</code>	(symb → symb') $x \rightarrow \ln(x + \sqrt{x^2 + 1})$
450006	<code>^ACOSH2LNext</code>	(symb → symb') $x \rightarrow \ln(x + \sqrt{x^2 - 1})$
452006	<code>^ATANH2LNext</code>	(symb → symb') $x \rightarrow \ln((1+x) / (1-x)) / 2$
454006	<code>^XROOT2ext</code>	(symb1 symb2 → symb') $x y \rightarrow \exp(\ln(y) / x)$
45A006	<code>^LN2ATAN</code>	(symb → symb') $x \rightarrow \ln(x)$

44.1.3 简化、求值和替换

地址	名称	说明
45B006	<code>^VAR=LIST</code>	(idnt { } → { }') 将初始列表中的所有元素替换为 idnt=element。
464006	<code>^SYMBEXEC</code>	(ob symb → ob') 如果 symb 是方程，则在 ob 中执行相应的变量替换；否则，尝试确定使 ob 为零的 symb。请注意，变量替换也适用于用户函数的替换。

地址	名称	说明
465006	<code>^MEVALext</code>	(ob { } { } ' → ob ') 将 ob 中出现的 list2 元素全部替换为 list1 中的对应元素。搜索 ob 时，从外层表达式向内层表达式进行。list2 和 list1 可以包含二级程序。如果设置了 vxflag，则 SIGN var 保持不变。
466006	<code>^CASNUMEVAL</code>	(symb list1 list2 → symb ') 对符号对象求值。两个列表的格式为 list1={idnt/lam1... idnt_n/lam_n} list2={value1...value_n}。在以 value1...value_n 替换前，不会对 list1 中的 idnt/lam 求值。
467006	<code>^CASCOMPEVAL</code>	(symb → symb ') 对符号对象求值。
468006	<code>^REPLACE2BY1</code>	(symb idnt a → symb ') 对符号对象求值，并将 idnt 替换为一个值；例如，对 $X=1/2$ 时的 $F(X)$ 求值。
469006	<code>^NR_REPLACE</code>	(symb idnt a → symb ') 与 REPLACE2BY1 类似，但禁止对 INT 求值。
46B006	<code>^CASCRUNCH</code>	(ob → %) 与 CRUNCH 类似，但在近似模式下运行。
46C006	<code>^APPROXCOMPEVAL</code>	(symb → symb ') 与 CASCOMPEVAL 类似，但在近似模式下运行。
11A007	<code>^ALGCASCOMPEVAL</code>	(expr → expr)
地址	名称	说明
297006	<code>^SLVARext</code>	(Lvar → Lvar ') 简化列表中所有被视为变量的元素。
298006	<code>^SIMPLIFY</code>	(symb → symb ') 像 EVAL 一样简化一个对象。
299006	<code>^SIMPlext</code>	(symb → symb ') 像 EXPAND 一样简化一个对象。对象必须是符号对象、实数或复数。
29A006	<code>^SYMEXPAN</code>	(symb → symb ') 像 EXPAN 一样简化一个对象。对象必须是 symb/real/cmplx。
29B006	<code>^SIMPVAR</code>	(ob → ob ') 简化变量。
2A0006	<code>^SIMPSYMS</code>	(inf sup fcn var → int (inf , sup , fcn , var))
2A2006	<code>^SIMPUSERFCN</code>	(ob1..obn #n ob → id []) 简化用户函数，并检查用户函数的导数。Ob 必须是 id、符号对象、二级程序或 romptr。
2A3006	<code>^EVALUSERFCN</code>	(V1..Vn #n fcn → f []) 在进行堆栈检查的情况下对用户函数求值。
2A4006	<code>^SIMP </code>	(ob list → ob ') 执行 WHERE 运算符。
2A9006	<code>^SIMPext</code>	(ob1 ob2 → ob1' ob2') 简化内部表示形式的两个对象。它会检查 o2 是否既非复数也非 irquad，因为用 larg 分解相应分式会产生“Try to recover Memory”。
2AD006	<code>^SIMPgcdext</code>	(o1 o2 gcd → o1/gcd o2/gcd) 将 o1 和 o2 除以 gcd。
2AE006	<code>^SIMP3ext</code>	(a b → g a'' b'') 计算 $g = \text{gcd}(a,b)$ 、 $a'' = a/g$ 和 $b'' = b/g$ 。
2B9006	<code>^TSIMP2ext</code>	(symb → symb) 进行超越函数简化。仅将 sqrt、^ 和 XROOT 转换为 EXP/LN。LN 以 -1/INV[-LN[]] 的形式返回，供 SERIES 使用。
地址	名称	说明
2BA006	<code>^TSIMPext</code>	(symb → symb) 进行超越函数简化。将超越函数转换为 EXP 和 LN。
2BB006	<code>^TSIMP3ext</code>	(symb → symb)

44.1.4 合并与展开

地址	名称	说明
26E006	<code>^COLCext</code>	(symb → symb ') 按 symb 的当前变量进行因式分解，并对 symb 的整数内容进行因式分解。
2FE006	<code>^TCOLLECT</code>	(symb → symb ') 先进行三角函数线性化，再合并角度相同的正弦项和余弦项。
2FF006	<code>^SIGMAEXPext</code>	(symb → symb ') 转换为 exp 和 ln，并进行指数函数线性化。
300006	<code>^LINEXPext</code>	(symb → Meta) Meta = arg_exp1 coef1 ... arg_expn coefn #2n.
301006	<code>^SIGMAEXP2ext</code>	(Meta → symb) 将 arg_exp/coef_meta 反向转换为符号对象。
303006	<code>^SINEXPA</code>	(symb → symb ') 展开 SIN。
316006	<code>^LNEXPA</code>	(symb → symb ') 展开 LN。
31C006	<code>^MTRIG2SYMB</code>	(Meta → symb) 将 trig-meta 反向转换为符号对象。
309006	<code>^COSEXPA</code>	(symb → symb ') 展开 COS。

地址	名称	说明
30F006	<code>^EXPEXPA</code>	(symb → symb') 展开 EXP。
31B006	<code>^LINEXPA</code>	(symb → Meta) 交替排列三角运算符和系数。
31D006	<code>^LNCOLCext</code>	(symb → symb') 合并对数项。
地址	名称	说明
31F006	<code>^TEXPAext</code>	(symb → symb) 超越函数展开的主程序。
240006	<code>^EXLR</code>	('a=b' → a b) (ob → x ob) 内部方程拆分器。

44.1.5 三角变换

地址	名称	说明
407006	<code>^HALFTAN</code>	(symb → symb') 将三角函数转换为半角的 TAN。
411006	<code>^TRIGTAN</code>	(symb → symb') 将 sin 和 cos 转换为同一角的 tan。
416006	<code>^TRIGext</code>	(symb → symb') 应用 $\sin^2 + \cos^2 = 1$ 简化三角表达式。如果设置了标志 -116，则尝试只保留 sin；否则只保留 cos。
417006	<code>^HYP2EXPext</code>	(symb → symb') 将双曲函数转换为 exp 和 ln，并将 XROOT 和 ^ 转换为 exp 和 ln。
418006	<code>^EXPLNext</code>	(symb → symb') 将所有超越函数转换为 exp 和 ln。
419006	<code>^SERIESEXPLN</code>	(symb → symb') 将 sqrt、^ 和 XROOT 转换为 EXP/LN。
426006	<code>^TAN2SCext</code>	(symb → symb') 将 tan 转换为 sin/cos。
429006	<code>^SIN2TCext</code>	(symb → symb') 将 sin 转换为 cos*tan。
430006	<code>^ATAN2Sext</code>	(symb → symb') 使用 $\text{asin}(x) = \text{atan}(x/\sqrt{1-x^2})$ 将 ATAN 转换为 ASIN。
433006	<code>^ASIN2Text</code>	(symb → symb') 使用 $\text{asin}(x) = \text{atan}(x/\sqrt{1-x^2})$ 将 ASIN 转换为 ATAN。
地址	名称	说明
436006	<code>^ASIN2Cext</code>	(symb → symb') 使用 $\text{asin}(x) = \pi/2 - \text{acos}(x)$ 将 ASIN 转换为 ACOS。
43A006	<code>^ACOS2Sext</code>	(symb → symb') 使用 $\text{acos}(x) = \pi/2 - \text{asin}(x)$ 将 ACOS 转换为 ASIN。
43F006	<code>^TAN2SC2ext</code>	(symb → symb') 将 TAN 转换为倍角的 SIN/COS。如果设置了标志 -116，则调用 TAN2SC2；否则调用 TAN2CS2。
456006	<code>^LN2ext</code>	(symb → symb') 如果 symb 包含 x，则返回 -1/inv(-ln(x))；否则返回 ln(x)。供 SERIES 使用。

44.1.6 除法、最大公因数与最小公倍数

地址	名称	说明
3E8006	<code>^PSEUDODIV</code>	(Q2 Q1 → a Q2*a/Q1 Q2*a/Q1)
3EA006	<code>^BESTDIV2</code>	(o2 o1 → quo mod)
3EC006	<code>^QUOText</code>	(o2 o1 → o2 div o1) 两个对象的欧几里得商，即使 o2 mod o1=0 也可使用。
3ED006	<code>^NEWWDIVext</code>	(ob2 ob1 → quo mod) 作欧几里得除法；ob2 和 ob1 可以是分式，结果会返回 Q 的一个分式。
3F3006	<code>^QUOTOBJext</code>	(a a-1...a0 bb_1...b0 #b #a flag → r q) SRPL 欧几里得除法：仅当 flag 为 TRUE 时，第 2 步才计算余数 r。
3F4006	<code>^DIVISIBLE?</code>	(a b → a/b T) (a b → ob F) 如果 b 整除 a，则返回 TRUE 和商；否则返回 FALSE。
3F5006	<code>^QDiv?</code>	(a b → a/b T) (a b → F) 如果 b 整除 a，则返回 TRUE 和商；否则返回 FALSE。
地址	名称	说明
3F6006	<code>^FastDiv?</code>	(P Q → P/Q PmodQ T) 作欧几里得除法。假定 P 和 Q 的系数为整数或高斯整数。在复数模式下，或稀疏短除法失败时，返回 FALSE。
3F7006	<code>^POTENCEext</code>	(z1 z2 → q r) 对小整数作逐步欧几里得除法。
2A5006	<code>^DENOLCMext</code>	(list → ob) 计算列表各元素分母的最小公倍数。如果输入不是列表，则返回该对象的分母。
2A6006	<code>^METADENOLCM</code>	(Meta → ob) 计算 Meta 各元素分母的最小公倍数。

地址	名称	说明
2B1006	<code>^LPGCDext</code>	<code>({} → {} ob) </code> 计算列表中所有元素的最大公因数。该算法远非最优。
2B2006	<code>^SLOWGCDext</code>	<code>(c 1 A B → c* gcd (A,B)) </code> 用欧几里得算法计算多项式最大公因数。当 A 或 B 包含 irrquads 时使用。 c 是原多项式各内容的最大公因数，在 <code>GCDHEUext</code> 失败后返回。
2B3006	<code>^QGcd</code>	<code>(ob2 ob1 → gcd) </code> 通用内部最大公因数。 (<code>LAM2: GCDext ob1, ob2 → pgcd</code>)。

第 45 章

符号元对象处理

本章介绍用于修改 **meta** 的命令字；**meta** 是展开后的符号对象。这些命令字用于修改表达式或对表达式进行运算。

45.1 参考

45.1.1 基本表达式操作

地址	名称	说明
157006	^SYMBINCOMP	(symb $\rightarrow$ ob1 .. obN #n) (ob $\rightarrow$ ob #1) ({} $\rightarrow$ { } #1) 将符号对象展开为元对象。对于其他对象，则将 #1 压入堆栈，把它们转换为仅含一个对象的元对象。
386006	^m-1&m+1	(meta $\rightarrow$ meta&1&+ meta&1&-) 创建 meta 的两个副本。在第一个副本后追加 1 和 +，在第二个副本后追加 1 和 -。
387006	^meta1/meta	(meta $\rightarrow$ meta 1&meta&/) 复制 meta ，并对它所表示的表达式取倒数。
388006	^1&meta	(Meta $\rightarrow$ 1&Meta) 将数字 1 前置到 meta 。
389006	^meta/2	(Meta $\rightarrow$ Meta&2&/) 将表达式除以二。
38A006	^addt2	(Meta $\rightarrow$ Meta&2) 将数字 2 追加到 meta 。
38B006	^addt/	(Meta $\rightarrow$ Meta&/) 将除法运算符追加到 meta 。
地址	名称	说明
38C006	^meta2*	(Meta $\rightarrow$ 2&Meta&*) 将表达式乘以 2。
459006	^metai*	(meta $\rightarrow$ meta*i) 将 meta 乘以 i。
38D006	^meta1-sq	(Meta $\rightarrow$ 1&Meta&SQ&-) 将 x 变为 1-x ² ，其中 x 是原表达式。
38E006	^metasq+1	(Meta $\rightarrow$ Meta&SQ&1&+) 将 x 变为 x ² +1，其中 x 是原表达式。
38F006	^metasq-1	(Meta $\rightarrow$ Meta&SQ&1&-) 将 x 变为 x ² -1，其中 x 是原方程。
390006	^meta-1	(Meta $\rightarrow$ Meta&1&-) 从表达式中减去一。
398006	^addt^	(Meat $\rightarrow$ Meta&^) 将幂运算符追加到元对象。
39C006	^top&addt*	(meta2 meta1 $\rightarrow$ meta2*meta1) top& addt*。不作检查。
39D006	^top&addt/	(meta2 meta1 $\rightarrow$ meta2/meta1) top& addt/。不作检查。
39E006	^addti	(meta $\rightarrow$ meta&i) 将 i（虚数单位）追加到表达式。

45.1.2 基本运算与函数应用

地址	名称	说明
393006	^metaadd	(Meta1 Meta2 $\rightarrow$ Meta1+Meta2) 通过简单化简将两个元对象相加。metaadd 检查 Meta1/2 是否为 Z0 或 ONE。
3AB006	^MetaAdd	(Meta2 Meta1 $\rightarrow$ Meta2+Meta1) 通过简单化简将两个元对象相加。检查无穷值，然后调用 metaadd。
1CE006	^ckaddt+	(Meta1 Meta2 $\rightarrow$ Meta1+Meta2) 通过简单化简将两个元对象相加。
地址	名称	说明
394006	^metasub	(Meta1 Meta2 $\rightarrow$ Meta1+Meta2) 通过简单化简将两个元对象相减。metasub 检查 Meta1/2 是否为 Z0 或 ONE。
3AD006	^MetaSub	(Meta2 Meta1 $\rightarrow$ Meta2-Meta1) 通过简单化简将两个元对象相减。检查无穷值，然后调用 metasub。
1CF006	^ckaddt-	(Meta1 Meta2 $\rightarrow$ Meta1+Meta2) 通过简单化简将两个元对象相减。
395006	^metamult	(Meta1 Meta2 $\rightarrow$ Meta1*Meta2) 通过简单化简将两个元对象相乘。检查 meta1 、 meta2 是否为 Z0 或 Z1，并检查 xNEG。
3AF006	^MetaMul	(Meta2 Meta1 $\rightarrow$ Meta2*Meta1) 通过简单化简将两个元对象相乘。检查无穷值/0，然后调用 metamult。
1CD006	^ckaddt*	(Meta1 Meta2 $\rightarrow$ Meta1*Meta2) 通过简单化简将两个元对象相乘。

地址	名称	说明
396006	$\wedge$ metadiv	(Meta2 Meta1 $\rightarrow$ Meta2/Meta1) 通过简单化简将两个元对象相除。检查无穷值和 0、meta2=1 或 Z-1，并检查 xNEG。
3B1006	$\wedge$ MetaDiv	(Meta2 Meta1 $\rightarrow$ Meta2/Meta1) 通过简单化简将两个元对象相除。检查无穷值和 0，然后调用 metadiv。
3F1006	$\wedge$ DIVMETAOBJ	(o1...on #n ob $\rightarrow$ {o1/ob...on/ob}) 将元对象的所有元素除以 ob，并测试 o=1。
397006	$\wedge$ meta $\wedge$	(Meta ob $\rightarrow$ Meta&ob $\wedge$) 将表达式提升到指定幂。如果 ob=1，则直接返回表达式。对于整数幂，检查 meta 末尾是否存在 xNEG。
地址	名称	说明
399006	$\wedge$ metapow	(Meta2 Meta1 $\rightarrow$ Meta2 $\wedge$ Meta1) 将表达式提升到任意表达式所指定的幂。如果 Meta1 的长度为 ONE，则调用 meta $\wedge$ 。
3B5006	$\wedge$ MetaPow	(Meta2 Meta1 $\rightarrow$ Meta2 $\wedge$ Meta1) 幂运算。检查无穷值，然后调用 metapow。
39B006	$\wedge$ metaxroot	(Meta2 Meta1 $\rightarrow$ Meta2&XROOT&Meta1) 对表达式开根。
3B9006	$\wedge$ metaneg	(meta $\rightarrow$ meta) 仅检查 meta 是否以 xNEG 结尾。
3BA006	$\wedge$ metackneg	(meta $\rightarrow$ meta) 与 metaneg 类似，但检查 meta=ob ONE。
3B7006	$\wedge$ MetaNeg	(Meta $\rightarrow$ Meta) 对 meta 取负。仅检查以 xNEG 结尾的 meta。
502006	$\wedge$ xSYMRE	(meta $\rightarrow$ meta') 取得复数 meta 的实部。仅展开 + - * / $\wedge$ 。
504006	$\wedge$ xSYMIM	(meta $\rightarrow$ meta') 取得复数 meta 的虚部。仅展开 + - * / $\wedge$ 。
50E006	$\wedge$ addtABS	(Meta $\rightarrow$ Meta') 对 meta 执行 ABS。先执行 CRUNCH 以确定符号。
510006	$\wedge$ addtABSEXACT	(Meta $\rightarrow$ Meta') 对 meta 执行 ABS。不执行 CRUNCH，只用精确方法确定符号。
511006	$\wedge$ addtSIGN	(Meta $\rightarrow$ Meta') 对 meta 执行 SIGN。
513006	$\wedge$ addtARG	(Meta $\rightarrow$ Meta') 对 meta 执行 ARG。
12D006	$\wedge$ addtXROOT	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 XROOT。与 o2 $\wedge$ o1 相比，XROOT(o2,o1) 为 o1 $\wedge$ [1/o2]。
12F006	$\wedge$ addtMIN	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 MIN。
131006	$\wedge$ addtMAX	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 MAX。
地址	名称	说明
133006	$\wedge$ addt<	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 <。
135006	$\wedge$ addt<=	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 <=。
137006	$\wedge$ addt>	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 >。
139006	$\wedge$ addt>=	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 >=。
13B006	$\wedge$ addt==	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 ==。
13D006	$\wedge$ addt!=	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 !=。
13F006	$\wedge$ addt%	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 %。
141006	$\wedge$ addt%CH	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 %CH。Meta2*(1+Meta'/100)=Meta1。
143006	$\wedge$ addt%T	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 %T。
145006	$\wedge$ addtMOD	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 MOD。
147006	$\wedge$ addtTRNC	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 TRNC。
149006	$\wedge$ addtRND	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 RND。
14B006	$\wedge$ addtCOMB	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 COMB。
14D006	$\wedge$ addtPERM	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 PERM。
14F006	$\wedge$ addtOR	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 OR。
151006	$\wedge$ addtAND	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 AND。
153006	$\wedge$ addtXOR	(Meta2 Meta1 $\rightarrow$ Meta') 对 meta 执行 XOR。
506006	$\wedge$ addtCONJ	(meta $\rightarrow$ meta') 取得复数 meta 的复共轭。
523006	$\wedge$ addtLN	(Meta $\rightarrow$ Meta') 对 meta 执行 LN。

地址	名称	说明
415006	$\wedge \sin 2 \tan$	(meta $\rightarrow$ meta ') $x \rightarrow \tan(x) / \sqrt{1+(\tan(x))^2}$
421006	$\wedge \tan 2 \exp$	(meta $\rightarrow$ meta ') $x \rightarrow (\exp(i2x)-1)/(i*(\exp(i2x)+1))$
423006	$\wedge \sin 2 \ln$	(meta $\rightarrow$ meta ') $x \rightarrow i * \ln(x+\sqrt{x^2-1})+\pi / 2$.
425006	$\wedge \cos 2 \ln$	(meta $\rightarrow$ meta ') $x \rightarrow \ln(x+\sqrt{x^2-1}) / i$
428006	$\wedge \sin / \cos$	(meta $\rightarrow$ meta ') $x \rightarrow \sin(x) / \cos(x)$
42B006	$\wedge \cos * \tan$	(meta $\rightarrow$ meta ') $x \rightarrow \cos(x) * \tan(x)$
42D006	$\wedge \sqrt{1-\sin^2}$	(meta $\rightarrow$ meta ') $x \rightarrow \sqrt{1-(\sin(x))^2}$.
42F006	$\wedge \sqrt{1-\cos^2}$	(meta $\rightarrow$ meta ') $x \rightarrow \sqrt{1-(\cos(x))^2}$.
432006	$\wedge \tan 2 \sin$	(meta $\rightarrow$ meta ') $x \rightarrow \arcsin(x / \sqrt{x^2+1})$
435006	$\wedge \sin 2 \atan$	(meta $\rightarrow$ meta ') $x \rightarrow \arctan(x / \sqrt{1-x^2})$
438006	$\wedge \pi / 2-\cos$	(meta $\rightarrow$ meta ') $x \rightarrow \pi / 2-\arcsin(x)$
439006	$\wedge \pi / 2-\sin$	(meta $\rightarrow$ meta ') $x \rightarrow \pi / 2-\arcsin(x)$
43B006	$\wedge \pi / 2-\cos$	(meta $\rightarrow$ meta ') $x \rightarrow \pi / 2-\arcsin(x)$
43E006	$\wedge \atan 2 \ln$	(meta $\rightarrow$ meta ') $x \rightarrow i / 2 * \ln((i+x) / (i-x))$
441006	$\wedge 2 * 1-\cos / \sin$	(meta $\rightarrow$ meta ') $x \rightarrow (1-\cos(2x)) / \sin(2x)$
地址	名称	说明
443006	$\wedge 2 * \sin / 1+\cos$	(meta $\rightarrow$ meta ') $x \rightarrow \sin(2x) / (1+\cos(2x))$
445006	$\wedge \sin 2 \exp$	(meta $\rightarrow$ meta ') $x \rightarrow (e^{(i * x)}-1) / e^{(i * x)} / (2i)$
447006	$\wedge \cos 2 \exp$	(meta $\rightarrow$ meta ') $x \rightarrow (e^{(i * x)}+1) / e^{(i * x)} / 2$
449006	$\wedge \sinh 2 \exp$	(meta $\rightarrow$ meta ') $x \rightarrow (e^x-1) / e^x / 2$
44B006	$\wedge \cosh 2 \exp$	(meta $\rightarrow$ meta ') $x \rightarrow (e^x+1) / e^x / 2$
44D006	$\wedge \tanh 2 \exp$	(meta $\rightarrow$ meta ') $x \rightarrow (e^{2x}-1) / (e^{2x}+1)$
44F006	$\wedge \sinh 2 \ln$	(meta $\rightarrow$ meta ') $x \rightarrow \ln(x+\sqrt{x^2+1})$
451006	$\wedge \cosh 2 \ln$	(meta $\rightarrow$ meta ') $x \rightarrow \ln(x+\sqrt{x^2-1})$
453006	$\wedge \atanh 2 \ln$	(meta $\rightarrow$ meta ') $x \rightarrow \ln((1+x) / (1-x)) / 2$
455006	$\wedge \text{xroot} 2 \exp \ln$	(meta1 meta2 $\rightarrow$ meta ') $x y \rightarrow \exp(\ln(y) / x)$
458006	$\wedge \exp 2 \sin \cos$	(meta $\rightarrow$ meta ') 以 EXP[RE]*[COS+i*SIN] 的形式返回 meta 的 EXP。

45.1.4 无穷与未定义值

地址	名称	说明
3A1006	$\wedge 1 \text{metaundef} \#$	(meta $\rightarrow$ meta #) 测试 meta 中是否存在 undef。# 是 undef 的位置。
3A0006	$\wedge 2 \text{metaundef} \#$	(meta2 meta1 $\rightarrow$ meta2 meta1 #) 测试 meta2 和 meta1 中是否存在 undef。# 是 undef 的位置。
3A2006	$\wedge \text{metaundef}$	($\rightarrow$ meta) 返回 undef meta。
3A4006	$\wedge 1 \text{metainf} \#$	(meta $\rightarrow$ meta #) 查找 meta 中无穷值的位置。长度大于 2 的 meta 被视为有限 meta。
地址	名称	说明
3A3006	$\wedge 2 \text{metainf} \#$	(meta2 meta1 $\rightarrow$ meta2 meta1 #) 查找无穷在 meta2 和 meta1 中的位置。长度大于 2 的元对象视为有限元对象。
3A5006	$\wedge \text{metainftype}$	(meta $\rightarrow$ #) 返回无穷的类型：正无穷返回 1，负无穷返回 2，无符号无穷返回 0。
3A6006	$\wedge \text{unsignedinf}$	($\rightarrow$ meta) 返回无符号无穷。
3A7006	$\wedge \text{plusinf}$	($\rightarrow$ meta) 返回正无穷。
3A8006	$\wedge \text{NDROPplusinf}$	(ob1...obn $\rightarrow$ meta) 用正无穷替换元对象。
3A9006	$\wedge \text{minusinf}$	($\rightarrow$ meta) 返回负无穷。
3AA006	$\wedge \text{NDROPminusinf}$	(ob1...obn $\rightarrow$ meta) 用负无穷替换元对象。

45.1.5 展开与化简

地址	名称	说明
3BB006	<code>^metasimp</code>	<code>(Meta → Meta)</code> 化简元对象。进行非递归有理化简。
118007	<code>^DISTRIB*</code>	<code>(meta → meta ' T)</code> <code>(meta → meta ' F)</code> 对 <code>*</code> 应用分配律。如果未进行分配，则返回 <code>FALSE</code> 。
3C2006	<code>^DISTRIB/</code>	<code>(meta → meta ' T)</code> <code>(meta → meta ' F)</code> 对 <code>/</code> 应用分配律。如果未进行分配，则返回 <code>FALSE</code> 。
304006	<code>^METASINEXPA</code>	<code>(Meta → Meta ')</code> 展开 <code>SIN</code> 。
305006	<code>^SINEXPA+</code>	<code>(Meta → Meta ')</code> 展开 <code>SIN(x+y)</code> 。
306006	<code>^SINEXPA-</code>	<code>(Meta → Meta ')</code> 展开 <code>SIN(x-y)</code> 。
地址	名称	说明
307006	<code>^SINEXPA*</code>	<code>(Meta → Meta ')</code> 展开 <code>SIN(x*y)</code> 。当 <code>x</code> 或 <code>y</code> 为整数时展开。
308006	<code>^SINEXPA*1</code>	<code>(Meta2 Meta1 → Meta ')</code> 展开 <code>SIN(x*y)</code> 。假定 <code>Meta1</code> 为整数。
30A006	<code>^METACOSEXPA</code>	<code>(Meta → Meta ')</code> 展开 <code>COS</code> 。
30B006	<code>^COSEXPA+</code>	<code>(Meta → Meta ')</code> 展开 <code>COS(x+y)</code> 。
30C006	<code>^COSEXPA-</code>	<code>(Meta → Meta ')</code> 展开 <code>COS(x-y)</code> 。
30D006	<code>^COSEXPA*</code>	<code>(Meta → Meta ')</code> 展开 <code>COS(x*y)</code> 。
30E006	<code>^COSEXPA*1</code>	<code>(meta2 meta1 → Meta ')</code> 展开 <code>COS(x*y)</code> 。 <code>meta1</code> 表示整数。
310006	<code>^METAEXPXPA</code>	<code>(Meta → Meta ')</code> 展开 <code>EXP</code> 。
311006	<code>^EXPXPA+</code>	<code>(Meta → Meta ')</code> 展开 <code>EXP(x+y)</code> 。
312006	<code>^EXPXPA-</code>	<code>(Meta → Meta ')</code> 展开 <code>EXP(x-y)</code> 。
313006	<code>^EXPXPA*</code>	<code>(Meta → Meta ')</code> 展开 <code>EXP(x*y)</code> 。
314006	<code>^EXPXPANEG</code>	<code>(Meta → Meta ')</code> 展开 <code>EXP(-x)</code> 。
315006	<code>^EXPXPA*1</code>	<code>(Meta2 meta1 → Meta ')</code> 展开 <code>EXP(x*y)</code> 。 <code>meta1</code> 表示整数。
317006	<code>^METALNEXPA</code>	<code>(Meta → Meta ')</code> 展开 <code>LN</code> 。
318006	<code>^LNEXPA*</code>	<code>(Meta → Meta ')</code> 展开 <code>LN(x*y)</code> 。
319006	<code>^LNEXPA/</code>	<code>(Meta → Meta ')</code> 展开 <code>LN(x/y)</code> 。
31A006	<code>^LNEXPA^</code>	<code>(Meta → Meta ')</code> 展开 <code>LN(x^y)</code> 。
地址	名称	说明
31E006	<code>^METATANEXPA</code>	<code>(meta → tan[meta])</code> 展开 <code>tan[meta]</code> 。

45.1.6 测试

地址	名称	说明
39A006	<code>^metafraction?</code>	<code>(Meta → Meta flag)</code> 测试元对象是否为整数之比。
3BC006	<code>^metapi?</code>	<code>(Meta → Meta#)</code> 测试元对象中是否存在 π 。 <code>#</code> 为 π 最后一次出现的位置；若不存在则为 <code>0</code> 。
3BD006	<code>^metaCOMPARE</code>	<code>(Meta2 Meta1 → Meta2 Meta1 #)</code> 比较两个元对象。 <code>#=0</code> : 未定义 <code>#=1</code> : <code>></code> <code>#=2</code> : <code><</code> <code>#=3</code> : <code>=</code> 假定一般情形，例如在实数模式中 <code>X^2 > 0</code> 。更谨慎的比较见下方 <code>STRICTmetaCOMPARE</code> 。
3BE006	<code>^STRICTmetaCOMPARE</code>	<code>(Meta2 Meta1 → Meta2 Meta1 #)</code> 比较两个元对象。 <code>#=0</code> : 未定义 <code>#=1</code> : <code>></code> <code>#=2</code> : <code><</code> <code>#=3</code> : <code>=</code> 与 <code>metaCOMPARE</code> 不同，它不假定一般情形。
3C3006	<code>^metareal?</code>	<code>(meta → meta flag)</code> 测试 <code>IM[meta]==0</code> 是否成立。

第 46 章

多项式

本章词条用于多项式计算。

46.1 参考

46.1.1 多项式计算

地址	名称	说明
118006	<code>^QAdd</code>	$(o1 \rightarrow o2+o1) $ 将两个多项式相加。
119006	<code>^RADDext</code>	$(o2\ o1 \rightarrow o2+o1) $ 内部 +。与 <code>^QAdd</code> 是同一个词条。
117006	<code>^SWAPRADD</code>	$(o2\ o1 \rightarrow o1+o2) $ 先执行 SWAP，再执行 <code>QAdd</code> 。
115006	<code>^QSub</code>	$(o2\ o1 \rightarrow o2-o1) $ 将两个多项式相减。
116006	<code>^RSUBext</code>	$(o2\ o1 \rightarrow o2-o1) $ 内部 -。与 <code>^QSub</code> 是同一个词条。
114006	<code>^SWAPRSUB</code>	$(o2\ o1 \rightarrow o1-o2) $ 先执行 SWAP，再执行 <code>QSub</code> 。
111006	<code>^QMul</code>	$(Q1\ Q2 \rightarrow Q) $ 对含扩展的多项式进行乘法。
112006	<code>^RMULText</code>	$(Q1\ Q2 \rightarrow Q) $ 对含扩展的多项式进行乘法。与 <code>^QMul</code> 是同一个词条。
110006	<code>^SWAPRMULT</code>	$(Q1\ Q2 \rightarrow Q) $ 先执行 SWAP，再执行 <code>^QMul</code> 。
11C006	<code>^QDiv</code>	$(o2\ o1 \rightarrow o2/o1) $ 内部 /。
地址	名称	说明
11B006	<code>^RDIVext</code>	$(o2\ o1 \rightarrow o2/o1) $ 内部 /。与 <code>^QDiv</code> 是同一个词条。
11A006	<code>^SWAPRDIV</code>	$(o2\ o1 \rightarrow o1/o2) $ 先执行 SWAP，再执行 <code>QDiv</code> 。
0D9006	<code>^QMod</code>	$(Q,\ Z \rightarrow Q \bmod Z)$
113006	<code>^RASOP</code>	$(n1/d1\ n2/d2 \rightarrow d1*d2\ n1*d2\ n2*d1) $ 供 <code>RADDext</code> 和 <code>RSUBext</code> 处理有理数输入时使用。
11F006	<code>^RP#</code>	$(o2\ \# \rightarrow o2^{\#}) $ 内部幂运算（不适用于矩阵）。
120006	<code>^MPext</code>	$(ob\ \# \text{ prg}^* \rightarrow ob^{\#}) $ 使用指定乘法程序的一般幂运算。
123006	<code>^RPext</code>	$(o2\ o1 \rightarrow o2^{o1}) $ 尝试将 <code>o1</code> 转换为整数以调用 <code>RP#</code> ；否则调用 <code>x^ext</code> 。
108006	<code>^DISTDIVext</code>	$(P\ Q \rightarrow \text{quo mod } T)$ $(P\ Q \rightarrow P\ Q\ F) $ 欧几里得除法。假定 <code>P</code> 和 <code>Q</code> 的系数均为整数。如果稀疏短除失败，则返回 <code>FALSE</code> 。
3E5006	<code>^PTAYLext</code>	$(P,\ r \rightarrow \text{symb}) $ 多项式的泰勒展开。
15B006	<code>^CARCOMPext</code>	$(Q1/Q2 \rightarrow Q1'/Q2') $ 从有理多项式中提取关于第一个变量的首项系数。
3EE006	<code>^QDivRem</code>	$(ob2\ ob1 \rightarrow \text{quo mod}) $ 对两个对象进行多项式欧几里得除法。对于列表多项式，分派给 <code>DIV2LISText</code> 。
3EF006	<code>^DIV2LISText</code>	$(Z0\ l1\ l2 \rightarrow \text{div mod}) $ 欧几里得除法； <code>l1</code> 和 <code>l2</code> 是列表多项式。先测试 <code>l1=l2</code> ，然后尝试快速除法；如果失败，则切换到 <code>SRPL</code> 除法。
3F8006	<code>^PDIV2ext</code>	$(A\ B \rightarrow Q\ R) $ 对一元多项式逐步进行欧几里得除法。
3F9006	<code>^PSetSign</code>	$(P1\ P2 \rightarrow \text{sign}[P2]*P1) $ 根据 <code>P2</code> 的首项系数设置 <code>P1</code> 的符号。
3C4006	<code>^ModExpa</code>	$(Zn\ \text{Fraction} \rightarrow \text{Fraction modulo } Zn) $ 对 <code>Fraction</code> 取模 <code>Zn</code> 。
地址	名称	说明
3C5006	<code>^ModAdd</code>	$(Q1\ Q2\ Zn \rightarrow Z) $ 模加法。 $Z = Q1+Q2 \pmod{Zn}$ 。
3C6006	<code>^ModSub</code>	$(Q1\ Q2\ Zn \rightarrow Z) $ 模减法。 $Z = Q1-Q2 \pmod{Zn}$ 。
3C7006	<code>^ModMul</code>	$(Q1\ Q2\ Zn \rightarrow Z) $ 模乘法。 $Z = Q1*Q2 \pmod{Zn}$ 。
3C8006	<code>^ModDiv</code>	$(Z1\ Z2\ Zn \rightarrow Z) $ 模除法。 $Z = Z1/Z2 \pmod{Zn}$ 。
3C9006	<code>^ModDiv2</code>	$(Q1\ Q2\ Zn \rightarrow \text{quo mod mod'}) $ 模除法。 $\text{mod}' = Q1 \bmod Q2 \bmod Zn$ 。如果 <code>Q1</code> 和 <code>Q2</code> 是整数，则 <code>Q1 mod Q2 mod Zn</code> 始终为 0。

地址	名称	说明
3CA006	<code>^ModInv</code>	$(Z \rightarrow Z_n \rightarrow Z')$ 模逆。 $Z' = \text{INV}(Z) \pmod{Z_n}$ 。如果 $\text{GCD}[Z, Z_n] \neq 1$ 或 $Z = 0$ ，则引发 <code>NONINTERR</code> （否则结果不可预测）。
3CB006	<code>^ModGcd</code>	$(Q1 \ Q2 \ Z_n \rightarrow Q')$ 模最大公因数。

46.1.2 因式分解

地址	名称	说明
08E006	<code>^BerlekampP</code>	$(P \rightarrow \#prime \rightarrow P \ F \ / \ P \ Lf \ \#prime \ T)$ 使用 Berlekamp 算法求一元多项式的模因子。
08F006	<code>^Berlekamp</code>	$(P \rightarrow P \ F \ / \ P \ Lf \ \#prime \ T)$ 使用 Berlekamp 算法求一元多项式的模因子，并通过前端更快地查找线性因子。输入多项式必须无平方因子，否则无法将其完全分解。由于内存限制，系数采用字节大小，并施加以下限制： prime <128 且 degree <256。如果不满足条件，则返回 <code>FALSE</code> 。BCD: prime ≤97。
地址	名称	说明
0A8006	<code>^ALG48FCTR?</code>	$(P \rightarrow [\ meta \ cst \ coeff \ TRUE \ \ P \ FALSE \])$ 对 Erable 格式的无平方因子多项式进行因式分解。
0A9006	<code>^MFactTriv</code>	$(P \rightarrow meta\text{-}factor \ P')$ 提取 P 的所有平凡幂因子。
0AA006	<code>^CheckPNoExt</code>	$(P \rightarrow P \ flag)$ 检查 P 是否不含任何 DOCOL （即扩展）。
0AB006	<code>^PPP</code>	$(P \rightarrow PP \ PC)$ 计算非常数 P 关于 X1 的本原多项式及其内容。如果 P 已修整，结果也会修整。
0AC006	<code>^PFactor</code>	$(P \rightarrow Lfk \ Z)$ 对 P 进行完全因式分解。结果会修整。
0AD006	<code>^PSqff</code>	$(P \rightarrow Lfk)$ 对取正的 P 进行无平方因子和平凡因式分解，包括整数内容。相同幂次的因子不一定合并或彼此相邻，但所有 Fi 均无平方因子。
0AE006	<code>^PHFctr</code>	$(P \rightarrow Lf)$ 对取正的多项式进行启发式因式分解。必须绑定 LAM FullFact? 。如果 LAM FullFact? 为 <code>TRUE</code> ，则进行完全因式分解；如果为 <code>FALSE</code> ，则只进行无平方因子和平凡因式分解。
0AF006	<code>^PHFctr1</code>	$(P \rightarrow Lf)$ 对本原多项式进行启发式因式分解。必须绑定 LAM FullFact? 。如果为 <code>TRUE</code> ，则进行完全因式分解；如果为 <code>FALSE</code> ，则只进行无平方因子和平凡因式分解。
0B0006	<code>^PHFctr0</code>	$(P \rightarrow Lf)$ 对非常数、无平方因子的一元本原多项式进行启发式因式分解。
0D8007	<code>^P2P#</code>	$(P \rightarrow P' \ \#)$ 提取多项式的平凡幂。 P 必须是有效多项式；如果是列表，则必须以非零系数开头。
地址	名称	说明
0B1006	<code>^DeCntMulti</code>	$(R \rightarrow L)$ 将带计数的列表转换为简单列表。 $R = \{ \{f1 \ \#k1\} \dots \{fn \ \#kn\} \}$ $L = \{ f1 \ f1 \ .. \ fn \ fn \}$ 。
0B2006	<code>^DoLS</code>	$(L \ S \ F \rightarrow L')$ 对 L 的每个元素应用程序 F(Li,S) 。
0B3006	<code>^PNFctr</code>	$(Z \rightarrow Lf)$ 将正整数作为多项式进行因式分解。 Z 为 1 时， Lf = {} 其他情况下， Lf = { {Z1 #k1} ... {Zn #kn} }。
0B4006	<code>^PSQFF</code>	$(P \rightarrow Lsqff)$ 计算本原 P 的无平方因子分解。如果 P 已修整，结果也会修整。
0B5006	<code>^LiftZAdic</code>	$(p \ z \ F \rightarrow L)$ 将 n-1 个 z-adic 因式分解提升为 n 个因式分解。
0B6006	<code>^LFCProd</code>	$(C \ L \rightarrow C \ P)$ 计算组合乘积。
0B7006	<code>^UFactor</code>	$(P \rightarrow Lf)$ 对无平方因子的一元本原多项式进行因式分解。
0B8006	<code>^UFactor1</code>	$(P \rightarrow Lf)$ 对次数 > 2、无平方因子的一元本原多项式进行因式分解。
0B9006	<code>^MonicLf</code>	$(Lfp \ p \rightarrow Lfp')$ 通过除以第 1 个因子的首项系数，将真模因式分解转换为首一因式分解。
0BA006	<code>^DemonicLf</code>	$(Lfp \ lc \ p \rightarrow Lfp')$ 通过将第 1 个因子乘以 lcoeff ，把首一模因式分解转换为真模因式分解。
地址	名称	说明
0BB006	<code>^LiftLinear</code>	$(\#root1 \ .. \ \#rootn \ \#n \rightarrow)$ 提升多项式的模根，以求一元多项式的线性因子。 Lflin = 已求得的真因子列表 Lfplin' = 剩余线性因子 P' = 剩余多项式 假定 UFactor 的 lambda 变量可用，并使用这些变量进行输入和输出。

地址	名称	说明
0BC006	<code>^LiftGeneral</code>	<code>(→)</code> 将模 p 的因式分解提升为模 p^k 的因式分解，其中 p^k 超过成功提取真因子所需的因子界。假定 <code>UFactor</code> 的 <code>lambda</code> 变量可用。
0BD006	<code>^UFactorDeg2</code>	<code>(P → Lf)</code> 对二次多项式进行因式分解。该多项式为无平方因子的一元本原多项式。
0BE006	<code>^CombineFac</code>	<code>(P Lfp p → Tf Tfp)</code> 将模因子组合成真因子。 P 是待分解的多项式， <code>Lfp</code> 是模因子列表， p 是模数。该词条返回已求得的真因子列表 <code>Tf</code> ，以及每个真因子所对应的模因子列表 <code>Tfp</code> 。
0BF006	<code>^CombProd</code>	<code>(lc Lfp p Cb → F)</code> 计算模组合。
0C0006	<code>^CombInit</code>	<code>(#r → Cb)</code> 将模组合列表初始化为 <code>{ 1 0 0 0 .. }</code> 。
0C1006	<code>^CombNext</code>	<code>(Cb → Cb' flag)</code> 取得下一个可能的模组合。假定 <code>Cb</code> 有效且位于 <code>tempob</code> 区域。
0C2006	<code>^RmCombNext</code>	<code>(Lf Cb → Lfrm Lf' Cb' flag)</code> 成功找到组合后，移除下一个可能的组合，并从因子列表中移除已用因子。
0C3006	<code>^PFactTriv</code>	<code>(P → P' Lf)</code> 提取 P 的所有平凡幂因子。
地址	名称	说明
0C4006	<code>^VarFactor</code>	<code>(P #var → P #n)</code> 计算给定变量的多少次幂是 P 的因子。
0C5006	<code>^PFactPowCnt</code>	<code>(P → P Lk flag)</code> 计算 P 中的平凡幂因子。只要任一幂次非零， <code>flag</code> 即为 <code>TRUE</code> 。
0C6006	<code>^PDivLk</code>	<code>(P Lk → P')</code> 将多项式除以其平凡幂。
282006	<code>^FEVIDENText</code>	<code>(P → meta-fact cst coeff)</code> 实数模式：在整数范围内完全因式分解。 复数模式：求 P 的所有一次因子。

46.1.3 一般多项式运算

地址	名称	说明
09B006	<code>^ONE{ }POLY</code>	<code>(ob → {ob} ob1 → Q)</code> 在构建多项式时替代 <code>ONE{ }N</code> 。
09C006	<code>^TWO{ }POLY</code>	<code>(ob1 ob2 → Q)</code> 在构建多项式时替代 <code>TWO{ }N</code> 。
09D006	<code>^THREE{ }POLY</code>	<code>(ob1 ob2 ob3 → Q)</code> 在构建多项式时替代 <code>THREE{ }N</code> 。
09E006	<code>^TWO::POLY</code>	<code>(ob1 ob2 → ::)</code> 在构建多项式时替代 <code>2Ob>Seco</code> 。
09F006	<code>^::POLY</code>	<code>(Meta → ::)</code> 在构建多项式时替代 <code>::N</code> 。与常规 <code>::N</code> 代码不同，这里会弹出二进制整数；进入通用 <code>polyxml</code> 代码的词条会强制执行这一点。
0A0006	<code>^{ }POLY</code>	<code>(Meta → Q)</code> 在构建多项式时替代 <code>{ }N</code> 。与常规 <code>{ }N</code> 代码不同，这里会弹出二进制整数。因此，可以像 <code>ONE{ }POLY</code> 和 <code>TWO{ }POLY</code> 一样，以固定大小进入此处代码。
0A7006	<code>^>POLY</code>	<code>(Meta → Q)</code> 构建多项式。
0A1006	<code>^>TPOLY</code>	<code>(P ob → P')</code> 在构建多项式时替代 <code>>TCOMP</code> 。
地址	名称	说明
0A2006	<code>^>HPOLY</code>	<code>(P ob → P')</code> 在构建多项式时替代 <code>>HCOMP</code> 。
0A3006	<code>^>TPOLYN</code>	<code>(P ob1 .. obn #n → P')</code> 用于构建多项式的改进版 <code>>TCOMP</code> 。
0A4006	<code>^>HPOLYN</code>	<code>(P ob1 .. obn #n → P')</code> 用于构建多项式的改进版 <code>>HCOMP</code> 。
0A5006	<code>^MKPOLY</code>	<code>(#n #k → P)</code> 构建第 n 个变量的 k 次幂多项式。
2AB006	<code>^MAKEPROFOND</code>	<code>(ob # → { { { ... { o } ... } } })</code> 将 <code>ob</code> 嵌入指定层数的列表中。
4F4006	<code>^TRIMext</code>	<code>(Q → Q')</code> 删除多项式中不必要的零。
4F5006	<code>^PTrim</code>	<code>(ob → ob')</code> 修整多项式。
0A6006	<code>^ONE>POLY</code>	<code>(Q → Q')</code> 增加变量深度。常数 (<code>Z</code> 、 <code>Irr</code> 、 <code>C</code>) 不作修改。
302006	<code>^TCHEBext</code>	<code>(zint → P)</code> 切比雪夫多项式。 <code>zint>0</code> 时为第一类， <code>zint<0</code> 时为第二类。
3DE006	<code>^LRDMext</code>	<code>(P # → [])</code> 左向重新定维。把 <code>0</code> 添加到多项式左侧，以得到长度为 <code>#+1</code> 的符号向量。
3DF006	<code>^RRDMext</code>	<code>({ } # → { })</code> 右向重新定维：与 <code>LRDM</code> 类似，但把 <code>0</code> 添加到右侧，并使用 <code>{ }</code> 。
3E0006	<code>^DEGREext</code>	<code>({ } → degre)</code> 列表多项式的次数。

地址	名称	说明
3E1006	^FHORNER	(P / d r → P[X]_div_[X-r] / d r P[r] / d) Horner 格式。
3E2006	^HORNext	(P r → P[X]_div_[X-r] r P[r]) Horner 格式。
3E4006	^MHORNext	(P r → P[X]_div_[X-r] r P[r]) 矩阵的 Horner 格式。
地址	名称	说明
3E6006	^LAGRANGEext	(M → symb) Lagrange 插值。矩阵格式为 [[x1 .. xn] [f(x1) .. f(xn)]] 返回满足 P(xi)=f(xi) 的多项式 P。
10F007	^RESULTANT	(P1 P2 → P) 两个多项式的结式。P 的深度比 P1 和 P2 的深度少一级。
110007	^RESULTANTLP	(res g h P1 P2 → +/-res g' h' P1' P2') 子结式算法的内循环。
111007	^RESPSHIFTQ	(P Q → P') P 与移位后 Q 的结式。gcd[Q(x-r),P(x)]!=1 等价于 r 是 P' 的根。P' 的深度与 P 和 Q 相同。
112007	^ADDONEVAR	(P → P') 在主变量下方添加一个变量。适用于多项式，不适用于分数。
0CF007	^SHRINKEVEN	(P → P') 在偶多项式中作变量替换 Y=X^2。
0D1007	^SHRINK2SYM	(N D → N' D') 利用对称性缩减两个多项式。
0D2007	^SHRINKSYM	(N → N') 利用对称性缩减一个多项式。N 的次数必须为偶数；如果为奇数，应将 N 除以 X+1。
0D3007	^SHRINK2ASYM	(N D → N' D') 利用反对称性缩减两个多项式。
0D4007	^SHRINKASYM	(N → N') 利用反对称性缩减一个多项式。N 的次数必须为偶数；如果为奇数，应将 N 除以 X+1。
103006	^PNMax	(P → Z) 取得 P 中范数最大的系数。
161006	^SWAPNDXF	(Qden Qnom → symb) 从有理多项式构建符号对象。
地址	名称	说明
162006	^NDXFext	(Qnom Qden → symb) 从有理多项式构建符号对象。
163006	^SWAPFXND	(symb ob → ob Qnom Qden) 将符号对象转换为有理多项式。
164006	^FXNDext	(symb → Qnom Qden) 将符号对象转换为有理多项式。
3D7006	^REGCDext	(a b → d u v au+bv=d)
3D8006	^EGCDext	(a b → d u v au+bv=d) 多项式的 Bézout 恒等式。
0EA006	^PEvalFast?	(Z Pn → Z Pn F / Pn[Z] T) 尝试使用快速寄存器算术在 X1=Z 处对 Pn 求值。出现以下任一情况时失败：Pn 不是一元多项式；Z 实际上是多项式；Z 的大小超出寄存器容量；在 Horner 求值期间发生任何溢出。
10E007	^FLAGRESULTANT	(symb1 symb2 → symb) 两个符号形式多项式的结式。

46.1.4 测试

地址	名称	说明
10B006	^Univar?	(P → P flag) 测试多项式是否为一元多项式。
10C006	^SUnivar?	(P → P flag) 测试多项式是否为一元多项式，且其系数是否受寄存器大小限制。
0CC007	^POLYPARITY	(poly → Z) 测试多项式（内部表示）为偶、奇还是均非。偶多项式时 Z=1，奇多项式时 Z=-1，既非偶也非奇时 Z=0。
0D6007	^POLYSYM	(P → Z) 测试多项式系数的对称性。对称时 z=1，反对称时 z=-1，其他情况为 0。
0D7007	^POLYASYM	(P → Z) 测试多项式系数的“反对称性”。对称时 Z=1，反对称时 Z=-1，其他情况为 0。

第 47 章

求根

本章收录与方程求根有关的词条。

47.1 参考

47.1.1 求根与数值求解器

地址	名称	说明
272006	<code>^MULMULText</code>	<code>({} % → {}')</code> 将因子列表中的各重数乘以 <code>coeff</code> 。
274006	<code>^METAMM2</code>	<code>(meta % → meta')</code> 将元对象的所有重数乘以%。
275006	<code>^COMPLISTText</code>	<code>({} → {}')</code>
276006	<code>^METACOMPRIM</code>	<code>(Meta → Meta')</code> 通过将相应重数相加，去除同一因子的重复项。
278006	<code>^METACOMP1</code>	<code>(f1...fk-1 mk-1 meta-res mk fk # → f1...fk-1 mk-1 meta-res)</code>
279006	<code>^ADDLISTText</code>	<code>({} %n ob → {}')</code> 将重数为% <code>n</code> 的 <code>ob</code> 添加到列表。检查 <code>ob</code> 是否已在 <code>{}</code> 中。
27A006	<code>^DIVISext</code>	<code>(ob → {divisors})</code> 返回 <code>ob</code> 的约数列表。
27B006	<code>^FACT1ext</code>	<code>(symb-poly → Lvar Q {})</code> <code>{}</code> 是 <code>symb</code> 关于当前变量的根/重数列表。
27C006	<code>^FACTOext</code>	<code>(symb → Lvar Q {})</code> <code>{}</code> 是 <code>symb</code> 的因子/重数列表。
27D006	<code>^ZFACTO</code>	<code>(C → {} C Lfact)</code>
地址	名称	说明
27E006	<code>^SOLVext</code>	<code>(symb → {})</code> 一元多项式的数值求解器。列表中包含不计重数的根。
27F006	<code>^FRND</code>	<code>(ob → ob')</code> 对% <code>%</code> 、 <code>C%</code> 或由这两种类型组成的列表进行浮点舍入。 <code>SOLVext</code> 使用它重建因子。
280006	<code>^BICARREE?</code>	<code>(P #5 → meta cst_coeff T)</code> <code>(P #5 → P #5 F)</code> <code>(P # → P # F)</code> 检查 <code>P</code> 是否为双二次四阶方程。如果是，则返回因子元对象及乘法系数。
281006	<code>^REALBICAR</code>	<code>(f1 #1 coef → meta rest T)</code>
113007	<code>^IROOTS</code>	<code>(P → list)</code> 求多项式的整数根。
283006	<code>^EVIDENText</code>	<code>(P → meta cst_coeff)</code> 返回多项式 <code>P</code> 的根。调用数值求解器。
284006	<code>^EVIDSOLV</code>	<code>(P → meta cst_coeff)</code> 返回一次、二次及某些其他多项式的根。如果精确求解失败，则调用数值求解器。
285006	<code>^DEG2ext</code>	<code>(P → {})</code> 返回二次多项式的根。
286006	<code>^METADEG2</code>	<code>(P → P meta)</code> 返回二次多项式的根。 <code>P</code> 必须为一次或二次多项式。
287006	<code>^METADEG1</code>	<code>(P → P meta)</code> 返回一次多项式的根。 <code>P</code> 必须为一次多项式。
288006	<code>^DEG1</code>	<code>(f → r)</code> 一次因子的根。 <code>f</code> 的深度比 <code>r</code> 多一级。
289006	<code>^FDEG2ext</code>	<code>(P → meta-fact cst_coef)</code> 返回二次多项式的因子及相应的乘法系数。测试是否为一次多项式。
地址	名称	说明
28B006	<code>^RACTOFACext</code>	<code>(r → n d)</code> 将根转换为因子。因子为 <code>n/d</code> ，深度比 <code>r</code> 多一级。
28C006	<code>^FACTORACext</code>	<code>(f → r cst_coef)</code> 通过求解一次因子，将因子转换为根。 <code>f</code> 和 <code>cst_coef</code> 的深度均比 <code>r</code> 多一级。
28D006	<code>^RFACText</code>	<code>(ob # → {} intob meta)</code> <code>{}</code> 是变量列表。根据 <code>#</code> ， <code>Meta</code> 由分子 (<code>N</code>)、分母 (<code>D</code>) 或二者 (<code>N/D</code>) 的根或因子组成。 <code>ZERO</code> : <code>N/D</code> 的根； <code>ONE</code> : <code>N</code> 的根； <code>TWO</code> : <code>D</code> 的根，并调用数值求解器； <code>THREE</code> : <code>D</code> 的根，不调用数值求解器； <code>FOUR</code> : <code>N/D</code> 的因子； <code>FIVE</code> : <code>N</code> 的因子； <code>SIX</code> : <code>D</code> 的因子，并调用数值求解器； <code>SEVEN</code> : <code>D</code> 的因子，不调用数值求解器。
28E006	<code>^RFACT2ext</code>	<code>(ob {} # → {} intob meta)</code> 与 <code>RFACText</code> 类似，但变量列表由参数给出。
28F006	<code>^RFACTSTEP3</code>	<code>(ob → meta-fact)</code> 关于主变量进行部分无平方因子分解，并提取平凡因子。第 3 步: <code>ob → meta-fact</code> 。
290006	<code>^RFACTSTEP5</code>	<code>(%m on → add-to-meta-res)</code> 对无平方因子多项式进行因式分解。

地址	名称	说明
291006	<code>^METASOLV</code>	<code>(pn cst_coeff → meta cst_coeff) </code> 进行非整数因式分解（平方根扩展和数值分解）。重数存放在 LAM 5 中。
293006	<code>^METASOLV2</code>	<code>(cst_coeff p → fr1 %m [fr2 %m] # cst_coeff) </code> 返回一次和二次多项式的根/因子。
294006	<code>^METASOLV4</code>	<code>(cst1 f1 ... fk #k cst2 → fr1 %m ... frn %m #2k cst_coeff) </code> 返回因子，或在需要时转换为根。 <code>#k=1,2</code> 或 <code>4</code> ; <code>fk</code> 为一次或二次多项式。
地址	名称	说明
295006	<code>^ADDMULTIPL</code>	<code>(meta cst_coeff → meta' cst_coeff) </code> 向 meta 添加重数。重数位于 LAM 5 中。
296006	<code>^FACTOOBJext</code>	<code>({ fact_mult } flag prg* prg^ → ob) </code> 使用 prg* 进行乘法、使用 prg^ 取重数次幂，根据因子列表（ <code>flag=TRUE</code> ）或根列表（ <code>flag=FALSE</code> ）重建对象。
093006	<code>^ALG48MSOLV</code>	<code>(Lp → Lidnt Lsol) </code> 计算 Groebner 基的多变量解。 LAM3 必须绑定到 Lvar ， LAM4 必须绑定到 Lidnt 。
094006	<code>^GMSOLV</code>	<code>(Lp → meta-sol) </code> 计算 Groebner 基的多变量解。 LAM1 必须绑定到变量数。一个解是列表 <code>{ o1 ... on }</code> ，其中 <code>#n=LAM1</code> ；嵌套在 <code>k-1</code> 层列表中的 <code>ok</code> 是第 <code>k</code> 个变量的值， <code>ok</code> 可以是 <code>undef</code> 。
095006	<code>^GBASIS</code>	<code>(Lp → G) </code> 计算 Groebner 基。若无解，则 <code>G = { 1 }</code> ；若恒为真，则 <code>G = { 0 }</code> 。
096006	<code>^GSOLVE</code>	<code>(Lp → Lg) </code> 计算因式分解后的 Groebner 基。 <code>Lg = { Lg1 Lg2 .. Lgn }</code> <code>Lgi</code> = 独立解（可能如此） 若无解，则 <code>Lg = { }</code> 若恒为真，则 <code>Lg = { { 0 } }</code> 。
097006	<code>^GFACTOR</code>	<code>(Lp fctr? → Lg) </code> 计算 Groebner 基或因式分解后的 Groebner 基。不会删除冗余的基。
099006	<code>^REDUCE</code>	<code>(p G → q) </code> 相对于给定的基约简多项式。
地址	名称	说明
09A006	<code>^FASTREDUCE</code>	<code>(r P → q T / r P F) </code> 适用于短系数多项式的 REDUCE 汇编版本。若约简过程中发生溢出，则返回 FALSE 。假定 r 是真正的多项式（非常数）； G 非空且不含零（已经修剪）。
37D006	<code>^ROOTM2ROOT</code>	<code>({ } / V → V') </code> 将根/重数列表转换为根向量。
0F2007	<code>^PASCAL_NEXTLINE</code>	<code>({ } → { }') </code> 求帕斯卡三角形的下一行。
0F3007	<code>^DELTAPSOLVE</code>	<code>(Q → P) </code> 求解 $P(x+1) \cdot P(x) = Q(x)$ 。内部多项式函数。

第 48 章 微积分运算

本章各词条涉及微积分的多个方面，例如极限、导数、部分分式展开和拉普拉斯变换。

48.1 参考

48.1.1 极限与级数展开

地址	名称	说明
46F006	<code>^SYMTAYLOR</code>	<code>(symb id %/z → symb)</code> 关于给定变量、按给定阶数在点 0 附近作泰勒级数展开（麦克劳林级数）。
471006	<code>^TRUNCDL</code>	<code>(DL-l reste-l → truncated_DL)</code> 截断级数展开。
472006	<code>^LIMSERIES!</code>	<code>(expression X=a X % zint →)</code> <code>a lim DL-l rest-l num-l/deno-l equiv-l lvar #</code> 级数展开。 当 $X=a-h$ 或 $X=-1/h$ 时， $\# = 1$ 。
477006	<code>^LIMIT!</code>	<code>(symb → DL-l reste-l num-l/deno-l equiv.-l</code> <code>lim. lvar flag)</code> <code>lim. = { symf direction }</code>
478006	<code>^LIMSTEP!</code>	<code>(symb → { DL-l reste-l num-l/deno-l</code> <code>equiv.-l } flag)</code>
47C006	<code>^LIMLIM!</code>	<code>(# lvar equiv-l → lvar lim)</code>
47F006	<code>^LIMCMPL!</code>	<code>(reste-1-l reste-2-l → reste-1)</code>
480006	<code>^LIMEQUFR!</code>	<code>(n/d # → n/d-l equiv %)</code>
481006	<code>^LIMEQU!</code>	<code>({ } # → { } / { }-equiv-l { }-equiv-l { # # # })</code>
地址	名称	说明
483006	<code>^LIM+-!</code>	<code>(DL1...DLn #n op → DL flag)</code> <code>DL = { DL-l reste-</code> <code>num-l/deno-l equiv-l }</code> 。
48C006	<code>^LIMDIVPC!</code>	<code>(#ordre num-l deno-l → num-l deno-l)</code>
48E006	<code>^LIMPROFEND!</code>	<code>(num deno #prof → num deno)</code>
490006	<code>^LIM%#!</code>	<code>(num-l deno-l { %...% } → num-l' deno-l'</code> <code>#prof { %...% })</code>
49E006	<code>^LIM#VARX!</code>	<code>(lvar lvar → #varx)</code>
4A1006	<code>^HORNEXP!</code>	<code>(lim lvar X-l reste-l → lvar DL reste-l)</code>
4B6006	<code>^VARCOMP!</code>	<code>(var1 var2 → flag)</code>
4BA006	<code>^VARCOMP32!</code>	<code>(var → 0:)</code>
4BD006	<code>^LIMVALOBJ!</code>	<code>(ob lvar → symb)</code>
4BE006	<code>^LIMVAL!</code>	<code>(ob → coeff val)</code>
4BF006	<code>^EQUIV!</code>	<code>({ } lequiv → equiv ordre)</code>
4C0006	<code>^LVARXNX2!</code>	<code>(ob → ob lvarx lvarnx)</code>
4C2006	<code>^FindCurVar</code>	<code>(symb → symb)</code> 必要时设置新的当前变量。
4C3006	<code>^LIMVAR!</code>	<code>(symb → symb lvar)</code>
15C006	<code>^RISCH13</code>	<code>({ } / { }' → { }')</code> 假定 $\{ \}$ 的长度为 1，用其中这个元素除以 $\{ \}$ 的所有元素。RISCHext 和 SERIES 使用此词条，以得到形式更美观的级数输出。

48.1.2 导数

地址	名称	说明
3DC006	<code>^PDer</code>	<code>({ } → der)</code>
1A1006	<code>^DERIVext</code>	<code>(ob id → ob')</code> <code>(ob sym → ob')</code> <code>(ob v → v')</code> 计算对象的导数。对于列表参数，计算关于列表中各变量的梯度。如果变量是符号对象，则使用其中的第一个变量。请注意，梯度是向量，因此结果以列表形式返回。
地址	名称	说明
1A3006	<code>^DERIVIDNT</code>	<code>(ob id → ob')</code> 关于某个标识符求导的主入口。
1A4006	<code>^DERIVIDNT1</code>	<code>(ob → ob')</code> 关于 LAM1 中存储的标识符求导的主入口。

地址	名称	说明
1A5006	^DERIV	(symb → symb ') symb 关于 LAM1 中存储变量的导数。
1A6006	^METADERIV	(Meta → Meta ') 元对象的导数。
1BD006	^METADER&NEG	(Meta → Meta ') 对 Meta 求导并取负。
1A9006	^METADER+	(Meta&+ → Meta ') Meta 加法的导数。
1AA006	^METADER-	(Meta&- → Meta ') Meta 减法的导数。
1AB006	^METADER*	(Meta&* → Meta ') Meta 乘法的导数。
1AC006	^METADER/	(Meta&/ → Meta ') Meta 除法的导数。
1AD006	^METADER^	(Meta&^ → Meta ') Meta 幂的导数。
1AE006	^METADERFCN	(Meta → Meta ') Meta 函数的导数。
1AF006	^METADERDER	(symb_id ; sym_fcn_ ; xDER #3 → Meta ') Meta 函数导数的导数。
1B0006	^METADERI4	(Meta → Meta ') Meta 定积分的导数。
1B1006	^METADERI3	(Meta → Meta ') Meta 不定积分的导数。
1B2006	^METADERIFTE	(Meta → Meta ') Meta IFTE 的导数。
1B4006	^METADEREXP	(Meta → Meta ') Meta EXP 的导数。
1B5006	^METADERLN	(Meta → Meta ') Meta LN 的导数。

地址	名称	说明
1B6006	^METADERLNPI	(Meta → Meta ') Meta LNPI 的导数。
1B7006	^METADERLOG	(Meta → Meta ') Meta LOG 的导数。
1B8006	^METADERALOG	(Meta → Meta ') Meta ALOG 的导数。
1B9006	^METADERABS	(Meta → Meta ') Meta ABS 的导数。
1BA006	^METADERINV	(Meta → Meta ') Meta INV 的导数。
1BB006	^METADERNEG	(Meta → Meta ') Meta NEG 的导数。
1BC006	^METADERSQRT	(Meta → Meta ') Meta SQRT 的导数。
1BE006	^METADERSQ	(Meta → Meta ') Meta SQ 的导数。
1BF006	^METADERSIN	(Meta → Meta ') Meta SIN 的导数。
1C0006	^METADERCOS	(Meta → Meta ') Meta COS 的导数。
1C1006	^METADERTAN	(Meta → Meta ') Meta TAN 的导数。
1C2006	^METADERSINH	(Meta → Meta ') Meta SINH 的导数。
1C3006	^METADERCOSH	(Meta → Meta ') Meta COSH 的导数。
1C4006	^METADERTANH	(Meta → Meta ') Meta TANH 的导数。
1C5006	^METADERASIN	(Meta → Meta ') Meta ASIN 的导数。
1C6006	^METADERACOS	(Meta → Meta ') Meta ACOS 的导数。
1C7006	^METADERATAN	(Meta → Meta ') Meta ATAN 的导数。
1C8006	^METADERASH	(Meta → Meta ') Meta ASINH 的导数。
1C9006	^METADERACH	(Meta → Meta ') Meta ACOSH 的导数。

地址	名称	说明
1CA006	^METADERATH	(Meta → Meta ') Meta ATANH 的导数。
1B3006	^DERARG	(meta-symb → arg1 ... argk der1 ... derk #k op) 求各参数的导数。
1CB006	^pshder*	(Meta1 Meta2 → Meta2&Meta1'&*) Meta 求导实用词条。
1CC006	^SQRTINVpshd*	(Meta1 Meta2 → Meta2&SQRT&INV&Meta1'&*) Meta 求导实用词条。

原书第 391 页

48.1.3 积分

地址	名称	说明
07F007	<code>^ODE_INT</code>	<code>(symb idnt → symb) </code> 积分并加上常数。
2C5006	<code>^IBP</code>	<code>(u' * v u → u * v - u * v') </code> 内部的分部积分。如果 <code>u</code> 是常数, 则返回 <code>INTVX(u'*v)+u</code> 。如果堆栈第 2 层是列表, 其形式必须为 <code>{ olduv u'*v }</code> , 随后 <code>olduv</code> 会加到堆栈第 2 层的 <code>u*v</code> 上。这样可以在代数模式中多次使用 <code>IBP</code> , 例如: <code>IBP (ASIN(X)^2,X)</code> <code>IBP (ANS(1),sqrt(1-X^2))</code> <code>IBP (ANS(1),C)</code> , 最后一步使用含常数 <code>cst C</code> 的积分。
2D0006	<code>^PREVALext</code>	<code>(symb inf sup x → symb x=sup - symb x=inf) </code> 在两个界限之间对原函数求值。不检查 <code>symb</code> 在该区间内的不连续点。
2D1006	<code>^WARNSING</code>	<code>(symb inf sup vx → symb inf sup vx) </code> 向用户发出奇点警告。
2D2006	<code>^INText</code>	<code>(symb x → int[\$,x, symb, xt]) </code> 返回未求值的积分。
地址	名称	说明
2D3006	<code>^INT3</code>	<code>(f(x) x y → F(y) where F'=f) </code> 不定积分。对欠定形式没有限制。
3DD006	<code>^INTEGRext</code>	<code>({} → prim)</code>

48.1.4 部分分式

地址	名称	说明
3D2006	<code>^PARTFRAC</code>	<code>(o → symb) </code> 对 <code>o</code> 关于当前变量作部分分式展开。
3D3006	<code>^INPARTFRAC</code>	<code>(o list → symb) </code> 对 <code>o</code> 作部分分式展开。 <code>lvar</code> 必须绑定到 <code>LAM2</code> ; 如果 <code>o</code> 采用外部格式, 则 <code>list</code> 为 <code>=lvar</code> ; 如果 <code>o</code> 仍采用内部格式, 则 <code>list</code> 为 <code>NULL{}</code> 。

48.1.5 微分方程

地址	名称	说明
07E007	<code>^DESOLVE</code>	<code>(list symb1 → list_sols)</code> <code>(symb symb1 → list_sols) </code> 求解常微分方程。对于某些 ODE, 返回的堆栈第 2 层不是 <code>symb1</code> 。
081007	<code>^LDECSOLV</code>	<code>(2nd_member char_eq → solution) </code> 常系数线性微分方程。
082007	<code>^LDEGENE</code>	<code>(eq. carac → sol generale)</code>
083007	<code>^LDEPART</code>	<code>(2nd membre, eq carac → eq. carac, sol part)</code>
084007	<code>^LDSSOLVext</code>	<code>(V M → v') </code> <code>M</code> 是方程组的矩阵, <code>V</code> 是右端项向量。
085007	<code>^ODETYPESTO</code>	<code>(type →) </code> 将 ODE 类型存储在变量 <code>ODETYPE</code> 中。
地址	名称	说明
086007	<code>^ODE_SEPAR</code>	<code>(symb → symb symb-y symb-x T)</code> <code>(symb → symb F) </code> 尝试将 <code>symb</code> 分离为 <code>y</code> 的函数与 <code>x</code> 的函数之积。

原书第 392 页

48.1.6 拉普拉斯变换

地址	名称	说明
087007	<code>^LAPext</code>	<code>(symb → symb') </code> 对 <code>polynomial*exp/sin/cos</code> 作拉普拉斯变换。未知变换返回 <code>LAP()</code> 。
088007	<code>^ILAPext</code>	<code>(symb → symb') </code> 对有理分式作拉普拉斯逆变换。积分部分使用狄拉克 δ 函数。
08B007	<code>^ILAPEXP</code>	<code>(ck rk → ck*exp[rk*x])</code>

第 49 章

求和

本章给出与求和有关的主要词条，以及这些词条使用的一些子例程。

49.1 参考

地址	名称	说明
0F9007	<code>^SUM</code>	<code>(sym idnt → sym) </code> 内部 SUM。可以指定变量。
0FB007	<code>^SUMVX</code>	<code>(sym → sym) </code> 内部 SUMVX。始终关于当前变量运算。
0FD007	<code>^RATSUM</code>	<code>(sym → sym) </code> 离散有理求和。
0FE007	<code>^FTAYL</code>	<code>(f shift → f') </code> 有理分式的泰勒平移。
0FF007	<code>^CSTFRACTION?</code>	<code>(ob → ob flag) </code> 有理分式的泰勒平移。如果 <code>ob</code> 是常数分式，则返回 TRUE。
104007	<code>^HYPERGEO</code>	<code>(symb → symb) </code> 检验并执行超几何求和。
100007	<code>^NONRATSUM</code>	<code>(z/symb → symb) </code> 离散求和（超几何情形）。
103007	<code>^meta_cst?</code>	<code>(meta → meta flag) </code> 检验 <code>meta</code> 关于当前变量是否为常数。
108007	<code>^ZEILBERGER</code>	<code>(f(n,k) n k d → C T)</code> <code>(f(n,k) n k d → F) </code> Zeilberger 算法。* 尚未实现*。
地址	名称	说明
109007	<code>^SYMPSI</code>	<code>(sym → Psi(x)) </code> 双伽马函数。
10B007	<code>^SYMPSIN</code>	<code>(sym int → Psi(x,n)) </code> 双伽马函数。
11C007	<code>^%%PSI</code>	<code>(%%x → %%) </code> 双伽马函数。
10D007	<code>^IBERNOULLI</code>	<code>(#/zint → Q) </code> 伯努利数。
0D9007	<code>^NDEvalN/D</code>	<code>(num deno n d → num' deno') </code> 在列表分数处对列表多项式求值。
0DA007	<code>^PEvalN/D</code>	<code>(P n d → num d #) </code> 在列表分数处对列表多项式求值。
3C1006	<code>^vgerxssSYMSUM</code>	<code>(Meta2 Meta1 → meta) </code> 对两个 <code>zint</code> 进行检验的符号求和。 <code>lam'sumvar</code> 绑定到 <code>'id/lam'</code> ， <code>lam'sumexpr</code> 绑定到 <code>'expr'</code> 。

第 50 章 模运算

本章各词条涉及模算术及其他模运算。

50.1 参考

50.1.1 模运算

地址	名称	说明
252006	<code>^FLAGFACTORMOD</code>	<code>(symb $\rightarrow$ symb)</code> 模意义下的 FACTOR 。
253006	<code>^MFACTORMOD</code>	<code>(M $\rightarrow$ M')</code> 矩阵在模意义下的 FACTOR 。
256006	<code>^LIFCext</code>	<code>({contfrac} $\rightarrow$ fraction)</code> 将连分数转换为分数。
0E1006	<code>^PEvalMod</code>	<code>(Q Z Zn $\rightarrow$ Q')</code> 计算多项式模 Zn 的值。
0E2006	<code>^QAddMod</code>	<code>(Q1 Q2 Zn $\rightarrow$ Q')</code> 多项式模 Zn 加法。
0E3006	<code>^QSubMod</code>	<code>(Q1 Q2 Zn $\rightarrow$ Q')</code> 多项式模 Zn 减法。
0E4006	<code>^QMulMod</code>	<code>(Q1 Q2 Zn $\rightarrow$ Q')</code> 多项式模 Zn 乘法。
0E5006	<code>^QDivMod</code>	<code>(Q1 Q2 Zn $\rightarrow$ Qquo Qrem)</code> 多项式模 Zn 除法。在普通除法中，余式的系数可能很快增长到数十位，因此应尽可能对系数进行归一化。
地址	名称	说明
0E6006	<code>^QInvMod</code>	<code>(Q Zn $\rightarrow$ Q')</code> 多项式模 Zn 求逆。
0E7006	<code>^QGcdMod</code>	<code>(Q1 Q2 Zn $\rightarrow$ Q')</code> 一元多项式模 Zn 的多项式 GCD 。结果已化为首一形式。
4C5006	<code>^ISOL1</code>	<code>(symb id $\rightarrow$ id symb')</code>
4C6006	<code>^ISOLALL</code>	<code>(symb id $\rightarrow$ id {})</code> 内部 SOLVE 。
4C7006	<code>^ISOL2ext</code>	<code>(symb id $\rightarrow$ symb')</code> <code>(symb id $\rightarrow$ {})</code> 如果设置了 isolflag ，则与 ISOL1 相同；否则返回找到的所有解的列表。
4C8006	<code>^BEZOUTMSOLV</code>	<code>(Lpoly Lidnt $\rightarrow$ Lidnt sols)</code> 如果 Lpoly 中没有扩张，则调用 ALG48 GSOLVE ；否则通过 Bézout “高斯”消元法求解。后一种情况下，若方程组看似欠定，则截断 Lidnt 。此时方程组必须为恰定方程组，且各多项式必须互素。
4C9006	<code>^ROOT{ }N</code>	<code>(meta of roots $\rightarrow$ list of roots)</code> 丢弃带标记的根。
4CA006	<code>^MHORNER</code>	<code>(poly-l {r1...rk} # $\rightarrow$ P[r1...rk])</code> 顶层调用。 Poly-l 可以是矩阵。
4CB006	<code>^MHORNER1</code>	<code>(P { r } $\rightarrow$ P[...r...])</code>
4CC006	<code>^SQFFext</code>	<code>(Q $\rightarrow$ { F1 mult1 .. Fn multn })</code>
4CD006	<code>^MSQFF</code>	<code>(Q $\rightarrow$ F1 mult1 .. Fn multn #2n)</code> 对象的完全无平方因子分解。结果以元对象形式给出。
4CE006	<code>^%1TWO</code>	<code>(ob $\rightarrow$ ob %1 #2)</code> 对未知 (?) 对象作无平方因子分解。参见 MSQFF 。
4CF006	<code>^MZSQFF</code>	<code>(Z $\rightarrow$ Z1 mult1 .. Zn multn #2n)</code> 整数的完全因式分解。
地址	名称	说明
4D0006	<code>^MZSQFF1</code>	<code>(Meta curfac %n newfac T $\rightarrow$ Meta curfac %n+1)</code> <code>(Meta curfac %n newfac F $\rightarrow$ Meta' newfac %1)</code> 将整数因子加入因子列表。如果该因子与上一个因子相同，则只增加重数。
4D2006	<code>^MLISTSQFF</code>	<code>(P $\rightarrow$ Meta)</code> 对多项式作完全无平方因子分解，并对所有系数的 GCD 进行递归调用。
4D3006	<code>^METASQFFext</code>	<code>(P-list $\rightarrow$ S1 %1 .. Se-1 %e-1 %e ee Te Re)</code> 无平方因子分解。
4DE006	<code>^LIDNText</code>	<code>(ob $\rightarrow$ {})</code> 取得 ob 中出现的所有 id 的列表。
4DF006	<code>^LVARXNText</code>	<code>(symb $\rightarrow$ symb x lvarnx lvarx)</code> 查找 symb 中依赖于当前变量和另一个变量的变量。由于存在 sqrt ，此处无法使用 LVAR 。
4E0006	<code>^ISPOLYNOMIAL?</code>	<code>(ob $\rightarrow$ flag)</code> 如果 symb 关于当前变量是多项式，则返回 TRUE 。

地址	名称	说明
4E1006	^2POLYNOMIAL?	(symb1 symb2 → symb1 symb2 flag) 如果 symb1 和 symb2 关于当前变量都是多项式，则返回 TRUE。
4E2006	^VXINDEP?	(symb → symb flag) 如果 symb 与当前变量无关，则返回 TRUE。
4E4006	^RLVARExt	(ob → {}) 递归搜索所有变量。
4E5006	^LLVARExt	(o → #depth o lvar)
4E6006	^VXLVARExt	(symb → symb lvar)
4E7006	^LVARExt	(ob → ob {}) 变量列表。平方根包含在有理运算符列表中。
地址	名称	说明
4E8006	^VX>LVARExt	(ob → ob {}) 与 LVARExt 类似，但使用 >HCOMP 添加当前变量。平方根包含在有理运算符列表中。
4E9006	^VX>	({} → {}') 如果 VX 在列表中，则将其移到列表开头；否则不执行任何操作。
4EA006	^VX!	({} → {}) 如果 VX 在列表中，则将其移到开头；否则将 VX 添加到列表开头。
4EC006	^LIDNTLVAR	(symb lidnt → symb lidnt lvar) lvar 是 symb 中的变量列表，但 lidnt 的元素会移到 lvar 开头。
4ED006	^LISTOPRAC	(→ {}) 返回有理运算符列表，并在列表末尾附加 sqrt 。
4EE006	^LISTOPext	(→ {}) 不含平方根的基本“有理”运算符列表。
4EF006	^LISTOPSQRT	(→ {}) 含平方根的基本“有理”运算符列表。
4F0006	^LVARExt	(ob listop → lidnt) (Meta listop → lidnt) 使用给定的基本“有理”运算符列表，确定 ob （或 Meta ）中的变量列表。
4F2006	^DEPTHext	(ob → #) 返回 ob 中嵌套列表的最大层数。
4F3006	^DEPTHOBJext	(objet # → depth)
4F6006	^TRIMOBJext	(ob → ob ') 修剪对象。
地址	名称	说明
4F7006	^NEWTRIMext	(Q → Q) 递归检验 Q 是否为仅含一个常数元素的列表。此方法比 TRIMOBJext 快得多，并且足以处理在运行过程中进行修剪的程序输出。
4F8006	^>POLYTRIM	(meta → {}) 等效于 {}POLY TRIMOBJext。
4F9006	^ELMGext	(ob → ob') 修剪小数值（小于 epsilon ）。
0E9006	^IsV>V?	(v1 v2 → flag) 如果 v1 按字典序位于 v2 之后，则返回 TRUE。
0EB006	^PZadic	(Q Z → Q')
104006	^LISTMAXext	(P → P Z T depth) (P → P ? F #0) gcdheu 的第 1 步：如果不能应用 gcdheu （例如 P 含有 irrquads ），则返回 FALSE；否则返回 TRUE。 Z 是 P 中所有整数的最大值；如果 P 中有复数，则为 2*max 。
0EC006	^GCDHEUext	(A B → a b c pr[pgcd] A'/G' B'/G' flag) 启发式 GCD。

第 51 章

符号表

符号表是一个列表，用于描述表达式在某个参数的不同区间内的符号。该列表含有奇数个元素，其形式如下：

```
{ value1 sign1.2 value2 sign2.3 ...signN-1.N valueN }
```

这些值是参数的关键值，通常包括 $-\infty$ 、 $+\infty$ 以及表达式的奇点或零点所在位置。这些值必须按顺序排列，可以是数值或符号表达式。各符号表示表达式在相邻值之间区间内的符号。符号为‘-’、‘+’和‘?’（当符号未知时）。要计算表达式关于当前变量的符号表，请使用词条 `SIGNE1ext`。例如，表达式 ‘ $X^2 - 1$ ’ 的符号表为：

```
{ -∞ ' + ' -1 ' - ' 1 ' + ' +∞ }
```

下面列出与符号表有关的词条。

51.1 参考

地址	名称	说明
237006	<code>^SIGNE</code>	(<code>symb</code> $\rightarrow$ <code>sign</code>) 计算表达式关于当前变量的符号表。 <code>UserRPL</code> 命令 <code>SIGNTAB</code> 的内部版本。
0DC007	<code>^SIGNE1ext</code>	(<code>expr</code> $\rightarrow$ <code>sign</code>) 多项式或有理表达式的符号表。
0DE007	<code>^SIGNUNDEF</code>	($\rightarrow$ <code>sign</code>) 返回未定义的符号表。
0DF007	<code>^SIGNPLUS</code>	($\rightarrow$ <code>sign</code>) 返回恒为正的符号表。
地址	名称	说明
0E0007	<code>^SIGNMOINS</code>	($\rightarrow$ <code>sign</code>) 返回恒为负的符号表。
0E1007	<code>^SIGNALN</code>	(<code>sign</code> $\rightarrow$ <code>sign</code>) 返回符号表的 <code>ln</code> 结果。
0E2007	<code>^SIGNEEXP</code>	(<code>sign</code> $\rightarrow$ <code>sign'</code>) 返回符号表的 <code>exp</code> 结果。
0E3007	<code>^SIGNESIN</code>	(<code>sign</code> $\rightarrow$ <code>sign'</code>) 返回符号表的 <code>sin</code> 结果。
0E4007	<code>^SIGNECOS</code>	(<code>sign</code> $\rightarrow$ <code>sign'</code>) 返回符号表的 <code>cos</code> 结果。
0E5007	<code>^SIGNETAN</code>	(<code>sign</code> $\rightarrow$ <code>sign'</code>) 返回符号表的 <code>tan</code> 结果。
0E6007	<code>^SIGNEATAN</code>	(<code>sign</code> $\rightarrow$ <code>sign'</code>) 返回符号表的 <code>atan</code> 结果。
0E7007	<code>^SIGNESQRT</code>	(<code>sign</code> $\rightarrow$ <code>sign'</code>) 返回符号表的 <code>sqrt</code> 结果。
0E8007	<code>^SUBSIGNE</code>	(<code>sign min max</code> $\rightarrow$ <code>sign'</code>) 截断符号表。
0E9007	<code>^SIGNERIGHT</code>	(<code>sign ob</code> $\rightarrow$ <code>sign'</code>) 将 <code>ob</code> 放在符号表末尾。
0EA007	<code>^SIGNELEFT</code>	(<code>sign ob</code> $\rightarrow$ <code>sign'</code>) 将 <code>ob</code> 放在符号表开头。
0EB007	<code>^>SIGNE</code>	(<code>sign</code> $\rightarrow$ <code>sign'</code>) 在符号表前添加 { <code>-infinity ?</code> }。
0EC007	<code>^SIGNE></code>	(<code>sign</code> $\rightarrow$ <code>sign'</code>) 在符号表后添加 { <code>? +infinity</code> }。
0ED007	<code>^SIGNMULText</code>	(<code>sign1 sign2</code> $\rightarrow$ <code>sign'</code>) 将两个符号表相乘。
0DB007	<code>^POSITIFext</code>	(<code>ob</code> $\rightarrow$ <code>ob flag</code>) 尝试判断 <code>ob</code> 是否为正。在内部表示中，结果取决于 <code>increaseflag</code> ：如果 <code>increaseflag</code> 已清除，则 <code>x-1</code> 为正；否则为负，因为假定 <code>x</code> 趋向 <code>+infinity</code> 或 0。
0EE007	<code>^ZSIGNECK</code>	(<code>ob</code> $\rightarrow$ <code>ob flag</code>) 返回表达式的符号；若无法确定符号则报错。
地址	名称	说明
0F0007	<code>^ZSIGNE</code>	(<code>ob</code> $\rightarrow$ <code>zint</code>) 返回表达式的符号。 <code>zint=1</code> 表示 <code>+</code> ， <code>-1</code> 表示 <code>-</code> ， <code>0</code> 表示 <code>undef</code> 。表达式不必是多项式/有理式。
0F1007	<code>^zsigne</code>	(<code>meta</code> $\rightarrow$ <code>zint</code>) 返回 <code>meta</code> 符号对象的符号。 <code>zint=1</code> 表示 <code>+</code> ， <code>-1</code> 表示 <code>-</code> ， <code>0</code> 表示 <code>undef</code> 。表达式不必是多项式/有理式。
07D007	<code>^CHECKSING</code>	(<code>symb inf sup vx</code> $\rightarrow$ <code>symb inf sup vx flag</code>) 检查 <code>expr</code> 中是否存在奇点。

第 52 章 错误

所有 CAS 错误消息的编号都以 DE 开头，完整列表见附录 E。
词条 ^ERABLEERROR 和 ^GETERABLEMSG 会在消息编号上加 DE00，因此只需指定消息编号的最后两位。也可以将第 22 章所述的错误命令用于 CAS 错误，但要使用完整的错误编号。

52.1 参考

地址	名称	说明
57E006	^ERABLEERROR	(# →) 调用 CAS 错误。
57D006	^GETERABLEMSG	(# → \$) 获取 erable 消息表中的字符串。
090006	^ErrInfRes	Error 305h 生成 “Infinite Result” 错误。
091006	^ErrUndefRes	Error 304h 生成 “Undefined Result” 错误。
092006	^ErrBadDim	Error 501h 生成 “Invalid Dimension” 错误。
57F006	^CANTFACTOR	Error DE1Ch 生成 “Unable to find factor” 错误。
580006	^TRANSCERROR	Error DE20h 生成 “Not reducible to a rational expression” 错误。
581006	^NONUNARYERR	Error DE21h 生成 “Non unary operator” 错误。
582006	^INTERNALERR	Error DE26h 生成 “CAS internal error” 错误。
地址	名称	说明
583006	^INVALIDOP	Error DE28h 生成 “Operator not implemented (SERIES)” 错误。
584006	^ISOLERR	Error DE2Ah 生成 “No solution found” 错误。
585006	^NONINTERR	Error DE2Ch 生成 “No solution in ring” 错误。
586006	^INTVARERR	Error DE32h 生成 “No name in expression” 错误。
587006	^Z>#ERR	Error DE35h 生成 “Integer too large” 错误。
0EF007	^SIGNEERROR	Error DE36h 生成 “Unable to find sign” 错误。
588006	^Z<0ERR	Error DE46h 生成 “Negative integer” 错误。
589006	^VXINDEPERR	Error DE47h 生成 “Parameter is cur. var. dependent” 错误。
58A006	^NONPOLYSYST	Error DE49h 生成 “Non polynomial system” 错误。
58B006	^COMPLEXERR	Error DE4Dh 生成 “Complex number not allowed” 错误。
58C006	^VALMUSTBE0	Error DE4Eh 生成 “Polyn. valuation must be 0” 错误。
58D006	^SWITCHNOTALLOWED	Error DE4Fh 生成 “Mode switch not allowed here” 错误。
119007	^NONALGERR	Error DE50h 生成 “Non algebraic in expression” 错误。
58E006	^ERR\$EVALext	(seco → action)
58F006	^Sys1IT	(ob →) 若显示标志已设置，则执行该对象。

第 53 章 CAS配置

本章词条提供了配置 CAS 运算的方法。这里可以完成的配置，与用户通过标志或 **MODES** 窗口能够完成的配置相同。

53.1 参考

地址	名称	说明
08F007	^CFGDISPLAY	(→) 显示 CAS 的当前配置。
090007	^NEWVX	(→) 提示用户输入新的当前变量。
091007	^NEWMODULO	(→) 提示用户输入新的模数。
092007	^SWITCHON	(#flag →) 询问用户是否允许通过切换系统标志 #flag 来开启某个模式；若用户不愿切换，则报错。
093007	^SWITCHOFF	(#flag →) 询问用户是否允许通过切换系统标志 #flag 来关闭某个模式；若用户不愿切换，则报错。
094007	^FLAGNAME	(# → # \$) 查找标志的名称。
1DC007	(^PUSHFLAGS)	(→) 用户 PUSH 命令的内部版本：将当前标志设置和路径存入 CASDIR/ENVSTK 变量。
地址	名称	说明
1DD007	(^POPFLAGS)	(→) 用户 POP 命令的内部版本：从 CASDIR/ENVSTK 变量中弹出最后压入的标志设置和路径。
095007	^COMPLEXON	(→) 开启复数模式。根据系统标志 120 的状态，可能会先询问用户。
096007	^COMPLEXOFF	(→) 关闭复数模式。根据系统标志 120 的状态，可能会先询问用户。
097007	^EXACTON	(→) 开启精确模式。根据系统标志 120 的状态，可能会先询问用户。
098007	^EXACTOFF	(→) 关闭精确模式。根据系统标志 120 的状态，可能会先询问用户。
099007	^COMPLEXMODE	(→) 设置复数模式，并刷新配置显示。
09A007	^SETCOMPLEX	(→) 设置复数模式。
09B007	^COMPLEX?	(→ flag) 测试复数模式。
09C007	^REALMODE	(→) 设置实数模式，并刷新配置显示。
09D007	^CLRCOMPLEX	(→) 设置实数模式。
09E007	^EXACTMODE	(→) 设置精确模式，并刷新配置显示。
09F007	^SETEXACT	(→) 设置精确模式和 gcd 模式。
0A0007	^NUMMODE	(→) 设置数值模式，并刷新配置显示。
0A1007	^CLREXACT	(→) 清除精确模式。
0A2007	^EXACT?	(→ flag) 测试精确模式。
地址	名称	说明
0A3007	^STEPBYSTEP	(→) 设置逐步运算标志，并刷新显示。
0A4007	^NOSTEPBYSTEP	(→) 清除逐步运算标志，并刷新显示。
0A5007	^VERBOSEMODE	(→) 设置详细模式，并刷新配置显示。
0A6007	^SILENTMODE	(→) 设置静默模式，并刷新配置显示。
0A7007	^RECURMODE	(→) 设置递归模式，并刷新配置显示。
0A8007	^NONRECMODE	(→) 设置非递归模式，并刷新配置显示。
0A9007	^PLUSAT0	(→) 设置正值模式，并刷新配置显示。
0AA007	^SETPLUSAT0	(→) 设置正值模式。
0AB007	^PLUSATINFTY	(→) 设置正无穷模式，并刷新配置显示。
0AC007	^CLRPLUSAT0	(→) 设置正无穷模式。
0AD007	^SPARSEDATA	(→) 设置完整数据模式，并刷新配置显示。
0AE007	^FULLDATA	(→) 设置稀疏模式，并刷新配置显示。
0AF007	^RIGORMODE	(→) 设置严格模式，并刷新配置显示。

地址	名称	说明
0B0007	^SLOPPYMODE	(→) 设置宽松模式，并刷新配置显示。
地址	名称	说明
0B1007	^SLOPPY?	(→ flag) 测试宽松模式。
1D2006	^SAVECASFLAGS	(→) 保存 CAS 标志和当前变量。
1D4006	^RESTORECASFLAGS	(→) 恢复 CAS 标志和当前变量。
1D5006	^CASFLAGEVAL	(→) 在标志保护下执行运行流中的下一个对象。
0C2007	^RCLMODULE	(→ z) 从主目录提取 MODULE。
0C3007	^RCLPERIOD	(→ sym) 从主目录提取 PERIOD。
0C4007	^RCLVX	(→ id) 从主目录提取 VX。
0C5007	^STOVX	(ob →) 将对象存入 VX。
0C6007	^STOMODULE	(ob →) 将对象存入 MODULE。
0C7007	^RCLEPS	(→ %) 从主目录提取 EPS。
0C8007	^ISIDREAL?	(id → id id T) (id → id F) 测试 id 是否在 REALASSUME 列表中。
0C9007	^ADDTOREAL	(id →) 将 idnt 添加到实变量列表。
0CA007	^RESETCASCFG	(→) 重置 CAS 配置。
1D0006	^VERNUMext	(→ %version) CAS 版本号。

第 54 章 CAS菜单

本章词条用于返回由 CAS 命令组成的内置菜单，或执行其他与菜单有关的操作。有关菜单的一般信息，请参阅第 37 章。

54.1 参考

地址	名称	说明
1D1006	<code>^MENUXYext</code>	<code>(#2 #1 → {})</code> 生成由给定编号之间的 Erable 命令组成的列表。
08D007	<code>^MENUext</code>	<code>(\$6...\$1 →)</code> 如果 CAS 静默标志未设置，则将六个字符串显示为菜单键；否则不执行任何操作。
0B2007	<code>^MENUCHOOSE?</code>	<code>(→ prg flag)</code> 返回最合适的 CHOOSE 命令。
0B3007	<code>^MENUCHOOSE</code>	<code>({} →)</code> 让用户进行选择。如果标志 -117 已设置，则只安装菜单；否则显示 CHOOSE 选择框。
0B4007	<code>^MENUGENE1</code>	<code>(→ {})</code> CAS 菜单。
0B5007	<code>^MENUBASE1</code>	<code>(→ {})</code> 基本代数菜单。
0B6007	<code>^MENUMPLX1</code>	<code>(→ {})</code> 复数运算菜单。
0B7007	<code>^MENUTRIG1</code>	<code>(→ {})</code> 三角函数运算菜单。
0B8007	<code>^MENUMAT1</code>	<code>(→ {})</code> 矩阵运算菜单。
地址	名称	说明
0B9007	<code>^MENUARIT1</code>	<code>(→ {})</code> 算术运算菜单。
0BA007	<code>^MENSOLVE1</code>	<code>(→ {})</code> 求解器菜单。
0BB007	<code>^MENUEXPLN1</code>	<code>(→ {})</code> 指数与对数运算菜单。
0BC007	<code>^MENUDIFF1</code>	<code>(→)</code> 微分运算菜单。

第 55 章

User RPL命令的内部版本

本章列出的词条与 User RPL 命令最为接近。

55.1 参考

地址	名称	说明
218006	<code>^ISPRIME</code>	<code>(z/% → %0/%1) </code> 内部 <code>ISPRIME</code> 。
1D6006	<code>^FLAGEXPAND</code>	<code>(symb → symb') </code> 内部 <code>xEXPAND</code> 。展开符号表达式。
1D8006	<code>^FLAGFACTOR</code>	<code>(symb → symb')</code> <code>(z → symb) </code> 内部 <code>xFACTOR</code> 。对符号表达式或数值进行因式分解。
1D9006	<code>^FLAGLISTEXEC</code>	<code>(symb { } → symb') </code> 内部 <code>xSUBST</code> ，用于堆栈第 1 层为数组或矩阵的情况。
1DA006	<code>^FLAGSYMBEXEC</code>	<code>(symb symb' → symb'') </code> 内部 <code>xSUBST</code> ，用于堆栈第 1 层为符号对象的情况。
1DB006	<code>^FLAGIDNTEXEC</code>	<code>(symb id → symb') </code> 内部 <code>xSUBST</code> ，用于堆栈第 1 层为 <code>id</code> 或 <code>lam</code> 的情况。
1DC006	<code>^FLAGINTVX</code>	<code>(symb → symb') </code> 内部 <code>xINTVX</code> 。
地址	名称	说明
1DD006	<code>^DERVX</code>	<code>(symb → symb') </code> 内部 <code>xDERVX</code> 。
1DE006	<code>^SOLVEFLOAT</code>	<code>(% → { }) </code> 内部 <code>xSOLVEVX</code> ，用于浮点数。
1DF006	<code>^SYMLIMIT</code>	<code>(symb symb' → symb'') </code> 内部 <code>xLIMIT</code> ，用于标量。
1E0006	<code>^FLAGMATRIXLIMIT</code>	<code>([] symb → []') </code> 内部 <code>xLIMIT</code> ，用于矩阵。
1E1006	<code>^TAYLOR0</code>	<code>(symb → symb') </code> 内部 <code>xTAYLOR0</code> 。
1E2006	<code>^FLAGSERIES</code>	<code>(symb id z → { } symb') </code> 内部 <code>xSERIES</code> 。
1E4006	<code>^PLOTADD</code>	<code>(symb →) </code> 内部 <code>xPLOTADD</code> 。
1E5006	<code>^FLAGIBP</code>	<code>(symb1 symb2 → symb3 symb4) </code> 内部 <code>xIBP</code> 。
1E6006	<code>^FLAGPREVAL</code>	<code>(symb1 symb2 symb3 → symb4) </code> 内部 <code>xPREVAL</code> 。在 <code>symb2</code> 和 <code>symb3</code> 两点对 <code>symb1</code> 求值，并求其差。
1E7006	<code>^MATRIXRISCH</code>	<code>([] id → symb') </code> 内部 <code>xRISCH</code> ，用于矩阵参数。
1E8006	<code>^FLAGRISCH</code>	<code>(symb id → symb') </code> 内部 <code>xRISCH</code> ，用于非矩阵参数。
1E9006	<code>^FLAGDERIV</code>	<code>(symb id → symb') </code> 内部 <code>xDERIV</code> 。
1EA006	<code>^FLAGLAP</code>	<code>(symb → symb') </code> 内部 <code>xLAP</code> 。
1EB006	<code>^FLAGILAP</code>	<code>(symb → symb') </code> 内部 <code>xILAP</code> 。
1EC006	<code>^FLAGDESOLVE</code>	<code>(symb symb' → symb'') </code> 内部 <code>xDESOLVE</code> 。
1ED006	<code>^FLAGLDSSOLV</code>	<code>(symb1 symb2 → symb3) </code> 内部 <code>xLDEC</code> 。
1EF006	<code>^FLAGTEXPAND</code>	<code>(symb → symb') </code> 内部 <code>xTEXPAND</code> 。
1F0006	<code>^FLAGLIN</code>	<code>(symb → symb') </code> 内部 <code>xLIN</code> 。
地址	名称	说明
1F1006	<code>^FLAGTSIMP</code>	<code>(symb → symb') </code> 内部 <code>xTSIMP</code> 。
1F2006	<code>^FLAGLNCOLLECT</code>	<code>(symb → symb') </code> 内部 <code>xLNCOLLECT</code> 。
1F3006	<code>^FLAGEXPLN</code>	<code>(symb → symb') </code> 内部 <code>xEXPLN</code> 。
1F4006	<code>^FLAGSINCOS</code>	<code>(symb → symb') </code> 内部 <code>xSINCOS</code> 。
1F5006	<code>^FLAGTLIN</code>	<code>(symb → symb') </code> 内部 <code>xTLIN</code> 。
1F6006	<code>^FLAGTCOLLECT</code>	<code>(symb → symb') </code> 内部 <code>TCOLLECT</code> 。
1F7006	<code>^FLAGTRIG</code>	<code>(symb → symb') </code> 内部 <code>xTRIG</code> 。
1F8006	<code>^FLAGTRIGCOS</code>	<code>(symb → symb') </code> 内部 <code>xTRIGCOS</code> 。
1F9006	<code>^FLAGTRIGSIN</code>	<code>(symb → symb') </code> 内部 <code>xTRIGSIN</code> 。
1FA006	<code>^FLAGTRIGTAN</code>	<code>(symb → symb') </code> 内部 <code>xTRIGTAN</code> 。

地址	名称	说明
1FB006	$\wedge$ FLAGTAN2SC	(symb $\rightarrow$ symb') 内部 xTAN2SC。
1FC006	$\wedge$ FLAGHALFTAN	(symb $\rightarrow$ symb') 内部 xHALFTAN。
1FD006	$\wedge$ FLAGTAN2SC2	(symb $\rightarrow$ symb') 内部 xTAN2SC2。
1FE006	$\wedge$ FLAGATAN2S	(symb $\rightarrow$ symb') 内部 xATAN2S。
1FF006	$\wedge$ FLAGASIN2T	(symb $\rightarrow$ symb') 内部 xASIN2T。
200006	$\wedge$ FLAGASIN2C	(symb $\rightarrow$ symb') 内部 xASIN2C。
201006	$\wedge$ FLAGACOS2S	(symb $\rightarrow$ symb') 内部 xACOS2S。
206006	$\wedge$ STEPIDIV2	(z1 z2 $\rightarrow$ z3 z4) 内部 xIDIV2。
207006	$\wedge$ FLAGDIV2	(symb1 symb2 $\rightarrow$ symb3 symb4) 内部 xDIV2。
地址	名称	说明
208006	$\wedge$ FLAGGCD	(symb1 symb2 $\rightarrow$ symb3) 内部 xGCD，用于有两个符号参数的情况。
209006	$\wedge$ PEGCD	(symb1 symb2 $\rightarrow$ symb3 symb4 symb5) 内部 xEGCD，用于多项式。
20B006	$\wedge$ ABCUV	(symb1 symb2 symb3 $\rightarrow$ symb4 symb5) 内部多项式 xABCUV。
20C006	$\wedge$ IABCUV	(z1 z2 z3 $\rightarrow$ z4 z5) 内部整数 xIABCUV。
20D006	$\wedge$ FLAGLGCD	({} $\rightarrow$ {} symb) 内部 xLGCD。
20E006	$\wedge$ FLAGLCM	(symb1 symb2 $\rightarrow$ symb3) 内部 xLCM。
20F006	$\wedge$ FLAGSIMP2	(symb1 symb2 $\rightarrow$ symb3 symb4) 内部 xSIMP2。
210006	$\wedge$ FLAGPARTFRAC	(symb $\rightarrow$ symb') 内部 xPARTFRAC。
211006	$\wedge$ FLAGPROPFRAC	(symb $\rightarrow$ symb') 内部 xPROPFRAC。
212006	$\wedge$ FLAGPTAYL	(P(X) r $\rightarrow$ P(X+r)) 内部 xPTAYL。
213006	$\wedge$ FLAGHORNER	(symb1 symb2 $\rightarrow$ symb3 symb4 symb5) 内部 xHORNER。
214006	$\wedge$ EULER	(z $\rightarrow$ z') 内部 xEULER。
216006	$\wedge$ FLAGCHINREM	(A1 A2 $\rightarrow$ A3) 内部 xCHINREM。
217006	$\wedge$ ICHINREM	(A1 A2 $\rightarrow$ A3) 内部 xICHINREM。
219006	$\wedge$ SOLVE1EQ	(symb id $\rightarrow$ {}) 内部 xSOLVE，用于单个方程。
21A006	$\wedge$ SOLVEMANYEQ	([] []' $\rightarrow$ {}) 内部 xSOLVE，用于方程数组。
21B006	$\wedge$ ZEROS1EQ	(symb id $\rightarrow$ {}) 内部 xZEROS，用于单个方程。
地址	名称	说明
21C006	$\wedge$ ZEROSMANYEQ	([] []' $\rightarrow$ {}) 内部 xZEROS，用于方程数组。
21D006	$\wedge$ FCOEF	([] $\rightarrow$ symb) 内部 xFCOEF。
21E006	$\wedge$ FROOTS	(symb $\rightarrow$ []) 内部 xFROOTS。
21F006	$\wedge$ FACTORS	(symb $\rightarrow$ {}) 内部 xFACTORS。
220006	$\wedge$ DIVIS	(symb $\rightarrow$ {}) 内部 xDIVIS。
223006	$\wedge$ rref	(M $\rightarrow$ A M') 内部 xrref。
229006	$\wedge$ MADNOCK	(M $\rightarrow$ symb1 [] ' [] ' symb3) 内部 xMAD。
22A006	$\wedge$ SYSTEM	([] []' $\rightarrow$ [] {} {}) 内部 xLINSOLVE。
22B006	$\wedge$ VANDERMONDE	({} $\rightarrow$ M) 内部 xVANDERMONDE。
22C006	$\wedge$ HILBERTNOCK	(z $\rightarrow$ M) 内部 xHILBERT。
22E006	$\wedge$ CURL	([exprs] [vars] $\rightarrow$ []) 内部 xCURL。
22F006	$\wedge$ DIVERGENCE	([exprs] [vars] $\rightarrow$ symb) 内部 xDIV。
230006	$\wedge$ LAPLACIAN	([expr] [vars] $\rightarrow$ symb) 内部 xLAPL。
231006	$\wedge$ HESSIAN	(symb A $\rightarrow$ M A' A' ') 内部 xHESS。
232006	$\wedge$ HERMITE	(z $\rightarrow$ symb) 内部 xHERMITE。
233006	$\wedge$ TCHEBNOCK	(%degree $\rightarrow$ symb) 内部 xTCHEBYCHEFF。

地址	名称	说明
234006	$\wedge$ LEGENDRE	($z \rightarrow \text{symb}$) 内部 xLEGENDRE。
235006	$\wedge$ LAGRANGE	($A \rightarrow \text{symb}$) 内部 xLAGRANGE。
236006	$\wedge$ FOURIER	($\text{symb } z \rightarrow C\%$) 内部 xFOURIER。
地址	名称	说明
238006	$\wedge$ TABVAR	($\text{symb} \rightarrow \text{symb} \{ \} \}$ grob) 内部 xTABVAR。
239006	$\wedge$ FLAGDIVPC	($\text{symb1 } \text{symb2 } z \rightarrow \text{symb3}$) 内部 xDIVPC。
23A006	$\wedge$ FLAGTRUNC	($\text{symb1 } \text{symb2} \rightarrow \text{symb3}$) 内部 xTRUNC。
23B006	$\wedge$ FLAGSEVAL	($\text{symb} \rightarrow \text{symb}'$) 内部 xSEVAL。
23C006	$\wedge$ XNUM	($\text{symb} \rightarrow \text{symb}'$) 内部 xXNUM。
23D006	$\wedge$ REORDER	($\text{symb id} \rightarrow \text{symb}'$) 内部 xREORDER。
23E006	$\wedge$ USERLVAR	($\text{symb} \rightarrow \text{symb} []$) 内部 xLVAR。
23F006	$\wedge$ USERLIDNT	($\text{symb} \rightarrow []$) 内部 xLNAME。
241006	$\wedge$ ADDTMOD	($\text{symb1 } \text{symb2} \rightarrow \text{symb3}$) 内部 xADDTMOD，用于标量。
242006	$\wedge$ MADDTMOD	($M \text{ } M' \rightarrow M''$) 内部 xADDTMOD，用于矩阵。
243006	$\wedge$ SUBTMOD	($\text{symb1 } \text{symb2} \rightarrow \text{symb3}$) 内部 xSUBTMOD，用于标量。
244006	$\wedge$ MSUBTMOD	($M \text{ } M' \rightarrow M''$) 内部 xSUBTMOD，用于矩阵。
245006	$\wedge$ MULTMOD	($\text{symb1 } \text{symb2} \rightarrow \text{symb3}$) 内部 xMULTMOD。

第 56 章 杂项

本章列出了无法归入前面任何章节的词条。

56.1 参考

56.1.1 详细模式显示例程

地址	名称	说明
579006	<code>^Verbose1</code>	<code>(\$ →) </code> 若详细模式开启，则在第 1 行显示消息。
57A006	<code>^Verbose2</code>	<code>(\$ →) </code> 若详细模式开启，则在第 2 行显示消息。
57B006	<code>^Verbose3</code>	<code>(\$ →) </code> 若详细模式开启，则在第 3 行显示消息。
57C006	<code>^VerboseN</code>	<code>(\$ # →) </code> 若详细模式开启，则在指定行显示消息。

56.1.2 求值

地址	名称	说明
257006	<code>^EvalNoCKx*</code>	<code>(ob ob' → ob')</code>
258006	<code>^EvalNoCKx+</code>	<code>(ob ob' → ob')</code>
259006	<code>^EvalNoCKx-</code>	<code>(ob ob' → ob')</code>
25A006	<code>^EvalNoCKx/</code>	<code>(ob ob' → ob')</code>
25B006	<code>^EvalNoCKx^</code>	<code>(ob ob' → ob')</code>
25C006	<code>^EvalNoCKxCHS</code>	<code>(ob → ob')</code>
25D006	<code>^EvalNoCKxINV</code>	<code>(ob → ob')</code>
25E006	<code>^EvalNoCKxMOD</code>	<code>(ob ob' → ob')</code>
地址	名称	说明
25F006	<code>^EvalNoCKxPERM</code>	<code>(ob ob' → ob')</code>
260006	<code>^EvalNoCKxCOMB</code>	<code>(ob ob' → ob')</code>
261006	<code>^EvalNoCKxOR</code>	<code>(ob ob' → ob')</code>
262006	<code>^EvalNoCKxAND</code>	<code>(ob ob' → ob')</code>
263006	<code>^EvalNoCKxXOR</code>	<code>(ob ob' → ob')</code>
264006	<code>^EvalNoCKxXROOT</code>	<code>(ob ob' → ob')</code>
265006	<code>^TABVALext</code>	<code>(fnct x { } → { }) </code> 数值表。

56.1.3 转换

地址	名称	说明
266006	<code>^TOLISText</code>	<code>(o1..on #n → Lvar Q1..Qn) </code> 将符号对象的元对象转换为内部形式。
267006	<code>^FROMLISText</code>	<code>(Lvar Meta L → L') </code> 将 Meta 对象的元素转换为用户格式。 Meta 不包含表示元素数量的 #n 。 L 是 Meta 各元素深度的列表。例如，要转换一个多项式、一个向量和一个矩阵： <code>Lvar = { X }</code> <code>Meta = { Z1 Z3 }</code> <code>{ Z0 Z1 }</code> <code>{ { Z1 { Z1 Z0 } } }</code> <code>L = { #0 #1 #2 }</code> <code>L' = { 'X+2' {01} {{1x}} }</code>

56.1.4 Qpi

地址	名称	说明
074007	<code>^QPI</code>	<code>(ob → ob')</code> 内部 <code>xxQ</code> 。
073007	<code>^QpiZ</code>	<code>(ob → symb)</code> 调用 <code>^Qpi%</code> ，并将所得（实数）整数转换为 <code>zint</code> 。
地址	名称	说明
075007	<code>^QpiSym</code>	<code>(symb → symb')</code> 内部 <code>xxQ</code> ，用于符号对象。
076007	<code>^QpiArray</code>	<code>([] → []')</code> 内部 <code>xxQ</code> ，用于数组。转换数组中的每个元素。
077007	<code>^QpiList</code>	<code>({} → { }')</code> 内部 <code>xxQ</code> ，用于列表。转换列表中的每个元素。
078007	<code>^Qpi</code>	<code>(%/C% → symb)</code> 内部 <code>xxQ</code> ，用于实数和复数。
079007	<code>^Qpi%</code>	<code>(% → symb)</code> 用于实数的 <code>xxQ</code> ，但不将数值转换为 <code>zint</code> 。
07A007	<code>^GetRoot</code>	<code>(%' → %' %')</code> 尝试寻找参数的一个完全平方数因子。该算法只尝试小于 1024^2-1 的数，并假定 <code>%</code> 是整数。返回结果满足 <code>%(%)^2*%"</code> 。对于不含完全平方因子的数， <code>%=1</code> 且 <code>%"=%</code> 。
07B007	<code>^Approx</code>	<code>(% → %' %')</code> 用分数近似表示实数。返回分子 <code>%'</code> 和分母 <code>%'</code> 。近似精度由当前显示格式决定。

56.1.5无穷

地址	名称	说明
2E2006	<code>^INFINIext</code>	<code>(→ '∞')</code>
2E3006	<code>^MINUSINFext</code>	<code>(→ '-∞')</code>
2E4006	<code>^PLUSINFext</code>	<code>(→ '+∞')</code>
2E5006	<code>^?ext</code>	<code>'?'</code> 将未定义符号压入堆栈。
2E6006	<code>^POSINFext</code>	<code>(symb → symb #)</code> 如果符号对象包含 <code>'∞'</code> ，则返回 <code>#1</code> 。
2E1006	<code>^TESTINFINI</code>	<code>(ob → ob flag)</code> 测试对象是否包含无穷。
地址	名称	说明
2E7006	<code>^POSUNDEFext</code>	<code>(symb → symb #)</code> 如果符号对象包含未定义符号 <code>'?'</code> ，则返回 <code>#1</code> 。

56.1.6 内置常量

地址	名称	说明
2EA006	<code>^pi</code>	<code>(→ 'π')</code>
2EB006	<code>^metapi</code>	<code>(→ π #1)</code>
2F1006	<code>^meta-pi</code>	<code>(→ π xNEG #2)</code>
2E8006	<code>^pisur2</code>	<code>(→ 'π/2')</code>
2F2006	<code>^metapi/2</code>	<code>(→ π 2 x/ #3)</code>
2E9006	<code>^pisur-2</code>	<code>(→ '-π/2')</code>
2F4006	<code>^meta-pi/2</code>	<code>(→ π 2 x/ xNEG #4)</code>
2F3006	<code>^metapi/4</code>	<code>(→ π 4 x/ #3)</code>
2F5006	<code>^meta-pi/4</code>	<code>(→ π 4 x/ xNEG #4)</code>
2F6006	<code>^pifois2</code>	<code>(→ '2*π')</code>
2EC006	<code>^'xPI</code>	<code>(→ xPI)</code>
2F9006	<code>^base_ln</code>	<code>(→ 'e')</code>
2FA006	<code>^meta_e</code>	<code>(→ e #1)</code>
2EE006	<code>^'xi</code>	<code>(→ xi)</code>
2ED006	<code>^metai</code>	<code>(→ i #1)</code>
2EF006	<code>^ipi</code>	<code>(→ 'i*π')</code>
2F0006	<code>^metaipi</code>	<code>(→ i π x* #3)</code>
2F8006	<code>^metapi*2</code>	<code>(→ π 2 x* #3)</code>

地址	名称	说明
2F7006	$\wedge\text{deuxipi}$	$(\rightarrow '2*i*\pi')$

56.1.7 列表应用

地址	名称	说明
3F0006	$\wedge\text{DIVOBJext}$	$(\{o1\dots on\} ob \rightarrow \{o1/ob\dots on/ob\})$ 将列表的所有元素除以 ob 。测试 ob 是否等于 1。
3F2006	$\wedge\text{LOPDext}$	$(\{o1\dots on\} ob \rightarrow \{o1/ob\dots on/ob\})$ 与调用 RDIVext 的 DIVOBJ 不同, LOPDext 调用 QUOText 进行除法。
地址	名称	说明
269006	$\wedge\text{LOPlex}$	$(\{ \} ob \text{ binop} \rightarrow \{ \})$ 将 « ob binop » 非递归地应用于列表的各元素。
26A006	$\wedge\text{LOPAext}$	$(\{ \} ob \text{ binop} \rightarrow \{ \})$ 将 « op binop » 递归地应用于列表的各元素, 而不是列表元素本身。
10F006	$\wedge\text{LOPMext}$	$(ob \{ \} \rightarrow \{ \})$ 将列表中的每个元素乘以给定对象。
45F006	$\wedge\text{LISTEXEC}$	$(ob \{ \} \rightarrow ob')$ $(ob \{ \} \rightarrow \{ \})$ 第一种情况下, 列表应为 $\{ 'X=1' 'Y=2' \dots \}$ 的形式; 第二种情况下, 应为 $\{ 'X=1' 'X=2' \}$ 的形式。第一种情况下, 将 ob 中出现的所有 X 替换为 1、 Y 替换为 2, 依此类推。第二种情况下, 依次在 X=1 、 X=2 时对 ob 求值。
460006	$\wedge\text{LISTEXEC1}$	$(\{ \} objet \rightarrow \{ \})$
461006	$\wedge\text{SECOEXEC}$	$(\{ \} prog \rightarrow \{ \})$ 对 ob 的每个元素执行 prog 。
268006	$\wedge\text{PFEXECext}$	$(symb \text{ prg} \rightarrow symb)$
26B006	$\wedge\text{LISTSECOext}$	$(composite \rightarrow composite)$ 将 1LAM 非递归地应用于列表的所有元素。
26D006	$\wedge\text{CK1TONOext}$	$(ob \rightarrow ob')$ 将 prg 应用于 ob ; 对于列表则递归应用。 prg 从运行流中取得。

56.1.8 Irrquads

地址	名称	说明
167006	$\wedge\text{TYPEIRRQ?}$	$(ob \rightarrow flag)$ ob 是否为 irrquad ?
168006	$\wedge\text{DTYPEIRRQ?}$	$(ob \rightarrow ob \text{ flag})$ 先执行 DUP , 再执行 $\wedge\text{TYPEIRRQ?}$ 。
165006	$\wedge\text{QXNDext}$	$(irr \rightarrow a \ b \ c)$ 如果堆栈第 1 层不是 irr , 则 b=0 且 c=1 。
地址	名称	说明
166006	$\wedge\text{NDXQext}$	$(a \ b \ c \rightarrow irr)$
2D8006	$\wedge\text{IRRQ\#ULTIMATE}$	$(ob \rightarrow \# \ c)$ 求出 « 的深度, 并返回 irr 的最终 c 。
508006	$\wedge\text{QCONJext}$	$(irr \rightarrow irr')$ 求 irr 的 irr 共轭。这不是复共轭。
509006	$\wedge\text{QABSext}$	$(irr \rightarrow irr \text{ sign})$ 求 irr 的符号。如果 irr 由 Z 构成, 则总能成功。
51A006	$\wedge\text{QNORMext}$	$(Zirr \rightarrow a^2-b*c^2)$ irrquad 的 irr 范数。这不是复数的模。
4D4006	$\wedge\text{SECOSQFFext}$	$(:: x \ll a \ b \ c \ x) \rightarrow \{ fact1 \ mult1 \dots factn \ multn \})$ 对 irrquad 和高斯整数进行因式分解。
124006	$\wedge\text{PREPARext}$	$(o1 \ o2 \rightarrow a1 \ b1 \ c1 \ a2 \ b2 \ c2)$ 返回 o1 和 o2 的 irrquad 分解, 其中 c1=c2 , 或 c1 与 c2 没有公因子。 c1<c2 ; 排序由 LESSCOMPLEX? 处理, 先按类型, 再按 CRC 。
2DA006	$\wedge\text{LISTIRRQ}$	$(ob \{ \} \rightarrow \{ \})$ 将对象中所有 irrquad 的 C 部分添加到列表中。

56.1.9 杂项

地址	名称	说明
3E7006	$\wedge\text{PSEUDOPREP}$	$(o2 \ o1 \rightarrow o2*a1.n^{\wedge} o1 \ a1.n^{\wedge})$
3FB006	$\wedge\text{HSECO2RCext}$	$(ob \rightarrow ob')$ 将常量从内部形式转换为用户形式。
3FC006	$\wedge\text{SECO2CMPext}$	$(seco \rightarrow symb)$ 对复数作反向转换。如果不在有理表达式的最顶层, 应禁用 polarflag 。

地址	名称	说明
3FF006	^VALOBJext	(# { ..{Q}.. } {var1..varn} → { ..{ob}.. }) 反向转换列表中嵌套在深度 # 处的对象。化简 var1..varn。
401006	^VAL2ext	(# { ..{Q}.. } {var1..varn} → { ..{ob}.. }) 反向转换列表中嵌套在深度 # 处的对象。不化简 var1..varn。若已设置 positivfflag, 则按幂次升序进行转换, 这对 SERIES 和 LIMIT 命令很有用。
402006	^INVAL2	(P # → symbpoly) LAM2 必须包含 Lvar, # 是深度。
403006	^METAVAL2	(# Meta_list → Meta_symb) LMA2 必须包含 Lvar, LAM1 会被修改。
404006	^VAL1	(ob → ob) LAM2 必须包含 Lvar, LAM1 会被修改。
405006	^VAL1M	(ob → Meta_symb) LAM2 必须包含 Lvar, LAM1 会被修改。
45C006	^IDNTEXEC	(symb idnt → symb') 尝试查找使 symb=0 的 idnt。在 symb' 中以等式'idnt=..' 的形式返回一个解。
121006	^MP0	(ob → ob 1) 返回所选类型的数字 1。符号对象/ROMPTR 形式的 1 看起来很奇怪; 它用于避免 infinity^0/undef^0 返回 1。
26C006	^rpnQOBJext	(ob → ob') prg 从堆栈中取得。检查 idnt 名称开头的 d1、d2、..., 以确定 idnt 是否表示用户函数的导数。每剥除一个前缀后, 如果所得 idnt 位于当前目录中, 便停止。示例: 如果'd2d1Y' 未定义, 而'd1Y' 定义为« », 则'd2d1Y' 返回 { #2 } « »; 如果 d2d1Y、d1Y 和 Y 均未定义, 则返回 { #2 #1 } 'Y'。

地址	名称	说明
29D006	^SIMPIDNT	(idnt → ob) 对 idnt 求值; 如果已定义, 则递归查找其内容。遇到循环定义时不报错, 但会显示警告。
29F006	^RCL1IDNT	(idnt/lam → ob) 递归取得 idnt 的内容。LAM1 至 LAM3 必须已绑定。
2A7006	^SWPSIMPNDXF	(ob2 ob1 → ob1/ob2) 化简后的分数 (内部形式)。
2A8006	^SIMPNDXFext	(ob2 ob1 → ob2/ob1) 化简后的分数 (内部形式)。
2B6006	^CMODext	(C2 C1 → C1 C2_mod_C1)
2BD006	^SQFF2ext	(l1...ln #n-1 → l1'...ln' #n-1)
2BE006	^PPZ	(p → p/pgcd pgcd) ob 是 P 的所有常数系数的 gcd。常数系数可以是整数、高斯整数或 irrquad; irrquad 使用其自身的“gcd”实现。
117007	^PPZZ	(ob → ob zint) 在 PPZ 的基础上进一步检查, 以确保返回 zint。
2BF006	^PZHSTR	(a z → a mod z)
2C0006	^HORNER1ext	(P r → P[r])
2C1006	^PEval	(P r → P[r]) P 必须是列表多项式。
2C6006	^SQRT_IN?	({ } → { } flag) 如果 { } 中有一个元素是包含 sqrt 的 symb, 则返回 TRUE。
2C7006	^IS_SQRT?	(symb → flag)
2C9006	^IS_XROOT?	(symb → flag)
2CA006	^STOPPRIMIT	(symb →) 将原函数存入 PRIMIT 变量。
2CB006	^CONTAINS_LN?	(symb → symb flag)
2D4006	^FOURIERext	(symb n → cn) 计算一个 2π 周期函数的第 n 个傅里叶系数。

地址	名称	说明
2D9006	^LESSCOMPLEX?	(ob1 ob2 → ob1 ob2 flag) 先按类型、再按 CRC 比较对象。如果 ob1 的复杂度低于 ob2, 则 flag 为 TRUE (ob1>ob2)。若 ob1 或 ob2 是 irrq, 则先查找 ob1 和 ob2 的最终类型。如果两者的最终类型相同, 则通过比较 « depth 排序。
2DD006	^TABLECOSext	(→ { }) 特殊 COS 值表 (k*pi/12)。
2DE006	^TABLETANext	(→ { }) 特殊 TAN 值表 (k*pi/12)。
101007	^LINEARAPPLY	(symb nonrat_prg rat_prg → symb) 应用线性性质。nonrat_prg 应用于非有理部分: symb → symb。rat_prg 应用于有理部分: symb → symb。对 symb 应用线性性质。

地址	名称	说明
106007	$\wedge A/B2PQR$	($A \ B \rightarrow P \ Q \ R$) 将分数 A/B 写成 $E[P]/P*Q/E[R]$ 。Q 与 R 的正向移位互素。
107007	$\wedge GOSPER?$	($P \ Q \ R \rightarrow P \ R \ Y \ T$) ($P \ Q \ R \rightarrow F$) 求解 $P = Q \ E[Y] - R \ Y$ 中的 Y。
0CB007	$\wedge FRACPARITY$	($fr \rightarrow Z$) 测试分数（内部表示）是偶函数、奇函数还是二者皆非。若为偶函数，Z=1；若为奇函数，Z=-1；若二者皆非，Z=0。
0D5007	$\wedge FR2ND\%$	($fraction-1 \rightarrow N \ D \ \%$) 提取分数的平凡幂。
4D1006	$\wedge MSECOSQFF$	($ob \rightarrow Meta$) 对扩展进行因式分解。

第 VI 部分 索引

附录 F

按名称排序的词条

地址	名称	页码	地址	名称	页码
0530B3	(BBDownArrow)	141	05B0B3	(BBEmpty?)	142
05C0B3	(BBGetDefltHeight)	142	03F0B3	(BBGetN)	141
0370B3	(BBGetNGrob)	141	0380B3	(BBGetNStr)	141
04B0B3	(BBIsChecked?)	141	0150B3	(BBMoveTo)	141
0590B3	(BBPgDown)	141	05A0B3	(BBPgUp)	142
0190B3	(BBRecalOff&Disp)	141	02F0B3	(BBReDrawBackgr)	141
03B0B3	(BBRereadChkEnbl)	141	0250B3	(BBReReadCoords)	141
03C0B3	(BBRereadFullScr)	141	0240B3	(BBReReadHeight)	141
03E0B3	(BBReReadNElems)	141	0230B3	(BBReReadPageSize)	141
0260B3	(BBReReadWidth)	141	0290B3	(BBRunCanclAction)	141
0280B3	(BBRunENTERAction)	141	0220B3	(BBRunEntryProc)	141
0540B3	(BBSpace)	141	0520B3	(BBUpArrow)	141
03D0B3	(BReReadMenus)	141	0190E0	BRRclC1	142
0000B3	Choose	141	0050B3	ChooseMenu0	141
0060B3	ChooseMenu1	141	0070B3	ChooseMenu2	141
0630B3	ChooseSimple	141	353F7	!!append\$	45
35369	!!append\$?	45	353EB	!!insert\$	45
25E67	!*triand	46	25E68	!*trior	46
353CD	!append\$	45	35F6A	!append\$SWAP	45
2F191	!DcompWidth	46	3533C	!insert\$	45
03EC2	#*	27	2632D	#*OVF	27
03DBC	#+	26	36851	#+-1	27
357FC	#+DUP	27	36093	#+OVER	27
34417	#+PICK	74	344DD	#+ROLL	73
35E39	#+SWAP	27	3453D	#+UNROLL	73
03DE0	#-	27	35552	#-#2/	27
36815	#-+1	27	3581F	#-DUP	27
360A7	#-OVER	27	34405	#-PICK	74
344CB	#-ROLL	73	35E4D	#-SWAP	27
3452B	#-UNROLL	73	03EF7	#/	27
03CC7	#0<>	28	03CA6	#0=	28
34A7E	#0=?SEMI	90	36383	#0=?SKIP	90
348FC	#0=case	90	36F15	#0=ITE	90
35B96	#0=UNTIL	94	03DEF	#1+	26
36A13	#1+'	84	073DB	#1+_ONE_DO	95
35830	#1+DUP	27	362CA	#1+LAST\$	45
35FB0	#1+NDROP	58, 72	34436	#1+PICK	74
344F2	#1+ROLL	73	2F222	#1+ROT	27
35E61	#1+SWAP	27	34552	#1+UNROLL	73
03E0E	#1-	27	36815	#1-	27
36851	#1-+	27	35E89	#1-1SWAP	27
35841	#1-DUP	27	3601B	#1-ROT	27
3628E	#1-SUB\$	45	28071	#1-SWAP	27, 58
281D5	#1-UNROT	27	361EE	#1-{ }N	56
35675	#10*	27	35620	#10+	27
03880	(#102A8)	25	3375B	(#110)	21
3E17B	(#111)	21	3562A	#12+	27
2777E	(#12F)	22	3378D	(#132)	22
337A1	(#134)	22	337AB	(#135)	22
337B5	(#136)	22	337BF	(#137)	22
337C9	(#138)	22	337D3	(#139)	22
337DD	(#13A)	22	337E7	(#13B)	22
337F1	(#13D)	22	337FB	(#13E)	22
3530D	#1<>	28	352FE	#1=	28
3639C	#1=?SKIP	90	36D8A	#1=case	90
03E6F	#2*	27	03E2D	#2+	26
34451	#2+PICK	74	34517	#2+ROLL	73
34564	#2+UNROLL	73	03E4E	#2-	27
03E8E	#2/	27	3380F	(#200)	22
3C8D0	(#2111)	24	03FEF	(#2614)	24
03FF9	(#2686)	24	36711	#2<>	28
352F1	#2=	28	091B4	(#2D541)	25
03FA9	(#2E48)	25	355FD	#3+	26
34465	#3+PICK	74	2D6006	^#3+ROLL	73
355DF	#3-	27	3A1C2	(#304)	22
3B9FA	(#313)	22	350F5	(#37258)	25
352E0	#3=	28	35602	#4+	26
34474	#4+PICK	74	355DA	#4-	27
3C11E	(#410)	22	3B928	(#411)	22
33837	(#412)	22	3BA2D	(#414)	22
0803F	(#414C1)	25	3B93D	(#415)	22

地址	名称	页码	地址	名称	页码
33841	(#444)	22	3C10F	(#450)	22
3B952	(#451)	22	33855	(#452)	22
3BA18	(#454)	22	3B913	(#455)	22
3A12D	(#4FF)	22	35607	#5+	26
355D5	#5-	27	3385F	(#510)	22
33869	(#511)	22	3BA09	(#515)	22
08ECE	(#536A8)	25	33873	(#550)	22
366FD	#5=	28	3C8DF	(#5B11)	25
356B8	#6*	27	3560C	#6+	26
355D0	#6-	27	277F6	(#605)	22
27800	(#606)	22	2780A	(#607)	22
27814	(#608)	22	2781E	(#609)	22
27828	(#60A)	23	27832	(#60B)	23
2783C	(#60C)	23	27846	(#60D)	23
2768E	(#60E)	23	27698	(#60F)	23
276AC	(#611)	23	276B6	(#612)	23
276C0	(#613)	23	276CA	(#614)	23
0657E	(#61441)	25	276D4	(#615)	23
276DE	(#616)	23	276E8	(#617)	23
27792	(#618)	23	2779C	(#619)	23
277A6	(#61A)	23	277B0	(#61B)	23
277BA	(#61C)	23	277C4	(#61D)	23
277CE	(#61E)	23	277D8	(#61F)	23
277E2	(#620)	23	277EC	(#621)	23
276F2	(#622)	23	276FC	(#623)	23
27706	(#624)	23	27710	(#628)	23
2771A	(#629)	23	27724	(#62A)	23
2772E	(#62B)	23	27738	(#62C)	23
27742	(#62D)	23	27788	(#62E)	23
35611	#7+	26	33891	(#700)	23
3C17A	(#710)	23	3C16B	(#750)	23
08DF7	(#7FF)	23	3569B	#8*	27
35616	#8+	26	27878	(#800)	23
3B976	(#822)	23	3C83C	(#82C)	23
3B967	(#855)	23	3C81E	(#85C)	24
3389B	(#861)	24	338A5	(#862)	24
338AF	(#865)	24	338B9	(#86E)	24
3362F	(#8F)	21	3E7FF	(#8F1)	24
3E759	(#8FD)	24	3561B	#9+	27
33643	(#92)	21	3364D	(#9A)	21
3EAFB	(#9F)	21	3E7E9	(#9F1)	24
3E743	(#9FD)	24	25F72	#:>\$	44
03CE4	#<	28	366BC	#<3	28
03D4E	#<>	28	36D9E	#<>case	90
36D76	#<case	90	36F29	#<ITE	90
03D19	#=	28	363B5	#=?SKIP	90
348D2	#=case	90	34939	#=casedrop	90
36590	#=casedrpfls	90	35C54	#=ITE	90
37752	#=POSCOMP	55	03D83	#>	28
25F77	#>\$	44	36739	#>1	28
36DB2	#>2case	90	363E2	#>?SKIP	90
36DCB	#>case	90	05A75	#>CHR	44
059CC	#>HXS	48	36F3D	#>ITE	90
07E50	#>ROMPTR	69	0EE006	^#>Z	180
3373D	#_102	21	2774C	(#A01)	24
27756	(#A02)	24	27760	(#A04)	24
2776A	(#A05)	24	27774	(#A06)	24
338CD	(#A11)	24	3D50D	(#A110)	25
338D7	(#A12)	24	338E1	(#A1A)	24
3D52B	(#A1A0)	25	3367F	(#A2)	21
338EB	(#A21)	24	338F5	(#A22)	24
338FF	(#A2A)	24	39E6B	(#A4)	21
33689	(#A5)	21	33909	(#A61)	24
33913	(#A62)	24	3391D	(#A65)	24
33927	(#A6E)	24	03826	(#A8241)	25
336A7	(#A9)	21	33931	(#AA1)	24
3D51C	(#AA10)	25	3393B	(#AA2)	24
33945	(#AAA)	24	2C4D2	(#AAA0)	25
3BD4C	(#AF)	21	03EB1	#AND	28
39277	(#B437D)	25	38275	(#BB)	21
3B7AD	(#BBBB)	25	3394F	(#C06)	24
33959	(#C07)	24	33963	(#C08)	24
33977	(#C0B)	24	3B904	(#C22)	24
3C800	(#C2C)	24	3B8F5	(#C55)	24
3C7E2	(#C5C)	24	3E7DA	(#C8)	21
33981	#CAlarmErr	24	3BD65	(#CF)	21

地址	名称	页码	地址	名称	页码
08F1F	(#D6A8)	25	038DC	(#E13A8)	25
33995	#EXITERR	25	07C007	^#FACT	181
38266	(#FFFF)	25	3736E	#FIVE#FOUR	25
3551D	#MAX	28	35511	#MIN	28
33783	(#NoRoomForSt)	22	37315	#ONE#27	25
33747	#SyntaxErr	21	3735C	#THREE#FOUR	25
3734A	#TWO#FOUR	25	37328	#TWO#ONE	25
3733A	#TWO#TWO	25	37294	#ZERO#ONE	25
37305	#ZERO#SEVEN	25	39C9F	(\$&ob)	47
36851	\$!+	27	25F7C	\$>GROB	68
25F81	\$>grob	68	25F86	\$>GROBCR	68
25F8B	\$>grobCR	68	05B15	\$>ID	77
3402C	\$_"	43	34056	\$_2DQ	43
3403A	\$_::	43	34002	\$_<<>>	43
3401E	\$_[]	43	34064	\$_ECHO	42
34076	\$_EXIT	42	340B4	\$_GRAD	42
34048	\$_LRParens	43	33FD2	\$_R<<	42
33FE2	\$_R<Z	42	340A4	\$_RAD	42
34088	\$_Undefined	42	33FF2	\$_XYZ	42
34010	\$_{}	43	33B45	(\$DER)	44
30385	%%*	34	3602F	%%*ROT	34
35EDE	%%*SWAP	34	36C7C	%%*UNROT	34
3032E	%%+	34	3033A	%%-	34
2FBE5	%%.1	32	30DC8	%%.4	32
2FBFF	%%.5	32	303D3	%%/	34
36BE6	%%/>%	34	2FB49	%%0	32
30112	%%0<	35	301F6	%%0<=	35
301A6	%%0<>	35	30145	%%0=	35
30173	%%0>	35	301CE	%%0>=	36
2FB63	%%1	32	3047D	%%1/	35
2FC19	%%10	32	30CC7	%%12	32
2FB7D	%%2	32	27A89	%%2PI	32
2FB97	%%3	32	2FBB1	%%4	32
2FBCB	%%5	32	30CEB	%%60	32
30BEA	%%7	32	3020A	%%<	35
30296	%%<=	35	3026A	%%>	35
2FF9B	%%>%	32	30280	%%>=	35
261CF	%%>C%	37	302DB	%%ABS	34
306F3	%%ACOSRAD	34	3073A	%%ANGLE	35
30757	%%ANGLEDEG	35	30767	%%ANGLERAD	34
306C3	%%ASINRAD	35	302FB	%%CHS	35
30642	%%COS	35	30653	%%COSDEG	35
307B2	%%COSH	35	30663	%%COSRAD	35
30507	%%EXP	35	30984	%%FLOOR	35
30912	%%H>HMS	104	30984	%%INT	35
30546	%%LN	35	3057F	%%LNP1	35
300C7	%%MAX	35	30EB0	%%P>R	35
2FADB	%%PI	32	11C007	^%%PSI	209
30E83	%%R>P	35	305F1	%%SIN	35
30602	%%SINDEG	35	30780	%%SINH	35
30612	%%SINRAD	34	304D5	%%SQRT	35
30693	%%TANRAD	35	3044A	%%^	34
303A7	%%*	33	3035F	%%+	33
25E69	%%+SWAP	33	3036C	%%-	33
339D3	%%-.5)	30	2FA09	%%-1	30
2B289	%%-1=case	91	2FA1E	%%-2	30
2FA33	%%-3	30	2FA48	%%-4	30
2FA5D	%%-5	30	2FA72	%%-6	30
2FA87	%%-7	30	2FA9C	%%-8	30
2FAB1	%%-9	30	2FB0A	%%-MAXREAL	30
2FB34	%%-MINREAL	30	27118	%%.1	30
339BE	%%.5	30	303E9	%%/	33
2F937	%%0	30	30123	%%0<	35
301BA	%%0<>	35, 87	30156	%%0=	35
2B149	%%0=case	91	30184	%%0>	35
301E2	%%0>=	35	2F94C	%%1	30
26F36	%%1+	33	26F4A	%%1-	33
270EE	%%1.8)	30	3049A	%%1/	33
339E8	%%10	31	35C18	%%10*	33
27E5D	%%100	31	2FCE6	%%11	31
2FCFB	%%12	31	2FC7D	%%1200)	31
2FD10	%%13	31	2FD25	%%14	31
2FD3A	%%15	31	2FCD1	%%15360)	31
2FD4F	%%16	31	2FD64	%%17	31
2FD79	%%18	31	339FD	%%180	31

地址	名称	页码	地址	名称	页码
2FD8E	%19	31	2B1A3	%1=case	91
4CE006	^%1TWO	210	2F961	%2	30
2FDA3	%20	31	33A12	(%200)	31
2FDB8	%21	31	2FDCE	%22	31
2FDE2	%23	31	2FDF7	%24	31
2FC92	(%2400)	31	2FE0C	%25	31
2FE21	%26	31	2FE36	%27	31
2FE4B	(%28)	31	2FE60	(%29)	31
2B20C	%2=case	91	2F976	%3	30
2FE75	(%30)	31	2FE8A	(%31)	31
2FE9F	(%32)	31	2FEB4	(%33)	31
2FEC9	(%34)	31	2FEDE	(%35)	31
33A27	%360	31	2F98B	%4	30
33A3C	(%400)	31	2FCA7	(%4800)	31
2F9A0	%5	30	2F9B5	%6	30
2F9CA	%7	31	2F9DF	%8	31
27103	%80	31	2F9F4	%9	31
2FCBC	(%9600)	31	3025C	%<	35
302A1	%<=	35	302B7	%<>	35
302AC	%=	35	30275	%>	35
2EFCB	%>#	48	2FFAC	%>%%	32
30346	%>%%-	33	30489	%>%%1/	33
30746	%>%%ANGLE	34	304E1	%>%%SQRT	33
35ECA	%>%%SWAP	32	3028B	%>=	35
05C27	%>C%	37	3005E	%>HMS	104
2F223	%>TAG	51	302EB	%ABS	33
262EC	%ABSCOECE	26	306DC	%ACOS	33
307FE	%ACOSH	33	305A5	%ALOG	33
30723	%ANGLE	34	306AC	%ASIN	33
307EB	%ASINH	33	3070C	%ATAN	33
30811	%ATANH	33	3095E	%CEIL	34
3041B	%CH	34	3030B	%CHS	33
3084D	%COMB	34	3062B	%COS	33
307C5	%COSH	33	3000D	%D>R	34
339A9	%e	30	3051A	%EXP	33
3052D	%EXPM1	33	30824	%EXPONENT	33
30AAF	%FACT	34	30971	%FLOOR	33
30938	%FP	33	3008B	%HMS+	104
300B3	%HMS-	104	30077	%HMS>	104
3094B	%IP	33	2B3FD	%IP>#	32
30559	%LN	33	30592	%LNP1	33
3056C	%LOG	33	3031B	%MANTISSA	33
300E0	%MAX	34	35DBC	%MAXorder	34
2FAF5	%MAXREAL	32	300F9	%MIN	34
2FB1F	%MINREAL	30	305C7	%MOD	34
30837	%NFACT	34	3046C	%NROOT	34
303B4	%OF	34	30860	%PERM	34
2FAC6	%PI	30	30EA6	%POL>%REC	34
30040	%R>D	34	309AD	%RAN	34
30A2F	%RANDOMIZE	34	30E79	%REC>%POL	34
302C2	%SGN	33	305DA	%SIN	33
30799	%SINH	33	30EDD	%SPH>%REC	34
304F4	%SQRT	33	303F6	%T	34
3067C	%TAN	33	307D8	%TANH	33
4EA61	(%TICKSday)	31	4EA4C	(%TICKShour)	31
4EA37	(%TICKSmin)	31	4EA22	(%TICKSsec)	31
4EA76	(%TICKSweek)	32	3045B	%^	33
05193	&\$	45	36FF6	&\$SWAP	45
0521F	&COMP	54	0518A	&HXS	48
06E97	'	84	25E6A	'DoBadKey	84
25E6B	'DoBadKeyT	84	2B90B	'DROPFALSE	84
3619E	'ERRJMP	84	27B43	'IDFUNCTION	84
27B7F	'IDPARAMETER	85	27B6B	'IDPOLAR	85
27155	'IDX	77	36A8B	'LAMLNAMESTO	102
36306	'NOP	84	06EEB	'R	83
36A27	'R'R	83	29ED0	'Rapndit	85
06F66	'REVAL	83	36A4A	'RRDROP	83
354CB	'RSaveRomWrd	117	354CB	'RSAVEWORD	117
29786	('RSWAP#1+)	28	36BBE	('x*)	84
36BD2	'xDER	84	36AA4	'xDEREQ	85
2EE006	^'xi	221	2EC006	^'xPI	221
073A5	+LOOP	95	25F9A	0LASTOWDOB!	117
25F9A	0LASTOWDOB!、0LastRomWrd!	117	388006	^1&meta	195
40A006	^1-x^2/1+x^2	197	34670	10GETLAM	78
344A8	(10PICK)	74	3466B	10PUTLAM	78

地址	名称	页码	地址	名称	页码
3616F	10UNROLL	73	3467A	11GETLAM	78
34675	11PUTLAM	79	34684	12GETLAM	78
3467F	12PUTLAM	79	3468E	13GETLAM	78
34689	13PUTLAM	79	34698	14GETLAM	78
34693	14PUTLAM	79	33B13	(14SPACES\$)	44
346A2	15GETLAM	78	3469D	15PUTLAM	79
346AC	16GETLAM	78	346A7	16PUTLAM	79
346B6	17GETLAM	78	346B1	17PUTLAM	79
346C0	18GETLAM	78	346BB	18PUTLAM	79
346CA	19GETLAM	78	346C5	19PUTLAM	79
362A2	1_#1-SUB	45	362A2	1_#1-SUB\$	45
35DEE	1ABND\$WAP	79	364FF	1GETABND	79
34616	1GETLAM	78	2F318	1GETLAM\$WP1+	79
35F42	1GET\$WAP	79	36518	1LAMBIND	77
3A4006	^1metainf#	198	3A1006	^1metaundef#	198
2B3A6	1NULLLAM{ }	79	34611	1PUTLAM	78
2F180	1REV	32	25E6D	1stkdecomp\$w	46
36676	2#0=OR	29	30E5B	2%>%	32
30E47	2%>%	32	34B4F	2'RCOLARPITE	89
441006	^2*1-cos/sin	198	443006	^2*sin/1+cos	198
346D4	20GETLAM	78	346CF	20PUTLAM	79
346DE	21GETLAM	78	346D9	21PUTLAM	79
346E8	22GETLAM	78	346E3	22PUTLAM	79
346F2	(23GETLAM)	78	346ED	(23PUTLAM)	79
346FC	(24GETLAM)	78	346F7	(24PUTLAM)	79
34706	(25GETLAM)	78	34701	(25PUTLAM)	79
34710	(26GETLAM)	78	3470B	(26PUTLAM)	79
3471A	(27GETLAM)	78	34715	(27PUTLAM)	79
34BAB	2@REVAL	84	173006	^2DMATRIX?	185
03258	2DROP	72	355A5	2DROP00	26
25E6E	2DropBadKey	122	35B32	2DROPFALSE	88
2D7006	^2DROPTRUE	88	392006	^2DROPZ0	180
031AC	2DUP	71	3674D	2DUP#+	27
358C2	2DUP#<	28	358DC	2DUP#	28
358F8	2DUP#>	28	36CA4	2DUP5ROLL	71
36621	2DUPEQ	88	35D30	2DUP\$WAP	71
3370B	2EXT	21	3632E	2GETEVAL	79
34620	2GETLAM	78	336E3	2GROB	21
336CF	(2HXS)	21	2F17E	2HXSLIST?	26
155006	^2LAMBIND	77	33459	2LIST	19
3A3006	^2metainf#	198	3A0006	^2metaundef#	198
2825E	(2NELCOMPDROP)	54	271F4	(2NULLLAM{ })	79
37087	2Ob>Seco	57	37046	2OVER	73
4E1006	^2POLYNOMIAL?	211	3461B	2PUTLAM	78
343E1	2RDROP	84	331B1	2REAL	18
3570C	2skipcola	85	35018	2\$WAP	72
12A006	^2SYMBINCOMP	56, 179	40D006	^2x/1+x^2	197
34BBB	3@REVAL	84	341D2	3DROP	72
2D5006	^3DUP	71	3462A	3GETLAM	78
156006	^3LAMBIND	77	27208	{3NULLLAM{ }}	79
34485	3PICK	73	36789	3PICK#+	28
36CCC	3PICK3PICK	74	360F7	3PICKOVER	73
35F1A	3PICK\$WAP	73	34625	3PUTLAM	78
343F3	3RDROP	84	33765	3REAL	21
35703	3skipcola	85	3422B	3UNROLL	73
341D7	4DROP	72	34634	4GETLAM	78
2B3B7	4NULLLAM{ }	79	3448A	4PICK	74
3679D	4PICK#+	28	35E20	4PICK#+\$WAP	28
35E20	4PICK+\$WAP	28	3610B	4PICKOVER	74
35F2E	4PICK\$WAP	74	3462F	4PUTLAM	78
3423A	4ROLL	72	3588B	4ROLLDROP	72
360E3	4ROLLOVER	72	36043	4ROLLROT	72
35F06	4ROLL\$WAP	72	34331	4UNROLL	73
343CF	4UNROLL3DROP	73	35D44	4UNROLLDUP	73
36057	4UNROLLROT	73	341DC	5DROP	72
3463E	5GETLAM	78	3448F	5PICK	74
34639	5PUTLAM	78	34257	5ROLL	72
358A7	5ROLLDROP	72	356D5	5skipcola	85
34357	5UNROLL	73	341E8	6DROP	72
34648	6GETLAM	78	34494	6PICK	74
34643	6PUTLAM	78	34281	6ROLL	72
3438D	6UNROLL	73	341F4	7DROP	72
34652	7GETLAM	78	34499	7PICK	74
3464D	7PUTLAM	78	342EA	7ROLL	73
35BEB	7UNROLL	73	3465C	8GETLAM	78
3449E	8PICK	74	34657	8PUTLAM	78

地址	名称	页码	地址	名称	页码
342BB	8ROLL	73	3615B	8UNROLL	73
34666	9GETLAM	78	344A3	(9PICK)	74
34661	9PUTLAM	78	28225	(9UNROLL)	73
05445	::N	55	3631A	::NEVAL	57
09F006	^::POLY	202	27F47	<DelKey	173
27EAF	<SkipKey	173	27F9A	>DelKey	173
25F15	>FONT	160	0525B	>H\$	45
052C6	>HCOMP	54	0A2006	^>HPOLY	202
0A4006	^>HPOLYN	202	25F90	>LANGUAGE	107
2620B	>MINIFONT	160	0A7006	^>POLY	202
4F8006	^>POLYTRIM	211	06F9F	>R	83
25E6F	>Review\$	46	0EB007	^>SIGNE	212
27EFB	>SkipKey	173	052EE	>T\$	45
05E81	>TAG	51	052FA	>TCOMP	54
0A1006	^>TPOLY	202	0A3006	^>TPOLYN	202
35A88	?>ROMPTR	69	25E70	?ATTN_QUIT	122
25E70	?ATTNQUIT	122	25E71	?BlinkCursor	107
361C6	?CARCOMP	54	25E72	?CaseKeyDef	90
25E73	?CaseRompPtr@	90	25E74	?ClrAlg	158
25E75	?ClrAlgSetPr	158	2F19F	?DispCommandLine	155, 172
2DFCC	?DispMenu	155, 164	2C341	?DispStack	155
2C311	?DispStatus	156	2E5006	^?ext	221
2EE5D	?FlashAlert	107	39332	{?GETMSG}	97
34A46	?GOTO	84	3705A	?Ob>Seco	57
25E77	?OKINALG	118	25E78	?PURGE_HERE	102
35AAB	?ROMPTR>	69	34AA1	?SEMI	88
3692D	?SEMIDROP	88	0712A	?SKIP	89
35DDA	?SKIPSWAP	89	2EF73	?Space/Go>	172
25E79	?STO_HERE	101	35F56	?SWAP	88
35F97	?SWAPDROP	89	293A3	{?symcomp}	63
0797B	@	101	07943	@LAM	78
33107	any	17	25F0F	a%>\$	44
106007	^A/B2PQR	224	20B006	^ABCUV	218
07497	ABND	77	04EA4	ABORT	97
25FA4	ABUFF	155	43C006	^ACOS2ASIN	191
424006	^ACOS2LN	191	425006	^acos2ln	198
43A006	^ACOS2Sext	193	451006	^acosh2ln	198
450006	^ACOSH2LNext	191	2EF76	AddLeadingSpace	172
279006	^ADDLISText	204	378006	^ADDMATOBJext	187
295006	^ADDMULTIPL	205	112007	^ADDONEVAR	203
13D006	^addt!=	196	13F006	^addt%	196
141006	^addt%CH	196	143006	^addt%T	196
38B006	^addt/	195	406006	^addt0meta	59
38A006	^addt2	195	133006	^addt<	196
135006	^addt<=	196	13B006	^addt==	196
137006	^addt>	196	139006	^addt>=	196
50E006	^addtABS	196	510006	^addtABSEXACT	196
53E006	^addtACOS	197	54F006	^addtACOSH	197
56E006	^addtALOG	197	151006	^addtAND	196
513006	^addtARG	196	53C006	^addtASIN	197
54C006	^addtASINH	197	540006	^addtATAN	197
549006	^addtATANH	197	560006	^addtCEIL	197
14B006	^addtCOMB	196	506006	^addtCONJ	196
535006	^addtCOS	197	544006	^addtCOSH	197
55A006	^addtD->R	197	558006	^addtEXP	197
570006	^addtEXPM	197	574006	^addtFACT	197
55E006	^addtFLOOR	197	564006	^addtFP	197
39E006	^addti	195	556006	^addtINV	197
562006	^addtIP	197	523006	^addtLN	196
56A006	^addtLNP1	197	56C006	^addtLOG	197
568006	^addtMANT	197	131006	^addtMAX	196
12F006	^addtMIN	196	145006	^addtMOD	196
241006	^ADDTMOD	219	577006	^addtNOT	197
14F006	^addtOR	196	0C9007	^ADDTOREAL	215
14D006	^addtPERM	196	55C006	^addtR->D	197
2EF75	AddTrailingSpace	172	149006	^addtRND	196
511006	^addtSIGN	196	537006	^addtSIN	197
53B006	^addtSINACOS	197	542006	^addtSINH	197
554006	^addtSQ	197	551006	^addtSQRT	197
539006	^addtTAN	197	410006	^addtTAN/2	197
546006	^addtTANH	197	147006	^addtTRNC	196
153006	^addtXOR	196	566006	^addtXPON	197
12D006	^addtXROOT	196	398006	^addt^	195
2AF37	AEQ1stcase	91	2B06A	AEQopscase	91
071AB	AGAIN	94	25FA9	ALARM?	105
2F178	ALARMS@	104	0A8006	^ALG48FCTR?	201

地址	名称	页码	地址	名称	页码
093006	^ALG48MSOLV	205	11A007	^ALGCASCOMPEVAL	192
2BE36	(ALGcase)	92	2AA43	AlgDecomp	47
256EA	AlgEntry?	158	2F1AF	AlgObEdit	172
41C006	^ALOG2EXP	191	03B46	AND	88
25E7C	AND\$	45	36D4E	ANDcase	89
36EED	ANDITE	89	36E6B	ANDNOTcase	89
2F31A	APNDCRLF	44, 108	2EF5A	apndvarlst	56
29F25	AppDisplay!	130	29F35	AppDisplay@	130
35BD7	APPEND_SPACE	45	0FE006	^AppendList	56
2A145	AppError!	130	2A158	AppError@	130
2A055	AppExitCond!	130	2A065	AppExitCond@	130
29F55	AppKeys!	130	29F75	AppKeys0	130
25690	AppMode?	130	07B007	^Approx	221
46C006	^APPROXCOMPEVAL	192	515006	^ARG2	38
17E006	^ARRAY2MATRIX	185	3312F	array	17
3384B	(ARRAYLISTREAL)	22	33391	ARRAYREAL	19
3382D	(ARRAYREALREAL)	22	003007	^ArrayToList	185
002007	^ArrayToMatrix	53	35E006	^ARSIZE	52
437006	^ASIN2ACOS	191	434006	^ASIN2ATAN	191
435006	^asin2atan	198	436006	^ASIN2Cext	193
422006	^ASIN2LN	191	423006	^asin2ln	198
433006	^ASIN2Text	193	44F006	^asinh2ln	198
44E006	^ASINH2LNext	191	2F1A5	AskQuestion	160
2F3B3	(AsnKey)	122	431006	^ATAN2ASIN	191
432006	^atan2asin	198	43E006	^atan2ln	198
43D006	^ATAN2LNext	191	430006	^ATAN2Sext	193
453006	^atanh2ln	198	452006	^ATANH2LNext	191
27882	Attn#	24	25FAE	ATTN?	122
338C3	ATTNERR	24	05040	ATTNFLG@	122
05068	ATTNFLGCLR	122	2EF6C	AtUserStack	117
2EEEF	AUTOSCALE	175	382006	^AXQ	188
0905F	BAK>OB	69	081D9	BAKNAME	69
35C006	^BANGARRY	187	2EFBF	BASE	106
2F9006	^base_ln	221	071A2	BEGIN	94
08F006	^Berlekamp	201	08E006	^BerlekampP	201
3EA006	^BESTDIV2	193	169006	^BESTMATRIXTYPE	53
4C8006	^BEZOUTMSOLV	210	0CF006	^BFactor	181
280006	^BICARREE?	204	25FB3	BIGDISPN	159
25FB8	BIGDISPROW1	159	25FBD	BIGDISPROW2	159
25FC2	BIGDISPROW3	159	25FC7	BIGDISPROW4	159
074D0	BIND	77	33107	BINT0	17
33111	BINT1	17	3316B	BINT10	17
334EF	BINT100	20	334F9	BINT101	20
33503	BINT102	20	3350D	BINT103	20
33517	BINT104	20	33521	BINT105	20
3352B	BINT106	20	33535	BINT107	20
3353F	BINT108	20	33549	BINT109	20
33175	BINT11	17	33553	BINT110	20
3355D	BINT111	20	33567	BINT112	20
33571	BINT113	20	3357B	BINT114	20
33585	BINT115	20	3358F	BINT116	20
33599	BINT117	20	335A3	BINT118	20
335AD	BINT119	20	3317F	BINT12	17
335B7	BINT120	20	335C1	BINT121	20
335CB	BINT122	20	335D5	BINT123	20
335DF	BINT124	20	335E9	BINT125	20
335F3	BINT126	20	335FD	BINT127	20
33607	BINT128	20	33611	BINT129	21
33189	BINT13	17	3361B	BINT130	21
3361B	BINT130d	21	33625	BINT131	21
33625	BINT131d	21	33193	BINT14	17
3319D	BINT15	18	331A7	BINT16	18
331B1	BINT17	18	331BB	BINT18	18
331C5	BINT19	18	3311B	BINT2	17
331CF	BINT20	18	331D9	BINT21	18
331E3	BINT22	18	331ED	BINT23	18
331F7	BINT24	18	33201	BINT25	18
3371F	BINT253	21	33729	BINT255d	21
3320B	BINT26	18	33215	BINT27	18
3321F	BINT28	18	33229	BINT29	18
33125	BINT3	17	33233	BINT30	18
3323D	BINT31	18	33247	BINT32	18
33251	BINT33	18	3325B	BINT34	18
33265	BINT35	18	3326F	BINT36	18
33279	BINT37	18	33283	BINT38	18
3328D	BINT39	18	3312F	BINT4	17

地址	名称	页码	地址	名称	页码
33297	BINT40	18	33387	BINT40h	19
332A1	BINT41	18	332AB	BINT42	18
332B5	BINT43	18	332BF	BINT44	18
332C9	BINT45	18	332D3	BINT46	18
332DD	BINT47	18	332E7	BINT48	18
332F1	BINT49	18	33139	BINT5	17
332FB	BINT50	18	33305	BINT51	19
3330F	BINT52	19	33319	BINT53	19
33323	BINT54	19	3332D	BINT55	19
33337	BINT56	19	33341	BINT57	19
3334B	BINT58	19	33355	BINT59	19
33143	BINT6	17	3335F	BINT60	19
33369	BINT61	19	33373	BINT62	19
3337D	BINT63	19	33387	BINT64	19
33391	BINT65	19	3339B	BINT66	19
333A5	BINT67	19	333AF	BINT68	19
333B9	BINT69	19	3314D	BINT7	17
333C3	BINT70	19	333CD	BINT71	19
333D7	BINT72	19	333E1	BINT73	19
333EB	BINT74	19	333F5	BINT75	19
333FF	BINT76	19	33409	BINT77	19
33413	BINT78	19	3341D	BINT79	19
33157	BINT8	17	33427	BINT80	19
33607	BINT80h	20	33431	BINT81	19
3343B	BINT82	19	33445	BINT83	19
3344F	BINT84	19	33459	BINT85	19
33463	BINT86	19	3346D	BINT87	19
33477	BINT88	19	33481	BINT89	20
33161	BINT9	17	3348B	BINT90	20
33495	BINT91	20	3349F	BINT92	20
334A9	BINT93	20	334B3	BINT94	20
334BD	BINT95	20	334C7	BINT96	20
334D1	BINT97	20	334DB	BINT98	20
334E5	BINT99	20	33585	BINT_115d	20
3358F	BINT_116d	20	335CB	BINT_122d	20
3361B	BINT_130d	21	33625	BINT_131d	21
33751	(BINT_263d)	21	33495	BINT_91d	20
334C7	BINT_96d	20	336D9	BINTC0h	21
0D4006	^BisPrime?	181	2EFC5	bit%*	48
2EFC9	bit%#+	48	2EFC7	bit%#-	48
2EFC3	bit%# /	48	2EFC4	bit%#*	48
2EFC8	bit%#+	48	2EFC6	bit%#-	48
2EFC2	bit%# /	48	2EFBC	bit*	48
2EFB9	bit+	48	2EFBA	bit-	48
2EFBD	bit /	48	2EFAC	bitAND	48
2EFB8	bitASR	48	2EFAF	bitNOT	48
2EFAD	bitOR	48	2EFB6	bitRL	49
2EFB7	bitRLB	49	2EFB4	bitRR	49
2EFB5	bitRRB	49	2EFB0	bitSL	49
2EFB1	bitSLB	49	2EFB2	bitSR	49
2EFB3	bitSRB	49	2EFAE	bitXOR	48
2F16D	Blank\$	45	2EF5E	BlankDA1	157
2EE5C	BlankDA12	157	2F31B	BlankDA2	157
2F31C	BlankDA2a	157	25E7E	BLANKIT	157
2F31D	BOTROW	158	25E7F	Box/StdLabel	67
25E80	Box/StdLbl:	67	0D5006	^BRabin	182
0D0006	^BrentPow	181	261D4	C%%0=	38
27E2E	C%%1	37	05DBC	C%%>%%	37
261D9	C%%>C%	37	261DE	C%%CHS	38
261E3	C%%CONJ	38	51E006	^C%%SQRT	38
27DBF	C%-1	37	2B2A7	C%-1=case	91
27DE4	C%0	37	261E8	C%0=	38
2B15D	C%0=case	91	27E09	C%1	37
25E81	C%1 /	37	2B1C1	C%1=case	91
2B22A	C%2=case	91	2F31F	C%>#	26
05D2C	C%>%	32	25E82	C%>%%	37
25E83	C%>%%SWAP	37	18B006	^C%>C%%	37
25E84	C%ABS	37	25E85	C%ACOS	38
25E86	C%ACOSH	38	25E87	C%ALOG	38
25E88	C%ARG	38	25E89	C%ASIN	38
25E8A	C%ASINH	38	25E8B	C%ATAN	38
25E8C	C%ATANH	38	261ED	C%CHS	37
261F2	C%CONJ	38	25E8D	C%COS	38
25E8E	C%COSH	38	25E8F	C%C^C	37
25E90	C%C^R	37	25E91	C%EXP	38
25E92	C%LN	38	25E93	C%LOG	38

地址	名称	页码	地址	名称	页码
25E94	C%R^C	37	25E95	C%SGN	38
25E96	C%SIN	38	25E97	C%SINH	38
25E98	C%SQRT	37	25E99	C%TAN	38
25E9A	C%TANH	38	188006	^C2C%%	37
25E9B	C>Im%	37	25E9C	C>Re%	37
34D00	CACHE	77	2EF91	CAL_CURS_POS	168
2EF90	CAL_CURS_POS_VIS	168	2B2F2	(CallEditCmd:)	172
57F006	^CANTFACTOR	213	050ED	CAR\$	44
05089	CARCOMP	54	15B006	^CARCOMPext	200
467006	^CASCOMPEVAL	192	46B006	^CASCRUNCH	192
349F9	case	89	34985	case2drop	89
349D6	case2DROP	89	365CC	case2drpfls	89
36C4F	caseDEADKEY	92	36C4F	caseDoBadKey	92
3495D	casedrop	89	349B1	caseDROP	89
36C36	caseDrpBadKy	92	365B3	casedrpfls	89
368FB	casedrptru	89	361B2	caseERRJMP	92
365E5	caseFALSE	89	36B53	caseSIZEERR	92
3652C	caseTRUE	89	1D5006	^CASFLAGEVAL	215
466006	^CASNUMEVAL	192	0516C	CDR\$	44
05153	CDRCOMP	54	25FEF	CENTER\$3x5	68
25FF4	CENTER\$5x7	68	2DA2B	cfC	32
2DA11	cfF	32	08F007	^CFGDISPLAY	214
2B5006	^CGCDext	182	05AB3	CHANGETYPE	107
3355D	char	20	2F1A7	CHARSEDT	45
2F320	CHECKHEIGHT	65	04708	CHECKKEY	121
25F2B	CHECKMENU	165	25EA6	CheckMenuRow	165
2F162	CHECKPICT	174	0AA006	^CheckPNoExt	201
2F163	CHECKPVARs	174	07D007	^CHECKSING	212
070002	(^Choose2)	136	072002	(^Choose3)	136
076002	(^Choose3CANCL)	136	074002	(^Choose3Index)	136
077002	(^Choose3OK)	136	073002	(^Choose3Save)	136
075002	(^ChooseDefHandler)	136	05A51	CHR>#	26
37AA5	CHR>\$	44	33D40	CHR_#	40
33D47	CHR_*	40	33D4E	CHR_+	40
33D55	CHR_,	40	33D5C	CHR_-	40
33F46	CHR_->	42	33D63	CHR_.	40
33D32	CHR_...	40	33D6A	CHR_/	40
33D71	CHR_0	40	33D2B	CHR_00	40
33D78	CHR_1	40	33D7F	CHR_2	40
33D86	CHR_3	40	33D8D	CHR_4	40
33D94	CHR_5	40	33D9B	CHR_6	40
33DA2	CHR_7	40	33DA9	CHR_8	40
33DB0	CHR_9	40	33DB7	CHR_:	40
33DBE	CHR_;	40	33DC5	CHR_<	40
33F4D	CHR_<<	42	33FBD	CHR_<=	42
33FCB	CHR_<>	42	33DCC	CHR_=	40
33DD3	CHR_>	41	33FC4	CHR_>=	42
33F54	CHR_>>	42	33FA1	CHR_[	41
33FA8	CHR_]	41	33DDA	CHR_A	41
33E90	CHR_a	41	33F5B	CHR_Angle	42
33DE1	CHR_B	41	33E97	CHR_b	41
33DE8	CHR_C	41	33E9E	CHR_c	41
33DEF	CHR_D	41	33EA5	CHR_d	41
33D39	CHR_DblQuote	40	33F62	CHR_Deriv	42
33DF6	CHR_E	41	33EAC	CHR_e	41
33DFD	CHR_F	41	33EB3	CHR_f	41
33E04	CHR_G	41	33EBA	CHR_g	41
33E0B	CHR_H	41	33EC1	CHR_h	41
33E12	CHR_I	41	33EC8	CHR_i	41
33F69	CHR_Integral	42	33E19	CHR_J	41
33ECF	CHR_j	41	33E20	CHR_K	41
33ED6	CHR_k	41	33E27	CHR_L	41
33EDD	CHR_l	41	33F70	CHR_LeftPar	40
33E2E	CHR_M	41	33EE4	CHR_m	42
33E35	CHR_N	41	33EEB	CHR_n	42
33F77	CHR_Newline	40	33E3C	CHR_O	41
33EF2	CHR_o	42	33E43	CHR_P	41
33EF9	CHR_p	42	33F7E	CHR_Pi	42
33E4A	CHR_Q	41	33F00	CHR_q	42
33E51	CHR_R	41	33F07	CHR_r	42
33F85	CHR_RightPar	40	33E58	CHR_S	41
33F0E	CHR_s	42	33F8C	CHR_Sigma	42
33F93	CHR_Space	40	33E5F	CHR_T	41
33F15	CHR_t	42	33E66	CHR_U	41
33F1C	CHR_u	42	33F9A	CHR_UndScore	41
33E6D	CHR_V	41	33F23	CHR_v	42

地址	名称	页码	地址	名称	页码
33E74	CHR_W	41	33F2A	CHR_w	42
33E7B	CHR_X	41	33F31	CHR_x	42
33E82	CHR_Y	41	33F38	CHR_y	42
33E89	CHR_Z	41	33F3F	CHR_z	42
33FAF	CHR_{	42	33FB6	CHR_}	42
2F215	CircleB	66	2F216	CircleG1	66
2F217	CircleG2	66	2F218	CircleW	66
2F219	CircleXor	67	2F13D	{CK#-}	27
51D006	^CK%SQRT	34	203006	^CK&CONV2INT	180
202006	^CK&CONVINT	180	25EA7	Ck&DecKeyLoc	121
2631E	CK&DISPATCH0	116	26328	CK&DISPATCH1	116
26323	CK&DISPATCH2	116	007002	^Ck&DoMsgBox	160
262B0	CK0	116	25E9D	CK0ATTNABORT	122
26292	CK0NOLASTWD	116	262B5	CK1	116
26300	CK1&Dispatch	116	187006	^CK1Cext	118, 182
25E9E	CK1NoBlame	117	26297	CK1NOLASTWD	116
26D006	^CK1TONOext	222	184006	^CK1Z	117, 180
262BA	CK2	116	26305	CK2&Dispatch	116
190006	^CK2FPOLY	179	2629C	CK2NOLASTWD	116
185006	^CK2Z	117, 180	262BF	CK3	116
2630A	CK3&Dispatch	116	262A1	CK3NOLASTWD	116
186006	^CK3Z	117, 180	262C4	CK4	116
2630F	CK4&Dispatch	116	262A6	CK4NOLASTWD	116
262C9	CK5	116	26314	CK5&Dispatch	117
262AB	CK5NOLASTWD	116	1CD006	^ckaddt*	195
1CE006	^ckaddt+	195	1CF006	^ckaddt-	195
181006	^CKALG	118	15A006	^CKCARCOMP	56
2F321	CkChr00	47	18F006	^CKFPOLYext	179
2F324	CKGROBFITS	65	158006	^CKINNERCOMP	56
19C006	^CKINT>0	183	521006	^CKLN	189
177006	^CKMATRIXELEM	179	262CE	CKN	116
25F25	CKNNOLASTWD	116	172006	^CKNUMARRY	53
2BCA2	cknumdsptchl	64	2EF06	CKPICT	174
36B7B	CKREAL	117	25E9F	CKREF	103
16F006	^CKSAMESIZE	185	3D2B4	CKSYMBTYPE	117
193006	^CKSYMREALCMP	119	261C0	CLCD10	157
19E006	^CLEANIDLAM	179	261C5	CLEARLCD	157
2F15B	CLEARMENU	165	2F2DC	ClearSelection	170
26021	CLEARV DISP	157	2A085	(Clipboard!)	171
2A0A5	(Clipboard0)	171	2A0B5	(Clipboard?)	171
2A095	(Clipboard@)	171	2EED7	CLKTICKS	104
2EEC9	CLOSEUART	108	2F15E	Clr16	157
2EED4	Clr8	157	2EED5	Clr8-15	157
2571E	ClrAlgEntry	158	26035	ClrAlphaAnn	158
2569A	ClrAppMode	130	25676	ClrAppSuspOK	131
2649F	(ClrBusyAnn)	158	09D007	^CLRCOMPLEX	214
2EE74	ClrDA1Bad	156	2EE7D	ClrDA1IsStat	155
2EE8D	ClrDA1OK	156	2EE75	ClrDA2aBad	156
2EE8E	ClrDA2aOK	156	2EEB3	ClrDA2bBad	157
2EE80	ClrDA2bIsEdL	157	2EE81	ClrDA2bNoCh	157
2EE8F	ClrDA2bOK	156	2EEA7	ClrDA2bTemp	156
2EE90	ClrDA2OK	156	2EEB5	ClrDA3Bad	157
2EE6E	ClrDA3OK	156	2EE6D	ClrDAsOK	156
0A1007	^CLREXACT	214	2603A	ClrLeftAnn	158
2EEAF	ClrNoRollDA2	157	2561C	(ClrNUsrKeyOK)	122
0AC007	^CLRPLUSAT0	214	2603F	ClrRightAnn	158
26044	ClrSysFlag	106	26049	ClrUserFlag	106
2EF7B	CMD_BAK	170	2EF8A	CMD_COPY	170
2F2FA	CMD_COPY.SBR	170	2EF88	CMD_CUT	170
2EF7D	CMD_DEB_LINE	170	2EF82	CMD_DEL	169
2EF80	CMD_DOWN	170	2EF81	CMD_DROP	169
2EF7E	CMD_END_LINE	170	2EF7C	CMD_NXT	170
2EF7A	CMD_PAGED	170	2EF77	CMD_PAGEL	170
2EF78	CMD_PAGER	170	2EF79	CMD_PAGEU	170
2EF74	CMD_PLUS	169	2F194	CMD_PLUS2	169
2F195	CMD_PLUS3	169	2EF83	CMD_STO_DEBUT	170
2EF84	CMD_STO_FIN	170	2EF7F	CMD_UP	170
2F326	CMDSTO	173	2B6006	^CMODext	223
3311B	cmp	17	526006	^CMPLXLN	189
518006	^CNORMext	182	25EA9	CodePl>%rc.p	121
262F1	COERCE	26	25EA0	COERCE\$22	45
2F244	(COERCE&CKSGN)	26	3F481	COERCE2	26
0F0006	^COERCE2Z	26	35D08	COERCEDUP	26
2602B	COERCEFLAG	87	35EB6	COERCESWAP	26
06FD1	COLA	85	34AD3	COLA_EVAL	85
359AD	COLAcase	90	35994	COLACOLA	85

地址	名称	页码	地址	名称	页码
36865	COLAITE	89	359C8	COLANOTcase	90
34AF4	COLARPITE	89	363FB	COLASKIP	85
26E006	^COLCext	192	0BE006	^CombineFac	202
0C0006	^CombInit	202	0C1006	^CombNext	202
0BF006	^CombProd	202	2EF9A	CommandLineHeight	172
262FB	COMPEVAL	84	09B007	^COMPLEX?	214
58B006	^COMPLEXERR	213	099007	^COMPLEXMODE	214
096007	^COMPLEXOFF	214	095007	^COMPLEXON	214
275006	^COMPLISText	204	2FD006	^COMPRIMext	56
3396D	Connecting	24	0FF006	^Contains?	88
2CB006	^CONTAINS_LN?	223	08D08	CONTEXT!	102
08D5A	CONTEXT@	102	204006	^CONVBACK2INT	180
205006	^CONVBACKINT	180	42B006	^cos*tan	198
447006	^cos2exp	198	446006	^COS2EXPext	191
42C006	^COS2ext	191	412006	^COS2TAN	191
413006	^cos2tan	197	408006	^COS2TAN/2	191
409006	^cos2tan/2	197	309006	^COSEXP	192
30D006	^COSEXP*	199	30E006	^COSEXP*1	199
30B006	^COSEXP+	199	30C006	^COSEXP-	199
44B006	^cosh2exp	198	44A006	^COSH2EXPext	191
08696	CREATE	102	25EA1	CREATEDIR	102
27195	CRLF\$	42	27D3F	CROSSGROB	65
25EA2	CRUNCH	63	4D5006	^CSQFFext	182
3EA01	(CST)	165	0FF007	^CSTFRACTION?	209
22E006	^CURL	218	26585	CURSOR@	168
2EEEE	CURSOR_END?	168	26030	CURSOR_OFF	168
26594	(CURSOR_PART)	168	15D006	^CXRIext	182
50C006	^CZABS	37	2EEB0	DA1Bad?	156
2EEAB	DA1IsStatus?	157	2EE62	DA1OK?	156
2BF3A	DA1OK?NOTIT	156	2EEB1	DA2aBad?	157
2EE66	DA2aLess1OK?	156	2BF53	DA2aOK?NOTIT	156
2EEB2	DA2bBad?	157	2EE7E	DA2bIsEdL?	157
2EEB7	DA2bNoCh?	157	2BF6C	DA2bOK?NOTIT	156
2EEB4	DA3Bad?	157	2EE63	DA3OK?	156
2BF85	DA3OK?NOTIT	156	36E07	dARRYcase	92
2EED0	DATE	104	2EED2	DATE+DAYS	104
2F329	Date>d\$	104	2F1AB	Date>hxs13	104
2F190	DcompWidth@	46	2EED1	DDAYS	104
0B1006	^DeCntMulti	201	2A893	Decomp#Disp	46
2A878	Decomp#Line	46	25EAA	DECOMP\$	47
2F1BF	Decomp%Short	47	2A842	DecompLine	46
2A8AE	DecompEcho	47	2A85D	DecompEdit	47
2A8C9	DecompStd1Line	46	2A8E4	DecompStd1Line32	46
041ED	DEEPSLEEP	107	288006	^DEG1	204
285006	^DEG2ext	204	3E0006	^DEGREext	202
2EF95	DEL_CMD	169	0F3007	^DELTAPSOLVE	205
0BA006	^DemonicLf	201	2A5006	^DENOLCMext	193
0314C	DEPTH	72	4F2006	^DEPTHext	211
4F3006	^DEPTHOBJext	211	1B3006	^DERARG	207
1A5006	^DERIV	207	1A1006	^DERIVext	206
1A3006	^DERIVIDNT	206	1A4006	^DERIVIDNT1	206
1DD006	^DERVX	217	07E007	^DESOLVE	208
2F7006	^deuxipi	222	36E93	dIDNTNcase	92
16E006	^DIMLIMITS	52	345006	^DIMRANM	184
25EBC	Disp5x7	159	25FB8	DISP@01	159
25FBD	DISP@09	159	25FC2	DISP@17	159
25FC7	DISP@25	159	2EF6F	DispBadToken	46
2F19E	DispCommandLine	155, 172	2EEFF	DispCoord1	159
2F32B	DISPCOORD2	160	2EE5A	DispEditLine	155
2F300	DispILPrompt	156	25FE5	DISPLASTROW	160
25FEA	DISPLASTROWBUT1	160	0C0007	^DISPLAYext	68
2DFE0	DispMenu	155, 164	2DFF4	DispMenu.1	155, 164
25FB3	DISPN	159	25FB8	DISPROW1	159
25EAB	DISPROW1*	159	25FE0	DISPROW10	159
25FBD	DISPROW2	159	25EAC	DISPROW2*	159
25FC2	DISPROW3	159	25FC7	DISPROW4	159
25FCC	DISPROW5	159	261F7	DISPROW6	159
25FD1	DISPROW7	159	25FD6	DISPROW8	159
25FDB	DISPROW9	159	38C00	(DISPST2&FREEZE)	159
2C305	DispStatus	156	25EAD	DISPSTATUS2	159
2C2F9	DispStsBound	156	2A7F7	DispTimeReq?	156
255B5	Distance	67	108006	^DISTDIVext	200
118007	^DISTRIB*	199	3C2006	^DISTRIB/	199
2F21A	Dither	67	3EF006	^DIV2LISText	200
22F006	^DIVERGENCE	218	220006	^DIVIS	218
27A006	^DIVISext	204	3F4006	^DIVISIBLE?	193

地址	名称	页码	地址	名称	页码
3F1006	^DIVMETAOBJ	64, 196	3F0006	^DIVOBJext	222
36E43	dLISTcase	92	073F7	DO	95
25EAE	DO#EXIT	97	25EAF	DO\$EXIT	97
25EB0	DO%EXIT	97	25EBE	Do1st/2nd+:	158
2F2F0	DO<Del	169	2F2EE	DO<Skip	170
2F2E4	DO>BEG	170	2F2F1	DO>Del	169
2F2E5	DO>END	170	2EF04	DO>LCD	66
2F2EF	DO>Skip	170	25EB1	DO>STR	47
1A7006	^DO>STRID	47	02E002	^DoAlert	160
25EBF	DoBadKey	122	2EEC1	DOBAUD	108
25EB2	DOBEEP	107	2EFA6	DOBIN	106
074E4	DOBIND	77	2EEC5	DOBUFLen	108
2EF02	DOC>PX	175	25EB3	DOCHR	44
09E002	^DoCKeyCancel	141	0A0002	^DoCKeyChAll	141
09F002	^DoCKeyCheck	141	09D002	^DoCKeyOK	141
0B0002	^DoCKeyUnChAll	141	2EF05	DOCLLCD	157
2EECB	DOCR	108	2EFA8	DODEC	106
2F2F9	DODEL.L	169	2EECD	DODELAY	108
25EC1	DoDelim	169	25EC2	DoDelims	169
25EB4	DODISP	159	2604E	DOENG	107
2EEF9	DOERASE	155	2F2DD	DoFarBS	169
2F2DE	DoFarDel	169	2F2E8	DOFIND	171
2EEBD	DOFINISH	108	25EC3	DoFirstRow	165
26053	DOFIX	107	2EF60	DOGRAPHIC	175
25EC4	DoHere:	102	2EFA5	DOHEX	106
2C371	DoInputForm	147	2EEC4	DOKERRM	108
25EC5	DoKeyOb	122	25886	DoLabel	68, 164
2EF03	DOLCD>	66	0B2006	^DoLS	201
25EC6	DoMenuKey	164	2589A	DoMenuKeyNS	165
2F2EA	DONEXT	171	2EFA7	DOOCT	106
2EEC2	DOPARITY	108	2EEBE	DOPKT	108
2EF01	DOPX>C	175	30A66	DORANDOMIZE	34
25EB5	DORCLE	175	2F2E9	DOREPL	171
2F2EB	DOREPLACE	171	2F2EC	DOREPLACE/NEXT	171
2AC0E	DoRunSafe	106	2EEC6	DOSBRK	108
26058	DOSCI	107	2EEC7	DOSRECV	108
2605D	DOSTD	107	2565A	DoStdKeys?	130
3B76C	(DOSTOALLF2)	106	25EB6	DOSTOE	175
2F23E	DOSTOSYSF	106	25EB7	DOSTR>	46
2EFAA	dostws	48	2F2DB	DOTEXTINFO	172
2EEC3	DOTRANSIO	108	25EB8	DOTVARS%	102
0BD002	^DOTVARS{}	102	25EB9	DOVARS	102
2F2D4	dowait	104	2EE60	DoWarning	160
25EBA	DPRADIX?	107	2F32C	DRAWBOX#	66
2F13F	DRAWLINE#3	66	36EA7	dREALNcase	92
03244	DROP	71	3683D	DROP#1-	27
282CC	(DROP%0)	32	36996	DROP'	84
2F1C6	DROP3PICK	74	28ACE	(DROP?symcomp)	63
25ECD	DropBadKey	122	2F32E	DROPDEADTRUE	84
357CE	DROPDUP	71	35289	DROFFALSE	88
364AF	DROPLOOP	95	37032	DROPNDROP	58, 71
04D3E	DROPNULL\$	44	3596D	DROPONE	26
3606B	DROPOVER	71	36342	DROPRDROP	84
36007	DROPROT	71	35733	DROPSWAP	71
3574D	DROPSWAPDROP	71, 72	35280	DROPTTRUE	88
26215	DropVStack	100	2E0006	^DROPZ0	180
2DF006	^DROPZ1	180	3558C	DROPZERO	26
35136	DTYPEARRY?	117	35172	DTYPECOL?	118
35109	DTYPECSTR?	117	115007	^DTYPEGAUSSINT?	118, 182
168006	^DTYPEIRRQ?	179, 222	35190	DTYPELIST?	117
170006	^DTYPENDO?	185	35118	DTYPEREAL?	117
171006	^DTYPFMAT?	118	34EBE	DUMP	77
03188	DUP	71	3532B	DUP#0<>	29
36441	DUP#0<>WHILE	95	352BD	DUP#0=	29
348F7	DUP#0=case	90	36D21	DUP#0=csDROP	90
3490E	DUP#0=csedrp	90	36ED4	DUP#0=IT	90
36F51	DUP#0=ITE	90	364C8	DUP#0_DO	95
35912	DUP#1+	27	34431	DUP#1+PICK	58, 71
35956	DUP#1-	27	3531C	DUP#1=	29
3571E	DUP#2+	27	366D0	DUP#<7	29
362DE	DUP\$>ID	77	36C0E	DUP%0=	35
3696E	DUP'	84	36513	DUP1LAMBIND	77
35D30	DUP3PICK	71	3674D	DUP3PICK#+	27
34797	DUP4PUTLAM	79	3432C	DUP4UNROLL	71
35C2C	DUP@	101	159006	^DUPCKLEN{}	56
35CE0	DUPDUP	71	36635	DUPEQ:	88

地址	名称	页码	地址	名称	页码
25EBB	DUPGROBDIM	65	3622A	DUPINCOMP	56
3645A	DUPINDEX@	95	357E2	DUPLen\$	44
3627A	DUPLenCOMP	54	36252	DUPNULL\$?	47
16C006	^DUPNULL[]?	185	36266	DUPNULLCOMP?	54
36ABD	DUPNULL{}?	56	36AEA	DUPONE	26
3611F	DUPPICK	71	0F7006	^DupQIsZero?	182
36133	DUPROLL	71	34FC0	DUPROM-WORD?	69
35C40	DUPROMPTR@	69	35FF3	DUPROT	71
35A56	DUPSAFE@	101	347AB	DUPTEMPENV	79
36B26	DUPTWO	26	350CD	DUPTYPEAPLET?	118
35136	DUPTYPEARRY?	117	350EB	DUPTYPEBINT?	118
35037	DUPTYPECHAR?	118	35127	DUPTYPECMP?	117
35172	DUPTYPECOL?	118	35109	DUPTYPECSTR?	117
351AE	DUPTYPEEXT?	118	35082	DUPTYPEFLASHPTR?	118
350BE	DUPTYPEFONT?	118	116007	^DUPTYPEGAUSSINT?	118, 182
35181	DUPTYPEGROB?	118	350FA	DUPTYPEHSTR?	118
35046	DUPTYPEIDNT?	117	350DC	DUPTYPELAM?	117
35190	DUPTYPELIST?	117	350AF	DUPTYPELNGCMP?	118
350A0	DUPTYPELNGREAL?	118	35118	DUPTYPEREAL?	117
35145	DUPTYPEROMP?	118	35154	DUPTYPERRR?	118
0CC006	^DupTypeS?	183	35163	DUPTYPESYMB?	118
3519F	DUPTYPETAG?	118	183006	^DUPTYPEZ?	118
35091	DUPTYPEZINT?	118	3457F	DUPUNROT	71, 72
36AD6	DUPZERO	26	10A006	^DupZIsEven?	182
0FB006	^DupZIsNeg?	182	0F9006	^DupZIsOne?	182
109006	^DupZIsTwo?	182	36A77	dvarlsBIND	77
18C006	^E%%>C%%	37	2EEE4	Echo\$Key	169
2F11C	Echo\$NoChr00	169	25F10	ederr	98
25ECE	EDITDECOMP\$	47	25F11	editdecomp\$w	47
36E57	EditExstCase	92	2EEE5	EditLevel1	172
257A2	EditLExists?	168	2EEEB	EDITLINE\$	168
25ED2	EditMenu	172	2F2DF	EditSelect	171
2EEE9	EditString	172	3D8006	^EGCDext	203
33157	EIGHT	17	331BB	EIGHTEEN	18
342BB	EIGHTROLL	73	33427	EIGHTY	19
33431	EIGHTYONE	19	33175	ELEVEN	17
4F9006	^ELMGext	211	371B3	Embedded?	55
03B2E	EQ	88	3663A	EQ:	88
34999	EQcase	91	34920	EQcasedrop	91
36EBB	EQIT	91	36F01	EQITE	91
25ED3	EqList?	56	37829	EQLookup	55
3664E	EQOR	88	3607F	EQOVER	88
03B97	EQUAL	88	36D62	EQUALcase	92
2AD81	EQUALcasedrop	92	36D08	EQUALcaserp	92
3660D	EQUALNOT	88	36E7F	EQUALNOTcase	92
36662	EQUALOR	88	3C0006	^EQUALPOS2META	59
3776B	EQUALPOSCOMP	55	3BF006	^EQUALPOSMETA	59
25ECF	EQUATION	175	462006	^EQUATION?	63
4BF006	^EQUIV!	206	011004	^EQW3Edit	172
019004	^EQW3GROB	68	01F004	^EQW3GROBmini	68
01A004	^EQW3GROBstk	68	01E004	^EQW3GROBsys	68
57E006	^ERABLEERROR	213	26071	ERASE&LEFT\$3x5	68
2606C	ERASE&LEFT\$5x7	68	3376F	(Err#Kill)	21
33819	(Err#NoLstArg)	22	33779	(Err#NoLstStk)	21
58E006	^ERR\$EVALext	213	092006	^ErrBadDim	213
26067	ERRBEEP	97	090006	^ErrInfRes	213
04ED1	ERRJMP	97	04CE6	ERROR@	97
04D33	ERRORCLR	97	36883	ERROROUT	97
04D0E	ERRORSTO	97	04E5E	ERRSET	97
04EB8	ERRTRAP	97	091006	^ErrUndefRes	213
214006	^EULER	218	06F8E	EVAL	83
2F2E3	EVAL.LINE	171	2F2FB	EVAL.SELECTION	171
26319	EvalNoCK	117	25F29	(EvalNoCK:)	117
257006	^EvalNoCKx*	220	258006	^EvalNoCKx+	220
259006	^EvalNoCKx-	220	25A006	^EvalNoCKx/	220
262006	^EvalNoCKxAND	220	25C006	^EvalNoCKxCHS	220
260006	^EvalNoCKxCOMB	220	25D006	^EvalNoCKxINV	220
25E006	^EvalNoCKxMOD	220	261006	^EvalNoCKxOR	220
25F006	^EvalNoCKxPERM	220	263006	^EvalNoCKxXOR	220
264006	^EvalNoCKxXROOT	220	25B006	^EvalNoCKx^	220
2A3006	^EVALUSERFCN	192	283006	^EVIDENText	204
284006	^EVIDSOLV	204	0A2007	^EXACT?	214
09E007	^EXACTMODE	214	098007	^EXACTOFF	214
097007	^EXACTON	214	2F2F8	EXEC_CMD	171
258EF	(ExitAction!)	164	3709B	ExitAtLOOP	95
04E37	EXITMSGSTO	97	240006	^EXLR	193

地址	名称	页码	地址	名称	页码
458006	^exp2sincos	198	34C82	EXPAND	45, 48
525006	^EXPANDLN	189	12C006	^EXPAND^	189
30F006	^EXPEXPA	193	313006	^EXPEXPA*	199
315006	^EXPEXPA*1	199	311006	^EXPEXPA+	199
312006	^EXPEXPA-	199	314006	^EXPEXPANEG	199
418006	^EXPLNext	193	41D006	^EXPM2EXP	191
33193	EXT	17	05481	EXTN	55, 60
3398B	EXTOBOB	24	336F7	EXTREAL	21
33701	EXTSYM	21	27B006	^FACT1ext	204
27C006	^FACTOext	204	296006	^FACTOOBJext	205
28C006	^FACTORACext	204	572006	^factorial	190
21F006	^FACTORS	218	573006	^facts	190
576006	^factzint	181	27E9B	failed	88
03AC0	FALSE	87	369FF	FALSE'	84
283E8	FalseFalse	87	36554	FALSETRUE	87
36554	FalseTrue	87	3F6006	^FastDiv?	193
09A006	^FASTREDUCE	205	255DD	FBoxB	67
255D3	FBoxG1	67	255D8	FBoxG2	67
255D3	FBoxW	67	255E2	FBoxXor	67
21D006	^FCOEF	218	289006	^FDEG2ext	204
282006	^FEVIDENText	202	3E1006	^FHORNER	203
3319D	FIFTEEN	18	332FB	FIFTY	18
3334B	FIFTYEIGHT	19	3332D	FIFTYFIVE	19
33323	FIFTYFOUR	19	33355	FIFTYNINE	19
33305	FIFTYONE	19	33341	FIFTYSEVEN	19
33337	FIFTYSIX	19	33319	FIFTYTHREE	19
3330F	FIFTYTWO	19	067004	^Filer	112
06D004	^FILER_MANAGER	112	06E004	^FILER_MANAGERTYPE	112
37798	Find1stTrue	55	4C2006	^FindCurVar	206
35A006	^FINDELN	187	2F2F2	FindStrInCmd	171
2F110	(FINDVARS)	63	264CC	FIRSTC@	168
33139	FIVE	17	3344F	FIVEFOUR	19
34257	FIVEROLL	72	33463	FIVESIX	19
33445	FIVETHREE	19	34357	FIVEUNROLL	73
201006	^FLAGACOS2S	218	200006	^FLAGASIN2C	218
1FF006	^FLAGASIN2T	218	1FE006	^FLAGATAN2S	218
225006	^FLAGAXQ	188	216006	^FLAGCHINREM	218
1E9006	^FLAGDERIV	217	1EC006	^FLAGDESOLVE	217
207006	^FLAGDIV2	218	239006	^FLAGDIVPC	219
1D6006	^FLAGEXPAND	217	1F3006	^FLAGEXPLN	217
1D8006	^FLAGFACTOR	217	252006	^FLAGFACTORMOD	210
226006	^FLAGGAUSS	188	208006	^FLAGGCD	218
1FC006	^FLAGHALFTAN	218	213006	^FLAGHORNER	218
1E5006	^FLAGIBP	217	1DB006	^FLAGIDNTEXEC	217
1EB006	^FLAGILAP	217	1DC006	^FLAGINTVX	217
22D006	^FLAGJORDAN	188	1EA006	^FLAGLAP	217
20E006	^FLAGLCM	218	1ED006	^FLAGLDSSOLV	217
20D006	^FLAGLGCD	218	1F0006	^FLAGLIN	217
1D9006	^FLAGLISTEXEC	217	1F2006	^FLAGLNCOLLECT	217
1E0006	^FLAGMATRIXLIMIT	217	094007	^FLAGNAME	214
210006	^FLAGPARTFRAC	218	1E6006	^FLAGPREVAL	217
211006	^FLAGPROPFRAC	218	212006	^FLAGPTAYL	218
224006	^FLAGQXA	188	10E007	^FLAGRESULTANT	203
1E8006	^FLAGRISCH	217	1E2006	^FLAGSERIES	217
23B006	^FLAGSEVAL	219	20F006	^FLAGSIMP2	218
1F4006	^FLAGSINCOS	217	227006	^FLAGSYLVESTER	188
1DA006	^FLAGSYMBEXEC	217	1FB006	^FLAGTAN2SC	218
1FD006	^FLAGTAN2SC2	218	1F6006	^FLAGTCOLLECT	217
1EF006	^FLAGTEXPAND	217	1F5006	^FLAGTLIN	217
1F7006	^FLAGTRIG	217	1F8006	^FLAGTRIGCOS	217
1F9006	^FLAGTRIGSIN	217	1FA006	^FLAGTRIGTAN	217
23A006	^FLAGTRUNC	219	1F1006	^FLAGTSIMP	217
25ED4	FlashMsg	160	2EE61	FlashWarning	160
192006	^FLOAT?	119	261CA	FLUSH	121
261CA	FLUSHKEYS	121	2621A	FONT>	160
33297	FORTY	18	332E7	FORTYEIGHT	18
332C9	FORTYFIVE	18	332BF	FORTYFOUR	18
332F1	FORTYNINE	18	332A1	FORTYONE	18
332DD	FORTYSEVEN	18	332D3	FORTYSIX	18
332B5	FORTYTHREE	18	332AB	FORTYTWO	18
3312F	FOUR	17	333B9	FOURFIVE	19
236006	^FOURIER	219	2D4006	^FOURIERext	223
3423A	FOURROLL	72	36043	FOURROLLROT	72
33193	FOURTEEN	17	333A5	FOURTHREE	19
3339B	FOUR TWO	19	33297	FOURTY	18
34331	FOURUNROLL	73	0D5007	^FR2ND%	224

地址	名称	页码	地址	名称	页码
0CB007	^FRACPARITY	224	27F006	^FRND	204
267006	^FROMLISText	220	21E006	^FROOTS	218
001004	^FSTR1	47	00D004	^FSTR13	47
003004	^FSTR3	47	004004	^FSTR4	47
005004	^FSTR5	47	006004	^FSTR6	47
007004	^FSTR7	47	009004	^FSTR9	47
0FE007	^FTAYL	209	0AE007	^FULLDATA	214
164006	^FXNDext	203	05F42	GARBAGE	107
383006	^GAUSS	188	095006	^GBASIS	205
26076	GBUFF	155	25ED5	GBUFFGROBDIM	155
0EC006	^GCDHEUext	211	2F105	GDISPCENTER	175
2F2F3	GET.W->	171	2F2F4	GET.W<-	171
34504	get1	58	2F2F6	GET_CUR_FONT.EXT	172
0371D	GETATELN	52	04A41	GETDF	163
26224	GetElemBotVStack	100	26229	GetElemTopVStack	100
2F339	GetEqN	175	57D006	^GETERABLEMSG	213
04E07	GETEXITMSG	97	26242	GetFontStkHeight	160
26247	GetHeader	155	2F106	GETINDEP	174
2EEBF	GetIOPAR	108	25ED6	GETKEY	121
25ED7	GETKEY*	121	25ED9	GetKeyOb	122
075A5	GETLAM	78	3483E	GETLAMPAIR	79
2F33C	getmatchtok	46	25EDA	GetMenu%	163
2624C	GetMetaVStack	100	25F17	GetMetaVStackDROP	99
2EEBC	GETNAME	108	25EDB	GetNextToken	46
2F33D	GETPARAM	174	2F107	GETPMIN&MAX	174
04A0B	GETPROC	163	2EEF5	GETPTYPE	174
2F10D	GETRES	175	07A007	^GetRoot	221
2F33E	GETSCALE	175	04D64	GETTHEMESG	97
04714	GETTOUCH	121	25967	GetUserKeys	122
25F18	GetVStack	99	26233	GetVStackProtectWo rd	100
2F0FE	GETXMAX	174	2F0FF	GETXMIN	174
2F10E	GETYMAX	174	2F100	GETYMIN	174
097006	^GFACTOR	205	094006	^GMSOLV	205
25588	Gor	67	107007	^GOSPER?	224
34A31	GOTO	84	2F2E6	GOTOLABEL	170
255A1	Grey?	67	3317F	grob	17
2607B	GROB!	65	26080	GROB!ZERO	65
368E7	GROB!ZERODRP	65	2F342	GROB+#	65
2EFDB	(GROB+)	65	25ED8	GROB>GDISP	65
2E0D5	Grob>Menu	67, 164	0BF007	^GROBADDext	66
26085	GROBDIM	65	36C68	GROBDIMw	65
096006	^GSOLVE	205	2558D	Gxor	67
25EDC	H/W>KeyCode	121	0ED006	^H>Z	180
407006	^HALFTAN	193	2608F	HARDBUFF	155
26094	HARDBUFF2	155	25EDE	HARDHEIGHT	155
2EED6	HBUFF_X_Y	158	26099	HEIGHTENGROB	155
232006	^HERMITE	218	231006	^HESSIAN	218
22C006	^HILBERTNOCK	218	08D92	HOMEDIR	102
2C0006	^HORNER1ext	223	4A1006	^HORNEXP!	206
3E2006	^HORNext	203	3FB006	^HSECO2RCext	222
33175	hxs	17	2F0EE	HXS#HXS	49
2F0EF	HXS<=HXS	49	2EFCF	HXS<HXS	49
2EFCC	HXS==HXS	49	05A03	HXS>#	26
2EFC0	HXS>\$	44	2EFC1	hxs>\$	44
2EFCA	HXS>%	32	2EFCE	HXS>=HXS	49
2EFCD	HXS>HXS	49	336C5	(HXSREAL)	21
417006	^HYP2EXPext	193	104007	^HYPERGEO	209
20C006	^IABCUV	218	10D007	^IBERNOULLI	209
2C5006	^IBP	208	217006	^ICHINREM	218
2F0EC	ICMPDRPRTDRP	56	33143	id	17
05BE9	ID>\$	44	2E11B	Id>Menu	67, 164
05F2E	ID>TAG	51	272F3	(ID_EQ)	77
27937	(ID_SIGMADAT)	77	33887	(IDLISTOB)	23
2B0CC	idntcase	92	45C006	^IDNTEXEC	223
191006	^IDNTLAM?	118	2B0EF	idntlamcase	92
334D1	IDREAL	20	3387D	(IDREALOB)	23
0716B	IDUP	83, 94	3D6006	^IEGCDext	181
034004	^IfCheckFieldType	148	033004	^IfCheckSetValue	148
031004	^IfDisplayFromData	148	03D004	^IfEnterKeyPress	148
027004	^IfGetCurrentField Value	148	02C004	^IfGetFieldChoosed ata	148
02D004	^IfGetFieldChoosed ecomp	148	02B004	^IfGetFieldDecompO bject	148
030004	^IfGetFieldInterna lValue	148	028004	^IfGetFieldMessage Handler	148

地址	名称	页码	地址	名称	页码
02A004	^IfGetFieldObjects Type	148	045004	^IfGetFieldPos	149
02E004	^IfGetFieldResetVa lue	148	029004	^IfGetFieldType	148
026004	^IfGetFieldValue	148	032004	^IfGetNbFields	148
04C004	^IfGetPrlgFromType s	148	04A004	^IfInitDepth	148
03A004	^IfKeyCalc	148	037004	^IfKeyChoose	148
038004	^IfKeyEdit	148	03B004	^IfKeyInvertCheck	148
039004	^IfKeyTypes	148	020004	^IfMain	147
042004	^IfMain2	148	03C004	^IfONKeyPress	148
043004	^IfPutFieldsOnStac k	148	035004	^IfReset	148
048004	^IfSetAllHelpStrin gs	149	047004	^IfSetAllLabelsMes sages	149
025004	^IfSetCurrentField Value	148	036004	^IfSetField	148
044004	^IfSetFieldPos	149	02F004	^IfSetFieldResetVa lue	148
024004	^IfSetFieldValue	147	021004	^IfSetFieldVisible	147
023004	^IfSetGrob	147	03F004	^IfSetHelpString	148
022004	^IfSetSelected	147	040004	^IfSetTitle	148
08B007	^ILAPEXP	208	088007	^ILAPext	208
2F0D4	(ILnot?)	117	25EDF	ImmedEntry?	158
35BAF	INCOMPDROP	56	2F0E8	INDEPVAR	174
07221	INDEX@	95	367D9	INDEX@#-	95
07270	INDEXSTO	95	3D9006	^INEGCD	181
2E2006	^INFINText	221	394C8	INHARDROM?	107
2EEE6	InitEd&Modes	173	2EEE7	InitEdLine	169, 173
2EEE8	InitEdModes	173	25EE0	InitMenu	164
25EE1	InitMenu%	164	25EE2	InitTrack:	163
366E9	INNER#1=	56	054AF	INNERCOMP	56
3BADA	(INNERCOMP>%)	56	35C68	INNERDUP	56
3D3006	^INPARTFRAC	208	2F154	(input\$)	124
2EF5F	InputLine	124	2F155	(input{})	124
25790	INSERT?	169	366006	^INSERT[]COL[]	187
365006	^INSERT[]ROW[]	187	25795	INSERT_MODE	169
364006	^INSERTCOL[]	187	2EF97	InsertEcho	169
362006	^INSERTROW[]	187	363006	^insertrow[]	187
2FC006	^INSERT{}N	56	2D3006	^INT3	208
33805	INTEGER337	22	3DD006	^INTEGRext	208
06B4E	INTEMNOTREF?	103	582006	^INTERNALERR	213
2D2006	^INText	208	586006	^INTVARERR	213
402006	^INVAL2	223	583006	^INVALIDOP	213
2609E	INVGROB	66	2E25C	(InvLabelGrob)	65
350006	^INXREdext	186	2EF006	^ipi	221
113007	^IROOTS	204	2D8006	^IRRQ#ULTIMATE	222
15F006	^IRXCext	37	2C7006	^IS_SQRT?	223
2C9006	^IS_XROOT?	223	0C8007	^ISIDREAL?	215
4C5006	^ISOL1	210	4C7006	^ISOL2ext	210
4C6006	^ISOLALL	210	584006	^ISOLERR	213
4E0006	^ISPOLYNOMIAL?	210	218006	^ISPRIME	217
283FC	ISTOP-INDEX	95	07249	ISTOP@	95
07295	ISTOPSTO	95	04D004	^IsUncompressDataS tring	149
0E9006	^IsV>V?	211	34A22	IT	89
34B3E	ITE	89	34ABE	ITE_DROP	89
36DDF	j%0=case	91	36D3A	jEQcase	91
07258	JINDEX@	95	072AD	JINDEXSTO	95
380006	^JORDAN	188	04D87	JstGetTHEMESG	97
04D87	JstGetTHEMSG	97	07264	JSTOP@	95
072C2	JSTOPSTO	95	2F347	JUMPBOT	159
2F348	JUMPLEFT	159	2F349	JUMPRIGHT	159
2F34A	JUMPTOP	159	25EE4	KeepUnit	61
25EE5	Key>StdKeyOb	122	25EE6	Key>U/SKeyOb	122
255006	^KEYEVAL	122	25EE3	KEYINBUFFER?	122
260A3	KILLGDISP	155	2F34D	KINVISLF	108
2F34E	KVIS	108	2F34F	KVISLF	108
361006	^la+ELEMsym	187	25877	LabelDef!	163
235006	^LAGRANGE	219	3E6006	^LAGRANGEext	203
3314D	lam	17	25F95	LANGUAGE>	107
087007	^LAPext	208	230006	^LAPLACIAN	218
362B6	LAST\$	45	2BC006	^LASTCOMP	54
25908	LastMenuDef!	163	2590D	LastMenuDef@	163
260A8	LastMenuRow!	163	260AD	LastMenuRow@	163
25EE7	LastNonNull	102	08326	LASTRAM~WORD	102

地址	名称	页码	地址	名称	页码
255F6	LBoxB	67	255EC	LBoxG1	67
255F1	LBoxG2	67	255E7	LBoxW	67
255FB	LBoxXor	67	375006	^LCFROG2M	184
081007	^LDECSOLV	208	082007	^LDEGENE	208
083007	^LDEPART	208	084007	^LDSSOLVext	208
0B1002	^LEDispItem	141	0B2002	^LEDispList	141
0B3002	^LEDispPrompt	141	25FF9	LEFT\$3x5	68
26008	LEFT\$3x5Arrow	68	2601C	LEFT\$3x5CR	68
26012	LEFT\$3x5CRArrow	68	25FFE	LEFT\$5x7	68
26003	LEFT\$5x7Arrow	68	26017	LEFT\$5x7CR	68
2600D	LEFT\$5x7CRArrow	68	2F352	LEFTCOL	158
234006	^LEGENDRE	219	05636	LEN\$	44
0567B	LENCOMP	54	05616	LENHXS	48
17B006	^LENMATRIX	187	2D9006	^LESSCOMPLEX?	223
0B6006	^LFCProd	201	4DE006	^LIDNText	210
4EC006	^LIDNTLVAR	211	256006	^LIFCext	210
0BC006	^LiftGeneral	202	0BB006	^LiftLinear	201
0B5006	^LiftZAdic	201	49E006	^LIM#VARX!	206
490006	^LIM%#!	206	483006	^LIM+~!	206
47F006	^LIMCMPL!	206	48C006	^LIMDIVPC!	206
481006	^LIMEQU!	206	480006	^LIMEQUFR!	206
477006	^LIMIT!	206	47C006	^LIMLIM!	206
48E006	^LIMPROFEND!	206	472006	^LIMSERIES!	206
478006	^LIMSTEP1!	206	4BE006	^LIMVAL!	206
4BD006	^LIMVALOBJ!	206	4C3006	^LIMVAR!	206
101007	^LINEARAPPLY	223	2556A	LineB	66
2556F	LineG1	66	25574	LineG2	66
2EFA0	LINEOFF	66	2EFA3	LINEOFF3	66
2EF9F	LINEON	66	2EFA2	LINEON3	66
25565	LineW	66	25579	LineXor	66
31B006	^LINEXPA	193	300006	^LINEXPext	192
080007	^LINSOLV	186	33139	list	17
17A006	^LIST2MATRIX	184	3343B	LISTCMP	19
45F006	^LISTEXEC	222	460006	^LISTEXEC1	222
2DA006	^LISTIRRQ	222	3346D	LISTLAM	19
104006	^LISTMAXext	211	4EE006	^LISTOPext	211
4ED006	^LISTOPRAC	211	4EF006	^LISTOPSQRT	211
0FD006	^ListPos	55	2F24E	LISTRCL	101
33431	LISTREAL	19	26B006	^LISTSECOext	222
001007	^ListToArray	53	4E5006	^LLVARDext	211
45A006	^LN2ATAN	191	456006	^LN2ext	193
527006	^LNATANext	189	31D006	^LNCOLCext	193
316006	^LNEXPA	192	318006	^LNEXPA*	199
319006	^LNEXPA/	199	31A006	^LNEXPA^	199
41A006	^LNP12LN	191	25EE8	LoadTouchTbl	164
25EE9	LockAlpha	158	41B006	^LOG2LN	191
377C5	Lookup	55	377DE	Lookup.1	55
07334	LOOP	95	269006	^LOPlext	222
26A006	^LOPAext	222	3F2006	^LOPDext	222
10F006	^LOPMext	222	01118	LowBat?	107
2B1006	^LPGCDext	194	3DE006	^LRDMext	202
4F0006	^LVARDext	211	4E7006	^LVARExt	211
4C0006	^LVARXNX2!	206	4DF006	^LVARXNXext	210
386006	^m~1&m+1	195	2AE32	M~1stcasechs	91
321006	^MADD	185	242006	^MADDTMOD	219
37E006	^MADJ	188	229006	^MADNOCK	218
376006	^MAKE2DMATRIX	184	377006	^make2dmatrix	184
373006	^MAKEARRY	184	2E189	MakeBoxLabel	67
2E1EB	MakeDirLabel	67	260B2	MAKEGROB	66
2E24D	MakeInvLabel	67	2E2AA	MakeLabel	68, 164
2F0DB	MAKEPICT#	66	2AB006	^MAKEPROFOND	202
2F355	MAKEPVARs	174	2E166	MakeStdLabel	67
357006	^MAKESYSText	186	27D5D	MARKGROB	65
326006	^MAT*	185	32A006	^MAT*SCL	185
320006	^MAT+	185	322006	^MAT~	185
333006	^MAT/	185	332006	^MAT/SCL	185
376B7	matchob?	55	334006	^MATCHS	185
34A006	^MATCNORM	186	372006	^MATCON	184
336006	^MATCONJ	185	32D006	^MATCROSS	185
346006	^MATDET	186	32E006	^MATDOT	185
37F006	^MATEGV	188	37C006	^MATEGVL	188
17D006	^MATEXPLODE	185	348006	^MATFNORM	186
371006	^MATIDN	184	338006	^MATIM	186
34E006	^MATINV	185	34D006	^MATRANK	186
344006	^MATRANM	184	347006	^MATRDET	186
337006	^MATRE	186	341006	^MATREDIM	184

地址	名称	页码	地址	名称	页码
34C006	^MATREF	186	34F006	^MATREFRREF	186
340006	^MATREPL	187	36D006	^MATRIX-COL	187
36B006	^MATRIX-ROW	187	178006	^MATRIX2ARRAY	53
179006	^MATRIX2LIST	185	35F006	^MATRIX>DIAG	187
369006	^MATRIXCSWAP	187	360006	^MATRIXDIAG>	187
174006	^MATRIXDIM	186	367006	^MATRIXRCI	186
368006	^MATRIXRCIJ	186	1E7006	^MATRIXRISCH	217
36A006	^MATRIXRSWAP	187	349006	^MATRNORM	186
34B006	^MATRREF	186	335006	^MATSQUARE	185
33E006	^MATSUB	187	339006	^MATTRACE	186
33B006	^MATTRAN	52	33C006	^mattran	186
33A006	^MATTRN	186	33D006	^mattrn	186
32C006	^MAT^	185	16D006	^MDIMS	52
35FD8	MDIMSDROP	52	05F61	MEM	107
33111	MEMERR	17	0B9007	^MENUARIT1	216
0B5007	^MENUBASE1	216	0B3007	^MENUCHOOSE	216
0B2007	^MENUCHOOSE?	216	0B6007	^MENUCMPLX1	216
25845	MenuDef@	163	0BC007	^MENUDIFF1	216
0BB007	^MENUEXPLN1	216	08D007	^MENUext	216
0B4007	^MENUGENE1	216	275FD	MenuKey	165
2589F	MenuKeyLS!	163	2588B	MenuKeyNS!	163
25890	MenuKeyNS@	163	258B3	MenuKeyRS!	163
27620	MenuMaker	164	0B8007	^MENUMAT1	216
2EEFC	MENUOFF	155	2EEFD	MENUOFF?	155
260B7	MenuRow!	163	260BC	MenuRow@	163
25863	MenuRowAct!	163	0BA007	^MENSOLVE1	216
0B7007	^MENUTRIG1	216	1D1006	^MENUXYext	216
2AFFB	MEQ1stcase	91	2B01B	MEQopscase	91
390006	^meta-1	195	2F1006	^meta-pi	221
2F4006	^meta-pi/2	221	2F5006	^meta-pi/4	221
389006	^meta/2	195	38D006	^metal-sq	195
387006	^metal/meta	195	38C006	^meta2*	195
103007	^meta_cst?	209	2FA006	^meta_e	221
393006	^metaadd	195	3AB006	^MetaAdd	195
3BA006	^metackneg	196	278006	^METACOMP1	204
3BD006	^metaCOMPARE	199	276006	^METACOMPRIM	204
30A006	^METACOSEXPA	199	287006	^METADEG1	204
286006	^METADEG2	204	2A6006	^METADENOLCM	193
1BD006	^METADER&NEG	207	1AB006	^METADER*	207
1A9006	^METADER+	207	1AA006	^METADER-	207
1AC006	^METADER/	207	1B9006	^METADERABS	207
1C9006	^METADERACH	207	1C6006	^METADERACOS	207
1B8006	^METADERALOG	207	1C8006	^METADERASH	207
1C5006	^METADERASIN	207	1C7006	^METADERATAN	207
1CA006	^METADERATH	207	1C0006	^METADERCOS	207
1C3006	^METADERCOSH	207	1AF006	^METADERDER	207
1B4006	^METADEREXP	207	1AE006	^METADERFCN	207
1B1006	^METADERI3	207	1B0006	^METADERI4	207
1B2006	^METADERIFTE	207	1BA006	^METADERINV	207
1A6006	^METADERIV	207	1B5006	^METADERLN	207
1B6006	^METADERLNP1	207	1B7006	^METADERLOG	207
1BB006	^METADERNEG	207	1BF006	^METADERSIN	207
1C2006	^METADERSINH	207	1BE006	^METADERSQ	207
1BC006	^METADERSQRT	207	1C1006	^METADERTAN	207
1C4006	^METADERTANH	207	1AD006	^METADER^	207
396006	^metadiv	196	3B1006	^MetaDiv	196
39F006	^metaEQUAL?	59	310006	^METAEXPEXPA	199
39A006	^metafraction?	199	2ED006	^metai	221
459006	^metai*	195	3A5006	^metainftype	198
198006	^METAINT?	59, 183	2F0006	^metaipi	221
4D8006	^METALISTVXXL	179	317006	^METALNEXPA	199
36C006	^METAMAT-ROW	187	36E006	^METAMATCSWAP	187
351006	^METAMATRED	186	36F006	^METAMATRSWAP	187
274006	^METAMM2	204	3AF006	^MetaMul	195
395006	^metamult	195	3B7006	^MetaNeg	196
3B9006	^metaneg	196	2EB006	^metapi	221
2F8006	^metapi*2	221	2F2006	^metapi/2	221
2F3006	^metapi/4	221	3BC006	^metapi?	199
352006	^METAPIVOT	186	199006	^METAPOSINT?	59, 183
399006	^metapow	196	3B5006	^MetaPow	196
3C3006	^metareal?	199	36FA6	metaROTDUP	58
3BB006	^metasimp	199	304006	^METASINEXPA	199
291006	^METASOLV	205	293006	^METASOLV2	205
294006	^METASOLV4	205	385006	^metasplit	59
38E006	^metasq+1	195	38F006	^metasq-1	195
4D3006	^METASQFFext	210	394006	^metasub	195

地址	名称	页码	地址	名称	页码
3AD006	^MetaSub	195	2F356	metatail	59
31E006	^METATANEXPA	199	3A2006	^metaundef	198
403006	^METAVAL2	223	39B006	^metaxroot	196
397006	^meta^	196	465006	^MEVALext	192
253006	^MFACTORMOD	210	0A9006	^MFactTriv	201
4CA006	^MHORNER	210	4CB006	^MHORNER1	210
3E4006	^MHORNext	203	2B083	Midlstcase	91
2625B	MINIFONT>	160	3A9006	^minusinf	198
2E3006	^MINUSINFext	221	3399F	MINUSONE	25
0A5006	^MKPOLY	202	4D2006	^MLISTSQFF	210
327006	^MMMULT	185	3D0006	^Mod	181
3C5006	^ModAdd	200	3C8006	^ModDiv	200
3C9006	^ModDiv2	200	3C4006	^ModExpa	200
3CB006	^ModGcd	201	275EE	Modifier	164
25EEA	ModifierKey?	121	3CA006	^ModInv	201
3C7006	^ModMul	200	3C6006	^ModSub	200
0B9006	^MonicLf	201	121006	^MP0	223
120006	^MPext	200	4D1006	^MSECOSQFF	224
4CD006	^MSQFF	210	323006	^MSUB	185
244006	^MSUBTMOD	219	31C006	^MTRIG2SYMB	192
272006	^MULMULText	204	245006	^MULTMOD	219
328006	^MVMULT	185	4CF006	^MZSQFF	210
4D0006	^MZSQFF1	210	05459	{ }N	55
35FB0	N+1DROP	58, 72	36B67	NcaseSIZEERR	92
36BAA	NcaseTYPEERR	92	2C2C0	nCustomMenu	165
0D9007	^NDEvalN/D	209	26260	nDISPSTACK	156
0326E	NDROP	58, 72	28211	NDROPFALSE	58, 88
3AA006	^NDROPminusinf	198	3A8006	^NDROPplusinf	198
3B3006	^NDROPZ0	180	3B4006	^NDROPZ1	180
391006	^NDROPZERO	58	031D9	NDUP	71
28143	NDUPN	71	162006	^NDXFext	203
166006	^NDXQext	222	3ED006	^NEWDIVext	193
33B39	NEWLINE\$	42	361DA	NEWLINE\$&\$	44
361DA	NEWLINE&\$	44	091007	^NEWMODULO	214
4F7006	^NEWTRIMext	211	090007	^NEWVX	214
37B54	NEXTCOMPOB	55	25EEB	NEXTLIBBAK	69
2FB006	^NEXTPext	56	0CA006	^NFactor	181
0CB006	^NFactorSpc	181	33161	NINE	17
331C5	NINETEEN	18	2EF07	nmetasyms	117
2EF96	NO_AFFCMD	172	25EED	NoAttn?Semi	122
256BE	NOBLINK	107	25EEE	NoEdit?case	92
2EEED	NoEditLine?	168	25EEF	NoExitAction	164
27E72	nohalt	131	260C1	NOHALTERR	98
119007	^NONALGERR	213	585006	^NONINTERR	213
2ADBD	nonopcase	92	58A006	^NONPOLYSYST	213
100007	^NONRATSUM	209	0A8007	^NONRECMODE	214
581006	^NONUNARYERR	213	25621	(NonUsrKeyOK?)	122
06E8E	NOP	83	0A4007	^NOSTEPBYSTEP	214
03AF2	NOT	88	34BD8	NOT?DROP	88
34A59	NOT?GOTO	84	34A92	NOT?SEMI	88
35F7E	NOT?SWAPDROP	89	0712A	NOT_IT	89
3640F	NOT_UNTIL	94	36428	NOT_WHILE	94
35C7C	NOTAND	88	34A13	NOTcase	89
34976	NOTcase2drop	89	349EA	NOTcase2DROP	89
3494E	NOTcasedrop	89	349C6	NOTcaseDROP	89
2B2C5	NOTcaseFALSE	90	36914	NOTcaseTRUE	89
36B3A	NOTcsdrpfls	89	260C6	NOTLISTcase	92
260CB	NOTROMPcase	92	260D0	NOTSECOcase	92
469006	^NR_REPLACE	192	2BB53	nscknum2	64
35D58	NTHCOMDDUP	54	35BC3	NTHCOMPDROP	54
056B6	NTHELCOMP	54	37784	NTHOF	55
055DF	NULL\$	42	0556F	NULL\$?	47
35D94	NULL\$SWAP	44	25EEC	NULL\$TEMP	44
055FD	NULL::	57	055B7	NULLCOMP?	54
27AA3	(NULLGROB)	65	055D5	NULLHXS	48
272FE	NULLID	77	2B3AB	NULLLAM	77
27FED	NullMenuKey	164	359006	^NULLVECTOR?	185
055E9	NULL{ }	56	29E67	nultrior	46
2B25C	num-1=case	91	2B11C	num0=case	91
2B176	num1=case	91	2B1DF	num2=case	91
2ADE0	numblstcase	92	0A0007	^NUMMODE	214
39CB3	{ob&\$}	47	37073	Ob>Seco	57
2F1AE	ObEdit	172	197006	^OBJ2REAL	33
2B8BE	(OBJ>R)	84	374006	^OBJDIMS2MAT	184
19A006	^OBJINT?	182	19B006	^OBJPOSINT?	182
05944	OCRC	107	2F257	OCRC%	107

地址	名称	页码	地址	名称	页码
07F007	^ODE_INT	208	086007	^ODE_SEPAR	208
085007	^ODEYPESTO	208	076AE	OFFSRRP	69
33111	ONE	17	36739	ONE#>	28
0A6006	^ONE>POLY	202	073CE	ONE_DO	95
363CE	ONE_EQ	28	36A63	ONECOLA	85
36B12	ONEDUP	25	3657C	ONEFALSE	26
369E6	ONEFALSE'	84	334EF	ONEHUNDRED	20
36B12	ONEONE	25	35EA2	ONESWAP	26
29D18	ONE{ }N	56	09B006	^ONE{ }POLY	202
03B75	OR	88	25EF0	OR\$	45
359E3	ORcase	89	280C1	ORDERXY#	66
280F8	ORDERXY%	66	365F9	ORNOT	88
05902	OSIZE	107	032C2	OVER	73
36775	OVER#+	28	367C5	OVER#-	28
36694	OVER#0=	28	36147	OVER#2+UNROL	58
366A8	OVER#<	29	35268	OVER#=	28
348E2	OVER#=case	90	36725	OVER#>	29
369AA	OVER'	84	36CF4	OVER5PICK	73
36183	OVERARSIZE	52	35CF4	OVERDUP	73
36482	OVERINDEX@	95	05622	OVERLEN\$	44
35D6C	OVERSWAP	73	35D6C	OVERUNROT	73
0D8007	^P2P#	201	215006	^PA2B2	181
2EF62	palparse	46	35B82	palrompdcmp	47
2B475	ParOuterLoop	130	2EF6E	ParseFail	46
3D2006	^PARTFRAC	208	0F2007	^PASCAL_NEXTLINE	205
2EF94	PASTE.EXT	171	25EF1	PATHDIR	102
228006	^PCAR	188	3DC006	^PDer	206
3F8006	^PDIV2ext	200	0C6006	^PDivLk	202
209006	^PEGCD	218	2C1006	^PEval	223
37B006	^PEVAL	187	0EA006	^PEvalFast?	203
0E1006	^PEvalMod	210	0DA007	^PEvalN/D	209
0AC006	^PFactor	201	0C5006	^PFactPowCnt	202
0C3006	^PFactTriv	202	268006	^PFEXCECext	222
0AE006	^PHFctr	201	0B0006	^PHFctr0	201
0AF006	^PHFctrl	201	2EA006	^pi	221
30017	PI/180	32	438006	^pi/2-acos	198
43B006	^pi/2-asin	198	439006	^pi/2-meta	198
032E2	PICK	74	2F258	PICTRCL	174
28A006	^PIext	56	2F6006	^pifois2	221
2E9006	^pisur-2	221	2E8006	^pisur2	221
354006	^PIVOTFLOAT	186	260DF	PIXOFF	66
260D5	PIXOFF3	66	260E4	PIXON	66
260DA	PIXON3	66	260EE	PIXON?	66
260E9	PIXON?3	66	255BF	PixonB	67
255C4	PixonG1	67	255C9	PixonG2	67
255BA	PixonW	67	255CE	PixonXor	67
36FE2	plDRPpZparg	64	1E4006	^PLOTADD	217
0A9007	^PLUSAT0	214	0AB007	^PLUSATINFTY	214
3A7006	^plusinf	198	2E4006	^PLUSINFext	221
0B3006	^PNFctr	201	103006	^PNMax	203
2B628	POLKeyUI	130	2B6CD	POLRestoreUI	130
2B6B4	POLResUI&Err	130	2B4AC	POLSaveUI	130
2B542	POLSetUI	130	0A0006	^{ }POLY	202
0D7007	^POLYASYM	203	0CC007	^POLYPARITY	203
0D6007	^POLYSYM	203	1DD007	{^POPFLAGS}	214
25F19	PopMetaVStack	100	25F1A	PopMetaVStackDROP	99
25F1B	PopVStack	99	25F1C	PopVStackAbove	100
378FA	POS\$	44	37906	POS\$REV	45
378FA	POSCHR	45	37906	POSCHREX	45
376EE	POSCOMP	55	2E6006	^POSINFext	221
0DB007	^POSITIFext	212	2E7006	^POSUNDEFext	221
3F7006	^POTENCEext	193	0AB006	^PPP	201
2BE006	^PPZ	223	117007	^PPZZ	223
124006	^PREPARext	222	4EB006	^prepvarlist	56
2D0006	^PREVALext	208	25EF2	PrevNonNull	102
08376	PREVRAM-WORD	102	2610C	PrgmEntry?	158
0C7006	^Prime+	182	0C8006	^Prime-	182
2EEF0	PromptIdUtil	45	0BD007	^PROMPTSTO1	102
3F9006	^PSetSign	200	3E8006	^PSEUDODIV	193
3E7006	^PSEUDOPREP	222	29A5D	psh	58
2963E	psh&	58	29821	psh1&	58
298C0	psh1&rev	59	29693	psh1top&	58
1CB006	^pshder*	207	2973B	pshtop&	58
29972	pshzer	59	29986	pshzerpsharg	64
0AD006	^PSqff	201	0B4006	^PSQFF	201
3E5006	^PTAYLext	200	4F5006	^PTrim	202

地址	名称	页码	地址	名称	页码
37118	PuHiddenVar	102	28071	pull	27, 58
260F3	PULLCMPEL	52	35B006	^PULLEL[S]	187
29754	pullpsh1&	59	260F8	PULLREALEL	52
28085	pullrev	58	2F076	puretemp?	62
2F0AC	PURGALARM%	104	08C27	PURGE	102
1DC007	(^PUSHFLAGS)	214	26265	PushMetaVStack	99
25F1D	PushMetaVStack&Dro	99	25F1E	PushVStack	99
	p				
25F1F	PushVStack&Clear	99	25F20	PushVStack&Keep	100
25F21	PushVStack&KeepDRO	100	35D006	^PUT[]	187
	P				
2F2F5	PUT_FONTE	173	2F2F7	PUT_STYLE	172
260FD	PUTCMPEL	52	26102	PUTEL	52
2626A	PutElemBotVStack	100	2626F	PutElemTopVStack	100
2EEF2	PUTINDEP	174	2EEF3	PUTINDEPLIST	174
075E9	PUTLAM	78	2B42A	PUTLIST	56
2EEF6	PUTPTYPE	174	26107	PUTREALEL	52
2EEF4	PUTRES	175	2EEF1	PUTSCALE	175
2F365	PUTXMAX	174	2F366	PUTXMIN	174
2F367	PUTYMAX	174	2F368	PUTYMIN	174
0EB006	^PZadic	211	2BF006	^PZHSTR	223
3701E	pZpargSWAPUn	64	293F8	P{ }N	55
509006	^QABSext	222	118006	^QAdd	200
0E2006	^QAddMod	210	508006	^QCONJext	222
11C006	^QDiv	200	3F5006	^QDiv?	193
0E5006	^QDivMod	210	3EE006	^QDivRem	200
2B3006	^QGcd	194	0E7006	^QGcdMod	210
0E6006	^QInvMod	210	0F8006	^QIsZero?	182
0D9006	^QMod	200	111006	^QMul	200
0E4006	^QMulMod	210	4FB006	^QNeg	189
51A006	^QNORMext	222	074007	^QPI	221
078007	^Qpi	221	079007	^Qpi%	221
076007	^QpiArray	221	077007	^QpiList	221
075007	^QpiSym	221	073007	^QpiZ	221
115006	^QSub	200	0E3006	^QSubMod	210
517006	^QUADRANT	38	3EC006	^QUOText	193
3F3006	^QUOTOBJext	193	381006	^QXA	188
165006	^QXNDext	222	400006	^R2SYM	179
18D006	^R2Zext	180	0701F	R>	83
2B8E6	(R>OBJ)	84	0F5006	^R>Z	180
07012	R@	83	50A006	^RABSext	189
28B006	^RACTOFACext	204	25EF3	RAD?	107
119006	^RADDext	200	082E3	RAM-WORDNAME	102
113006	^RASOP	200	0FD007	^RATSUM	209
29F006	^RCL1IDNT	223	2F196	RCL_CMD	168
2F197	RCL_CMD2	168	2EF85	RCL_CMD_DEB	170
2EF86	RCL_CMD_FIN	170	2F199	RCL_CMD_MODE	169
2EF87	RCL_CMD_POS	168	2F314	RCLALARM%	104
0C7007	^RCLEPS	215	370AF	RclHiddenVar	102
0C2007	^RCLMODULO	215	0C3007	^RCLPERIOD	215
2F259	RCLSYSF	106	2F25A	(RCLSYSF2)	106
2F25B	RCLUSERF	106	2F25C	(RCLUSERF2)	106
0C4007	^RCLVX	215	505006	^RCONJext	189
11B006	^RDIVext	200	06FB7	RDROP	84
3597F	RDROPCOLA	84	2962A	RDROPFALSE	88
26111	RDUP	84	261FC	Re>C%	37
33111	real	17	195006	^REAL?	119
281006	^REALBICAR	204	36DF3	REALcase	92
33233	REALEXT	18	528006	^REALLN	189
09C007	^REALMODE	214	331A7	REALOB	18
33733	REALOBOB	21	331B1	REALREAL	18
33797	(REALSTRSTR)	22	3320B	REALSYM	18
25EF4	RECLAIMDISP	155	0A7007	^RECURMODE	214
099006	^REDUCE	205	0F6007	^REDUCEMETASYST	186
0F5007	^REDUCEMETASYST	186	3D7006	^REGCDext	203
23D006	^REORDER	219	071E5	REPEAT	94
047C7	REPKEY?	122	25583	Repl	67
085D3	REPLACE	102	468006	^REPLACE2BY1	192
2F2ED	REPLACEALL	171	2F2FC	REPLACEALLNOSCREEN	171
359F7	REQcase	91	35A10	REQcasedrop	92
0CA007	^RESETCASCFG	215	2F0A1	RESETDEPTH	72
35BFF	RESOROMP	69	111007	^RESPSHIFTQ	203
1D4006	^RESTORECASFLAGS	215	089002	(^RestoreHARDBUFF)	136
25F22	RestoreSysFlags	106	37186	RestVarRes	102
10F007	^RESULTANT	203	110007	^RESULTANTLP	203
28187	reversym	72	258C7	ReviewKey!	164

地址	名称	页码	地址	名称	页码
28E006	^RFACT2ext	204	28D006	^RFACText	204
28F006	^RFACTSTEP3	204	290006	^RFACTSTEP5	204
25F24	RIGHT\$3x6	68	2F36B	RIGHTCOL	158
0AF007	^RIGORMODE	214	500006	^RIMext	189
15C006	^RISCH13	206	15E006	^RIXCext	37
4E4006	^RLVARext	211	0C2006	^RmCombNext	202
112006	^RMULText	200	32F006	^RNDARRY	185
30F14	RNDXY	34	4FC006	^RNEGext	189
03325	ROLL	73	29A8F	roll2ND	58
36FCE	roll2top&	58	35FC4	ROLLDROP	73
35D80	ROLLSWAP	73	36FCE	rolltwotop&	58
34FCD	ROM-WORD?	69	33715	ROMPANY	21
3319D	rompointer	18	08CCC	ROMPTR>#	69
07E99	ROMPTR@	69	37D006	^ROOTM2ROOT	205
4C9006	^ROOT{ }N	210	03295	ROT	72
36761	ROT#+	28	35E07	ROT#+SWAP	28
367B1	ROT#-	28	36801	ROT#1+	28
28001	ROT#1+UNROT	28	35E07	ROT+SWAP	28
3574D	ROT2DROP	71, 72	35CA4	ROT2DUP	72
35CB8	ROTAND	88	341A8	ROTDROP	72
34195	ROTDROPSWAP	72	3579C	ROTDUP	72
35CCC	ROTOVER	72	343BD	ROTROT2DROP	72, 73
3416E	ROTSWAP	72	36FBA	ROTUntop&	58
2F36C	Rows8-15	158	11F006	^RP#	200
123006	^RPext	200	070FD	RPIT	89
070C3	RPITE	89	2A964	RPNDecomp#Disp	46
2A944	RPNDecomp#Line	46	2A904	RPNDecomp1Line	46
2A984	RPNDecompEcho	47	2A924	RPNDecompEdit	47
2A9A4	RPNDecompStd1Line	46	2A9C4	RPNDecompStd1Line3	46
			2		
26C006	^rpnQOBJext	223	3DF006	^RRDMext	202
4FE006	^RREext	189	223006	^rref	218
368C9	RSKIP	84	261A2	rstfmt1	107
116006	^RSUBext	200	34144	RSWAP	84
004002	^RunChooseSimple	141	2AB69	RunInApprox	106
0B954	(RunInNewContext)	171	2A9E9	RunRPN:	117
2ABF0	RunSafeFlags	106	2ABD7	RunSafeFlagsNoErro	106
			r		
0F2006	^S>Z	180	0F3006	^S>Z?	180
35A5B	SAFE@	101	25EF7	SAFE@_HERE	101
1D3006	^SAFEPURGE	102	35A29	SAFESTO	101
175006	^SAMEMATRIX	185	176006	^SAMEMATSCTYPE	185
1D2006	^SAVECASFLAGS	215	261A7	savefmt1	107
088002	(^SaveHARDBUFF)	136	2F05E	SaveLastEdit	173
25EFB	SaveLastMenu	163	34D58	SAVESTACK	78
25F23	SaveSysFlags	106	3712C	SaveVarRes	102
329006	^SCL*MAT	185	2F36D	SCROLLDOWN	159
0C1007	^SCROLLext	159	2F36E	SCROLLLEFT	159
2F36F	SCROLLRIGHT	159	2F370	SCROLLUP	158
255B0	ScrollLVGrob	67	33157	seco	17
3FC006	^SECO2CMPext	222	2E107	Seco>Menu	67, 164
461006	^SECOEXEC	222	4D4006	^SECOSQFFext	182, 222
2F2E7	SELECT.FONT	173	2F2E1	SELECT.LINE	171
2F2E2	SELECT.LINEEND	171	0312B	SEMI	84
34AAD	SEMILOOP	95	2EEBB	SENDLIST	108
25EF8	SEP\$NL	45	419006	^SERIESEXPLN	193
25719	SetAlgEntry	158	26139	SetAlphaAnn	158
25695	SetAppMode	130	25671	SetAppSuspOK	131
261AC	setbeep	107	26116	SETCIRCERR	98
09A007	^SETCOMPLEX	214	2F378	SetCursor	170
2F379	SetDA123NoCh	157	2EE65	SetDA12a3NCh	157
2EE65	SetDA12a3NoCh	157	2EE6F	SetDA12NoCh	157
2EE71	SetDA12Temp	156	2EE70	SetDA13NoCh	157
2EE78	SetDA1Bad	156	2EEAC	SetDA1IsStat	157
2EE72	SetDA1NoCh	157	2EE69	SetDA1Temp	156
2EE67	SetDA1Valid	156	2EE94	SetDA23NoCh	157
2EE79	SetDA2aBad	156	2EE6C	SetDA2aEcho	157
2EE73	SetDA2aNoCh	157	2EE8A	SetDA2aTemp	156
2EF98	SetDA2aValid	156	2EE7A	SetDA2bBad	157
2EE7F	SetDA2bIsEdL	157	2EE76	SetDA2bNoCh	157
2EE6A	SetDA2bTemp	156	2EEA5	SetDA2bTempF	156
2EE68	SetDA2bValid	156	2EE93	SetDA2NoCh	157
2F37A	SetDA2OKTemp	156	2EE91	SetDA2Valid	156
2EE7B	SetDA3Bad	157	2EE77	SetDA3NoCh	157
2EE6B	SetDA3Temp	156	2EF99	SetDA3Valid	156
2EEA0	SetDA3ValidF	156	2EE7C	SetDAsNoCh	157

地址	名称	页码	地址	名称	页码
2EE64	SetDasTemp	156	2FFBD	SETDEG	107
2565F	SetDoStdKeys	130	09F007	^SETEXACT	214
2FFEF	SETGRAD	107	07638	SETHASH	69
26283	SetHeader	155	3714A	SetHiddenRes	102
2F37B	SetIOPARErr	98, 108	2F458	SETIVLERR	98
25EFC	SetKeysNS	163	2613E	SetLeftAnn	158
04FB6	SETMEMERR	98	2564D	SetNAppKeyOK	130
262E7	SETNONEXTERR	98	2EEAE	SetNoRollDA2	157
25617	(SetNUsrKeyOK)	122	0AA007	^SETPLUSAT0	214
04FF2	SETPORTNOTAV	98	26143	SetPrgmEntry	158
2FFDB	SETRAD	107	2580E	SetRebuild	163
26148	SetRightAnn	158	05016	SETROMPERR	98
262D8	SETSIZEERR	98	25EFD	SetSomeRow	165
262E2	SETSTACKERR	98	2645E	setStdEditWid	46
26459	setStdWid	46	2614D	SetSysFlag	106
25EFE	SetThisRow	164	262DD	SETTYPEERR	98
26152	SetUserFlag	106	2622E	SetVStackProtectWo rd	100
3314D	SEVEN	17	342EA	SEVENROLL	73
331B1	SEVENTEEN	18	333C3	SEVENTY	19
333EB	SEVENTYFOUR	19	3341D	SEVENTYNINE	19
0CD006	^SFactor	181	0426A	ShowInvRomp	107
0D3007	^SHRINK2ASYM	203	0D1007	^SHRINK2SYM	203
0D4007	^SHRINKASYM	203	0CF007	^SHRINKEVEN	203
0D2007	^SHRINKSYM	203	3721C	Sig?ErrJump	98
301006	^SIGMAEXP2ext	192	2FF006	^SIGMAEXPext	192
237006	^SIGNE	212	0DC007	^SIGNE1ext	212
0EC007	^SIGNE>	212	0E6007	^SIGNEATAN	212
0E4007	^SIGNECOS	212	0EF007	^SIGNEERROR	213
0E2007	^SIGNEEXP	212	0EA007	^SIGNELEFT	212
0E1007	^SIGNELN	212	0E9007	^SIGNERIGHT	212
0E3007	^SIGNESIN	212	0E7007	^SIGNESQRT	212
0E5007	^SIGNETAN	212	0E0007	^SIGNMOINS	212
0ED007	^SIGNMULText	212	0DF007	^SIGNPLUS	212
0DE007	^SIGNUNDEF	212	0A6007	^SILENTMODE	214
299006	^SIMP1ext	192	2AE006	^SIMP3ext	192
2A9006	^SIMPext	192	2AD006	^SIMPGCDext	192
29D006	^SIMPIDNT	223	298006	^SIMPLIFY	192
2A8006	^SIMPNDXFext	223	2A0006	^SIMPSYMBs	192
2A2006	^SIMPUSERFCN	192	29B006	^SIMPVAR	192
2A4006	^SIMP	192	428006	^sin/cos	198
445006	^sin2exp	198	444006	^SIN2EXPext	191
42E006	^SIN2ext	191	414006	^SIN2TAN	191
415006	^sin2tan	198	40B006	^SIN2TAN/2	191
40C006	^sin2tan/2	197	42A006	^SIN2TC	191
429006	^SIN2TCext	193	303006	^SINEXPA	192
307006	^SINEXPA*	199	308006	^SINEXPA*1	199
305006	^SINEXPA+	199	306006	^SINEXPA-	199
449006	^sinh2exp	198	448006	^SINH2EXPext	191
0D3006	^SIsPrime?	181	33143	SIX	17
34281	SIXROLL	72	331A7	SIXTEEN	18
3335F	SIXTY	19	333AF	SIXTYEIGHT	19
33387	SIXTYFOUR	19	33369	SIXTYONE	19
3337D	SIXTYTHREE	19	33373	SIXTYTWO	19
3438D	SIXUNROLL	73	0714D	SKIP	85
35715	skipcola	85	0B1007	^SLOPPY?	215
0B0007	^SLOPPYMODE	215	26120	SLOW	104
2B2006	^SLOWGCDext	194	297006	^SLVARext	192
2BB3A	sncknum2	64	219006	^SOLVE1EQ	218
0F7007	^SOLVECRAMER	186	21A006	^SOLVEMANYEQ	218
0F4007	^SOLVEMETASYST	186	1DE006	^SOLVEXFLOAT	217
27E006	^SOLVext	204	25EFF	SolvMenuInit	165
2F37E	SORTASLOW	104	100006	^SortList	56
33B55	SPACE\$	42	0AD007	^SPARSEDATA	214
0CE006	^SPollard	181	2BD006	^SQFF2ext	223
4CC006	^SQFFext	210	42F006	^sqrt1-cos^2	198
42D006	^sqrt1-sin^2	198	41E006	^SQRT2LNEXP	191
41F006	^sqrt2lnexp	191	2C6006	^SQRT_IN?	223
1CC006	^SQRTINVpshd*	207	2BB21	sscknum2	64
25F12	sstDISP	159	26242	StackFontHeight	160
26288	StackLineHeight	160	25F00	StartMenu	164
2EEDA	STATCLST	53	2EEDE	STATMEAN	53
2EEDC	STATN	53	2EEDB	STATSADD%	53
2EEDD	STATSMAX	53	2EEDF	STATSMIN	53
2EEE0	STATSTDEV	53	2EEE1	STATTOT	53
2EEE2	STATVAR	53	25F01	Std/BoxLabel	67

地址	名称	页码	地址	名称	页码
2716D	StdIOPAR	108	27D7B	(StdLabelGrob)	65
25F02	StdMenuKeyLS	163	25F03	StdMenuKeyNS	163
0A3007	^STEPBYSTEP	214	206006	^STEPIDIV2	218
25F13	stkdecomp\$w	46	07D27	STO	101
369BE	STO'	84	2F198	STO_CMD_MODE	169
2EF8B	STO_CURS_POS	169	2EF8C	STO_CURS_POS2	169
2EF8D	STO_CURS_POS3	170	2EF8E	STO_CURS_POS4	170
2EF8F	STO_CURS_POS_VIS	170	2F3A9	(STOALLF)	106
2F3AA	(STOALLF2)	106	2F37F	STOALM	104
37104	StoHiddenVar	102	2F062	StoIOPAR	108
07D1B	STOLAM	78	370006	^STOMAText	187
0C6007	^STOMODULO	215	2CA006	^STOPRIMIT	223
2F25F	(STOSYSF)	106	2F260	(STOSYSF2)	106
356006	^STOSYSText	186	2F261	(STOUSERF)	106
2F262	(STOUSERF2)	106	0C5007	^STOVX	215
33125	str	17	2E0F3	Str>Menu	67, 164
3BE006	^STRICTmetaCOMPARE	199	37ABE	STRIPTAGS	51
37AEB	STRIPTAGS12	51	33319	STRLIST	19
33823	STRREALREAL	22	2557E	Sub	67
05733	SUB\$	45	2A5CA	SUB\$1#	45
35DA8	SUB\$SWAP	45	05821	SUBCOMP	55
25597	SubGor	66	2612F	SUBGROB	65
2559C	SubGxor	66	05815	SUBHXS	48
33F006	^submeta	59	3760D	SubMetaOb	59
37685	SubMetaOb1	59	25592	SubRepl	66
0E8007	^SUBSIGNE	212	243006	^SUBTMOD	219
0F9007	^SUM	209	4D6006	^SUMSQRext	182
0FB007	^SUMVX	209	10C006	^SUnivar?	203
25F04	SuspendOK?	130	03223	SWAP	72
357BB	SWAP#-	27	3592B	SWAP#1+	28, 58
28099	SWAP#1+SWAP	28	36829	SWAP#1-	28
280AD	SWAP#1-SWAP	28	36C22	SWAP%/%	34
362F2	SWAP%>C%	37	35346	SWAP%\$	45
36982	SWAP'	84	368B5	SWAP2DUP	72
36CB8	SWAP3PICK	72	36CE0	SWAP4PICK	74
36C90	SWAP4ROLL	72	39C8B	(SWAP>HCOMP)	54
3700A	SWAPCKREF	103	3635B	SWAPCOLA	85
2F073	SWAPcompSWAP	63	3421A	SWAPDROP	72
28989	(SWAPDROP#1-)	28	35857	SWAPDROPDUP	72
35872	SWAPDROPSWAP	72, 73	374BE	SWAPDROPTRUE	88
3576E	SWAPDUP	72	163006	^SWAPFXND	203
3623E	SWAPINCOMP	56	3646E	SWAPINDEX@	95
36496	SWAPLOOP	95	161006	^SWAPNDXF	203
36AFE	SWAPON	26	3457F	SWAPOVER	71, 72
367ED	SWAPOVER#-	28	117006	^SWAPRADD	200
11A006	^SWAPRDIV	200	4FF006	^SWAPRIM	189
110006	^SWAPRMULT	200	4FA006	^SWAPRNEG	189
341BA	SWAPROT	72, 73	0AC003	^SWAPROWS	187
4FD006	^SWAPRRE	189	114006	^SWAPRSUB	200
2812F	SWAPTRUE	88	36946	SWAPUnDROP	59
3695A	SWAPUnNDROP	58	58D006	^SWITCHNOTALLOWED	213
093007	^SWITCHOFF	214	092007	^SWITCHON	214
3592B	SWP1+	28, 58	2A7006	^SWPSIMPNDXF	223
51B006	^SXSQRext	189	384006	^SYLVESTER	188
3316B	sym	17	33161	symp	17
464006	^SYMBEXEC	191	157006	^SYMBINCOMP	56, 179, 195
0546D	SYMBN	55, 63	2BD8C	(SYMBN:)	63
33639	SYMBREAL	21	33657	SYMBUNIT	21
286E7	symcomp	63	29A006	^SYMEXPAN	192
336BB	SYMEXT	21	33693	SYMID	21
3369D	SYMLAM	21	1DF006	^SYMLIMIT	217
3366B	SYMOB	21	109007	^SYMPSCI	209
10B007	^SYMPSPIN	209	33675	SYMREAL	21
2EF26	SYMSHOW	64	336B1	SYMSYM	21
46F006	^SYMTAYLOR	206	26134	SYNTAXERR	98
58F006	^Sys1IT	213	2F064	Sys@	101
005002	(^sysCHOOSE)	136	08D92	SYSCONTEXT	102
2EF67	SysDisplay	155	36F79	SysITE	92
2EF66	SysMenuCheck	164	08DD4	SYSRP?	102
2F380	SysSTO	101	22A006	^SYSTEM	218
355006	^SYSText	186	2EED7	SysTime	104
2DD006	^TABLECOSext	223	2DE006	^TABLETANext	223
265006	^TABVALext	220	238006	^TABVAR	219
33189	TAGGED	17	336ED	TAGGEDANY	21
37B04	TAGOBS	51	275C6	TakeOver	164

地址	名称	页码	地址	名称	页码
442006	^TAN2CS2	191	420006	^TAN2EXP	191
421006	^tan2exp	198	427006	^TAN2SC	191
440006	^TAN2SC2	191	43F006	^TAN2SC2ext	193
426006	^TAN2SCext	193	40E006	^TAN2TAN/2	191
40F006	^tan2tan/2	197	44D006	^tanh2exp	198
44C006	^TANH2EXPext	191	1E1006	^TAYLOR0	217
302006	^TCHEBext	202	233006	^TCHEBNOCK	218
2FE006	^TCOLLECT	192	2F09A	TempConv	61
3316B	TEN	17	2E1006	^TESTINFINI	221
26170	TestSysFlag	106	26175	TestUserFlag	106
31F006	^TEXPAext	193	33189	THIRTEEN	17
33233	THIRTY	18	33283	THIRTYEIGHT	18
33265	THIRTYFIVE	18	3325B	THIRTYFOUR	18
3328D	THIRTYNINE	18	3323D	THIRTYONE	18
33279	THIRTYSEVEN	18	33251	THIRTYTHREE	18
33247	THIRTYTWO	18	2F158	(THISCHAR)	168
33125	THREE	17	33319	THREEFIVE	19
36216	THREE{ }N	56	09D006	^THREE{ }POLY	202
2F003	(Ticks>Date)	104	2F004	(Ticks>Rpt)	104
2F002	(Ticks>TOD)	104	34BEF	ticR	83
2EED3	TIMESTR	45, 104	16A006	^{}TO[]	184
26166	TOADISP	155	2EECF	TOD	104
2F381	TOD>t\$	104	2616B	TOGDISP	155
2F382	TOGGLELINE#3	66	2577F	(TogInsert)	169
2EFA1	TOGLINE	66	2EFA4	TOGLINE3	66
2F21B	ToGray	67	33AA7	(tok\$)	43
33AB3	(tok&)	43	33B85	tok'	43
33BD9	(tok*)	43	33BF1	(tok+)	43
33B91	tok,	43	33BFD	tok-	43
272D9	tok->	43	33B9D	tok.	43
33BE5	(tok/)	43	33C4D	tok0	43
33C59	tok1	43	33C65	(tok2)	43
33C71	(tok3)	43	33C7D	(tok4)	43
33C89	(tok5)	43	33C95	(tok6)	43
33CA1	(tok7)	43	33CAD	tok8	43
26206	tok8cktrior	46	261BB	tok8trior	46
33CB9	tok9	43	33BA9	(tok;)	43
33AD7	tok<<	43	33C09	tok=	43
29E99	tok=casedrop	92	33ACB	(tok>>)	43
33A6B	(tok)	44	33A51	(tok)	44
33B55	tok_	42	2D848	tok_g	43
2D86D	tok_m	43	2D8AD	tok_s	43
33AEF	(tokanglesign)	43	33C2D	(tokCTGROB)	43
33C3F	(tokCTSTR)	43	33C21	(tokDER)	43
33ABF	tokESC	43	33AE3	tokexponent	43
33BB5	(toklparen)	44	33B79	tokquote	43
33BC1	(tokrparen)	44	33A8F	toksharp	43
33AFB	(tokSIGMA)	44	33C15	(tokSQRT)	44
33B61	(tokUNKNOWN)	44	33A9B	(tokuscore)	44
33B07	(tokWHERE)	44	33A77	tok{	43
33A83	(tok})	44	33BCD	(tok^)	44
364E1	toLEN_DO	95	266006	^TOLISText	220
296A7	top&	58	39C006	^top&addt*	195
39D006	^top&addt/	195	2F383	TOP16	158
2F384	TOP8	158	2F385	TOPROW	158
07709	TOSRRP	69	06657	TOTEMPOB	103
35C90	TOTEMPSWAP	103	16B006	^[]TO{ }	184
580006	^TRANSCERROR	213	330006	^TRCARRY	185
30F28	TRCXY	34	416006	^TRIGext	193
411006	^TRIGTAN	193	4F4006	^TRIMext	202
4F6006	^TRIMOBJext	211	03A81	TRUE	87
369D2	TRUE'	84	36540	TRUEFALSE	87
36540	TrueFalse	87	27E87	TrueTrue	87
471006	^TRUNC DL	206	2B9006	^TSIMP2ext	192
2BB006	^TSIMP3ext	192	2BA006	^TSIMPext	192
3326F	TTHIRTYSIX	18	2F034	TURNMENUOFF	155
2F031	TURNMENUON	155	041A7	TurnOff	107
3317F	TWELVE	17	331CF	TWENTY	18
3321F	TWENTYEIGHT	18	33201	TWENTYFIVE	18
331F7	TWENTYFOUR	18	33229	TWENTYNINE	18
331D9	TWENTYONE	18	33215	TWENTYSEVEN	18
3320B	TWENTYSIX	18	331ED	TWENTYTHREE	18
331E3	TWENTYTWO	18	3311B	TWO	17
09E006	^TWO::POLY	202	36202	TWO{ }N	56
09C006	^TWO{ }POLY	202	03C64	TYPE	117
350D2	TYPEAPLET?	118	3513B	TYPEARRY?	117

地址	名称	页码	地址	名称	页码
350F0	TYPEBINT?	118	352AD	TYPECARRY?	117
3503C	TYPECHAR?	118	03F95	(TYPECMP)	25
3512C	TYPECMP?	117	35177	TYPECOL?	118
3510E	TYPECSTR?	117	03FDB	(TYPEEREL)	25
03FE5	(TYPEEXT)	25	351B3	TYPEEXT?	118
35087	TYPEFLASHPTR?	118	350C3	TYPEFONT?	118
114007	^TYPEGAUSSINT?	118, 182	35186	TYPEGROB?	118
350FF	TYPEHSTR?	118	03FA9	TYPEIDNT	25
3504B	TYPEIDNT?	117	194006	^TYPEIDNTLAM?	117
167006	^TYPEIRRQ?	179, 222	03FD1	(TYPELAM)	25
350E1	TYPELAM?	117	03F9F	(TYPELIST)	25
35195	TYPELIST?	117	350B4	TYPELNGCMP?	118
350A5	TYPELNGREAL?	118	35292	TYPERARRY?	117
03F8B	TYPEREAL	25	3511D	TYPEREAL?	117
196006	^TYPEREALZINT?	119	3514A	TYPEROMP?	118
20D6F	(TYPERRP)	25	35159	TYPERRP?	118
03FBD	(TYPESYMB)	25	35168	TYPESYMB?	118
351A4	TYPETAGGED?	118	182006	^TYPEZ?	118
35096	TYPEZINT?	118	33193	unitob	17
2F07B	U>nbr	61	2F099	U>NCQ	61
0B7006	^UFactor	201	0B8006	^UFactor1	201
0BD006	^UFactorDeg2	202	2F07C	UM#?	62
2F07D	UM%	61	2F07E	UM%CH	61
2F07F	UM%T	61	2D74F	um*	60
2F080	UM*	61	2F081	UM+	61
2F082	UM-	61	2D759	um/	60
2F083	UM/	61	2F085	UM<=?	62
2F086	UM<?	62	2F087	UM=?	62
2F088	UM>=?	62	2F089	UM>?	62
2F07A	UM>U	61	2F08A	UMABS	61
2D999	UMCEIL	61	2F08B	UMCHS	61
2F08C	UMCONV	61	2F08D	UMCOS	61
2D777	umEND	60	2D985	UMFLOOR	61
2D971	UMFP	61	2D95D	UMIP	61
2F08E	UMMAX	61	2F08F	UMMIN	61
2D76D	umP	60	2D9CB	UMRND	61
2F090	UMSI	61	2D949	UMSIGN	61
2F091	UMSIN	61	2F092	UMSQ	61
2F093	UMSQRT	61	2F094	UMTAN	61
2D9EE	UMTRC	61	2F095	UMU>	61
2F096	UMXROOT	61	2D763	um^	60
2F098	Unbr>U	61	262F6	UNCOERCE	32
36BFA	UNCOERCE%%	32	3F495	UNCOERCE2	32
29CB9	uncrunch	63	34FA6	undo	78
2F019	UNIT>\$	61	10B006	^Univar?	203
25F08	UnLockAlpha	158	0339E	UNROLL	73
29B12	unroll2ND	58	3422B	UNROT	73
343BD	UNROT2DROP	72, 73	35872	UNROTDROP	72, 73
35D1C	UNROTDUP	73	360CF	UNROTOVER	73
341BA	UNROTSWAP	72, 73	3A6006	^unsignedinf	198
071C8	UNTIL	94	2F193	UobROT	59
2F265	UPDIR	102	2F266	USER\$>TAG	51
463006	^USERFCN?	63	36F65	UserITE	92
25F09	UserKeys?	122	23F006	^USERLIDNT	219
23E006	^USERLVAR	219	324006	^VADD	185
404006	^VAL1	223	405006	^VAL1M	223
401006	^VAL2ext	223	58C006	^VALMUSTBE0	213
3FF006	^VALOBJext	223	22B006	^VANDERMONDE	218
45B006	^VAR=LIST	191	4B6006	^VARCOMP!	206
4BA006	^VARCOMP32!	206	0C4006	^VarFactor	202
2F267	VARSIZE	107	37A006	^VBINARYOP	187
579006	^Verbose1	220	57A006	^Verbose2	220
57B006	^Verbose3	220	0A5007	^VERBOSEMODE	214
57C006	^VerboseN	220	2EF93	VERIF_SELECTION	170
2F388	VerifyTOD	104	1D0006	^VERNUMext	215
2F389	VERSTRING	108	26125	VERYSLOW	104
2612A	VERYVERYSLOW	104	3C1006	^vgerxssSYMSUM	209
2F2E0	ViewEditGrob	173	2F21F	ViewGrobObject	159
2F19A	ViewLevell	172	2F21D	ViewObject	159
2F21E	ViewStrObject	159	32B006	^VPMULT	185
342006	^VRRDM	184	343006	^VRRDMmeta	184
325006	^VSUB	185	379006	^VUNARYOP	187
4EA006	^VX!	211	4E9006	^VX>	211
4E8006	^VX>LVARExt	211	4E2006	^VXINDEP?	211
589006	^VXINDEPERR	213	4E6006	^VXLVARExt	211
4DB006	^VXXL0	179	4DA006	^VXXL1ext	179

地址	名称	页码	地址	名称	页码
4DD006	^VXXL2	179	4DC006	^VXXL2NR	179
4D7006	^VXXLext	179	4D9006	^VXXLFext	179
2F268	Wait/GetKey	122	25F0B	WaitForKey	122
2D1006	^WARNSING	208	071EE	WHILE	94
2EF61	WINDOW#	159	2F38A	WINDOWBOT?	159
2617F	WINDOWCORNER	158	26184	WINDOWDOWN	158
26189	WINDOWLEFT	158	2F38B	WINDOWLEFT?	159
2618E	WINDOWRIGHT	158	2F38C	WINDOWRIGHT?	159
2F38D	WINDOWTOP?	159	26193	WINDOWUP	158
26198	WINDOWXY	158	370C3	WithHidden	102
2EFBE	WORDSIZE	48	09A003	(^WRAP\$)	45
08E007	^WRITEMENU	164	127006	^x*ext	189
125006	^x+ext	189	126006	^x-ext	189
129006	^x/ext	189	128006	^x=ext	190
50D006	^xABSext	189	533006	^xACOSext	190
54E006	^xACOSHext	190	532006	^xASINext	190
54B006	^xASINHext	190	534006	^xATANext	190
548006	^xATANHext	190	52C006	^xCOSext	190
52F006	^xCOSHext	190	17F006	^XEQ>ARRAY	53
17C006	^XEQARRY>	53	2F296	XEQORDER	102
25F14	XEQPGDIR	102	2F297	XEQPURGEPICT	175
2F2A3	XEQRCL	101	2F2A7	XEQSETLIB	69
2F2A9	XEQSHOWLS	64	25E79	XEQSTOID	101
25F0C	XEQStoKey	101	3BC43	XEQTYPE	117
529006	^xEXPext	190	0BE007	^XGROBext	68
33625	XHI	21	3361B	XHI-1	21
503006	^xIMext	189	52A006	^xINVext	189
2EF92	XLINe_SIZE?	173	522006	^xLNext	189
2F38E	xnsgeneral	59	23C006	^XNUM	219
03ADA	XOR	88	25F0D	XOR\$	45
501006	^xREext	189	455006	^xroot2expln	198
454006	^XROOT2ext	191	52D006	^xSINext	190
530006	^xSINHext	190	2F38F	xnsgeneral	59
553006	^xSQext	189	51C006	^XSQRext	189
3E823	xSTO>	101	53F006	^xSYMACOS	190
550006	^xSYMACOSH	190	56F006	^xSYMALOG	190
53D006	^xSYMASIN	190	54D006	^xSYMASINH	190
541006	^xSYMATAN	190	54A006	^xSYMATANH	190
561006	^xSYMCEIL	190	536006	^xSYMCOS	190
545006	^xSYMCOSh	190	559006	^xSYMEXP	197
571006	^xSYMEXPM1	190	575006	^xSYMFACT	190
55F006	^xSYMFLOOR	190	565006	^xSYMFP	190
504006	^xSYMIM	196	557006	^xSYMINV	189
563006	^xSYMIP	190	56B006	^xSYMLNP1	190
56D006	^xSYMLOG	190	569006	^xSYMMAnt	190
578006	^xSYMNOT	190	502006	^xSYMRE	196
538006	^xSYMSIN	190	543006	^xSYMSINH	190
555006	^xSYMSQ	189	552006	^xSYMSQRT	189
53A006	^xSYMTAN	190	547006	^xSYMTANH	190
567006	^xSYMXPON	190	52E006	^xTANext	190
531006	^xTANHext	190	52B006	^xvext	189
3421A	XY>Y	72	25F0E	XYGROBDISP	65
341D2	XYZ>	72	3574D	XYZ>Y	71, 72
3416E	XYZ>YXZ	72	341A8	XYZ>YZ	72
343BD	XYZ>Z	72, 73	3636F	XYZ>ZCOLA	85
35EF2	XYZ>ZTRUE	88	35872	XYZ>ZX	72, 73
3422B	XYZ>ZXY	73	34195	XYZ>ZY	72
341BA	XYZ>ZYX	72, 73	341D7	XYZW>	72
343CF	XYZW>W	73	34331	XYZW>WXYZ	73
36C90	XYZW>YWZX	72	3423A	XYZW>YZWX	72
12B006	^x^ext	189	331006	^Yext	52
33387	YHI	19	18A006	^Z2%%	32
0EF006	^Z2BIN	26	18E006	^Z2Sext	180
265BC	Z<	182	588006	^Z<0ERR	213
265D0	Z<=	182	265C6	Z<>	182
265C1	Z=	182	265B7	Z>	182
19D006	^Z>#	26	587006	^Z>#ERR	213
265CB	Z>=	182	0F6006	^Z>R	32
0F1006	^Z>S	44	0F4006	^Z>ZH	180
102006	^ZAbs	180	50B006	^ZABS	180
10E006	^ZBit?	181	10D006	^ZBits	181
108007	^ZEILBERGER	209	33107	ZERO	17
073C3	ZERO_DO	95	36568	ZEROFALSE	26
3709B	ZEROISTOPSTO	95	360BB	ZEROOVER	26
21B006	^ZEROS1EQ	218	21C006	^ZEROSMANYEQ	218
35E75	ZEROSWAP	26	37287	ZEROZERO	25

地址	名称	页码	地址	名称	页码
37394	ZEROZEROONE	26	373A8	ZEROZEROTWO	26
37380	ZEROZEROZERO	25	27D006	^ZFACTO	204
0C9006	^ZFactor	181	2B8006	^ZGcd	181
2B7006	^ZGCDext	181	0FC006	^ZIsNeg?	182
0FA006	^ZIsOne?	182	0D2006	^ZIsPrime?	181
0DD006	^ZMod	181	107006	^ZNLT?	182
105006	^ZNMax	181	106006	^ZNMin	181
0D1006	^ZPrime?	181	0F0007	^ZSIGNE	212
0F1007	^zsigne	212	0EE007	^ZSIGNECK	212
0E0006	^ZSQRT	181	0D8006	^ZTrialDiv	182
0D6006	^ZTrialDiv2	182	0D7006	^ZTrialPrime?	182
101006	^ZTrim	180	189006	^ZZ2C%%ext	37
05F0B3	(~\$>grobOrGROB)	68	0860B0	~grobAlertIcon	65
0870B0	~grobCheckKey	65	0050B0	~IFMenuRow1	147
0060B0	~IFMenuRow2	147	01E0E8	~INTEMPOB?	103
0040B1	~MsgBoxMenu	160	0DE0B0	~nNullBind	77

附录 G

按地址排序的词条

地址	名称	页码	地址	名称	页码
0000B3	Choose	141	001004	^FSTR1	47
001007	^ListToArray	53	01118	LowBat?	107
002007	^ArrayToMatrix	53	003004	^FSTR3	47
003007	^ArrayToList	185	0312B	SEMI	84
0314C	DEPTH	72	03188	DUP	71
031AC	2DUP	71	031D9	NDUP	71
03223	SWAP	72	03244	DROP	71
03258	2DROP	72	0326E	NDROP	58, 72
03295	ROT	72	032C2	OVER	73
032E2	PICK	74	03325	ROLL	73
0339E	UNROLL	73	0371D	GETATELN	52
03826	(#A8241)	25	03880	(#102A8)	25
038DC	(#E13A8)	25	03A81	TRUE	87
03AC0	FALSE	87	03ADA	XOR	88
03AF2	NOT	88	03B2E	EQ	88
03B46	AND	88	03B75	OR	88
03B97	EQUAL	88	03C64	TYPE	117
03CA6	#0=	28	03CC7	#0<>	28
03CE4	#<	28	03D19	#=	28
03D4E	#<>	28	03D83	#>	28
03DBC	#+	26	03DE0	#-	27
03DEF	#1+	26	03E0E	#1-	27
03E2D	#2+	26	03E4E	#2-	27
03E6F	#2*	27	03E8E	#2/	27
03EB1	#AND	28	03EC2	#*	27
03EF7	#/	27	03F8B	TYPEREAL	25
03F95	(TYPECMP)	25	03F9F	(TYPELIST)	25
03FA9	(#2E48)	25	03FA9	TYPEIDNT	25
03FBD	(TYPESYMB)	25	03FD1	(TYPELAM)	25
03FDB	(TYPEEREL)	25	03FE5	(TYPEEXT)	25
03FEF	(#2614)	24	03FF9	(#2686)	24
004002	^RunChooseSimple	141	004004	^FSTR4	47
0040B1	~MsgBoxMenu	160	041A7	TurnOff	107
041ED	DEEPSLEEP	107	0426A	ShowInvRomp	107
04708	CHECKKEY	121	04714	GETTOUCH	121
047C7	REPKEY?	122	04A0B	GETPROC	163
04A41	GETDF	163	04CE6	ERROR@	97
04D0E	ERRORSTO	97	04D33	ERRORCLR	97
04D3E	DROPNULL\$	44	04D64	GETTHEMSG	97
04D87	JstGetTHEMSG	97	04D87	JstGETTHEMSG	97
04E07	GETEXITMSG	97	04E37	EXITMSGSTO	97
04E5E	ERRSET	97	04EA4	ABORT	97
04EB8	ERRTRAP	97	04ED1	ERRJMP	97
04FB6	SETMEMERR	98	04FF2	SETPORTNOTAV	98
005002	(^sysCHOOSE)	136	005004	^FSTR5	47
05016	SETTROMPERR	98	05040	ATTNFLG@	122
05068	ATTNFLGCLR	122	05089	CARCOMP	54
0050B0	~IFMenuRow1	147	0050B3	ChooseMenu0	141
050ED	CAR\$	44	05153	CDRCOMP	54
0516C	CDR\$	44	0518A	&HXS	48
05193	&\$	45	0521F	&COMP	54
0525B	>H\$	45	052C6	>HCOMP	54
052EE	>T\$	45	052FA	>TCOMP	54
05445	: :N	55	05459	{ }N	55
0546D	SYMBN	55, 63	05481	EXTN	55, 60
054AF	INNERCOMP	56	0556F	NULL\$?	47
055B7	NULLCOMP?	54	055D5	NULLHXS	48
055DF	NULL\$	42	055E9	NULL{ }	56
055FD	NULL: :	57	05616	LENHXS	48
05622	OVERLEN\$	44	05636	LEN\$	44
0567B	LENCOMP	54	056B6	NTHELCOMP	54
05733	SUB\$	45	05815	SUBHXS	48
05821	SUBCOMP	55	05902	OSIZE	107
05944	OCRC	107	059CC	#>HXS	48
05A03	HXS>#	26	05A51	CHR>#	26
05A75	#>CHR	44	05AB3	CHANGETYPE	107
05B15	\$>ID	77	05BE9	ID>\$	44
05C27	%>C%	37	05D2C	C%>%	32
05DBC	C%>%%	37	05E81	>TAG	51
05F2E	ID>TAG	51	05F42	GARBAGE	107
05F61	MEM	107	006004	^FSTR6	47

地址	名称	页码	地址	名称	页码
0060B0	~IFMenuRow2	147	0060B3	ChooseMenu1	141
0657E	(#61441)	25	06657	TOTEMPOB	103
06B4E	INTEMNOTREF?	103	06E8E	NOP	83
06E97	'	84	06EEB	'R	83
06F66	'REVAL	83	06F8E	EVAL	83
06F9F	>R	83	06FB7	RDROP	84
06FD1	COLA	85	007002	^Ck&DoMsgBox	160
007004	^FSTR7	47	07012	R@	83
0701F	R>	83	0070B3	ChooseMenu2	141
070C3	RPITE	89	070FD	RPIT	89
0712A	?SKIP	89	0712A	NOT_IT	89
0714D	SKIP	85	0716B	IDUP	83, 94
071A2	BEGIN	94	071AB	AGAIN	94
071C8	UNTIL	94	071E5	REPEAT	94
071EE	WHILE	94	07221	INDEX@	95
07249	ISTOP@	95	07258	JINDEX@	95
07264	JSTOP@	95	07270	INDEXSTO	95
07295	ISTOPSTO	95	072AD	JINDEXSTO	95
072C2	JSTOPSTO	95	07334	LOOP	95
073A5	+LOOP	95	073C3	ZERO_DO	95
073CE	ONE_DO	95	073DB	#1+_ONE_DO	95
073F7	DO	95	07497	ABND	77
074D0	BIND	77	074E4	DOBIND	77
075A5	GETLAM	78	075E9	PUTLAM	78
07638	SETHASH	69	076AE	OFFSRRP	69
07709	TOSRRP	69	07943	@LAM	78
0797B	@	101	07D1B	STOLAM	78
07D27	STO	101	07E50	#>ROMPTR	69
07E99	ROMPTR@	69	0803F	(#414C1)	25
081D9	BAKNAME	69	082E3	RAM-WORDNAME	102
08326	LASTRAM-WORD	102	08376	PREVRAM-WORD	102
085D3	REPLACE	102	08696	CREATE	102
08C27	PURGE	102	08CCC	ROMPTR>#	69
08D08	CONTEXT!	102	08D5A	CONTEXT@	102
08D92	HOMEDIR	102	08D92	SYSCONTEXT	102
08DD4	SYSRRP?	102	08DF7	(#7FF)	23
08ECE	(#536A8)	25	08F1F	(#D6A8)	25
009004	^FSTR9	47	0905F	BAK>OB	69
091B4	(#2D541)	25	0B954	(RunInNewContext)	171
00D004	^FSTR13	47	011004	^EQW3Edit	172
0150B3	(BBMoveTo)	141	019004	^EQW3GROB	68
0190B3	(BBRecalOff&Disp)	141	0190E0	BRRc1C1	142
01A004	^EQW3GROBStk	68	01E004	^EQW3GROBsys	68
01E0E8	~INTEMPOB?	103	01F004	^EQW3GROBmini	68
020004	^IfMain	147	20D6F	(TYPERRP)	25
021004	^IfSetFieldVisible	147	022004	^IfSetSelected	147
0220B3	(BBRunEntryProc)	141	023004	^IfSetGrob	147
0230B3	(BBReReadPageSize)	141	024004	^IfSetFieldValue	147
0240B3	(BBReReadHeight)	141	025004	^IfSetCurrentField Value	148
0250B3	(BBReReadCoords)	141	25565	LineW	66
2556A	LineB	66	2556F	LineG1	66
25574	LineG2	66	25579	LineXor	66
2557E	Sub	67	25583	Repl	67
25588	Gor	67	2558D	Gxor	67
25592	SubRepl	66	25597	SubGor	66
2559C	SubGxor	66	255A1	Grey?	67
255B0	ScrollVGrob	67	255B5	Distance	67
255BA	PixonW	67	255BF	PixonB	67
255C4	PixonG1	67	255C9	PixonG2	67
255CE	PixonXor	67	255D3	FBoxG1	67
255D3	FBoxW	67	255D8	FBoxG2	67
255DD	FBoxB	67	255E2	FBoxXor	67
255E7	LBoxW	67	255EC	LBoxG1	67
255F1	LBoxG2	67	255F6	LBoxB	67
255FB	LBoxXor	67	25617	(SetNUsrKeyOK)	122
2561C	(ClrNUsrKeyOK)	122	25621	(NonUsrKeyOK?)	122
2564D	SetNAppKeyOK	130	2565A	DoStdKeys?	130
2565F	SetDoStdKeys	130	25671	SetAppSuspOK	131
25676	ClrAppSuspOK	131	25690	AppMode?	130
25695	SetAppMode	130	2569A	ClrAppMode	130
256BE	NOBLINK	107	256EA	AlgEntry?	158
25719	SetAlgEntry	158	2571E	ClrAlgEntry	158
2577F	(TogInsert)	169	25790	INSERT?	169
25795	INSERT_MODE	169	257A2	EditLExists?	168
2580E	SetRebuild	163	25845	MenuDef@	163

地址	名称	页码	地址	名称	页码
25863	MenuRowAct!	163	25877	LabelDef!	163
25886	DoLabel	68, 164	2588B	MenuKeyNS!	163
25890	MenuKeyNS@	163	2589A	DoMenuKeyNS	165
2589F	MenuKeyLS!	163	258B3	MenuKeyRS!	163
258C7	ReviewKey!	164	258EF	(ExitAction!)	164
25908	LastMenuDef!	163	2590D	LastMenuDef@	163
25967	GetUserKeys	122	25E67	!*triand	46
25E68	!*trior	46	25E69	%+SWAP	33
25E6A	'DoBadKey	84	25E6B	'DoBadKeyT	84
25E6D	1stkdecomp\$w	46	25E6E	2DropBadKey	122
25E6F	>Review\$	46	25E70	?ATTN_QUIT	122
25E70	?ATTNQUIT	122	25E71	?BlinkCursor	107
25E72	?CaseKeyDef	90	25E73	?CaseRomptr@	90
25E74	?ClrAlg	158	25E75	?ClrAlgSetPr	158
25E77	?OKINALG	118	25E78	?PURGE_HERE	102
25E79	?STO_HERE	101	25E79	XEQSTOID	101
25E7C	AND\$	45	25E7E	BLANKIT	157
25E7F	Box/StdLabel	67	25E80	Box/StdLbl:	67
25E81	C%1/	37	25E82	C%>%%	37
25E83	C%>%%SWAP	37	25E84	C%ABS	37
25E85	C%ACOS	38	25E86	C%ACOSH	38
25E87	C%ALOG	38	25E88	C%ARG	38
25E89	C%ASIN	38	25E8A	C%ASINH	38
25E8B	C%ATAN	38	25E8C	C%ATANH	38
25E8D	C%COS	38	25E8E	C%COSH	38
25E8F	C%C^C	37	25E90	C%C^R	37
25E91	C%EXP	38	25E92	C%LN	38
25E93	C%LOG	38	25E94	C%R^C	37
25E95	C%SGN	38	25E96	C%SIN	38
25E97	C%SINH	38	25E98	C%SQRT	37
25E99	C%TAN	38	25E9A	C%TANH	38
25E9B	C>Im%	37	25E9C	C>Re%	37
25E9D	CKOATTNABORT	122	25E9E	CK1NoBlame	117
25E9F	CKREF	103	25EA0	COERCE\$22	45
25EA1	CREATEDIR	102	25EA2	CRUNCH	63
25EA6	CheckMenuRow	165	25EA7	Ck&DecKeyLoc	121
25EA9	CodePl>%rc.p	121	25EAA	DECOMP\$	47
25EAB	DISPROW1*	159	25EAC	DISPROW2*	159
25EAD	DISPSTATUS2	159	25EAE	DO#EXIT	97
25EAF	DO\$EXIT	97	25EB0	DO%EXIT	97
25EB1	DO>STR	47	25EB2	DOBEEP	107
25EB3	DOCHR	44	25EB4	DODISP	159
25EB5	DORCLE	175	25EB6	DOSTOE	175
25EB7	DOSTR>	46	25EB8	DOTVARS%	102
25EB9	DOVARS	102	25EBA	DPRADIX?	107
25EBB	DUPGROBDIM	65	25EBC	Disp5x7	159
25EBE	Dolst/2nd+:	158	25EBF	DoBadKey	122
25EC1	DoDelim	169	25EC2	DoDelims	169
25EC3	DoFirstRow	165	25EC4	DoHere:	102
25EC5	DoKeyOb	122	25EC6	DoMenuKey	164
25ECD	DropBadKey	122	25ECE	EDITDECOMP\$	47
25ECF	EQUATION	175	25ED2	EditMenu	172
25ED3	EqList?	56	25ED4	FlashMsg	160
25ED5	GBUFFGROBDIM	155	25ED6	GETKEY	121
25ED7	GETKEY*	121	25ED8	GROB>GDISP	65
25ED9	GetKeyOb	122	25EDA	GetMenu%	163
25EDB	GetNextToken	46	25EDC	H/W>KeyCode	121
25EDE	HARDHEIGHT	155	25EDF	ImmedEntry?	158
25EE0	InitMenu	164	25EE1	InitMenu%	164
25EE2	InitTrack:	163	25EE3	KEYINBUFFER?	122
25EE4	KeepUnit	61	25EE5	Key>StdKeyOb	122
25EE6	Key>U/SKeyOb	122	25EE7	LastNonNull	102
25EE8	LoadTouchTbl	164	25EE9	LockAlpha	158
25EEA	ModifierKey?	121	25EEB	NEXTLIBBAK	69
25EEC	NULL\$TEMP	44	25EED	NoAttn?Semi	122
25EEE	NoEdit?case	92	25EEF	NoExitAction	164
25EF0	OR\$	45	25EF1	PATHDIR	102
25EF2	PrevNonNull	102	25EF3	RAD?	107
25EF4	RECLAIMDISP	155	25EF7	SAFE@_HERE	101
25EF8	SEP\$NL	45	25EFB	SaveLastMenu	163
25EFC	SetKeysNS	163	25EFD	SetSomeRow	165
25EFE	SetThisRow	164	25EFF	SolvMenuInit	165
25F00	StartMenu	164	25F01	Std/BoxLabel	67
25F02	StdMenuKeyLS	163	25F03	StdMenuKeyNS	163
25F04	SuspendOK?	130	25F08	UnLockAlpha	158
25F09	UserKeys?	122	25F0B	WaitForKey	122

地址	名称	页码	地址	名称	页码
25F0C	XEQStoKey	101	25F0D	XOR\$	45
25F0E	XYGROBDISP	65	25F0F	a%>\$	44
25F10	ederr	98	25F11	editdecomp\$w	47
25F12	sstDISP	159	25F13	stkdecomp\$w	46
25F14	XEQPGDIR	102	25F15	>FONT	160
25F17	GetMetaVStackDROP	99	25F18	GetVStack	99
25F19	PopMetaVStack	100	25F1A	PopMetaVStackDROP	99
25F1B	PopVStack	99	25F1C	PopVStackAbove	100
25F1D	PushMetaVStack&Dro	99	25F1E	PushVStack	99
	P				
25F1F	PushVStack&Clear	99	25F20	PushVStack&Keep	100
25F21	PushVStack&KeepDRO	100	25F22	RestoreSysFlags	106
	P				
25F23	SaveSysFlags	106	25F24	RIGHT\$3x6	68
25F25	CKNNOLASTWD	116	25F29	(EvalNoCK:)	117
25F2B	CHECKMENU	165	25F72	#:>\$	44
25F77	#>\$	44	25F7C	\$>GROB	68
25F81	\$>grob	68	25F86	\$>GROBCR	68
25F8B	\$>grobCR	68	25F90	>LANGUAGE	107
25F95	LANGUAGE>	107	25F9A	OLASTOWDOB!	117
25F9A	OLASTOWDOB!、0LastR	117	25FA4	ABUFF	155
	omWrd!				
25FA9	ALARM?	105	25FAE	ATTN?	122
25FB3	BIGDISPN	159	25FB3	DISPN	159
25FB8	BIGDISPROW1	159	25FB8	DISP@01	159
25FB8	DISPROW1	159	25FBD	BIGDISPROW2	159
25FBD	DISP@09	159	25FBD	DISPROW2	159
25FC2	BIGDISPROW3	159	25FC2	DISP@17	159
25FC2	DISPROW3	159	25FC7	BIGDISPROW4	159
25FC7	DISP@25	159	25FC7	DISPROW4	159
25FCC	DISPROW5	159	25FD1	DISPROW7	159
25FD6	DISPROW8	159	25FDB	DISPROW9	159
25FE0	DISPROW10	159	25FE5	DISPLASTROW	160
25FEA	DISPLASTROWBUT1	160	25FEF	CENTER\$3x5	68
25FF4	CENTER\$5x7	68	25FF9	LEFT\$3x5	68
25FFE	LEFT\$5x7	68	26003	LEFT\$5x7Arrow	68
026004	^IfGetFieldValue	148	26008	LEFT\$3x5Arrow	68
2600D	LEFT\$5x7CRArrow	68	26012	LEFT\$3x5CRArrow	68
26017	LEFT\$5x7CR	68	2601C	LEFT\$3x5CR	68
26021	CLEARVDISP	157	2602B	COERCEFLAG	87
26030	CURSOR_OFF	168	26035	ClrAlphaAnn	158
2603A	ClrLeftAnn	158	2603F	ClrRightAnn	158
26044	ClrSysFlag	106	26049	ClrUserFlag	106
2604E	DOENG	107	26053	DOFIX	107
26058	DOSCI	107	2605D	DOSTD	107
26067	ERRBEEP	97	2606C	ERASE&LEFT\$5x7	68
26071	ERASE&LEFT\$3x5	68	26076	GBUFF	155
2607B	GROB!	65	26080	GROB! ZERO	65
26085	GROBDIM	65	2608F	HARDBUFF	155
26094	HARDBUFF2	155	26099	HEIGHTENGROB	155
2609E	INVGROB	66	260A3	KILLGDISP	155
260A8	LastMenuRow!	163	260AD	LastMenuRow@	163
260B2	MAKEGROB	66	0260B3	(BBReReadWidth)	141
260B7	MenuRow!	163	260BC	MenuRow@	163
260C1	NOHALTERR	98	260C6	NOTLISTcase	92
260CB	NOTROMPcase	92	260D0	NOTSEC0case	92
260D5	PIXOFF3	66	260DA	PIXON3	66
260DF	PIXOFF	66	260E4	PIXON	66
260E9	PIXON?3	66	260EE	PIXON?	66
260F3	PULLCMPEL	52	260F8	PULLREALEL	52
260FD	PUTCMPEL	52	26102	PUTEL	52
26107	PUTREALEL	52	2610C	PrgmEntry?	158
26111	RDUP	84	26116	SETCIRCERR	98
26120	SLOW	104	26125	VERYSLOW	104
2612A	VERYVERYSLOW	104	2612F	SUBGROB	65
26134	SYNTAXERR	98	26139	SetAlphaAnn	158
2613E	SetLeftAnn	158	26143	SetPrgmEntry	158
26148	SetRightAnn	158	2614D	SetSysFlag	106
26152	SetUserFlag	106	26166	TOADISP	155
2616B	TOGDISP	155	26170	TestSysFlag	106
26175	TestUserFlag	106	2617F	WINDOWCORNER	158
26184	WINDOWDOWN	158	26189	WINDOWLEFT	158
2618E	WINDOWRIGHT	158	26193	WINDOWUP	158
26198	WINDOWXY	158	261A2	rstfmt1	107
261A7	savefmt1	107	261AC	setbeep	107
261BB	tok8trior	46	261C0	CLCD10	157

地址	名称	页码	地址	名称	页码
261C5	CLEARLCD	157	261CA	FLUSH	121
261CA	FLUSHKEYS	121	261CF	%>C%	37
261D4	C%%0=	38	261D9	C%%>C%	37
261DE	C%%CHS	38	261E3	C%%CONJ	38
261E8	C%0=	38	261ED	C%CHS	37
261F2	C%CONJ	38	261F7	DISPROW6	159
261FC	Re>C%	37	26206	tok8cktrior	46
2620B	>MINIFONT	160	26215	DropVStack	100
2621A	FONT>	160	26224	GetElemBotVStack	100
26229	GetElemTopVStack	100	2622E	SetVStackProtectWo rd	100
26233	GetVStackProtectWo rd	100	26242	GetFontStkHeight	160
26242	StackFontHeight	160	26247	GetHeader	155
2624C	GetMetaVStack	100	2625B	MINIFONT>	160
26260	nDISPSTACK	156	26265	PushMetaVStack	99
2626A	PutElemBotVStack	100	2626F	PutElemTopVStack	100
26283	SetHeader	155	26288	StackLineHeight	160
26292	CK0NOLASTWD	116	26297	CK1NOLASTWD	116
2629C	CK2NOLASTWD	116	262A1	CK3NOLASTWD	116
262A6	CK4NOLASTWD	116	262AB	CK5NOLASTWD	116
262B0	CK0	116	262B5	CK1	116
262BA	CK2	116	262BF	CK3	116
262C4	CK4	116	262C9	CK5	116
262CE	CKN	116	262D8	SETSIZEERR	98
262DD	SETTYPEERR	98	262E2	SETSTACKERR	98
262E7	SETNONEXTERR	98	262EC	%ABSCOERCE	26
262F1	COERCE	26	262F6	UNCOERCE	32
262FB	COMPEVAL	84	26300	CK1&Dispatch	116
26305	CK2&Dispatch	116	2630A	CK3&Dispatch	116
2630F	CK4&Dispatch	116	26314	CK5&Dispatch	117
26319	EvalNoCK	117	2631E	CK&DISPATCH0	116
26323	CK&DISPATCH2	116	26328	CK&DISPATCH1	116
2632D	#*OVF	27	26459	setStdWid	46
2645E	setStdEditWid	46	2649F	(ClrBusyAnn)	158
264CC	FIRSTC@	168	26585	CURSOR@	168
26594	(CURSOR_PART)	168	265B7	Z>	182
265BC	Z<	182	265C1	Z=	182
265C6	Z<>	182	265CB	Z>=	182
265D0	Z<=	182	26F36	%1+	33
26F4A	%1-	33	027004	^IfGetCurrentField Value	148
270EE	(%1.8)	30	27103	%80	31
27118	%1	30	27155	'IDX	77
2716D	StdIOPAR	108	27195	CRLF\$	42
271F4	(2NULLLAM{ })	79	27208	(3NULLLAM{ })	79
272D9	tok->	43	272F3	(ID_EQ)	77
272FE	NULLID	77	275C6	TakeOver	164
275EE	Modifier	164	275FD	MenuKey	165
27620	MenuMaker	164	2768E	(#60E)	23
27698	(#60F)	23	276AC	(#611)	23
276B6	(#612)	23	276C0	(#613)	23
276CA	(#614)	23	276D4	(#615)	23
276DE	(#616)	23	276E8	(#617)	23
276F2	(#622)	23	276FC	(#623)	23
27706	(#624)	23	27710	(#628)	23
2771A	(#629)	23	27724	(#62A)	23
2772E	(#62B)	23	27738	(#62C)	23
27742	(#62D)	23	2774C	(#A01)	24
27756	(#A02)	24	27760	(#A04)	24
2776A	(#A05)	24	27774	(#A06)	24
2777E	(#12F)	22	27788	(#62E)	23
27792	(#618)	23	2779C	(#619)	23
277A6	(#61A)	23	277B0	(#61B)	23
277BA	(#61C)	23	277C4	(#61D)	23
277CE	(#61E)	23	277D8	(#61F)	23
277E2	(#620)	23	277EC	(#621)	23
277F6	(#605)	22	27800	(#606)	22
2780A	(#607)	22	27814	(#608)	22
2781E	(#609)	22	27828	(#60A)	23
27832	(#60B)	23	2783C	(#60C)	23
27846	(#60D)	23	27878	(#800)	23
27882	Attn#	24	27937	(ID_SIGMADAT)	77
27A89	%%2PI	32	27AA3	(NULLGROB)	65
27B43	'IDFUNCTION	84	27B6B	'IDPOLAR	85
27B7F	'IDPARAMETER	85	27D3F	CROSSGROB	65

地址	名称	页码	地址	名称	页码
27D5D	MARKGROB	65	27D7B	(StdLabelGrob)	65
27DBF	C%-1	37	27DE4	C%0	37
27E09	C%1	37	27E2E	C%1	37
27E5D	%100	31	27E72	nohalt	131
27E87	TrueTrue	87	27E9B	failed	88
27EAF	<SkipKey	173	27EFB	>SkipKey	173
27F47	<DelKey	173	27F9A	>DelKey	173
27FED	NullMenuKey	164	28001	ROT#1+UNROT	28
028004	^IfGetFieldMessage Handler	148	28071	#1-SWAP	27, 58
28071	pull	27, 58	28085	pullrev	58
28099	SWAP#1+SWAP	28	280AD	SWAP#1-SWAP	28
0280B3	(BBRunENTERAction)	141	280C1	ORDERXY#	66
280F8	ORDERXY%	66	2812F	SWAPTRUE	88
28143	NDUPN	71	28187	reversym	72
281D5	#1-UNROT	27	28211	NDROPFALSE	58, 88
28225	(9UNROLL)	73	2825E	(2NELCOMPDROP)	54
282CC	(DROP%0)	32	283E8	FalseFalse	87
283FC	ISTOP-INDEX	95	286E7	symcomp	63
28989	(SWAPDROP#1-)	28	28ACE	(DROP?symcomp)	63
029004	^IfGetFieldType	148	0290B3	(BBRunCancelAction)	141
293A3	(?symcomp)	63	293F8	P{ }N	55
2962A	RDROPFALSE	88	2963E	psh&	58
29693	psh1top&	58	296A7	top&	58
2973B	pshtop&	58	29754	pullpsh1&	59
29786	('RSWAP#1+)	28	29821	psh1&	58
298C0	psh1&rev	59	29972	pshzer	59
29986	pshzerpsharg	64	29A5D	psh	58
29A8F	roll2ND	58	29B12	unroll2ND	58
29CB9	uncrunch	63	29D18	ONE{ }N	56
29E67	nultrior	46	29E99	tok=casedrop	92
29ED0	'Rapndit	85	29F25	AppDisplay!	130
29F35	AppDisplay@	130	29F55	AppKeys!	130
29F75	AppKeys0	130	02A004	^IfGetFieldObjects Type	148
2A055	AppExitCond!	130	2A065	AppExitCond@	130
2A085	(Clipboard!)	171	2A095	(Clipboard@)	171
2A0A5	(Clipboard0)	171	2A0B5	(Clipboard?)	171
2A145	AppError!	130	2A158	AppError@	130
2A5CA	SUB\$1#	45	2A7F7	DispTimeReq?	156
2A842	DecompLine	46	2A85D	DecompEdit	47
2A878	Decomp#Line	46	2A893	Decomp#Disp	46
2A8AE	DecompEcho	47	2A8C9	DecompStd1Line	46
2A8E4	DecompStd1Line32	46	2A904	RPNDecomp1Line	46
2A924	RPNDecompEdit	47	2A944	RPNDecomp#Line	46
2A964	RPNDecomp#Disp	46	2A984	RPNDecompEcho	47
2A9A4	RPNDecompStd1Line	46	2A9C4	RPNDecompStd1Line3 2	46
2A9E9	RunRPN:	117	2AA43	AlgDecomp	47
2AB69	RunInApprox	106	2ABD7	RunSafeFlagsNoErro r	106
2ABF0	RunSafeFlags	106	2AC0E	DoRunSafe	106
2AD81	EQUALcasedrop	92	2ADBD	nonopcase	92
2ADE0	num1stcase	92	2AE32	M-1stcasechs	91
2AF37	AEQ1stcase	91	2AFFB	MEQ1stcase	91
02B004	^IfGetFieldDecompO bject	148	2B01B	MEQopscase	91
2B06A	AEQopscase	91	2B083	Mid1stcase	91
2B0CC	idntcase	92	2B0EF	idntlamcase	92
2B11C	num0=case	91	2B149	%0=case	91
2B15D	C%0=case	91	2B176	num1=case	91
2B1A3	%1=case	91	2B1C1	C%1=case	91
2B1DF	num2=case	91	2B20C	%2=case	91
2B22A	C%2=case	91	2B25C	num-1=case	91
2B289	%-1=case	91	2B2A7	C%-1=case	91
2B2C5	NOTcaseFALSE	90	2B2F2	(CallEditCmd:)	172
2B3A6	1NULLLAM{ }	79	2B3AB	NULLLAM	77
2B3B7	4NULLLAM{ }	79	2B3FD	%IP>#	32
2B42A	PUTLIST	56	2B475	ParOuterLoop	130
2B4AC	POLSaveUI	130	2B542	POLSetUI	130
2B628	POLKeyUI	130	2B6B4	POLResUI&Err	130
2B6CD	POLRestoreUI	130	2B8BE	{OBJ>R}	84
2B8E6	{R>OBJ}	84	2B90B	'DROPFALSE	84
2BB21	sscknum2	64	2BB3A	sncknum2	64
2BB53	nscknum2	64	2BCA2	cknumdsptch1	64
2BD8C	(SYMBN:)	63	2BE36	(ALGcase)	92

地址	名称	页码	地址	名称	页码
2BF3A	DA1OK?NOTIT	156	2BF53	DA2aOK?NOTIT	156
2BF6C	DA2bOK?NOTIT	156	2BF85	DA3OK?NOTIT	156
02C004	^IfGetFieldChooseData	148	2C2C0	nCustomMenu	165
2C2F9	DispStsBound	156	2C305	DispStatus	156
2C311	?DispStatus	156	2C341	?DispStack	155
2C371	DoInputForm	147	2C4D2	(#AAA0)	25
02D004	^IfGetFieldChooseDecomp	148	2D74F	um*	60
2D759	um/	60	2D763	um^	60
2D76D	umP	60	2D777	umEND	60
2D848	tok_g	43	2D86D	tok_m	43
2D8AD	tok_s	43	2D949	UMSIGN	61
2D95D	UMIP	61	2D971	UMFP	61
2D985	UMFLOOR	61	2D999	UMCEIL	61
2D9CB	UMRND	61	2D9EE	UMTRC	61
2DA11	cfF	32	2DA2B	cfC	32
2DFCC	?DispMenu	155, 164	2DFE0	DispMenu	155, 164
2DFF4	DispMenu.1	155, 164	02E002	^DoAlert	160
02E004	^IfGetFieldResetValue	148	2E0D5	Grob>Menu	67, 164
2E0F3	Str>Menu	67, 164	2E107	Seco>Menu	67, 164
2E11B	Id>Menu	67, 164	2E166	MakeStdLabel	67
2E189	MakeBoxLabel	67	2E1EB	MakeDirLabel	67
2E24D	MakeInvLabel	67	2E25C	(InvLabelGrob)	65
2E2AA	MakeLabel	68, 164	2EE5A	DispEditLine	155
2EE5C	BlankDA12	157	2EE5D	?FlashAlert	107
2EE60	DoWarning	160	2EE61	FlashWarning	160
2EE62	DA1OK?	156	2EE63	DA3OK?	156
2EE64	SetDAsTemp	156	2EE65	SetDA12a3NCh	157
2EE65	SetDA12a3NoCh	157	2EE66	DA2aLess1OK?	156
2EE67	SetDA1Valid	156	2EE68	SetDA2bValid	156
2EE69	SetDA1Temp	156	2EE6A	SetDA2bTemp	156
2EE6B	SetDA3Temp	156	2EE6C	SetDA2aEcho	157
2EE6D	ClrDAsOK	156	2EE6E	ClrDA3OK	156
2EE6F	SetDA12NoCh	157	2EE70	SetDA13NoCh	157
2EE71	SetDA12Temp	156	2EE72	SetDA1NoCh	157
2EE73	SetDA2aNoCh	157	2EE74	ClrDA1Bad	156
2EE75	ClrDA2aBad	156	2EE76	SetDA2bNoCh	157
2EE77	SetDA3NoCh	157	2EE78	SetDA1Bad	156
2EE79	SetDA2aBad	156	2EE7A	SetDA2bBad	157
2EE7B	SetDA3Bad	157	2EE7C	SetDAsNoCh	157
2EE7D	ClrDA1IsStat	155	2EE7E	DA2bIsEdL?	157
2EE7F	SetDA2bIsEdL	157	2EE80	ClrDA2bIsEdL	157
2EE81	ClrDA2bNoCh	157	2EE8A	SetDA2aTemp	156
2EE8D	ClrDA1OK	156	2EE8E	ClrDA2aOK	156
2EE8F	ClrDA2bOK	156	2EE90	ClrDA2OK	156
2EE91	SetDA2Valid	156	2EE93	SetDA2NoCh	157
2EE94	SetDA23NoCh	157	2EEA0	SetDA3ValidF	156
2EEA5	SetDA2bTempF	156	2EEA7	ClrDA2bTemp	156
2EEAB	DA1IsStatus?	157	2EEAC	SetDA1IsStat	157
2EEAE	SetNoRollDA2	157	2EEAF	ClrNoRollDA2	157
2EEB0	DA1Bad?	156	2EEB1	DA2aBad?	157
2EEB2	DA2bBad?	157	2EEB3	ClrDA2bBad	157
2EEB4	DA3Bad?	157	2EEB5	ClrDA3Bad	157
2EEB7	DA2bNoCh?	157	2EEBB	SENDLIST	108
2EEBC	GETNAME	108	2EEBD	DOFINISH	108
2EEBE	DOPKT	108	2EEBF	GetIOPAR	108
2EEC1	DOBAUD	108	2EEC2	DOPARITY	108
2EEC3	DOTRANSIO	108	2EEC4	DOKERRM	108
2EEC5	DOBUFLEN	108	2EEC6	DOSBRK	108
2EEC7	DOSRECV	108	2EEC9	CLOSEUART	108
2EECB	DOCR	108	2EECD	DODELAY	108
2EECF	TOD	104	2EED0	DATE	104
2EED1	DDAYS	104	2EED2	DATE+DAYS	104
2EED3	TIMESTR	45, 104	2EED4	Clr8	157
2EED5	Clr8-15	157	2EED6	HBUFF_X_Y	158
2EED7	CLKTICKS	104	2EED7	SysTime	104
2EEDA	STATCLST	53	2EEDB	STATSADD%	53
2EEDC	STATN	53	2EEDD	STATSMAX	53
2EEDF	STATMEAN	53	2EEDF	STATSMIN	53
2EEE0	STATSTDEV	53	2EEE1	STATTOT	53
2EEE2	STATVAR	53	2EEE4	Echo\$Key	169
2EEE5	EditLevell	172	2EEE6	InitEd&Modes	173
2EEE7	InitEdLine	169, 173	2EEE8	InitEdModes	173
2EEE9	EditString	172	2EEEA	CURSOR_END?	168

地址	名称	页码	地址	名称	页码
2EEEE	EDITLINE\$	168	2EED	NoEditLine?	168
2EEF	AUTOSCALE	175	2EEF0	PromptIdUtil	45
2EEF1	PUTSCALE	175	2EEF2	PUTINDEP	174
2EEF3	PUTINDEPLIST	174	2EEF4	PUTRES	175
2EEF5	GETPTYPE	174	2EEF6	PUTPTYPE	174
2EEF9	DOERASE	155	2EEFC	MENUOFF	155
2EEFD	MENUOFF?	155	2EEFF	DispCoord1	159
2EF01	DOPX>C	175	2EF02	DOC>PX	175
2EF03	DOLCD>	66	2EF04	DO>LCD	66
2EF05	DOCLLCD	157	2EF06	CKPICT	174
2EF07	nmetasyms	117	2EF26	SYMSHOW	64
2EF5A	apndvarlst	56	2EF5E	BlankDA1	157
2EF5F	InputLine	124	2EF60	DOGRAPHIC	175
2EF61	WINDOW#	159	2EF62	palparse	46
2EF66	SysMenuCheck	164	2EF67	SysDisplay	155
2EF6C	AtUserStack	117	2EF6E	ParseFail	46
2EF6F	DispBadToken	46	2EF73	?Space/Go>	172
2EF74	CMD_PLUS	169	2EF75	AddTrailingSpace	172
2EF76	AddLeadingSpace	172	2EF77	CMD_PAGE_L	170
2EF78	CMD_PAGER	170	2EF79	CMD_PAGE_U	170
2EF7A	CMD_PAGED	170	2EF7B	CMD_BAK	170
2EF7C	CMD_NXT	170	2EF7D	CMD_DEB_LINE	170
2EF7E	CMD_END_LINE	170	2EF7F	CMD_UP	170
2EF80	CMD_DOWN	170	2EF81	CMD_DROP	169
2EF82	CMD_DEL	169	2EF83	CMD_STO_DEBUT	170
2EF84	CMD_STO_FIN	170	2EF85	RCL_CMD_DEB	170
2EF86	RCL_CMD_FIN	170	2EF87	RCL_CMD_POS	168
2EF88	CMD_CUT	170	2EF8A	CMD_COPY	170
2EF8B	STO_CURS_POS	169	2EF8C	STO_CURS_POS2	169
2EF8D	STO_CURS_POS3	170	2EF8E	STO_CURS_POS4	170
2EF8F	STO_CURS_POS_VIS	170	2EF90	CAL_CURS_POS_VIS	168
2EF91	CAL_CURS_POS	168	2EF92	XLIN_SIZE?	173
2EF93	VERIF_SELECTION	170	2EF94	PASTE.EXT	171
2EF95	DEL_CMD	169	2EF96	NO_AFFCMD	172
2EF97	InsertEcho	169	2EF98	SetDA2aValid	156
2EF99	SetDA3Valid	156	2EF9A	CommandLineHeight	172
2EF9F	LINEON	66	2EFA0	LINEOFF	66
2EFA1	TOGLINE	66	2EFA2	LINEON3	66
2EFA3	LINEOFF3	66	2EFA4	TOGLINE3	66
2EFA5	DOHEX	106	2EFA6	DOBIN	106
2EFA7	DOOCT	106	2EFA8	DODEC	106
2EFAA	dostws	48	2EFAC	bitAND	48
2EFAD	bitOR	48	2EFAE	bitXOR	48
2EFAF	bitNOT	48	2EFB0	bitSL	49
2EFB1	bitSLB	49	2EFB2	bitSR	49
2EFB3	bitSRB	49	2EFB4	bitRR	49
2EFB5	bitRRB	49	2EFB6	bitRL	49
2EFB7	bitRLB	49	2EFB8	bitASR	48
2EFB9	bit+	48	2EFBA	bit-	48
2EFBC	bit*	48	2EFBD	bit/	48
2EFBE	WORDSIZE	48	2EFBF	BASE	106
2EFC0	HXS>\$	44	2EFC1	hxs>\$	44
2EFC2	bit%#/#	48	2EFC3	bit%#/#	48
2EFC4	bit%#*	48	2EFC5	bit%#*	48
2EFC6	bit%#-	48	2EFC7	bit%#-	48
2EFC8	bit%#+	48	2EFC9	bit%#+	48
2EFCA	HXS>%	32	2EFCB	%>#	48
2EFCC	HXS==HXS	49	2EFCD	HXS>HXS	49
2EFCE	HXS>=HXS	49	2EFCF	HXS<HXS	49
2EFDB	(GROB+)	65	2F002	(Ticks>TOD)	104
2F003	(Ticks>Date)	104	02F004	^IfSetFieldResetVal ue	148
2F004	(Ticks>Rpt)	104	2F019	UNIT>\$	61
2F031	TURNMENUON	155	2F034	TURNMENUOFF	155
2F05E	SaveLastEdit	173	2F062	StoIOPAR	108
2F064	Sys@	101	2F073	SWAPcompSWAP	63
2F076	puretemp?	62	2F07A	UM>U	61
2F07B	U>nbr	61	2F07C	UM#?	62
2F07D	UM%	61	2F07E	UM%CH	61
2F07F	UM%T	61	2F080	UM*	61
2F081	UM+	61	2F082	UM-	61
2F083	UM/	61	2F085	UM<=?	62
2F086	UM<?	62	2F087	UM=?	62
2F088	UM>=?	62	2F089	UM>?	62
2F08A	UMABS	61	2F08B	UMCHS	61
2F08C	UMCONV	61	2F08D	UMCOS	61

地址	名称	页码	地址	名称	页码
2F08E	UMMAX	61	2F08F	UMMIN	61
2F090	UMSI	61	2F091	UMSIN	61
2F092	UMSQ	61	2F093	UMSQRT	61
2F094	UMTAN	61	2F095	UMU>	61
2F096	UMXROOT	61	2F098	Unbr>U	61
2F099	U>NCQ	61	2F09A	TempConv	61
2F0A1	RESETDEPTH	72	2F0AC	PURGALARM%	104
02F0B3	(BBReDrawBackgr)	141	2F0D4	(ILnot?)	117
2F0DB	MAKEPICT#	66	2F0E8	INDEPVAR	174
2F0EC	ICMPDRPRTDRP	56	2F0EE	HXS#HXS	49
2F0EF	HXS<=HXS	49	2F0FE	GETXMAX	174
2F0FF	GETXMIN	174	2F100	GETYMIN	174
2F105	GDISPCENTER	175	2F106	GETINDEP	174
2F107	GETPMIN&MAX	174	2F10D	GETRES	175
2F10E	GETYMAX	174	2F110	(FINDVARS)	63
2F11C	Echo\$NoChr00	169	2F13D	(CK#-)	27
2F13F	DRAWLINE#3	66	2F154	(input\$)	124
2F155	(input{})	124	2F158	(THISCHAR)	168
2F15B	CLEARMENU	165	2F15E	Clr16	157
2F162	CHECKPICT	174	2F163	CHECKPVAR	174
2F16D	Blank\$	45	2F178	ALARMS@	104
2F17E	2HXSLIST?	26	2F180	1REV	32
2F190	DcompWidth@	46	2F191	!DcompWidth	46
2F193	UobROT	59	2F194	CMD_PLUS2	169
2F195	CMD_PLUS3	169	2F196	RCL_CMD	168
2F197	RCL_CMD2	168	2F198	STO_CMD_MODE	169
2F199	RCL_CMD_MODE	169	2F19A	ViewLevel1	172
2F19E	DispCommandLine	155, 172	2F19F	?DispCommandLine	155, 172
2F1A5	AskQuestion	160	2F1A7	CHARSEdit	45
2F1AB	Date>hxs13	104	2F1AE	ObEdit	172
2F1AF	AlgObEdit	172	2F1BF	Decomp%Short	47
2F1C6	DROP3PICK	74	2F215	CircleB	66
2F216	CircleG1	66	2F217	CircleG2	66
2F218	CircleW	66	2F219	CircleXor	67
2F21A	Dither	67	2F21B	ToGray	67
2F21D	ViewObject	159	2F21E	ViewStrObject	159
2F21F	ViewGrobObject	159	2F222	#1+ROT	27
2F223	%>TAG	51	2F23E	DOSTOSYSF	106
2F244	(COERCE&CKSGN)	26	2F24E	LISTRCL	101
2F257	OCRC%	107	2F258	PICTRCL	174
2F259	RCLSYSF	106	2F25A	(RCLSYSF2)	106
2F25B	RCLUSERF	106	2F25C	(RCLUSERF2)	106
2F25F	(STOSYSF)	106	2F260	(STOSYSF2)	106
2F261	(STOUSERF)	106	2F262	(STOUSERF2)	106
2F265	UPDIR	102	2F266	USER\$>TAG	51
2F267	VARSIZE	107	2F268	Wait/GetKey	122
2F296	XEQORDER	102	2F297	XEQPURGEPICT	175
2F2A3	XEQRCL	101	2F2A7	XEQSETLIB	69
2F2A9	XEQSHOWLS	64	2F2D4	dowait	104
2F2DB	DOTEXTINFO	172	2F2DC	ClearSelection	170
2F2DD	DoFarBS	169	2F2DE	DoFarDel	169
2F2DF	EditSelect	171	2F2E0	ViewEditGrob	173
2F2E1	SELECT.LINE	171	2F2E2	SELECT.LINEEND	171
2F2E3	EVAL.LINE	171	2F2E4	DO>BEG	170
2F2E5	DO>END	170	2F2E6	GOTOLABEL	170
2F2E7	SELECT.FONT	173	2F2E8	DOFIND	171
2F2E9	DOREPL	171	2F2EA	DONEXT	171
2F2EB	DOREPLACE	171	2F2EC	DOREPLACE/NEXT	171
2F2ED	REPLACEALL	171	2F2EE	DO<Skip	170
2F2EF	DO>Skip	170	2F2F0	DO<Del	169
2F2F1	DO>Del	169	2F2F2	FindStrInCmd	171
2F2F3	GET.W->	171	2F2F4	GET.W<-	171
2F2F5	PUT_FONTE	173	2F2F6	GET_CUR_FONT.EXT	172
2F2F7	PUT_STYLE	172	2F2F8	EXEC_CMD	171
2F2F9	DODEL.L	169	2F2FA	CMD_COPY.SBR	170
2F2FB	EVAL.SELECTION	171	2F2FC	REPLACEALLNOSCREEN	171
2F300	DispILPrompt	156	2F314	RCLALARM%	104
2F318	1GETLAMSWP1+	79	2F31A	APNDCRLF	44, 108
2F31B	BlankDA2	157	2F31C	BlankDA2a	157
2F31D	BOTROW	158	2F31F	C%>#	26
2F320	CHECKHEIGHT	65	2F321	CkChr00	47
2F324	CKGROBFITS	65	2F326	CMDSTO	173
2F329	Date>d\$	104	2F32B	DISPCOORD2	160
2F32C	DRAWBOX#	66	2F32E	DROPDEADTRUE	84
2F339	GetEqN	175	2F33C	getmatchtok	46
2F33D	GETPARAM	174	2F33E	GETSCALE	175

地址	名称	页码	地址	名称	页码
2F342	GROB+#	65	2F347	JUMPBOT	159
2F348	JUMPLEFT	159	2F349	JUMPRIGHT	159
2F34A	JUMPTOP	159	2F34D	KINVISLF	108
2F34E	KVIS	108	2F34F	KVISLF	108
2F352	LEFTCOL	158	2F355	MAKEPVARs	174
2F356	metatail	59	2F365	PUTXMAX	174
2F366	PUTXMIN	174	2F367	PUTYMAX	174
2F368	PUTYMIN	174	2F36B	RIGHTCOL	158
2F36C	Rows8-15	158	2F36D	SCROLLDOWN	159
2F36E	SCROLLLEFT	159	2F36F	SCROLLRIGHT	159
2F370	SCROLLUP	158	2F378	SetCursor	170
2F379	SetDA123NoCh	157	2F37A	SetDA2OKTemp	156
2F37B	SetIOPARErr	98, 108	2F37E	SORTASLOW	104
2F37F	STOALM	104	2F380	SysSTO	101
2F381	TOD>t\$	104	2F382	TOGGLELINE#3	66
2F383	TOP16	158	2F384	TOP8	158
2F385	TOPROW	158	2F388	VerifyTOD	104
2F389	VERSTRING	108	2F38A	WINDOWBOT?	159
2F38B	WINDOWLEFT?	159	2F38C	WINDOWRIGHT?	159
2F38D	WINDOWTOP?	159	2F38E	xnsgeneral	59
2F38F	xsgeneral	59	2F3A9	{STOALLF}	106
2F3AA	{STOALLF2}	106	2F3B3	{AsnKey}	122
2F458	SETIVLEERR	98	2F937	%0	30
2F94C	%1	30	2F961	%2	30
2F976	%3	30	2F98B	%4	30
2F9A0	%5	30	2F9B5	%6	30
2F9CA	%7	31	2F9DF	%8	31
2F9F4	%9	31	2FA09	%-1	30
2FA1E	%-2	30	2FA33	%-3	30
2FA48	%-4	30	2FA5D	%-5	30
2FA72	%-6	30	2FA87	%-7	30
2FA9C	%-8	30	2FAB1	%-9	30
2FAC6	%PI	30	2FADB	%%PI	32
2FAF5	%MAXREAL	32	2FB0A	%-MAXREAL	30
2FB1F	%MINREAL	30	2FB34	%-MINREAL	30
2FB49	%%0	32	2FB63	%%1	32
2FB7D	%%2	32	2FB97	%%3	32
2FBB1	%%4	32	2FBCB	%%5	32
2FBE5	%%.1	32	2FBFF	%%.5	32
2FC19	%%10	32	2FC7D	(%1200)	31
2FC92	(%2400)	31	2FCA7	(%4800)	31
2FCBC	(%9600)	31	2FCD1	(%15360)	31
2FCE6	%11	31	2FCFB	%12	31
2FD10	%13	31	2FD25	%14	31
2FD3A	%15	31	2FD4F	%16	31
2FD64	%17	31	2FD79	%18	31
2FD8E	%19	31	2FDA3	%20	31
2FDB8	%21	31	2FDCE	%22	31
2FDE2	%23	31	2FDF7	%24	31
2FE0C	%25	31	2FE21	%26	31
2FE36	%27	31	2FE4B	(%28)	31
2FE60	(%29)	31	2FE75	(%30)	31
2FE8A	(%31)	31	2FE9F	(%32)	31
2FEB4	(%33)	31	2FEC9	(%34)	31
2FEDE	(%35)	31	2FF9B	%%>%	32
2FFAC	%>%%	32	2FFBD	SETDEG	107
2FFDB	SETRAD	107	2FFEF	SETGRAD	107
030004	^IfGetFieldInterna	148	3000D	%D>R	34
	lValue				
30017	PI/180	32	30040	%R>D	34
3005E	%>HMS	104	30077	%HMS>	104
3008B	%HMS+	104	300B3	%HMS-	104
300C7	%%MAX	35	300E0	%MAX	34
300F9	%MIN	34	30112	%%0<	35
30123	%0<	35	30145	%%0=	35
30156	%0=	35	30173	%%0>	35
30184	%0>	35	301A6	%%0<>	35
301BA	%0<>	35, 87	301CE	%%0>=	36
301E2	%0>=	35	301F6	%%0<=	35
3020A	%%<	35	3025C	%<	35
3026A	%%>	35	30275	%>	35
30280	%%>=	35	3028B	%>=	35
30296	%%<=	35	302A1	%<=	35
302AC	%=	35	302B7	%<>	35
302C2	%SGN	33	302DB	%%ABS	34
302EB	%ABS	33	302FB	%%CHS	35

地址	名称	页码	地址	名称	页码
3030B	%CHS	33	3031B	%MANTISSA	33
3032E	%%+	34	3033A	%%-	34
30346	%>%%-	33	3035F	%+	33
3036C	%-	33	30385	%*	34
303A7	%*	33	303B4	%OF	34
303D3	%%/	34	303E9	%/	33
303F6	%T	34	3041B	%CH	34
3044A	%%^	34	3045B	%^	33
3046C	%NROOT	34	3047D	%%1/	35
30489	%>%%1/	33	3049A	%1/	33
304D5	%SQRT	35	304E1	%>%SQRT	33
304F4	%SQRT	33	30507	%EXP	35
3051A	%EXP	33	3052D	%EXPM1	33
30546	%LN	35	30559	%LN	33
3056C	%LOG	33	3057F	%LNP1	35
30592	%LNP1	33	305A5	%ALOG	33
305C7	%MOD	34	305DA	%SIN	33
305F1	%SIN	35	30602	%SINDEG	35
30612	%SINRAD	34	3062B	%COS	33
30642	%COS	35	30653	%COSDEG	35
30663	%COSRAD	35	3067C	%TAN	33
30693	%TANRAD	35	306AC	%ASIN	33
306C3	%ASINRAD	35	306DC	%ACOS	33
306F3	%ACOSRAD	34	3070C	%ATAN	33
30723	%ANGLE	34	3073A	%ANGLE	35
30746	%>%ANGLE	34	30757	%ANGLEDEG	35
30767	%ANGLERAD	34	30780	%SINH	35
30799	%SINH	33	307B2	%COSH	35
307C5	%COSH	33	307D8	%TANH	33
307EB	%ASINH	33	307FE	%ACOSH	33
30811	%ATANH	33	30824	%EXPONENT	33
30837	%NFACT	34	3084D	%COMB	34
30860	%PERM	34	30912	%H>HMS	104
30938	%FP	33	3094B	%IP	33
3095E	%CEIL	34	30971	%FLOOR	33
30984	%FLOOR	35	30984	%INT	35
309AD	%RAN	34	30A2F	%RANDOMIZE	34
30A66	DORANDOMIZE	34	30AAF	%FACT	34
30BEA	%7	32	30CC7	%12	32
30CEB	%60	32	30DC8	%%.4	32
30E47	2%-%%	32	30E5B	2%>%	32
30E79	%REC>%POL	34	30E83	%R>P	35
30EA6	%POL>%REC	34	30EB0	%P>R	35
30EDD	%SPH>%REC	34	30F14	RNDXY	34
30F28	TRCXY	34	031004	^IfDisplayFromData	148
032004	^IfGetNbFields	148	033004	^IfCheckSetValue	148
33107	any	17	33107	BINT0	17
33107	ZERO	17	33111	BINT1	17
33111	MEMERR	17	33111	ONE	17
33111	real	17	3311B	BINT2	17
3311B	cmp	17	3311B	TWO	17
33125	BINT3	17	33125	str	17
33125	THREE	17	3312F	array	17
3312F	BINT4	17	3312F	FOUR	17
33139	BINT5	17	33139	FIVE	17
33139	list	17	33143	BINT6	17
33143	id	17	33143	SIX	17
3314D	BINT7	17	3314D	lam	17
3314D	SEVEN	17	33157	BINT8	17
33157	EIGHT	17	33157	seco	17
33161	BINT9	17	33161	NINE	17
33161	symb	17	3316B	BINT10	17
3316B	sym	17	3316B	TEN	17
33175	BINT11	17	33175	ELEVEN	17
33175	hxs	17	3317F	BINT12	17
3317F	grob	17	3317F	TWELVE	17
33189	BINT13	17	33189	TAGGED	17
33189	THIRTEEN	17	33193	BINT14	17
33193	EXT	17	33193	FOURTEEN	17
33193	unitob	17	3319D	BINT15	18
3319D	FIFTEEN	18	3319D	rompointer	18
331A7	BINT16	18	331A7	REALOB	18
331A7	SIXTEEN	18	331B1	2REAL	18
331B1	BINT17	18	331B1	REALREAL	18
331B1	SEVENTEEN	18	331BB	BINT18	18
331BB	EIGHTEEN	18	331C5	BINT19	18

地址	名称	页码	地址	名称	页码
331C5	NINETEEN	18	331CF	BINT20	18
331CF	TWENTY	18	331D9	BINT21	18
331D9	TWENTYONE	18	331E3	BINT22	18
331E3	TWENTYTWO	18	331ED	BINT23	18
331ED	TWENTYTHREE	18	331F7	BINT24	18
331F7	TWENTYFOUR	18	33201	BINT25	18
33201	TWENTYFIVE	18	3320B	BINT26	18
3320B	REALSYM	18	3320B	TWENTYSIX	18
33215	BINT27	18	33215	TWENTYSEVEN	18
3321F	BINT28	18	3321F	TWENTYEIGHT	18
33229	BINT29	18	33229	TWENTYNINE	18
33233	BINT30	18	33233	REALEXT	18
33233	THIRTY	18	3323D	BINT31	18
3323D	THIRTYONE	18	33247	BINT32	18
33247	THIRTYTWO	18	33251	BINT33	18
33251	THIRTYTHREE	18	3325B	BINT34	18
3325B	THIRTYFOUR	18	33265	BINT35	18
33265	THIRTYFIVE	18	3326F	BINT36	18
3326F	TTHIRTSIX	18	33279	BINT37	18
33279	THIRTYSEVEN	18	33283	BINT38	18
33283	THIRTYEIGHT	18	3328D	BINT39	18
3328D	THIRTYNINE	18	33297	BINT40	18
33297	FORTY	18	33297	FOURTY	18
332A1	BINT41	18	332A1	FORTYONE	18
332AB	BINT42	18	332AB	FORTYTWO	18
332B5	BINT43	18	332B5	FORTYTHREE	18
332BF	BINT44	18	332BF	FORTYFOUR	18
332C9	BINT45	18	332C9	FORTYFIVE	18
332D3	BINT46	18	332D3	FORTYSIX	18
332DD	BINT47	18	332DD	FORTYSEVEN	18
332E7	BINT48	18	332E7	FORTYEIGHT	18
332F1	BINT49	18	332F1	FORTYNINE	18
332FB	BINT50	18	332FB	FIFTY	18
33305	BINT51	19	33305	FIFTYONE	19
3330F	BINT52	19	3330F	FIFTYTWO	19
33319	BINT53	19	33319	FIFTYTHREE	19
33319	STRLIST	19	33319	THREEFIVE	19
33323	BINT54	19	33323	FIFTYFOUR	19
3332D	BINT55	19	3332D	FIFTYFIVE	19
33337	BINT56	19	33337	FIFTYSIX	19
33341	BINT57	19	33341	FIFTYSEVEN	19
3334B	BINT58	19	3334B	FIFTYEIGHT	19
33355	BINT59	19	33355	FIFTYNINE	19
3335F	BINT60	19	3335F	SIXTY	19
33369	BINT61	19	33369	SIXTYONE	19
33373	BINT62	19	33373	SIXTYTWO	19
3337D	BINT63	19	3337D	SIXTYTHREE	19
33387	BINT40h	19	33387	BINT64	19
33387	SIXTYFOUR	19	33387	YHI	19
33391	ARRYREAL	19	33391	BINT65	19
3339B	BINT66	19	3339B	FOUR TWO	19
333A5	BINT67	19	333A5	FOURTHREE	19
333AF	BINT68	19	333AF	SIXTYEIGHT	19
333B9	BINT69	19	333B9	FOURFIVE	19
333C3	BINT70	19	333C3	SEVENTY	19
333CD	BINT71	19	333D7	BINT72	19
333E1	BINT73	19	333EB	BINT74	19
333EB	SEVENTYFOUR	19	333F5	BINT75	19
333FF	BINT76	19	33409	BINT77	19
33413	BINT78	19	3341D	BINT79	19
3341D	SEVENTYNINE	19	33427	BINT80	19
33427	EIGHTY	19	33431	BINT81	19
33431	EIGHTYONE	19	33431	LISTREAL	19
3343B	BINT82	19	3343B	LISTCMP	19
33445	BINT83	19	33445	FIVETHREE	19
3344F	BINT84	19	3344F	FIVEFOUR	19
33459	2LIST	19	33459	BINT85	19
33463	BINT86	19	33463	FIVESIX	19
3346D	BINT87	19	3346D	LISTLAM	19
33477	BINT88	19	33481	BINT89	20
3348B	BINT90	20	33495	BINT91	20
33495	BINT_91d	20	3349F	BINT92	20
334A9	BINT93	20	334B3	BINT94	20
334BD	BINT95	20	334C7	BINT96	20
334C7	BINT_96d	20	334D1	BINT97	20
334D1	IDREAL	20	334DB	BINT98	20

地址	名称	页码	地址	名称	页码
334E5	BINT99	20	334EF	BINT100	20
334EF	ONEHUNDRED	20	334F9	BINT101	20
33503	BINT102	20	3350D	BINT103	20
33517	BINT104	20	33521	BINT105	20
3352B	BINT106	20	33535	BINT107	20
3353F	BINT108	20	33549	BINT109	20
33553	BINT110	20	3355D	BINT111	20
3355D	char	20	33567	BINT112	20
33571	BINT113	20	3357B	BINT114	20
33585	BINT115	20	33585	BINT_115d	20
3358F	BINT116	20	3358F	BINT_116d	20
33599	BINT117	20	335A3	BINT118	20
335AD	BINT119	20	335B7	BINT120	20
335C1	BINT121	20	335CB	BINT122	20
335CB	BINT_122d	20	335D5	BINT123	20
335DF	BINT124	20	335E9	BINT125	20
335F3	BINT126	20	335FD	BINT127	20
33607	BINT128	20	33607	BINT80h	20
33611	BINT129	21	3361B	BINT130	21
3361B	BINT130d	21	3361B	BINT_130d	21
3361B	XHI-1	21	33625	BINT131	21
33625	BINT131d	21	33625	BINT_131d	21
33625	XHI	21	3362F	(#8F)	21
33639	SYMBREAL	21	33643	(#92)	21
3364D	(#9A)	21	33657	SYMBUNIT	21
3366B	SYMOB	21	33675	SYMREAL	21
3367F	(#A2)	21	33689	(#A5)	21
33693	SYMID	21	3369D	SYMLAM	21
336A7	(#A9)	21	336B1	SYMSYM	21
336BB	SYMEXT	21	336C5	(HXSREAL)	21
336CF	(2HXS)	21	336D9	BINTC0h	21
336E3	2GROB	21	336ED	TAGGEDANY	21
336F7	EXTREAL	21	33701	EXTSYM	21
3370B	2EXT	21	33715	ROMPANY	21
3371F	BINT253	21	33729	BINT255d	21
33733	REALOBOB	21	3373D	#_102	21
33747	#SyntaxErr	21	33751	(BINT_263d)	21
3375B	(#110)	21	33765	3REAL	21
3376F	(Err#Kill)	21	33779	(Err#NoLstStk)	21
33783	(#NoRoomForSt)	22	3378D	(#132)	22
33797	(REALSTRSTR)	22	337A1	(#134)	22
337AB	(#135)	22	337B5	(#136)	22
337BF	(#137)	22	337C9	(#138)	22
337D3	(#139)	22	337DD	(#13A)	22
337E7	(#13B)	22	337F1	(#13D)	22
337FB	(#13E)	22	33805	INTEGER337	22
3380F	(#200)	22	33819	(Err#NoLstArg)	22
33823	STRREALREAL	22	3382D	(ARRYREALREAL)	22
33837	(#412)	22	33841	(#444)	22
3384B	(ARRYLISTREAL)	22	33855	(#452)	22
3385F	(#510)	22	33869	(#511)	22
33873	(#550)	22	3387D	(IDREALOB)	23
33887	(IDLISTOB)	23	33891	(#700)	23
3389B	(#861)	24	338A5	(#862)	24
338AF	(#865)	24	338B9	(#86E)	24
338C3	ATTNERR	24	338CD	(#A11)	24
338D7	(#A12)	24	338E1	(#A1A)	24
338EB	(#A21)	24	338F5	(#A22)	24
338FF	(#A2A)	24	33909	(#A61)	24
33913	(#A62)	24	3391D	(#A65)	24
33927	(#A6E)	24	33931	(#AA1)	24
3393B	(#AA2)	24	33945	(#AAA)	24
3394F	(#C06)	24	33959	(#C07)	24
33963	(#C08)	24	3396D	Connecting	24
33977	(#C0B)	24	33981	#CArmErr	24
3398B	EXTOBOB	24	33995	#EXITERR	25
3399F	MINUSONE	25	339A9	%e	30
339BE	%5	30	339D3	(%-.5)	30
339E8	%10	31	339FD	%180	31
33A12	(%200)	31	33A27	%360	31
33A3C	(%400)	31	33A51	(tokj)	44
33A6B	(toki)	44	33A77	tok{	43
33A83	(tokj)	44	33A8F	toksharp	43
33A9B	(tokuscore)	44	33AA7	(tok\$)	43
33AB3	(tok&)	43	33ABF	tokESC	43
33ACB	(tok>>)	43	33AD7	tok<<	43

地址	名称	页码	地址	名称	页码
33AE3	tokexponent	43	33AEF	(tokanglesign)	43
33AFB	(tokSIGMA)	44	33B07	(tokWHERE)	44
33B13	(14SPACES\$)	44	33B39	NEWLINE\$	42
33B45	(\$DER)	44	33B55	SPACE\$	42
33B55	tok_	42	33B61	(tokUNKNOWN)	44
33B79	tokquote	43	33B85	tok'	43
33B91	tok,	43	33B9D	tok.	43
33BA9	(tok;)	43	33BB5	(toklparen)	44
33BC1	(tokrparen)	44	33BCD	(tok')	44
33BD9	(tok*)	43	33BE5	(tok/)	43
33BF1	(tok+)	43	33BFD	tok-	43
33C09	tok=	43	33C15	(tokSQRT)	44
33C21	(tokDER)	43	33C2D	(tokCTGROB)	43
33C3F	(tokCTSTR)	43	33C4D	tok0	43
33C59	tok1	43	33C65	(tok2)	43
33C71	(tok3)	43	33C7D	(tok4)	43
33C89	(tok5)	43	33C95	(tok6)	43
33CA1	(tok7)	43	33CAD	tok8	43
33CB9	tok9	43	33D2B	CHR_00	40
33D32	CHR_...	40	33D39	CHR_DblQuote	40
33D40	CHR_#	40	33D47	CHR_*	40
33D4E	CHR_+	40	33D55	CHR_,	40
33D5C	CHR_-	40	33D63	CHR_.	40
33D6A	CHR_/	40	33D71	CHR_0	40
33D78	CHR_1	40	33D7F	CHR_2	40
33D86	CHR_3	40	33D8D	CHR_4	40
33D94	CHR_5	40	33D9B	CHR_6	40
33DA2	CHR_7	40	33DA9	CHR_8	40
33DB0	CHR_9	40	33DB7	CHR_:	40
33DBE	CHR_;	40	33DC5	CHR_<	40
33DCC	CHR_=	40	33DD3	CHR_>	41
33DDA	CHR_A	41	33DE1	CHR_B	41
33DE8	CHR_C	41	33DEF	CHR_D	41
33DF6	CHR_E	41	33DFD	CHR_F	41
33E04	CHR_G	41	33E0B	CHR_H	41
33E12	CHR_I	41	33E19	CHR_J	41
33E20	CHR_K	41	33E27	CHR_L	41
33E2E	CHR_M	41	33E35	CHR_N	41
33E3C	CHR_O	41	33E43	CHR_P	41
33E4A	CHR_Q	41	33E51	CHR_R	41
33E58	CHR_S	41	33E5F	CHR_T	41
33E66	CHR_U	41	33E6D	CHR_V	41
33E74	CHR_W	41	33E7B	CHR_X	41
33E82	CHR_Y	41	33E89	CHR_Z	41
33E90	CHR_a	41	33E97	CHR_b	41
33E9E	CHR_c	41	33EA5	CHR_d	41
33EAC	CHR_e	41	33EB3	CHR_f	41
33EBA	CHR_g	41	33EC1	CHR_h	41
33EC8	CHR_i	41	33ECF	CHR_j	41
33ED6	CHR_k	41	33EDD	CHR_l	41
33EE4	CHR_m	42	33EEB	CHR_n	42
33EF2	CHR_o	42	33EF9	CHR_p	42
33F00	CHR_q	42	33F07	CHR_r	42
33F0E	CHR_s	42	33F15	CHR_t	42
33F1C	CHR_u	42	33F23	CHR_v	42
33F2A	CHR_w	42	33F31	CHR_x	42
33F38	CHR_y	42	33F3F	CHR_z	42
33F46	CHR_>	42	33F4D	CHR_<<	42
33F54	CHR_>>	42	33F5B	CHR_Angle	42
33F62	CHR_Deriv	42	33F69	CHR_Integral	42
33F70	CHR_LeftPar	40	33F77	CHR_Newline	40
33F7E	CHR_Pi	42	33F85	CHR_RightPar	40
33F8C	CHR_Sigma	42	33F93	CHR_Space	40
33F9A	CHR_UndScore	41	33FA1	CHR_[]	41
33FA8	CHR_[]	41	33FAF	CHR_{}	42
33FB6	CHR_}	42	33FBD	CHR_<=	42
33FC4	CHR_>=	42	33FCB	CHR_<>	42
33FD2	\$_R<<	42	33FE2	\$_R<Z	42
33FF2	\$_XYZ	42	34002	\$_<<>>	43
034004	~IfCheckFieldType	148	34010	\$_{}	43
3401E	\$_[]	43	3402C	\$_"	43
3403A	\$_::	43	34048	\$_LRParens	43
34056	\$_2DQ	43	34064	\$_ECHO	42
34076	\$_EXIT	42	34088	\$_Undefined	42
340A4	\$_RAD	42	340B4	\$_GRAD	42
34144	RSWAP	84	3416E	ROTSWAP	72

地址	名称	页码	地址	名称	页码
3416E	XYZ>YXZ	72	34195	ROTDROPSWAP	72
34195	XYZ>ZY	72	341A8	ROTDROP	72
341A8	XYZ>YZ	72	341BA	SWAPROT	72, 73
341BA	UNROTSWAP	72, 73	341BA	XYZ>ZYX	72, 73
341D2	3DROP	72	341D2	XYZ>	72
341D7	4DROP	72	341D7	XYZW>	72
341DC	5DROP	72	341E8	6DROP	72
341F4	7DROP	72	3421A	SWAPDROP	72
3421A	XY>Y	72	3422B	3UNROLL	73
3422B	UNROT	73	3422B	XYZ>ZXY	73
3423A	4ROLL	72	3423A	FOURROLL	72
3423A	XYZW>YZWX	72	34257	5ROLL	72
34257	FIVEROLL	72	34281	6ROLL	72
34281	SIXROLL	72	342BB	8ROLL	73
342BB	EIGHTROLL	73	342EA	7ROLL	73
342EA	SEVENROLL	73	3432C	DUP4UNROLL	71
34331	4UNROLL	73	34331	FOURUNROLL	73
34331	XYZW>WXYZ	73	34357	5UNROLL	73
34357	FIVEUNROLL	73	3438D	6UNROLL	73
3438D	SIXUNROLL	73	343BD	ROTROT2DROP	72, 73
343BD	UNROT2DROP	72, 73	343BD	XYZ>Z	72, 73
343CF	4UNROLL3DROP	73	343CF	XYZW>W	73
343E1	2RDROP	84	343F3	3RDROP	84
34405	#-PICK	74	34417	#+PICK	74
34431	DUP#1+PICK	58, 71	34436	#1+PICK	74
34451	#2+PICK	74	34465	#3+PICK	74
34474	#4+PICK	74	34485	3PICK	73
3448A	4PICK	74	3448F	5PICK	74
34494	6PICK	74	34499	7PICK	74
3449E	8PICK	74	344A3	(9PICK)	74
344A8	(10PICK)	74	344CB	#-ROLL	73
344DD	#+ROLL	73	344F2	#1+ROLL	73
34504	get1	58	34517	#2+ROLL	73
3452B	#-UNROLL	73	3453D	#+UNROLL	73
34552	#1+UNROLL	73	34564	#2+UNROLL	73
3457F	DUPUNROT	71, 72	3457F	SWAPOVER	71, 72
34611	1PUTLAM	78	34616	1GETLAM	78
3461B	2PUTLAM	78	34620	2GETLAM	78
34625	3PUTLAM	78	3462A	3GETLAM	78
3462F	4PUTLAM	78	34634	4GETLAM	78
34639	5PUTLAM	78	3463E	5GETLAM	78
34643	6PUTLAM	78	34648	6GETLAM	78
3464D	7PUTLAM	78	34652	7GETLAM	78
34657	8PUTLAM	78	3465C	8GETLAM	78
34661	9PUTLAM	78	34666	9GETLAM	78
3466B	10PUTLAM	78	34670	10GETLAM	78
34675	11PUTLAM	79	3467A	11GETLAM	78
3467F	12PUTLAM	79	34684	12GETLAM	78
34689	13PUTLAM	79	3468E	13GETLAM	78
34693	14PUTLAM	79	34698	14GETLAM	78
3469D	15PUTLAM	79	346A2	15GETLAM	78
346A7	16PUTLAM	79	346AC	16GETLAM	78
346B1	17PUTLAM	79	346B6	17GETLAM	78
346BB	18PUTLAM	79	346C0	18GETLAM	78
346C5	19PUTLAM	79	346CA	19GETLAM	78
346CF	20PUTLAM	79	346D4	20GETLAM	78
346D9	21PUTLAM	79	346DE	21GETLAM	78
346E3	22PUTLAM	79	346E8	22GETLAM	78
346ED	(23PUTLAM)	79	346F2	(23GETLAM)	78
346F7	(24PUTLAM)	79	346FC	(24GETLAM)	78
34701	(25PUTLAM)	79	34706	(25GETLAM)	78
3470B	(26PUTLAM)	79	34710	(26GETLAM)	78
34715	(27PUTLAM)	79	3471A	(27GETLAM)	78
34797	DUP4PUTLAM	79	347AB	DUPTEMPENV	79
3483E	GETLAMP AIR	79	348D2	#=case	90
348E2	OVER#=case	90	348F7	DUP#0=case	90
348FC	#0=case	90	3490E	DUP#0=casedrp	90
34920	EQcasedrop	91	34939	#=casedrop	90
3494E	NOTcasedrop	89	3495D	casedrop	89
34976	NOTcase2drop	89	34985	case2drop	89
34999	EQcase	91	349B1	caseDROP	89
349C6	NOTcaseDROP	89	349D6	case2DROP	89
349EA	NOTcase2DROP	89	349F9	case	89
34A13	NOTcase	89	34A22	IT	89
34A31	GOTO	84	34A46	?GOTO	84
34A59	NOT?GOTO	84	34A7E	#0=?SEMI	90

地址	名称	页码	地址	名称	页码
34A92	NOT?SEMI	88	34AA1	?SEMI	88
34AAD	SEMILOOP	95	34ABE	ITE_DROP	89
34AD3	COLA_EVAL	85	34AF4	COLARPITE	89
34B3E	ITE	89	34B4F	2'RCOLARPITE	89
34BAB	2@REVAL	84	34BBB	3@REVAL	84
34BD8	NOT?DROP	88	34BEF	ticR	83
34C82	EXPAND	45, 48	34D00	CACHE	77
34D58	SAVESTACK	78	34EBE	DUMP	77
34FA6	undo	78	34FC0	DUPROM-WORD?	69
34FCD	ROM-WORD?	69	035004	^IfReset	148
35018	2SWAP	72	35037	DUPTYPECHAR?	118
3503C	TYPECHAR?	118	35046	DUPTYPEIDNT?	117
3504B	TYPEIDNT?	117	35082	DUPTYPEFLASHPTR?	118
35087	TYPEFLASHPTR?	118	35091	DUPTYPEZINT?	118
35096	TYPEZINT?	118	350A0	DUPTYPELNGREAL?	118
350A5	TYPELNGREAL?	118	350AF	DUPTYPELNGCMP?	118
350B4	TYPELNGCMP?	118	350BE	DUPTYPEFONT?	118
350C3	TYPEFONT?	118	350CD	DUPTYPEAPLET?	118
350D2	TYPEAPLET?	118	350DC	DUPTYPELAM?	117
350E1	TYPELAM?	117	350EB	DUPTYPEBINT?	118
350F0	TYPEBINT?	118	350F5	(#37258)	25
350FA	DUPTYPEHSTR?	118	350FF	TYPEHSTR?	118
35109	DTYPECSTR?	117	35109	DUPTYPECSTR?	117
3510E	TYPECSTR?	117	35118	DTYPEREAL?	117
35118	DUPTYPEREAL?	117	3511D	TYPEREAL?	117
35127	DUPTYPECMP?	117	3512C	TYPECMP?	117
35136	DTYPEARRY?	117	35136	DUPTYPEARRY?	117
3513B	TYPEARRY?	117	35145	DUPTYPEROMP?	118
3514A	TYPEROMP?	118	35154	DUPTYPERRP?	118
35159	TYPERRP?	118	35163	DUPTYPESYMB?	118
35168	TYPESYMB?	118	35172	DTYPECOL?	118
35172	DUPTYPECOL?	118	35177	TYPECOL?	118
35181	DUPTYPEGROB?	118	35186	TYPEGROB?	118
35190	DTYPELIST?	117	35190	DUPTYPELIST?	117
35195	TYPELIST?	117	3519F	DUPTYPETAG?	118
351A4	TYPETAGGED?	118	351AE	DUPTYPEEXT?	118
351B3	TYPEEXT?	118	35268	OVER#	28
35280	DROPTRUE	88	35289	DROFFALSE	88
35292	TYPERARRY?	117	352AD	TYPECARRY?	117
352BD	DUP#0	29	352E0	#3	28
352F1	#2	28	352FE	#1	28
3530D	#1<	28	3531C	DUP#1	29
3532B	DUP#0<	29	3533C	!insert\$	45
35346	SWAP&\$	45	35369	!!append\$?	45
353CD	!append\$	45	353EB	!!insert\$	45
353F7	!!append\$	45	354CB	'RSaveRomWrd	117
354CB	'RSAYEWORD	117	35511	#MIN	28
3551D	#MAX	28	35552	#-#2/	27
3558C	DROPZERO	26	355A5	2DROP00	26
355D0	#6-	27	355D5	#5-	27
355DA	#4-	27	355DF	#3-	27
355FD	#3+	26	35602	#4+	26
35607	#5+	26	3560C	#6+	26
35611	#7+	26	35616	#8+	26
3561B	#9+	27	35620	#10+	27
3562A	#12+	27	35675	#10*	27
3569B	#8*	27	356B8	#6*	27
356D5	5skipcola	85	35703	3skipcola	85
3570C	2skipcola	85	35715	skipcola	85
3571E	DUP#2+	27	35733	DROPSWAP	71
3574D	DROPSWAPDROP	71, 72	3574D	ROT2DROP	71, 72
3574D	XYZ>Y	71, 72	3576E	SWAPDUP	72
3579C	ROTDUP	72	357BB	SWAP#-	27
357CE	DROPDUP	71	357E2	DUPLN\$	44
357FC	#+DUP	27	3581F	#-DUP	27
35830	#1+DUP	27	35841	#1-DUP	27
35857	SWAPDROPDUP	72	35872	SWAPDROPSWAP	72, 73
35872	UNROTDROP	72, 73	35872	XYZ>ZX	72, 73
3588B	4ROLLDROP	72	358A7	5ROLLDROP	72
358C2	2DUP#<	28	358DC	2DUP#	28
358F8	2DUP#>	28	35912	DUP#1+	27
3592B	SWAP#1+	28, 58	3592B	SWP1+	28, 58
35956	DUP#1-	27	3596D	DROPONE	26
3597F	RDROPCOLA	84	35994	COLACOLA	85
359AD	COLAcase	90	359C8	COLANOTcase	90
359E3	ORcase	89	359F7	REQcase	91

地址	名称	页码	地址	名称	页码
35A10	REQcasedrop	92	35A29	SAFESTO	101
35A56	DUPSAFE@	101	35A5B	SAFE@	101
35A88	?>ROMPTR	69	35AAB	?ROMPTR>	69
35B32	2DROPFALSE	88	35B82	palrompdcmp	47
35B96	#0=UNTIL	94	35BAF	INCOMPDROP	56
35BC3	NTHCOMPDROP	54	35BD7	APPEND_SPACE	45
35BEB	7UNROLL	73	35BFF	RESOROMP	69
35C18	%10*	33	35C2C	DUP@	101
35C40	DUPROMPTR@	69	35C54	#=ITE	90
35C68	INNERDUP	56	35C7C	NOTAND	88
35C90	TOTEMPSWAP	103	35CA4	ROT2DUP	72
35CB8	ROTAND	88	35CCC	ROTOVER	72
35CE0	DUPDUP	71	35CF4	OVERDUP	73
35D08	COERCEDUP	26	35D1C	UNROTDUP	73
35D30	2DUPSWAP	71	35D30	DUP3PICK	71
35D44	4UNROLLDUP	73	35D58	NTHCOMDDUP	54
35D6C	OVERSWAP	73	35D6C	OVERUNROT	73
35D80	ROLLSWAP	73	35D94	NULL\$SWAP	44
35DA8	SUB\$SWAP	45	35DBC	%MAXorder	34
35DDA	?SKIPSWAP	89	35DEE	1ABND\$SWAP	79
35E07	ROT#+SWAP	28	35E07	ROT+SWAP	28
35E20	4PICK#+SWAP	28	35E20	4PICK+SWAP	28
35E39	#+SWAP	27	35E4D	#-SWAP	27
35E61	#1+SWAP	27	35E75	ZEROSWAP	26
35E89	#1-1SWAP	27	35EA2	ONESWAP	26
35EB6	COERCESWAP	26	35ECA	%>%\$SWAP	32
35EDE	%*\$SWAP	34	35EF2	XYZ>ZTRUE	88
35F06	4ROLLSWAP	72	35F1A	3PICKSWAP	73
35F2E	4PICKSWAP	74	35F42	1GETSWAP	79
35F56	?SWAP	88	35F6A	!append\$SWAP	45
35F7E	NOT?SWAPDROP	89	35F97	?SWAPDROP	89
35FB0	#1+NDROP	58, 72	35FB0	N+1DROP	58, 72
35FC4	ROLLDROP	73	35FD8	MDIMSDROP	52
35FF3	DUPROT	71	036004	^IfSetField	148
36007	DROPROT	71	3601B	#1-ROT	27
3602F	%*ROT	34	36043	4ROLLROT	72
36043	FOURROLLROT	72	36057	4UNROLLROT	73
3606B	DROPOVER	71	3607F	EQOVER	88
36093	#+OVER	27	360A7	#-OVER	27
360BB	ZEROOVER	26	360CF	UNROTOVER	73
360E3	4ROLLOVER	72	360F7	3PICKOVER	73
3610B	4PICKOVER	74	3611F	DUPPICK	71
36133	DUPROLL	71	36147	OVER#2+UNROL	58
3615B	8UNROLL	73	3616F	10UNROLL	73
36183	OVERAR\$IZE	52	3619E	'ERRJMP	84
361B2	caseERRJMP	92	361C6	?CARCOMP	54
361DA	NEWLINE\$&\$	44	361DA	NEWLINE&\$	44
361EE	#1-{}N	56	36202	TWO{}N	56
36216	THREE{}N	56	3622A	DUPINCOMP	56
3623E	SWAPINCOMP	56	36252	DUPNULL\$?	47
36266	DUPNULLCOMP?	54	3627A	DUPLENCOMP	54
3628E	#1-SUB\$	45	362A2	1_#1-SUB	45
362A2	1_#1-SUB\$	45	362B6	LAST\$	45
362CA	#1+LAST\$	45	362DE	DUP\$>ID	77
362F2	SWAP%>C%	37	36306	'NOP	84
3631A	::NEVAL	57	3632E	2GETEVAL	79
36342	DROPRDROP	84	3635B	SWAPCOLA	85
3636F	XYZ>ZCOLA	85	36383	#0=?SKIP	90
3639C	#1=?SKIP	90	363B5	#=?SKIP	90
363CE	ONE_EQ	28	363E2	#>?SKIP	90
363FB	COLASKIP	85	3640F	NOT_UNTIL	94
36428	NOT_WHILE	94	36441	DUP#0<>WHILE	95
3645A	DUPINDEX@	95	3646E	SWAPINDEX@	95
36482	OVERINDEX@	95	36496	SWAPLOOP	95
364AF	DROPLOOP	95	364C8	DUP#0_DO	95
364E1	toLEN_DO	95	364FF	1GETABND	79
36513	DUP1LAMBIND	77	36518	1LAMBIND	77
3652C	caseTRUE	89	36540	TRUEFALSE	87
36540	TrueFalse	87	36554	FALSETRUE	87
36554	FalseTrue	87	36568	ZEROFALSE	26
3657C	ONEFALSE	26	36590	#=casedrpfls	90
365B3	casedrpfls	89	365CC	case2drpfls	89
365E5	caseFALSE	89	365F9	ORNOT	88
3660D	EQUALNOT	88	36621	2DUPEQ	88
36635	DUPEQ:	88	3663A	EQ:	88
3664E	EQOR	88	36662	EQUALOR	88

地址	名称	页码	地址	名称	页码
36676	2#0=OR	29	36694	OVER#0=	28
366A8	OVER#<	29	366BC	#<3	28
366D0	DUP#<7	29	366E9	INNER#1=	56
366FD	#5=	28	36711	#2<>	28
36725	OVER#>	29	36739	#>1	28
36739	ONE#>	28	3674D	2DUP#+	27
3674D	DUP3PICK#+	27	36761	ROT#+	28
36775	OVER#+	28	36789	3PICK#+	28
3679D	4PICK#+	28	367B1	ROT#-	28
367C5	OVER#-	28	367D9	INDEX@#-	95
367ED	SWAPOVER#-	28	36801	ROT#1+	28
36815	#-+1	27	36815	#1-	27
36829	SWAP#1-	28	3683D	DROP#1-	27
36851	#+-1	27	36851	#1-+	27
36851	\$1+	27	36865	COLAITE	89
36883	ERROROUT	97	368B5	SWAP2DUP	72
368C9	RSKIP	84	368E7	GROB!ZERODRP	65
368FB	casedrptru	89	36914	NOTcaseTRUE	89
3692D	?SEMIDROP	88	36946	SWAPUndROP	59
3695A	SWAPUndROP	58	3696E	DUP'	84
36982	SWAP'	84	36996	DROP'	84
369AA	OVER'	84	369BE	STO'	84
369D2	TRUE'	84	369E6	ONEFALSE'	84
369FF	FALSE'	84	36A13	#1+'	84
36A27	'R'R	83	36A4A	'RRDROP	83
36A63	ONECOLA	85	36A77	dvarlsBIND	77
36A8B	'LAMLNAMESTO	102	36AA4	'xDEREQ	85
36ABD	DUPNULL{ }?	56	36AD6	DUPZERO	26
36AEA	DUPONE	26	36AFE	SWAPONE	26
36B12	ONEDUP	25	36B12	ONEONE	25
36B26	DUPTWO	26	36B3A	NOTcsdrpfls	89
36B53	caseSIZEERR	92	36B67	NcaseSIZEERR	92
36B7B	CKREAL	117	36BAA	NcaseTYPEERR	92
36BBE	('x*)	84	36BD2	'xDER	84
36BE6	%%/>%	34	36BFA	UNCOERCE%%	32
36C0E	DUP%0=	35	36C22	SWAP%%/	34
36C36	caseDrpBadKy	92	36C4F	caseDEADKEY	92
36C4F	caseDoBadKey	92	36C68	GROBDIMw	65
36C7C	%%*UNROT	34	36C90	SWAP4ROLL	72
36C90	XYZW>YWZX	72	36CA4	2DUP5ROLL	71
36CB8	SWAP3PICK	72	36CCC	3PICK3PICK	74
36CE0	SWAP4PICK	74	36CF4	OVER5PICK	73
36D08	EQUALcasedrp	92	36D21	DUP#0=csDROP	90
36D3A	jEQcase	91	36D4E	ANDcase	89
36D62	EQUALcase	92	36D76	#<case	90
36D8A	#1=case	90	36D9E	#<>case	90
36DB2	#>2case	90	36DCB	#>case	90
36DDF	j%0=case	91	36DF3	REALcase	92
36E07	dARRYcase	92	36E43	dLISTcase	92
36E57	EditExstCase	92	36E6B	ANDNOTcase	89
36E7F	EQUALNOTcase	92	36E93	dIDNTNcase	92
36EA7	dREALNcase	92	36EBB	EQIT	91
36ED4	DUP#0=IT	90	36EED	ANDITE	89
36F01	EQITE	91	36F15	#0=ITE	90
36F29	#<ITE	90	36F3D	#>ITE	90
36F51	DUP#0=ITE	90	36F65	UserITE	92
36F79	SysITE	92	36FA6	metaROTDUP	58
36FBA	ROTUntop&	58	36FCE	roll2top&	58
36FCE	rolltwotop&	58	36FE2	plDRPpZparg	64
36FF6	&\$SWAP	45	037004	^IfKeyChoose	148
3700A	SWAPCKREF	103	3701E	pZpargSWAPUn	64
37032	DROPNDROP	58, 71	37046	2OVER	73
3705A	?Ob>Seco	57	37073	Ob>Seco	57
37087	2Ob>Seco	57	3709B	ExitAtLOOP	95
3709B	ZEROISTOPSTO	95	370AF	RclHiddenVar	102
0370B3	(BBGetNGrob)	141	370C3	WithHidden	102
37104	StoHiddenVar	102	37118	PuHiddenVar	102
3712C	SaveVarRes	102	3714A	SetHiddenRes	102
37186	RestVarRes	102	371B3	Embedded?	55
3721C	Sig?ErrJump	98	37287	ZEROZERO	25
37294	#ZERO#ONE	25	37305	#ZERO#SEVEN	25
37315	#ONE#27	25	37328	#TWO#ONE	25
3733A	#TWO#TWO	25	3734A	#TWO#FOUR	25
3735C	#THREE#FOUR	25	3736E	#FIVE#FOUR	25
37380	ZEROZEROZERO	25	37394	ZEROZEROONE	26
373A8	ZEROZEROTWO	26	374BE	SWAPDROPTRUE	88

地址	名称	页码	地址	名称	页码
3760D	SubMetaOb	59	37685	SubMetaOb1	59
376B7	matchob?	55	376EE	POSCOMP	55
37752	#=POSCOMP	55	3776B	EQUALPOSCOMP	55
37784	NTHOF	55	37798	Find1stTrue	55
377C5	Lookup	55	377DE	Lookup.1	55
37829	EQLookup	55	378FA	POS\$	44
378FA	POSCHR	45	37906	POS\$REV	45
37906	POSCHRREV	45	37AA5	CHR>\$	44
37ABE	STRIPTAGS	51	37AEB	STRIPTAGS12	51
37B04	TAGOB\$	51	37B54	NEXTCOMPOB	55
038004	^IfKeyEdit	148	0380B3	{BBGetNStr}	141
38266	{#FFFF}	25	38275	{#BB}	21
38C00	{DISPST2&FREEZE}	159	039004	^IfKeyTypes	148
39277	{#B437D}	25	39332	{?GETMSG}	97
394C8	INHARDROM?	107	39C8B	{SWAP>HCOMP}	54
39C9F	{&ob}	47	39CB3	{ob&\$}	47
39E6B	{#A4}	21	03A004	^IfKeyCalc	148
3A12D	{#4FF}	22	3A1C2	{#304}	22
03B004	^IfKeyInvertCheck	148	03B0B3	{BBRereadChkEnbl}	141
3B76C	{DOSTOALLF2}	106	3B7AD	{#BBBB}	25
3B8F5	{#C55}	24	3B904	{#C22}	24
3B913	{#455}	22	3B928	{#411}	22
3B93D	{#415}	22	3B952	{#451}	22
3B967	{#855}	23	3B976	{#822}	23
3B9FA	{#313}	22	3BA09	{#515}	22
3BA18	{#454}	22	3BA2D	{#414}	22
3BADA	{INNERCOMP>%}	56	3BC43	XEQTYPE	117
3BD4C	{#AF}	21	3BD65	{#CF}	21
03C004	^IfONKeyPress	148	03C0B3	{BBRereadFullScr}	141
3C10F	{#450}	22	3C11E	{#410}	22
3C16B	{#750}	23	3C17A	{#710}	23
3C7E2	{#C5C}	24	3C800	{#C2C}	24
3C81E	{#85C}	24	3C83C	{#82C}	23
3C8D0	{#2111}	24	3C8DF	{#5B11}	25
03D004	^IfEnterKeyPress	148	03D0B3	{BReReadMenus}	141
3D2B4	CKSYMBTYPE	117	3D50D	{#A110}	25
3D51C	{#AA10}	25	3D52B	{#A1A0}	25
03E0B3	{BBReReadNElems}	141	3E17B	{#111}	21
3E743	{#9FD}	24	3E759	{#8FD}	24
3E7DA	{#C8}	21	3E7E9	{#9F1}	24
3E7FF	{#8F1}	24	3E823	xSTO>	101
3EA01	{CST}	165	3EAFB	{#9F}	21
03F004	^IfSetHelpString	148	03F0B3	{BBGetN}	141
3F481	COERCE2	26	3F495	UNCOERCE2	32
040004	^IfSetTitle	148	042004	^IfMain2	148
043004	^IfPutFieldsOnStack	148	044004	^IfSetFieldPos	149
045004	^IfGetFieldPos	149	047004	^IfSetAllLabelsMessages	149
048004	^IfSetAllHelpStrings	149	04A004	^IfInitDepth	148
04B0B3	{BBI\$Checked?}	141	04C004	^IfGetPrlgFromTypes	148
04D004	^IsUncompressDataString	149	4EA22	{%TICKSsec}	31
4EA37	{%TICKSmin}	31	4EA4C	{%TICKShour}	31
4EA61	{%TICKSday}	31	4EA76	{%TICKSweek}	32
0520B3	{BBUpArrow}	141	0530B3	{BBDownArrow}	141
0540B3	{BBSpace}	141	0590B3	{BBPgDown}	141
05A0B3	{BBPgUp}	142	05B0B3	{BBEmpty?}	142
05C0B3	{BBGetDefltHeight}	142	05F0B3	{~\$>grobOrGROB}	68
0630B3	ChooseSimple	141	067004	^Filer	112
06D004	^FILER_MANAGER	112	06E004	^FILER MANAGERTYPE	112
070002	{^Choose2}	136	072002	{^Choose3}	136
073002	{^Choose3Save}	136	073007	^QpiZ	221
074002	{^Choose3Index}	136	074007	^QPI	221
075002	{^ChooseDefHandler}	136	075007	^QpiSym	221
076002	{^Choose3CANCL}	136	076007	^QpiArray	221
077002	{^Choose3OK}	136	077007	^QpiList	221
078007	^Qpi	221	079007	^Qpi%	221
07A007	^GetRoot	221	07B007	^Approx	221
07C007	^#FACT	181	07D007	^CHECKSING	212
07E007	^DESOLVE	208	07F007	^ODE_INT	208
080007	^LINSOLV	186	081007	^LDECSOLV	208
082007	^LDEGENE	208	083007	^LDEPART	208
084007	^LDSSOLVext	208	085007	^ODETYPESTO	208

地址	名称	页码	地址	名称	页码
086007	^ODE_SEPAR	208	0860B0	~grobAlertIcon	65
087007	^LAPext	208	0870B0	~grobCheckKey	65
088002	(^SaveHARDBUFF)	136	088007	^ILAPext	208
089002	(^RestoreHARDBUFF)	136	08B007	^ILAPEXP	208
08D007	^MENUext	216	08E006	^BerlekampP	201
08E007	^WRITEMENU	164	08F006	^Berlekamp	201
08F007	^CFGDISPLAY	214	090006	^ErrInfRes	213
090007	^NEWVX	214	091006	^ErrUndefRes	213
091007	^NEWMODULO	214	092006	^ErrBadDim	213
092007	^SWITCHON	214	093006	^ALG48MSOLV	205
093007	^SWITCHOFF	214	094006	^GMSOLV	205
094007	^FLAGNAME	214	095006	^GBASIS	205
095007	^COMPLEXON	214	096006	^GSOLVE	205
096007	^COMPLEXOFF	214	097006	^GFACTOR	205
097007	^EXACTON	214	098007	^EXACTOFF	214
099006	^REDUCE	205	099007	^COMPLEXMODE	214
09A003	(^WRAP\$)	45	09A006	^FASTREDUCE	205
09A007	^SETCOMPLEX	214	09B006	^ONE{}POLY	202
09B007	^COMPLEX?	214	09C006	^TWO{}POLY	202
09C007	^REALMODE	214	09D002	^DoCKeyOK	141
09D006	^THREE{}POLY	202	09D007	^CLRCOMPLEX	214
09E002	^DoCKeyCancel	141	09E006	^TWO::POLY	202
09E007	^EXACTMODE	214	09F002	^DoCKeyCheck	141
09F006	^::POLY	202	09F007	^SETEXACT	214
0A0002	^DoCKeyChAll	141	0A0006	^{}POLY	202
0A0007	^NUMMODE	214	0A1006	^>TPOLY	202
0A1007	^CLREXACT	214	0A2006	^>HPOLY	202
0A2007	^EXACT?	214	0A3006	^>TPOLYN	202
0A3007	^STEPBYSTEP	214	0A4006	^>HPOLYN	202
0A4007	^NOSTEPBYSTEP	214	0A5006	^MKPOLY	202
0A5007	^VERBOSEMODE	214	0A6006	^ONE>POLY	202
0A6007	^SILENTMODE	214	0A7006	^>POLY	202
0A7007	^RECURMODE	214	0A8006	^ALG48FCTR?	201
0A8007	^NONRECMODE	214	0A9006	^MFactTriv	201
0A9007	^PLUSAT0	214	0AA006	^CheckPNoExt	201
0AA007	^SETPLUSAT0	214	0AB006	^PPP	201
0AB007	^PLUSATINFTY	214	0AC003	^SWAPROWS	187
0AC006	^PFactor	201	0AC007	^CLRPLUSAT0	214
0AD006	^PSqff	201	0AD007	^SPARSEDATA	214
0AE006	^PHFctr	201	0AE007	^FULLDATA	214
0AF006	^PHFctrl	201	0AF007	^RIGORMODE	214
0B0002	^DoCKeyUnChAll	141	0B0006	^PHFctr0	201
0B0007	^SLOPPYMODE	215	0B1002	^LEDISPitem	141
0B1006	^DeCntMulti	201	0B1007	^SLOPPY?	215
0B2002	^LEDISPlist	141	0B2006	^DoLS	201
0B2007	^MENUCHOOSE?	216	0B3002	^LEDISPprompt	141
0B3006	^PNFctr	201	0B3007	^MENUCHOOSE	216
0B4006	^PSQFF	201	0B4007	^MENUGENE1	216
0B5006	^LiftZAdic	201	0B5007	^MENUBASE1	216
0B6006	^LFCProd	201	0B6007	^MENUMCPLX1	216
0B7006	^UFactor	201	0B7007	^MENUTRIG1	216
0B8006	^UFactor1	201	0B8007	^MENUMAT1	216
0B9006	^MonicLf	201	0B9007	^MENUARIT1	216
0BA006	^DemonicLf	201	0BA007	^MENSOLVE1	216
0BB006	^LiftLinear	201	0BB007	^MENUEXPLN1	216
0BC006	^LiftGeneral	202	0BC007	^MENUDIFF1	216
0BD002	^DOTVARS{}	102	0BD006	^UFactorDeg2	202
0BD007	^PROMPTSTO1	102	0BE006	^CombineFac	202
0BE007	^XGROBext	68	0BF006	^CombProd	202
0BF007	^GROBADDext	66	0C0006	^CombInit	202
0C0007	^DISPLAYext	68	0C1006	^CombNext	202
0C1007	^SCROLLext	159	0C2006	^RmCombNext	202
0C2007	^RCLMODULO	215	0C3006	^PFactTriv	202
0C3007	^RCLPERIOD	215	0C4006	^VarFactor	202
0C4007	^RCLVX	215	0C5006	^PFactPowCnt	202
0C5007	^STOVX	215	0C6006	^PDivLk	202
0C6007	^STOMODULO	215	0C7006	^Prime+	182
0C7007	^RCLEPS	215	0C8006	^Prime-	182
0C8007	^ISIDREAL?	215	0C9006	^ZFactor	181
0C9007	^ADDTOREAL	215	0CA006	^NFactor	181
0CA007	^RESETCASCFG	215	0CB006	^NFactorSpc	181
0CB007	^FRACPARITY	224	0CC006	^DupTypeS?	183
0CC007	^POLYPARITY	203	0CD006	^SFactor	181
0CE006	^SPollard	181	0CF006	^BFactor	181
0CF007	^SHRINK EVEN	203	0D0006	^BrentPow	181
0D1006	^ZPrime?	181	0D1007	^SHRINK2SYM	203

地址	名称	页码	地址	名称	页码
0D2006	^ZIsPrime?	181	0D2007	^SHRINKSYM	203
0D3006	^SIsPrime?	181	0D3007	^SHRINK2ASYM	203
0D4006	^BIsPrime?	181	0D4007	^SHRINKASYM	203
0D5006	^BRabin	182	0D5007	^FR2ND%	224
0D6006	^ZTrialDiv2	182	0D6007	^POLYSYM	203
0D7006	^ZTrialPrime?	182	0D7007	^POLYASYM	203
0D8006	^ZTrialDiv	182	0D8007	^P2F#	201
0D9006	^QMod	200	0D9007	^NDEvalN/D	209
0DA007	^PEvalN/D	209	0DB007	^POSITIFext	212
0DC007	^SIGNElxt	212	0DD006	^ZMod	181
0DE007	^SIGNUNDEF	212	0DE0B0	~nNullBind	77
0DF007	^SIGNPLUS	212	0E0006	^ZSQRT	181
0E0007	^SIGNMOINS	212	0E1006	^PEvalMod	210
0E1007	^SIGNELN	212	0E2006	^QAddMod	210
0E2007	^SIGNEEXP	212	0E3006	^QSubMod	210
0E3007	^SIGNESIN	212	0E4006	^QMulMod	210
0E4007	^SIGNECOS	212	0E5006	^QDivMod	210
0E5007	^SIGNETAN	212	0E6006	^QInvMod	210
0E6007	^SIGNETATAN	212	0E7006	^QGcdMod	210
0E7007	^SIGNESQRT	212	0E8007	^SUBSIGNE	212
0E9006	^IsV>V?	211	0E9007	^SIGNERIGHT	212
0EA006	^PEvalFast?	203	0EA007	^SIGNELEFT	212
0EB006	^PZadic	211	0EB007	^>SIGNE	212
0EC006	^GCDHEUext	211	0EC007	^SIGNE>	212
0ED006	^H>Z	180	0ED007	^SIGNMULText	212
0EE006	^#>Z	180	0EE007	^ZSIGNECK	212
0EF006	^Z2BIN	26	0EF007	^SIGNEERROR	213
0F0006	^COERCE2Z	26	0F0007	^ZSIGNE	212
0F1006	^Z>S	44	0F1007	^zsigne	212
0F2006	^S>Z	180	0F2007	^PASCAL_NEXTLINE	205
0F3006	^S>Z?	180	0F3007	^DELTAPSOLVE	205
0F4006	^Z>ZH	180	0F4007	^SOLVEMETASYST	186
0F5006	^R>Z	180	0F5007	^REDUCEMETASYST	186
0F6006	^Z>R	32	0F6007	^REDUCEMETAPSYST	186
0F7006	^DupQIsZero?	182	0F7007	^SOLVECRAMER	186
0F8006	^QIsZero?	182	0F9006	^DupZIsOne?	182
0F9007	^SUM	209	0FA006	^ZIsOne?	182
0FB006	^DupZIsNeg?	182	0FB007	^SUMVX	209
0FC006	^ZIsNeg?	182	0FD006	^ListPos	55
0FD007	^RATSUM	209	0FE006	^AppendList	56
0FE007	^FTAYL	209	0FF006	^Contains?	88
0FF007	^CSTFRACTION?	209	100006	^SortList	56
100007	^NONRATSUM	209	101006	^ZTrim	180
101007	^LINEARAPPLY	223	102006	^ZAbs	180
103006	^PNMax	203	103007	^meta_cst?	209
104006	^LISTMAXext	211	104007	^HYPERGEO	209
105006	^ZNMax	181	106006	^ZNMin	181
106007	^A/B2PQR	224	107006	^ZNL?	182
107007	^GOSPER?	224	108006	^DISTDIVext	200
108007	^ZEILBERGER	209	109006	^DupZIsTwo?	182
109007	^SYMPSI	209	10A006	^DupZIsEven?	182
10B006	^Univar?	203	10B007	^SYMPSIN	209
10C006	^SUnivar?	203	10D006	^ZBits	181
10D007	^IBERNOULLI	209	10E006	^ZBit?	181
10E007	^FLAGRESULTANT	203	10F006	^LOPMext	222
10F007	^RESULTANT	203	110006	^SWAPRMULT	200
110007	^RESULTANTLP	203	111006	^QMul	200
111007	^RESPSHIFTQ	203	112006	^RMULTText	200
112007	^ADDONEVAR	203	113006	^RASOP	200
113007	^IROOTS	204	114006	^SWAPRSUB	200
114007	^TYPEGAUSSINT?	118, 182	115006	^QSub	200
115007	^DTYPEGAUSSINT?	118, 182	116006	^RSUBext	200
116007	^DUPTYPEGAUSSINT?	118, 182	117006	^SWAPRADD	200
117007	^PPZZ	223	118006	^QAdd	200
118007	^DISTRIB*	199	119006	^RADDext	200
119007	^NONALGERR	213	11A006	^SWAPRDIV	200
11A007	^ALGCASCOMPEVAL	192	11B006	^RDIVext	200
11C006	^QDiv	200	11C007	^%%PSI	209
11F006	^RP#	200	120006	^MPext	200
121006	^MP0	223	123006	^RPext	200
124006	^PREPARext	222	125006	^x+ext	189
126006	^x-ext	189	127006	^x*ext	189
128006	^x=ext	190	129006	^x/ext	189
12A006	^2SYMBINCOMP	56, 179	12B006	^x^ext	189
12C006	^EXPAND^	189	12D006	^addtXROOT	196
12F006	^addtMIN	196	131006	^addtMAX	196

地址	名称	页码	地址	名称	页码
133006	^addt<	196	135006	^addt<=	196
137006	^addt>	196	139006	^addt>=	196
13B006	^addt==	196	13D006	^addt!=	196
13F006	^addt%	196	141006	^addt%CH	196
143006	^addt%T	196	145006	^addtMOD	196
147006	^addtTRNC	196	149006	^addtRND	196
14B006	^addtCOMB	196	14D006	^addtPERM	196
14F006	^addtOR	196	151006	^addtAND	196
153006	^addtXOR	196	155006	^2LAMBIND	77
156006	^3LAMBIND	77	157006	^SYMBINCOMP	56, 179, 195
158006	^CKINNERCOMP	56	159006	^DUPCKLEN{}	56
15A006	^CKCARCOMP	56	15B006	^CARCOMPext	200
15C006	^RISCH13	206	15D006	^CXRIext	182
15E006	^RIXCext	37	15F006	^IRXCext	37
161006	^SWAPNDXF	203	162006	^NDXFext	203
163006	^SWAPFXND	203	164006	^FXNDext	203
165006	^QXNDext	222	166006	^NDXQext	222
167006	^TYPEIRRQ?	179, 222	168006	^DTYPEIRRQ?	179, 222
169006	^BESTMATRIXTYPE	53	16A006	^{}TO[]	184
16B006	^[]TO{}	184	16C006	^DUPNULL[]?	185
16D006	^MDIMS	52	16E006	^DIMLIMITS	52
16F006	^CKSAMESIZE	185	170006	^DTYPEENDO?	185
171006	^DTYPEFMAT?	118	172006	^CKNUMARRY	53
173006	^2DMATRIX?	185	174006	^MATRIXDIM	186
175006	^SAMEMATRIX	185	176006	^SAMEMATSTYPE	185
177006	^CKMATRIXELEM	179	178006	^MATRIX2ARRAY	53
179006	^MATRIX2LIST	185	17A006	^LIST2MATRIX	184
17B006	^LENMATRIX	187	17C006	^XEQARRY>	53
17D006	^MATEXPLODE	185	17E006	^ARRAY2MATRIX	185
17F006	^XEQ>ARRAY	53	181006	^CKALG	118
182006	^TYPEZ?	118	183006	^DUPTYPEZ?	118
184006	^CK1Z	117, 180	185006	^CK2Z	117, 180
186006	^CK3Z	117, 180	187006	^CK1Cext	118, 182
188006	^C2C%	37	189006	^Z2C%ext	37
18A006	^Z2%	32	18B006	^C%>C%	37
18C006	^E%>C%	37	18D006	^R2Zext	180
18E006	^Z2Sext	180	18F006	^CKFPOLYext	179
190006	^CK2FPOLY	179	191006	^IDNTLAM?	118
192006	^FLOAT?	119	193006	^CKSYMREALCMP	119
194006	^TYPEIDNTLAM?	117	195006	^REAL?	119
196006	^TYPEREALZINT?	119	197006	^OBJ2REAL	33
198006	^METAINT?	59, 183	199006	^METAPOSINT?	59, 183
19A006	^OBJINT?	182	19B006	^OBJPOSINT?	182
19C006	^CKINT>0	183	19D006	^Z>#	26
19E006	^CLEANIDLAM	179	1A1006	^DERIVext	206
1A3006	^DERIVIDNT	206	1A4006	^DERIVIDNT1	206
1A5006	^DERIV	207	1A6006	^METADERIV	207
1A7006	^DO>STRID	47	1A9006	^METADER+	207
1AA006	^METADER-	207	1AB006	^METADER*	207
1AC006	^METADER/	207	1AD006	^METADER^	207
1AE006	^METADERFCN	207	1AF006	^METADERDER	207
1B0006	^METADERI4	207	1B1006	^METADERI3	207
1B2006	^METADERIFTE	207	1B3006	^DERARG	207
1B4006	^METADEREXP	207	1B5006	^METADERLN	207
1B6006	^METADERLNP1	207	1B7006	^METADERLOG	207
1B8006	^METADERALOG	207	1B9006	^METADERABS	207
1BA006	^METADERINV	207	1BB006	^METADERNEG	207
1BC006	^METADERSQRT	207	1BD006	^METADER&NEG	207
1BE006	^METADERSQ	207	1BF006	^METADERSIN	207
1C0006	^METADERCOS	207	1C1006	^METADERTAN	207
1C2006	^METADERSINH	207	1C3006	^METADERCOSH	207
1C4006	^METADERTANH	207	1C5006	^METADERASIN	207
1C6006	^METADERACOS	207	1C7006	^METADERATAN	207
1C8006	^METADERASH	207	1C9006	^METADERACH	207
1CA006	^METADERATH	207	1CB006	^pshder*	207
1CC006	^SQRTINVpshd*	207	1CD006	^ckaddt*	195
1CE006	^ckaddt+	195	1CF006	^ckaddt-	195
1D0006	^VERNUMext	215	1D1006	^MENUXYext	216
1D2006	^SAVECASFLAGS	215	1D3006	^SAFEPURGE	102
1D4006	^RESTORECASFLAGS	215	1D5006	^CASFLAGEVAL	215
1D6006	^FLAGEXPAND	217	1D8006	^FLAGFACTOR	217
1D9006	^FLAGLISTEXEC	217	1DA006	^FLAGSYMBEXEC	217
1DB006	^FLAGIDNTEXEC	217	1DC006	^FLAGINTVX	217
1DC007	(^PUSHFLAGS)	214	1DD006	^DERVX	217
1DD007	(^POPFLAGS)	214	1DE006	^SOLVEXFLOAT	217

地址	名称	页码	地址	名称	页码
1DF006	^SYMLIMIT	217	1E0006	^FLAGMATRIXLIMIT	217
1E1006	^TAYLOR0	217	1E2006	^FLAGSERIES	217
1E4006	^PLOTADD	217	1E5006	^FLAGIBP	217
1E6006	^FLAGPREVAL	217	1E7006	^MATRIXRISCH	217
1E8006	^FLAGRISCH	217	1E9006	^FLAGDERIV	217
1EA006	^FLAGLAP	217	1EB006	^FLAGILAP	217
1EC006	^FLAGDESOLVE	217	1ED006	^FLAGLDSSOLV	217
1EF006	^FLAGTEXPAND	217	1F0006	^FLAGLIN	217
1F1006	^FLAGTSIMP	217	1F2006	^FLAGLNCOLLECT	217
1F3006	^FLAGEXPLN	217	1F4006	^FLAGINCOS	217
1F5006	^FLAGTLIN	217	1F6006	^FLAGTCOLLECT	217
1F7006	^FLAGTRIG	217	1F8006	^FLAGTRIGCOS	217
1F9006	^FLAGTRIGSIN	217	1FA006	^FLAGTRIGTAN	217
1FB006	^FLAGTAN2SC	218	1FC006	^FLAGHALFTAN	218
1FD006	^FLAGTAN2SC2	218	1FE006	^FLAGATAN2S	218
1FF006	^FLAGASIN2T	218	200006	^FLAGASIN2C	218
201006	^FLAGACOS2S	218	202006	^CK&CONVINT	180
203006	^CK&CONV2INT	180	204006	^CONVBAC2INT	180
205006	^CONVBACINT	180	206006	^STEPIDIV2	218
207006	^FLAGDIV2	218	208006	^FLAGGCD	218
209006	^PEGCD	218	20B006	^ABCUV	218
20C006	^IABCUV	218	20D006	^FLAGLGCD	218
20E006	^FLAGLCM	218	20F006	^FLAGSIMP2	218
210006	^FLAGPARTFRAC	218	211006	^FLAGPROPFRAC	218
212006	^FLAGPTAYL	218	213006	^FLAGHORNER	218
214006	^EULER	218	215006	^PA2B2	181
216006	^FLAGCHINREM	218	217006	^ICHINREM	218
218006	^ISPRIME	217	219006	^SOLVE1EQ	218
21A006	^SOLVEMANYEQ	218	21B006	^ZEROS1EQ	218
21C006	^ZEROSMANYEQ	218	21D006	^FCOEF	218
21E006	^FROOTS	218	21F006	^FACTORS	218
220006	^DIVIS	218	223006	^rref	218
224006	^FLAGQXA	188	225006	^FLAGAXQ	188
226006	^FLAGGAUSS	188	227006	^FLAGSYLVESTER	188
228006	^PCAR	188	229006	^MADNOCK	218
22A006	^SYSTEM	218	22B006	^VANDERMONDE	218
22C006	^HILBERTNOCK	218	22D006	^FLAGJORDAN	188
22E006	^CURL	218	22F006	^DIVERGENCE	218
230006	^LAPLACIAN	218	231006	^HESSIAN	218
232006	^HERMITE	218	233006	^TCHEBNOCK	218
234006	^LEGENDRE	219	235006	^LAGRANGE	219
236006	^FOURIER	219	237006	^SIGNE	212
238006	^TABVAR	219	239006	^FLAGDIVPC	219
23A006	^FLAGTRUNC	219	23B006	^FLAGSEVAL	219
23C006	^XNUM	219	23D006	^REORDER	219
23E006	^USERLVAR	219	23F006	^USERLIDNT	219
240006	^EXLR	193	241006	^ADDTMOD	219
242006	^MADDTMOD	219	243006	^SUBTMOD	219
244006	^MSUBTMOD	219	245006	^MULTMOD	219
252006	^FLAGFACTORMOD	210	253006	^MFACTORMOD	210
255006	^KEYEVAL	122	256006	^LIFCext	210
257006	^EvalNoCKx*	220	258006	^EvalNoCKx+	220
259006	^EvalNoCKx-	220	25A006	^EvalNoCKx/	220
25B006	^EvalNoCKx^	220	25C006	^EvalNoCKxCHS	220
25D006	^EvalNoCKxINV	220	25E006	^EvalNoCKxMOD	220
25F006	^EvalNoCKxPERM	220	260006	^EvalNoCKxCOMB	220
261006	^EvalNoCKxOR	220	262006	^EvalNoCKxAND	220
263006	^EvalNoCKxXOR	220	264006	^EvalNoCKxXROOT	220
265006	^TABVALext	220	266006	^TOLISText	220
267006	^FROMLISText	220	268006	^PFEXECext	222
269006	^LOP1ext	222	26A006	^LOPAext	222
26B006	^LISTSECOext	222	26C006	^rpnQOJBext	223
26D006	^CK1TONOext	222	26E006	^COLCext	192
272006	^MULMULText	204	274006	^METAMM2	204
275006	^COMPLISText	204	276006	^METACOMPRIM	204
278006	^METACOMP1	204	279006	^ADDLISText	204
27A006	^DIVISext	204	27B006	^FACT1ext	204
27C006	^FACTOext	204	27D006	^ZFACTO	204
27E006	^SOLVext	204	27F006	^FRND	204
280006	^BICARREE?	204	281006	^REALBICAR	204
282006	^FEVIDENText	202	283006	^EVIDENText	204
284006	^EVIDSOLV	204	285006	^DEG2ext	204
286006	^METADEG2	204	287006	^METADEG1	204
288006	^DEG1	204	289006	^FDEG2ext	204
28A006	^PIext	56	28B006	^RACTOFACext	204
28C006	^FACTORACext	204	28D006	^RFACText	204

地址	名称	页码	地址	名称	页码
28E006	^RFACT2ext	204	28F006	^RFACTSTEP3	204
290006	^RFACTSTEP5	204	291006	^METASOLV	205
293006	^METASOLV2	205	294006	^METASOLV4	205
295006	^ADDMULTIPL	205	296006	^FACTOOBJext	205
297006	^SLVARext	192	298006	^SIMPLIFY	192
299006	^SIMPlext	192	29A006	^SYMEXPAN	192
29B006	^SIMPVAR	192	29D006	^SIMPIDNT	223
29F006	^RCL1IDNT	223	2A0006	^SIMPSYMS	192
2A2006	^SIMPUSERFCN	192	2A3006	^EVALUSERFCN	192
2A4006	^SIMP	192	2A5006	^DENOLCMext	193
2A6006	^METADENOLCM	193	2A7006	^SWPSIMPNDXF	223
2A8006	^SIMPNDXFext	223	2A9006	^SIMPext	192
2AB006	^MAKEPROFOND	202	2AD006	^SIMPGCdext	192
2AE006	^SIMP3ext	192	2B1006	^LPGCdext	194
2B2006	^SLOWGCdext	194	2B3006	^QGcd	194
2B5006	^CGCdext	182	2B6006	^CMODext	223
2B7006	^ZGCdext	181	2B8006	^ZGcd	181
2B9006	^TSIMP2ext	192	2BA006	^TSIMPext	192
2BB006	^TSIMP3ext	192	2BC006	^LASTCOMP	54
2BD006	^SQFF2ext	223	2BE006	^PPZ	223
2BF006	^PZHSTR	223	2C0006	^HORNER1ext	223
2C1006	^PEval	223	2C5006	^IBP	208
2C6006	^SQRT_IN?	223	2C7006	^IS_SQRT?	223
2C9006	^IS_XROOT?	223	2CA006	^STOPRIMIT	223
2CB006	^CONTAINS_LN?	223	2D0006	^PREVALext	208
2D1006	^WARNSING	208	2D2006	^INText	208
2D3006	^INT3	208	2D4006	^FOURIERext	223
2D5006	^3DUP	71	2D6006	^#3+ROLL	73
2D7006	^2DROPTTRUE	88	2D8006	^IRRQ#ULTIMATE	222
2D9006	^LESSCOMPLEX?	223	2DA006	^LISTIRRQ	222
2DD006	^TABLECOSext	223	2DE006	^TABLETANext	223
2DF006	^DROPZ1	180	2E0006	^DROPZ0	180
2E1006	^TESTINFINI	221	2E2006	^INFINIext	221
2E3006	^MINUSINFext	221	2E4006	^PLUSINFext	221
2E5006	^?ext	221	2E6006	^POSINFext	221
2E7006	^POSUNDEFext	221	2E8006	^pisur2	221
2E9006	^pisur-2	221	2EA006	^pi	221
2EB006	^metapi	221	2EC006	^'xPI	221
2ED006	^metai	221	2EE006	^'xi	221
2EF006	^ipi	221	2F0006	^metaipi	221
2F1006	^meta-pi	221	2F2006	^metapi/2	221
2F3006	^metapi/4	221	2F4006	^meta-pi/2	221
2F5006	^meta-pi/4	221	2F6006	^pifois2	221
2F7006	^deuxipi	222	2F8006	^metapi*2	221
2F9006	^base_ln	221	2FA006	^meta_e	221
2FB006	^NEXTPext	56	2FC006	^INSERT{ }N	56
2FD006	^COMPRIMext	56	2FE006	^TCOLLECT	192
2FF006	^SIGMAEXPext	192	300006	^LINEXPext	192
301006	^SIGMAEXP2ext	192	302006	^TCHEBext	202
303006	^SINEXPA	192	304006	^METASINEXPA	199
305006	^SINEXPA+	199	306006	^SINEXPA-	199
307006	^SINEXPA*	199	308006	^SINEXPA*1	199
309006	^COSEXPA	192	30A006	^METACOSEXPA	199
30B006	^COSEXPA+	199	30C006	^COSEXPA-	199
30D006	^COSEXPA*	199	30E006	^COSEXPA*1	199
30F006	^EXPEXPA	193	310006	^METAEXPEXPA	199
311006	^EXPEXPA+	199	312006	^EXPEXPA-	199
313006	^EXPEXPA*	199	314006	^EXPEXPANEG	199
315006	^EXPEXPA*1	199	316006	^LNEXPA	192
317006	^METALNEXPA	199	318006	^LNEXPA*	199
319006	^LNEXPA/	199	31A006	^LNEXPA^	199
31B006	^LINEXPA	193	31C006	^MTRIG2SYMB	192
31D006	^LNCOLCext	193	31E006	^METATANEXPA	199
31F006	^TEXPAext	193	320006	^MAT+	185
321006	^MADD	185	322006	^MAT-	185
323006	^MSUB	185	324006	^VADD	185
325006	^VSUB	185	326006	^MAT*	185
327006	^MMULT	185	328006	^MVMULT	185
329006	^SCL*MAT	185	32A006	^MAT*SCL	185
32B006	^VPMULT	185	32C006	^MAT^	185
32D006	^MATCROSS	185	32E006	^MATDOT	185
32F006	^RNDARRY	185	330006	^TRCARRY	185
331006	^Yext	52	332006	^MAT/SCL	185
333006	^MAT/	185	334006	^MATCHS	185
335006	^MATSQUARE	185	336006	^MATCONJ	185
337006	^MATRE	186	338006	^MATIM	186

地址	名称	页码	地址	名称	页码
339006	^MATTRACE	186	33A006	^MATTRN	186
33B006	^MATTRAN	52	33C006	^mattran	186
33D006	^mattrn	186	33E006	^MATSUB	187
33F006	^submeta	59	340006	^MATREPL	187
341006	^MATREDIM	184	342006	^VRRDM	184
343006	^VRRDMmeta	184	344006	^MATRANM	184
345006	^DIMRANM	184	346006	^MATDET	186
347006	^MATRDET	186	348006	^MATFNORM	186
349006	^MATRNORM	186	34A006	^MATCNORM	186
34B006	^MATRREF	186	34C006	^MATREF	186
34D006	^MATRANK	186	34E006	^MATINV	185
34F006	^MATREFRREF	186	350006	^INXREdext	186
351006	^METAMATRED	186	352006	^METAPIVOT	186
354006	^PIVOTFLOAT	186	355006	^SYSText	186
356006	^STOSYSText	186	357006	^MAKESYSText	186
359006	^NULLVECTOR?	185	35A006	^FINDELN	187
35B006	^PULLEL[S]	187	35C006	^BANGARRY	187
35D006	^PUT[]	187	35E006	^ARSize	52
35F006	^MATRIX>DIAG	187	360006	^MATRIXDIAG>	187
361006	^la+ELEMsym	187	362006	^INSERTROW[]	187
363006	^insertrow[]	187	364006	^INSERTCOL[]	187
365006	^INSERT[]ROW[]	187	366006	^INSERT[]COL[]	187
367006	^MATRIXRCI	186	368006	^MATRIXRCIJ	186
369006	^MATRIXCSWAP	187	36A006	^MATRIXRSWAP	187
36B006	^MATRIX-ROW	187	36C006	^METAMAT-ROW	187
36D006	^MATRIX-COL	187	36E006	^METAMATCSWAP	187
36F006	^METAMATRSWAP	187	370006	^STOMAText	187
371006	^MATIDN	184	372006	^MATCON	184
373006	^MAKEARRY	184	374006	^OBJDIMS2MAT	184
375006	^LCPROG2M	184	376006	^MAKE2DMATRIX	184
377006	^make2dmatrix	184	378006	^ADDMATOBJext	187
379006	^VUNARYOP	187	37A006	^VBINARYOP	187
37B006	^EVAL	187	37C006	^MATEGVL	188
37D006	^ROOTM2ROOT	205	37E006	^MADJ	188
37F006	^MATEGV	188	380006	^JORDAN	188
381006	^QXA	188	382006	^AXQ	188
383006	^GAUSS	188	384006	^SYLVESTER	188
385006	^metasplit	59	386006	^m-1&m+1	195
387006	^meta1/meta	195	388006	^1&meta	195
389006	^meta/2	195	38A006	^addt2	195
38B006	^addt/	195	38C006	^meta2*	195
38D006	^meta1-sq	195	38E006	^metasq+1	195
38F006	^metasq-1	195	390006	^meta-1	195
391006	^NDROPZERO	58	392006	^2DROPZ0	180
393006	^metaadd	195	394006	^metasub	195
395006	^metamult	195	396006	^metadiv	196
397006	^meta^	196	398006	^addt^	195
399006	^metapow	196	39A006	^metafraction?	199
39B006	^metaxroot	196	39C006	^top&addt*	195
39D006	^top&addt/	195	39E006	^addti	195
39F006	^metaEQUAL?	59	3A0006	^2metaundef#	198
3A1006	^1metaundef#	198	3A2006	^metaundef	198
3A3006	^2metainf#	198	3A4006	^1metainf#	198
3A5006	^metainftype	198	3A6006	^unsignedinf	198
3A7006	^plusinf	198	3A8006	^NDROPplusinf	198
3A9006	^minusinf	198	3AA006	^NDROPminusinf	198
3AB006	^MetaAdd	195	3AD006	^MetaSub	195
3AF006	^MetaMul	195	3B1006	^MetaDiv	196
3B3006	^NDROPZ0	180	3B4006	^NDROPZ1	180
3B5006	^MetaPow	196	3B7006	^MetaNeg	196
3B9006	^metaneg	196	3BA006	^metackneg	196
3BB006	^metasimp	199	3BC006	^metapi?	199
3BD006	^metaCOMPARE	199	3BE006	^STRICTmetaCOMPARE	199
3BF006	^EQUALPOSMETA	59	3C0006	^EQUALPOS2META	59
3C1006	^vgerxssSYMSUM	209	3C2006	^DISTRIB/	199
3C3006	^metareal?	199	3C4006	^ModExpa	200
3C5006	^ModAdd	200	3C6006	^ModSub	200
3C7006	^ModMul	200	3C8006	^ModDiv	200
3C9006	^ModDiv2	200	3CA006	^ModInv	201
3CB006	^ModGcd	201	3D0006	^Mod	181
3D2006	^PARTFRAC	208	3D3006	^INPARTFRAC	208
3D6006	^IEGCDext	181	3D7006	^REGCDext	203
3D8006	^EGCDext	203	3D9006	^INEGCD	181
3DC006	^PDer	206	3DD006	^INTEGRext	208
3DE006	^LRDMext	202	3DF006	^RRDMext	202
3E0006	^DEGREext	202	3E1006	^FHORNER	203

地址	名称	页码	地址	名称	页码
3E2006	^HORNExt	203	3E4006	^MHORNExt	203
3E5006	^PTAYLExt	200	3E6006	^LAGRANGEExt	203
3E7006	^PSEUDOPREP	222	3E8006	^PSEUDODIV	193
3EA006	^BESTDIV2	193	3EC006	^QUOTExt	193
3ED006	^NEWDIVExt	193	3EE006	^QDivRem	200
3EF006	^DIV2LISTExt	200	3F0006	^DIVOBJExt	222
3F1006	^DIVMETAOBJ	64, 196	3F2006	^LOPDExt	222
3F3006	^QUOTOBJExt	193	3F4006	^DIVISIBLE?	193
3F5006	^QDiv?	193	3F6006	^FastDiv?	193
3F7006	^POTENCEExt	193	3F8006	^PDIV2Ext	200
3F9006	^PSetSign	200	3FB006	^HSECO2RCExt	222
3FC006	^SECO2CMPExt	222	3FF006	^VALOBJExt	223
400006	^R2SYM	179	401006	^VAL2Ext	223
402006	^INVAL2	223	403006	^METAVAL2	223
404006	^VAL1	223	405006	^VAL1M	223
406006	^addt0meta	59	407006	^HALFTAN	193
408006	^COS2TAN/2	191	409006	^cos2tan/2	197
40A006	^1-x^2/1+x^2	197	40B006	^SIN2TAN/2	191
40C006	^sin2tan/2	197	40D006	^2x/1+x^2	197
40E006	^TAN2TAN/2	191	40F006	^tan2tan/2	197
410006	^addtTAN/2	197	411006	^TRIGTAN	193
412006	^COS2TAN	191	413006	^cos2tan	197
414006	^SIN2TAN	191	415006	^sin2tan	198
416006	^TRIGExt	193	417006	^HYP2EXPExt	193
418006	^EXPLNExt	193	419006	^SERIESEXPLN	193
41A006	^LNP12LN	191	41B006	^LOG2LN	191
41C006	^ALOG2EXP	191	41D006	^EXPM2EXP	191
41E006	^SQRT2LNEXP	191	41F006	^sqrt2lnexp	191
420006	^TAN2EXP	191	421006	^tan2exp	198
422006	^ASIN2LN	191	423006	^asin2ln	198
424006	^ACOS2LN	191	425006	^acos2ln	198
426006	^TAN2SCExt	193	427006	^TAN2SC	191
428006	^sin/cos	198	429006	^SIN2TCExt	193
42A006	^SIN2TC	191	42B006	^cos*tan	198
42C006	^COS2Ext	191	42D006	^sqrt1-sin^2	198
42E006	^SIN2Ext	191	42F006	^sqrt1-cos^2	198
430006	^ATAN2SExt	193	431006	^ATAN2ASIN	191
432006	^atan2asin	198	433006	^ASIN2Text	193
434006	^ASIN2ATAN	191	435006	^asin2atan	198
436006	^ASIN2CExt	193	437006	^ASIN2ACOS	191
438006	^pi/2-acos	198	439006	^pi/2-meta	198
43A006	^ACOS2SExt	193	43B006	^pi/2-asin	198
43C006	^ACOS2ASIN	191	43D006	^ATAN2LNExt	191
43E006	^atan2ln	198	43F006	^TAN2SC2Ext	193
440006	^TAN2SC2	191	441006	^2*1-cos/sin	198
442006	^TAN2CS2	191	443006	^2*sin/1+cos	198
444006	^SIN2EXPExt	191	445006	^sin2exp	198
446006	^COS2EXPExt	191	447006	^cos2exp	198
448006	^SINH2EXPExt	191	449006	^sinh2exp	198
44A006	^COSH2EXPExt	191	44B006	^cosh2exp	198
44C006	^TANH2EXPExt	191	44D006	^tanh2exp	198
44E006	^ASINH2LNExt	191	44F006	^asinh2ln	198
450006	^ACOSH2LNExt	191	451006	^acosh2ln	198
452006	^ATANH2LNExt	191	453006	^atanh2ln	198
454006	^XROOT2Ext	191	455006	^xroot2expln	198
456006	^LN2Ext	193	458006	^exp2sincos	198
459006	^metai*	195	45A006	^LN2ATAN	191
45B006	^VAR=LIST	191	45C006	^IDNTEXEC	223
45F006	^LISTEXEC	222	460006	^LISTEXEC1	222
461006	^SECOEXEC	222	462006	^EQUATION?	63
463006	^USERFCN?	63	464006	^SYMBEXEC	191
465006	^MEVALExt	192	466006	^CASNUMEVAL	192
467006	^CASCOMPEVAL	192	468006	^REPLACE2BY1	192
469006	^NR_REPLACE	192	46B006	^CASCRUNCH	192
46C006	^APPROXCOMPEVAL	192	46F006	^SYMTAYLOR	206
471006	^TRUNC DL	206	472006	^LIMSERIES!	206
477006	^LIMIT!	206	478006	^LIMSTEP1!	206
47C006	^LIMLIM!	206	47F006	^LIMCMPL!	206
480006	^LIMEQUFR!	206	481006	^LIMEQU!	206
483006	^LIM+-!	206	48C006	^LIMDIVPC!	206
48E006	^LIMPROFEND!	206	490006	^LIM%#!	206
49E006	^LIM#VARX!	206	4A1006	^HORNEXP!	206
4B6006	^VARCOMP!	206	4BA006	^VARCOMP32!	206
4BD006	^LIMVALOBJ!	206	4BE006	^LIMVAL!	206
4BF006	^EQUIV!	206	4C0006	^LVARXNX2!	206
4C2006	^FindCurVar	206	4C3006	^LIMVAR!	206

地址	名称	页码	地址	名称	页码
4C5006	^ISOL1	210	4C6006	^ISOLALL	210
4C7006	^ISOL2ext	210	4C8006	^BEZOUTMSOLV	210
4C9006	^ROOT{ }N	210	4CA006	^MHORNER	210
4CB006	^MHORNER1	210	4CC006	^SQFFext	210
4CD006	^MSQFF	210	4CE006	^%1TWO	210
4CF006	^MZSQFF	210	4D0006	^MZSQFF1	210
4D1006	^MSECOSQFF	224	4D2006	^MLISTSQFF	210
4D3006	^METASQFFext	210	4D4006	^SECOSQFFext	182, 222
4D5006	^CSQFFext	182	4D6006	^SUMSQRext	182
4D7006	^VXXLext	179	4D8006	^METALISTVXXL	179
4D9006	^VXXLFext	179	4DA006	^VXXL1ext	179
4DB006	^VXXL0	179	4DC006	^VXXL2NR	179
4DD006	^VXXL2	179	4DE006	^LIDNText	210
4DF006	^LVARXNXext	210	4E0006	^ISPOLYNOMIAL?	210
4E1006	^2POLYNOMIAL?	211	4E2006	^VXINDEP?	211
4E4006	^RLVARext	211	4E5006	^LLVARDext	211
4E6006	^VXLVARext	211	4E7006	^LVARext	211
4E8006	^VX>LVARext	211	4E9006	^VX>	211
4EA006	^VX!	211	4EB006	^prepvarlist	56
4EC006	^LIDNTLVAR	211	4ED006	^LISTOPRAC	211
4EE006	^LISTOPext	211	4EF006	^LISTOPSQRT	211
4F0006	^LVARDext	211	4F2006	^DEPTHext	211
4F3006	^DEPTHOBJext	211	4F4006	^TRIMext	202
4F5006	^PTrim	202	4F6006	^TRIMOBJext	211
4F7006	^NEWTRIMext	211	4F8006	^>POLYTRIM	211
4F9006	^ELMGext	211	4FA006	^SWAPRNEG	189
4FB006	^QNeg	189	4FC006	^RNEGext	189
4FD006	^SWAPRRE	189	4FE006	^RREext	189
4FF006	^SWAPRIM	189	500006	^RIMext	189
501006	^xREext	189	502006	^xSYMRE	196
503006	^xIMext	189	504006	^xSYMIM	196
505006	^RCONJext	189	506006	^addtCONJ	196
508006	^QCONJext	222	509006	^QABSext	222
50A006	^RABSext	189	50B006	^ZABS	180
50C006	^CZABS	37	50D006	^xABSext	189
50E006	^addtABS	196	510006	^addtABSEXACT	196
511006	^addtSIGN	196	513006	^addtARG	196
515006	^ARG2	38	517006	^QUADRANT	38
518006	^CNORMext	182	51A006	^QNORMext	222
51B006	^SXSQRext	189	51C006	^XSQRext	189
51D006	^CK%SQRT	34	51E006	^C%SQRT	38
521006	^CKLN	189	522006	^xLNNext	189
523006	^addtLN	196	525006	^EXPANDLN	189
526006	^CMPLXLN	189	527006	^LNATANext	189
528006	^REALLN	189	529006	^xEXPext	190
52A006	^xINVext	189	52B006	^xvext	189
52C006	^xCOSeext	190	52D006	^xSINext	190
52E006	^xTANext	190	52F006	^xCOSHext	190
530006	^xSINHext	190	531006	^xTANHext	190
532006	^xASINext	190	533006	^xACOSext	190
534006	^xATANext	190	535006	^addtCOS	197
536006	^xSYMCOS	190	537006	^addtSIN	197
538006	^xSYMSIN	190	539006	^addtTAN	197
53A006	^xSYMTAN	190	53B006	^addtSINACOS	197
53C006	^addtASIN	197	53D006	^xSYMASIN	190
53E006	^addtACOS	197	53F006	^xSYMACOS	190
540006	^addtATAN	197	541006	^xSYMATAN	190
542006	^addtSINH	197	543006	^xSYMSINH	190
544006	^addtCOSH	197	545006	^xSYMCOSh	190
546006	^addtTANH	197	547006	^xSYMTANH	190
548006	^xATANHext	190	549006	^addtATANH	197
54A006	^xSYMATANH	190	54B006	^xASINHext	190
54C006	^addtASINH	197	54D006	^xSYMASINH	190
54E006	^xACOSHext	190	54F006	^addtACOSH	197
550006	^xSYMACOSH	190	551006	^addtSQRT	197
552006	^xSYMSQRT	189	553006	^xSQext	189
554006	^addtSQ	197	555006	^xSYMSQ	189
556006	^addtINV	197	557006	^xSYMINV	189
558006	^addtEXP	197	559006	^xSYMEXP	197
55A006	^addtD->R	197	55C006	^addtR->D	197
55E006	^addtFLOOR	197	55F006	^xSYMFLOOR	190
560006	^addtCEIL	197	561006	^xSYMCEIL	190
562006	^addtIP	197	563006	^xSYMIP	190
564006	^addtFP	197	565006	^xSYMFP	190
566006	^addtXPON	197	567006	^xSYMXPON	190
568006	^addtMANT	197	569006	^xSYMMANT	190

地址	名称	页码	地址	名称	页码
56A006	^addtLNP1	197	56B006	^xSYMLNP1	190
56C006	^addtLOG	197	56D006	^xSYMLOG	190
56E006	^addtALOG	197	56F006	^xSYMALOG	190
570006	^addtEXPM	197	571006	^xSYMEXPM1	190
572006	^factorial	190	573006	^facts	190
574006	^addtFACT	197	575006	^xSYMFACT	190
576006	^factzint	181	577006	^addtNOT	197
578006	^xSYMNOT	190	579006	^Verbose1	220
57A006	^Verbose2	220	57B006	^Verbose3	220
57C006	^VerboseN	220	57D006	^GETERABLEMSG	213
57E006	^ERABLEERROR	213	57F006	^CANTFACTOR	213
580006	^TRANSCERROR	213	581006	^NONUNARYERR	213
582006	^INTERNALERR	213	583006	^INVALIDOP	213
584006	^ISOLERR	213	585006	^NONINTERR	213
586006	^INTVARERR	213	587006	^Z>#ERR	213
588006	^Z<0ERR	213	589006	^VXINDEPERR	213
58A006	^NONPOLYSYST	213	58B006	^COMPLEXERR	213
58C006	^VALMUSTBE0	213	58D006	^SWITCHNOTALLOWED	213
58E006	^ERR\$EVALext	213	58F006	^Sys1IT	213